Python

杨秉学

2026 年 6 月 8 日

目录

第一部分　基础配置 **1**

第一章　pip **3**
一、 下载 3
二、 国内源 3
1) 配置国内源 3
三、 使用 4
1) 升级 4
2) 搜索包 4
3) 安装包 4
4) 查看包 4
5) 升级包 5
6) 卸载包 5

第二章　(Ana/mini)conda **7**
一、 简介 7
1) conda 7
2) 虚拟环境 7
二、 安装 7
1) 安装 anaconda 7
(1) Linux 安装 7
2) 安装 miniconda 7
(1) Linux 安装 7
(2) Windows 安装 8
三、 配置国内源 9
1) linux 9
2) windows 9
(1) 手动配置 10
(2) 命令行配置 10
(3) 移除镜像 11
(4) 验证 11
四、 虚拟环境 12
1) 修改默认虚拟环境路径 12
(1) Linux 12
(2) Windows 13
2) 创建虚拟环境 13

3) 查看虚拟环境 14
4) 激活虚拟环境 14
5) 删除虚拟环境 14
6) 虚拟环境重新命名 14
五、 包管理 14
1) 查询包 14
2) 安装包 15
(1) 在线安装 15
1 conda 安装 15
2 pip 安装 15
(2) 离线安装 15
(3) 批量安装 15
3) 卸载包 16
4) 查看包 16
5) 更新包 16
6) 清空缓存 17
六、 Vscode 配置 17

第三章 venv 环境 19
一、 概述 19
二、 使用 19
三、 常用命令 19
1) 指定 python 版本 19
四、 与 Vscode 结合 19

第四章 Jupter Notebook 21
一、 简介 21
1) notebook 运行逻辑 21
二、 安装 22
1) 安装 IPython(非必须) 22
2) 安装 Jupyter 22
3) 添加 Kernel 22
4) 代码补全 22
三、 使用 22
1) 本地使用 22
2) 远程使用 23
(1) 远程服务器配置 23
(2) 本地访问远程 JupterLab 24
1 配置防火墙访问 24
2 SSH 端口转发 25
3 配置密码 25

第五章 Jupter Notebook 语法 27
一、 JupyterLab 的语法指令 27
1) writefile 写到一个文件 27

2) pycat 显示一个文件 27
3) run 执行一个文件 27
4) 计算代码运行时间 28
5) whos 查看当前内存中的变量 28
6) lsmagic 查看所有的魔法指令 28

第六章 自定义包 **31**
一、 模块 31
二、 概述 31

第二部分 基本语法 **33**

第七章 操作数相关 **35**
一、 概述 35
1) 标识符 (变量名) 35

第八章 基本数据类型 **37**
一、 Number——数字类型 37
1) int 整数类型 37
(1) bool 布尔类型 37
(2) 数字分隔符 37
2) float——浮点数 (科学计数法) 38
(1) 浮点数精度问题 38
3) bool——布尔类型 39
4) complex——复数类型 39
二、 类型转换 39
1) type 39
2) 显式类型转换 39
三、 缓存重用机制 40

第九章 [序列] 数据结构 **43**
一、 序列 43
1) 序列内置函数 43
(1) len 序列的长度 43
(2) max 序列的最大值 44
(3) min 序列的最小值 45
(4) sum 返回序列的和 45
(5) sorted 返回排序后的序列 46
(6) reversed 返回逆序的序列 47
(7) enumerate 返回索引和元素组成的元组 47
2) 序列的通用操作 48
(1) [] 索引访问 48
(2) index 查找元素的索引 49
(3) count 统计元素出现的次数 50
(4) 切片 51

(5) split 分割 . . . 52
(6) in/not in 成员关系 . . . 52
(7) 拼接 . . . 53
(8) join 连接 . . . 54
(9) * 重复 . . . 55
(10) append 追加 . . . 56
(11) extend 扩展 . . . 56
(12) insert 插入 . . . 57
(13) update 更新 . . . 58
(14) remove 删除 . . . 58
(15) pop 弹出 . . . 59
(16) clear 清空 . . . 60
(17) del 删除 . . . 60
3) 序列间转换 . . . 61
(1) list 将序列转为列表 . . . 61
(2) tuple 将序列转为元组 . . . 62
(3) set 将序列转为集合 . . . 63
二、 列表 (list) . . . 63
1) 创建 . . . 63
(1) 直接生成 . . . 63
(2) list 显式转换 . . . 64
(3) 列表推导式 . . . 64
2) 修改 . . . 65
3) 栈 . . . 65
4) collections 实现栈和队列 . . . 66
三、 字符串 . . . 66
1) 定义 . . . 66
四、 集合 . . . 66
1) 定义 . . . 66
(1) 集合推导式 . . . 66
2) 集合间运算 . . . 67
五、 字典 . . . 68
1) 创建 . . . 68
(1) 直接生成 . . . 68
(2) 字典推导式 . . . 68
六、 元组 (tuple) . . . 69
1) 定义 . . . 69
2) 创建 . . . 69
(1) 直接生成 . . . 69
(2) tuple 显式转换 . . . 69
(3) 元组推导式 (生成器表达式) . . . 69

第十章 (动作行为) 语法相关 71
一、 行为规范 . . . 71
1) 注释 . . . 71

2) 行与缩进 71
3) 多行语句 71
二、 运算符 72
1) 算术运算符 72
2) 赋值运算符 73
3) 位运算符 74
4) 比较运算符 75
5) 逻辑运算符 76
三、 逻辑 76
1) 条件 76
2) 三元运算符 77
3) 循环 77
4) while 77
5) for 78

第十一章 异常 **81**
一、 异常 81
二、 异常的种类 82
1) 文件 82
2) 除零 82
3) 输入异常 82
4) 捕获多个异常 83
5) 捕获所有异常 83
6) 自定义异常返回信息——raise 84
7) 自定义异常 84
三、 异常的处理 84

第十二章 函数与模块 **87**
一、 迭代器与生成器 87
1) 迭代器 87
(1) 创建迭代器 87
(2) StopIteration 异常 88
2) 生成器 89
二、 Lambda 表达式 89
三、 内建函数 90
四、 闭包 91
五、 装饰器 (Decorator) 91
六、 模块 92

第十三章 装饰器 (decorator) **93**
一、 概述 93

第十四章 [标准库] math (标量) 数学库 **97**
一、 数学常数 97
二、 三角函数相关 97

1) 角度转换 97
2) 基本三角函数 97
3) 反三角函数 97
三、 指数与对数相关 98
四、 幂函数与开方 98
五、 特殊功能函数 98
1) 取整相关 98
2) 数值处理 98
3) 特殊判断 98

第十五章 自定义包 **101**
一、 模块 101
二、 概述 101

第三部分 系统编程 **103**

第十六章 多线程编程 **105**
一、 线程 105
二、 线程同步 106
三、 线程池 109
1) ThreadPoolExecutor 109
2) 返回值 110
四、 Python 中的多线程 111

第十七章 多进程编程 **113**
一、 进程 113
二、 进程间通信 114
1) Queue 115
三、 进程池 116
1) 同步执行 116
2) 异步执行 117

第十八章 系统信息 **121**
一、 获取 CPU 信息 121
二、 获取内存信息 121
三、 获取磁盘信息 121
四、 获取网络信息 122
1) 网络连接信息 124

第十九章 进程管理 **125**
一、 创建进程 125
二、 获取进程 125
1) 显示所有进程及对应的名称 125
2) 根据名称获取进程 ID 126
三、 结束进程 127

第二十章 pywin23 概述 **129**
一、 概述 129
二、 安装 129

第二十一章 进程管理 **131**
一、 创建进程 131
二、 获取进程 131
1) 显示所有进程及对应的名称 131
2) 根据名称获取进程 ID 132

第二十二章 GUI 操作 **133**
一、 获取窗口 133
二、 窗口截图 133
三、 窗口操作 133

第四部分 存储 **135**

第二十三章 输入输出 **137**
一、 input 输入 137
二、 print 输出 137
1) 分隔符 137
2) 结尾符 138
3) 输出位置 138
4) 格式化输出 138
三、 文件的内建函数和方法 139

第二十四章 文本操作的应用 **141**

第二十五章 上下文管理 **153**
一、 with 简化资源管理 153
1) 基本语法 153
2) 工作原理 155
二、 创建自定义的上下文管理器 155
1) 类的方式 155
2) contextlib 模块 156

第五部分 [编程范式] 面向对象编程 **157**

第二十六章 面向对象 **159**
一、 类与对象 159
1) 类 159
2) 继承 159
3) 接口 160
4) 封装 160
(1) 私有属性 160

(2) 私有方法 160
5) 多态 161

第六部分 图形可视化 163

第二十七章 PySide 概述 165
一、 PySide 简介 165
1) Qt 、PyQt 、PySide 之间的关系 165
2) PySide 与 PyQt 的比较 166
3) 主要模块 166
二、 安装 PySide 167
1) 常见问题 167
(1) PySide 找不到 libstdc++ 167
(2) conda 的 python 版本与系统的版本不一致 167

第二十八章 [Python 库] Matplotlib 概述 171
一、 简介 171
二、 安装 171
三、 基本框架 171

第二十九章 [matplotlib] 绘图 175
一、 折线/散点图 175
1) plt.plot 折线/散点图 175
(1) -o 将散点图连线 178
(2) 设置散点图属性 180
(3) 绘制折线图 180
2) plt.scatter 散点图 181
二、 条形图/柱形图 186
1) plt.bar 垂直条形图 186
2) plt.bar 堆叠条形图 187
3) plt.bar 并列条形图 188
4) plt.barh 水平条形图 189
三、 pie 饼图 190
四、 boxplot 箱线图 192
五、 直方图 194
1) plt.hist 一维频率直方图 194
2) plt.hist2d 二维频率直方图 197
3) plt.hexbin 六边形区间划分 199
六、 imshow 热力图 199
七、 等高线图 200
1) plt.contour 等高线图 200
2) plt.contourf 带填充色的等高线图 203
八、 三维图 206
1) plt.plot3D 三维数据点与线 206
2) plt.contour3D 三维等高线图 206

3) plt.plot_wireframe 线框图 . . . 209
4) plt.plot_surface 曲面图 . . . 210
(1) plt.plot_surface 极坐标曲面图 . . . 213
(2) plt.plot_trisurf 曲面三角剖分 . . . 214

第三十章 [matplotlib] 图片样式与排版 215
一、 默认配置 . . . 215
二、 显示中文 . . . 216
三、 图片的属性 . . . 217
四、 matplotlib 线条 . . . 218
1) 线条的颜色 . . . 218
2) 线条的粗细 . . . 220
3) 线条的样式 . . . 221
4) 线条的标记 . . . 222
五、 图片的边界 . . . 224
六、 坐标轴 . . . 225
1) 坐标记号标签 . . . 226
2) 坐标轴的标签 . . . 227
3) 坐标轴移动 . . . 228
4) 添加图例 . . . 229
5) 坐标轴的网格 . . . 231
6) 特殊点的注释 . . . 232
七、 箭头 . . . 233
八、 图片风格 . . . 234
九、 多图的排列 . . . 236
1) plt.subplot() . . . 236
(1) add_subplot() 添加子图 . . . 238
2) plt.subplots() . . . 239
3) GridSpec . . . 240
4) 自定义子图大小和间距 . . . 241

第三十一章 [Python 库] Pyecharts 概述 243
一、 简介 . . . 243
二、 安装 . . . 243

第三十二章 图表类型 245

第三十三章 Manim 概述 247
一、 简介 . . . 247
二、 Manim(ManimCE) . . . 247
1) 安装 . . . 247
2) 使用 . . . 248
3) 测试 . . . 248
三、 ManimGL . . . 249
四、 ManimCairo . . . 249

第三十四章 Manim 基本图形 251
一、 点 . . . 251
二、 线 . . . 251
1) 直线 . . . 251
2) 虚线 . . . 252
3) 带箭头的线 . . . 253
三、 圆 . . . 253
1) 正圆 . . . 253
2) 椭圆 . . . 254
3) 圆弧 . . . 254
四、 多边形 . . . 255
1) 三角形 . . . 255
2) 矩形 . . . 257
(1) 正方形 . . . 257
(2) 矩形 . . . 258
(3) 圆角矩形 . . . 259
3) 星形 . . . 261
4) 任意多边形 . . . 262
(1) 多边形 . . . 262
(2) 任意正多边形 . . . 263
五、 自定义图片 . . . 264
1) SVG . . . 264
六、 公共样式 . . . 266
七、 图像样式 . . . 266
1) 边框 . . . 267
2) 颜色 . . . 268
(1) 预制颜色 . . . 268
(2) 填充和透明度 . . . 269
八、 坐标系 . . . 271
1) 一维坐标系 . . . 271
2) 二维坐标系 . . . 272
(1) NumberPlane 实数平面 . . . 272
(2) ComplexPlane 复数平面 . . . 273
(3) PolarPlane 极坐标平面 . . . 274
(4) Axes 二维笛卡尔坐标轴 . . . 275
3) 三维坐标系 . . . 276
(1) ThreeDAxes 三维笛卡尔坐标系 . . . 276

第三十五章 Manim 文字和公式 281
一、 Text 文字 . . . 281
1) 一般文本 . . . 281
2) 多行文本 . . . 282
3) 富文本 . . . 282
二、 Tex 公式 . . . 284
1) 公式 . . . 284

2) 列表 . . . 285
3) 标题 . . . 285
三、 文本样式 . . . 287
1) 颜色相关 . . . 288
(1) color 单色 . . . 288
(2) gradient 渐变色 . . . 289
(3) 按字符设置颜色 . . . 290
2) 字体相关 . . . 292
(1) font 字体 . . . 292
(2) font_size 字体大小 . . . 292
(3) weight 字重 . . . 293
(4) slant 倾斜 . . . 295
(5) 按字符设置 . . . 296

第三十六章 manim 动画效果 299
一、 创建 . . . 299
1) Create 创建图形 . . . 299
2) Write 书写效果 . . . 300
3) FadeIn 淡入效果 . . . 300
4) add 添加元素 . . . 301
二、 销毁 . . . 301
1) Uncreate 擦除图形 . . . 301
2) Unwrite 擦除文字 . . . 302
3) FadeOut 淡出效果 . . . 302
三、 移动 . . . 303
1) shift 移动指定距离 . . . 303
2) move_to 移动到指定位置 . . . 303
3) MoveAlongPath 沿路径移动 . . . 304
4) TracedPath 保留运动路径 . . . 305
四、 Rotate 旋转 . . . 306
五、 变换 . . . 307
1) 图形变换 . . . 307
(1) TransformFromCopy 保留原图形 . . . 307
(2) ReplacementTransform 原图形直接变换成新的图形 . . . 308
2) 文字变换 . . . 308
(1) TransformFromCopy 保留原文字 . . . 308
(2) ReplacementTransform 原文字直接变换成新的文字 . . . 309
六、 动画效果属性 . . . 311
七、 光照样式 . . . 311
八、 三维对象样式 . . . 311
九、 动画组合 . . . 312
1) LaggedStart 延迟启动 . . . 312
2) Succession 顺序启动 . . . 313

十、 动画联动 314
十一、 相机 319
1) 相机概念 319
2) 相机移动 319
(1) move_to 移动焦点 319
(2) set 改变视野 320
3) 鹰眼效果 321
4) 追踪物体 321
5) 3D 场景 322

第七部分 互联 325

第三十七章 urllib 327
一、 简介 327
二、 urllib.request 327
1) Get 328
2) Post 328
3) Http Headers 328
4) Proxy 329
三、 requests 329
1) 安装 330
2) Get 330
3) Post 330

第三十八章 django 概述 331
一、 概述 331
二、 MVC 模型 331
1) MTV 模型 331
2) Django 的工作流程 332
三、 Django 的安装 333
1) conda 安装 333
2) pip 安装 333
四、 主项目 333
1) 创建项目 333
2) 查询帮助 333
3) 启动项目 336
五、 创建子应用 336
(1) 子应用的作用 336
1) 创建子应用 336
2) 注册子应用 337

第三十九章 Django 模型 (数据库) 339
一、 Django 模型简介 339
二、 数据库的配置 339
1) sqlite3 数据库配置 339

(1) settings.py 配置 339
2) MySQL 数据库配置 339
(1) 安装 MySQL 驱动 339
(2) 提前配置数据库 340
(3) 配置 settings.py 341
3) PostgreSQL 数据库配置 343
(1) 安装 psycopg2 343
(2) [可选] 配置 PostgreSQL 的远程访问 343
(3) 配置 settings.py 344
4) Redis 数据库配置 345
三、 定义模型 345
1) 前提准备————创建子应用 345
2) 定义表结构 346
(1) ORM 的字段类型 346
(2) 外键关联 347
3) 添加 INSTALLED_APPS 347
4) 生成迁移文件 348
(1) 查看数据库 349
5) 常见报错 349
(1) mysqlclient 报错 350
四、 对象关系映射 (ORM) 350
1) ORM 与数据库的对应关系 350
2) 数据库操作 350
(1) 增 350
(2) 查询 351
(3) 改 352
(4) 删 352
五、 后台管理 353
1) 创建超级用户 353
2) 设置中文 353
3) 注册模型 353
4) 问题 355

第四十章 视图 357
一、 视图 357
二、 视图函数 357
1) 视图函数——HttpResponse 357
2) 渲染模板——render 357
三、 URL 配置 (URLconf) 358
1) path() 将 URL 模式映射到视图 359
2) [弃用] url()/re_path 将 URL 模式映射到视图 360
3) [不单独使用] include() 导入 URL 模式 361
四、 路由视图过程 362

第四十一章 模板 template 365

一、 模板概述 365
1) 模板的定义 368
二、 静态文件 369
1) 静态文件的作用 369
2) 静态文件的配置 369

第八部分 数据处理与分析 371

第四十二章 [Python 库] Pandas 373
一、 简介 373
二、 安装 373

第四十三章 Pandas 的数据结构 375
一、 Series 375
二、 DataFrame 375
三、 Index 375

第四十四章 Pandas 数据的引用和读取 377
一、 数据的引用 377
二、 文件的读取与写入 377
1) CSV 377
2) Excel 377
3) JSON 377
4) HDF5 377

第四十五章 Pandas 数据的聚合和排序 379

第九部分 人工智能 381

第四十六章 PyTorch 383
一、 概述 383
1) 与 TensorFlow 的比较 383
二、 PyTorch 的历史 384
三、 PyTorch 的主要应用 384
四、 Pytorch 环境搭建 385
1) 前提条件 385
2) CPU 版本 385
3) GPU 版本 385
(1) conda 安装 385
(2) whl 安装 386
4) 问题 386
5) 后续辅助 386
五、 PyTorch 模块结构 387
1) Torch 相关 387
(1) PyTorch 安装目录 387

(2) PyTorch 的主要模块 387
2) torchvision 相关 389
(1) torchvision 安装目录 389
(2) torchvision 的主要模块 389

第四十七章 PyTorch 基础 393
一、 Tensor 概述 393
1) Tensor 简介 393
2) Tensor 的作用 393
3) torch.Tensor 和 torch.tensor 393
4) Tensor 的历史演变 394
5) Tensor 的结构属性 394
二、 Tensor 操作 395
1) Tensor 操作分类 395
2) 创建 Tensor 396
(1) 直接创建 397
(2) 从 NumPy 数组创建 398
(3) 概率分布创建张量 399
(4) 根据数值创建 Tensor 405
(5) 创建特殊张量 411
3) Tensor 的转换 412
4) 数学运算函数 413
(1) torch.add() 张量逐元素相加 413
(2) torch.addcdiv() 张量逐元素相加并除以第三个张量 414
(3) torch.addcmul() 张量逐元素相加并乘以第三个张量 414
(4) torch.sub() 415
(5) torch.mul() 逐元素相乘 415
(6) torch.div() 416
(7) torch.matmul() 矩阵乘法 416
(8) torch.dot() 417
(9) torch.pow() 417
(10) torch.exp() 417
(11) torch.log() 418
(12) torch.log2() 418
(13) torch.log10() 418
(14) torch.sqrt() 419
(15) torch.sin() 419
(16) torch.cos() 419
(17) torch.tan() 420
(18) torch.asin() 420
(19) torch.acos() 421
(20) torch.atan() 421
(21) torch.atan2() 421

(22) torch.sinh() . . . 422
(23) torch.cosh() . . . 422
(24) torch.tanh() . . . 422
(25) torch.mean() . . . 423
(26) torch.sum() . . . 423
(27) torch 取整函数 . . . 423
5) 随机抽样函数 . . . 424
(1) torch.seed() . . . 424
(2) torch.manual_seed(seed) . . . 424
(3) torch.initial_seed() . . . 425
(4) torch.get_rng_state() . . . 425
(5) torch.set_rng_state(new_state) . . . 425
6) 统计函数 . . . 426
(1) torch.prob() 所有元素的积 . . . 426
(2) torch.sum() 所有元素的和 . . . 426
(3) torch.mean() 所有元素的平均值 . . . 427
(4) torch.max() 所有元素的最大值 . . . 428
(5) torch.min() 所有元素的最小值 . . . 429
(6) torch.median() 所有元素的中位数 . . . 430
(7) torch.mode() 所有元素的众数 . . . 432
(8) torch.var() 所有元素的方差 . . . 433
(9) torch.std() 所有元素的标准差 . . . 437
7) Tensor 的形状 . . . 437
(1) 获取 Tensor 的形状 . . . 438
(2) Tensor 的形状变换 . . . 438
8) Tensor 的索引 . . . 442
(1) index_select() 按照索引获取张量中的元素 . . . 442
(2) masked_select() 按照掩码获取张量中的元素 . . . 445
9) Tensor 的广播机制 . . . 448
10) 归并操作 . . . 450
11) 比较操作 . . . 452
12) 矩阵操作 . . . 454
13) 矩阵转秩 (维度转换) . . . 457
(1) permute() . . . 457
(2) transpose() . . . 458
14) 增减维度 . . . 458
(1) squeeze() 删除维度为 1 的维度 . . . 458
(2) 手动删除维度 . . . 462
(3) unsqueeze() 增加维度 . . . 462
15) Tensor 的连接操作 . . . 464
(1) cat() 不变维度连接 . . . 464
(2) stack() 增加维度连接 . . . 470
16) Tensor 的切片 . . . 475
(1) chunk() 均分 . . . 475

(2) split() 按照指定大小切分 478
(3) unbind() 按照指定维度切分 481
17) 根据操作张量中的元素分类 483
(1) 逐元素操作 483
(2) in-place 操作 486
(3) 内存布局连续 487
三、 autograd 自动求导微分 490
1) Tensor 与 Variable 490
2) 自动求导要点 491
3) 计算图 492
(1) 计算图的推导 492
(2) 计算图的元素 496
(3) 计算图的属性 grad_fn 497
4) autograd 包 499
(1) torch.autograd 简介 499
(2) torch.autograd.backward() 500
1 标量反向传播 504
2 非标量反向传播 508
(3) torch.autograd.grad() 511
(4) torch.autograd.Function 513
5) autograd 的注意事项 519
(1) 梯度不会自动清零 519
(2) 叶子结点不可执行 in-place 520
(3) detach 切断一些分支的反向传播 520
四、 Tensor 与 Numpy 比较 522

第四十八章 PyTorch 数据处理 533
一、 数据处理工具箱概述 533
二、 utils.data 533
1) Dataset 数据交互模块 534
2) DataLoader 数据加载模块 535
三、 torchvision 图形处理工具箱 537
1) torchvision 安装 537
2) torchvision.datasets 537
3) torchvision.models 537
4) torchvision.transforms 537
5) torchvision.utils 538
四、 可视化工具 538
1) TensorBoard 538
(1) 安装 539
(2) 使用 539

第四十九章 PyTorch 神经网络工具 543
一、 神经网络核心组件 543
二、 构建神经网络的主要工具 544

1) nn.Module . . . 544
2) nn.functional . . . 544
(1) nn.Module 和 nn.functional 的区别 . . . 544
三、 构建模型的方法 . . . 547
1) 继承 nn.Module 基类构建模型 . . . 548
2) 使用 nn.Sequential 按层顺序构建模型 . . . 550
3) 继承 nn.Module 基类构建模型，再使用相关模型容器进行封装 . . . 554
四、 损失优化 . . . 557
1) Pytorch 激活函数 . . . 557
2) Pytorch 损失函数 . . . 558
3) Pytorch 优化器 . . . 564
五、 训练模型 . . . 567
1) 加载和预处理数据集 . . . 567
2) 定义损失函数 . . . 567
3) 定义优化方法 . . . 567
4) 循环训练模型 . . . 567
5) 循环测试或验证模型 . . . 568
(1) 比较 model.train() 和 model.eval() 的区别 . . . 568
6) 可视化结果 . . . 568
六、 正则化 . . . 568
1) L1 正则化 . . . 568
2) L2 正则化 . . . 568
3) Dropout . . . 569
4) Batch Normalization . . . 569
七、 训练技巧 . . . 569
八、 保存和加载模型 . . . 569
九、 实例 . . . 569
1) 模拟余弦函数曲线 . . . 569

第十部分 办公自动化 573

第五十章 excel 自动化概述 575
一、 Excel 介绍 . . . 575
二、 处理 Excel 文件的 Python 模块 . . . 575

第五十一章 Pandas 处理 Excel 577
一、 Excel 文件操作 . . . 577
1) read_excel 文件读取 . . . 577
2) DataFrame.to_excel 文件写入 . . . 577
二、 Sheet 工作表 (sheet) . . . 578
1) 读取工作表 (sheet) . . . 578
2) 写入工作表 (sheet) . . . 578
三、 Cell 单元格 . . . 578

1) 获取单元格 579
(1) 按行/列索引获取单元格 579
(2) 按区块获取单元格 579
2) 数据筛选 579
3) sort_values 排序 579
4) 数据运算 579
5) drop 删除行/列 580
6) 数据表的拼接 580
(1) merge 拼接 580
(2) concat 全连接 581
(3) append 纵向拼接 582

第五十二章 [麻烦] xlrd、xlwt 与 xlutils 库 585
一、 安装 585
二、 读取 Excel 内容 586
三、 写入 Excel 内容 587
四、 类型转换 588
五、 合并单元格 591
六、 使用 Excel 函数 592
七、 复制现有的 Excel 文件并添加新的数据 597
八、 合并多个 Excel 文件 598
九、 修改现有的 Excel 文件 599

第五十三章 [简洁] openpyxl 库 601
一、 安装 601
二、 创建 Excel 文件 601
三、 读取 Excel 内容 601
1) load_workbook() 打开文件 602
2) sheet 表单 602
四、 写入 Excel 内容 603
五、 修改 Excel 文件单元格内容 604
六、 修改 Excel 文件样式 605
1) 基本样式 605
2) 自定义样式 606
七、 操作大型 Excel 文件 608
1) read_only 模式 609
2) write_only 模式 610

第五十四章 [必须安装 Excel] xlwings 库 611
一、 简介 611
1) xlwings 的数据结构 611
二、 安装 612
三、 App 应用程序 612
四、 Book 工作簿 (Excel 文件) 612
1) 新建/保存关闭工作簿 612

2) 获取当前活动工作簿 613
3) 删除工作簿 614
五、 Sheet 工作表 (sheet) 614
1) 新建工作表 (sheet) 614
(1) 获取工作表 (sheet) 615
(2) 按索引/名称获取工作表 615
2) 获取工作簿中所有工作表 615
3) 删除工作表 (sheet) 616
4) 重命名工作表 616
六、 Range 区域/单元格 617
1) 获取单元格 617
2) 清除单元格 617
3) 删除单元格 618
4) 插入单元格 619
七、 xlwings 与 panads 的结合 619

第五十五章 word 自动化 621
一、 docx 库 621
1) 安装 621
2) 创建 Word 文档 621

第五十六章 web 自动化 623
一、 selenium 介绍 623
1) 组成部分 623
2) Selenium WebDriver 623
二、 selenium 安装 623
三、 启动浏览器 624
1) Selenium 操作流程 625

第五十七章 OCR 识别 627
一、 在线识别 627
二、 离线识别 627
1) pytesseract 627

Index 629

第一部分

基础配置

第一章 pip

一、 下载

到pip 官网下载 pip 安装包。

```
# 下载pip安装包
wget https://bootstrap.pypa.io/get-pip.py
# 安装pip
python get-pip.py

# 查看pip版本
pip --version # Python2.x 版本命令
pip3 --version    # Python3.x 版本命令

# 升级pip
## Linux 或 macOS
pip install --upgrade pip    # python2.x
pip3 install --upgrade pip   # python3.x

## Windows 平台升级:
python -m pip install -U pip   # python2.x
python -m pip3 install -U pip    # python3.x
```

二、 国内源

```
阿里云 http://mirrors.aliyun.com/pypi/simple/
中国科技大学 https://pypi.mirrors.ustc.edu.cn/simple/
豆瓣(douban) http://pypi.douban.com/simple/
清华大学 https://pypi.tuna.tsinghua.edu.cn/simple/
中国科学技术大学 http://pypi.mirrors.ustc.edu.cn/simple/
```

1) 配置国内源

```
# 临时生效
pip install -i https://pypi.tuna.tsinghua.edu.cn/simple SomePackage

# 永久生效
```

```
pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
```

1) Linux 修改 `/.pip/pip.conf` (没有就创建一个)，内容如下：

```
[global]
index-url = https://pypi.tuna.tsinghua.edu.cn/simple
```

2) Windows 在用户目录中创建一个 `pip` 目录，如 `C:\Users\tony\pip`，新建文件 `pip.ini`，内容如下：

```
[global]
index-url = https://pypi.tuna.tsinghua.edu.cn/simple
```

三、 使用

1) 升级

```
# 升级pip
pip install -U pip
## 如果这个升级命令出现问题 ，可以使用以下命令:
pip install --upgrade pip
```

2) 搜索包

```
pip search SomePackage
```

3) 安装包

```
pip install SomePackage              # 最新版本
pip install SomePackage==1.0.4       # 指定版本
pip install 'SomePackage>=1.0.4'     # 最小版本
pip install 'SomePackage>=1.0.4,<2.0' # 指定范围
pip install -i https://pypi.tuna.tsinghua.edu.cn/simple Django # 指定源
```

4) 查看包

```
pip show SomePackage
# 查看已安装的包
pip list
# 查看指定包的详细信息
```

```
pip show --verbose SomePackage
pip show -f SomePackage
```

5) 升级包

```
#  查看可升级的包
pip list -o
# 升级包
pip install --upgrade SomePackage
```

6) 卸载包

```
pip uninstall SomePackage
```

第二章 (Ana/mini)conda

一、 简介

1) conda

anaconde 是 Python 的全家桶，包含了编辑器、解释器以及 conda 包管理工具，但是太占地方

miniconda 是一个更小的包，只包含了解释器和 conda 包管理工具，编辑器需要自己安装。

2) 虚拟环境

虚拟环境是一个独立的 Python 环境 (具备完整第三方依赖的 python)，可以在不同的项目中使用不同的 Python 版本和包。

二、 安装

1) 安装 anaconda

(1) Linux 安装

参考 [1]

```
#如果不喜欢一打开conda执行下面语句
conda config --set auto_activate_base false
```

2) 安装 miniconda

(1) Linux 安装

访问官网下载 miniconda 安装包，然后执行

```
$> wget
    https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
wget
    https://mirrors.tuna.tsinghua.edu.cn/anaconda/miniconda/Miniconda3-py39_4.9.2-Linux-x86_64.sh
```

```
$> chmod +x Miniconda3-latest-Linux-x86_64.sh
$> ./Miniconda3-latest-Linux-x86_64.sh
Miniconda3 will now be installed into this location:
/home/tony/miniconda3

  - Press ENTER to confirm the location
  - Press CTRL-C to abort the installation
  - Or specify a different location below
[/home/tony/miniconda3] >>> /home/tony/.application/miniconda3

#如果不喜欢一打开conda执行下面语句
conda config --set auto_activate_base false
```

(2) Windows 安装

访问清华大学镜像站下载 miniconda 安装包，然后执行

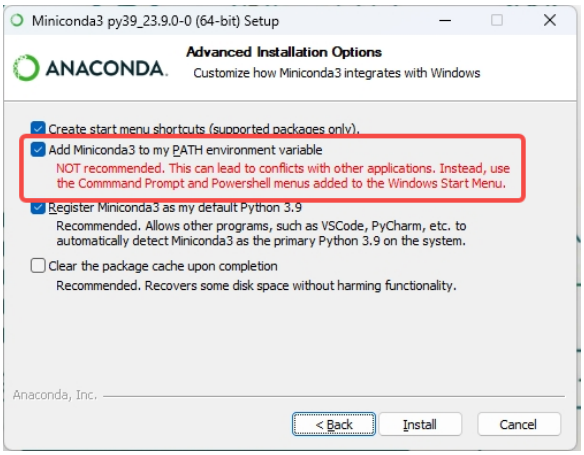

图 2.1 windows 安装 miniconda 配置环境变量

☞ 就算这步没有选择，可以安装以后手动配置环境变量，但是这样做会更麻烦一些。

如图2.1所示，勾选上配置环境变量，然后点击下一步，然后点击安装，等待安装完成。

如果安装成功，可以在命令行中输入`conda`，如果出现如图2.2所示的结果，说明安装成功。

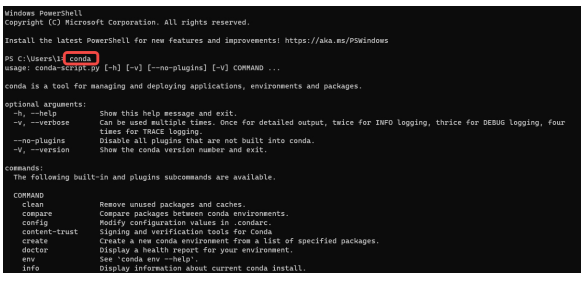

图 2.2 conda 安装成功检验

三、 配置国内源

各系统都可以通过修改用户目录下的 `.condarc` 文件来使用 TUNA 镜像源

1) linux

```
$> vim ~/.condarc
show_channel_urls: true
channels:
  - defaults
show_channel_urls: true
default_channels:
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/r
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/msys2
custom_channels:
  conda-forge: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
  msys2: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
  bioconda: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
  menpo: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
  pytorch: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
  pytorch-lts: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
  simpleitk: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
  deepmodeling: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/

$> conda info
```

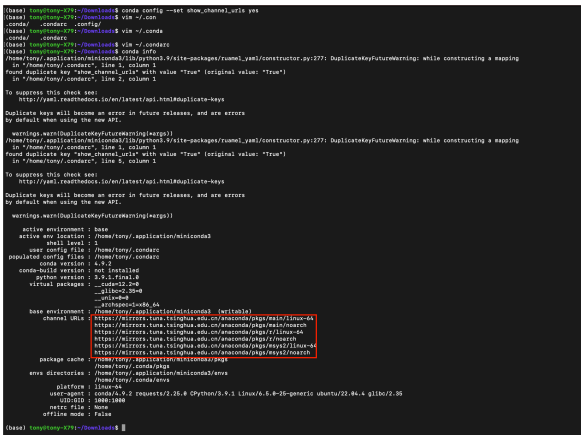

图 2.3 conda 使用终端配置国内源

2) windows

(1) 手动配置

Windows 用户无法直接创建名为 `.condarc` 的文件，可先执行

```
conda config --set show_channel_urls yes
```

生成该文件之后再修改。

```
show_channel_urls: true
channels:
  - defaults
default_channels:
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/r
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/msys2
custom_channels:
  conda-forge: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
  msys2: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
  bioconda: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
  menpo: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
  pytorch: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
  pytorch-lts: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
  simpleitk: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
  deepmodeling: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/
```

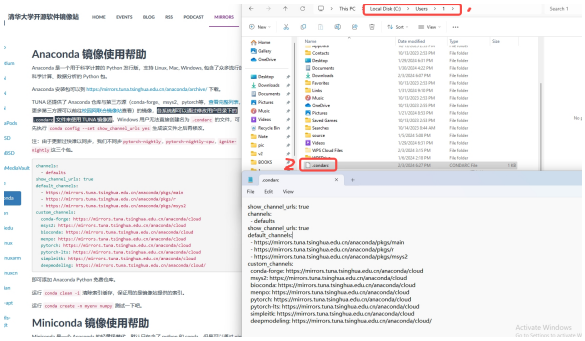

图 2.4 conda 使用配置文件配置国内源

(2) 命令行配置

如果嫌麻烦，可以直接在命令行中输入

```
# 配置中科大源
conda config --add channels
    https://mirrors.ustc.edu.cn/anaconda/pkgs/main/
conda config --add channels
    https://mirrors.ustc.edu.cn/anaconda/pkgs/free/
conda config --add channels
    https://mirrors.ustc.edu.cn/anaconda/cloud/conda-forge/
conda config --add channels
    https://mirrors.ustc.edu.cn/anaconda/cloud/msys2/
```

```
conda config --add channels
    https://mirrors.ustc.edu.cn/anaconda/cloud/bioconda/
conda config --add channels
    https://mirrors.ustc.edu.cn/anaconda/cloud/menpo/
conda config --add channels https://mirrors.ustc.edu.cn/anaconda/cloud/
# 阿里镜像源
conda config --add channels https://mirrors.aliyun.com/pypi/simple/
# 清华镜像源
conda config --add channels
    https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main
conda config --add channels
    https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/r
conda config --add channels
    https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/msys2
# 清除索引缓存，保证用的是镜像站提供的索引
conda clean -i
# 显示检索路径，每次安装包时会将包源路径显示出来
conda config --set show_channel_urls yes
conda config --set always_yes True
# 显示所有镜像通道路径命令
conda config --show channels
# conda在指定环境下安装包
condaăinstallă-nămachine_algorithmămatplotlib
```

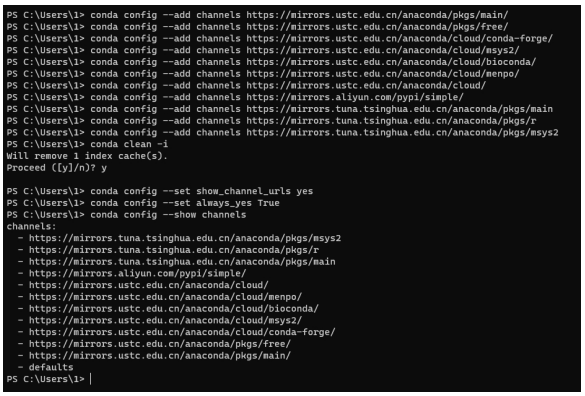

图 2.5 conda 使用终端配置国内源

(3) 移除镜像

如果想要移除镜像，可以在命令行中输入

```
conda config --remove channels 源名称或链接
```

(4) 验证

验证是否配置成功，可以在命令行中输入

```
conda config --show-sources
# 或
conda info
```

如果出现如图2.5所示的结果，说明配置成功。

四、 虚拟环境

像 Conda 这样的包管理工具，管理的是虚拟环境

1) 修改默认虚拟环境路径

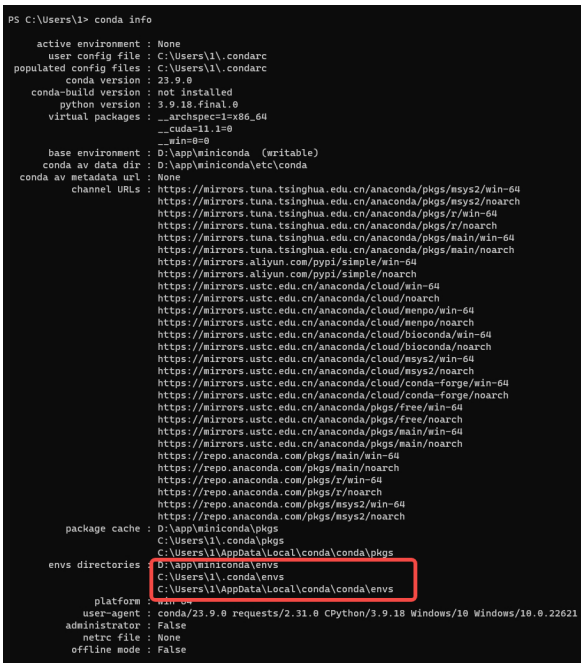

图 2.6 默认虚拟环境的默认路径

(1) Linux

```
$> mkdir  -p /home/tony/.local/conda_env # Linux
$> cat ~/.condarc
show_channel_urls: true
channels:
  - defaults
default_channels:
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/r
```

```
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/msys2
custom_channels:
  conda-forge: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
  msys2: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
  bioconda: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
  menpo: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
  pytorch: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
  pytorch-lts: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
  simpleitk: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
  deepmodeling: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/

envs_dirs:
 - /home/tony/.local/conda_env
```

(2) Windows

在`C:\Users\username\.condarc`文件中添加如图2.7所示的内容，然后在命令行中输入

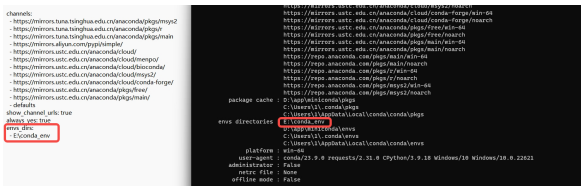

图 2.7 改变 conda 虚拟环境的默认路径

2) 创建虚拟环境

```
# 创建一个名为myenv的虚拟环境，指定Python版本是3.8，安装目录为E:\conda_env
conda create --name myenv python=3.9 -p E:\conda_env
```

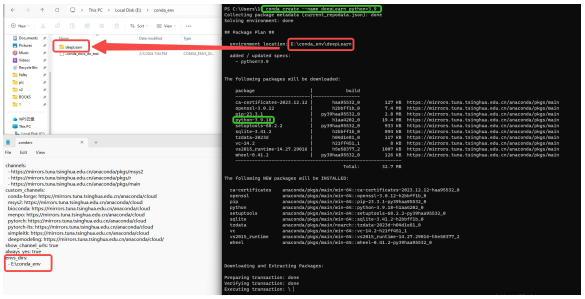

图 2.8 conda 创建虚拟环境

3) 查看虚拟环境

```
# 查看所有虚拟环境
conda info --envs
```

图 2.9 查看虚拟环境

4) 激活虚拟环境

```
# 激活虚拟环境
conda activate <env_name>
# 退出虚拟环境
conda deactivate
```

5) 删除虚拟环境

在删除虚拟环境之前，需要先退出虚拟环境

```
# 删除虚拟环境
conda remove --name <env_name> --all
```

6) 虚拟环境重新命名

```
# 重新命名虚拟环境
conda create --name new_name --clone old_name
```

五、 包管理

1) 查询包

```
# 查询包
conda search <package_name>

# 安装指定版本的包
conda install <package_name>=<version>
```

2) 安装包

(1) 在线安装

[I] conda 安装

conda 安装包会自动安装相关依赖的包

```
conda install pytorch torchvision torchaudio pytorch-cuda=11.8 -c pytorch
    -c nvidia
# -c 标志指定了要从哪个频道（channel）安装软件包。
# 频道是一组预先构建的二进制软件包的存储库，可以从中获取 Python
    包和其他依赖项。频道可以位于 Anaconda 或第三方存储库中。
# 在使用 -c 标志时，可以在其后指定要使用的频道名称或 URL。
```

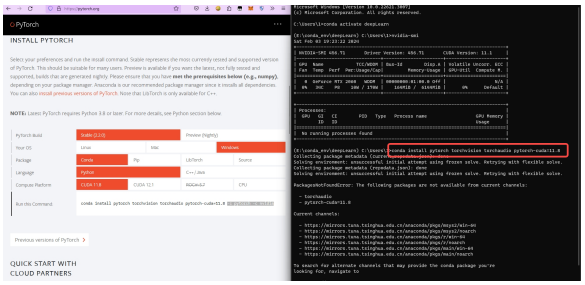

图 2.10 conda 安装 pytorch

[II] pip 安装

pip 安装会比较快，但是相关依赖的包可能不会被安装

```
pip3 install torch torchvision torchaudio --index-url
    https://download.pytorch.org/whl/cu118
```

(2) 离线安装

如果有的包就一个可以到pypi上下载编译好的`whl`文件，然后在命令行中输入

```
pip install 包名
conda install 包名
```

(3) 批量安装

事先写好一个`requirements.txt`文件

```
numpy
transformers==4.27.1
```

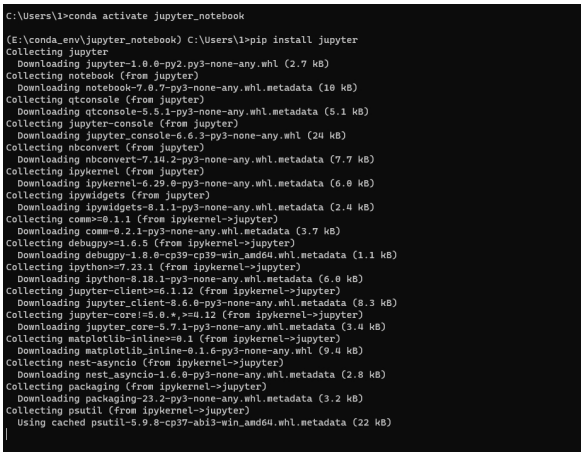

图 2.11 pip 安装 jupyter

```
torch>=1.10
matplotlib
```

然后在命令行中输入

```
conda install --file requirements.txt
pip install -r requirements.txt
```

3) 卸载包

```
# 卸载包
conda remove <package_name>
conda uninstall <package_name>
```

4) 查看包

```
# 查看所有已安装的包
conda list
# 查看指定环境下的包
conda list -n <env_name>
```

5) 更新包

```
# 更新所有包
conda update --all

# 更新指定包
```

```
conda update <package_name>

## 更新conda本身
conda update conda
```

6) 清空缓存

```
# 清空缓存
conda clean --all
```

六、 Vscode 配置

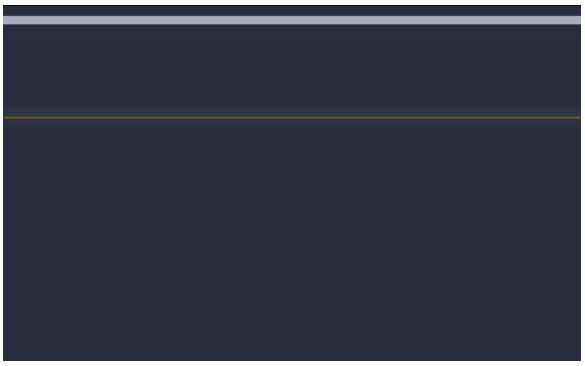

图 2.12 修改 python 解释器

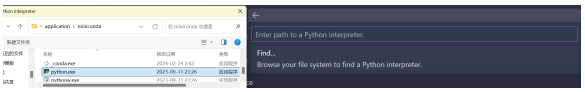

图 2.13 选择 conda 的 python 解释器

第三章 venv 环境

一、 概述

`venv`就是用来为一个应用创建一套“隔离”的 Python 运行环境

二、 使用

```
# 创建虚拟环境
python -m venv my_env

# 激活虚拟环境
## Linux
source my_env/bin/activate
## Windows
my_env\Scripts\activate.bat

# 退出虚拟环境
deactivate
```

三、 常用命令

1) 指定 python 版本

```
# 创建虚拟环境
python -m venv my_env --python=python3.6
```

四、 与 Vscode 结合

https://code.visualstudio.com/docs/python/environments

第四章 Jupter Notebook

一、 简介

Jupter Notebook 是基于网页的用于交互计算的应用程序。
其实 Jupter Notebook 与 Jupter lab 没什么大的区别
其功能

1) 开发
2) 运行代码
3) 文档编写
4) 展示结果

☞ Jupyter Notebook 建议仅用于学习目的，不推荐用于项目开发。

1) notebook 运行逻辑

Jupyter Notebook 不仅仅是一个简单的 web 应用程序，它还需要关联指定的 kernel，用于执行我们的代码。

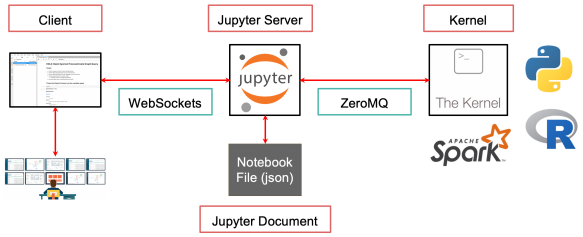

图 4.1 Jupyter Notebook 运行逻辑

如图4.1所示，左边是用户写的代码，传输到中间的 Jupyter server, server 本身不能执行代码，server 把代码传给 Kernel，Kernel 才是实际执行代码的组件，执行代码的地方。Kernel 执行完代码，把结果返回给 server，再返回到用户的网页。

Kernel 不仅可以是 python.exe，也可以是其他语言的解释器，如 Julia, R 等，更多 kernel 可以看支持的kernel 列表.

在 pytorch 开发中，kernel 其实就是某一个 python 解释器——python.exe，我们需要为当前的 notebook 配置相应的 kernel，来进入相应的虚拟环境，这样才能运行代码。

二、 安装

1) 安装 IPython(非必须)

```
# 安装
conda install ipython
# 或
pip install IPython # 安装IPython
```

2) 安装 Jupyter

```
# 安装
conda install jupyter
# 或
pip install jupyter
```

3) 添加 Kernel

```
# 1.进入虚拟环境
conda activate mypytorch
# 2.安装ipykernel工具包（安装jupyter）
conda install ipykernel
# 或
pip install ipykernel
# 3.为当前环境添加kernel
python -m ipykernel install --user --name=mypytorch
```

4) 代码补全

```
# 安装
pip install jupyter_contrib_nbextensions
## 有时会不匹配报错 pip install -i
    https://pypi.tuna.tsinghua.edu.cn/simple jupyter_contrib_nbextensions

conda install -c conda-forge jupyter_contrib_nbextensions
```

三、 使用

1) 本地使用

在那个目录下打开命令行，打开 jupyter lab 以后就是那个目录。

```
# 切换到目标目录
cd 目标目录
# 激活虚拟环境
conda activate jupyter_notebook
# 在命令行中输入
jupyter notebook
# 或
jupyter lab # (推荐)更加强大的版本
```

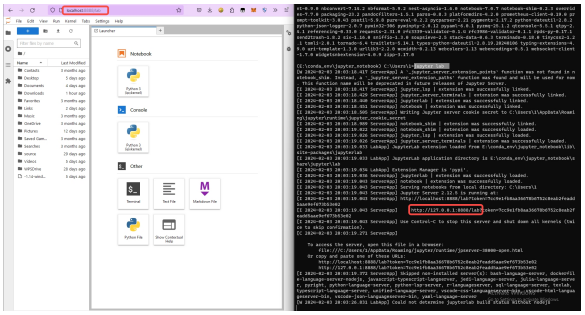

图 4.2 启动 jupyter lab

2) 远程使用

(1) 远程服务器配置

如图4.3所示，配置远程连接 JupterNotebook。

```
$(conda_env)> jupyter notebook --generate-config
Writing default config to: /home/tony/.jupyter/jupyter_notebook_config.py

(conda_env)> jupyter lab --generate-config
Writing default config to: /home/tony/.jupyter/jupyter_lab_config.py

$(conda_env)> vim ~/.jupyter/jupyter_notebook_config.py
c.NotebookApp.ip = '*'                          # 允许访问此服务器的
    IP，星号表示任意 IP
c.NotebookApp.open_browser = False              # 运行时不打开本机浏览器
#c.NotebookApp.port = 8890                      # 使用端口，默认是 8888
c.NotebookApp.enable_mathjax = True             # 启用 MathJax
c.NotebookApp.allow_remote_access = True        #允许远程访问
#c.NotebookApp.allow_root = True                # 允许以root用户启动

$(conda_env)> jupyter notebook --no-browser --port=8888

$(conda_env)> jupyter lab --no-browser --port=8888
$(conda_env)> nohup jupyter lab --no-browser --port=8888 > $(date
    '+%Y-%m-%d-%H-%M-%S')-jupyter.log 2>&1 &
```

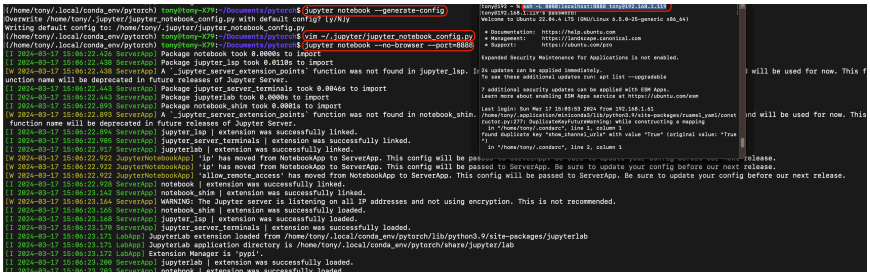

图 4.3 配置远程连接 JupterNotebook

(2) 本地访问远程 JupterLab

[I] 配置防火墙访问

```
$(conda_env)> jupyter lab --no-browser --port=8888 #
    在远程服务器上启动JupterLab
# 临时设置
$ sudo firewall-cmd --list-ports        # 列出所有开放都得端口
$ sudo firewall-cmd --add-port=8888/tcp # 添加临时的开放端口
$ sudo firewall-cmd --remove-port=8888/tcp # 关闭临时的开放端口

# 永久设置
$ firewall-cmd --list-ports --permanent # 查看所有永久开放的端口（默认为空）
$ sudo firewall-cmd --add-port=8888/tcp --permanent # 添加永久的开放端口
$ sudo firewall-cmd --remove-port=8888/tcp --permanent # 关闭永久的开放端口

# 配置结束后需要输入重载命令并重启防火墙以生效配置
$ sudo firewall-cmd --reload
$ sudo systemctl restart firewalld
```

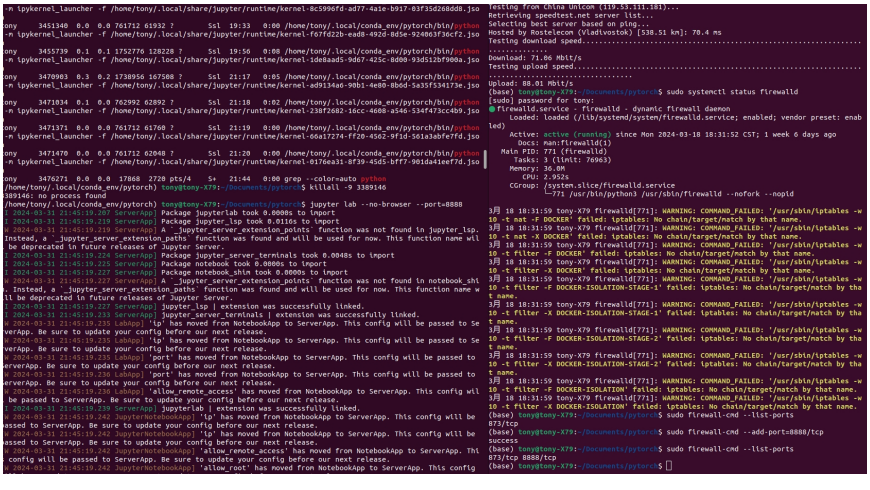

图 4.4 远程开启 jupyter lab 防火墙

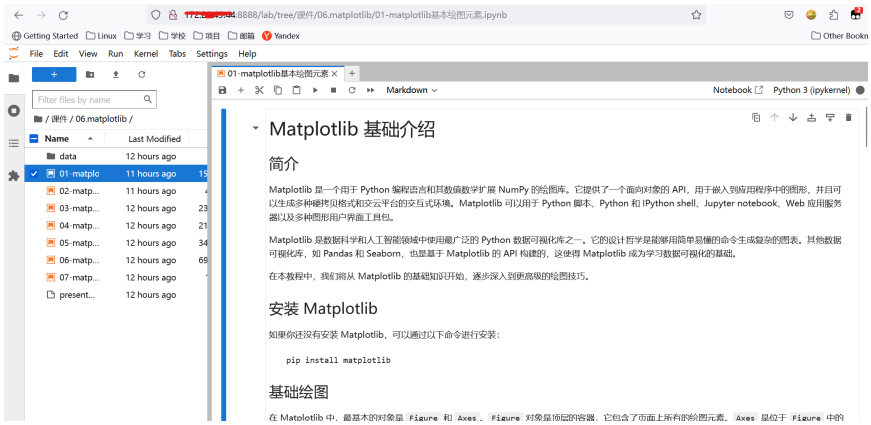

图 4.5 配置防火墙访问 JupterLab

[II] SSH 端口转发

配置完需要使用 ssh 的端口转发 (-L) 功能，将远程的端口转发到本地 (8080)，然后在本地访问即可, 如图4.6所示。

```
# ssh -L [本地端口]:localhost:[远程端口] [远程用户名]@[远程IP] -p
    [ssh连接端口]
ssh -L 8080:localhost:8888 tony@192.168.1.119
```

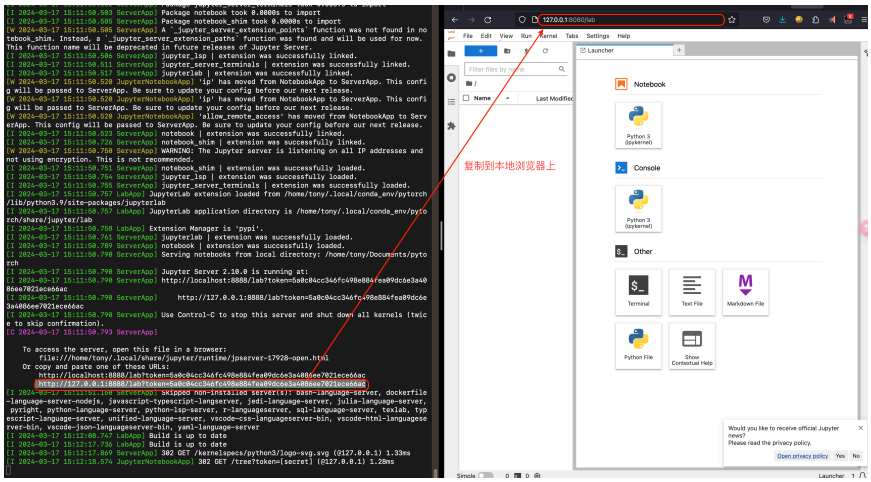

图 4.6 访问远程 JupterLab, 这里是 localhost:8080

[III] 配置密码

按照如图4.7所示设置密码。然后按照如图4.8所示登入。第一次登入需要输入密码，以后就不需要了。

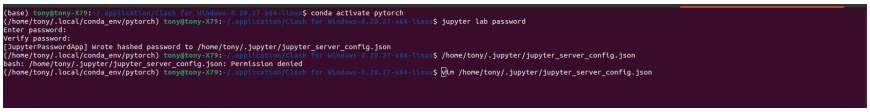

图 4.7 jupyterlab 设置密码

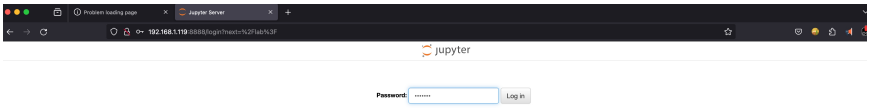

图 4.8 jupyterlab 登入界面

第五章 Jupter Notebook 语法

一、 JupyterLab 的语法指令

1) writefile 写到一个文件

```
%%writefile 文件名
内容
```

将当前 cell 的内容写入到文件中。

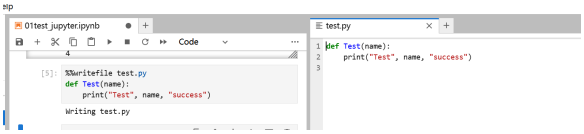

图 5.1 writefile 指令

2) pycat 显示一个文件

```
%pycat 文件名
```

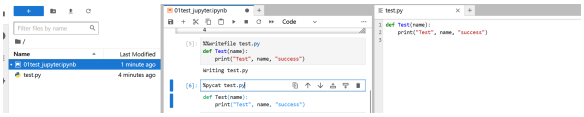

图 5.2 pycat 指令

3) run 执行一个文件

```
%run 文件名
函数名(参数)
```

当然也可以通过`import 文件名`导入文件,`from 文件名 import 函数名`导入函数。

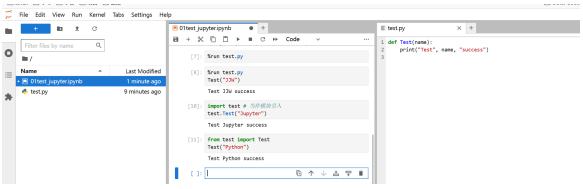

图 5.3 run 指令

4) 计算代码运行时间

多轮测试，取平均值。

```
%timeit 函数名(参数) # 测试一条语句的运行时间

%%timeit # 测试一个cell的运行时间
```

只测试一次。

```
%time 函数名(参数) # 测试一条语句的运行时间

%%time # 测试一个cell的运行时间
```

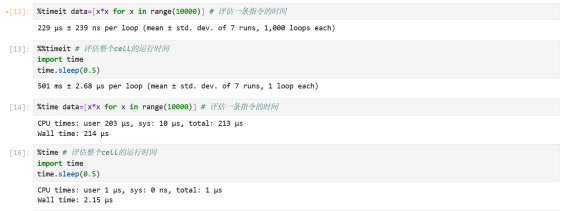

图 5.4 timeit 和 time 指令

5) whos 查看当前内存中的变量

使用`%whos`或者`%who`查看当前内存中的变量。
包括定义的变量、函数、模块等。

6) lsmagic 查看所有的魔法指令

```
[10]: import test # 当作模块引入
      test.Test("Jupyter")

      Test Jupyter success

[11]: from test import Test
      Test("Python")

      Test Python success

*[12]: %timeit data=[x*x for x in range(10000)] # 评估一条指令的时间

       229 µs ± 239 ns per loop (mean ± std. dev. of 7 runs, 1,000 loops each)

[13]: %%timeit # 评估整个cell的运行时间
      import time
      time.sleep(0.5)

      501 ms ± 2.68 µs per loop (mean ± std. dev. of 7 runs, 1 loop each)

[14]: %time data=[x*x for x in range(10000)] # 评估一条指令的时间

      CPU times: user 203 µs, sys: 10 µs, total: 213 µs
      Wall time: 214 µs

[16]: %time # 评估整个cell的运行时间
      import time
      time.sleep(0.5)

      CPU times: user 1 µs, sys: 0 ns, total: 1 µs
      Wall time: 2.15 µs

[17]: a = 123

[18]: import torch

[19]: %whos

      Variable   Type        Data/Info
      --------------------------------
      Test       function    <function Test at 0x79d4143409d0>
      a          int         123
      data       list        n=10000
      i          int         4
      test       module      <module 'test' from '/hom<...>/python/pytorch/test.py'>
      time       module      <module 'time' (built-in)>
      torch      module      <module 'torch' from '/ho<...>kages/torch/__init__.py'>
```

图 5.5 whos 指令

```
[20]: %lsmagic

[20]: root
        line
        cell
          js "DisplayMagics"
          javascript "DisplayMagics"
          latex "DisplayMagics"
          svg "DisplayMagics"
          html "DisplayMagics"
          markdown "DisplayMagics"
          prun "ExecutionMagics"
          debug "ExecutionMagics"
          timeit "ExecutionMagics"
          time "ExecutionMagics"
          capture "ExecutionMagics"
          code_wrap "ExecutionMagics"
          sx "OSMagics"
          system "OSMagics"
          ! "OSMagics"
          writefile "OSMagics"
          script "ScriptMagics"
          sh "Other"
          bash "Other"
          perl "Other"
          ruby "Other"
          python "Other"
          python2 "Other"
          python3 "Other"
          pypy "Other"
          SVG "Other"
          HTML "Other"
          file "Other"
```

图 5.6 lsmagic 指令

第六章 自定义包

一、 模块

在 python 中，一个扩展名为"`.py`" 的文件称为一个**模块** (module).

模块可认为是一盒 (箱) 主题积木，通过它可以拼出某一主题 (某个系统、某个复杂求解问题) 的东西。

与函数或类不同，一个函数或类相当于一块积木，而一个模块中可以包括很多函数或类.

二、 概述

Python 中，"Package" 就是一个目录，其中包括一组模块和一个"`__init__.py`" 文件。其中 `__init__.py` 文件定义了包的属性和方法，通常只是一个空文件，但是必须存在，**切勿随意删除若删除** `__init__.py` 文件，则该 Package 仅仅是一个目录，无法被导入或包含其他模型库和嵌套包.

第二部分

基本语法

第七章 操作数相关

Python 是弱类型语言, 变量的类型是根据赋值的值来决定的。

一、 概述

变量是数据操作和数值计算的起始点，算法用到的各种数据都是存储在变量内。

Python 是一门" 弱类型" 语言，所有的变量无须声明即可使用，变量的数据类型可以随时改变. Python3 中有六个标准的数据类型，分类如下:

1) 不可变数据类型
    1) Number (数字或数值)
    2) String (字符串)
    3) Thple(元组)
2) 可变数据类型——序列
    1) List (列表)
    2) Set (集合)
    3) Dictionary (字典)

1) 标识符 (变量名)

1) 第一个字符必须以字母 (`a-z`, `A-Z`) 或下划线 `_` 。
2) 标识符的其他的部分由字母、数字和下划线组成。
3) 标识符对大小写敏感，count 和 Count 是不同的标识符。
4) 标识符对长度无硬性限制，但建议保持简洁（一般不超过 20 个字符）。
5) 禁止使用保留关键字，如 `if`、`for`、`class` 等不能作为标识符。

```
# 合法标识符:
age = 25
user_name = "Alice"
_total = 100
MAX_SIZE = 1024
calculate_area()
StudentInfo
__private_var

# 非法标识符:
2nd_place = "silver"     # 错误: 以数字开头
user-name = "Bob"        # 错误: 包含连字符
class = "Math"           # 错误: 使用关键字
$price = 9.99           # 错误: 包含特殊字符
for = "loop"            # 错误: 使用关键字
```

Python 3 允许使用 Unicode 字符作为标识符，可以用中文作为变量名，非 ASCII 标识符也是允许的了。

```
姓名 = "张三"  # 合法
 = 3.14159   # 合法
```

第八章 基本数据类型

一、 Number——数字类型

Python 支持三种不同的 Number 数值类型:

1) 整型 (int)：整数类型，表示没有小数部分的数字。

2) 浮点型 (float)：浮点数类型，表示有小数部分的数字。

3) 复数 (complex)：复数类型，表示实部和虚部

如果需要对数据内置的类型进行转换，只需要将数据类型作为函数名即可，如 `int(x)` 转换为一个整数.

1) int 整数类型

(1) bool 布尔类型

Python 中 bool 是 int 的子类 (继承 `int`), 故`True==1`、`False==0`。
如要切实判断，则采用: `XXX is True`.

(2) 数字分隔符

为了提高数字的的可读性, Python 3.x 允许使用下划线 `_` 作为数字（包括整数和小数）的分隔符。通常每隔三个数字添加一个下划线, 类似于英文数字中的逗号。下划线不会影响数字本身的值。

```
>>> 1_000_000
1000000
>>> click = 1_301_547
>>> distance = 384_000_000
>>> print("Python 教程阅读量: ", click)
>>> print("地球和月球的距离: ", distance)
Python 教程阅读量:  1301547
地球和月球的距离:  384000000
```

2) float——浮点数 (科学计数法)

Python 3.x 支持科学计数法, 用`e`或`E`表示幂。

```
>>> 1e6
1000000.0
>>> 1e-6
0.000001
>>> 1e+6
1000000.0
>>> 1e+0006
1000000.0
>>> 1e+0000006
1000000.0
>>> 2.1E5
210000.0
```

(1) 浮点数精度问题

Python 中浮点类型之间的运算, 其结果并不像我们想象的那样

```
>>> 0.1+0.2
0.30000000000000004
>>> 0.1+0.1-0.2
0.0
>>> 0.1+0.1+0.1-0.3
5.551115123125783e-17
>>> 0.1+0.1+0.1-0.2
0.10000000000000003
```

如果需要非常精确的结果, 可以使用`decimal`模块（其实就是别人开发好的程序, 我们可以直接拿来用）, 它实现的十进制数运算适合会计方面的应用和有高精度要求的应用。例如：

```
import decimal
a = decimal.Decimal("10.0")
b = decimal.Decimal("3")
print(10.0/3)
print(a/b)
```

```
3.3333333333333335
3.3333333333333333333333333333
```

如果`decimal`模块还是无法满足需求, 还可以使用`fractions`模块, 例如：

```
>>> from fractions import Fraction
>>> a = Fraction(1, 3)
>>> b = Fraction(1, 3)
>>> a + b
```

```
Fraction(2, 3)
>>> a - b
Fraction(0, 3)
>>> a * b
Fraction(1, 9)
>>> a / b
Fraction(3, 1)
>>> print(Fraction(10,3))
10/3
```

3) bool——布尔类型

布尔类型包含

1) `True`: 真

2) `False`: 假

4) complex——复数类型

二、 类型转换

1) type

```
Python 3.9.19 (main, Mar 21 2024, 17:11:28)
[GCC 11.2.0] :: Anaconda, Inc. on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> type(8)
<class 'int'>
>>> type("8")
<class 'str'>
>>> type(False)
<class 'bool'>
>>> type(true)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'true' is not defined
>>> num = 20 + 15j
>>> type(num)
<class 'complex'>
```

2) 显式类型转换

```
>>> int("8") # 将字符串转换为整数
8
>>> float("8")
8.0
```

```
>>> str(8) # 将整数转换为字符串
'8'
>>> bool(8) # 将整数转换为布尔值
True
>>> bool(0)
False
```

三、 缓存重用机制

Python 缓冲机制是为提高程序执行的效率服务的, 实际上就是在 Python 解释器启动时从内存空间中开辟出一小部分, 用来存储高频使用的数据, 这样可以大大减少高频使用的数据创建时申请内存和销毁时撤销内存的开销。

数据类型	是否可以重用
范围在 [-5, 256] 之间的小整数	如果之前在程序中创建过, 就直接存入缓存, 后续不再创建。
bool 类型	
字符串类型数据	
大于 256 的整数	只要在本代码块内创建过, 就直接缓存, 后续不再创建。
大于 0 的浮点型小数	
小于 0 的浮点型小数	不进行缓存, 每次都需要额外创建。
小于 -5 的整数	

表 8.1 缓存重用机制

```
#范围在 [-5, 256] 之间的小整数
int1 = -5
int2 = -5
print("[-5, 256] 情况下的两个变量: ", id(int1), id(int2))
#bool 类型
bool1 = True
bool2 = True
print("bool 类型情况下的两个变量: ",id(bool1),id(bool2))

#对于字符串
s1 = "3344"
s2 = "3344"
print("字符串情况下的两个交量", id(s1), id(s2))
#大于 256 的整数
int3 = 257
int4 = 257
print("大于 256 的整数情况下的两个变量", id(int3), id(int4))
#大于 0 的浮点数
f1 = 256.4
f2 = 256.4
```

```
print("大于 0 的浮点数情况下的两个变量", id(f1), id(f2))
#小于 0 的浮点数
f3 = -2.45
f4 = -2.45
print("小于 0 的浮点数情况下的两个变量", id(f3), id(f4))
#小于 -5 的整数
n1 = -6
n2 = -6
print("小于 -5 的整数情况下的两个变量", id(n1), id(n2))
```

```
[-5, 256] 情况下的两个变量:  2169665579120 2169665579120
bool 类型情况下的两个变量:  140731067889768 140731067889768
字符串情况下的两个交量 2169668343408 2169668343408
大于 256 的整数情况下的两个变量 2169668069648 2169668069648
大于 0 的浮点数情况下的两个变量 2169666229936 2169666229936
小于 0 的浮点数情况下的两个变量 2169668070320 2169668070320
小于 -5 的整数情况下的两个变量 2169668070032 2169668070032
```

```
def fun():
    #[-5,256]
    int1 = -5
    int2 = -5
    print("fun 中 -5 的存储状态",id(int1), id(int2))

    #bool 类型
    bool2 = True
    bool3 = True
    print("fun 中 bool 类型的存储状态",id(bool3),id(bool2))

    #字符串类型
    s1 = "3344"
    s2 = "3344"
    print("fun 中 3344 字符串的存储状态", id(s1), id(s2))

    #大于 256
    int3 = 257
    int4 = 257
    print("fun 中 257 的存储状态", id(int3), id(int4))

    #浮点类型
    f1 = 256.4
    f2 = 256.4
    print("fun 中 256.4 的存储状态",id(f1), id(f2))

    #小于 -5
```

```
    n1 = -6
    n2 = -6
    print("fun 中 -6 的存储状态", id(n1), id(n2))

fun()
```

```
fun 中 -5 的存储状态 2022312994928 2022312994928
fun 中 bool 类型的存储状态 140731067889768 140731067889768
fun 中 3344 字符串的存储状态 2022319364656 2022319364656
fun 中 257 的存储状态 2022319089936 2022319089936
fun 中 256.4 的存储状态 2022313645744 2022313645744
fun 中 -6 的存储状态 2022319090032 2022319090032
```

第九章 [序列] 数据结构

一、 序列

序列: 一块可存放多个值的连续内存空间，这些值按一定顺序排列，可通过每个值所在位置的编号（称为索引）访问它们。

1) 序列内置函数

下面的函数适用于所有序列类型，包括字符串、列表、元组等。

(1) len 序列的长度

1) 字符串

```
chinese_zodiac = "鼠牛虎兔龙蛇马羊猴鸡狗猪"
print(len(chinese_zodiac))
>>> 12
```

2) 列表

```
name = ['张三', '李四', '王五']
print(len(name))
>> 3
```

3) 元组

```
name = ('张三', '李四', '王五')
print(len(name))
>> 3
```

4) 字典

```
name = {'name': '张三', 'age': 18}
print(len(name))
>> 2
```

5) 集合

```
name = {'张三', '李四', '王五'}
print(len(name))
>> 3
```

(2) max 序列的最大值

1) 字符串

```
chinese_zodiac = "鼠牛虎兔龙蛇马羊猴鸡狗猪"
print(max(chinese_zodiac))
>>> 龙

my_string = "hello"
print(max(my_string))  # 输出: 'o'
>>> o
```

2) 列表

```
name = ['张三', '李四', '王五']
print(max(name))
>>> 王五

my_list = [1, 2, 3, 4, 5]
print(max(my_list))  # 输出: 5
>>> 5
```

3) 元组

```
name = ('张三', '李四', '王五')
print(max(name))
>>> 王五

my_tuple = (1, 2, 3)
print(max(my_tuple))  # 输出: 3
>>> 3
```

4) 字典

```
my_dict = {'a': 1, 'b': 2, 'c': 3}
print(max(my_dict))  # 输出: ('c', 3)
>>> c
print(max(my_dict.values()))
>>> 3
```

5) 集合

```
my_set = {1, 2, 3, 4}
print(max(my_set))  # 输出：4
>>> 4
```

(3) min 序列的最小值

1) 字符串

```
chinese_zodiac = "鼠牛虎兔龙蛇马羊猴鸡狗猪"
print(min(chinese_zodiac))  # 输出：'鼠'
>>> 鼠
```

2) 列表

```
name = ['张三', '李四', '王五']
print(min(name))  # 输出：'张三'
>>> 张三
```

3) 元组

```
name = ('张三', '李四', '王五')
print(min(name))  # 输出：'张三'
>>> 张三
```

4) 字典

```
my_dict = {'a': 1, 'b': 2, 'c': 3}
print(min(my_dict))  # 输出：('a', 1)
>>> a
print(min(my_dict.values()))
>>> 1
```

5) 集合

```
my_set = {1, 2, 3, 4}
print(min(my_set))  # 输出：1
>>> 1
```

(4) sum 返回序列的和

1) 字符串

字符串不能使用 `sum` 函数。

2) 列表

```
my_list = [1, 2, 3, 4, 5]
print(sum(my_list))  # 输出：15
```

3) 元组

```
my_tuple = (1, 2, 3)
print(sum(my_tuple))  # 输出：6
```

4) 字典

字典只能对值进行求和。

```
# 字典（需要对值进行求和）
my_dict = {'a': 1, 'b': 2, 'c': 3}
print(sum(my_dict.values()))  # 输出：6
```

5) 集合

```
my_set = {1, 2, 3, 4}
print(sum(my_set))  # 输出：10
```

(5) sorted 返回排序后的序列

1) 字符串

```
my_string = "cba"
print(sorted(my_string))
>>> ['a', 'b', 'c']
```

2) 列表

```
my_list = [3, 1, 4, 1, 5]
print(sorted(my_list))
>>> [1, 1, 3, 4, 5]
```

3) 元组（排序后转为列表）

```
my_tuple = (3, 1, 4, 1, 5)
print(sorted(my_tuple))
>>> [1, 1, 3, 4, 5]
```

4) 字典（按键排序）

```
my_dict = {'b': 2, 'a': 1, 'c': 3}
print(sorted(my_dict))
>>> ['a', 'b', 'c']
```

5) 集合（排序后转为列表）

```
my_set = {3, 1, 4}
print(sorted(my_set))
>>> [1, 3, 4]
```

(6) reversed 返回逆序的序列

1) 字符串

```
my_string = "hello"
print(''.join(reversed(my_string)))  # 输出："olleh"
>>> olleh
```

2) 列表

```
my_list = [1, 2, 3, 4, 5]
print(list(reversed(my_list)))  # 输出：[5, 4, 3, 2, 1]
>>> [5, 4, 3, 2, 1]
```

3) 元组

```
my_tuple = (1, 2, 3, 4, 5)
print(tuple(reversed(my_tuple)))  # 输出：(5, 4, 3, 2, 1)
>>> (5, 4, 3, 2, 1)
```

4) 字典（不能直接逆序，但可以逆序键或值）

```
my_dict = {'a': 1, 'b': 2, 'c': 3}
print(list(reversed(my_dict)))  # 输出：['c', 'b', 'a'] (逆序键)
>>> ['c', 'b', 'a']
```

5) 集合

集合是无序的，所以不能使用 `reversed` 函数。

(7) enumerate 返回索引和元素组成的元组

1) 字符串

字符串不能使用 `enumerate` 函数。

2) 列表

```
>>> my_list = ['a', 'b', 'c']
>>> for index, value in enumerate(my_list):
...     print(index, value)
...
```

```
0 a
1 b
2 c
```

3) 元组

```
>>> my_tuple = ('a', 'b', 'c')
>>> for index, value in enumerate(my_tuple):
...     print(index, value)
...
0 a
1 b
2 c
```

4) 字典

字典不能使用 `enumerate` 函数。

5) 集合

集合不能使用 `enumerate` 函数。

2) 序列的通用操作

(1) [] 索引访问

1) 字符串

```
>>> chinese_zodiac = "鼠牛虎兔龙蛇马羊猴鸡狗猪"
>>> print(chinese_zodiac[0])
鼠
>>> print(chinese_zodiac[0:4])
鼠牛虎兔
>>> print(chinese_zodiac[-1])
猪
>>> print(chinese_zodiac[-3:-1])
鸡狗
>>> print(chinese_zodiac[-3:])
鸡狗猪
>>> print(chinese_zodiac[0:-1:2])
鼠虎龙马猴狗
```

2) 列表

```
变量名 = [值1, 值2, 值3,...]

name = ['张三', '李四', '王五']
print(name[1])
>> 李四
```

3) 元组

```
变量名 = (值1, 值2, 值3,...)

name = ('张三', '李四', '王五')
print(name[1])
>> 李四
```

4) 字典

```
变量名 = {键1:值1, 键2:值2, 键3:值3,...}

name = {'name':'张三', 'age':18, 'sex':'男'}
print(name['name'])
>> 张三
```

5) 集合

```
变量名 = {值1, 值2, 值3,...}

name = {'张三', '李四', '王五'}
print(name[1])
>> 李四
```

(2) index 查找元素的索引

1) 字符串

字符串没有 `index` 方法，但是可以使用 `find` 方法。

```
# 字符串
my_string = "Hello, World!"
index_of_world = my_string.find("World")  # 查找子字符串 "World"
    的索引
print(index_of_world)  # 输出: 7

# 未找到时返回 -1
index_of_python = my_string.find("Python")
print(index_of_python)  # 输出: -1
```

2) 列表

```
变量名 = [1, 2, 3, 4, 5]

print(变量名.index(3))
>> 2

# 查找不存在的元素会抛出 ValueError
# print(my_list.index(6))  # ValueError: 6 is not in list
```

3) 元组

```
变量名 = (1, 2, 3, 4, 5)

print(变量名.index(3))
>> 2
```

4) 字典

字典没有 `index` 方法，因为字典是键值对集合，而不是简单的元素序列。通常，我们会直接通过键来访问值。

```
变量名 = {'key1': 'value1', 'key2': 'value2'}

print(变量名.keys())
>> dict_keys(['key1', 'key2'])

print(变量名.values())
>> dict_values(['value1', 'value2'])

print(变量名.items())
>> dict_items([('key1', 'value1'), ('key2', 'value2')])
```

5) 集合

集合没有 `index` 方法，因为集合是无序的。

(3) count 统计元素出现的次数

1) 字符串

```
my_string = "Hello, world! Hello, everyone!"
count_of_hello = my_string.count("Hello")  # 统计子字符串 "Hello"
    出现的次数
print(count_of_hello)
>>> 2
```

2) 列表

```
my_list = [1, 2, 3, 1, 2, 3, 1, 2, 3]
count_of_1 = my_list.count(1)  # 统计元素 1 出现的次数
print(count_of_1)
>>> 3
```

3) 元组

```
my_tuple = (1, 2, 3, 1, 2, 3, 1, 2, 3)
count_of_1 = my_tuple.count(1)  # 统计元素 1 出现的次数
print(count_of_1)
>>> 3
```

4) 字典

字典没有 `count` 方法，因为字典是键值对集合，而不是简单的元素序列。

5) 集合

集合没有 `count` 方法，因为集合中的元素是唯一的，每个元素最多只出现一次。

(4) 切片

```
list[start:stop:step]
# start: 起始位置
# stop: 终止位置
# step: 步长
```

返回 左闭右开区间

1) `[start:stop:step]`: 返回一个新的序列

2) `start`: 起始位置

3) `stop`: 终止位置

4) `step`: 步长

1) 字符串

```
>>> chinese_zodiac = "鼠牛虎兔龙蛇马羊猴鸡狗猪"
>>> print(chinese_zodiac[0:4])
鼠牛虎兔
>>> print(chinese_zodiac[0:-1:2])
鼠虎龙马猴狗
```

2) 列表

```
变量名 = [值1, 值2, 值3,...]
变量名[起始位置:终止位置:步长]

name = ['张三', '李四', '王五', '乔治', '佩奇']
print(name[1:3])
>> ['李四', '王五']
print(name[3:])
>> ['乔治', '佩奇']
```

3) 元组

```
变量名 = (值1, 值2, 值3,...)
变量名[起始位置:终止位置:步长]
```

```
name = ('张三', '李四', '王五', '乔治', '佩奇')
print(name[1:3])
>> ('李四', '王五')
print(name[3:])
>> ('乔治', '佩奇')
```

4) 字典

```
变量名 = {键1:值1, 键2:值2, 键3:值3,...}
变量名[起始位置:终止位置:步长]

name = {'name':'张三', 'age':18, 'sex':'男'}
print(name['name'])
>> 张三
```

5) 集合

因为集合是无序的，所以不能使用切片。

(5) split 分割

1) 字符串

```
>>> chinese_zodiac = "鼠牛虎兔龙蛇马羊猴鸡狗猪"
>>> print(chinese_zodiac.split(" "))    # 默认分割符为空格
['鼠', '牛', '虎', '兔', '龙', '蛇', '马', '羊', '猴', '鸡', '狗',
    '猪']
>>> print(chinese_zodiac.split(" ", 3)) # 最多分割3次
['鼠', '牛', '虎', '兔龙蛇马羊猴鸡狗猪']
>>> english_zodiac = "rat ox tiger rabbit dragon snake horse sheep
    monkey rooster dog pig"
>>> print(english_zodiac.split(" ", 3))
['rat', 'ox', 'tiger', 'rabbit dragon snake horse sheep monkey
    rooster dog pig']
```

2) 列表、元组、字典、集合 **都**不支持 `split` 方法。

(6) in/not in 成员关系

1) `in`: 判断某个元素是否在序列中

2) `not in`: 判断某个元素是否不在序列中

1) 字符串

```
>>> chinese_zodiac = "鼠牛虎兔龙蛇马羊猴鸡狗猪"
>>> print("狗" in chinese_zodiac)
True
>>> print("狗" not in chinese_zodiac)
False
```

2) 列表

```
name = ['张三', '李四', '王五']
print('李四' in name)
>> True
print('李四' not in name)
>> False
```

3) 元组

```
name = ('张三', '李四', '王五')
print('李四' in name)
>> True
print('李四' not in name)
>> False
```

4) 字典

```
name = {'name': '张三', 'age': 18}
print('name' in name)
>> True
print('name' not in name)
>> False
```

5) 集合

```
name = {'张三', '李四'}
print('张三' in name)
>> True
print('张三' not in name)
>> False
```

(7) 拼接

1) `+`: 将两个序列拼接

2) `join`: 将序列中的元素以指定字符连接

1) 字符串

```
>>> a = "123"
>>> b = "456"
>>> print(a + b)
123456
```

2) 列表

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]
print(list1 + list2)
>>> [1, 2, 3, 4, 5, 6]
```

3) 元组

```
tuple1 = (1, 2, 3)
tuple2 = (4, 5, 6)
print(tuple1 + tuple2)
>>> (1, 2, 3, 4, 5, 6)
```

4) 字典

字典不能使用 `+` 进行拼接。

但是字典有 `|` 并集，可以进行拼接。

```
set1 = {1, 2, 3}
set2 = {4, 5, 6}
print(set1 | set2)
>>> {1, 2, 3, 4, 5, 6}
```

5) 集合

集合不能使用 `+` 进行拼接。

(8) join 连接

1) 字符串

```
my_list = ['a', 'b', 'c']
result = ''.join(my_list)  # 使用空字符串连接
print(result)
>>> abc

result_with_dash = '-'.join(my_list)
print(result_with_dash)
>>> a-b-c
```

2) 列表

```
>>> chinese_zodiac = "鼠牛虎兔龙蛇马羊猴鸡狗猪"
>>> print(" ".join(chinese_zodiac))
鼠 牛 虎 兔 龙 蛇 马 羊 猴 鸡 狗 猪
```

3) 元组

```
>>> my_tuple = ('a', 'b', 'c')
>>> print(my_tuple * 3)
('a', 'b', 'c', 'a', 'b', 'c', 'a', 'b', 'c')
```

4) 字典

```
# 字典
my_dict = {'a': 1, 'b': 2, 'c': 3}

# 连接字典的键
keys_result = ''.join(my_dict.keys())
print(keys_result)
>>> abc

# 连接字典的值（需要先将值转换为字符串）
values_result = ''.join(map(str, my_dict.values()))
print(values_result)
>>> 123
```

5) 集合

```
my_set = {1, 2, 3, 4, 5}

# 连接集合的元素
elements_result = ''.join(map(str, my_set))
print(elements_result)
>>> 12345
```

(9) *重复

1) 字符串

```
>>> chinese_zodiac = "鼠牛虎兔龙蛇马羊猴鸡狗猪"
>>> print(chinese_zodiac * 3)
鼠牛虎兔龙蛇马羊猴鸡狗猪鼠牛虎兔龙蛇马羊猴鸡狗猪鼠牛虎兔龙蛇马羊猴鸡狗猪
```

2) 列表

```
my_list = [1, 2, 3]
print(my_list * 2)
>>> [1, 2, 3, 1, 2, 3]
```

3) 元组

```
my_tuple = (1, 2, 3)
print(my_tuple * 2)
>>> (1, 2, 3, 1, 2, 3)
```

4) 字典

字典不支持重复操作。

5) 集合

集合不支持重复操作。

(10) append 追加

1) 字符串

字符串不支持 `append` 方法。

2) 列表

```
my_list = [1, 2, 3]
my_list.append(4)
print(my_list)
>>> [1, 2, 3, 4]
```

3) 元组

元组不支持 `append` 方法。

4) 字典

```
my_dict = {'a': 1, 'b': 2}
my_dict['c'] = 3
print(my_dict)
>>> {'a': 1, 'b': 2, 'c': 3}
```

5) 集合

```
my_set = {1, 2, 3}
my_set.add(4)
print(my_set)
>>> {1, 2, 3, 4}
```

(11) extend 扩展

1) 字符串

```
my_str = 'abc'
my_str.extend('def')
print(my_str)
>>> abcdef
```

2) 列表

`extend`方法 只接收列表中的元素逐一添加到另一个列表中。

该方法会将参数列表中的元素逐一添加到原列表中，而不是将参数列表作为一个整体添加到原列表中。

```
my_list = [1, 2, 3]
my_list.extend([4, 5])
print(my_list)
>>> [1, 2, 3, 4, 5]
```

3) 元组

元组不支持 `extend` 方法。

4) 字典

```
my_dict = {'a': 1, 'b': 2}
my_dict.update({'c': 3, 'd': 4})
print(my_dict)
>>> {'a': 1, 'b': 2, 'c': 3, 'd': 4}
```

5) 集合

```
my_set = {1, 2, 3}
my_set.update({4, 5})
print(my_set)
>>> {1, 2, 3, 4, 5}
```

(12) insert 插入

1) 字符串

字符串不支持 `insert` 方法。

2) 列表

```
my_list = [1, 2, 3]
my_list.insert(1, 1.5)
print(my_list)
>>> [1, 1.5, 2, 3]
```

3) 元组

元组不支持 `insert` 方法。

4) 字典

```
my_dict = {'a': 1, 'b': 2}
my_dict.insert('c', 3)
```

```
print(my_dict)
>>> {'a': 1, 'b': 2}
```

5) 集合

集合不支持 `insert` 方法。

(13) update 更新

1) 字符串

字符串是不可变的，因此没有 `update` 方法。

2) 列表

列表的 `update` 方法是 `extend` 方法。

3) 元组

元组不支持 `update` 方法。

4) 字典

```
my_dict = {'a': 1, 'b': 2}
my_dict.update({'b': 3, 'c': 4})  # 更新键 'b' 的值，并添加键 'c'
print(my_dict)
>>> {'a': 1, 'b': 3, 'c': 4}
```

5) 集合

```
my_set = {1, 2, 3}
my_set.update({3, 4, 5})  # 添加元素 4 和 5
print(my_set)
>>> {1, 2, 3, 4, 5}
```

(14) remove 删除

1) 字符串

字符串不支持 `remove` 方法。

2) 列表

```
my_list = [1, 2, 3, 2]
my_list.remove(2)
print(my_list)
>>> [1, 3, 2]
```

3) 元组

元组不支持 `remove` 方法。

4) 字典

```
my_dict = {'a': 1, 'b': 2}
del my_dict['b']
print(my_dict)
>>> {'a': 1}
```

5) 集合

```
my_set = {1, 2, 3, 2}
my_set.remove(2)  # 删除第一个匹配的2
print(my_set)  # 输出：{1, 3, 2}
```

集合还有一个`discard`方法，如果元素不存在，`remove`会报错，而`discard`不会。

```
my_set = {1, 2, 3, 4, 5}
my_set.discard(4)
print(my_set)
>>> {1, 2, 3, 5}
```

(15) pop 弹出

1) 字符串

字符串不支持`pop`方法。

2) 列表

```
my_list = [1, 2, 3, 2]
my_list.remove(2)  # 删除第一个匹配的2
print(my_list)  # 输出：[1, 3, 2]
my_list.pop(1)  # 删除索引为1的元素
print(my_list)  # 输出：[1, 2]
```

3) 元组

元组不支持`pop`方法。

4) 字典

```
my_dict = {'a': 1, 'b': 2}
my_dict.pop('a')
>>> 1
```

5) 集合

集合的`pop`方法是随机删除一个元素。

```
## pop方法:随机删除一个元素
set1 = {1, 2, 3, 4, 5, 6, 7}
set1.pop()
print(set1)
>>> {2, 3, 4, 5, 6, 7}
```

(16) clear 清空

1) 字符串

字符串是不可变的，因此没有 `clear` 方法。

2) 列表

```
## clear方法:清空列表
list1 = [1, 2, 3, 4, 5, 6, 7]
list1.clear()
print(list1)
>>> []
```

3) 元组

元组是不可变的，因此没有 `clear` 方法。

4) 字典

```
## clear方法:清空字典
dict1 = {'name': 'Jack', 'age': 20, 'gender': 'male'}
dict1.clear()
print(dict1)
>>> {}
```

5) 集合

```
my_set = {1, 2, 3, 4, 5}
my_set.clear()  # 清空集合
print(my_set)
>>> set()
```

(17) del 删除

实际开发中并不经常使用 `del` 方法，因为 Python 自带的垃圾回收机制会自动销毁无用的对象。

1) 字符串

字符串是不可变的，因此没有 `del` 方法。

2) 列表

```
intlist = [1, 45, 8, 34]
print(intlist)
del intlist[1]
print(intlist)
>>> [1, 8, 34]
del intlist
print(intlist)
>>> NameError: name 'intlist' is not defined
```

3) 元组

```
my_tuple = (1, 2, 3, 4, 5)
print(my_tuple)
>>> (1, 2, 3, 4, 5)
del my_tuple
print(my_tuple)
>>> NameError: name 'my_tuple' is not defined
```

4) 字典

```
my_dict = {'a': 1, 'b': 2, 'c': 3}
del my_dict['b']  # 删除键为 'b' 的键值对
print(my_dict)
>>> {'a': 1, 'c': 3}

del my_dict
print(my_dict)
>>> NameError: name 'my_dict' is not defined
```

5) 集合

```
my_set = {1, 2, 3, 4, 5}
del my_set
print(my_set)
>>> NameError: name 'my_set' is not defined
```

3) 序列间转换

(1) list 将序列转为列表

1) 字符串

```
my_string = "hello"
print(list(my_string))  # 输出: ['h', 'e', 'l', 'l', 'o']
>>> ['h', 'e', 'l', 'l', 'o']
```

2) 列表

列表不存在转换为列表的问题，所以转换后的结果还是原列表。

3) 元组

```
my_tuple = (1, 2, 3)
print(list(my_tuple))  # 输出: [1, 2, 3]
>>> [1, 2, 3]
```

4) 字典

```
my_dict = {'a': 1, 'b': 2, 'c': 3}
print(list(my_dict))  # 输出: ['a', 'b', 'c']
>>> ['a', 'b', 'c']
```

5) 集合

```
my_set = {1, 2, 3}
print(list(my_set))  # 输出: [1, 2, 3]
>>> [1, 2, 3]
```

(2) tuple 将序列转为元组

1) 字符串

```
my_string = "hello"
print(tuple(my_string))
>>> ('h', 'e', 'l', 'l', 'o')
```

2) 列表

```
my_list = [1, 2, 3]
print(tuple(my_list))
>>> (1, 2, 3)
```

3) 元组

元组不存在转换为元组的问题，所以转换后的结果还是原元组。

4) 字典

```
my_dict = {'a': 1, 'b': 2}
print(tuple(my_dict))
>>> ('a', 'b')
```

5) 集合

```
my_set = {1, 2, 3}
print(tuple(my_set))
>>> (1, 2, 3)
```

(3) set 将序列转为集合

1) 字符串

```
my_string = "hello"
print(set(my_string))
>>> {'h', 'e', 'l', 'o'}
```

2) 列表

```
my_list = [1, 2, 3]
print(set(my_list))
>>> {1, 2, 3}
```

3) 元组

```
my_tuple = (1, 2, 3)
print(set(my_tuple))
>>> {1, 2, 3}
```

4) 字典

```
my_dict = {'a': 1, 'b': 2}
print(set(my_dict))
>>> {'b', 'a'}
```

5) 集合

集合不存在转换为集合的问题，所以转换后的结果还是原集合。

二、 列表 (list)

1) 创建

(1) 直接生成

```
变量名 = [值1, 值2, 值3,...]
```

```
name = ['张三', '李四', '王五', 1, 2.3, True]
>>> ['张三', '李四', '王五', 1, 2.3, True]
```

(2) list 显式转换

`range` 函数用于生成一个数字序列，可以通过 `list` 函数将其转换为列表。

```
变量名 = list(range(起始值, 终止值, 步长))
```

```
>>> list(range(10))
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> list(range(1, 10))
[1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> list(range(1, 10, 2))
[1, 3, 5, 7, 9]
```

(3) 列表推导式

```
变量名 = [表达式 for 变量 in 可迭代对象 (if 条件)]
```

```
a = [1, 2, 3, 4, 5]
b = [i * 2 for i in a]
print(b)
>> [2, 4, 6, 8, 10]
```

```
# 生成从1-10所有偶数的平方
alist = [i**2 for i in range(1, 11) if i % 2 == 0]
print(alist)
>> [4, 16, 36, 64, 100]
```

```
# 过滤掉长度小于或等于3的字符串列表，并将剩下的转换成大写字母:
names = ['Bob','Tom','alice','Jerry','Wendy','Smith']
new_names = [name.upper() for name in names if len(name)>3]
print(new_names)
>>> ['ALICE', 'JERRY', 'WENDY', 'SMITH']
```

```
# 计算 30 以内可以被 3 整除的整数:
multiples = [i for i in range(30) if i % 3 == 0]
print(multiples)
>>> [0, 3, 6, 9, 12, 15, 18, 21, 24, 27]
```

2) 修改

直接对元素赋值即可

```
a = [1, 2, 3]
a[0] = 4
print(a)
>> [4, 2, 3]
```

3) 栈

使用 list 列表模拟栈功能的实现方法是，

1) 使用 `append()` 方法在末尾存入数据；
2) 使用 `pop()` 方法在末尾读取数据。

```
stack = []
stack.append(1)
stack.append("hello")
stack.append(2)
stack.append(3)
print(stack)
>> [1, 'hello', 2, 3]

print(stack.pop())
>> 3

print(stack.pop())
>> 2

print(stack.pop())
>> hello
```

4) collections 实现栈和队列

面使用 list 实现队列的例子中，插入数据的部分是通过 insert() 方法实现的，这种方法效率并不高，因为每次从列表的开头插入一个数据，列表中所有元素都得向后移动一个位置。

标准库的 `collections` 模块中的 `deque` 结构体，它被设计成在两端存入和读取都很快的特殊 list，可以用来实现栈和队列的功能。

三、 字符串

1) 定义

字符串是 Python 中最常用的数据类型。我们可以使用引号 (' 或") 来创建字符串。不能修改。

四、 集合

1) 定义

集合是一个 无序的、不重复的数据集合。

(1) 集合推导式

```
变量名 = {表达式 for 变量 in 可迭代对象 (if 条件)}
```

```
# 计算数字 1,2,3 的平方数
num_set = {i**2 for i in (1,2,3)}
num_set
>>> {1, 4, 9}
```

```
# 判断不是 abc 的字母并输出:
letters = 'abcdefghijklmnopqrstuvwxyz'
not_abc = {x for x in letters if x not in 'abc'}
print(not_abc)
>>> {'u', 'j', 'p', 'i', 'r', 'f', 'q', 'v', 'x', 'y', 'z', 'h',
    'l', 't', 'o', 'w', 'm', 'e', 'n', 'g', 'k', 's', 'd'}
```

2) 集合间运算

不同于其他容器类型 (如列表、元组等)，集合类型支持集合间的运算。

```
a = {1, 2, 3, 4, 5}
b = {4, 5, 6, 7, 8}

# 交集
## & 符号
print(a & b)
>>> {4, 5}
## intersection方法
print(a.intersection(b))
>>> {4, 5}

# 并集
## | 符号
print(a | b)
>>> {1, 2, 3, 4, 5, 6, 7, 8}
## union方法
print(a.union(b))
>>> {1, 2, 3, 4, 5, 6, 7, 8}

# 差集
## - 符号
print(a - b)
>>> {1, 2, 3}
print(b - a)
>>> {8, 6, 7}
## difference方法
print(a.difference(b))
>>> {1, 2, 3}
print(b.difference(a))
>>> {8, 6, 7}

# 对称差集
## ^ 符号
print(a ^ b)
>>> {1, 2, 3, 6, 7, 8}
## symmetric_difference方法
print(a.symmetric_difference(b))
>>> {1, 2, 3, 6, 7, 8}
```

五、 字典

字典是一种无序的数据集合，使用键-值（key-value）存储，具有极快的查找速度。

```
# 字典数据没有下标，通过键来查找

变量名 = {键1:值1，键2:值2，键3:值3,...}
```

1) 创建

(1) 直接生成

```
# 增加
dict1 = {'name': '张三', 'age': 18}

# 添加键值对
dict1['sex'] = '男'
print(dict1)
>>> {'name': '张三', 'age': 18, 'sex': '男'}

# 修改
dict1['age'] = 20
print(dict1)
>>> {'name': '张三', 'age': 20, 'sex': '男'}
```

(2) 字典推导式

```
变量名 = {键:值 for 变量 in 可迭代对象 (if 条件)}
```

```
a = [1, 2, 3, 4, 5]
b = {i: i * 2 for i in a}
print(b)
>> {1: 2, 2: 4, 3: 6, 4: 8, 5: 10}
```

```
# 生成从1-10所有偶数的平方
adict = {i: i**2 for i in range(1, 11) if i % 2 == 0}
print(adict)
```

```
>>> {2: 4, 4: 16, 6: 36, 8: 64, 10: 100}
```

```
names = ['Bob','Tom','alice','Jerry','Wendy','Smith']
# 将列表中各字符串值为键，各字符串的长度为值，组成键值对
newdict = {key:len(key) for key in names}
print(newdict)
>>>
{'Bob': 3, 'Tom': 3, 'alice': 5, 'Jerry': 5, 'Wendy': 5, 'Smith':
    5}
```

六、 元组 (tuple)

1) 定义

元组与列表类似，不同之处在于元组的元素 不能修改。

2) 创建

(1) 直接生成

```
# 直接创建元组
t2 = (1, 2, 3)
print(t2)
>>> (1, 2, 3)
```

(2) tuple 显式转换

```
# 通过 tuple 方法创建元组
t1 = tuple([1, 2, 3])
print(t1)
>>> (1, 2, 3)
```

(3) 元组推导式 (生成器表达式)

```
变量名 = (表达式 for 变量 in 可迭代对象 (if 条件))
```

```
1  a = (x for x in range(1,10))
2  print(a)
3  >>> <generator object <genexpr> at 0x7f16c56bf840>
```

不同于列表与字典，元组推到其的结果是生成器，可以使用`tuple()`方法将生成器转换为元组。

```
1  a = (x for x in range(1,10))
2  a = tuple(a)
3  print(a)
4  >>> (1, 2, 3, 4, 5, 6, 7, 8, 9)
```

第十章 (动作行为) 语法相关

一、 行为规范

1) 注释

Python 中单行注释以 # 开头

```
# 这是一个注释
print("hello world!")
```

多行注释使用 '''...'''

2) 行与缩进

python 最具特色的就是使用缩进来表示代码块, 缩进的空格数是可变的，但是**同一个代码块的语句必须包含相同的缩进空格数**。

```
if True:
    print ("True")
else:
    print ("False")
```

3) 多行语句

Python 通常是一行写完一条语句，但如果语句很长，我们可以使用反斜杠 \ 来实现多行语句

```
item_one = 1
item_two = 2
item_three = 3
total = item_one + \
        item_two + \
        item_three
print(total) # 输出为 6
```

二、 运算符

python 中常用的运算符有：

1) 算术运算符

1) `+` 加法
2) `-` 减法
3) `*` 乘法
4) `/` 除法
5) `//` 整除
6) `%` 取余
7) `**` 幂

```
x = 5
y = 2
print(x + y)
print(x - y)
print(x * y)
print(x / y)
print(x // y)
print(x % y)
print(x ** y)
```

```
7
3
10
2.5
2
1
25
```

其中`+`还可以拼接字符串，`*`可以重复字符串

```
x = 'abc'
y = 'def'
print(x + y)
print(x * 3)
```

```
abcdef
abcabcabc
```

2) 赋值运算符

1) `=`简单赋值
2) `+=`加法赋值
3) `-=`减法赋值
4) `*=`乘法赋值
5) `/=`除法赋值
6) `//=`整除赋值
7) `%=`取余赋值
8) `**=`幂赋值
9) `|=`或赋值
10) `&=`与赋值
11) `^=`异或赋值
12) `>>=`右移赋值
13) `<<=`左移赋值

```
x = 5
x += 2
print("x += 2:", x)
x -= 2
print("x -= 2:", x)
x *= 2
print("x *= 2:", x)
x /= 2
print("x /= 2:", x)
x //= 2
print("x //= 2:", x)
x **= 2
print("x **= 2:", x)
x %= 2
print("x %= 2:", x)

y = 16
y |= 2
print("y |= 2:", y)
y=16
y &= 4
print("y &= 2:", y)
y=16
y >>= 2
print("y >>= 2:", y)
y <<= 2
print("y <<= 2:", y)
y ^= 2
print("y ^= 2:", y)
```

```
x += 2: 7
x -= 2: 5
x *= 2: 10
x /= 2: 5.0
x //= 2: 2.0
x **= 2: 4.0
x %= 2: 0.0
y |= 2: 18
y &= 2: 0
y >>= 2: 4
y <<= 2: 16
y ^= 2: 18
```

3) 位运算符

1) `&` 与

2) `|` 或

3) `^` 异或

4) `~` 取反

5) `«` 左移

6) `»` 右移

```
x = 16
y = 2
print("x & y:", x & y)
print("x | y:", x | y)
print("x ^ y:", x ^ y)
print("~x:", ~x)
print("x << 2:", x << 2)
print("x >> 2:", x >> 2)
```

```
x & y: 0
x | y: 18
x ^ y: 18
~x: -17
x << 2: 64
x >> 2: 4
```

4) 比较运算符

1) `==` 两个变量的值是否相等

2) `!=` 不等于

3) `>` 大于

4) `<` 小于

5) `>=` 大于等于

6) `<=` 小于等于

7) `is` 判断两个变量所引用的对象是否相同

8) `is not` 判断两个变量所引用的对象是否不相同

```python
import time #引入 time 模块

t1 = time.gmtime() # gmtime()用来获取当前时间(秒)
t2 = time.gmtime()

print(t1 == t2) #输出 True
print(t1 is t2) #输出 False
```

5) 逻辑运算符

1) `and` 与

2) `or` 或

3) `not` 非

```python
x = 5
y = 2
print(x > y and x < y)
print(x > y or x < y)
print(not x > y)
```

```
False
True
False
```

三、 逻辑

1) 条件

```python
x = 'abc'
if x == 'abc':
    print('x is abc')
elif x == 'def':
```

```
    print('x is def')
else:
    print('x is not abc or def')
```

2) 三元运算符

```
表达式1 if 条件 else 表达式2
```

```
x = 'abc'
print('x is abc') if x == 'abc' else print('x is not abc')
```

```
x is abc
```

```
a = 10
b = 15
max = a if a>b else b
print("max=", max)
```

```
max= 15
```

3) 循环

4) while

```
i = 0
while i < 5:
    print(i)
    i += 1
```

5) for

Python 中的`for`和 C 语言中的`for`有所不同, Python 的`for`是用来遍历序列的。

```
for i in range(5):
    print(i)
```

类型	包含的运算符
算术运算符	**(指数) ~(按位翻转) +(一元加号) -(一元减号) *(乘) /(除) %(取模) //(取整除) +(加) -(减) «(左移) »(右移)
位运算符	&(按位与) |(按位或) ^(按位异或) ~(按位取反)
赋值运算符	=(基础赋值) +=(加后赋值) -=(减后赋值) *=(乘后赋值) /=(除后赋值) %=(取模后赋值) //=(取整除后赋值) **=(指数后赋值) «=(左移后赋值) »=(右移后赋值)
逻辑运算符	not(逻辑非) and(逻辑与) or(逻辑或)
关系运算符	<(小于) <=(小于等于) >(大于) >=(大于等于) ==(等于) != 或 <>(不等于)
身份运算符	is(对象同一性) is not(对象非同一性)
成员运算符	in(存在于序列) not in(不存在于序列)
其他运算符	sizeof(Python 3.11+ 新增) ?:(三元运算符) .(属性访问) [](列表/字典访问) ()(元组/函数调用) @(矩阵乘法) ,(元组拼接)

表 10.1 Python 运算符分类 (基于 Python 3.x 语法)

第十一章 异常

一、 异常

异常 ≠ 错误
异常是在程序运行过程中出现的错误，而错误是程序的语法错误。
异常处理的一般流程是：检测到错误，引发异常，捕获异常，处理异常。

```
# 第一种 try ... except
## 1.当try代码块中的代码报错时，
##  只有抛出与except关键字后异常类型相同的异常，才会执except代码块中的代码
## 2.如果except关键字后没有接异常类型，
##  那么任何异常都会触发except代码块中的代码逻辑
try:
    # 代码块
    pass
except Exception as e:
    # 异常处理
    pass

# 第二种 try ... except ... finally
try:
    # 代码块
    pass
except Exception as e:
    # 异常处理
    pass
finally:
    # 无论是否发生异常都会执行
    pass

# 第三种 try ... except ... else
try:
    # 代码块
    pass
except Exception as e:
    # 异常处理
    pass
else:
    # 无异常时执行
    pass

# 第四种 try ... except ... else ... finally
```

```
try:
    # 代码块
    pass
except Exception as e:
    # 异常处理
    pass
else:
    # 无异常时执行
    pass
finally:
    # 无论是否发生异常都会执行
```

二、 异常的种类

1) 文件

```
try:
    f = open('test.txt', 'r')
    print(f.read())
except FileNotFoundError as e:
    print('文件不存在')
finally:
    f.close()
```

2) 除零

```
try:
    print(5 / 0)
except ZeroDivisionError as e:
    print('除零错误')
```

3) 输入异常

```
year = int(input('请输入年份: '))

try:
    if year % 4 == 0 and year % 100 != 0 or year % 400 == 0:
        print('闰年')
    else:
        print('平年')
except ValueError as e:
    print('输入错误')
```

4) 捕获多个异常

```
try:
    print(5 / 0)
    year = int(input('请输入年份: '))
except ZeroDivisionError as e:
    print('除零错误')
except ValueError as e:
    print('输入错误')
```

```
try:
    print(5 / 0)
    year = int(input('请输入年份: '))
except (ZeroDivisionError, ValueError) as e: #
    用一个元组来捕获多个异常
    print('输入错误')
```

5) 捕获所有异常

```
try:
    print(5 / 0)
    year = int(input('请输入年份: '))
except Exception as e:  # Exception是所有异常的基类
    print('发生异常')
```

6) 自定义异常返回信息——raise

```
try:
    raise NameError('自定义异常')
except NameError as e:
    print("My custom Exception")
```

7) 自定义异常

三、 异常的处理

通过try ... except可以捕获异常，但如果捕捉到异常无法处理呢？此时最好的方式就是向上抛出异常。

异常处理的原则是处理可预见的异常，向上抛出当前无法处理的异常，最终的目的就是方便程序编写者可以很轻松地定位出现异常的位置，以及出现异常的原因。

```
# raise
raise [exceptionName('reason')]
## 中括号“[]”括起的部分是可选参数，
## exceptionName的作用就是抛出指定类型的异常，
## reason则是抛出异常时附带的说明信息。
```

```
def error_fun():
    try:
        name = input('Enter your name: ')
        if name.isspace():
            raise ValueError('Name cannot be empty')
        if name.isdigit():
            raise ValueError('Name cannot be a number')
```

```
            name = name + 123

        # 无法处理本层抛出的ValueError,会抛出该错误到上一层
        except ValueError as e:
            print(f'错误类型: {e}') #
        f的作用是将{}中的内容替换为变量的值
        else:
            print('Name:', name)

# error_fun()层的上一层是__main__, 所以抛出的错误会被__main__处理
try:
    error_fun()
except ValueError as e:
    print(f'主函数抛出异常: {e}, Please try again.')
    error_fun()
```

第十二章 函数与模块

一、 迭代器与生成器

1) 迭代器

```
# 迭代器
# 迭代器是一个可以记住遍历的位置的对象
# 迭代器对象从集合的第一个元素开始访问，直到所有的元素被访问完结束
# 迭代器只能往前不会后退
# 迭代器有两个基本的方法：iter() 和 next()

list = [1, 2, 3, 4]
it = iter(list) # 创建迭代器对象
print(next(it)) # 输出迭代器的下一个元素
print(next(it))
```

(1) 创建迭代器

把一个类作为一个迭代器使用需要在类中实现两个方法 `__iter__()` 与 `__next__()`。

```
1  class MyNumbers:
2    def __iter__(self):
3      self.a = 1
4      return self
5
6    def __next__(self):
7      x = self.a
8      self.a += 1
9      return x
10
11 myclass = MyNumbers()
12 myiter = iter(myclass)
13
14 print(next(myiter))
15 print(next(myiter))
```

```
print(next(myiter))
print(next(myiter))
print(next(myiter))
```

```
1
2
3
4
5
```

(2) StopIteration 异常

StopIteration 异常用于标识迭代的完成，防止出现无限循环的情况，在 `__next__()` 方法中我们可以设置在完成指定循环次数后触发 StopIteration 异常来结束迭代。

```
class MyNumbers:
  def __iter__(self):
    self.a = 1
    return self

  def __next__(self):
    if self.a <= 10:
      x = self.a
      self.a += 1
      return x
    else:
      raise StopIteration

myclass = MyNumbers()
myiter = iter(myclass)

for x in myiter:
  print(x)
```

```
1
2
3
4
5
6
7
```

```
8
9
10
```

2) 生成器

生成器是一个返回迭代器的函数，只能用于迭代操作，在调用生成器运行的过程中，每次遇到 yield 时函数会暂停并保存当前所有的运行信息，返回 yield 的值，并在下一次执行`next()`方法时从当前位置继续运行

```
def fibonacci(n): # 生成器函数 - 斐波那契
    a, b, counter = 0, 1, 0
    while True:
        if counter > n:
            return
        yield a
        a, b = b, a + b
        counter += 1

f = fibonacci(10) # f 是一个迭代器，由生成器返回生成

for i in f:
    print(i, end=' ')
```

二、 Lambda 表达式

在很多场景，不太需要函数名字和返回值

```
# Lambda表达式
# lambda [arg1 [,arg2,.....argn]]:expression

sum = lambda arg1, arg2: arg1 + arg2
print("相加后的值为：", sum(10, 20))
```

```
G = 9.8  # 重力加速度常数
retrofire = lambda t, rtf, C: -C*G if t >= rtf else 0.0

# 等价于
def retrofire(t, rtf, C):
    if t >= rtf:
        return -C * G
    else:
        return 0.0
# 测试
print(retrofire(10, 5, 2)) # 输出 -19.6=-2*9.8
print(retrofire(3, 5, 2))  # 输出 0.0
```

三、 内建函数

```
# 内建函数
# abs() 返回数字的绝对值
print(abs(-10))

# map() 会根据提供的函数对指定序列做映射
def square(x):
    return x ** 2
print(list(map(square, [1, 2, 3, 4, 5])))
>>> [1, 4, 9, 16, 25]
a = [1, 2, 3, 4, 5]
print(list(map(lambda x: x+1 ,a)))
>>> [2, 3, 4, 5, 6]
b = [6, 7, 8, 9, 10]
print(list(map(lambda x, y: x + y, a, b)))
>>> [7, 9, 11, 13, 15]

# reduce() 函数会对参数序列中元素进行累积
## 在python3里面，reduce()函数需要导入functools模块
from functools import reduce
def add(x, y):
    return x + y
print(reduce(add, [1, 2, 3, 4, 5]))
>>> 15

# filter() 函数用于过滤序列
## 在python3里面，filter()返回的是一个filter对象，需要用list()转换成列表
def is_odd(n):
    return n % 2 == 1
print(list(filter(is_odd, [1, 2, 3, 4, 5])))
>>> [1, 3, 5]


# sorted() 函数对所有可迭代的对象进行排序操作
print(sorted([1, 3, 2, 5, 4]))
>>> [1, 2, 3, 4, 5]
print(sorted([1, 3, 2, 5, 4], reverse=True))
>>> [5, 4, 3, 2, 1]

# zip() 函数用于将可迭代的对象作为参数，将对象中对应的元素打包成一个个元组
a = [1, 2, 3]
b = [4, 5, 6]
print(list(zip(a, b)))
>>> [(1, 4), (2, 5), (3, 6)]
## 字典值键互换
dict1 = {'a': 1, 'b': 2}
print(dict(zip(dict1.values(), dict1.keys())))
>>> {1: 'a', 2: 'b'}
```

四、 闭包

函数嵌套函数，就是闭包,就是返回值是函数的函数

```
# 不使用闭包
def addfunc():
    num1 = 2
    num2 = 3
    print(num1 + num2)

num = fun()

# 闭包
def addfunc1():
    num1 = 2
    def addfunc2(num2):
        print(num1 + num2)
    return addfunc2
    # addfunc2 表示一个函数的引用/函数名
    # addfunc2() 表示调用这个函数
num1 = addfunc1(3)  # 这里返回的是一个函数
num1(4)             # 这里调用的是返回的函数
>>> 6
```

```
# 计算 a *x +b =y 的函数
# 计算 a * x + b = y
def func(a, b):
    def com_func(x):
        return a *x + b
    return com_func

fun = func(1, 4)
print(fun(1))
print(fun(4))
print(fun(5))
```

五、 装饰器 (Decorator)

使用@符号，把一个函数作为参数传递给另一个函数
闭包传进来的是一个变量，装饰器传进来的是一个函数

```
# 装饰器
def decorator(func):
    def wrapper():
        print('调用函数')
        func()
    return wrapper

@decorator
def fun():
```

```
    print('hello world')

fun() # 相当于执行了 decorator(fun)()
```

```
# 使用装饰器计算两个数的和
def tips(func):
    def nei(a, b):
        print('start')
        func(a, b)
        print('end')
    return nei

@tips
def add(a, b):
    print(a + b)

add(1, 2) # 相当于执行了 tips(add)(1, 2)
```

六、 模块

模块是代码量变得相当大时，为了将需要重复使用的有组织的代码放在一起，方便维护和使用，这部分代码可以附加到其他代码中使用，附加的过程就是`import`。

```
# 模块
import 模块名              # 导入整个模块
from 模块名 import 函数名 #
    导入模块中的一个函数(不推荐，因为可能会覆盖掉原有的函数)
from 模块名 import *       # 导入模块中的所有函数
import 模块名 as 别名      # 导入模块并给模块取别名
```

```
# 自定义模块
## @file mymodule.py
def add(a, b):
    return a + b

def sub(a, b):
    return a - b

# 同一文件夹下的test.py
## 如果不在同一文件夹下，需要将mymodule.py放到sys.path中的一个路径下
## @file test.py
import mymodule
print(mymodule.add(1, 2))
print(mymodule.sub(1, 2))
```

第十三章 装饰器 (decorator)

一、 概述

装饰器（decorators）是 Python 中的一种高级功能，它允许你动态地修改函数或类的行为。

装饰器是一种函数，它接受一个函数作为参数，并返回一个新的函数或修改原来的函数。

```
#funA 作为装饰器函数
def funA(fn):
    #...
    fn() # 执行传入的 fn 参数
    #...
    return '...'

@funA
def funB():
    #...
```

等价于

```
def funA(fn):
    #...
    fn() # 执行传入的 fn 参数
    #...
    return '...'

def funB():
    #...

funB = funA(funB)
```

使用函数装饰器 A() 去装饰另一个函数 B()，其底层执行了如下 2 步操作：

1) 将 B 作为参数传给 A() 函数；

2) 将 A() 函数执行完成的返回值反馈回 B。

Python 的标准库（Standard Library）非常丰富，提供了大量内置模块，用于处理各种常见任务，无需额外安装第三方包。以下是 Python 标准库中一些常用模块及其主要用途的分类介绍（基于 Python 3.11+ 版本）：

1) 文本处理

(1) `string`:字符串操作工具,如模板、常量(如`string.ascii_letters`)。
(2) `re`: 正则表达式支持，用于模式匹配和文本提取。
(3) `difflib`: 比较序列（如字符串）的差异。
(4) `textwrap`: 格式化文本段落（如自动换行）。
(5) `unicodedata`: 访问 Unicode 字符数据库(如标准化、名称查询)。

2) 数据结构与算法

(1) `collections`: 扩展容器类型，如`deque`,`Counter`,`defaultdict`,`OrderedDict`。
(2) `heapq`: 堆队列算法（优先队列)。
(3) `bisect`: 维护有序列表的二分查找。
(4) `array`: 高效存储同类型基本数据（如整数、浮点数)。
(5) `queue`:线程安全的队列实现(如`Queue`,`LifoQueue`,`PriorityQueue`)。

3) 数值与数学

(1) `math`: 数学函数（如`sin`,`log`,`sqrt`)。
(2) `cmath`: 复数数学函数。
(3) `random`: 生成伪随机数。
(4) `statistics`: 统计函数（如均值、中位数、标准差)。
(5) `decimal`: 高精度十进制浮点运算。
(6) `fractions`: 有理数（分数）运算。

4) 日期与时间

(1) `datetime`:处理日期和时间(`date`,`time`,`datetime`,`timedelta`)。
(2) `time`: 底层时间函数（如`time()`,`sleep()`)。
(3) `calendar`: 日历相关功能。
(4) `zoneinfo`（Python 3.9+): 时区支持（基于 IANA 时区数据库)。

5) 文件与目录操作

(1) `os`: 操作系统接口（如环境变量、路径、进程)。
(2) `pathlib`(Python 3.4+):面向对象的路径操作(推荐替代`os.path`)。
(3) `shutil`: 高级文件操作（复制、移动、删除目录等)。
(4) `glob`: 文件名模式匹配（如`*.txt`)。
(5) `fnmatch`: Unix shell 风格的通配符匹配。
(6) `tempfile`: 创建临时文件和目录。
(7) `zipfile` / `tarfile`: 读写 ZIP/TAR 压缩文件。

6) 数据持久化

(1) `pickle`：Python 对象序列化/反序列化。
(2) `json`：JSON 编码与解码。
(3) `csv`：读写 CSV 文件。
(4) `sqlite3`：轻量级嵌入式 SQL 数据库接口。
(5) `dbm`：键值对数据库（类似字典的持久化存储）。

7) 数据压缩与归档

(1) `zlib`, `gzip`, `bz2`, `lzma`：不同压缩格式的支持。
(2) `tarfile`, `zipfile`：归档文件操作（见上文）。

8) 加密与哈希

(1) `hashlib`：安全哈希算法（如 SHA256, MD5）。
(2) `hmac`：基于密钥的消息认证。
(3) `secrets`（Python 3.6+）：生成安全随机数（用于密码、令牌等）。

9) 网络与互联网

(1) `socket`：底层网络通信。
(2) `http.client`, `http.server`：HTTP 客户端/服务器。
(3) `urllib`：URL 处理（`urlopen`, `parse`, `request` 等子模块）。
(4) `email`：解析和构建电子邮件消息。
(5) `smtplib` / `poplib` / `imaplib`：发送/接收邮件协议。
(6) `ftplib`：FTP 客户端。
(7) `xml.etree.ElementTree`：解析和创建 XML 文档。
(8) `html.parser`：简单的 HTML 解析器。

10) 并发与并行

(1) `threading`：线程支持。
(2) `multiprocessing`：多进程支持（绕过 GIL）。
(3) `asyncio`（Python 3.4+）：异步 I/O、事件循环、协程。
(4) `concurrent.futures`：高层并发接口（线程池/进程池）。

11) 开发工具

(1) `unittest`：单元测试框架。
(2) `doctest`：从文档字符串中提取测试用例。
(3) `pdb`：Python 调试器。
(4) `traceback`：打印或格式化异常栈信息。

(5) `logging`：灵活的日志记录系统。

(6) `argparse`：命令行参数解析（推荐替代 `optparse`）。

(7) `profile`, `cProfile`：性能分析工具。

12) 运行时服务

(1) `sys`：访问解释器相关的变量和函数（如 `argv`, `exit()`）。

(2) `builtins`：内置函数和异常（通常不需要显式导入）。

(3) `gc`：垃圾回收控制。

(4) `inspect`：检查活动对象（函数、类、帧等）。

(5) `importlib`：动态导入模块的工具。

13) 国际化

(1) `locale`：本地化设置（如数字、日期格式）。

(2) `gettext`：多语言支持（i18n）。

14) 其他实用模块

(1) `typing`：类型提示支持（如 `List[int]`, `Optional[str]`）。

(2) `enum`：枚举类型。

(3) `dataclasses`（Python 3.7+）：自动生成类的常用方法（如 `__init__`, `__repr__`）。

(4) `contextlib`：上下文管理器工具（如 `@contextmanager`）。

(5) `functools`：高阶函数和可调用对象操作（如 `lru_cache`, `partial`）。

(6) `itertools`：高效迭代器工具（如 `chain`, `product`, `combinations`）。

(7) `operator`：函数形式的标准操作符（如 `add`, `itemgetter`）。

第十四章 [标准库] math (标量) 数学库

一、 数学常数

```
import math
math.pi  # 圆周率   3.141592653589793
math.e   # 自然对数底数 e  2.718281828459045
```

二、 三角函数相关

1) 角度转换

```
import math
math.degrees(x)  # 弧度转角度 (180ř/Ex)
math.radians(x)  # 角度转弧度 (/180řEx)


# 例如:
math.radians(180)  # 180ř =
math.degrees(math.pi)  #  = 180ř
```

2) 基本三角函数

```
import math
math.sin(x)    # 正弦函数 (x为弧度)
math.cos(x)    # 余弦函数
math.tan(x)    # 正切函数
math.hypot(x,y) # 直角三角形斜边 (xš+yš)
```

3) 反三角函数

```
import math

math.asin(x)   # 反正弦 (返回弧度)
math.acos(x)   # 反余弦
math.atan(x)   # 反正切
```

三、 指数与对数相关

```
math.exp(x)    # e的x次方 (等价于 e^x)
math.log(x)    # 自然对数 ln(x)
math.log1p(x)  # ln(1+x) (更精确的x接近0时的计算)
math.log10(x)  # 常用对数 lg(x)
math.log2(x)   # 二进制对数 log(x)
```

四、 幂函数与开方

```
math.sqrt(x)   # 平方根 x
math.pow(x,y)  # x的y次方 (等价于x**y)
math.cbrt(x)   # 立方根 x
math.isqrt(x)  # 整数平方根 (返回整数部分)
```

五、 特殊功能函数

1) 取整相关

```
math.ceil(x)   # 向上取整 (返回x的最小整数)
math.floor(x)  # 向下取整 (返回x的最大整数)
math.trunc(x)  # 截断取整 (直接去除小数部分)
```

2) 数值处理

```
math.fabs(x)   # 绝对值 (返回浮点数)
math.factorial(x) # 阶乘 x!
math.fsum(iterable) # 高精度迭代求和
math.modf(x)   # 分解为(小数部分, 整数部分)元组
```

3) 特殊判断

```
math.isinf(x)  # 判断是否为无穷大
math.isnan(x)  # 判断是否为NaN（非数值）
```

第十五章 自定义包

一、 模块

在 python 中，一个扩展名为"`.py`" 的文件称为一个**模块** (module).

模块可认为是一盒 (箱) 主题积木，通过它可以拼出某一主题 (某个系统、某个复杂求解问题) 的东西。

与函数或类不同，一个函数或类相当于一块积木，而一个模块中可以包括很多函数或类.

二、 概述

Python 中，"Package" 就是一个目录，其中包括一组模块和一个"`__init__.py`"文件。其中 `__init__.py` 文件定义了包的属性和方法，通常只是一个空文件，但是必须存在，**切勿随意删除若删除** `__init__.py` 文件，则该 Package 仅仅是一个目录，无法被导入或包含其他模型库和嵌套包.

第三部分

系统编程

第十六章 多线程编程

一、 线程

Python3 中用到`threading`模块来实现多线程编程，线程是操作系统能够进行运算调度的最小单位，它被包含在进程之中，是进程中的实际运作单位。

```
Thread(group=None, target=None, name=None, args=(), kwargs={}, *,
    daemon=None)
'''
group: 线程组，目前还没有实现，库引用中应该写None
target: 要执行的方法
name: 线程名
args/kwargs: 方法的参数
daemon: 是否是守护线程
'''

start() # 启动线程
join(timeout=None) # 等待线程终止
```

```
1 # 不推荐的写法
2 import threading
3
4 def func(arg1, arg2):
5     for i in range(5):
6         print('threading: %s, %s %s' % (i, arg1+i, i+arg2))
7
8 t = threading.Thread(target=func, args=(1,2))
9 t.start()
```

```
1 # 推荐的写法
2 ##
      上面的方法虽然用起来简单，但是不推荐使用，因为这样写不利于维护，韩式推荐使用继承的方式来创建线程
```

```python
import threading
from threading import current_thread

def func(arg1, arg2):
    for i in range(5):
        print('threading: %s, %s %s' % (i, arg1+i, i+arg2))

class Mythread(threading.Thread):
    def __init__(self, func, args, tname=''):
        super(Mythread, self).__init__()
        self.func = func    # func 存放的是要执行的函数
        self.args = args    # args 存放的是函数的参数
        self.tname = tname  # tname 存放的线程的名字，线程的名字是可选的，
                            # 如果不传入线程名字，线程会自动分配一个名字

    ## 执行线程的具体逻辑
    ## 通过重写run方法的方式可以自定义线程具体的执行逻辑，
    ## 相比上面的方法， 这种方法更加灵活、直观。
    def run(self):
        print("Runing Threading: ",self.tname)
        print(current_thread().getName(), "start")
        print('run')
        self.func(*self.args)
        print(current_thread().getName(), "stop")

t1 = Mythread(func, (1, 2), tname='thread_name')
t1.start()
t1.join() # join方法会将主线程挂起，直到子线程运行结束再继续执行主线程中的逻辑

print(current_thread().getName(), "end")
```

二、 线程同步

线程同步是为了保证多个线程能够安全的访问共享的数据，Python 中使用`Lock`对象来实现线程同步。

```python
Lock()
'''
acquire(blocking=True, timeout=-1) # 获取锁
release() # 释放锁
'''
```

如果没有锁直接使用多线程，会出现数据混乱的情况。

程序中多个线程公用相同的资源是导致线程之间产生冲突的主要原因，为了避免这种冲突，要求线程在操作公用资源时要获取锁，只有拿到锁的线程才可以使用公用资源，没有获取锁的线程只能先等待。

```python
# 错误示例
# 因为没有加锁，所以会出现数据混乱的情况
import time
import threading

class MyThread(threading.Thread):
    def __init__(self, input, output):
        super(MyThread, self).__init__()
        # 调用父类的构造函数
        # super()函数是用于调用父类(超类)的一个方法
        # 参数 :
        #   MyThread - 类名，表示要调用的父类
        #   self - 对象，表示当前类的实例
        self.input = input
        self.output = output

    def run(self):
        for line in self.input.readlines():
            time.sleep(1) # 模拟耗时操作 1s
            self.output.write(line)
        print(f'写入: {line.strip()}')

def main():
    txt1 = open('./threading/1.txt', 'r') # 读取文件
    txt2 = open('./threading/2.txt', 'r') # 读取文件
    txt3 = open('./threading/3.txt', 'a') # 追加文件
    t1 = MyThread(txt1, txt3)
    t2 = MyThread(txt2, txt3)
    ```
    创建的两个线程会同时运行run方法，
    此时就会造成3.txt中的内容混乱，
    因为t1线程运行run方法插入了一行数据后，
    t2线程也在运行，将2.txt中的某行数据插入了 3.txt中。
    ```
    t1.start()
    t2.start()
    t1.join() # 主线程等待子线程结束
    t2.join() # 主线程等待子线程结束
    txt1.close()
    txt2.close()
    txt3.close()

if __name__ == '__main__':
    main()
```

```python
# 正确示例
import time
import threading

class MyThread(threading.Thread):
    def __init__(self, input, output, lock): # 增加一个lock参数
        super(MyThread, self).__init__()
        self.input = input
        self.output = output
        self.lock = lock            # 保存锁对象

    def run(self):
        print(self.getName(), '获取锁')
        self.lock.acquire()         # 获取锁,lock状态变为locked,
                                    # 并且阻塞其他线程获取lock对象
        for line in self.input.readlines():
            time.sleep(1)
            self.output.write(line)
        self.lock.release()         # 释放锁,lock状态变为unlocked,
                                    # 其他线程可以获取lock对象
        print(f'写入: {line.strip()}')
        print(self.getName(), '释放锁')

def main():
    lock = threading.Lock()         # 创建一个锁对象
    txt1 = open('./threading/1.txt', 'r') # 读取文件
    txt2 = open('./threading/2.txt', 'r') # 读取文件
    txt3 = open('./threading/3.txt', 'a') # 追加文件
    t1 = MyThread(txt1, txt3, lock)
    t2 = MyThread(txt2, txt3, lock)
    t1.start()
    t2.start()
    t1.join() # 主线程等待子线程结束
    t2.join() # 主线程等待子线程结束
    txt1.close()
    txt2.close()
    txt3.close()

if __name__ == '__main__':
    main()
```

```
$ Python ./threading_with_lock.py
Thread-1 获取锁
Thread-2 获取锁
写入: Hello 1.txt
Thread-1 释放锁
写入: Hello 2.txt
Thread-2 释放锁
```

三、 线程池

有些程序对效率有较高的要求，但自己创建且维护多个线程显得繁杂，此时就可以使用线程池。线程池至少会包含一个线程，但通常它都会由多个线程构成。线程池会对池子中的线程进行管理，用户只需要关心自己程序的逻辑即可

1) ThreadPoolExecutor

python 中`concurrent`标准, 提供了线程池功能

```
ThreadPoolExecutor(max_workers=None, thread_name_prefix='',
    initializer=None, initargs=())
'''
max_workers: 线程池中最多能同时运行的线程数
thread_name_prefix: 线程名的前缀
initializer: 初始化函数
initargs: 初始化函数的参数
'''
submit(fn, *args, **kwargs) # 提交任务
shutdown(wait=True) # 关闭线程池
```

```
1  import time
2  from concurrent.futures import ThreadPoolExecutor
3  # 导入了 concurrent.futures 中的 ThreadPoolExecutor类
4  # 通过实例化
       ThreadPoolExecutor类来创建线程池，该类的参数为线程池中拥有的线程数
5  #
       通过submit方法将要执行的方法与该方法对应的参数作为参数提交到线程池中，
6  # 线程池会自动安排闲置的线程去执行相应的任务。
7  import threading
8
9  def worker(input_file, output_file, lock, thread_name):
10     print(thread_name, '获取锁')
11     with lock:  # 使用上下文管理器来自动获取和释放锁
12         for line in input_file.readlines():
13             time.sleep(1)
```

```
            output_file.write(line)
            print(f'写入: {line.strip()}')
    print(thread_name, '释放锁')

def main():
    lock = threading.Lock()
    txt1 = open('./threading/1.txt', 'r')
    txt2 = open('./threading/2.txt', 'r')
    txt3 = open('./threading/3.txt', 'a')

    with ThreadPoolExecutor(max_workers=8) as executor:
        futures = []
        # 使用submit方法提交任务
        futures.append(executor.submit(worker, txt1, txt3, lock,
    "线程1"))
        futures.append(executor.submit(worker, txt2, txt3, lock,
    "线程2"))

        # 等待所有线程完成
        for future in futures:
            future.result()

    txt1.close()
    txt2.close()
    txt3.close()

if __name__ == '__main__':
    main()
```

2) 返回值

```
# 线程执行的方法有相应的返回值
as_completed(fs, timeout=None)
'''
fs: 一个Future对象的迭代器
timeout: 超时时间
'''
```

```
import time
from concurrent.futures import ThreadPoolExecutor, as_completed

executor = ThreadPoolExecutor(max_workers=8)

```

```python
def longtime(s):
    print(f"\033[32m线程 {s} 开始\033[0m")
    time.sleep(2)
    print(f'\033[92m线程 {s} 结束\033[0m')
    return s+2

all_task = []
for i in range(5):
    # 提交任务
    ## 通过submit方法将 long_time 函数提交给线程池
    ## long_time 函数的参数是 i
    ## long_time 函数的返回值是 i+2
    task = executor.submit(longtime, i)
    # 保存任务到列表
    all_task.append(task)
    print(f"\033[35m任务 {i} 提交\033[0m")

for future in as_completed(all_task):
    data = future.result()
    print(f"\033[95m任务 {i} 完成，返回值为{data} \033[0m")
```

```
$ Python ./threading_pool_return.py
线程 0 开始
任务 0 提交
线程 1 开始
任务 1 提交
线程 2 开始
任务 2 提交
线程 3 开始
任务 3 提交
线程 4 开始
任务 4 提交
线程 4 结束
线程 2 结束
线程 0 结束
任务 6 完成，返回值为6
线程 3 结束
任务 4 完成，返回值为4
线程 1 结束
任务 2 完成，返回值为2
任务 5 完成，返回值为5
任务 3 完成，返回值为3
```

四、 Python 中的多线程

Python 中的线程与其他主流语言（如 C++、Java 等）提供的线程有较大差别，主要表现在 Python 的线程无法利用多核 CPU，对程序速度的提升非常

有限。

之所以有这样的问题，因为 Python 线程面临着解释器级别的锁：GIL（Global Interpreter Lock, 全局解释器锁），每个线程要运行都需要获得 GIL 锁，因为 GIL 锁的存在，Python 多线程其实依旧以类似单线程的方式在运行。

因此，Python 中的多线程适合 处理 I/O 密集型任务，但不适合处理 CPU 密集型任务。

第十七章 多进程编程

一、 进程

Python3 中用到`multiprocessing`模块来实现多进程编程，进程是操作系统能够进行运算调度的最小单位，是程序的一次执行过程，是一个动态的过程：有它自己的地址空间，有它自己的内存，有它自己的数据，有它自己的打开的文件等等。进程的切换会消耗大量的资源，所以进程间的通信相对复杂。

```
Process([group [, target [, name [, args [, kwargs]]]]])
'''
group: 线程组，目前还没有实现，库引用中应该写None
target: 要执行的方法
name: 进程名
args/kwargs: 方法的参数
'''
start() # 启动进程
join([timeout]) # 等待进程终止
```

```
1 # 不推荐的写法
2 import multiprocessing
3
4 def func(arg1, arg2):
5     for i in range(5):
6         print('multiprocessing: %s, %s %s' % (i, arg1+i, i+arg2))
7
8 p = multiprocessing.Process(target=func, args=(1,2))
9 p.start()
```

一个好的方式是继承`multiprocessing.Process`类，然后重写`run`方法，这样可以更加灵活、直观。

```python
# 推荐的写法
##
    上面的方法虽然用起来简单，但是不推荐使用，因为这样写不利于维护，韩式推荐使用继承的
import multiprocessing

def func(arg1, arg2):
    for i in range(5):
        print('multiprocessing: %s, %s %s' % (i, arg1+i, i+arg2))

class Myprocess(multiprocessing.Process):
    def __init__(self, func, args, pname=''):
        super(Myprocess, self).__init__()
        self.func = func    # func 存放的是要执行的函数
        self.args = args    # args 存放的函数的参数
        self.pname = pname  # pname
    存放的进程的名字，进程的名字是可选的，
                            #
    如果不传入进程名字，进程会自动分配一个名字

    ## 执行进程的具体逻辑
    ## 通过重写run方法的方式可以自定义进程具体的执行逻辑，
    ## 相比上面的方法， 这种方法更加灵活、直观。
    def run(self):
        print("Runing Process: ",self.pname)
        print('run')
        self.func(*self.args)
        print(current_process().name, "end")

if __name__ == '__main__':
    p = Myprocess(func, (1, 2), 'myprocess')
    p.start()
    p.join()
```

```
$ python  Myprocess.py
Runing Process:  myprocess
Myprocess-1 start
run
multiprocessing: 0, 1 2
multiprocessing: 1, 2 3
multiprocessing: 2, 3 4
multiprocessing: 3, 4 5
multiprocessing: 4, 5 6
Myprocess-1 stop
```

二、 进程间通信

进程与线程的区别

与线程不同，线程如同车间里的工人，工人之间可以直接使用车间里的任意资源；

但进程表示车间本身，车间与车间是相互独立的，各自有自身的空间让自己的线程完成具体的子任务，车间之间的信息与资源是不互通的，如何在不同进程中传递信息是进程在使用过程中需要解决的问题。

1) Queue

```python
# 利用Queue实现生产者消费者模型
# 利用Queue实现生产者消费者模型
import os
import time
from multiprocessing import Process, Queue

def producer(q):
    for i in range(5):
        print(f'\033[32m生产者: {os.getpid()}生产了商品{i}\033[0m')
        # 将商品放入队列
        q.put(i)
        time.sleep(1)

def consumer(q):
    while True:
        try:
            # 从队列中获取商品
            i = q.get(timeout=5)
            print(f'\033[95m消费者: {os.getpid()}消费了商品{i}\033[0m')
        except:
            break

if __name__ == '__main__':
    q = Queue()
    p = Process(target=producer, args=(q,))
    c = Process(target=consumer, args=(q,))
    p.start() # 生产者进程开始生产商品
    c.start() # 消费者进程开始消费商品
    p.join()  # 主进程挂起，等待子进程producer
    c.join()  # 主进程挂起，等待子进程consumer
    print('主进程结束')
```

```
$ python queue.py
生产者: 19896 生产了商品0
消费者: 5856 消费了商品0
生产者: 19896 生产了商品1
消费者: 5856 消费了商品1
生产者: 19896 生产了商品2
消费者: 5856 消费了商品2
生产者: 19896 生产了商品3
消费者: 5856 消费了商品3
生产者: 19896 生产了商品4
消费者: 5856 消费了商品4
主进程结束
```

三、 进程池

进程池是一种能够提供多个进程的服务，进程池中的进程可以执行任务，当任务执行完毕后，进程会自动回到进程池中，等待下一个任务。

```
Pool([processes [, initializer [, initargs [, maxtasksperchild [,
    context]]]]])
'''
processes: 进程池中的进程数
initializer: 初始化函数
initargs: 初始化函数的参数
maxtasksperchild: 每个子进程最多执行的任务数
context: 上下文
'''
apply(func[, args[, kwds]]) # 同步执行
apply_async(func[, args[, kwds]]) # 异步执行
map(func, iterable[, chunksize]) # 同步执行
map_async(func, iterable[, chunksize]) # 异步执行
```

1) 同步执行

```
1 # 进程同步
2 # 演示如何使用 multiprocessing.Pool 来创建进程池，
3 # 以及如何使用 apply 和 map 方法来执行函数
4
5 from multiprocessing import Pool
6 import os
7 import time
```

```python

def square(x):
    print(f'Process {os.getpid()} calculating square of {x}')
    time.sleep(1)  # 模拟耗时操作
    return x * x

def worker(num):
    print(f'Process {os.getpid()} doubling {num}')
    time.sleep(1)  # 模拟耗时操作
    return num * 2

if __name__ == "__main__":
    with Pool(processes=4) as pool: # 创建进程池
        # 使用 apply 同步执行
        print("Starting apply()")
        ## apply方法会阻塞主进程，直到子进程执行完毕并返回结果
        result_apply = pool.apply(square, (10,))
        print(f'Result from apply: {result_apply}')

        # 使用 map 同步执行
        print("Starting map()")
        numbers = [1, 2, 3, 4, 5]
        result_map = pool.map(worker, numbers)
        print(f'Results from map: {result_map}')
```

```
$ python pool.py
Starting apply()
Process 17668 calculating square of 10
Result from apply: 100
Starting map()
Process 4084 doubling 1
Process 32672 doubling 2
Process 12144 doubling 3
Process 17668 doubling 4
Process 12144 doubling 5
Results from map: [2, 4, 6, 8, 10]
```

2) 异步执行

```python
# 进程异步
# 演示如何使用 multiprocessing.Pool 来创建进程池，
# 以及如何使用 apply_async 和 map_async 方法来异步执行函数

from multiprocessing import Pool
```

```python
import os
import time

def square(x):
    print(f'Process {os.getpid()} calculating square of {x}')
    time.sleep(1)  # 模拟耗时操作
    return x * x

def worker(num):
    print(f'Process {os.getpid()} doubling {num}')
    time.sleep(1)  # 模拟耗时操作
    return num * 2

if __name__ == "__main__":
    with Pool(processes=4) as pool:
        # 使用 apply_async 异步执行
        print("Starting apply_async()")
        result_apply_async = pool.apply_async(square, (10,))

        # 在等待结果的同时，可以做其他事情
        print("Doing some other work while waiting for apply_async result")
        time.sleep(2)  # 模拟其他工作耗时
        print(f'Result from apply_async: {result_apply_async.get()}')  # 获取结果

        # 使用 map_async 异步执行
        print("Starting map_async()")
        numbers = [1, 2, 3, 4, 5]
        result_map_async = pool.map_async(worker, numbers)

        # 在等待结果的同时，可以做其他事情
        print("Doing some other work while waiting for map_async results")
        time.sleep(2)  # 模拟其他工作耗时
        print(f'Results from map_async: {result_map_async.get()}')  # 获取结果
```

```
$ python pool_async.py
Starting apply_async()
Doing some other work while waiting for apply_async result
Process 35528 calculating square of 10
Result from apply_async: 100
Starting map_async()
Doing some other work while waiting for map_async results
Process 6792 doubling 1
Process 36048 doubling 2
Process 36592 doubling 3
```

```
Process 35528 doubling 4
Process 35528 doubling 5
Results from map_async: [2, 4, 6, 8, 10]
```

第十八章 系统信息

一、 获取 CPU 信息

```
>>> import psutil
>>> psutil.cpu_count() # CPU逻辑数量
20
>>> psutil.cpu_count(logical=False) # CPU物理核心
12
# 12说明是双核超线程，20则是20核非超线程
```

二、 获取内存信息

```
>>> import psutil
>>> psutil.virtual_memory() # 获取内存信息
svmem(total=33997406208, available=18719956992, percent=44.9,
    used=15277449216, free=18719956992)
# total: 总内存，available: 可用内存，percent: 使用率，used: 已用内存，
    free: 空闲内存
>>> psutil.swap_memory() # 获取交换分区信息
sswap(total=2147483648, used=64000000, free=2083483648, percent=3.0,
    sin=0, sout=0)
# total: 总交换分区，used: 已用交换分区，free: 空闲交换分区，percent:
    使用率，sin: 交换分区输入，sout: 交换分区输出
```

三、 获取磁盘信息

```
>>> import psutil
>>> psutil.disk_partitions() # 磁盘分区信息
[sdiskpart(device='C:\\', mountpoint='C:\\', fstype='NTFS',
    opts='rw,fixed'), sdiskpart(device='D:\\', mountpoint='D:\\',
    fstype='NTFS', opts='rw,fixed'), sdiskpart(device='G:\\',
    mountpoint='G:\\', fstype='FAT32', opts='rw,fixed')]
# device: 设备名，mountpoint: 挂载点，fstype: 文件系统类型，opts: 挂载选项

>>> psutil.disk_usage('/') # 磁盘使用情况
```

```
sdiskusage(total=511243677696, used=457558290432, free=53685387264,
    percent=89.5)
# total: 总容量, used: 已用容量, free: 空闲容量, percent: 使用率

>>> psutil.disk_io_counters() # 磁盘IO
sdiskio(read_count=7223356, write_count=3159065, read_bytes=272073475072,
    write_bytes=253278097920, read_time=8198, write_time=4701)
# read_count: 读次数, write_count: 写次数, read_bytes: 读字节数,
    write_bytes: 写字节数, read_time: 读时间, write_time: 写时间
```

四、 获取网络信息

```
>>> import psutil
>>> psutil.net_io_counters() # 获取网络读写字节 / 包的个数
snetio(bytes_sent=1097595849, bytes_recv=185204061, packets_sent=1197785,
    packets_recv=412550, errin=0, errout=0, dropin=0, dropout=0)
# bytes_sent: 发送字节数, bytes_recv: 接收字节数, packets_sent: 发送包数,
    packets_recv: 接收包数, errin: 错误接收数, errout: 错误发送数, dropin:
    丢弃接收数, dropout: 丢弃发送数

>>> psutil.net_if_addrs() # 获取网络接口信息
{'以太网':
    [snicaddr(family=<AddressFamily.AF_LINK: -1>,
    address='E0-BE-03-96-F1-A1',
    netmask=None,
    broadcast=None,
    ptp=None),

    snicaddr(family=<AddressFamily.AF_INET: 2>,
    address='192.168.10.101',
    netmask='255.255.255.0',
    broadcast=None, ptp=None),
    snicaddr(family=<AddressFamily.AF_INET6: 23>,
    address='2409:8a18:601:5334::49e',
    netmask=None,
    broadcast=None,
    ptp=None),

    snicaddr(family=<AddressFamily.AF_INET6: 23>,
    address='2409:8a18:601:5334:c401:e201:52de:444d',
    netmask=None,
    broadcast=None,
    ptp=None),

    snicaddr(family=<AddressFamily.AF_INET6: 23>,
    address='fdd5:5fdd:d72f::49e',
    netmask=None,
    broadcast=None,
    ptp=None),

    snicaddr(family=<AddressFamily.AF_INET6: 23>,
```

```
    address='fe80::3d12:1c86:bef:a4b',
    netmask=None,
    broadcast=None,
    ptp=None)],

'VMware Network Adapter VMnet1':
    [snicaddr(family=<AddressFamily.AF_LINK: -1>,
    address='00-50-56-C0-00-01',
    netmask=None,
    broadcast=None,
    ptp=None),
    snicaddr(family=<AddressFamily.AF_INET: 2>,
    address='192.168.159.1',
    netmask='255.255.255.0',
    broadcast=None,
    ptp=None),
    snicaddr(family=<AddressFamily.AF_INET6: 23>,
    address='fe80::c89b:fef2:cb75:2d15',
    netmask=None,
    broadcast=None,
    ptp=None)],

'VMware Network Adapter VMnet8':
    [snicaddr(family=<AddressFamily.AF_LINK: -1>,
    address='00-50-56-C0-00-08',
    netmask=None,
    broadcast=None,
    ptp=None),
    snicaddr(family=<AddressFamily.AF_INET: 2>,
    address='192.168.226.1',
    netmask='255.255.255.0',
    broadcast=None,
    ptp=None),
    snicaddr(family=<AddressFamily.AF_INET6: 23>,
    address='fe80::d4c9:92d1:1156:ccd3', netmask=None, broadcast=None,
    ptp=None)],

'Loopback Pseudo-Interface 1':
    [snicaddr(family=<AddressFamily.AF_INET: 2>, address='127.0.0.1',
    netmask='255.0.0.0', broadcast=None, ptp=None),
    snicaddr(family=<AddressFamily.AF_INET6: 23>, address='::1',
    netmask=None, broadcast=None, ptp=None)]
}

>>> psutil.net_if_stats() # 获取网络接口状态
{
    '以太网': snicstats(isup=True, duplex=<NicDuplex.NIC_DUPLEX_FULL: 2>,
    speed=1000, mtu=1500, flags=''),
    'VMware Network Adapter VMnet1': snicstats(isup=True,
    duplex=<NicDuplex.NIC_DUPLEX_FULL: 2>, speed=100, mtu=1500, flags=''),
    'VMware Network Adapter VMnet8': snicstats(isup=True,
    duplex=<NicDuplex.NIC_DUPLEX_FULL: 2>, speed=100, mtu=1500, flags=''),
    'Loopback Pseudo-Interface 1': snicstats(isup=True,
    duplex=<NicDuplex.NIC_DUPLEX_FULL: 2>, speed=1073, mtu=1500, flags='')
}
```

1) 网络连接信息

```
>>> import psutil
>>> psutil.net_connections()
[
    sconn(fd=83, family=<AddressFamily.AF_INET6: 30>, type=1,
    laddr=addr(ip='::127.0.0.1', port=62911),
    raddr=addr(ip='::127.0.0.1', port=3306), status='ESTABLISHED',
    pid=3725),
    sconn(fd=84, family=<AddressFamily.AF_INET6: 30>, type=1,
    laddr=addr(ip='::127.0.0.1', port=62905),
    raddr=addr(ip='::127.0.0.1', port=3306), status='ESTABLISHED',
    pid=3725),
    sconn(fd=93, family=<AddressFamily.AF_INET6: 30>, type=1,
    laddr=addr(ip='::', port=8080), raddr=(), status='LISTEN', pid=3725),
    sconn(fd=103, family=<AddressFamily.AF_INET6: 30>, type=1,
    laddr=addr(ip='::127.0.0.1', port=62918),
    raddr=addr(ip='::127.0.0.1', port=3306), status='ESTABLISHED',
    pid=3725),
    sconn(fd=105, family=<AddressFamily.AF_INET6: 30>, type=1, ...,
    pid=3725),
    sconn(fd=106, family=<AddressFamily.AF_INET6: 30>, type=1, ...,
    pid=3725),
    sconn(fd=107, family=<AddressFamily.AF_INET6: 30>, type=1, ...,
    pid=3725),
    ...
    sconn(fd=27, family=<AddressFamily.AF_INET: 2>, type=2, ..., pid=1)
]
```

第十九章 进程管理

一、 创建进程

```
import subprocess

subprocess.Popen('calc.exe')    # 打开计算器
subprocess.Popen('notepad.exe') # 打开记事本
subprocess.Popen('cmd.exe')     # 打开命令行
subprocess.Popen('resmon.exe')  # 打开资源管理器
subprocess.Popen('taskmgr.exe') # 打开任务管理器
subprocess.Popen('D:\\application\\wps\\WPS Office\\ksolaunch.exe') #
    打开WPS
subprocess.Popen('D:/application/wps/WPS Office/ksolaunch.exe') # 打开WPS
```

二、 获取进程

1) 显示所有进程及对应的名称

```
import psutil

# 获取所有进程的 PID
pids = psutil.pids()

# 遍历每个 PID，获取进程名称并打印
for pid in pids:
    try:
        # 实例化一个 Process 对象
        process = psutil.Process(pid)
        # 获取并打印进程名称
        print(f"PID: {pid}, Name: {process.name()}")
    except (psutil.NoSuchProcess, psutil.AccessDenied,
    psutil.ZombieProcess):
        # 忽略访问被拒绝、进程不存在或僵尸进程的情况
        pass
```

```
PID: 0, Name: System Idle Process
PID: 4, Name: System
```

```
...
PID: 25836, Name: ChsIME.exe
PID: 25932, Name: chrome.exe
PID: 26128, Name: Code.exe
PID: 26236, Name: wsl.exe
PID: 26436, Name: conhost.exe
PID: 26608, Name: wps.exe
PID: 26612, Name: wslhost.exe
```

2) 根据名称获取进程 ID

```
import psutil

def get_pids_by_name(process_name):
    pids = []
    for proc in psutil.process_iter(['pid', 'name']):
        try:
            if proc.info['name'] == process_name:
                pids.append(proc.info['pid'])
        except (psutil.NoSuchProcess, psutil.AccessDenied,
    psutil.ZombieProcess):
            pass  # 忽略访问被拒绝或进程不存在的情况
    return pids

process_name_to_find = "python"  # 替换为你想要查找的进程名称
pids = get_pids_by_name(process_name_to_find)

print(f"进程名称 '{process_name_to_find}' 的进程 ID 列表: {pids}")
```

```
进程名称 'Code.exe' 的进程 ID 列表: [2108, 2692, 4676, 9472, 11052, 11964,
    12184, 14440, 14872, 15052, 15584, 16896, 17392, 17784, 18924, 19040,
    19132, 20708, 22544, 23080, 23868, 26128]
```

```
import psutil

>>> psutil.pids() # 所有进程ID
[3865, 3864, 3863, 3856, 3855, 3853, 3776, ..., 45, 44, 1, 0]
>>> p = psutil.Process(3776) #
    获取指定进程ID=3776, 其实就是当前Python交互环境
>>> p.name() # 进程名称
'python3.6'
>>> p.exe() # 进程exe路径
'/Users/michael/anaconda3/bin/python3.6'
>>> p.cwd() # 进程工作目录
'/Users/michael'
>>> p.cmdline() # 进程启动的命令行
['python3']
>>> p.ppid() # 父进程ID
3765
>>> p.parent() # 父进程
```

```
<psutil.Process(pid=3765, name='bash') at 4503144040>
>>> p.children() # 子进程列表
[]
>>> p.status() # 进程状态
'running'
>>> p.username() # 进程用户名
'michael'
>>> p.create_time() # 进程创建时间
1511052731.120333
>>> p.terminal() # 进程终端
'/dev/ttys002'
>>> p.cpu_times() # 进程使用的CPU时间
pcputimes(user=0.081150144, system=0.053269812, children_user=0.0,
    children_system=0.0)
>>> p.memory_info() # 进程使用的内存
pmem(rss=8310784, vms=2481725440, pfaults=3207, pageins=18)
>>> p.open_files() # 进程打开的文件
[]
>>> p.connections() # 进程相关网络连接
[]
>>> p.num_threads() # 进程的线程数量
1
>>> p.threads() # 所有线程信息
[pthread(id=1, user_time=0.090318, system_time=0.062736)]
>>> p.environ() # 进程环境变量
{'SHELL': '/bin/bash', 'PATH':
    '/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin:...', 'PWD':
    '/Users/michael', 'LANG': 'zh_CN.UTF-8', ...}
>>> p.terminate() # 结束进程
Terminated: 15 <-- 自己把自己结束了
```

三、 结束进程

```
import psutil

# 获取指定进程ID的进程对象
p = psutil.Process(pid)

# 结束进程
p.terminate()
```

第二十章 pywin23 概述

一、 概述

`pywin32`是一个针对 Windows 平台对 Python 做的扩展包装了 Windows 系统的 Win32 API，能创建和使用 COM 对象和图形窗口界面

二、 安装

```
pip install pywin32
```

第二十一章 进程管理

一、 创建进程

```
import subprocess

subprocess.Popen('calc.exe')    # 打开计算器
subprocess.Popen('notepad.exe') # 打开记事本
subprocess.Popen('cmd.exe')     # 打开命令行
subprocess.Popen('resmon.exe')  # 打开资源管理器
subprocess.Popen('taskmgr.exe') # 打开任务管理器
subprocess.Popen('D:\\application\\wps\\WPS Office\\ksolaunch.exe') #
    打开WPS
subprocess.Popen('D:/application/wps/WPS Office/ksolaunch.exe') # 打开WPS
```

二、 获取进程

1) 显示所有进程及对应的名称

```
import psutil

# 获取所有进程的 PID
pids = psutil.pids()

# 遍历每个 PID，获取进程名称并打印
for pid in pids:
    try:
        # 实例化一个 Process 对象
        process = psutil.Process(pid)
        # 获取并打印进程名称
        print(f"PID: {pid}, Name: {process.name()}")
    except (psutil.NoSuchProcess, psutil.AccessDenied,
    psutil.ZombieProcess):
        # 忽略访问被拒绝、进程不存在或僵尸进程的情况
        pass
```

```
PID: 0, Name: System Idle Process
PID: 4, Name: System
```

```
...
PID: 25836, Name: ChsIME.exe
PID: 25932, Name: chrome.exe
PID: 26128, Name: Code.exe
PID: 26236, Name: wsl.exe
PID: 26436, Name: conhost.exe
PID: 26608, Name: wps.exe
PID: 26612, Name: wslhost.exe
```

2) 根据名称获取进程 ID

```python
import psutil

def get_pids_by_name(process_name):
    pids = []
    for proc in psutil.process_iter(['pid', 'name']):
        try:
            if proc.info['name'] == process_name:
                pids.append(proc.info['pid'])
        except (psutil.NoSuchProcess, psutil.AccessDenied,
    psutil.ZombieProcess):
            pass  # 忽略访问被拒绝或进程不存在的情况
    return pids

process_name_to_find = "python"  # 替换为你想要查找的进程名称
pids = get_pids_by_name(process_name_to_find)

print(f"进程名称 '{process_name_to_find}' 的进程 ID 列表: {pids}")
```

```python
import psutil

# 获取所有进程
psutil.pids()
# 获取指定进程
p = psutil.Process(1234) # 1234为进程PID
```

第二十二章 GUI 操作

一、 获取窗口

二、 窗口截图

三、 窗口操作

第四部分

存储

第二十三章 输入输出

一、 input 输入

```
name = input('请输入你的名字: ')
print('你好', name)
```

二、 print 输出

`print()` 函数的详细语法格式如下

```
print (value,...,sep='',end='\n',file=sys.stdout,flush=False)
```

从上面的语法格式可以看出, value 参数可以接受任意多个变量或值, 因此 `print()` 函数完全可以输出多个值。例如如下代码：

```
user_name = 'Charlie'
user_age = 8
#同时输出多个变量和字符串
print("读者名: ",user_name,"年龄: ",user_age)
```

1) 分隔符

```
>>> print("读者名: " ,user_name,"年龄: ",user_age,sep='|')
读者名: |Charlie|年龄: |8
```

2) 结尾符

```
print('你好', '世界')
print('你好', '世界', sep='')
print('你好', '世界', sep='', end='')
print('你好', '世界', sep='', end='\n')
print('你好', '世界', sep='', end='\t')
print('你好', '世界', sep='', end='\r')
print('你好', '世界', sep='', end='\b')
```

```
你好 世界
你好世界
你好世界
你好世界
你好世界
你好世界
```

3) 输出位置

`print()`函数默认输出到屏幕。实际上，完全可以通过改变该参数让`print()`函数输出到特定文件中，

```
with open('test.txt', 'w') as f:
    print('Hello, world!', file=f)
```

4) 格式化输出

转换说明符（Conversion Specifier）只是一个占位符，它会被后面表达式（变量、常量、数字、字符串、加减乘除等各种形式）的值代替。

```
# 输出整数
print('%d' % 123)
# 输出浮点数
print('%f' % 3.1415926)
# 输出字符串
print('%s' % 'hello')
# 输出字符
print('%c' % 'a')
# 输出十六进制
print('%x' % 255)
```

转换说明符	解释
%d、%i	转换为带符号的十进制整数
%o	转换为带符号的八进制整数
%x、%X	转换为带符号的十六进制整数
%e	转化为科学计数法表示的浮点数（e 小写）
%E	转化为科学计数法表示的浮点数（E 大写）
%f、%F	转化为十进制浮点数
%g	智能选择使用%f 或%e 格式
%G	智能选择使用%F 或%E 格式
%c	格式化字符及其 ASCII 码
%r	使用 repr() 函数将表达式转换为字符串
%s	使用 str() 函数将表达式转换为字符串

表 23.1 转换说明符

```
# 输出八进制
print('%o' % 255)
```

```
123
3.141593
hello
a
ff
377
```

三、 文件的内建函数和方法

```
# 文件的打开 —— open()
'''
 文件的打开模式:
 open(file, mode='r', buffering=-1, encoding=None, errors=None,
    newline=None, closefd=True, opener=None)

 其中
  mode
    r 以只读方式打开文件
    w 以写入的方式打开文件，会覆盖已存在的文件
    a 以追加的方式打开文件，如果文件存在，文件指针会放在文件的结尾
    b 以二进制模式打开文件
'''
f = open('test.txt', 'r')

# 文件的读取 —— read()
print(f.read())      # 读取全部内容
```

```
print(f.readline()) # 读取一行
print(f.readlines())# 读取所有行

# 文件的关闭 —— close()
f.close()

f = open('test.txt', 'w')
# 文件的写入 —— write()
f.write('hello world')
f.close()

# 文件指针的移动 —— seek()
f = open('test.txt', 'a')
# seek(offset, whence)
# offset: 偏移量,
# whence: 偏移的起始位置
#   0: 文件的开头
#   1: 当前位置
#   2: 文件的结尾

print('当前文件指针位置: %s' % f.tell())
print('当前读取到一个字符, 字符为: ', f.read(1))
print('当前文件指针位置: ', f.tell())

f.seek(0) # 移动到文件的开头
print('移动到文件开头后的文件指针位置: ', f.tell())

## 显示文件指针位置—— tell()
print('当前文件指针位置: %s' % f.tell())

print('执行seek(5, 1)后: ')
f.seek(5, 1) # 从当前位置向后移动5个字符
print('当前文件指针位置: %s' % f.tell())
```

第二十四章 文本操作的应用

删除文本注释.py

```python
import re
import os
import readline  # 支持终端 Tab 补全
import glob      # 用于文件路径匹配

def complete_path(text, state):
    """实现终端文件路径 Tab 补全"""
    results = glob.glob(text + '*') + [None]
    return results[state]

# 注册路径补全
readline.set_completer(complete_path)
readline.parse_and_bind("tab: complete")

def extract_comments(file_content, language):
    """
    根据语言类型提取注释。
    :param file_content: 文件内容
    :param language: 用户选择的语言类型
    """
    if language == "C/C++":
        single_line_pattern = r"(?m)^\s*//.*$|(?<=\s)//.*$"
        multi_line_pattern = r"(?s)/\*.*?\*/"
    elif language == "Python":
        single_line_pattern = r"(?m)(^\s*#.*$|\s+#.*$)"
        multi_line_pattern = r"(?s)('''.*?'''|\"\"\".*?\"\"\")"
    elif language == "LaTeX":
        single_line_pattern = r"(?m)^\s*%.*$|(?<=\s)%.*$"
        multi_line_pattern = None

    # 提取注释
    single_line_comments = re.findall(single_line_pattern,
    file_content)
    multi_line_comments = re.findall(multi_line_pattern,
```

```
        file_content) if multi_line_pattern else []
34
35      return [c for c in (single_line_comments +
        multi_line_comments) if c.strip()]
36
37  def list_comments_with_index(comments):
38      """列出所有检测到的注释"""
39      print("检测到以下注释：")
40      for idx, comment in enumerate(comments, 1):
41          print(f"{idx}. {repr(comment.strip())}")
42
43  def get_comments_to_keep(comments):
44      """用户选择需要保留的注释"""
45      user_input =
        input("输入需要保留的注释编号（用逗号分隔），直接回车全部删除：
        ")
46      if not user_input.strip():
47          return []
48
49      try:
50          indices = [int(x) - 1 for x in user_input.split(',')]
51          return [comments[i] for i in indices if 0 <= i <
        len(comments)]
52      except ValueError:
53          print("输入格式不正确，将默认删除所有注释。")
54          return []
55
56  def remove_comments(file_content, comments_to_keep, all_comments):
57      """删除未保留的注释"""
58      for comment in all_comments:
59          if comment not in comments_to_keep:
60              file_content = file_content.replace(comment, '', 1)
61      return file_content
62
63  def clean_file(input_file, language):
64      """主函数：读取文件，处理注释，生成新文件"""
65      with open(input_file, 'r', encoding='utf-8') as f:
66          file_content = f.read()
67
68      comments = extract_comments(file_content, language)
69
70      if not comments:
71          print("未检测到任何注释。")
72          return
73
74      list_comments_with_index(comments)
75      comments_to_keep = get_comments_to_keep(comments)
76
77      cleaned_content = remove_comments(file_content,
        comments_to_keep, comments)
```

```
78
79      # 生成新文件名并写入
80      base_name, ext = os.path.splitext(input_file)
81      new_file_name = f"{base_name}_clean{ext}"
82      with open(new_file_name, 'w', encoding='utf-8') as f:
83          f.write(cleaned_content)
84
85      print(f"处理完成，新文件已保存为：{new_file_name}")
86
87  def main():
88      """程序入口，用户选择语言和输入文件"""
89      print("请选择要处理的语言类型：")
90      print("1. C/C++")
91      print("2. Python")
92      print("3. LaTeX")
93
94      language_choice = input("请输入对应的数字：  ").strip()
95      language_map = {"1": "C/C++", "2": "Python", "3": "LaTeX"}
96
97      if language_choice not in language_map:
98          print("无效选择，请重新运行程序。")
99          return
100
101     language = language_map[language_choice]
102     input_file = input(f"请输入要处理的 {language} 文件路径：  ")
103
104     if os.path.isfile(input_file):
105         clean_file(input_file, language)
106     else:
107         print("文件不存在，请检查路径是否正确。")
108
109 if __name__ == "__main__":
110     main()
```

递归删除整个文件夹中注释.py

```
1  import re
2  import os
3  import shutil
4  import readline  # 支持终端 Tab 补全
5  import glob      # 用于文件路径匹配
6
7  def complete_path(text, state):
8      """
9      实现终端文件路径Tab补全功能。
```

```
10
11      Args:
12          text (str): 用户当前输入的文本
13          state (int): 补全状态索引
14
15      Returns:
16          str: 匹配的路径补全结果，如果没有更多匹配则返回None
17      """
18      results = glob.glob(text + '*') + [None]
19      return results[state]
20
21  # 注册路径补全
22  readline.set_completer(complete_path)
23  readline.parse_and_bind("tab: complete")
24
25  def get_file_language(file_path):
26      """
27      根据文件后缀名判断编程语言类型。
28
29      Args:
30          file_path (str): 文件路径
31
32      Returns:
33          str: 语言类型字符串，如果不支持则返回None
34
35      支持的文件类型:
36          - C/C++: .c, .cpp, .cc, .cxx, .h, .hpp, .hxx
37          - Python: .py, .pyw
38          - LaTeX: .tex, .latex
39          - Bash: .sh, .bash
40          - Lua: .lua
41      """
42      ext = os.path.splitext(file_path)[1].lower()
43
44      if ext in ['.c', '.cpp', '.cc', '.cxx', '.h', '.hpp', '.hxx']:
45          return 'C/C++'
46      elif ext in ['.py', '.pyw']:
47          return 'Python'
48      elif ext in ['.tex', '.latex']:
49          return 'LaTeX'
50      elif ext in ['.sh', '.bash']:
51          return 'Bash'
52      elif ext in ['.lua']:
53          return 'Lua'
54      else:
55          return None
56
57  def extract_comments(file_content, language):
58      """
59      根据编程语言类型提取文件中的注释内容。
```

```
60
61      Args:
62          file_content (str): 文件内容
63          language (str): 编程语言类型
64
65      Returns:
66          list: 提取到的注释列表
67
68      支持的注释格式:
69          - C/C++: 单行注释(//) 和 多行注释(/* */)
70          - Python: 单行注释(#) 和 文档字符串(''' ''' 或 """ """)
71          - LaTeX: 单行注释(%)
72          - Bash: 单行注释(#) 和 多行注释(<< 'EOF' ... EOF)
73          - Lua: 单行注释(--) 和 多行注释(--[[ ]])
74      """
75      if language == "C/C++":
76          # C/C++ 单行注释 // 和多行注释 /* */
77          single_line_pattern = r"(?m)^\s*//.*$|(?<=\s)//.*$"
78          multi_line_pattern = r"(?s)/\*.*?\*/"
79      elif language == "Python":
80          # Python 单行注释 # 和文档字符串 ''' """
81          single_line_pattern = r"(?m)(^\s*#.*$|\s+#.*$)"
82          multi_line_pattern = r"(?s)('''.*?'''|\"\"\".*?\"\"\")"
83      elif language == "LaTeX":
84          # LaTeX 单行注释 %
85          single_line_pattern = r"(?m)^\s*%.*$|(?<=\s)%.*$"
86          multi_line_pattern = None
87      elif language == "Bash":
88          # Bash 单行注释 # 和多行注释 << 'EOF' ... EOF 或 << "EOF"
    ... EOF
89          single_line_pattern = r"(?m)(^\s*#.*$|\s+#.*$)"
90          multi_line_pattern =
    r"(?s)<<\s*['\"]?(\w+)['\"]?\s*\n.*?\n\1"
91      elif language == "Lua":
92          # Lua 单行注释 -- 和多行注释 --[[ ]] 或 --[=[ ]=] 等
93          single_line_pattern =
    r"(?m)^\s*--(?!\[).*$|(?<=\s)--(?!\[).*$"
94          multi_line_pattern = r"(?s)--\[=*\[.*?\]=*\]"
95      else:
96          return []
97
98      # 提取注释
99      single_line_comments = re.findall(single_line_pattern,
    file_content)
100     multi_line_comments = re.findall(multi_line_pattern,
    file_content) if multi_line_pattern else []
101
102     return [c for c in (single_line_comments +
    multi_line_comments) if c.strip()]
103
```

```
104 def list_comments_with_index(comments):
105     """
106     列出检测到的所有注释并显示编号，用于用户选择保留哪些注释。
107 
108     Args:
109         comments (list): 注释列表
110 
111     说明:
112         此函数会在控制台打印所有检测到的注释，每个注释前面都有编号，
113         方便用户在交互模式下选择需要保留的注释。
114     """
115     print("检测到以下注释: ")
116     for idx, comment in enumerate(comments, 1):
117         print(f"{idx}. {repr(comment.strip())}")
118 
119 def get_comments_to_keep(comments):
120     """
121     交互式获取用户希望保留的注释。
122 
123     Args:
124         comments (list): 注释列表
125 
126     Returns:
127         list: 用户选择保留的注释列表
128 
129     说明:
130         此函数提示用户输入需要保留的注释编号（用逗号分隔），
131 
    如果用户直接回车则删除所有注释。输入格式错误时默认删除所有注释。
132     """
133     user_input =
    input("输入需要保留的注释编号（用逗号分隔），直接回车全部删除:
    ")
134     if not user_input.strip():
135         return []
136 
137     try:
138         indices = [int(x) - 1 for x in user_input.split(',')]
139         return [comments[i] for i in indices if 0 <= i <
    len(comments)]
140     except ValueError:
141         print("输入格式不正确，将默认删除所有注释。")
142         return []
143 
144 def remove_comments(file_content, comments_to_keep, all_comments):
145     """
146     从文件内容中移除未被保留的注释。
147 
148     Args:
149         file_content (str): 原始文件内容
```

```
150         comments_to_keep (list): 用户选择保留的注释列表
151         all_comments (list): 所有检测到的注释列表
152 
153     Returns:
154         str: 移除指定注释后的文件内容
155 
156     说明:
157         此函数会遍历所有检测到的注释，如果注释不在保留列表中，
158         就从文件内容中移除该注释。使用replace方法进行替换，
159         每次只替换一个匹配项以避免误删。
160     """
161     for comment in all_comments:
162         if comment not in comments_to_keep:
163             file_content = file_content.replace(comment, '', 1)
164     return file_content
165 
166 def clean_file(input_file, output_file, interactive=True):
167     """
168     处理单个文件的注释清理。
169 
170     Args:
171         input_file (str): 输入文件的完整路径
172         output_file (str): 输出文件的完整路径
173         interactive (bool): 是否使用交互模式，默认True
174 
175     Returns:
176         bool: 处理成功返回True，失败返回False
177 
178     说明:
179         1. 根据文件后缀名自动识别语言类型
180         2. 读取文件内容并提取注释
181         3. 在交互模式下让用户选择保留的注释
182         4. 移除未保留的注释并写入输出文件
183         5. 如果文件不支持或没有注释，将直接复制原文件
184     """
185     # 根据文件后缀名判断语言类型
186     language = get_file_language(input_file)
187     if not language:
188         print(f"跳过不支持的文件类型：{input_file}")
189         return False
190 
191     # 尝试读取文件内容
192     try:
193         with open(input_file, 'r', encoding='utf-8') as f:
194             file_content = f.read()
195     except Exception as e:
196         print(f"读取文件失败 {input_file}: {e}")
197         return False
198 
199     # 提取注释
```

```
200     comments = extract_comments(file_content, language)
201
202     # 如果没有注释，直接复制文件
203     if not comments:
204         print(f"文件 {input_file} 未检测到任何注释，直接复制。")
205         try:
206             with open(output_file, 'w', encoding='utf-8') as f:
207                 f.write(file_content)
208             return True
209         except Exception as e:
210             print(f"写入文件失败 {output_file}: {e}")
211             return False
212
213     # 根据交互模式处理注释选择
214     if interactive:
215         print(f"\n处理文件: {input_file} ({language})")
216         list_comments_with_index(comments)
217         comments_to_keep = get_comments_to_keep(comments)
218     else:
219         # 非交互模式下删除所有注释
220         comments_to_keep = []
221
222     # 移除注释并生成清理后的内容
223     cleaned_content = remove_comments(file_content,
    comments_to_keep, comments)
224
225     # 尝试写入输出文件
226     try:
227         with open(output_file, 'w', encoding='utf-8') as f:
228             f.write(cleaned_content)
229         print(f"处理完成: {input_file} -> {output_file}")
230         return True
231     except Exception as e:
232         print(f"写入文件失败 {output_file}: {e}")
233         return False
234
235 def process_directory(input_dir, output_dir=None,
    interactive=True):
236     """
237     递归处理目录中所有支持的文件，移除注释并输出到新目录。
238
239     Args:
240         input_dir (str): 要处理的输入目录路径
241         output_dir (str, optional): 输出目录路径，默认为输入目录名
    + '_clean'
242         interactive (bool): 是否使用交互模式，默认True
243
244     说明:
245         1. 递归遍历输入目录中的所有文件和子目录
246         2. 对于支持的文件类型 (C/C++, Python, LaTeX, Bash,
```

```
    Lua) 进行注释清理
247         3. 对于不支持的文件类型，直接复制到输出目录
248         4. 保持原有的目录结构和文件层次关系
249         5. 在交互模式下，每个文件都会让用户选择保留的注释
250         6. 如果输出目录已存在，会询问用户是否删除重建
251     """
252     # 检查输入目录是否存在
253     if not os.path.isdir(input_dir):
254         print(f"目录不存在: {input_dir}")
255         return
256 
257     # 设置默认输出目录
258     if output_dir is None:
259         output_dir = f"{input_dir}_clean"
260 
261     # 处理已存在的输出目录
262     if os.path.exists(output_dir):
263         response = input(f"输出目录 {output_dir}
    已存在，是否删除重建？(y/n): ").strip().lower()
264         if response == 'y':
265             shutil.rmtree(output_dir)
266         else:
267             print("操作已取消。")
268             return
269 
270     # 创建输出目录
271     os.makedirs(output_dir, exist_ok=True)
272 
273     # 初始化计数器
274     processed_count = 0  # 已处理的文件数量
275     skipped_count = 0    # 跳过的文件数量
276 
277     # 递归遍历输入目录
278     for root, dirs, files in os.walk(input_dir):
279         # 计算当前目录相对于输入目录的相对路径
280         rel_path = os.path.relpath(root, input_dir)
281 
282         # 确定当前目录对应的输出目录路径
283         if rel_path != '.':
284             current_output_dir = os.path.join(output_dir,
    rel_path)
285         else:
286             current_output_dir = output_dir
287 
288         # 创建对应的输出目录 (如果不存在)
289         os.makedirs(current_output_dir, exist_ok=True)
290 
291         # 处理当前目录下的所有文件
292         for file in files:
293             input_file = os.path.join(root, file)
```

```
294             output_file = os.path.join(current_output_dir, file)
295
296             # 检查文件是否为支持的类型
297             if get_file_language(input_file):
298                 # 支持的文件类型，进行注释处理
299                 if clean_file(input_file, output_file,
    interactive):
300                     processed_count += 1
301                 else:
302                     skipped_count += 1
303             else:
304                 # 不支持的文件类型，直接复制到输出目录
305                 try:
306                     shutil.copy2(input_file, output_file)
307                     skipped_count += 1
308                 except Exception as e:
309                     print(f"复制文件失败 {input_file}: {e}")
310                     skipped_count += 1
311
312     # 打印处理结果统计
313     print(f"\n处理完成! ")
314     print(f"已处理文件: {processed_count} 个")
315     print(f"跳过文件: {skipped_count} 个")
316     print(f"输出目录: {output_dir}")
317
318 def main():
319     """
320     程序主入口函数，负责用户交互和参数设置。
321
322     功能说明:
323         1. 显示程序介绍和支持的文件类型
324         2. 获取用户输入的目录路径
325         3. 询问用户是否使用交互模式
326         4. 设置输出目录路径
327         5. 调用process_directory函数开始处理
328
329     支持的文件类型:
330         - C/C++: .c, .cpp, .cc, .cxx, .h, .hpp, .hxx
331         - Python: .py, .pyw
332         - LaTeX: .tex, .latex
333         - Bash: .sh, .bash
334         - Lua: .lua
335
336     交互模式说明:
337         -
    交互模式(y): 对每个有注释的文件，都会显示所有注释并让用户选择保留哪些
338         - 批处理模式(n): 自动删除所有检测到的注释，无需用户干预
339     """
340     # 显示程序标题和功能介绍
341     print("代码注释清理工具 - 目录批处理版本")
```

```
342     print("支持的文件类型: C/C++ (.c, .cpp, .h, .hpp等), Python
        (.py), LaTeX (.tex), Bash (.sh), Lua (.lua)")
343     print()
344
345     # 获取用户输入的目录路径
346     input_dir = input("请输入要处理的目录路径: ").strip()
347
348     # 验证目录是否存在
349     if not os.path.isdir(input_dir):
350         print("目录不存在, 请检查路径是否正确。")
351         return
352
353     # 询问是否使用交互模式
354     print("\n模式选择: ")
355     print("交互模式(y): 每个文件都会询问保留哪些注释")
356     print("批处理模式(n): 自动删除所有注释")
357     interactive_choice = input("是否使用交互模式？(y/n, 默认n):
        ").strip().lower()
358     interactive = interactive_choice == 'y'
359
360     # 设置输出目录
361     default_output = f"{input_dir}_clean"
362     print(f"\n输出目录设置 (默认: {default_output}) ")
363     output_dir =
        input(f"请输入输出目录路径 (直接回车使用默认) : ").strip()
364     if not output_dir:
365         output_dir = default_output
366
367     # 显示处理参数确认
368     print(f"\n=== 处理参数确认 ===")
369     print(f"输入目录: {input_dir}")
370     print(f"输出目录: {output_dir}")
371     print(f"处理模式: {'交互模式' if interactive else
        '批处理模式'}")
372     print("=" * 20)
373
374     # 开始处理目录
375     process_directory(input_dir, output_dir, interactive)
376
377 if __name__ == "__main__":
378     main()
```

```
$> python uncomment_dir.py
代码注释清理工具 - 目录批处理版本
支持的文件类型: C/C++ (.c, .cpp, .h, .hpp等), Python (.py), LaTeX (.tex),
    Bash (.sh), Lua (.lua)

请输入要处理的目录路径: ./old
```

```
模式选择:
交互模式(y): 每个文件都会询问保留哪些注释
批处理模式(n): 自动删除所有注释
是否使用交互模式？(y/n, 默认n):

输出目录设置 (默认: ./old_clean)
请输入输出目录路径 (直接回车使用默认) :

=== 处理参数确认 ===
输入目录: ./old
输出目录: ./old_clean
处理模式: 批处理模式
====================
输出目录 ./old_clean 已存在, 是否删除重建？(y/n): y
处理完成: ./old/bash/7z_unrar.sh -> ./old_clean/bash/7z_unrar.sh
处理完成: ./old/bash/7z_unzip.sh -> ./old_clean/bash/7z_unzip.sh
处理完成: ./old/C/zipfolder.c -> ./old_clean/C/zipfolder.c
处理完成: ./old/C/zip_split_folder.c -> ./old_clean/C/zip_split_folder.c
处理完成: ./old/python/audio2video.py -> ./old_clean/python/audio2video.py
处理完成: ./old/python/chinese_num_covert.py ->
    ./old_clean/python/chinese_num_covert.py
处理完成: ./old/python/git_add_batch.py ->
    ./old_clean/python/git_add_batch.py
处理完成: ./old/python/git_batch_folder.py ->
    ./old_clean/python/git_batch_folder.py
处理完成: ./old/python/merged_avi.py -> ./old_clean/python/merged_avi.py
处理完成: ./old/python/merged_flv.py -> ./old_clean/python/merged_flv.py
处理完成: ./old/python/merged_mp4.py -> ./old_clean/python/merged_mp4.py
处理完成: ./old/python/merged_ts.py -> ./old_clean/python/merged_ts.py
文件 ./old/python/name.py 未检测到任何注释, 直接复制。
文件 ./old/python/name_dir.py 未检测到任何注释, 直接复制。
处理完成: ./old/python/uncomment.py -> ./old_clean/python/uncomment.py
处理完成: ./old/python/uncomment_dir.py ->
    ./old_clean/python/uncomment_dir.py
处理完成: ./old/python/unzip.py -> ./old_clean/python/unzip.py
处理完成: ./old/python/unzip_file.py -> ./old_clean/python/unzip_file.py
处理完成: ./old/python/video_timeline.py ->
    ./old_clean/python/video_timeline.py
处理完成: ./old/python/video_timeline_cur_folder.py ->
    ./old_clean/python/video_timeline_cur_folder.py
处理完成: ./old/python/zip.py -> ./old_clean/python/zip.py
处理完成: ./old/python/zip100.py -> ./old_clean/python/zip100.py

处理完成!
已处理文件: 22 个
跳过文件: 4 个
输出目录: ./old_clean
```

第二十五章 上下文管理

一、 with 简化资源管理

在 Python 编程中，资源管理是一个重要但容易被忽视的环节。`with` 关键字为我们提供了一种优雅的方式来处理文件操作、数据库连接等需要明确释放资源的场景。

with 是 Python 中的一个关键字，用于上下文管理协议（Context Management Protocol）。它简化了资源管理代码，特别是那些需要明确释放或清理的资源（如文件、网络连接、数据库连接等）。

1) 基本语法

```
'''
expression 返回一个支持上下文管理协议的对象
as variable 是可选的，用于将表达式结果赋值给变量
'''
with expression [as variable]:
    # 代码块
```

1) 文件操作

```
# 文件操作
with open('example.txt', 'r') as file:
    content = file.read()
    print(content)
# 文件已自动关闭
```

```
# 同时打开多个文件
with open('input.txt', 'r') as infile, open('output.txt',
    'w') as outfile:
```

```
    content = infile.read()
    outfile.write(content.upper())
```

2) 数据库连接

```
import sqlite3

with sqlite3.connect('database.db') as conn:
    cursor = conn.cursor()
    cursor.execute('SELECT * FROM users')
    results = cursor.fetchall()
# 连接自动关闭
```

3) 线程锁

```
import threading
lock = threading.Lock()
with lock:
    # 临界区代码
    print("这段代码是线程安全的")
```

4) 临时修改系统状态

```
import decimal
with decimal.localcontext() as ctx:
    ctx.prec = 42  # 临时设置高精度
    # 执行高精度计算
# 精度恢复原设置
```

2) 工作原理

with 语句背后是 Python 的上下文管理协议，该协议要求对象实现两个方法：

1) `__enter__()`：进入上下文时调用，返回值赋给 as 后的变量。

2) `__exit__()`：在 with 块结束后调用，用于清理资源。

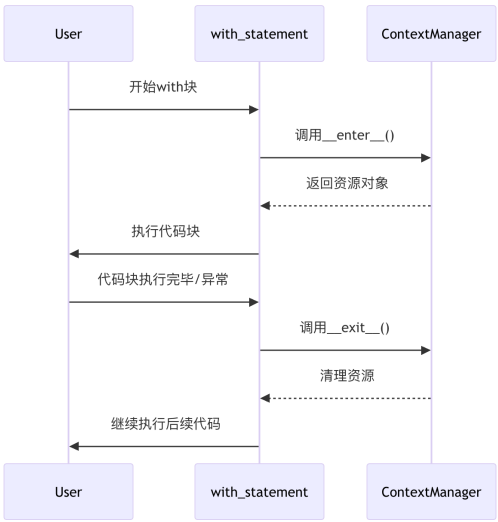

图 25.1 with 上下文管理执行流程

二、 创建自定义的上下文管理器

1) 类的方式

我们可以通过实现 `__enter__()` 和 `__exit__()` 方法创建自定义的上下文管理器

```
def __enter__(self):
    # 创建资源
    return self

'''
exc_type: 异常类型
exc_val: 异常值
exc_tb: 异常追踪信息

如果 __exit__() 返回 True，则表示异常已被处理，不会继续传播；返回 False 或
    None，异常会继续向外传播。
'''
def __exit__(self, exc_type, exc_val, exc_tb):
```

```
    # 释放资源
    return False
```

```
class Timer:
    def __enter__(self):
        import time
        self.start = time.time()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        import time
        self.end = time.time()
        print(f"耗时: {self.end - self.start:.2f}秒")
        return False

# 使用示例
with Timer() as t:
    # 执行一些耗时操作
    sum(range(1000000))
```

2) contextlib 模块

```
from contextlib import contextmanager

@contextmanager
def tag(name):
    print(f"<{name}>")
    yield
    print(f"</{name}>")

# 使用示例
with tag("h1"):
    print("这是一个标题")
```

第五部分

[编程范式] 面向对象编程

第二十六章 面向对象

一、 类与对象

1) 类

定义一个类，使用关键字 class，类名的命名规则与变量名相同，首字母大写。

```
class Person[(father)]: # 类首字母大写 [] 部分可选，表示继承的父类
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def say(self):
        print('hello world')

p = Person('Tom', 20) # 实例化
print(p.name)
print(p.age)
p.say()
```

2) 继承

Python 支持多继承，使用逗号分隔。

```
class Teacher:
    def teaching(self):
        print('I am teaching')

class Dad:
    def cooking(self):
        print('I am cooking')

class Son(Teacher, Dad):
    def playing(self):
        print('I am playing')

s = Son()
s.teaching()
s.cooking()
```

```
s.playing()
```

3) 接口

Python 中没有接口的概念，但是可以使用抽象类来实现接口的功能。

```
class Animal:
    def run(self):
        pass # 占位符

class Dog(Animal):
    def run(self):
        print('dog is running')

class Cat(Animal):
    def run(self):
        print('cat is running')

d = Dog()
d.run()

c = Cat()
c.run()
```

4) 封装

(1) 私有属性

Python 中没有私有属性的概念，但是可以通过在属性名前加两个下划线来实现私有属性。

```
class Person:
    def __init__(self, name, age):
        self.__name = name
        self.__age = age

    def say(self):
        print('hello world')

p = Person('Tom', 20)
print(p.__name) # 不能访问

p.say()
```

(2) 私有方法

Python 中没有私有方法的概念，但是可以通过在方法名前加一个下划线来实现私有方法。

```
class Person:
    def __init__(self, name, age):
        self.__name = name
        self.__age = age

    def __say(self):
        print('hello world')

    def say(self):
        self.__say()

p = Person('Tom', 20)
p.say()
```

5) 多态

多态是指不同的对象对同一消息做出不同的响应。

```
class Animal:
    def run(self):
        pass

class Dog(Animal):
    def run(self):
        print('dog is running')

class Cat(Animal):
    def run(self):
        print('cat is running')

def run(animal):
    animal.run()

d = Dog()
run(d)

c = Cat()
run(c)
```

第六部分

图形可视化

第二十七章 PySide 概述

一、 PySide 简介

PySide6 是 Qt 官方支持和维护的 Python 绑定库，它允许开发者在 Python 中创建基于 Qt 的图形用户界面（GUI）应用程序。与 PyQt 相比，PySide6 在最新版本的 Qt 中得到了官方支持，并且使用 LGPL 许可证，这意味着你可以在自己的项目中使用 PySide6，而不需要开源你的应用程序的源代码

简单来说，PySide6 就是将 Qt 6 提供的功能“翻译”成 Python 语言，让你可以用 Python 来开发图形界面应用程序。

1) Qt 、PyQt 、PySide 之间的关系

Qt 是一个广泛使用的框架，用于开发跨平台的图形用户界面（GUI）应用程序。它提供了创建桌面和移动应用程序的工具。

PyQt 和 PySide 是两种流行的 Python 绑定库，允许你用 Python 编写 Qt 应用程序。

1) Qt：这是一个用 C++ 编写的核心框架，用于开发 GUI 应用程序。它包括小部件、图形、网络等库。

2) PyQt：

    1) 由 Riverbank Computing 开发。
    2) 提供了 Qt 库到 Python 的绑定。
    3) 通常使用 GPL 或商业许可证，这可能要求为专有软件购买许可证。

3) PySide：

    1) 由 Qt 公司（之前是诺基亚和 Digia）开发。
    2) 提供了 Qt 库到 Python 的绑定。
    3) 使用 LGPL 许可证，这允许更灵活的许可证，尤其适用于商业应用程序。

PySide6/PyQt6 都是基于 Qt6, 它们之间的代码没什么区别;PySide2/PyQt5 都是基于 Qt5, 它们之间的代码也没什么区别。以下代码可以相互替换

```
from PySide6 import *
from PyQt6 import *
```

```
from PySide2 import *
from PyQt5 import *
```

2) PySide 与 PyQt 的比较

1) 信号与槽的命名：

下面代码等价：

```
from PySide6.QtCore import Signal, Slot
from PyQt6.QtCore import pyqtSignal as Signal, pyqtSlot as Slot
```

2) 枚举类型的快捷方式：

```
Qt.Wednesday            # PySide6
Qt.DayOfWeek.Wednesday  # PyQt6/PySide6
```

但是有`qtpy`模块，使得 PySide 和 PyQt 可以通用，不需要修改代码, 即使 Python 环境下只有 PyQt6 和 qtpy 模块，也可以运行 PySide6 的代码。

```
from qtpy.QtCore import Qt
```

3) 主要模块

1) QtCore：提供核心功能，如事件处理、定时器、文件和目录操作、线程等。

2) QtGui：提供图形功能，如 2D 图形绘制、字体、图像和窗口管理。

3) QtWidgets：提供传统的桌面小部件，用于构建经典的用户界面组件。

4) QtQuick 和 QML：支持现代的声明式用户界面设计，适用于创建动态和交互丰富的界面。

5) QtNetwork：提供网络功能，如 TCP/IP 套接字、HTTP 请求等。

6) QtSql：提供对 SQL 数据库的访问和操作功能。

二、 安装 PySide

https://doc.qt.io/qtforpython-6/quickstart.html#

```
$> conda create --name myenv python=3.9
$> conda activate myPySide
$> pip install pyside6
$> pip install pyside6==6.4.1

# 查看已安装的包
$> pip list | grep PySide6
$> pip show pyside6
Name: PySide6
Version: 6.7.2
Summary: Python bindings for the Qt cross-platform application and UI
    framework
Home-page: https://pyside.org
Author:
Author-email: Qt for Python Team <pyside@qt-project.org>
License: LGPL
Location:
    /home/tony/.application/Miniconda/envs/myPySide/lib/python3.9/site-packages
Requires: PySide6-Addons, PySide6-Essentials, shiboken6
Required-by:
```

1) 常见问题

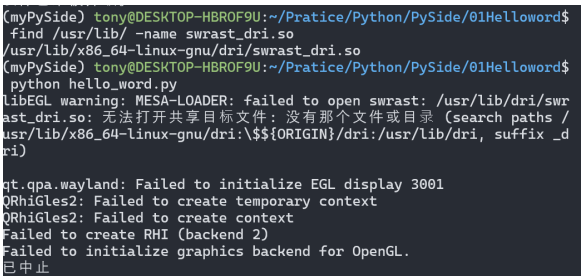

图 27.1 找不到 swrast

(1) PySide 找不到 libstdc++

```
ln -sf /usr/lib/x86_64-linux-gnu/libstdc++.so.6
    /home/tony/.application/Miniconda/bin/../lib/libstdc++.so.6
```

(2) conda 的 python 版本与系统的版本不一致

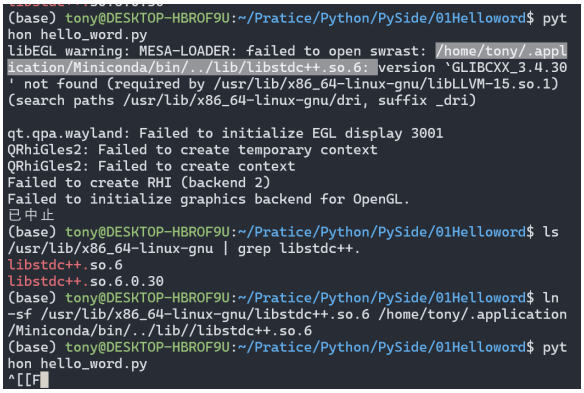

图 27.2 PySide 找不到 libstdc++

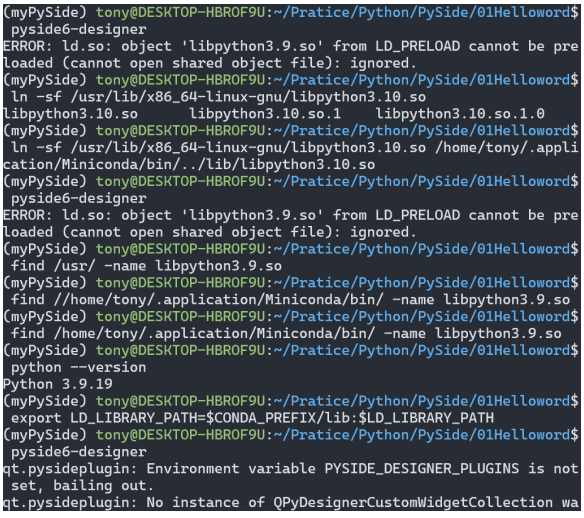

图 27.3 conda 的 python 版本与系统的版本不一致

```
export LD_LIBRARY_PATH=$CONDA_PREFIX/lib:$LD_LIBRARY_PATH
```

第二十八章 [Python 库] Matplotlib 概述

一、 简介

matplotlib 是 python 中一个非常强大的 2D 函数绘图模块，它提供了子模块 `pyplot` 和 `pylab`。

pylab 是对 pyplot 和 numpy 模块的封装，更适合在 IPython 交互式环境中使用。

对于一个项目来说，官方建议分别导入使用，这样代码更清晰，即：

```
import numpy as np
import matplotlib.pyplot as plt

# 不要使用
import pylab as pl
```

二、 安装

```
conda install matplotlib
# 或
pip install matplotlib
```

三、 基本框架

```
# 分别导入 numpy 和 pyplot 模块
import numpy as np
```

```
3  import matplotlib.pyplot as plt
4
5  # 生成 X 坐标, 256个采样值足够图像平滑
6  X = np.linspace(-np.pi, np.pi, 256, endpoint=True)
7
8  # 生成 Y 坐标
9  C,S = np.cos(X), np.sin(X)
10
11 # 绘制正余弦
12 plt.plot(X,S)
13 plt.plot(X,C)
14
15 # 显示图像
16 plt.show()
```

```
# 必须导入的库
import matplotlib.pyplot as plt
# legend 功能: 用于为图形添加图例
plt.legend(*args, **kwargs)
# 参数说明:
# loc : 图例的位置
## 'best': 自动选择最佳位置
## 'upper right': 右上角
## 'upper left': 左上角
## 'lower left': 左下角
## 'lower right': 右下角
## 'right': 右侧
## 'center left': 左侧居中
## 'center right': 右侧居中
## 'lower center': 底部居中
## 'upper center': 顶部居中
## 'center': 中央
# fontsize : 图例文字的大小
# title : 图例标题
# shadow : 是否添加阴影
# bbox_to_anchor : 图例相对于图形的位置
```

```
# 必须导入的库
import matplotlib.pyplot as plt
# title 功能: 为图表添加标题
plt.title(label, fontdict=None, loc='center', pad=None, **kwargs)
# 参数说明:
# label : 标题文本
# fontdict : 设置字体的字典, 如字体大小、颜色等
# loc : 标题的位置
## 'left': 左对齐
## 'center': 居中对齐
## 'right': 右对齐
# pad : 标题与图像之间的间距
```

```
# 必须导入的库
import matplotlib.pyplot as plt
# show 功能：显示当前所有绘图
plt.show(*args, **kwargs)
# 参数说明：
# block ：是否阻塞脚本的运行，直到窗口关闭
## True：阻塞（默认）
## False：不阻塞
```

第二十九章 [matplotlib] 绘图

一、 折线/散点图

1) plt.plot 折线/散点图

`plt.plot()` 用于绘制散点图，默认情况下，`plt.plot(x, y)` 会连接每个点，生成一条平滑的连线图。

```
# 必须导入的库
import matplotlib.pyplot as plt
# plot 功能：用于绘制散点图
plt.plot(*args, scalex=True, scaley=True, data=None, **kwargs)
# 参数说明：
# x : x坐标
# y : y坐标
# fmt : 线条样式
## 'o': 点
## '-o': 点连线
## '-H': 六边形连线
# data : 数据源
# scalex, scaley : 是否自动缩放x或y轴
```

```
import numpy as np
import matplotlib.pyplot as plt

# 生成从 0 到 1.1，步长为 0.01 的数据
data = np.arange(0, 1.1, 0.01)

# 设置图表的标题
plt.title('lines')  # 添加标题

# 设置 x 轴和 y 轴的名称
plt.xlabel('x')  # 添加 x 轴的名称
plt.ylabel('y')  # 添加 y 轴的名称

# 设置 x 轴和 y 轴的范围
plt.xlim((0, 1))  # 确定 x 轴范围
```

```
plt.ylim((0, 1))  # 确定 y 轴范围

# 设置 x 轴和 y 轴的刻度
plt.xticks([0, 0.2, 0.4, 0.6, 0.8, 1])  # 规定 x 轴刻度
plt.yticks([0, 0.2, 0.4, 0.6, 0.8, 1])  # 确定 y 轴刻度

# 使用 plot 函数绘制连线图
plt.plot(data, data)  # 绘制 y = x 曲线
plt.plot(data, data**2)  # 绘制 y = x^2 曲线

# 添加图例
plt.legend(['y=x', r'$y=x^2$'])  # r'' 使得数学公式正确显示

# 保存图像
plt.savefig('../tmp/默认连线图.png')

# 显示图像
plt.show()
```

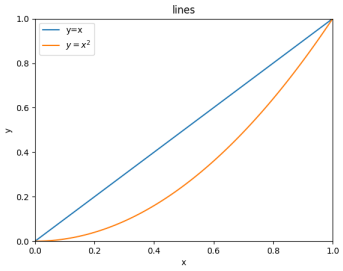

图 29.1 plot 默认连线图

```
import matplotlib.pyplot as plt
import numpy as np
fig = plt.figure(figsize=(6,4))
x = np.linspace(0, 10, 30)
y = np.sin(x)

# 等价于 plt.plot(x, y, mark='o', color='blue')
plt.plot(x, y, 'ob')
```

```
$> cat people.csv
指标,年末总人口(万人),0-14岁人口(万人),15-64岁人口(万人),65岁及以上人口(万人)
```

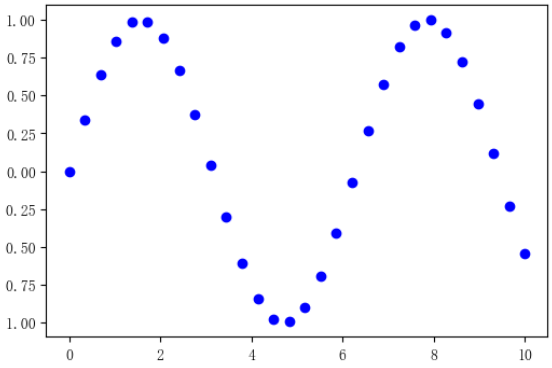

图 29.2 plot 散点图

```
2000年,126743,29012,88910,8821
2001年,127627,28716,89849,9062
2002年,128453,28774,90302,9377
2003年,129227,28559,90976,9692
2004年,129988,27947,92184,9857
2005年,130756,26504,94197,10055
2006年,131448,25961,95068,10419
2007年,132129,25660,95833,10636
2008年,132802,25166,96680,10956
2009年,133450,24659,97484,11307
2010年,134091,22259,99938,11894
2011年,134735,22164,100283,12288
2012年,135404,22287,100403,12714
2013年,136072,22329,100582,13161
2014年,136782,22558,100469,13755
2015年,137462,22715,100361,14386
2016年,138271,23008,100260,15003
2017年,139008,23348,99829,15831
2018年,139538,23523,99357,16658
2019年,140005,23493,98914,17599
```

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] =' SimHei'
# 设置中文显示
plt.rcParams['axes.unicode_minus'] = False
data = pd.read_csv('../data/people.csv')
name = data.columns  # 提取其中的columns字段，视为数据的标签
values = data.values  # 提取其中的values字段，数据的存在位置
plt.figure(figsize=(9, 7))  # 设置画布
plt.scatter(values[:, 0], values[:, 1], marker='o')  # 绘制散点图
plt.xlabel('年份')  # 添加横轴标签
```

```
plt.ylabel('年末总人口（万人）')  # 添加y轴名称
plt.xticks(range(0, 20), values[range(0, 20), 0], rotation=45)
plt.title('2000-2019年年末总人口散点图')  # 添加图表标题
plt.savefig('../tmp/2000-2019年年末总人口散点图.png') # 保存图表
plt.show()
```

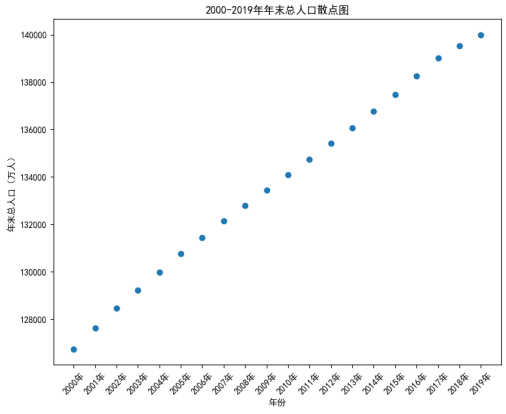

图 29.3 2000-2019 年年末总人口散点图

(1) -o 将散点图连线

使用 `plt.plot()` 可以绘制连线的散点图。

```
import matplotlib.pyplot as plt
import numpy as np

# 数据
x = np.linspace(0, 10, 30)
y = np.sin(x)

# 绘制连线的散点图
plt.plot(x, y, '-o', color='blue')  # 使用线条连接散点
plt.title('连线的散点图')
plt.show()
```

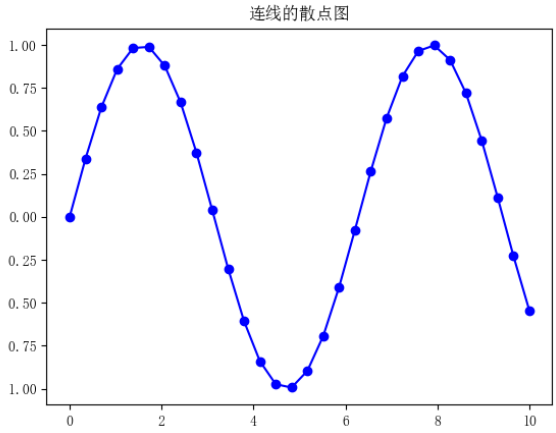

图 29.4 连线的散点图

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# 设置中文显示
plt.rcParams['font.sans-serif'] = 'SimHei'  # 解决中文乱码问题
plt.rcParams['axes.unicode_minus'] = False  # 设置正常显示负号

# 读取数据
data = pd.read_csv('../data/people.csv')
name = data.columns  # 提取columns字段，视为数据的标签
values = data.values  # 提取values字段，数据的存在位置

# 创建画布，并设置左右两个子图
plt.figure(figsize=(16, 7))  # 设置画布大小

# 左侧子图
plt.subplot(1, 2, 1)  # 创建第一个子图
plt.plot(values[:, 0], values[:, 1], color='r', linestyle='--',
    marker='o')  # 绘制折线图
plt.xlabel('年份')  # 添加横轴标签
plt.ylabel('年末总人口（万人）')  # 添加y轴名称
plt.xticks(range(0, 20), values[range(0, 20), 0], rotation=45)
plt.title('2000-2019年年末总人口点线图')  # 添加图表标题

# 右侧子图
plt.subplot(1, 2, 2)  # 创建第二个子图
plt.plot(values[:, 0], values[:, 2], 'bs-',  # 蓝色方块实线
         values[:, 0], values[:, 3], 'ro-.',  # 红色圆点虚线
         values[:, 0], values[:, 4], 'gH--')  # 绿色六边形点划线
plt.xlabel('年份')  # 添加横轴标签
plt.ylabel('年末总人口（万人）')  # 添加y轴名称
plt.xticks(range(0, 20), values[range(0, 20), 0], rotation=45)
```

```
33 plt.title('2000-2019年各年龄段总人口点线图')  # 添加图表标题
34 plt.legend(['0-14岁人口', '15-64岁人口', '65岁及以上人口'])
35
36 # 保存合并的图表
37 plt.tight_layout()  # 调整子图布局，避免重叠
38 plt.savefig('../tmp/2000-2019年总人口点线图.png')
39 plt.show()
```

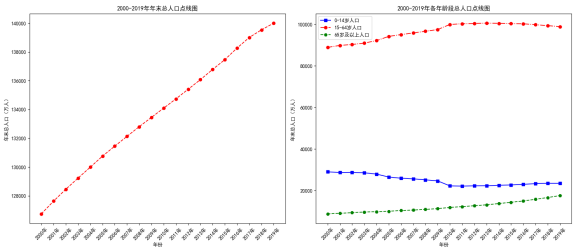

图 29.5 2000-2019 年总人口点线图

(2) 设置散点图属性

```
1  import matplotlib.pyplot as plt
2  import numpy as np
3  fig = plt.figure(figsize=(6,4))
4  x = np.linspace(0, 10, 30)
5  y = np.sin(x)
6
7  plt.plot(x, y, '-H', color='gray',    # 线条颜色 -H 代表六边形
8           markersize=15, linewidth=4, # 标记大小，线宽
9           markerfacecolor='white',    # 标记填充色
10          markeredgecolor='gray',     # 标记边框色
11          markeredgewidth=2)          # 标记边框宽度
```

(3) 绘制折线图

```
1  import numpy as np
2  import pandas as pd
```

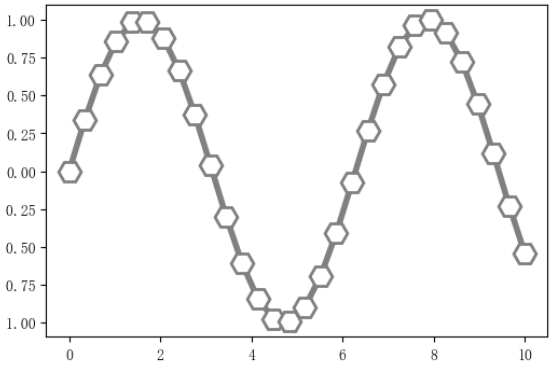

图 29.6 设置散点图属性

```
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] =' SimHei'
# 设置中文显示
plt.rcParams['axes.unicode_minus'] = False
data = pd.read_csv('../data/people.csv')
name = data.columns  # 提取其中的columns字段，视为数据的标签
values = data.values  # 提取其中的values字段，数据的存在位置
plt.figure(figsize=(9, 7))  # 设置画布
plt.plot(values[:, 0], values[:, 1], color='r', linestyle='--')
    # 绘制折线图
plt.xlabel('年份')  #添加横轴标签
plt.ylabel('年末总人口（万人）')  # 添加y轴名称
plt.xticks(range(0, 20), values[range(0, 20), 0], rotation=45)
plt.title('2000-2019年年末总人口折线图')  # 添加图表标题
plt.savefig('../tmp/2000-2019年年末总人口折线图.png')
plt.show()
```

2) plt.scatter 散点图

`plt.scatter()` 用于绘制散点图，允许设置点的大小、颜色、形状等属性。

```
scatter(x, y, s=None, c=None, marker=None, cmap=None, norm=None,
    vmin=None, vmax=None,
        alpha=None, linewidths=None, verts=None, edgecolors=None,
        hold=None, data=None, **kwargs)
# 参数说明:
# x : x坐标
# y : y坐标
# s : 点的大小
# c : 点的颜色
# marker : 点的形状
```

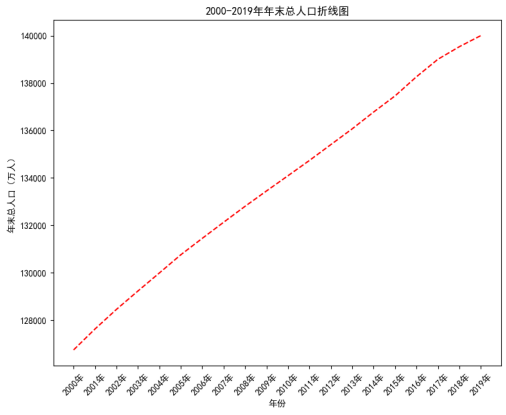

图 29.7 2000-2019 年年末总人口折线图

```
# cmap : 颜色映射表
# alpha : 透明度
# edgecolors : 点的边缘颜色
```

```
import matplotlib.pyplot as plt
import numpy as np

# 数据
x = np.random.rand(50)
y = np.random.rand(50)
sizes = np.random.rand(50) * 1000  # 点的大小
colors = np.random.rand(50)        # 点的颜色

# 绘制散点图
plt.scatter(x, y, s=sizes, c=colors, alpha=0.6, cmap='viridis')
plt.colorbar()  # 添加颜色条
plt.title('散点图示例')
plt.show()
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] =' SimHei'
# 设置中文显示
```

```
plt.rcParams['axes.unicode_minus'] = False
data = pd.read_csv('../data/people.csv')
name = data.columns  # 提取其中的columns字段，视为数据的标签
values = data.values  # 提取其中的values字段，数据的存在位置
plt.figure(figsize=(9, 7))  # 设置画布
plt.scatter(values[:, 0], values[:, 2], marker='o', c='red')   # 选取values中的第一列和第三列数据绘制散点图
plt.scatter(values[:, 0], values[:, 3], marker='D', c='blue')  # 选取values中的第一列和第四列数据绘制散点图
plt.scatter(values[:, 0], values[:, 4], marker='v', c='black') # 选取values中的第一列和第五列数据绘制散点图
plt.xlabel('年份')  # 添加横轴标签
plt.ylabel('年末总人口（万人）')  # 添加纵轴标签
plt.xticks(range(0, 20), values[range(0, 20), 0], rotation=45)
plt.title('2000-2019年各年龄段年末总人口散点图')  # 添加图表标题
plt.legend(['0-14岁人口', '15-64岁人口', '65岁及以上人口'])
    #添加图例
plt.savefig('../tmp/2000-2019年各年龄段年末总人口散点图.png')
plt.show()
```

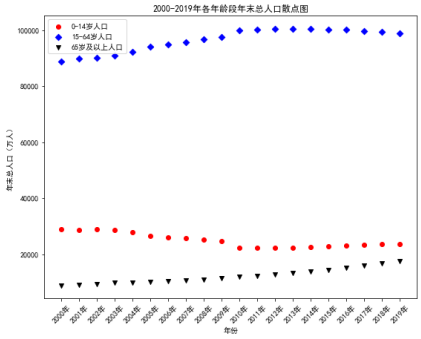

图 29.8 2000-2019 年各年龄段年末总人口数散点图

```
import matplotlib.pyplot as plt
import numpy as np

def scatter_create_color_graph():
    """
    绘制不同颜色值的散点图。
    x, y: 散点的坐标
    c: 颜色值（以十六进制表示）
    """
    x = [i for i in range(20)]
```

```
11      y = [i for i in range(20)]
12
13      plt.xticks(x)
14      plt.yticks(y)
15      c = np.linspace(0, 0xffffff, 20, endpoint=False)
16      plt.scatter(x, y, c=c, s=200, marker='o')
17      plt.title('不同颜色值绘制的散点图')
18
19  def scatter_create_markers_graph():
20      """
21      绘制不同标记大小和颜色的散点图。
22      marker: 标记类型
23      s: 标记大小
24      c: 标记颜色
25      """
26      x = np.array([i for i in range(20)])
27      y = np.array([i for i in range(20)])
28
29      plt.xticks(x)
30      plt.yticks(y)
31      plt.scatter(x, y, c='orange', s=200, marker='v')
32      plt.scatter(x + 1, y, c='gray', s=100, marker='^')
33      plt.title('不同标记大小和颜色绘制的散点图')
34
35  def scatter_create_size_graph():
36      """
37      根据坐标调整标记大小的散点图。
38      s: 根据位置动态调整标记大小
39      """
40      x = np.array([i for i in range(10)])
41      y = np.array([0] * len(x))
42      plt.ylim(-0.5, 1.5)
43      plt.yticks([0, 1])
44      plt.xticks(x)
45
46      sizes = [20 * (n + 1) ** 2 for n in range(len(x))]
47      plt.scatter(x, y, c='m', s=sizes)
48
49      sizes = [20 * (10 - n) ** 2 for n in range(len(x))]
50      plt.scatter(x, y + 1, c='m', s=sizes)
51      plt.title('根据坐标调整标记大小')
52
53  def scatter_create_random_graph():
54      """
55      随机坐标散点图。
56      x, y: 随机生成的坐标
57      marker: 标记类型
58      s: 标记大小
59      alpha: 标记透明度
60      """
```

```
61      x = np.random.randn(100)
62      y = np.random.randn(100)
63
64      plt.scatter(x, y, c='m', marker='p', s=500, alpha=0.6)
65      plt.title('随机坐标散点图')
66
67  def scatter_create_guess_graph():
68      """
69      多元高斯分布二维图。
70      s: 根据分布值调整标记大小
71      alpha: 标记透明度
72      """
73      mu_vec = np.array([0,0])
74      cov_mat = np.array([[1,0],[0,1]])
75      X = np.random.multivariate_normal(mu_vec, cov_mat, 1000)
76      R = X ** 2
77      R_sum = R.sum(axis = 1)
78
79      plt.scatter(X[:,0], X[:,1], color='m', marker='o',
80                  s = 32.*R_sum, edgecolor='black', alpha=0.5)
81      plt.title('多元高斯分布二维图')
82
83  def scatter_create_gradual_graph():
84      """
85      绘制同点渐变晕化效果。
86      c: 渐变颜色值
87      s: 标记大小
88      """
89      c = np.linspace(0xffff00, 0xffffff, 20, endpoint=False)
90      for i in range(19,-1,-1):
91          size = i * 10000 + 10
92          cval = hex(int(c[i]))[2:]
93          color = "#" + '0' * (6 - len(cval)) + cval
94          plt.scatter(0, 0, s=size, c=color)
95      plt.title('同点渐变晕化')
96
97  # 创建3x2的子图布局
98  plt.figure(figsize=(15, 10))
99
100 # 绘制每个子图
101 plt.subplot(3, 2, 1)
102 scatter_create_color_graph()
103
104 plt.subplot(3, 2, 2)
105 scatter_create_markers_graph()
106
107 plt.subplot(3, 2, 3)
108 scatter_create_size_graph()
109
110 plt.subplot(3, 2, 4)
```

```
111 scatter_create_random_graph()
112
113 plt.subplot(3, 2, 5)
114 scatter_create_guess_graph()
115
116 plt.subplot(3, 2, 6)
117 scatter_create_gradual_graph()
118
119 # 调整布局并显示图像
120 plt.tight_layout()
121 plt.show()
```

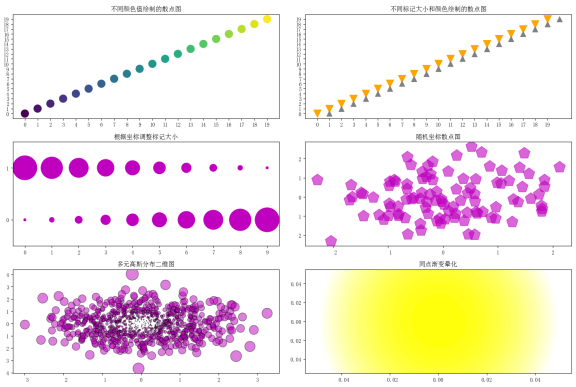

图 29.9 不同 scatter 属性演示

二、 条形图/柱形图

1) plt.bar 垂直条形图

`plt.bar()` 用于绘制垂直的条形图。

```
# 必须导入的库
import matplotlib.pyplot as plt
# bar 功能：绘制垂直条形图
plt.bar(x, height, width=0.8, bottom=None, align='center', **kwargs)
# 参数说明：
# x : 条形的 x 坐标
# height : 条形的高度
# width : 条形的宽度
# bottom : 条形的底部位置
# align : 对齐方式
## 'center'：居中
## 'edge'：边缘对齐
```

```python
import matplotlib.pyplot as plt

# 数据
categories = ['A', 'B', 'C', 'D']
values = [10, 24, 36, 12]

# 绘制垂直条形图
plt.bar(categories, values, color='skyblue')
plt.title('垂直条形图')
plt.show()
```

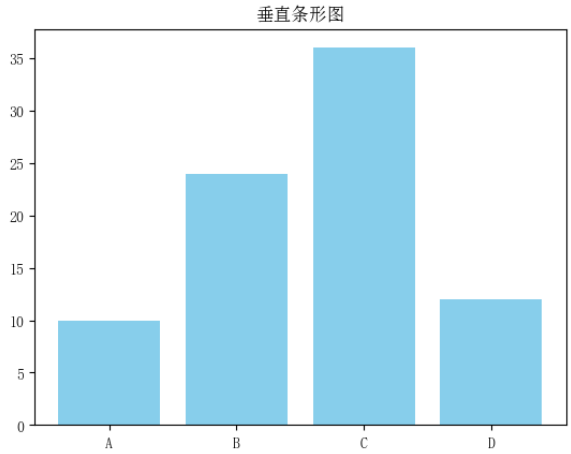

图 29.10 垂直条形图

2) plt.bar 堆叠条形图

`plt.bar()` 可以通过设置底部来实现堆叠条形图。

```python
import matplotlib.pyplot as plt

# 数据
categories = ['A', 'B', 'C', 'D']
values1 = [10, 20, 30, 40]
values2 = [5, 15, 25, 10]

# 绘制堆叠条形图
plt.bar(categories, values1, color='lightblue', label='Values 1')
```

```
plt.bar(categories, values2, bottom=values1, color='orange',
    label='Values 2')
plt.title('堆叠条形图')
plt.legend()
plt.show()
```

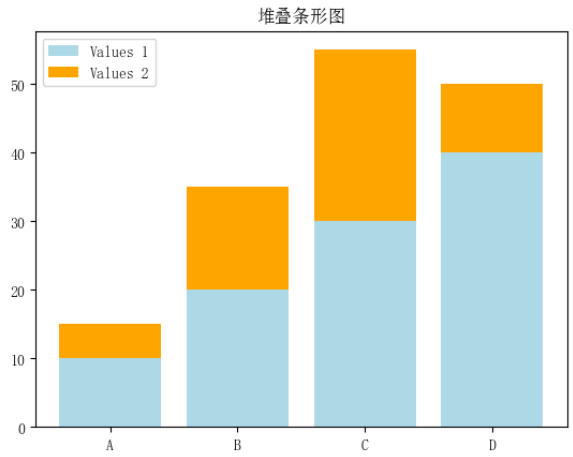

图 29.11 堆叠条形图

3) plt.bar 并列条形图

并列条形图可以通过调整条形的宽度和位置来实现。

```
import matplotlib.pyplot as plt
import numpy as np

# 数据
categories = np.arange(4)
values1 = [10, 20, 30, 40]
values2 = [5, 15, 25, 10]

# 绘制并列条形图
plt.bar(categories - 0.2, values1, width=0.4, color='lightblue',
    label='Values 1')
plt.bar(categories + 0.2, values2, width=0.4, color='orange',
    label='Values 2')
plt.title('并列条形图')
plt.legend()
plt.show()
```

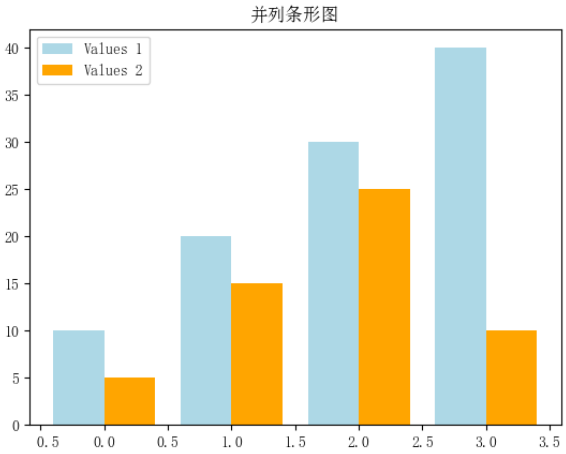

图 29.12 并列条形图

4) plt.barh 水平条形图

`plt.barh()` 用于绘制水平条形图。

```
# 必须导入的库
import matplotlib.pyplot as plt
# barh 功能：绘制水平条形图
plt.barh(y, width, height=0.8, left=None, align='center', **kwargs)
# 参数说明：
# y : 条形的 y 坐标
# width : 条形的宽度
# height : 条形的高度
# left : 条形的左侧位置
# align : 对齐方式
## 'center'：居中
## 'edge'：边缘对齐
```

```
1  import matplotlib.pyplot as plt
2
3  # 数据
4  categories = ['A', 'B', 'C', 'D']
5  values = [10, 24, 36, 12]
6
7  # 绘制水平条形图
8  plt.barh(categories, values, color='skyblue')
```

```
plt.title('水平条形图')
plt.show()
```

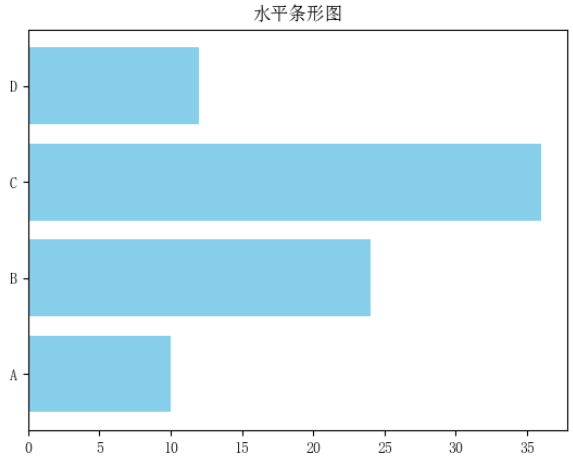

图 29.13 水平条形图

三、 pie 饼图

`plt.pie()` 用于绘制饼图。

```
# 必须导入的库
import matplotlib.pyplot as plt
# pie 功能：绘制饼图
plt.pie(x, explode=None, labels=None, colors=None, autopct=None,
    pctdistance=0.6, **kwargs)
# 参数说明:
# x : 每个饼块的大小
# explode : 是否突出显示饼块
# labels : 饼块的标签
# colors : 饼块的颜色
# autopct : 显示百分比的格式
# pctdistance : 标签距离圆心的比例
```

```
import matplotlib.pyplot as plt

# 数据
sizes = [15, 30, 45, 10]
labels = ['A', 'B', 'C', 'D']
colors = ['gold', 'yellowgreen', 'lightcoral', 'lightskyblue']
```

```
7
8  # 绘制饼图
9  plt.pie(sizes, labels=labels, colors=colors, autopct='%1.1f%%',
       shadow=True, startangle=140)
10 plt.title('饼图示例')
11 plt.axis('equal')  # 保持圆形
12 plt.show()
```

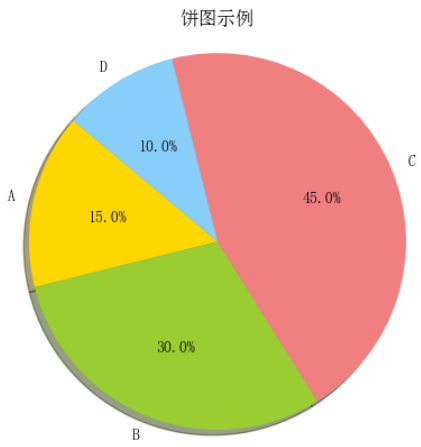

图 29.14 饼图示例

```
1  import matplotlib.pyplot as plt
2  labels = ['English', 'Maths', 'Chemistry']
3  scores = [90, 75, 88]
4  explode = (0, 0, 0.1) # 突出显示Chemistry
5  plt.pie(scores, explode=explode, labels=labels,
6          autopct='%1.1f%%', shadow=True, startangle=60)
7  plt.axis('equal')
8  plt.legend(loc="upper right")
9  plt.title('学科成绩比例(突出显示Chemistry)')
10 plt.show()
```

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3  import pandas as pd
```

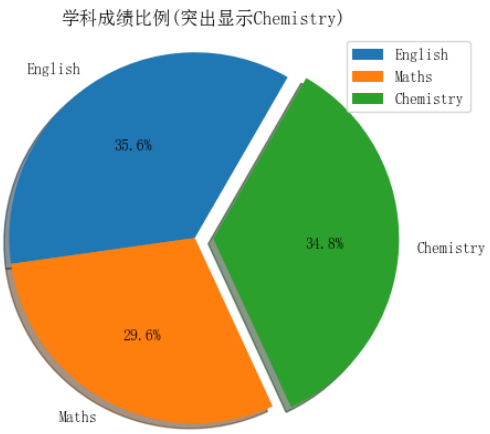

图 29.15 学科成绩比例 (突出显示 Chemistry)

```
4  plt.rcParams['font.sans-serif'] = 'SimHei'  # 设置中文显示
5  plt.rcParams['axes.unicode_minus'] = False
6
7  # 读取数据
8  data = pd.read_csv('../data/people.csv')
9  name = data.columns  # 提取其中的columns字段，视为数据的标签
10 values = data.values  # 提取其中的values字段，数据的存在位置
11 label = ['0-14岁人口', '15-64岁人口', '65岁及以上人口']  # 刻度标签
12 plt.figure(figsize=(6, 6))      #
       将画布设定为正方形，则绘制的饼图是正圆
13
14 # 绘制饼图
15 explode = [0.01, 0.01, 0.01]    # 设定各项离心n个半径
16 plt.pie(values[-1, 2:5], explode=explode, labels=label,
       autopct='%1.1f%%')
17 # values[-1, 2:5] 代表最后一行的第2、3、4列数据
18 # explode=explode 代表各项离心n个半径
19 # labels=label 代表各项标签
20 # autopct='%1.1f%%' 代表数值保留固定小数位
21
22 plt.title('2019年各年龄段年末总人口饼图')  # 添加图表标题
23 plt.savefig('../tmp/2019年各年龄段年末总人口饼图.png')
24 plt.show()
```

四、 boxplot 箱线图

`plt.boxplot()` 用于绘制箱线图。

```
# 必须导入的库
```

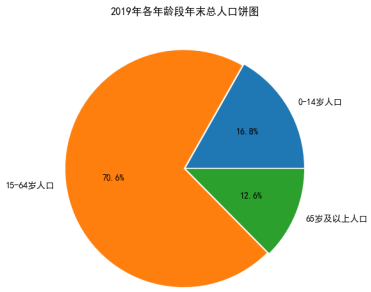

图 29.16 2019 年各年龄段年末总人口饼图

```
import matplotlib.pyplot as plt
# boxplot 功能: 绘制箱线图
plt.boxplot(x, notch=None, vert=None, patch_artist=None, widths=None,
    **kwargs)
# 参数说明:
# x : 数据
# notch : 是否为凹槽形
# vert : 是否垂直显示
# patch_artist : 是否填充箱体
# widths : 箱体的宽度
```

```
import matplotlib.pyplot as plt
import numpy as np

# 数据
np.random.seed(10)
data = [np.random.normal(0, std, 100) for std in range(1, 4)]

# 绘制箱线图
plt.boxplot(data, vert=True, patch_artist=True, labels=['A', 'B',
    'C'])
plt.title('箱线图')
plt.show()
```

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

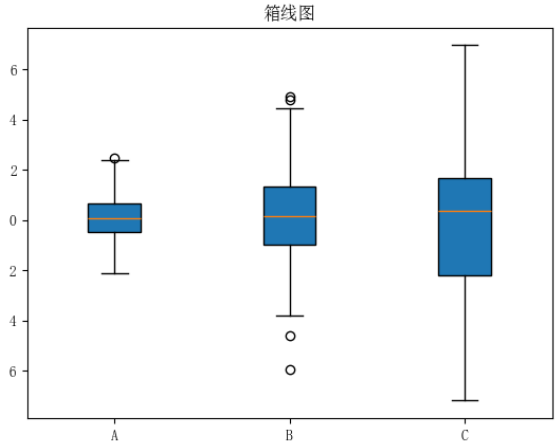

图 29.17 箱型图演示

```
4  plt.rcParams['font.sans-serif'] = 'SimHei'  # 设置中文显示
5  plt.rcParams['axes.unicode_minus'] = False
6  data = pd.read_csv('../data/people.csv')
7  name = data.columns  # 提取其中的columns字段，视为数据的标签
8  values = data.values  # 提取其中的values字段，数据的存在位置
9
10
11 label= ['0-14岁人口', '15-64岁人口', '65岁及以上人口']  # 定义标签
12 gdp = (list(values[:, 2]), list(values[:, 3]), list(values[:, 4]))
13 plt.figure(figsize=(7, 6))
14
15 # 绘制箱线图
16 plt.boxplot(gdp, notch=True, labels=label, meanline=True)
17 # notch=True 代表箱体是否为凹槽形
18 # labels=label 代表各项标签
19 # meanline=True 代表是否显示均值线
20
21 plt.title('2000-2019年各年龄段年末总人口箱线图')
22 plt.savefig('../tmp/2000-2019年各年龄段年末总人口箱线图.png')
23 plt.show()
```

五、 直方图

1) plt.hist 一维频率直方图

```
# 必须导入的库
import matplotlib.pyplot as plt
# hist 功能：绘制一维频率直方图
```

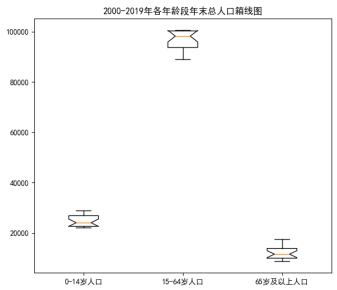

图 29.18 2000-2019 年各年龄段年末总人口箱线图

```
plt.hist(x, bins=None, density=None, **kwargs)
# 参数说明:
# x : 数据
# bins : 分箱的数量或位置
# density : 是否显示概率密度
```

`plt.hist()` 用于绘制一维频率直方图。

```
import matplotlib.pyplot as plt
import numpy as np

# 数据
data = np.random.randn(1000)

# 绘制一维频率直方图
plt.hist(data, bins=30, alpha=0.7, color='blue')
plt.title('一维频率直方图')
plt.show()
```

```
import matplotlib.pyplot as plt
import numpy as np

# 生成随机数据
data = np.random.randn(1000)

# 创建图像和子图
plt.figure(figsize=(8, 4))

# 第一个子图: histtype='step'
```

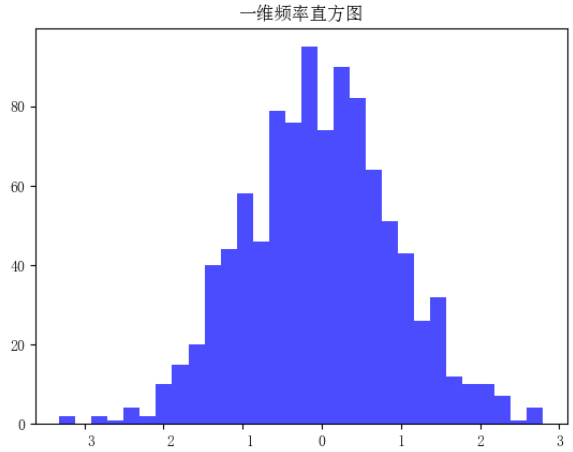

图 29.19 一维频率直方图

```
11 plt.subplot(1, 2, 1)
12 plt.title('step')
13 plt.hist(data, bins=50, density=True, alpha=1,  # density=True
       代替旧的 normed=True
14          histtype='step', color='grey', edgecolor='black')  #
       线条模式，不填充
15
16 # 第二个子图：histtype='stepfilled'
17 plt.subplot(1, 2, 2)
18 plt.title('stepfilled')
19 plt.hist(data, bins=50, density=True, alpha=1,
20          histtype='stepfilled', color='grey', edgecolor='none')
       # 填充的模式
21
22 # 调整布局并显示图像
23 plt.tight_layout()
24 plt.show()
```

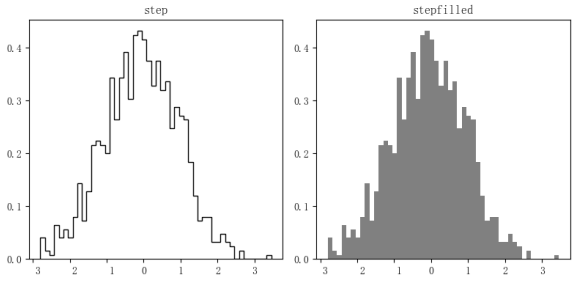

图 29.20 不同 histtype 类型的直方图

stepfilled 与透明性参数 alpha 搭配使用的效果非常好：

```
import matplotlib.pyplot as plt
import numpy as np

# 生成随机数据
x1 = np.random.normal(0, 2, 1000)    # 均值为0, 标准差为2的正态分布
x2 = np.random.normal(-2, 1, 1000)   # 均值为-2, 标准差为1的正态分布
x3 = np.random.normal(2, 2, 1000)    # 均值为2, 标准差为2的正态分布

# 设置直方图的样式参数
kwargs = dict(histtype='stepfilled', alpha=0.5, density=True,
    bins=40)

# 创建图像
plt.figure(figsize=(8, 4))

# 绘制直方图
plt.hist(x1, **kwargs, color='blue', label='x1')  # 第一个数据集
plt.hist(x2, **kwargs, color='orange', label='x2')  # 第二个数据集
plt.hist(x3, **kwargs, color='green', label='x3')  # 第三个数据集

# 添加图例
plt.legend()

# 显示图像
plt.title('不同透明度的频次直方图')
plt.show()
```

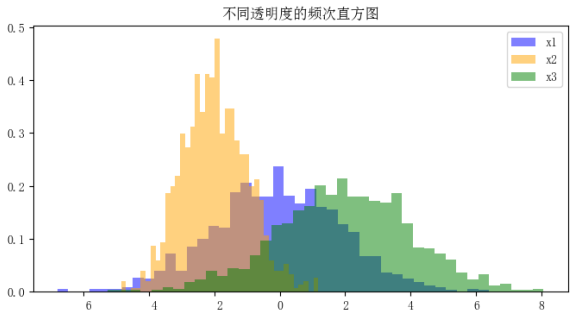

图 29.21 不同透明度的频次直方图

2) plt.hist2d 二维频率直方图

`plt.hist2d()` 用于绘制二维频率直方图。

```
# 必须导入的库
import matplotlib.pyplot as plt
# hist2d 功能：绘制二维频率直方图
plt.hist2d(x, y, bins=10, range=None, density=False, cmap=None, **kwargs)
# 参数说明：
# x : x坐标
# y : y坐标
# bins : 分箱数
# range : x 和 y 的值范围
# density : 是否归一化
# cmap : 颜色映射
```

```
1  import matplotlib.pyplot as plt
2  import numpy as np
3
4  # 数据
5  x = np.random.randn(1000)
6  y = np.random.randn(1000)
7
8  # 绘制二维频率直方图
9  plt.hist2d(x, y, bins=30, cmap='Blues')
10 plt.colorbar()
11 plt.title('二维频率直方图')
12 plt.show()
```

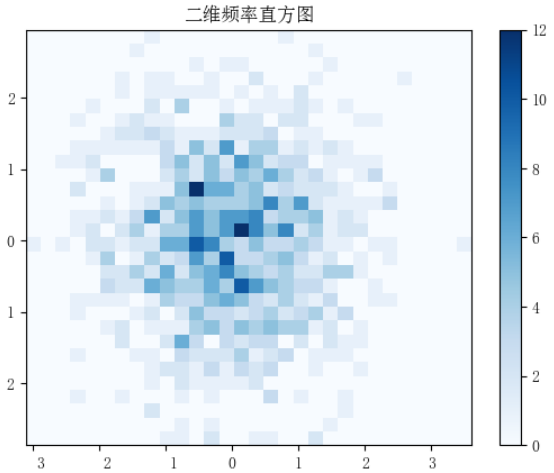

图 29.22 二维频率直方图

3) plt.hexbin 六边形区间划分

二维频次直方图是由与坐标轴正交的方块分割而成的，还有一种常用的方式是用正六边形分割。`plt.hexbin()` 用于绘制六边形区间划分。

```
# 必须导入的库
import matplotlib.pyplot as plt
# hexbin 功能：绘制六边形分箱图
plt.hexbin(x, y, gridsize=30, cmap=None, **kwargs)
# 参数说明：
# x ：x坐标
# y ：y坐标
# gridsize ：网格大小
# cmap ：颜色映射
```

```
import matplotlib.pyplot as plt
import numpy as np

# 数据
x = np.random.randn(10000)
y = np.random.randn(10000)

# 绘制六边形区间划分
plt.hexbin(x, y, gridsize=30, cmap='Greens')
plt.colorbar()
plt.title('六边形区间划分')
plt.show()
```

六、 imshow 热力图

`plt.imshow()` 用于绘制热力图。

```
# 必须导入的库
import matplotlib.pyplot as plt
# imshow 功能：显示图像数据
plt.imshow(X, cmap=None, interpolation=None, **kwargs)
# 参数说明：
# X ：图像数据
# cmap ：颜色映射
# interpolation ：插值方法
## 'nearest'：最近邻插值
## 'bilinear'：双线性插值
## 'bicubic'：双三次插值
```

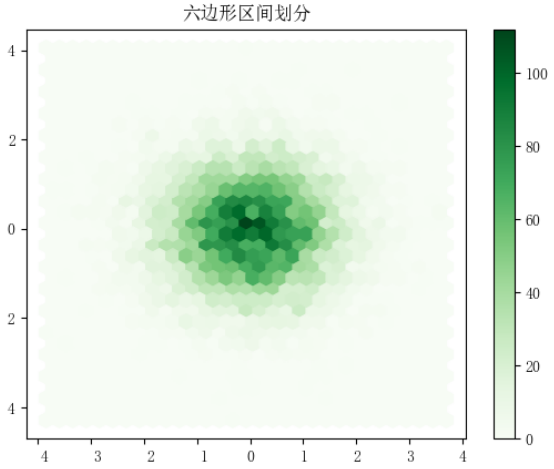

图 29.23 六边形区间划分

```python
import matplotlib.pyplot as plt
import numpy as np

# 数据
data = np.random.rand(10, 10)

# 绘制热力图
plt.imshow(data, cmap='hot', interpolation='nearest')
plt.colorbar()
plt.title('热力图')
plt.show()
```

七、 等高线图

1) plt.contour 等高线图

`plt.contour()` 用于绘制等高线图。

```python
# 必须导入的库
import matplotlib.pyplot as plt
# contour 功能：绘制等高线图
plt.contour(X, Y, Z, levels=None, colors=None, **kwargs)
# 参数说明：
# X, Y : 网格坐标
# Z : 等高线数据
# levels : 等高线的层次
```

```
# colors : 线条颜色
```

```
import matplotlib.pyplot as plt
import numpy as np

# 数据
x = np.linspace(-3.0, 3.0, 100)
y = np.linspace(-3.0, 3.0, 100)
X, Y = np.meshgrid(x, y)
Z = np.exp(-X**2 - Y**2)

# 绘制等高线图
plt.contour(X, Y, Z, levels=10, cmap='viridis') # levels=10
    表示等高线的数量 cmaps='viridis' 表示颜色
plt.title('等高线图')
plt.show()
```

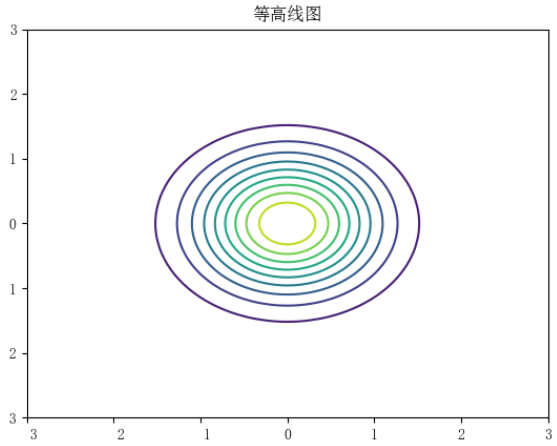

图 29.24 level=10,camaps=viridis 的等高线图

```
import matplotlib.pyplot as plt
import numpy as np

# 定义函数 f(x, y)
def f(x, y):
    return np.sin(x) ** 10 + np.cos(10 + y * x)

```

```
# 设置绘图风格
# plt.style.use('seaborn-v0_8-white')
plt.rcParams['font.family'] = 'SimSun'

# 生成数据
x = np.linspace(0, 5, 50)
y = np.linspace(0, 5, 40)
X, Y = np.meshgrid(x, y)
Z = f(X, Y)

# 绘制等高线图
plt.contour(X, Y, Z, colors='black')

# 显示图像
plt.title('等高线图')
plt.show()
```

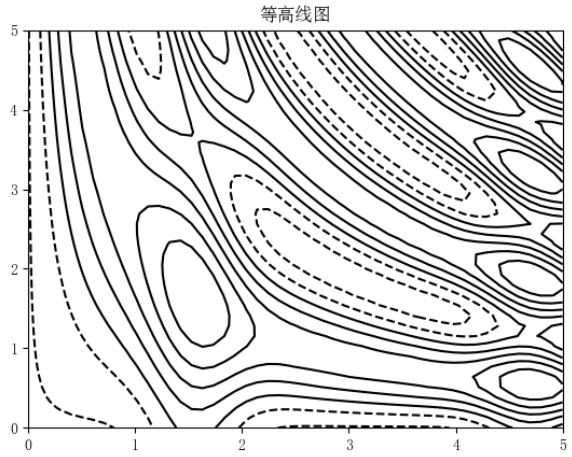

图 29.25 实线和虚线是通过默认的等高线分布自动生成的

实线：通常表示正值等高线的部分，代表函数在这些区域内的高度（或值）大于某一基准值。

虚线：通常表示负值等高线的部分，代表函数在这些区域内的高度（或值）小于某一基准值。

```
import matplotlib.pyplot as plt
import numpy as np

# 定义函数 f(x, y)
def f(x, y):
```

```
    return np.sin(x) ** 10 + np.cos(10 + y * x)

# 生成数据
x = np.linspace(0, 5, 50)
y = np.linspace(0, 5, 40)
X, Y = np.meshgrid(x, y)
Z = f(X, Y)

# 绘制等高线图，并等分为20份，使用copper颜色方案
plt.contour(X, Y, Z, 20, cmap='copper')

# 显示图像
plt.title('使用copper色彩方案的等高线图')
plt.show()
```

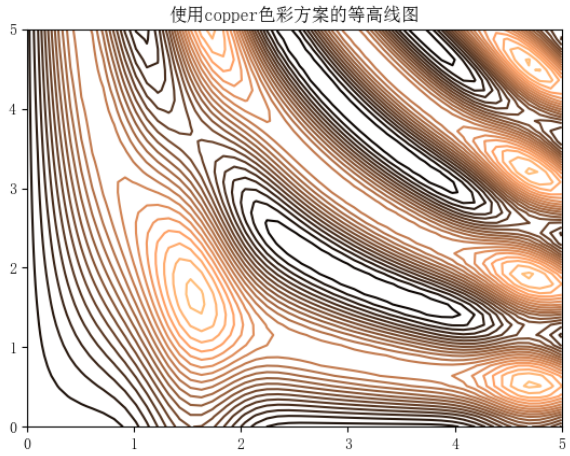

图 29.26 使用 copper 色彩方案的等高线图

2) plt.contourf 带填充色的等高线图

`plt.contourf()` 用于绘制带填充色的等高线图。

```
# 必须导入的库
import matplotlib.pyplot as plt
# contourf 功能：绘制填充的等高线图
plt.contourf(X, Y, Z, levels=None, cmap=None, **kwargs)
# 参数说明：
# X, Y : 网格坐标
# Z : 等高线数据
# levels : 等高线的层次
# cmap : 颜色映射
```

```python
import matplotlib.pyplot as plt
import numpy as np

# 定义函数 f(x, y)
def f(x, y):
    return np.sin(x) ** 10 + np.cos(10 + y * x)

# 生成数据
x = np.linspace(0, 5, 50)
y = np.linspace(0, 5, 40)
X, Y = np.meshgrid(x, y)
Z = f(X, Y)

# 绘制填充的等高线图，并等分为20份，使用copper颜色方案
plt.contourf(X, Y, Z, 20, cmap='copper')

# 添加颜色条
plt.colorbar()

# 显示图像
plt.title('颜色填充的等高线图')
plt.show()
```

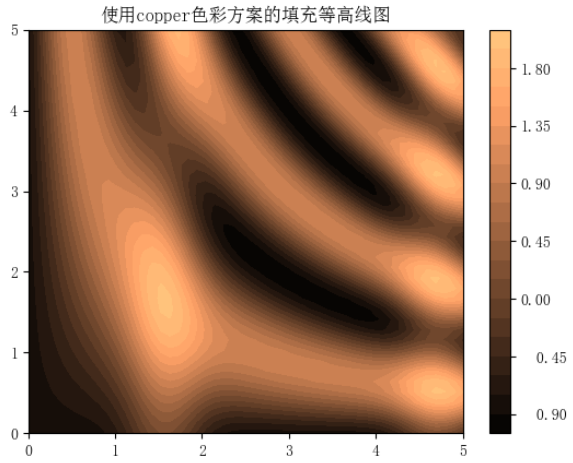

图 29.27 颜色填充的等高线图

```python
import matplotlib.pyplot as plt
```

```python
import numpy as np

# 定义函数 f(x, y)
def f(x, y):
    return np.sin(x) ** 10 + np.cos(10 + y * x)

# 生成数据
x = np.linspace(0, 5, 50)
y = np.linspace(0, 5, 40)
X, Y = np.meshgrid(x, y)
Z = f(X, Y)

# 绘制填充的等高线图，使用 hot 颜色方案，透明度为 0.75
plt.contourf(X, Y, Z, 20, alpha=0.75, cmap='hot')

# 画等高线，设置为黑色，线宽为 0.5
contours = plt.contour(X, Y, Z, 5, colors='black', linewidths=0.5)

# 为等高线添加标签
plt.clabel(contours, inline=True, fontsize=10)

# 添加颜色条
plt.colorbar()

# 显示图像
plt.title('使用 hot 色彩方案并添加标签的等高线图')
plt.show()
```

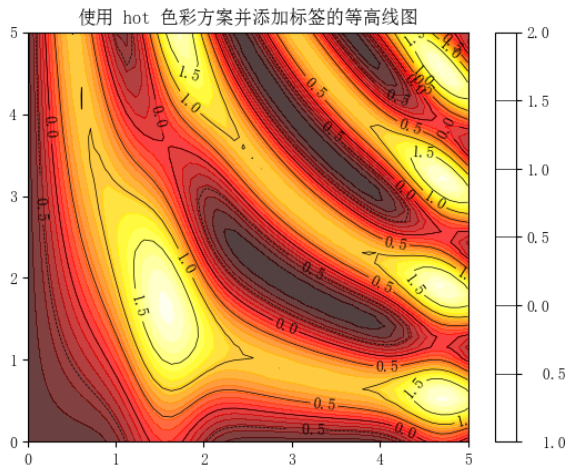

图 29.28 使用 hot 色彩方案并添加标签的等高线图

八、 三维图

Matplotlib 原本只能画 2D 图，后来扩展了 mplot3d 工具箱，它用来画三维图。

```
from mpl_toolkits import mplot3d
# 或
from mpl_toolkits.mplot3d import Axes3D
```

1) plt.plot3D 三维数据点与线

`plt.plot3D()` 用于绘制三维数据点与线。

```
# 必须导入的库
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
# plot3D 功能：绘制三维数据点与线
plt.plot3D(x, y, z, color=None, **kwargs)
# 参数说明：
# x, y, z : 三维坐标
# color : 线条颜色
```

```
1  import matplotlib.pyplot as plt
2  import numpy as np
3  from mpl_toolkits.mplot3d import Axes3D
4
5  # 数据
6  z = np.linspace(0, 1, 100)
7  x = z * np.sin(25 * z)
8  y = z * np.cos(25 * z)
9
10 # 绘制三维数据点与线
11 fig = plt.figure()
12 ax = fig.add_subplot(111, projection='3d')
13 ax.plot3D(x, y, z, 'gray')
14 plt.title(r'三维数据点与线 $x=z \cdot \sin(25z)$, $y=z \cdot
       \cos(25z)$')
15 plt.show()
```

2) plt.contour3D 三维等高线图

`plt.contour3D()` 用于绘制三维等高线图。

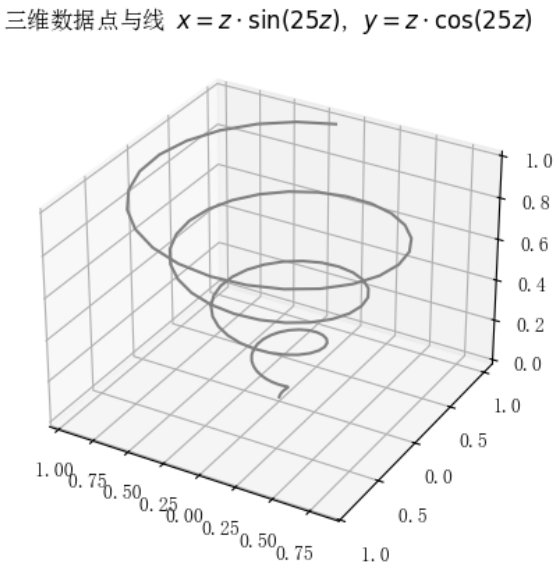

图 29.29 三维数据点与线 $x = z \cdot \sin(25z)$, $y = z \cdot \cos(25z)$

```
# 必须导入的库
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
# contour3D 功能：绘制三维等高线图
plt.contour3D(X, Y, Z, levels=None, cmap=None, **kwargs)
# 参数说明：
# X, Y, Z : 三维网格数据
# levels : 等高线层数
# cmap : 颜色映射
```

```
# contour3D()函数用法
ax.contour3D(X, Y, Z, 50, cmap='binary')
```

```
import matplotlib.pyplot as plt
import numpy as np
from mpl_toolkits.mplot3d import Axes3D

# 数据
x = np.linspace(-3, 3, 100)
y = np.linspace(-3, 3, 100)
X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X**2 + Y**2))

# 绘制三维等高线图
```

```
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.contour3D(X, Y, Z, 50, cmap='binary')
plt.title('三维等高线图')
plt.show()
```

```
import matplotlib.pyplot as plt
import numpy as np
from mpl_toolkits.mplot3d import Axes3D

# 定义数据
x = np.linspace(-6, 6, 50)
y = np.linspace(-6, 6, 50)
X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X**2 + Y**2))

# 定义绘图函数
def draw(ax, X, Y, Z):
    ax.contour3D(X, Y, Z, 40, cmap='hot')
    ax.set_xlabel('x')
    ax.set_ylabel('y')
    ax.set_zlabel('z')

# 创建图形和子图
fig = plt.figure(figsize=(10, 8))

# 第一个子图，默认视角
ax1 = fig.add_subplot(2, 2, 1, projection='3d')
draw(ax1, X, Y, Z)

# 第二个子图，调整俯仰角和方位角
ax2 = fig.add_subplot(2, 2, 2, projection='3d')
draw(ax2, X, Y, Z)
ax2.view_init(60, 35)  # 调整俯仰角为60度，方位角为35度

# 第三个子图，调整为俯视角度
ax3 = fig.add_subplot(2, 2, 3, projection='3d')
draw(ax3, X, Y, Z)
ax3.view_init(-90, 0)  # 调整俯仰角为-90度

# 第四个子图，反向视角
ax4 = fig.add_subplot(2, 2, 4, projection='3d')
draw(ax4, X, Y, Z)
ax4.view_init(-180, 35)  # 反方向，调整为-180度的俯仰角

```

```
40  # 调整布局并显示图像
41  plt.tight_layout()
42  plt.show()
```

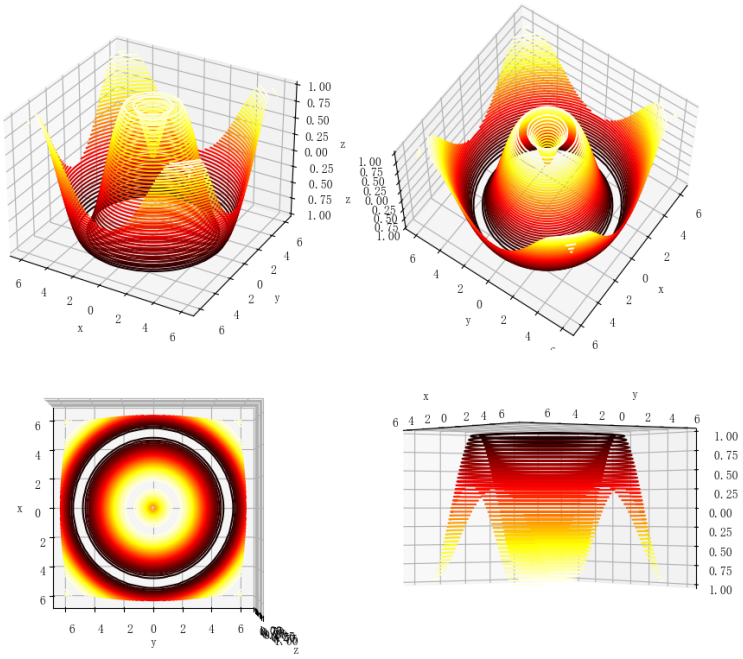

图 29.30 3D 等高线不同视图

3) plt.plot_wireframe 线框图

`plt.plot_wireframe()` 用于绘制线框图。

```
# 必须导入的库
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
# plot_wireframe 功能：绘制线框图
plt.plot_wireframe(X, Y, Z, rstride=1, cstride=1, **kwargs)
# 参数说明：
# X, Y, Z : 网格数据
# rstride : 行步长
# cstride : 列步长
```

```
1  import matplotlib.pyplot as plt
```

```python
import numpy as np
from mpl_toolkits.mplot3d import Axes3D

# 数据
x = np.linspace(-5, 5, 50)
y = np.linspace(-5, 5, 50)
X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X**2 + Y**2))

# 绘制线框图
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.plot_wireframe(X, Y, Z, color='black')
plt.title('线框图')
plt.show()
```

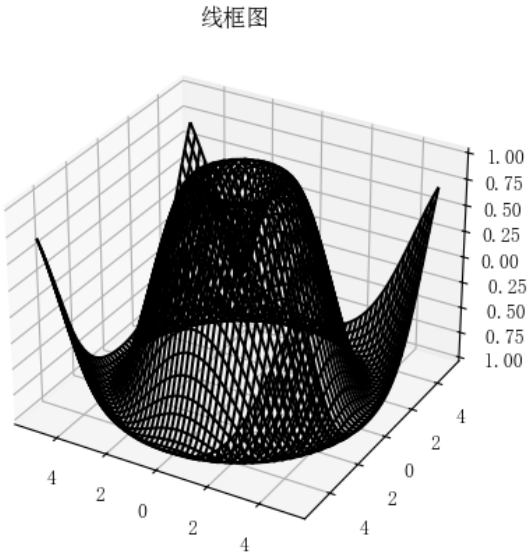

图 29.31 线框图演示

4) plt.plot_surface 曲面图

`plt.plot_surface()` 用于绘制曲面图。

```python
# 必须导入的库
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
# plot_surface 功能：绘制曲面图
plt.plot_surface(X, Y, Z, cmap=None, **kwargs)
# 参数说明：
```

```
# X, Y, Z : 网格数据
# cmap : 颜色映射
```

```python
import matplotlib.pyplot as plt
import numpy as np
from mpl_toolkits.mplot3d import Axes3D

# 数据
x = np.linspace(-5, 5, 50)
y = np.linspace(-5, 5, 50)
X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X**2 + Y**2))

# 绘制曲面图
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(X, Y, Z, cmap='viridis')
plt.title('曲面图')
plt.show()
```

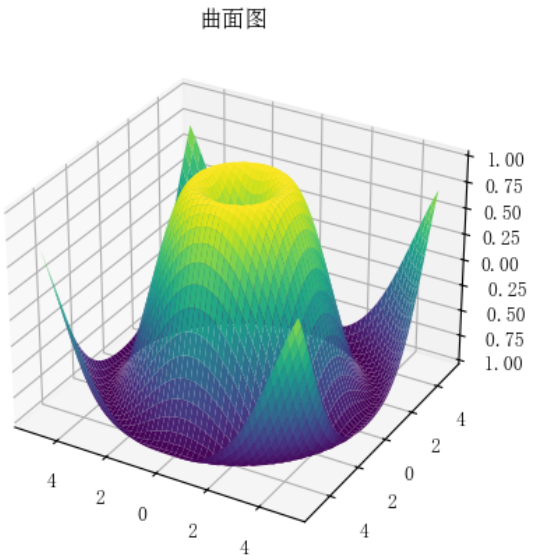

图 29.32 曲面图演示

例子 29.1

绘制函数 $f(x,y) = (x^2+y^2)\sin\left(\frac{1}{x^2+y^2}\right)$ 的三维曲面图。

方法:

```python
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# 定义函数 f(x, y)
def f(x, y):
    with np.errstate(divide='ignore', invalid='ignore'):
        z = (x**2 + y**2) * np.sin(1 / (x**2 + y**2))
        z[np.isnan(z)] = 0  # 处理 x^2 + y^2 = 0 的情况, 避免除以零
        z[np.isinf(z)] = 0  # 处理无穷大的情况
    return z

# 创建 x 和 y 的网格数据
x = np.linspace(-1, 1, 300)
y = np.linspace(-1, 1, 300)
X, Y = np.meshgrid(x, y)

# 计算函数值
Z = f(X, Y)

# 创建三维图像
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

# 绘制三维曲面图
surf = ax.plot_surface(X, Y, Z, cmap='viridis')

# 添加颜色条
fig.colorbar(surf, ax=ax, shrink=0.5, aspect=5)

# 设置标题和标签
ax.set_title(r'$f(x, y) = (x^2 + y^2)\sin\left(\frac{1}{x^2 + y^2}\right)$')
ax.set_xlabel('X axis')
ax.set_ylabel('Y axis')
ax.set_zlabel('Z axis')

# 显示图像
plt.show()
```

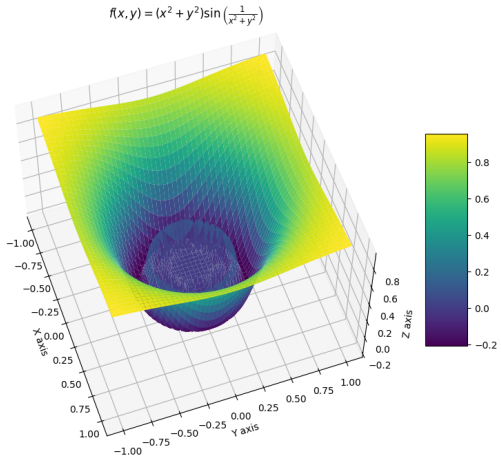

图 29.33 曲面图 $f(x,y)=(x^2+y^2)\sin\left(\frac{1}{x^2+y^2}\right)$

(1) plt.plot_surface 极坐标曲面图

极坐标曲面图的绘制可以通过曲面的极坐标变换实现

```python
import matplotlib.pyplot as plt
import numpy as np
from mpl_toolkits.mplot3d import Axes3D

# 数据
r = np.linspace(0, 1, 50)
p = np.linspace(0, 2*np.pi, 50)
R, P = np.meshgrid(r, p)
Z = R**2

# 极坐标转化为笛卡尔坐标
X = R * np.cos(P)
Y = R * np.sin(P)

# 绘制极坐标曲面图
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(X, Y, Z, cmap='viridis')
plt.title('极坐标曲面图')
plt.show()
```

(2) plt.plot_trisurf 曲面三角剖分

`plt.plot_trisurf()` 用于绘制曲面三角剖分图。

```
# 必须导入的库
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
# plot_trisurf 功能：绘制曲面三角剖分
plt.plot_trisurf(x, y, z, cmap=None, **kwargs)
# 参数说明：
# x, y, z : 三维坐标
# cmap : 颜色映射
```

```
1  import matplotlib.pyplot as plt
2  import numpy as np
3  from mpl_toolkits.mplot3d import Axes3D
4
5  # 数据
6  x = np.random.rand(100) * 10
7  y = np.random.rand(100) * 10
8  z = np.sin(np.sqrt(x**2 + y**2))
9
10 # 绘制曲面三角剖分图
11 fig = plt.figure()
12 ax = fig.add_subplot(111, projection='3d')
13 ax.plot_trisurf(x, y, z, cmap='viridis', edgecolor='none')
14 plt.title('曲面三角剖分')
15 plt.show()
```

第三十章 [matplotlib] 图片样式与排版

一、 默认配置

matplotlib 的相关配置主要包括以下几种，用户可以自定义它们：

1) 图片大小 (`figsize`) 和分辨率 (`dpi`)

2) 线宽 (`linewidth`)、颜色 (`color`)、风格 (`linestyle`) 和标记 (`marker`)

3) 坐标轴 (`axis`)、坐标轴标签 (`xlabel, ylabel`) 以及网格 (`grid`) 的属性

4) 文字 (`text`) 与字体 (`font`) 属性。

所有的默认属性均保存在 `matplotlib.rcParams` 字典中。

```
import matplotlib.pyplot as plt

# 查看所有的 rcParams 及其默认值
for key, value in plt.rcParams.items():
    print(f"{key}: {value}")
```

可以对 `plt.rcParams` 进行修改，以**全局**改变默认的配置。例如，修改图片的尺寸和分辨率：

```
import matplotlib.pyplot as plt

# 查看特定的 rcParams 值
print(plt.rcParams['figure.figsize'])  # 图像尺寸
print(plt.rcParams['figure.dpi'])      # 图像分辨率
print(plt.rcParams['lines.linewidth']) # 线条宽度

```

```
# 修改默认的图像尺寸和分辨率
plt.rcParams['figure.figsize'] = [10, 6]
plt.rcParams['figure.dpi'] = 120
print(plt.rcParams['figure.figsize'])  # 图像尺寸
print(plt.rcParams['figure.dpi'])      # 图像分辨率

# 绘制一个简单的图形，使用新的设置
plt.plot([1, 2, 3, 4], [1, 4, 2, 3])
plt.show()
```

如果不想全局修改默认配置，可以使用 `plt.rc()` 函数来临时修改配置：

```
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 6), dpi=120)
plt.plot([1, 2, 3, 4], [1, 4, 2, 3])
plt.show()
```

二、 显示中文

```
# 1.定位matplotlib的字体库路径
import matplotlib
matplotlib.matplotlib_fname()
>>>
    '/home/changjie/.local/conda_env/pytorch_test/lib/python3.10/site-packages/matplotlib/

# 2.将中文字体文件**.ttf，拷贝到该目录的fonts/ttf下
cp /usr/share/fonts/emacs_cnfonts/chinese/*
    /home/changjie/.local/conda_env/pytorch_test/lib/python3.10/site-packages/matplotlib/m

# 3删除 matplotlib 缓存
cd ~/.cache/matplotlib
rm * -r

# 4.查看是否成功
import matplotlib.pyplot as plt
from matplotlib import font_manager
# 获取字体管理器并列出所有TTF字体
fonts = [f.name for f in font_manager.fontManager.ttflist]
sorted_fonts = sorted(fonts)
# 打印排序后的字体列表
print(sorted_fonts)

# 5.使用字体
```

```
## 1)用plt.rcParams设置
plt.rcParams['font.family'] = 'SimSun'
plt.rcParams['axes.unicode_minus'] = False  #
    解决保存图像时负号'-'显示为方块的问题

## 2)用FontProperties, 并指定字体文件
from matplotlib.font_manager import FontProperties
import matplotlib.pyplot as plt
font = FontProperties(fname='/mnt/share/code/font/SimHei.ttf', size=20)
plt.title('测试', fontproperties=font)
```

三、 图片的属性

参数	默认值	描述
num	1	图像的数量
figsize	figure.figsize	图像的长和宽（英寸）
dpi	figure.dpi	分辨率（像素/英寸）
facecolor	figure.facecolor	绘图区域的背景颜色
edgecolor	figure.edgecolor	绘图区域边缘的颜色
frameon	True	是否绘制图像边缘

表 30.1 图像的属性

高 dpi 显示图像更细腻，但是图像尺寸也会变大。使用默认值即可。如果图像非常复杂，为了看清细节，我们可以调整宽高的英寸数。

```
1  import matplotlib.pyplot as plt
2  plt.rcParams['font.family'] = 'SimSun'
3
4  # 示例 1: 默认参数
5  plt.figure(num=1)
6  plt.plot([0, 1, 2], [0, 1, 4], label='默认参数')
7  plt.title('默认参数')
8  plt.legend()
9  plt.show()
10
11 # 示例 2: 设置图像大小 (figsize)
12 plt.figure(num=2, figsize=(10, 5))
13 plt.plot([0, 1, 2], [0, 1, 4], label='figsize = (10, 5)')
14 plt.title('设置图像大小')
15 plt.legend()
16 plt.show()
17
18 # 示例 3: 设置分辨率 (dpi)
19 plt.figure(num=3, dpi=150)
```

```
20 plt.plot([0, 1, 2], [0, 1, 4], label='dpi = 150')
21 plt.title('设置分辨率')
22 plt.legend()
23 plt.show()
24
25 # 示例 4: 设置背景颜色 (facecolor)
26 plt.figure(num=4, facecolor='lightgray')
27 plt.plot([0, 1, 2], [0, 1, 4], label='facecolor = lightgray')
28 plt.title('设置背景颜色')
29 plt.legend()
30 plt.show()
31
32 # 示例 5: 设置边缘颜色 (edgecolor)
33 plt.figure(num=5, edgecolor='red')
34 plt.plot([0, 1, 2], [0, 1, 4], label='edgecolor = red')
35 plt.title('设置边缘颜色')
36 plt.legend()
37 plt.show()
38
39 # 示例 6: 关闭边框 (frameon=False)
40 plt.figure(num=6, frameon=False)
41 plt.plot([0, 1, 2], [0, 1, 4], label='frameon = False')
42 plt.title('关闭边框')
43 plt.legend()
44 plt.show()
```

四、 matplotlib 线条

1) 线条的颜色

matplotlib 支持使用以下缩写字符设置线条的颜色：

表 30.2 线条颜色

缩写字符	颜色
'b'	blue
'g'	green
'r'	red
'c'	cyan
'm'	magenta
'y'	yellow
'k'	black
'w'	white

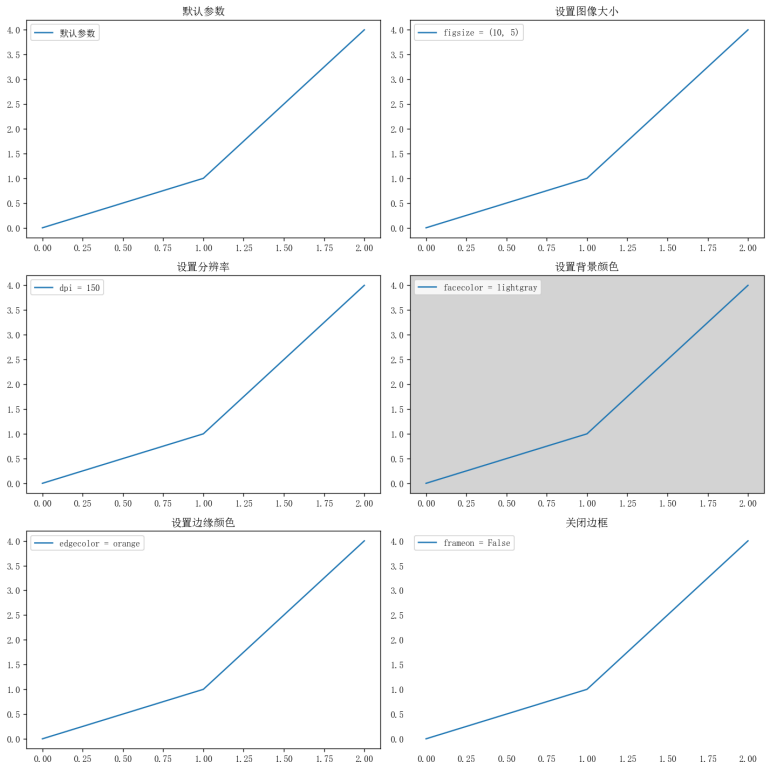
默认参数
默认参数
设置图像大小
figsize = (10, 5)
设置分辨率
dpi = 150
设置背景颜色
facecolor = lightgray
设置边缘颜色
edgecolor = orange
关闭边框
frameon = False

图 30.1 修改图片参数

```python
import matplotlib.pyplot as plt

# 颜色列表
colors = ['b', 'g', 'r', 'c', 'm', 'y', 'k', 'w']
labels = ['blue', 'green', 'red', 'cyan', 'magenta', 'yellow',
    'black', 'white']

plt.figure(figsize=(10, 6))
for i, color in enumerate(colors):
    plt.plot([0, 1, 2], [i, i + 1, i + 2], color=color,
    label=labels[i])

plt.title('线条的颜色')
plt.legend(loc='upper left')
plt.show()
```

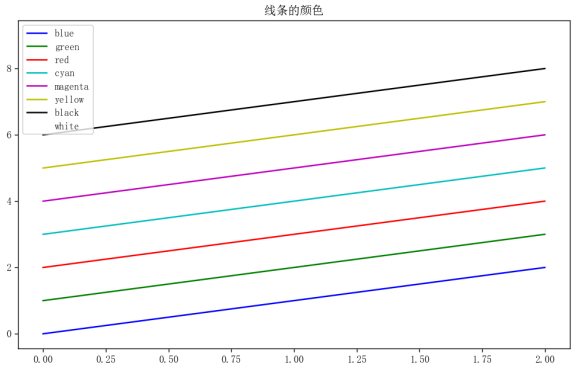

图 30.2 线条的颜色

2) 线条的粗细

线条的粗细可以通过 `linewidth` 或 `lw` 参数指定，取值为浮点数，表示线条的宽度点数。以下是不同 `linewidth` 设置的示例：

```python
import matplotlib.pyplot as plt

# 线条粗细列表
linewidths = [0.5, 1.0, 1.5, 2.0, 2.5, 3.0]

plt.figure(figsize=(10, 6))
```

```
for i, lw in enumerate(linewidths):
    # 这里不指定颜色就是使用默认的颜色序列来绘制线条。
    plt.plot([0, 1, 2], [i, i + 1, i + 2], linewidth=lw,
    label=f'linewidth = {lw}')

plt.title('线条的粗细')
plt.legend(loc='upper left')
plt.show()
```

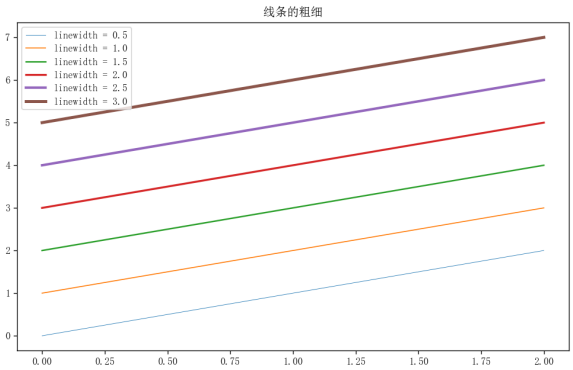

图 30.3 线条的粗细

3) 线条的样式

线条样式可以使用 `linestyle` 或 `ls` 参数来指定。支持的样式有以下几种：

表 30.3 线条样式

样式缩写	描述
`'-'`	实线
`'--'`	短划线
`':'`	点划线
`'-.'`	虚线

```
import matplotlib.pyplot as plt

# 线条样式列表
linestyles = ['-', '--', ':', '-.']
labels = ['实线', '短划线', '点划线', '虚线']

```

```
plt.figure(figsize=(10, 6))
for i, ls in enumerate(linestyles):
    plt.plot([0, 1, 2], [i, i + 1, i + 2], linestyle=ls,
    label=labels[i])

plt.title('线条的样式')
plt.legend(loc='upper left')
plt.show()
```

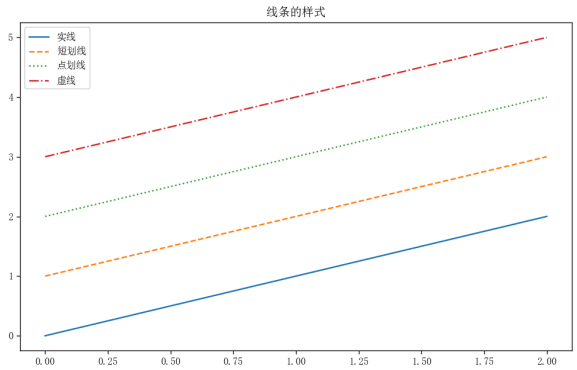

图 30.4 线条的样式

4) 线条的标记

线条标记是用来在图上标注数据点的符号，可以使用 `marker` 参数指定。以下是常用的标记样式：

```
import matplotlib.pyplot as plt

# 标记样式列表
markers = ['.', ',', 'o', 'v', '^', '<', '>', '1', '2', '3', '4',
    's', 'p', '*', 'h', 'H', '+', 'x', 'D', 'd', '|', '_']
labels = [
    'point', 'pixel', 'circle', 'triangle_down', 'triangle_up',
    'triangle_left', 'triangle_right',
    'tri_down', 'tri_up', 'tri_left', 'tri_right', 'square',
    'pentagon', 'star', 'hexagon1',
    'hexagon2', 'plus', 'x', 'diamond', 'thin_diamond', 'vline',
    'hline'
]

plt.figure(figsize=(12, 8))
```

表 30.4 线条标记

标记缩写	描述
'.'	点 (point marker)
','	像素 (pixel marker)
'o'	圆圈 (circle marker)
'v'	向下三角形 (triangle_down marker)
'^'	向上三角形 (triangle_up marker)
'<'	向左三角形 (triangle_left marker)
'>'	向右三角形 (triangle_right marker)
'1'	下三角 (tri_down marker)
'2'	上三角 (tri_up marker)
'3'	左三角 (tri_left marker)
'4'	右三角 (tri_right marker)
's'	正方形 (square marker)
'p'	五边形 (pentagon marker)
'*'	星形 (star marker)
'h'	六边形 1 (hexagon1 marker)
'H'	六边形 2 (hexagon2 marker)
'+'	加号 (plus marker)
'x'	叉号 (x marker)
'D'	菱形 (diamond marker)
'd'	细菱形 (thin_diamond marker)
'|'	竖线 (vline marker)
'_'	横线 (hline marker)

```
12 for i, marker in enumerate(markers):
13     plt.plot([0, 1, 2], [i, i + 1, i + 2], marker=marker,
       label=labels[i])
14
15 plt.title('线条的标记')
16 plt.legend(loc='upper left', fontsize='small')
17 plt.show()
```

```
1 import matplotlib.pyplot as plt
2
3 # 定义标记样式和方向属性
4 markers = ['o', 'v', '^', '<', '>', '8', 's', 'p', '*', 'h', 'H',
      'D', 'd', 'P', 'X']
5 fillstyles = ['full', 'left', 'right', 'bottom', 'top', 'none']
      # 标记的填充样式
```

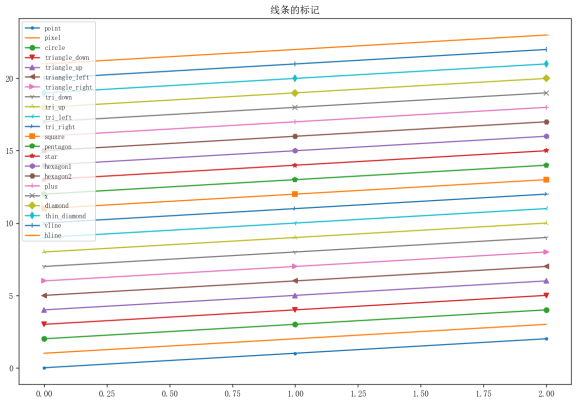

图 30.5 线条的标记

```
6
7  # 设置图像大小和子图网格
8  fig, axs = plt.subplots(len(markers), len(fillstyles),
       figsize=(12, 18))
9
10 # 循环绘制每种标记和填充样式
11 for i, marker in enumerate(markers):
12     for j, fillstyle in enumerate(fillstyles):
13         axs[i, j].plot(0, 0, marker=marker, markersize=20,
       markerfacecolor='orange', fillstyle=fillstyle)
14         axs[i, j].set_xticks([])
15         axs[i, j].set_yticks([])
16         axs[i, j].set_xlim(-1, 1)
17         axs[i, j].set_ylim(-1, 1)
18         axs[i, j].text(0, 0, f"('{marker}',{fillstyle})",
       ha='center', va='center')
19
20 plt.tight_layout()
21 plt.show()
```

五、 图片的边界

图片的边界可以通过 `plt.xlim()` 和 `plt.ylim()` 函数来设置。以下示例展示了如何设置图片的边界：

```
1  import matplotlib.pyplot as plt
2  import numpy as np
3
```

```
# 创建数据
x = np.linspace(-np.pi, np.pi, 100)
y1 = np.sin(x)
y2 = np.cos(x)

# 创建两个子图用于比较
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 6))

# 第一个子图：默认上下限
ax1.plot(x, y1, color='blue', label='sin(x)')
ax1.plot(x, y2, color='orange', label='cos(x)')
ax1.set_xlim(-np.pi, np.pi)  # 设置 x 轴的上下限
ax1.set_ylim(-1.0, 1.0)      # 设置 y 轴的上下限
ax1.set_title('原始图像')
ax1.legend()

# 第二个子图：扩展 y 轴的上下限 10%
ax2.plot(x, y1, color='blue', label='sin(x)')
ax2.plot(x, y2, color='orange', label='cos(x)')
ax2.set_xlim(-np.pi, np.pi)  # 设置 x 轴的上下限
ax2.set_ylim(-1.1, 1.1)      # 扩展 y 轴的上下限 10%
ax2.set_title('扩展 y 轴上下 10%')
ax2.legend()

# 调整布局并显示图像
plt.tight_layout()
plt.show()
```

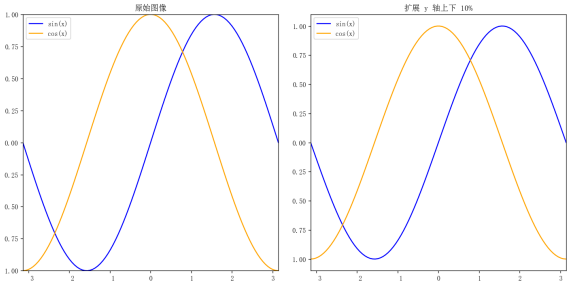

图 30.6 图片的边界

六、 坐标轴

1) 坐标记号标签

坐标记号标签通过 `plt.xticks()` 和 `plt.yticks()` 函数来设置。其主要功能包括：

1) 设置刻度位置：指定 x 轴上显示刻度的位置。
2) 设置刻度标签：自定义刻度位置的显示标签，可以使用字符串、数学符号等。
3) 美化图表：通过设置刻度和标签，可以让图表看起来更加专业、易于理解。

```
import matplotlib.pyplot as plt
import numpy as np

# 数据
x = np.linspace(-np.pi, np.pi, 100)
y = np.sin(x)

plt.plot(x, y)

# 设置 x 轴的刻度和标签
plt.xticks([-np.pi, -np.pi/2, 0, np.pi/2, np.pi], ['-', '-/2',
    '0', '/2', ''])
## 第1个参数是刻度的位置，第2个参数是刻度的标签

# 设置 y 轴的刻度和标签
plt.yticks([-1, -0.5, 0, 0.5, 1], ['-1', '-0.5', '0', '0.5', '1'])

plt.title('使用 xticks 和 yticks 设置刻度')
plt.show()
```

```
import matplotlib.pyplot as plt
import numpy as np

# 创建数据
x = np.linspace(-np.pi, np.pi, 100)
y1 = np.sin(x)
y2 = np.cos(x)

# 创建两个子图用于比较
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 6))
```

```
11
12 # 第一个子图：不使用 xticks 和 yticks
13 ax1.plot(x, y1, color='blue', label='sin(x)')
14 ax1.plot(x, y2, color='orange', label='cos(x)')
15 ax1.set_title('未设置 xticks 和 yticks')
16 ax1.legend()
17
18 # 第二个子图：使用 xticks 和 yticks
19 ax2.plot(x, y1, color='blue', label='sin(x)')
20 ax2.plot(x, y2, color='orange', label='cos(x)')
21 ax2.set_xticks([-np.pi, -np.pi/2, 0, np.pi/2, np.pi])  # 设置 x
       轴刻度
22 ax2.set_xticklabels(['-', '-/2', '0', '/2', ''])  # 设置 x
       轴刻度标签
23 ax2.set_yticks([-1, -0.5, 0, 0.5, 1])  # 设置 y 轴刻度
24 ax2.set_yticklabels(['-1', '-0.5', '0', '0.5', '1'])  # 设置 y
       轴刻度标签
25 ax2.set_title('设置 xticks 和 yticks')
26 ax2.legend()
27
28 # 调整布局并显示图像
29 plt.tight_layout()
30 plt.show()
```

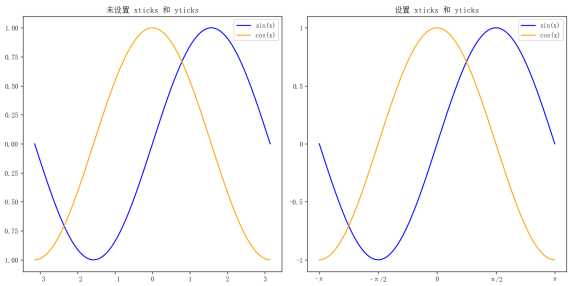

图 30.7 ticks 设置坐标

2) 坐标轴的标签

坐标轴的标签可以通过 `plt.xlabel()` 和 `plt.ylabel()` 函数来设置。以下示例展示了如何设置坐标轴的标签：

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
```

```python
# 创建数据
x = np.linspace(-np.pi, np.pi, 100)
y1 = np.sin(x)
y2 = np.cos(x)

# 创建两个子图用于比较
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 6))

# 第一个子图：不使用 xlabel 和 ylabel
ax1.plot(x, y1, color='blue', label='sin(x)')
ax1.plot(x, y2, color='orange', label='cos(x)')
ax1.set_xlim(-np.pi, np.pi)  # 设置 x 轴的上下限
ax1.set_ylim(-1.0, 1.0)      # 设置 y 轴的上下限
ax1.set_title('未设置 xlabel 和 ylabel')
ax1.legend()

# 第二个子图：使用 xlabel 和 ylabel
ax2.plot(x, y1, color='blue', label='sin(x)')
ax2.plot(x, y2, color='orange', label='cos(x)')
ax2.set_xlim(-np.pi, np.pi)  # 设置 x 轴的上下限
ax2.set_ylim(-1.0, 1.0)      # 设置 y 轴的上下限
ax2.set_xlabel('X 轴 (radians)')  # 设置 x 轴标签
ax2.set_ylabel('Y 轴 (value)')    # 设置 y 轴标签
ax2.set_title('设置 xlabel 和 ylabel')
ax2.legend()

# 调整布局并显示图像
plt.tight_layout()
plt.show()
```

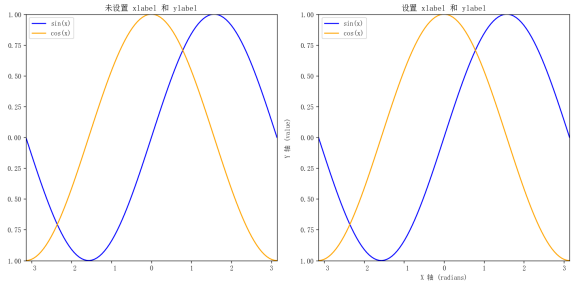

图 30.8 label 标签的设置

3) 坐标轴移动

坐标轴线和上面的记号连在一起就形成了脊柱（Spines，一条线段上有一系列的凸起，很像脊柱骨），它记录了数据区域的范围。

默认是放到图的四边，matplotlib 中的坐标轴线和记号线可以通过 `spines` 属性进行设置。以下示例展示了如何将坐标轴移到图的中间：

```
import matplotlib.pyplot as plt
import numpy as np

# 创建数据
x = np.linspace(-np.pi, np.pi, 100)
y1 = np.sin(x)
y2 = np.cos(x)

# 创建两个子图用于比较
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 6))

# 第一个子图：默认坐标轴在图的四边
ax1.plot(x, y1, color='blue', label='sin(x)')
ax1.plot(x, y2, color='orange', label='cos(x)')
ax1.set_title('默认坐标轴位置')
ax1.legend()

# 第二个子图：将坐标轴移到图的中间
ax2.plot(x, y1, color='blue', label='sin(x)')
ax2.plot(x, y2, color='orange', label='cos(x)')
ax2.spines['top'].set_color('none')  # 隐藏上边框
ax2.spines['right'].set_color('none')  # 隐藏右边框
ax2.spines['left'].set_position(('data', 0))  # 将左侧的 y 轴移到 x = 0 位置
ax2.spines['bottom'].set_position(('data', 0))  # 将底部的 x 轴移到 y = 0 位置
ax2.set_title('坐标轴移到图的中间')
ax2.legend()

# 调整布局并显示图像
plt.tight_layout()
plt.show()
```

4) 添加图例

图例是用来区分不同数据的标签，可以通过 `plt.legend()` 函数来添加。以下示例展示了如何添加图例：

```
import matplotlib.pyplot as plt
```

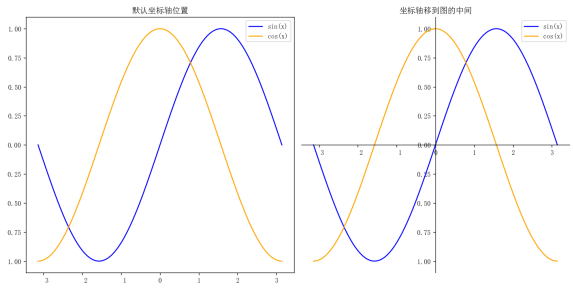

图 30.9 spines 移动坐标轴

```
import numpy as np

# 创建数据
x = np.linspace(-np.pi, np.pi, 100)
y1 = np.sin(x)
y2 = np.cos(x)

# 创建两个子图用于比较
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 6))

# 第一个子图：不使用图例
ax1.plot(x, y1, color='orange', linewidth=1.0, linestyle='-',
    label='sin(x)')
ax1.plot(x, y2, color='blue', linewidth=1.0, linestyle='-',
    label='cos(x)')
ax1.set_title('未添加图例')
# 没有使用 ax1.legend()，因此没有图例

# 第二个子图：使用图例
ax2.plot(x, y1, color='orange', linewidth=1.0, linestyle='-',
    label='sin(x)')
ax2.plot(x, y2, color='blue', linewidth=1.0, linestyle='-',
    label='cos(x)')
ax2.set_title('添加图例')
ax2.legend(loc='upper left', fontsize='large')  #
    使用图例，放在左上角，字体较大

# 调整布局并显示图像
plt.tight_layout()
plt.show()
```

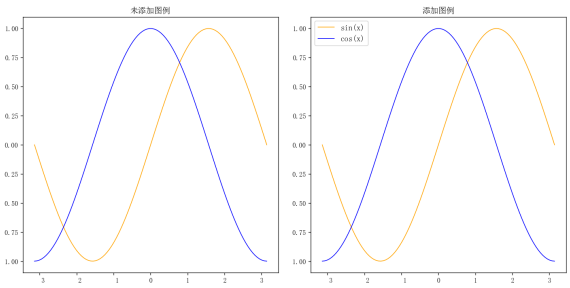

图 30.10 legend 添加图例

5) 坐标轴的网格

坐标轴的网格可以通过 `plt.grid()` 函数来设置。以下示例展示了如何添加坐标轴的网格：

```
import matplotlib.pyplot as plt
import numpy as np

# 创建数据
x = np.linspace(-np.pi, np.pi, 100)
y1 = np.sin(x)
y2 = np.cos(x)

# 创建两个子图用于比较
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 6))

# 第一个子图：不使用网格
ax1.plot(x, y1, color='orange', linewidth=1.0, linestyle='-',
    label='sin(x)')
ax1.plot(x, y2, color='blue', linewidth=1.0, linestyle='-',
    label='cos(x)')
ax1.set_title('未使用网格')
ax1.legend(loc='upper left')

# 第二个子图：使用网格
ax2.plot(x, y1, color='orange', linewidth=1.0, linestyle='-',
    label='sin(x)')
ax2.plot(x, y2, color='blue', linewidth=1.0, linestyle='-',
    label='cos(x)')
ax2.set_title('使用网格')
ax2.legend(loc='upper left')
ax2.grid(True)  # 开启网格

# 调整布局并显示图像
plt.tight_layout()
plt.show()
```

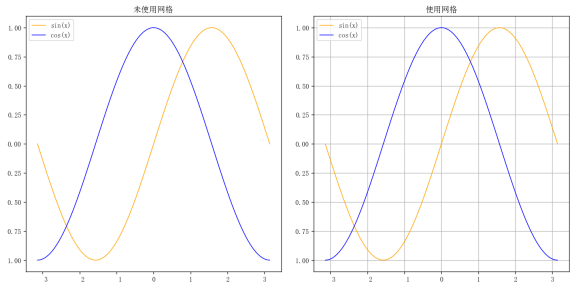

图 30.11 grid 添加网格

6) 特殊点的注释

特殊点的注释可以通过 `plt.annotate()` 函数来设置。`plt.annotate()` 函数的主要参数包括：

1) `s`：注释的内容。

2) `xy`：注释的位置。

3) `xytext`：注释的文本位置。

4) `arrowprops`：箭头的属性。

```
import matplotlib.pyplot as plt
import numpy as np

# 创建数据
x = np.linspace(-np.pi, np.pi, 100)
y1 = np.sin(x)
y2 = np.cos(x)
t = 2 * np.pi / 3  # 特殊点的位置
t2 = -2 * np.pi / 3  # 特殊点的位置

# 创建两个子图用于比较
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

# 第一个子图：不添加注释
ax1.plot(x, y1, color='orange', linewidth=1.5, linestyle='-',
    label='sin(x)')
```

```
16 ax1.plot(x, y2, color='blue', linewidth=1.5, linestyle='-',
       label='cos(x)')
17 ax2.scatter([t2], [np.cos(t2)], 30, color='blue') # 添加特殊点
18 ax1.scatter([t], [np.sin(t)], 30, color='orange') # 添加特殊点
19 ax1.set_title('未添加注释')
20 ax1.axhline(0, color='black', linewidth=0.5)
21 ax1.axvline(0, color='black', linewidth=0.5)
22 ax1.legend(loc='upper left')
23
24 # 第二个子图：添加注释
25 ax2.plot(x, y1, color='orange', linewidth=1.5, linestyle='-',
       label='sin(x)')
26 ax2.plot(x, y2, color='blue', linewidth=1.5, linestyle='-',
       label='cos(x)')
27 ax2.scatter([t2], [np.cos(t2)], 30, color='blue')
28 ax2.scatter([t], [np.sin(t)], 30, color='orange')
29 ax2.set_title('添加注释')
30 ax2.axhline(0, color='black', linewidth=0.5)
31 ax2.axvline(0, color='black', linewidth=0.5)
32
33 # 添加注释
34 ax2.annotate(r'$\sin(\frac{2\pi}{3})=\frac{\sqrt{3}}{2}$',
35              xy=(t, np.sin(t)), xycoords='data',
36              xytext=(+10, +30), textcoords='offset points',
       fontsize=16,
37              arrowprops=dict(arrowstyle="->",
       connectionstyle="arc3,rad=.2"))
38
39 ax2.annotate(r'$\cos(-\frac{2\pi}{3})=-\frac{1}{2}$',
40              xy=(t2, np.cos(t2)), xycoords='data',
41              xytext=(-90, 0), textcoords='offset points',
       fontsize=16,
42              arrowprops=dict(arrowstyle="->",
       connectionstyle="arc3,rad=.2"))
43
44 ax2.legend(loc='upper left')
45
46 # 调整布局并显示图像
47 plt.tight_layout()
48 plt.show()
```

七、 箭头

箭头可以通过`plt.arrow()`函数来设置。以下示例展示了如何添加箭头：

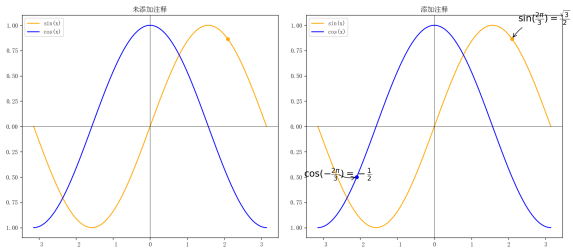

图 30.12 annotation 添加注释

```
import matplotlib.pyplot as plt

# 创建图像
plt.figure(figsize=(8, 6))

# 绘制箭头
plt.arrow(0, 0, 0.5, 0.5, width=0.02, head_width=0.1,
    head_length=0.1, fc='blue', ec='black')
## 从 (0, 0) 开始, 到 (0.5, 0.5) 结束
## width 是箭头的宽度, head_width 是箭头的底部宽度, head_length
    是箭头的长度
## fc 是箭头的填充颜色, ec 是箭头的边框颜色

plt.text(0.6, 0.6, '箭头', fontsize=12, color='black',
    ha='center', va='center')

# 设置坐标轴
plt.xlim(-1, 1)
plt.ylim(-1, 1)

# 添加标题
plt.title('示例: 使用 plt.arrow() 添加箭头')

# 显示图像
plt.grid(True)
plt.show()
```

八、 图片风格

matplotlib 提供了一些预设的风格，

```
print(plt.style.available[:])
>>>
```

```
['bmh', 'classic', 'dark_background', 'fivethirtyeight', 'ggplot']
```

可以通过`plt.style.use()`函数来设置。以下示例展示了如何设置图片风格：

```python
import matplotlib.pyplot as plt
import numpy as np

# 数据生成
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)

# 定义绘图风格
styles = ['Solarize_Light2', '_classic_test_patch', '_mpl-gallery', '_mpl-gallery-nogrid',
          'bmh', 'classic', 'dark_background', 'fast', 'fivethirtyeight', 'ggplot',
          'grayscale', 'seaborn-v0_8', 'seaborn-v0_8-bright', 'seaborn-v0_8-colorblind',
          'seaborn-v0_8-dark', 'seaborn-v0_8-dark-palette', 'seaborn-v0_8-darkgrid',
          'seaborn-v0_8-deep', 'seaborn-v0_8-muted', 'seaborn-v0_8-notebook',
          'seaborn-v0_8-paper', 'seaborn-v0_8-pastel', 'seaborn-v0_8-poster',
          'seaborn-v0_8-talk', 'seaborn-v0_8-ticks', 'seaborn-v0_8-white',
          'seaborn-v0_8-whitegrid', 'tableau-colorblind10']

# 计算行数和列数，每行3个
n_cols = 3
n_rows = (len(styles) + n_cols - 1) // n_cols  # 计算需要的行数

# 创建图和子图
fig, axs = plt.subplots(n_rows, n_cols, figsize=(10, n_rows * 3))
axs = axs.flatten()  # 展平成一维数组便于遍历

# 绘制每种风格
for i, style in enumerate(styles):
    plt.style.use(style)  # 设置当前风格
    axs[i].plot(x, y1, label='sin(x)')
    axs[i].plot(x, y2, label='cos(x)')
    axs[i].set_title(f'Style: {style}', fontsize=10)
    axs[i].legend(loc='upper right', fontsize=8)
    axs[i].grid(True)

# 隐藏多余的子图
```

```
for j in range(len(styles), len(axs)):
    axs[j].axis('off')

# 调整布局
plt.tight_layout()
plt.show()
```

九、 多图的排列

Matplotlib 提供了多种方法来绘制子图，常用的方法包括 `plt.subplot()`、`plt.subplots()` 和 `GridSpec`。

1) plt.subplot()

`plt.subplot()` 可以快速创建子图。

```
plt.subplot(nrows, ncols, index)
# nrows: 子图的行数
# ncols: 子图的列数
# index: 子图的索引，从1开始
```

```
import matplotlib.pyplot as plt
import numpy as np

# 创建数据
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)

# 创建一个2行1列的子图布局，绘制第一个子图
plt.subplot(2, 1, 1) # 创建一个2行1列的布局，子图在第1个位置
plt.plot(x, y1)
plt.title('Sine Function')

# 创建一个2行1列的子图布局，绘制第二个子图
plt.subplot(2, 1, 2)
plt.plot(x, y2)
plt.title('Cosine Function')

# 显示图像
plt.tight_layout()  # 自动调整子图间距
plt.show()
```

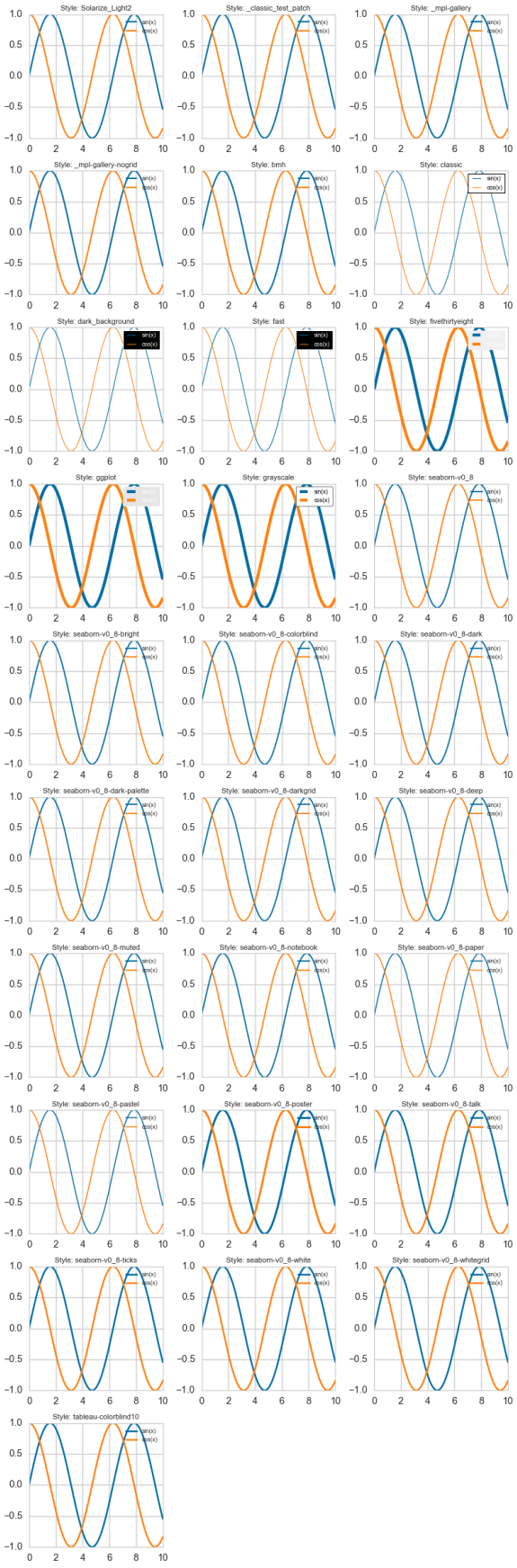

图 30.13 style 设置绘图风格

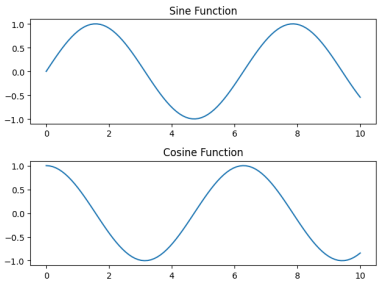

图 30.14 plt.subplot() 创建子图演示

(1) add_subplot() 添加子图

`plt.subplot()` 函数返回的是当前子图对象，可以通过 `add_subplot()` 方法来添加子图。

```
ax = fig.add_subplot(nrows, ncols, index)
# nrows: 子图的行数
# ncols: 子图的列数
# index: 子图的索引，从1开始
```

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  # 创建数据
5  rad = np.arange(0, np.pi * 2, 0.01)
6
7  # 创建图形对象 p1
8  p1 = plt.figure(figsize=(8, 10), dpi=80)  # 确定画布大小
9
10 # 第一个子图
11 ax1 = p1.add_subplot(2, 1, 1)  #
       创建一个两行1列的子图，并开始绘制第一幅
12 plt.title('lines')  # 添加标题
13 plt.xlabel('x1')  # 添加x轴的名称
14 plt.ylabel('y1')  # 添加y轴的名称
15 plt.xlim((0, 1))  # 确定x轴范围
16 plt.ylim((0, 1))  # 确定y轴范围
17 plt.xticks([0, 0.2, 0.4, 0.6, 0.8, 1])  # 规定x轴刻度
18 plt.yticks([0, 0.2, 0.4, 0.6, 0.8, 1])  # 确定y轴刻度
19 plt.plot(rad, rad)  # 添加 y = x 曲线
```

```
20 plt.plot(rad, rad**2)  # 添加 y = x^2 曲线
21 plt.legend(['y=x', 'y=x^2'])
22
23 # 第二个子图
24 ax2 = p1.add_subplot(2, 1, 2)  # 开始绘制第2幅
25 plt.title('sin/cos')  # 添加标题
26 plt.xlabel('x2')  # 添加x轴的名称
27 plt.ylabel('y2')  # 添加y轴的名称
28 plt.xlim((0, np.pi * 2))  # 确定x轴范围
29 plt.ylim((-1, 1))  # 确定y轴范围
30 plt.xticks([0, np.pi / 2, np.pi, np.pi * 1.5, np.pi * 2])  #
       规定x轴刻度
31 plt.yticks([-1, -0.5, 0, 0.5, 1])  # 确定y轴刻度
32 plt.plot(rad, np.sin(rad))  # 添加 sin 曲线
33 plt.plot(rad, np.cos(rad))  # 添加 cos 曲线
34 plt.legend(['sin', 'cos'])
35
36 # 保存并显示图像
37 plt.savefig('../tmp/subplot通过add_subplot添加子图.png')
38 plt.show()
```

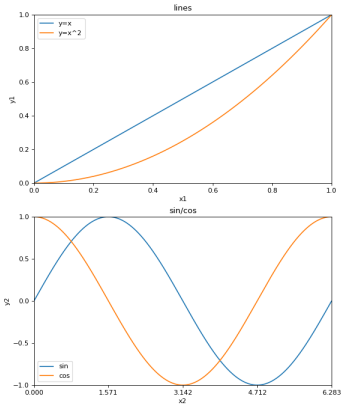

图 30.15 subplot 通过 add_subplot 添加子图

2) plt.subplots()

`plt.subplots()` 是 Matplotlib 中创建子图的推荐方法，它返回一个图形对象和一个包含所有子图的数组对象。

```
fig, axs = plt.subplots(nrows, ncols)
# nrows: 子图的行数
```

```
# ncols: 子图的列数
```

```
import matplotlib.pyplot as plt
import numpy as np

# 创建数据
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)

# 使用 subplots 创建 1行2列的子图布局
fig, axs = plt.subplots(1, 2, figsize=(10, 4))

# 绘制第一个子图
axs[0].plot(x, y1, 'r-')
axs[0].set_title('Sine Function')

# 绘制第二个子图
axs[1].plot(x, y2, 'b-')
axs[1].set_title('Cosine Function')

# 调整子图布局
plt.tight_layout()
plt.show()
```

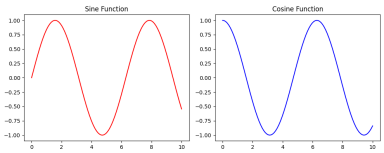

图 30.16 plt.subplots() 创建子图演示

3) GridSpec

`GridSpec` 提供了更灵活的方式来创建复杂的子图布局。通过 `GridSpec` 可以更细致地控制子图的大小和位置。

```
from matplotlib.gridspec import GridSpec
gs = GridSpec(nrows, ncols)
# nrows: 子图的行数
# ncols: 子图的列数

ax = fig.add_subplot(gs[row, col])
```

```
# gs[row, col]: 获取第 row 行第 col 列的子图
# row: 子图的行索引，从0开始
# col: 子图的列索引，从0开始
```

```
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.gridspec import GridSpec

# 创建数据
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)
y3 = np.tan(x)

# 创建一个新的图形
fig = plt.figure(figsize=(8, 6))

# 创建一个2x2的网格布局
gs = GridSpec(2, 2, figure=fig)

# 绘制子图1，占据第1行的全部
ax1 = fig.add_subplot(gs[0, :])
ax1.plot(x, y1, 'g-')
ax1.set_title('Sine Function')

# 绘制子图2，占据第2行第1列
ax2 = fig.add_subplot(gs[1, 0])
ax2.plot(x, y2, 'r-')
ax2.set_title('Cosine Function')

# 绘制子图3，占据第2行第2列
ax3 = fig.add_subplot(gs[1, 1])
ax3.plot(x, y3, 'b-')
ax3.set_title('Tangent Function')

# 调整子图布局
plt.tight_layout()
plt.show()
```

4) 自定义子图大小和间距

使用`plt.subplots_adjust()`可以调整子图之间的间距，方便对布局进行微调。

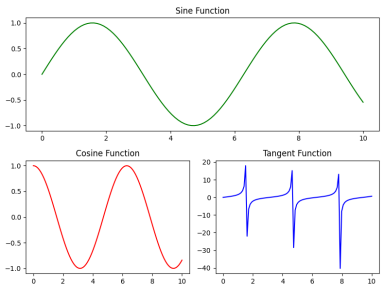

图 30.17 GridSpec 创建子图演示

```
plt.subplots_adjust(left=None, bottom=None, right=None, top=None,
    wspace=None, hspace=None)
# left: 左边距比例，默认为0.125
# bottom: 下边距比例，默认为0.11
# right: 右边距比例，默认为0.9
# top: 上边距比例，默认为0.88
# wspace: 水平间距比例，默认为0.2
# hspace: 垂直间距比例，默认为0.2
```

```python
import matplotlib.pyplot as plt
import numpy as np

# 创建数据
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)

# 创建一个2x1的子图
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(8, 6))

# 绘制子图
ax1.plot(x, y1, 'm-')
ax1.set_title('Sine Function')

ax2.plot(x, y2, 'c-')
ax2.set_title('Cosine Function')

# 调整子图之间的间距
plt.subplots_adjust(hspace=0.4, wspace=0.3)
plt.show()
```

第三十一章 [Python 库] Pyecharts 概述

一、 简介

Echarts 是一个由百度开源的数据可视化，凭借着良好的交互性，精巧的图表设计，得到了众多开发者的认可。而 Python 是一门富有表达力的语言，很适合用于数据处理。

1) V0.5x 版本：不在维护，支持 Python2.7，3.4+

2) V1.0 版本：v0.5.X 和 v1 间不兼容，仅支持 python3.6 及以上版本

3) V2.0 版本：支持 Python3.6 Python 3.11

二、 安装

```
pip install pyecharts

# 检查版本
$ python
Python 3.10.14 (main, May  6 2024, 19:42:50) [GCC 11.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import pyecharts
s.__version__)>>>
>>> print(pyecharts.__version__)
2.0.6
```

源码安装

```
$ git clone https://github.com/pyecharts/pyecharts.git
$ cd pyecharts
$ pip install -r requirements.txt -i
    https://pypi.tuna.tsinghua.edu.cn/simple
$ python setup.py install
# 或者执行 python install.py
```

第三十二章 图表类型

第三十三章 Manim 概述

一、 简介

Manim 最初由 Grant Sanderson 创建，用于他的 YouTube 频道 3Blue1Brown。随着频道的受欢迎程度增加，许多人想使用 Manim 来制作自己的项目。然而，最初的 Manim 仅供个人使用，其他用户很难安装和使用。

1) 2019 年底: Grant 开始在一个新分支（Shaders 分支）上工作，以实现更快的 OpenGL 渲染。
2) 2020 年中: 一组开发者将其分叉成现在的社区版（ManimCE）。
3) 2021 年初: Grant 将 Shaders 分支合并回主分支，形成了 ManimGL。

二、 Manim(ManimCE)

1) Manim 官网: https://www.manim.community/
2) Manim 项目地址: https://github.com/ManimCommunity/manim#command-line-arguments
3) Manim 文档: https://docs.manim.community/en/stable/#

PyPi 包名：`manim`

1) 安装

```
pip install manim
pip install manim  -i https://mirrors.aliyun.com/pypi/simple/
```

```
conda create -n my-manim-environment
conda activate my-manim-environment
conda install -c conda-forge manim
```

2) 使用

```
manim [options] <filename> <scene_name>
# 1.<filename>: 指定包含动画场景的Python脚本文件名（如myfile.py）
# 2.<scene_name>:
    指定要渲染的场景名称(可选，如果不指定，Manim会渲染文件中的第一个场景)
# 3.<options>: 可选参数
-h, --help: 显示帮助信息，列出所有可用的参数。
-p, --preview: 渲染完成后自动播放动画。
-ql, --quality low:
    以低质量（快速预览模式）渲染动画，适用于快速测试（15fps，480p）。
-qm, --quality medium: 以中等质量渲染动画（30fps，720p）。
-qh, --quality high: 以高质量渲染动画（60fps，1080p）。
-qk, --quality 4k: 以4K超高清质量渲染动画（60fps，2160p）。
-s, --save_last_frame: 只渲染场景的最后一帧并保存为图像文件。
-i, --save_as_gif: 将渲染的场景保存为GIF文件。
--renderer [cairo | opengl]:
    指定使用哪种渲染器（默认为cairo，opengl为高性能渲染器）。
-r, --resolution <height>x<width>: 设置输出视频的分辨率。例如-r
    1920x1080为1080p高清分辨率。
    -r 854,480:生成 480p 分辨率的视频
    -r 1280,720:生成 720p 分辨率的视频
    -r 1920,1080:生成 1080p 分辨率的视频
    -r 2560,1440:生成 1440p 分辨率的视频
    -r 3840,2160:生成 2160p 分辨率的视频
--fps <number>: 设置视频帧率（每秒帧数）。
-o, --output_file <path>: 指定输出文件的路径和文件名。
--config_file <path>: 指定自定义配置文件路径，用于设置渲染器的配置。
--progress_bar: 显示渲染进度条。
--log_level [DEBUG | INFO | WARNING | ERROR | CRITICAL]:
    设置日志级别，控制输出信息的详细程度。
--write_to_movie: 强制将场景输出为视频文件（mp4格式）

# 最常用的是 -pql 参数，用于快速预览
manim -ql -p myfile.py MyScene
```

3) 测试

```
# 测试中文能否显示
# manim -p .\sample.py
# -*- coding: utf-8 -*-
from manim import *

class TransformExample(Scene):
    def construct(self):
        banner = ManimBanner()
        banner.shift(UP * 0.5)
        self.play(banner.create(), run_time=1)
```

```
        self.play(banner.animate.scale(0.3), run_time=0.5)
        self.play(banner.expand(), run_time=1)

        t = Text("测试中文能否显示").next_to(banner, DOWN * 2)
        tex = VGroup(
            Text("测试数学公式:", font_size=30),
            Tex(r"$\sum_{n=1}^\infty \frac{1}{n^2} =
    \frac{\pi^2}{6}$"),
        )
        tex.arrange(RIGHT, buff=SMALL_BUFF)
        tex.next_to(t, DOWN)
        self.play(Write(t), run_time=1)
        self.play(Write(tex), run_time=1)

        self.wait()
```

三、 ManimGL

由 Grant Sanderson 维护的最新版本，具有更多实验性功能，PyPI 包名为 `manimgl`。

https://3b1b.github.io/manim/getting_started/installation.html#

四、 ManimCairo

Manim 的 Cairo 渲染器，OpenGL 渲染前的旧版本，仅用于渲染 Grant 的旧视频，PyPI 包名为 `manimlib`。

第三十四章 Manim 基本图形

一、 点

点是最简单图形，也是其他所有图形的基础。绘制其他任何图形时，都是用点来定位的。

```
Dot([x, y, z])
Dot([x, y, z], color=color)
```

```
# manim -pql 01Dot_demo.py DotSample
from manim import *
class DotSample(Scene):
    def construct(self):
        # 绘制 9个点
        for x in range(-1, 2):
            for y in range(1, -2, -1):
                p = Dot([x, y, 0], color=PURPLE)
                self.play(Create(p), run_time=0.5)
```

二、 线

1) 直线

提供任意 2 个点，manim 通过 `Line` 来绘制线。

```
Line(start=[x1, y1, z1], end=[x2, y2, z2])
Line(start=[x1, y1, z1], end=[x2, y2, z2], color=color)
```

```
# manim -pql 01Line_demo.py LineSample
```

```
from manim import *
class LineSample(Scene):
    def construct(self):
        # 绘制 9个点
        count = 0
        for x in range(-1, 2):
            for y in range(1, -2, -1):
                p1 = Dot([x, y, 0], color=PURPLE)
                p2 = Dot([x+1, y+1, 0], color=PURPLE)
                l = Line(p1.get_center(), p2.get_center(), color=BLUE)
                self.play(Create(p1), Create(p2), Create(l), run_time=0.5+count)
                count += 0.5
```

2) 虚线

提供任意 2 个点，manim 通过 `DashedLine` 来绘制虚线。

```
DashedLine(start=[x1, y1, z1], end=[x2, y2, z2])
DashedLine(start=[x1, y1, z1], end=[x2, y2, z2], color=color)
```

```
# manim -pql 01DashedLine_demo.py DashedLineSample
from manim import *
class DashedLineSample(Scene):
    def construct(self):
        self._dashedLines()
        self.wait()

    def _dashedLines(self):
        dl = DashedLine([-1, 1, 0], [1, 1, 0])
        self.play(Create(dl), run_time=0.5)

        dl = DashedLine([-1, 0, 0], [1, 0, 0])
        self.play(Create(dl), run_time=0.5)

        dl = DashedLine([-1, -1, 0], [1, -1, 0])
        self.play(Create(dl), run_time=0.5)
```

3) 带箭头的线

提供任意 2 个点，manim 通过 `Arrow` 来绘制带箭头的线。

```
Arrow(start=[x1, y1, z1], end=[x2, y2, z2])
Arrow(start=[x1, y1, z1], end=[x2, y2, z2], color=color)
```

```python
# manim -pql 01Arrow_demo.py ArrowSample
from manim import *
class ArrowSample(Scene):
    def construct(self):
        self._arrows()

    def _arrows(self):
        a = Arrow([-1, 1, 0], [1, 1, 0])
        self.play(Create(a), run_time=0.5)

        a = Arrow([-1, 0, 0], [1, 0, 0])
        self.play(Create(a), run_time=0.5)

        a = Arrow([-1, -1, 0], [1, -1, 0])
        self.play(Create(a), run_time=0.5)
```

三、 圆

1) 正圆

绘制圆只要提供半径即可，圆心默认在屏幕的中心。绘制圆使用 `Circle`。

```
Circle(radius=r)
Circle(radius=r, color=color)

# 通过指定圆心
Circle(radius=r, arc_center=[x, y, z])
```

```python
# manim -pql 01Circle_demo.py CircleSample
from manim import *
class CircleSample(Scene):
    def construct(self):
        self._circles()
```

```
6
7       def _circles(self):
8           c = Circle(radius=1)
9           self.play(Create(c), run_time=0.5)
10
11          c = Circle(radius=1, color=PURPLE, arc_center=[-2, 3, 0])
12          self.play(Create(c), run_time=0.5)
```

2) 椭圆

绘制椭圆只要提供长轴和短轴即可，圆心默认在屏幕的中心。绘制椭圆使用 `Ellipse`。

```
# width和 height分别控制椭圆最大宽度和最大高度
Ellipse(width=w, height=h)
Ellipse(width=w, height=h, color=color)

# 通过指定圆心
Ellipse(width=w, height=h, arc_center=[x, y, z])
```

```
1  # manim -pql 01Ellipse_demo.py EllipseSample
2  from manim import *
3  class EllipseSample(Scene):
4      def construct(self):
5          self._ellipses()
6
7      def _ellipses(self):
8          e = Ellipse(width=2, height=1)
9          self.play(Create(e), run_time=0.5)
10
11         e = Ellipse(width=3, height=1.5, color=PURPLE,
       arc_center=[-2, 3, 0])
12         self.play(Create(e), run_time=0.5)
```

3) 圆弧

绘制圆弧只要提供半径和角度即可，圆心默认在屏幕的中心。绘制圆弧使用 `Arc`。

```
# 1. angle: 圆弧的弧度
```

```
# 2. start_angle: 开始的角度，默认 0
# 3. radius: 圆弧的半径
Arc(radius=r, angle=angle)
Arc(radius=r, angle=angle, color=color)

# 通过指定圆心
Arc(radius=r, angle=angle, start_angle=start_angle, arc_center=[x, y, z])
```

```python
# manim -pql 01Arc_demo.py ArcSample
from manim import *
class ArcSample(Scene):
    def construct(self):
        self._arcs()

    def _arcs(self):
        # 90度的圆弧，默认圆心在屏幕中心，半径为1
        a = Arc(radius=1, angle=PI/2)
        self.play(Create(a), run_time=0.5)

        # 180度圆弧，半径2
        a = Arc(angle=PI, radius=2, color=BLUE)
        self.play(Create(a), run_time=0.5)

        # 30度圆弧，半径2，从270度开始绘制
        a = Arc(angle=PI / 6, start_angle=PI * 1.5, radius=2, color=RED)
        self.play(Create(a), run_time=0.5)

        # 90度圆弧，半径1，圆心在[-2，3，0]
        a = Arc(radius=1, angle=PI/2, color=PURPLE, arc_center=[-2, 3, 0])
        self.play(Create(a), run_time=0.5)
```

四、 多边形

1) 三角形

绘制等边三角形只要提供边长即可，圆心默认在屏幕的中心。绘制等边三角形使用 `Triangle`。

```python
# 创建等边三角形
```

```
# manim -pql 01Triangle_demo.py TriangleSample
from manim import *
class PolygonSample(Scene):
    def construct(self):
        self._triangles()
        self.wait()

    def _triangles(self):
        t = Triangle()
        self.play(Create(t))
```

```
Triangle(**kwargs)
# 1.vertices:
    类型: 列表或元组
    描述: 定义三角形的三个顶点。通常以 (x1, y1), (x2, y2), (x3, y3)
    的形式传递, 表示三角形的三个角的坐标。

# 2.fill_color:
    类型: 字符串 (颜色名称或十六进制颜色代码)
    描述: 指定三角形的填充颜色。例如, 可以使用 'red'、'blue' 或 '#FF5733'
    等。

# 3.border_color:
    类型: 字符串 (颜色名称或十六进制颜色代码)
    描述: 指定三角形边框的颜色。

# 4.border_width:
    类型: 浮点数或整数
    描述: 设置三角形边框的宽度, 以像素为单位。

# 5.opacity:
    类型: 浮点数, 通常在 0 到 1 之间
    描述: 设置三角形的透明度, 0 表示完全透明, 1 表示完全不透明。

# 6.position:
    类型: 列表或元组
    描述: 指定三角形的绘制位置, 可能以 (x, y)
    的形式传递, 表示三角形的起始点。

# 7.rotation:
    类型: 浮点数
    描述: 指定三角形的旋转角度, 以弧度或度为单位。
```

```
# manim -pql 01Triangle_demo.py TriangleSample
from manim import *

class TriangleSample(Scene):
```

```
    def construct(self):
        self._triangles()

    def _triangles(self):
        # 1.创建简单的三角形
        t = Triangle()
        self.play(Create(t))
        self.wait(2)

        # 清空画布
        self.play(Uncreate(t))
```

2) 矩形

(1) 正方形

绘制正方形只要提供边长即可，圆心默认在屏幕的中心。绘制正方形使用 `Square`。

```
Square(side_length=l)
Square(side_length=l, color=color)
```

```
# manim -pql 01Square_demo.py SquareSample
from manim import *

class SquareSample(Scene):
    def construct(self):
        self._squares()

    def _squares(self):
        # 1.创建简单的正方形
        s1 = Square(side_length=1)
        self.play(Create(s1))

        s2 = Square(side_length=1.5)
        self.play(Create(s2), run_time=0.5)

        s3 = Square() # 默认边长为2
        self.play(Create(s3), run_time=0.5)
        self.wait(2)
        self.play(Uncreate(s3))
        self.play(Uncreate(s2))
        self.play(Uncreate(s1))

```

```
        # 2.自定义正方形
        s = Square(side_length=5, color=BLUE)
        s.move_to([-2, 3, 0])
        s.rotate(PI/4)
        self.play(Create(s), run_time=2)
        self.wait(2)
```

(2) 矩形

绘制矩形只要提供宽度和高度即可，圆心默认在屏幕的中心。绘制矩形使用 `Rectangle`。

```
Rectangle(width=w, height=h)
Rectangle(width=w, height=h, color=color)
```

```
# manim -pql 01Rectangle_demo.py RectangleSample
from manim import *

class RectangleSample(Scene):
    def construct(self):
        self._rectangles()

    def _rectangles(self):
        # 1.创建简单的矩形
        r1 = Rectangle(width=2, height=1)
        self.play(Create(r1))

        r2 = Rectangle(width=2.5, height=1.5)
        self.play(Create(r2), run_time=0.5)

        r3 = Rectangle() # 默认宽度为4，高度为2
        self.play(Create(r3), run_time=0.5)
        self.wait(2)
        self.play(Uncreate(r3))
        self.play(Uncreate(r2))
        self.play(Uncreate(r1))

        # 2.自定义矩形
        r = Rectangle(width=4, height=2, color=BLUE,
    fill_color=RED, fill_opacity=0.5)
        r.move_to([-2, 3, 0])
        r.rotate(PI/4)
        self.play(Create(r), run_time=2)
        self.wait(2)
```

(3) 圆角矩形

带有圆角的矩形。角部不是尖锐的，而是圆滑的。使用`RoundedRectangle`绘制圆角矩形。

```
RoundedRectangle(**kwargs)
# 1.corner_radius:
    类型：浮点数或整数
    描述：设置圆角矩形的圆角半径，以像素为单位。

# 2.width:
    类型：浮点数或整数
    描述：设置圆角矩形的宽度，以像素为单位。

# 3.height:
    类型：浮点数或整数
    描述：设置圆角矩形的高度，以像素为单位。

# 4.fill_color:
    类型：字符串（颜色名称或十六进制颜色代码）
    描述：指定圆角矩形的填充颜色。例如，可以使用 'red'、'blue' 或 '#FF5733'
    等。

# 5.border_color:
    类型：字符串（颜色名称或十六进制颜色代码）
    描述：指定圆角矩形边框的颜色。

# 6.border_width:
    类型：浮点数或整数
    描述：设置圆角矩形边框的宽度，以像素为单位。

# 7.opacity:
    类型：浮点数，通常在 0 到 1 之间
    描述：设置圆角矩形的透明度，0 表示完全透明，1 表示完全不透明。

# 8.position:
    类型：列表或元组
    描述：指定圆角矩形的绘制位置，可能以 (x, y)
    的形式传递，表示圆角矩形的起始点。

# 9.rotation:
    类型：浮点数
    描述：指定圆角矩形的旋转角度，以弧度或度为单位。
```

```
1  # manim -pql 02Rectangle_demo.py RoundedRectangleSample
```

```python
from manim import *

class RoundedRectangleSample(Scene):
    def construct(self):
        self._rounded_rectangle()

    def _rounded_rectangle(self):
        # 创建一个圆角矩形，并设置其宽度和高度
        rect1 = RoundedRectangle(width=4, height=2)
        self.play(Create(rect1))

        rect2 = RoundedRectangle(width=4, height=2, corner_radius=0.5)
        self.play(Create(rect2), run_time=0.5)

        rect3 = RoundedRectangle(corner_radius=0.4, width=2, height=1.5)
        self.play(Create(rect3), run_time=0.5)

        rect4 = RoundedRectangle(corner_radius=0.5, width=2, height=1.5, color=BLUE, fill_color=RED, fill_opacity=0.75)
        rect4.move_to([-2, 3, 0])
        rect4.rotate(PI/4)
        self.play(Create(rect4), run_time=2)

        self.wait(2)
        self.play(Uncreate(rect4))
        self.play(Uncreate(rect1))
        self.play(Uncreate(rect2))
        self.play(Uncreate(rect3))

        # 创建一个圆角矩形，并设置其所有属性
        rect = RoundedRectangle(
            width=4,  # 宽度
            height=2,  # 高度
            corner_radius=0.5,  # 圆角半径
            fill_color=BLUE,  # 填充颜色
            stroke_color=RED,  # 边框颜色
            stroke_width=5,  # 边框宽度
            fill_opacity=0.75  # 透明度
        )
        rect.move_to([0, 0, 0])
        rect.rotate(PI/4)
        self.play(Create(rect), run_time=2)
        self.wait(2)
```

3) 星形

绘制星形只要提供内外半径和边数即可，圆心默认在屏幕的中心。绘制星形使用 `Star`。

```
Star(n=5, *, outer_radius=1, inner_radius=None, density=2,
    start_angle=1.5707963267948966, **kwargs)
# 1.n:
    类型: 整数
    描述: 指定星形的边数。
    默认值: 5
    通常是奇数(如 5、7、9 等)
    提示: 如果 n 不是正整数，则使用默认值 5。
    提示: 如果 n 小于 3，则使用默认值 5。
    提示: 如果 n 为偶数，则使用默认值 5。

# 2.outer_radius:
    类型: 浮点数或整数
    描述: 指定星形的外半径，以像素为单位。
    默认值: 1

# 3.inner_radius:
    类型: 浮点数或整数
    描述: 指定星形的内半径，以像素为单位。
    默认值: None
    提示: 如果 inner_radius 为 None，则使用 outer_radius 的 0.382 倍。

# 4.density:
    类型: 整数
    描述: 指定星形的密度。
    默认值: 2
    提示: 如果 density 不是正整数，则使用默认值 2。
    提示: 如果 density 小于 1，则使用默认值 2。

# 5.start_angle:
    类型: 浮点数
    描述: 指定星形的起始角度，以弧度为单位。
    默认值: PI/2
    提示: 如果 start_angle 不是有效的弧度值，则使用默认值 PI/2。
    提示: 如果 start_angle 小于 0 或大于 2PI，则使用默认值 PI/2。

# 6.fill_color:
    类型: 字符串 (颜色名称或十六进制颜色代码)
    描述: 指定星形的填充颜色。例如，可以使用 'red'、'blue' 或 '#FF5733' 等。
    提示: 如果 fill_color 不是有效的颜色名称或十六进制颜色代码，则使用默认值
    WHITE。

# 7.border_color:
    类型: 字符串 (颜色名称或十六进制颜色代码)
    描述: 指定星形边框的颜色。
    提示: 如果 border_color
    不是有效的颜色名称或十六进制颜色代码，则使用默认值 WHITE。

# 8.border_width:
```

```
    类型：浮点数或整数
    描述：设置星形边框的宽度，以像素为单位。
    默认值：0
    提示：如果 border_width 小于 0，则使用默认值 0。
```

```python
# manim -pql 01Star_demo.py StarSample
from manim import *
class StarSample(Scene):
    def construct(self):
        self._stars()

    def _stars(self):
        # 1.创建简单的星形
        s1 = Star()
        self.play(Create(s1), run_time=0.5)
        self.wait(2)
        self.play(Uncreate(s1))

        s2 = Star(n=5, outer_radius=2)
        self.play(Create(s2), run_time=0.5)
        self.wait(2)
        self.play(Uncreate(s2))

        s3 = Star(7, outer_radius=2, density=2, color=RED)
        s4 = Star(7, outer_radius=2, density=3, color=PURPLE)
        self.play(Create(s3), Create(s4), run_time=0.5)
```

4) 任意多边形

(1) 多边形

绘制任意多边形只要提供顶点即可，圆心默认在屏幕的中心。绘制任意多边形使用 `Polygon`。

```
Polygon(v1, v2, v3, ...)
Polygon(v1, v2, v3, ..., color=color)
```

```python
# manim -pql 01Polygon_demo.py PolygonSample
from manim import *
class PolygonSample(Scene):
    def construct(self):
```

```
5           self._polygons()
6
7       def _polygons(self):
8           # 1.创建简单的多边形
9           p = Polygon([-3, 1, 0], [-1, 1, 0], [-2, -1, 0])
10          self.play(Create(p), run_time=0.5)
11
12          p = Polygon([1, 1, 0], [2, 0, 0], [3, 1, 0], [3, -1, 0],
        [1, -1, 0])
13          self.play(Create(p), run_time=0.5)
14
15          # 2.自定义三角形
16          # 定义三角形的三个顶点
17          vertices = [[-2, 1, 0], [-1, -1, 0], [0, 1, 0]]
18          triangle = Polygon(
19              *vertices,
20              fill_color=BLUE,  # 填充颜色
21              stroke_color=RED,  # 边框颜色
22              stroke_width=5,  # 边框宽度
23              fill_opacity=0.75  # 透明度
24          )
25
26          # 设置三角形的位置
27          triangle.move_to([0, 0, 0])
28
29          # 旋转三角形 45 度 (PI/4 弧度)
30          triangle.rotate(PI/4)
31
32          # 显示三角形
33          self.play(Create(triangle), run_time=2)
34
35          # 保持显示一段时间
36          self.wait(2)
```

(2) 任意正多边形

虽然 manim 提供了绘制任意多边形的对象 `Polygon`，利用 Polygon 绘制正多边形理论上是完全可行的。不过，要自己去计算各个正多边形的坐标点显然有些费时费力，所以，manim 中还提供了一个专门用来绘制正多边形的对象 `RegularPolygon`.

```
RegularPolygon(n=n, **kwargs)
# 1.n_sides:
    类型: 整数
    描述: 指定正多边形的边数。
    默认值: 3
    提示: 3 <= n_sides <= 12
```

```
    提示: 如果 n_sides 不在指定范围内，则使用默认值 3。
    提示: 如果 n_sides 为 0 或负数，则使用默认值 3。
```

```python
# manim -pql 02RegularPolygon_demo.py RegularPolygonSample
from manim import *

class RegularPolygonSample(Scene):
    def construct(self):
        self._regular_polygon()

    def _regular_polygon(self):
        # 创建一个正五边形
        p1 = RegularPolygon() # 默认边数为5
        self.play(Create(p1))
        self.wait(2)
        self.play(Uncreate(p1))

        p2 = RegularPolygon(n=4)
        self.play(Create(p2), run_time=0.5)
        self.wait(2)
        self.play(Uncreate(p2))

        p3 = RegularPolygon(n=10)
        self.play(Create(p3), run_time=0.5)
        p3.move_to([0, 3, 0])
        p3.rotate(PI/4)
        self.play(Create(p3), run_time=2)
        p3.play(Uncreate(p3))

        self.wait(2)
```

五、 自定义图片

1) SVG

可以在https://www.svgrepo.com/网站上下载 svg 图片，然后使用`SVGMobject`来加载 svg 图片。

```python
from manim import *
import random

```

```
4  class EmojiPattern(Scene):
5      def construct(self):
6          # 设置背景颜色
7          self.camera.background_color = "#faf7f0"  #
       将场景的背景颜色设置为浅黄色
8
9          # 加载SVG文件
10         svg_path = "chinese-lion-svgrepo-com.svg"  #
       这里要将SVG文件路径替换为你下载的SVG文件路径
11         emoji = SVGMobject(svg_path).scale(0.5)  #
       将SVG文件加载为Manim对象并缩小到50%
12
13         # 定义颜色列表
14         colors = [RED, ORANGE, YELLOW, GREEN, TEAL, BLUE, PURPLE]
        # 可用的随机颜色列表
15
16         # 创建5x5的emoji网格，并赋予随机颜色和移动
17         rows = 5  # 网格行数
18         cols = 5  # 网格列数
19         x_spacing = 1.5  # x方向上每个emoji的间隔
20         y_spacing = 1.5  # y方向上每个emoji的间隔
21
22         emojis = []  # 用于存储所有emoji的列表
23         for i in range(rows):
24             for j in range(cols):
25                 emoji_copy = emoji.copy()  # 复制emoji对象
26                 emoji_copy.set_color(random.choice(colors))  #
       为emoji赋予随机颜色
27                 emoji_copy.move_to((j * x_spacing - 3, i *
       y_spacing - 3, 0))  # 将每个emoji移动到网格中的位置
28                 emojis.append(emoji_copy)  #
       将复制的emoji添加到列表中
29                 self.add(emoji_copy)  # 将emoji添加到场景中
30
31         # 添加底部文本
32         text = Text("Dragon Boat Festival",
       font_size=24).to_edge(DOWN)  # 在底部添加文字 “Dragon Boat
       Festival”
33         self.add(text)  # 将文本添加到场景中
34
35         # 让emoji随机移动并变化颜色，持续30秒
36         for _ in range(30):
37             self.play(
38                 # 随机移动每个emoji
39                 *[ApplyMethod(emoji.shift,
       np.random.uniform(-0.5, 0.5, 3)) for emoji in emojis],
40                 # 随机改变每个emoji的颜色
41                 *[ApplyMethod(emoji.set_color,
       random.choice(colors)) for emoji in emojis],
42                 run_time=1, rate_func=there_and_back  #
```

```
        动画持续1秒 ，效果为“来回”
43              )
44
45          # 暂停以查看最终场景
46          self.wait(2)  # 保持最终的场景显示2秒
47
48  if __name__ == "__main__":
49      from manim import config, tempconfig
50      config.background_color = "#faf7f0"  # 设置整个背景颜色为浅黄色
51      EmojiPattern().render()  # 渲染场景
```

六、 公共样式

属性	备注
color	颜色，默认为白色
opacity	透明度，默认为 0
gloss	反光度
shadow	阴影

表 34.1 manim 中的公共样式

七、 图像样式

绘制各类基本图形（点，线，圆，多边形等）时，每个图形都有自己的默认的样式（如颜色，填充颜色，边框颜色，边框宽度等）。但是，manim 也提供了一些方法来自定义图形的样式。

属性	备注
fill_color	填充的颜色
fill_opacity	填充的透明度
stroke_color	轮廓线的颜色
stroke_opacity	轮廓线的透明度
stroke_width	轮廓线的粗细
draw_stroke_behind_fill	是否在填充后在绘制轮廓线
background_image_fill	图形填充图片的路径

表 34.2 manim 中的图像样式

1) 边框

可以通过`stroke_color`和`stroke_width`来设置图形的边框颜色和边框宽度。

```
# 1.stroke_color:
    类型: 字符串 (颜色名称或十六进制颜色代码)
    描述: 指定图形的边框颜色。
    提示: 如果 stroke_color
    不是有效的颜色名称或十六进制颜色代码, 则使用默认值 WHITE。

# 2.stroke_width:
    类型: 浮点数或整数
    描述: 设置图形的边框宽度, 以像素为单位。
    默认值: 4
    提示: 如果 stroke_width 小于 0, 则使用默认值 4。
```

```python
# manim -pql 03Line_demo.py LineSample
from manim import *
class LineSample(Scene):
    def construct(self):
        self._lines()

    def _lines(self):
        # 1.创建简单的直线
        l1 = Line(
            [-1, 1, 0],
            [1, 1, 0],
            stroke_width=1,
        )
        self.play(Create(l1), run_time=0.5)

        l2 = Line(
            [-1, 0, 0],
            [1, 0, 0],
            stroke_color=RED,
            stroke_width=2,
        )
        self.play(Create(l2), run_time=0.5)

        l3 = Line(
            [-1, -1, 0],
            [1, -1, 0],
            stroke_color=BLUE,
            stroke_width=10,
        )
        self.play(Create(l3), run_time=0.5)
```

2) 颜色

(1) 预制颜色

各种基本图形本身有默认的颜色，比如线默认是白色，圆默认红色，多边形默认蓝色等。

除了默认颜色，manim 还预置了一系列的颜色，可以直接按颜色的名称来使用。

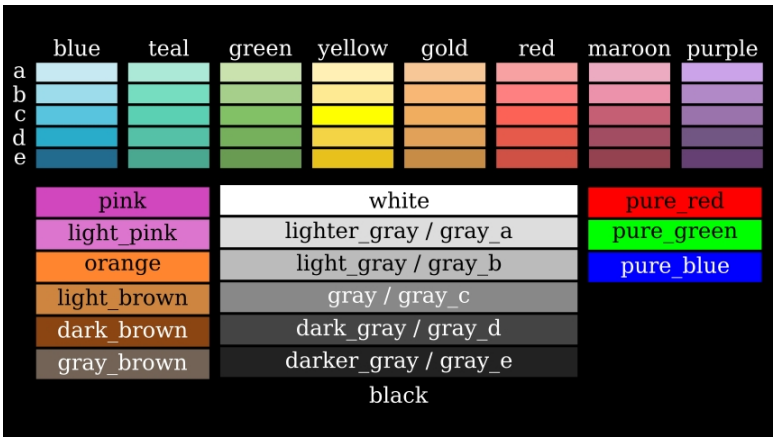

图 34.1 manim 预置颜色

可以通过`color`来设置图形的颜色。

```
from manim import *

class ColorLineDemo(Scene):
    def construct(self):
        # 定义所有颜色系列及其不同深度
        color_groups = [
            [BLUE_E, BLUE_D, BLUE_C, BLUE_B, BLUE_A],        # blue 系列
            [TEAL_E, TEAL_D, TEAL_C, TEAL_B, TEAL_A],        # teal 系列
            [GREEN_E, GREEN_D, GREEN_C, GREEN_B, GREEN_A],   # green 系列
            [YELLOW_E, YELLOW_D, YELLOW_C, YELLOW_B, YELLOW_A], # yellow 系列
```

```
            [GOLD_E, GOLD_D, GOLD_C, GOLD_B, GOLD_A],        # gold 系列
            [RED_E, RED_D, RED_C, RED_B, RED_A],             # red 系列
            [MAROON_E, MAROON_D, MAROON_C, MAROON_B, MAROON_A], # maroon 系列
            [PURPLE_E, PURPLE_D, PURPLE_C, PURPLE_B, PURPLE_A], # purple 系列
            [PINK, LIGHT_PINK, ORANGE, LIGHT_BROWN, DARK_BROWN], # 单色系列
            [GRAY_BROWN, WHITE, LIGHTER_GRAY, LIGHT_GRAY, GRAY], # 灰色系列
            [DARK_GRAY, DARKER_GRAY, BLACK],                  # 深灰系列
            [PURE_RED, PURE_GREEN, PURE_BLUE]                # 纯色系列
        ]

        # 起始位置的 y 坐标
        y_start = 3
        # 每个颜色组的初始 x 坐标
        x_start = -6

        # 绘制每个颜色组及其深度
        for i, color_group in enumerate(color_groups):
            # 调整 x 坐标以便水平排列
            x_pos = x_start + (i * 1.6)
            for j, color in enumerate(color_group):
                # 创建每条线的y坐标偏移
                y_pos = y_start - (j * 0.6)
                line = Line(
                    [x_pos, y_pos, 0],
                    [x_pos + 1.2, y_pos, 0],
                    stroke_width=10,
                    color=color
                )
                # 动画显示线条
                self.play(Create(line), run_time=0.2)

        # 保持显示2秒
        self.wait(2)
```

(2) 填充和透明度

可以通过`fill_color`和`fill_opacity`来设置图形的填充颜色和填充透明度。

```
# 1.fill_color:
    类型: 字符串 (颜色名称或十六进制颜色代码)
    描述: 指定图形的填充颜色。
    提示: 如果 fill_color 不是有效的颜色名称或十六进制颜色代码, 则使用默认值
    WHITE。

# 2.fill_opacity:
    类型: 浮点数或整数
    描述: 设置图形的填充透明度, 取值范围为 0 到 1。
    默认值: 0
    提示: 如果 fill_opacity 小于 0 或大于 1, 则使用默认值 0。
```

```python
# manim -pql 03Line_demo.py LineSample
from manim import *
class LineSample(Scene):
    def construct(self):
        self._lines()

    def _lines(self):
        # 1.创建简单的直线
        l1 = Line(
            [-1, 1, 0],
            [1, 1,0],
            fill_color=BLUE,
            fill_opacity=0.5,
            stroke_width=1,
        )
        self.play(Create(l1), run_time=0.5)

        l2 = Line(
            [-1, 0, 0],
            [1, 0, 0],
            fill_color=RED,
            fill_opacity=0.75,
            stroke_width=2,
        )
        self.play(Create(l2), run_time=0.5)

        c1 = Circle(
            radius=1,
            fill_color=BLUE,
            fill_opacity=0.5,
            stroke_width=1,
        )
        self.play(Create(c1), run_time=0.5)
```

八、 坐标系

没有引入坐标系之前，在绘制图形时，也有一个隐含的坐标系，它和屏幕的像素相关。

比如，我们之前示例中的各个图形，屏幕的中心就是坐标原点（[0,0]），横轴坐标的范围大概是 [−3.5,3.5]，纵轴的坐标范围大概是 [−4,4]，这个范围与设置的视频分辨率有关，分辨率设置的越高的话，坐标范围越大。

1) 一维坐标系

数轴（`NumberLine`）是最基本的一维坐标系，它的关键参数是

1) `x_range`: 设置数轴的范围和间隔
2) `length`: 设置数轴的长度

```python
# manim -pql 01NumberLineExample.py NumberLineExample
from manim import *

class NumberLineExample(Scene):
    def construct(self):
        # 第一条数轴：范围从 -10 到 10，步长为 2，长度为 10，包含数字
        number_line_1 = NumberLine(
            x_range=[-10, 10, 2],  # 设置数轴范围为 -10 到 10，步长 为 2
            length=10,  # 设置数轴长度为 10
            include_numbers=True  # 在数轴上显示数字
        )
        number_line_1.to_edge(UP)  # 将第一条数轴移到屏幕上方

        # 第二条数轴：范围从 -3 到 3，步长为 0.5，长度为 12，包含数字
        number_line_2 = NumberLine(
            x_range=[-3, 3, 0.5],  # 设置数轴范围为 -3 到 3，步长为 0.5
            length=12,  # 设置数轴长度为 12
            include_numbers=True  # 在数轴上显示数字
        )
        number_line_2.next_to(number_line_1, DOWN, buff=1)  # 将第二条数轴放在第一条数轴下方

        # 第三条数轴：范围从 -5 到 5，步长为 1，长度为 6，包含数字和箭头，旋转10度
        number_line_3 = NumberLine(
            x_range=[-5, 5 + 1, 1],  # 设置数轴范围为 -5 到 6，步长 为 1
```

```
            length=6,  # 设置数轴长度为 6
            include_numbers=True,  # 在数轴上显示数字
            include_tip=True,  # 在数轴的两端加上箭头
            rotation=10 * DEGREES  # 数轴旋转10度
        )
        number_line_3.next_to(number_line_2, DOWN, buff=1)  # 将第三条数轴放在第二条数轴下方

        # 动画显示三条数轴
        self.play(Create(number_line_1))
        self.play(Create(number_line_2))
        self.play(Create(number_line_3))

        # 保持场景显示2秒
        self.wait(2)

if __name__ == "__main__":
    from manim import config
    config.background_color = "#ffffff"  # 设置背景为白色
    scene = NumberLineExample()
    scene.render()
```

2) 二维坐标系

(1) NumberPlane 实数平面

实数平面（`NumberPlane`）是最基本的二绶坐标系，它的关键参数是

1) `x_range`: 设置X 轴横轴的范围和间隔
2) `y_range`: 设置Y 轴纵轴的范围和间隔
3) `x_length`: 设置X 轴横轴的长度
4) `y_length`: 设置Y 轴纵轴的长度

```
# manim -pql 01NumberPlaneExample.py NumberPlaneExample
from manim import *

class NumberPlaneExample(Scene):
    def construct(self):
        # 第一张网格: x 轴范围为 -4 到 11, y 轴范围为 -3 到 3, x 轴长度为 3, y 轴长度为 2
        number_plane_1 = NumberPlane(
```

```
            x_range=(-4, 11, 1),  # 设置 x 轴范围为 -4 到 11，步长为 1
            y_range=(-3, 3, 1),   # 设置 y 轴范围为 -3 到 3，步长为 1
            x_length=3,  # 设置 x 轴的物理长度为 3
            y_length=2   # 设置 y 轴的物理长度为 2
        )
        number_plane_1.shift(LEFT * 2)  # 将第一张网格移到屏幕左侧

        # 第二张网格：x 轴范围为 -4 到 11，x 轴长度为 3，y 轴长度为 4（没有指定 y 轴范围）
        number_plane_2 = NumberPlane(
            x_range=(-4, 11, 1),  # 设置 x 轴范围为 -4 到 11，步长为 1
            x_length=3,  # 设置 x 轴的物理长度为 3
            y_length=4   # 设置 y 轴的物理长度为 4
        )
        number_plane_2.shift(RIGHT * 2)  # 将第二张网格移到屏幕右侧

        # 动画显示两张网格
        self.play(Create(number_plane_1))
        self.play(Create(number_plane_2))

        # 保持场景显示2秒
        self.wait(2)

if __name__ == "__main__":
    from manim import config
    config.background_color = "#ffffff"  # 设置背景为白色
    scene = NumberPlaneExample()
    scene.render()
```

(2) ComplexPlane 复数平面

```
# manim -pql 01ComplexPlaneExample.py ComplexPlaneExample
from manim import *

class ComplexPlaneExample(Scene):
    def construct(self):
        # 创建复平面，并添加坐标系
        plane = ComplexPlane().add_coordinates()

        # 创建第一个点，表示复数 2 + i，颜色为黄色
        d1 = Dot(plane.n2p(2 + 1j), color=YELLOW)  # 使用 n2p
```

```
        将复数转换为平面上的点
11          label1 = Tex("2+i").next_to(d1, UR, buff=0.1)  #
        在点的右上 角添加标签 "2+i"
12
13          # 创建第二个点，表示复数 -3 - 2i，颜色为黄色
14          d2 = Dot(plane.n2p(-3 - 2j), color=YELLOW)  # 使用 n2p
        将复 数转换为平面上的点
15          label2 = Tex("-3-2i").next_to(d2, UR, buff=0.1)  #
        在点的右 上角添加标签 "-3-2i"
16
17          # 动画显示复平面、点和标签
18          self.play(Create(plane))
19          self.play(FadeIn(d1, label1))
20          self.play(FadeIn(d2, label2))
21
22          # 保持场景显示2秒
23          self.wait(2)
24
25  if __name__ == "__main__":
26      from manim import config
27      config.background_color = "#fafafa"  # 设置浅色背景
28      scene = ComplexPlaneExample()
29      scene.render()
```

(3) PolarPlane 极坐标平面

PolarPlane是一个极坐标平面,它通过角度和与原点的距离来定位位置，经常被用于导航类的系统中，与直角坐标相比，在这类系统中能极大的简化计算。它的关键参数有：

1) azimuth_step: 分割的角度个数
2) size: 极坐标在屏幕中显示的大小
3) radius_step：极坐标半径的间隔
4) radius_max：极坐标最大半径

```
1  # manim -pql 01PolarPlaneExample.py PolarPlaneExample
2  from manim import *
3
4  class PolarPlaneExample(Scene):
5      def construct(self):
6          # 创建极坐标平面
7          plane = PolarPlane(
```

```
            azimuth_step=30,    # 设置方位角步长为 30 度
            size=6,             # 设置极坐标平面的大小为 6
            radius_step=1,       # 设置半径的步长为 1
            radius_max=3         # 设置最大半径为 3
        ).add_coordinates()      # 添加坐标轴的数字

        # 将极坐标平面居中放置
        plane.move_to(ORIGIN)

        # 动画显示极坐标平面
        self.play(Create(plane))

        # 保持场景显示2秒
        self.wait(2)

if __name__ == "__main__":
    from manim import config
    config.background_color = "#fafafa"  # 设置浅色背景
    scene = PolarPlaneExample()
    scene.render()
```

(4) Axes 二维笛卡尔坐标轴

二维的笛卡尔坐标系（Axes）使用的比较多，它在平面坐标系之上，又提供了更多的配置，可以更加灵活的配置数轴，除了上面平面坐标系提到的那 4 个关键参数之外，还有 2 个配置坐标轴的参数也很重要：

1) x_axis_config: 配置 X 轴如何显示的参数

2) y_axis_config: 配置 Y 轴如何显示的参数

```
# manim -pql 01LogPlotExample.py LogPlotExample
from manim import *

class LogPlotExample(Scene):
    def construct(self):
        # 创建带有对数 y 轴的坐标系
        ax = Axes(
            x_range=[0, 10, 1],    # x 轴范围为 0 到 10, 步长为 1
            y_range=[-2, 6, 1],    # y 轴范围为 -2 到 6, 对数缩放
            x_length=6,            # x 轴长度为 6
            tips=False,            # 去掉轴的箭头
            axis_config={"include_numbers": True},  #
    在轴上显示数字
```

```
            y_axis_config={"scaling": LogBase(custom_labels=True)},  # y 轴采用对数缩放
        )

        # 绘制 y = x^2 的函数曲线，注意 x_min 必须大于 0，因为 x=0 时，y 是负无穷
        graph = ax.plot(lambda x: x**2, x_range=[0.001, 10], use_smoothing=False)

        # 将坐标轴和函数曲线添加到场景中
        self.play(Create(ax), Create(graph))

        # 保持场景显示2秒
        self.wait(2)

if __name__ == "__main__":
    from manim import config
    config.background_color = "#ffffff"  # 设置浅色背景
    scene = LogPlotExample()
    scene.render()
```

3) 三维坐标系

(1) ThreeDAxes 三维笛卡尔坐标系

三维的笛卡尔坐标系（`ThreeDAxes`）与二维坐标系的参数类似，只是多了一个维度（Z 轴）的配置，其配置参数与 X 轴和 Y 轴类似。显示三维图形时，有 2 点需要额外注意，

1) 场景要继承 `ThreeDScene`

2) 要调整下默认的相机位置，也就是视角的位置，默认视角是从 Z 轴顶部向下看的。

```
# manim -pql  01CoordinateSample.py  CoordinateSample
from manim import *

# 创建三维坐标轴并绘制一个蓝色圆形
class CoordinateSample(ThreeDScene):
    def construct(self):
        # 创建三维坐标轴，设置 y 轴的长度为 8
        axes = ThreeDAxes(y_length=8)

        # 创建一个蓝色的圆形，半径为 2
```

```
        circle = Circle(color=BLUE, radius=2)

        # 将坐标轴和圆形组合成一个 VGroup，以便同时显示
        vg = VGroup(axes, circle)

        # 默认的相机视角

        # 动画显示坐标轴和圆形
        self.play(Create(vg), run_time=2)

        # 暂停以查看最终场景
        self.wait()

# 运行场景
if __name__ == "__main__":
    from manim import config
    config.background_color = "#ffffff"  # 设置背景颜色为白色
    scene = CoordinateSample()
    scene.render()
```

```
# manim -pql  01CoordinateSample.py  CoordinateSample
from manim import *

# 创建三维坐标轴并绘制一个蓝色圆形
class CoordinateSample(ThreeDScene):
    def construct(self):
        # 创建三维坐标轴，设置 y 轴的长度为 8
        axes = ThreeDAxes(y_length=8)

        # 创建一个蓝色的圆形，半径为 2
        circle = Circle(color=BLUE, radius=2)

        # 将坐标轴和圆形组合成一个 VGroup，以便同时显示
        vg = VGroup(axes, circle)

        # 调整相机视角的代码 phi是与Z轴之间的角度，theta是围绕Z轴旋转的角度
        self.set_camera_orientation(phi=75 * DEGREES, theta=30 * DEGREES)

        # 动画显示坐标轴和圆形
        self.play(Create(vg), run_time=2)

        # 暂停以查看最终场景
        self.wait()
```

```
24
25 # 运行场景
26 if __name__ == "__main__":
27     from manim import config
28     config.background_color = "#ffffff"  # 设置背景颜色为白色
29     scene = CoordinateSample()
30     scene.render()
```

例子 34.1

实现全互联的神将网络

方法:

```
1      # manim -pql 02NeuralNetwoek_01.py
2      from manim import *
3
4      class NeuralNetwork(Scene):
5          def construct(self):
6              # 定义每一层的神经元数量
7              #layer_sizes = [1, 8, 16, 8, 1]
8              layer_sizes = [1, 8, 8, 1]
9
10             # 定义每层神经元的位置
11             layers = self.create_layers(layer_sizes)
12
13             # 连接所有层的节点
14             self.connect_layers(layers)
15
16             # 保持显示
17             self.wait(2)
18
19         def create_layers(self, layer_sizes):
20             """
21             创建每一层的神经元，并返回各层的节点集合
22             """
23             layers = []
24             x_position = -5  # 起始x坐标
25             for layer_size in layer_sizes:
26                 layer = self.create_layer(layer_size,
       x_position)
27                 layers.append(layer)
28                 x_position += 2  # 各层间隔
29
30             return layers
```

```
31
32          def create_layer(self, layer_size, x_position):
33              """
34              创建单层神经元，并返回该层的节点集合
35              """
36              layer = VGroup()  #
        创建一个组来包含这一层的所有节点
37              y_start = -(layer_size - 1) / 2  # 起始y坐标
38              for i in range(layer_size):
39                  neuron = Circle(radius=0.2, color=WHITE)  #
        创建一个神经元（圆形）
40                  neuron.move_to([x_position, y_start + i, 0])
         # 移动到指定位置
41                  layer.add(neuron)
42                  self.play(Create(neuron), run_time=0.1)  #
        动画显示神经 元
43
44              return layer
45
46          def connect_layers(self, layers):
47              """
48              连接相邻层之间的所有神经元
49              """
50              for i in range(len(layers) - 1):
51                  layer1 = layers[i]
52                  layer2 = layers[i + 1]
53                  for neuron1 in layer1:
54                      for neuron2 in layer2:
55                          connection =
        Line(neuron1.get_center(), neuron2.get_center(),
        stroke_width=1, color=WHITE)
56                          self.play(Create(connection),
        run_time=0.05)  # 动画显示连接线
57
```

第三十五章 Manim 文字和公式

一、 Text 文字

Text 系列常用的对象主要有三种：

1) Text：一般用于单行的文本

2) Paragraph：一般用于多行的文本

3) MarkupText：一般用于富文本显示

1) 一般文本

```python
# manim -pql 01TextSample.py TextSample
from  manim import *

class TextSample(Scene):
    def construct(self):
        text1 = Text("Hello, World!")
        self.play(Write(text1))
        text2 = Text("Hello, World!", font="Arial")
        self.play(ReplacementTransform(text1, text2))
        self.play(FadeOut(text2))

        # 创建不同语言的文本
        english_text = Text("hello world")  # 英文
        chinese_text = Text("你好，世界")    # 中文
        japanese_text = Text("こんにちは世界") # 日文
        russian_text = Text(", ")   # 俄文
        arabic_text = Text(" ", font="sans-serif") # 阿拉伯文
        # 对文本进行位置调整，使其按顺序排列
        english_text.to_edge(UP * 2)
        chinese_text.next_to(english_text, DOWN, buff=0.5)
        japanese_text.next_to(chinese_text, DOWN, buff=0.5)
        russian_text.next_to(japanese_text, DOWN, buff=0.5)
        arabic_text.next_to(russian_text, DOWN, buff=0.5)
```

```
        # 将所有文本依次显示到场景中
        self.play(Write(english_text))
        self.play(Write(chinese_text))
        self.play(Write(japanese_text))
        self.play(Write(russian_text))
        self.play(Write(arabic_text))

        self.wait(2)
```

2) 多行文本

```
# manim -pql 01ParagraphSample.py ParagraphSample
from manim import *

class ParagraphSample(Scene):
    def construct(self):
        # 创建包含古诗的 Paragraph 对象
        paragraph = Paragraph(
            "   春晓\n",          # 标题
            "春眠不觉晓",         # 第一行
            "处处闻啼鸟",         # 第二行
            "夜来风雨声",         # 第三行
            "花落知多少",         # 第四行
            alignment="center"   # 文本居中对齐
        )

        # 设置诗的标题样式
        title = paragraph[0]  # 提取标题部分
        title.set_color(YELLOW)  # 设置标题颜色

        # 动画显示古诗
        self.play(Write(paragraph, run_time=3))

        # 保持场景显示2秒
        self.wait(2)
```

3) 富文本

使用 `MarkupText` 类来创建带有 HTML 样式的文本。

1) `<b>...</b>`：加粗

2) `<i>...</i>`：斜体

3) `<u>...</u>`：下划线

4) `<s>...</s>`：中划线

5) `<span underline="double">...</span>`：双下划线

6) `<span underline="error">...</span>`：错误标记（通常是红色波浪线）

7) `<sub>...</sub>`：下标

8) `<sup>...</sup>`：上标

设置位置

1) `to_edge()`：设置文本的位置

2) `next_to()`：设置文本相对于另一个对象的位置

3) `shift()`：设置文本的位置偏移

4) `move_to()`：设置文本的位置

5) `scale()`：设置文本的缩放比例

```python
# manim -pql 01MarkupTextDemo.py MarkupTextDemo

from manim import *

class MarkupTextDemo(Scene):
    def construct(self):
        # 创建不同样式的文本对象
        bold_italic_text = MarkupText("<b>bold</b>
    <i>斜体文字</i>")
        underline_strike_text = MarkupText("<u>下划线</u>
    <s>中划线</s>")
        double_underline_error = MarkupText(
            '<span underline="double">双下划线</span> <span
    underline="error">错误标记</span>'
        )
        subscript_superscript =
    MarkupText("下标和上标：H<sub>2</sub>O
    H<sub>3</sub>O<sup>+</sup>")

        # 设置每个文本的位置
```

```
16          bold_italic_text.to_edge(UP * 2)
17          underline_strike_text.next_to(bold_italic_text, DOWN,
        buff=0.5)
18          double_underline_error.next_to(underline_strike_text,
        DOWN, buff=0.5)
19          subscript_superscript.next_to(double_underline_error,
        DOWN, buff=0.5)
20
21          # 动画显示文本
22          self.play(Write(bold_italic_text))
23          self.play(Write(underline_strike_text))
24          self.play(Write(double_underline_error))
25          self.play(Write(subscript_superscript))
26
27          # 保持显示2秒
28          self.wait(2)
```

二、 Tex 公式

Tex 系列的对象主要是用来显示数学公式，基于 Tex 之上，manim 还封装了列表和标题两种常用的对象。

不过，Tex 默认只支持显示英文，所以多用于显示公式，显示文本还是 `Text` 系列的对象比较常用。

1) 公式

`Tex` 可以显示任何 Latex 格式的数学公式：

```
1  # manim -pql 01MathEquations.py MathEquations
2  from manim import *
3
4  class MathEquations(Scene):
5      def construct(self):
6          # 创建不同的数学公式
7          pythagorean = Tex(r"$a^2 + b^2 = c^2$")
8          einstein = Tex(r"$E = mc^2$")
9          euler = Tex(r"$e^{i\pi} + 1 = 0$")
10
11         # 设置每个公式的位置
12         pythagorean.to_edge(UP * 2)
13         einstein.next_to(pythagorean, DOWN, buff=0.5)
14         euler.next_to(einstein, DOWN, buff=0.5)
15
```

```
        # 动画显示公式
        self.play(Write(pythagorean))
        self.play(Write(einstein))
        self.play(Write(euler))

        # 保持显示2秒
        self.wait(2)
```

2) 列表

`BulletedList`会将传入的多个字符串以列表的方式排列起来

```
# manim -pql 01MathListDemo.py MathListDemo
from manim import *

class MathListDemo(Scene):
    def construct(self):
        # 创建包含数学公式的项目符号列表
        bulleted_list = BulletedList(
            r"$a^2 + b^2 = c^2$",   # 毕达哥拉斯定理
            r"$E = mc^2$",          # 爱因斯坦质能方程
            r"$e^{i\pi} + 1 = 0$"   # 欧拉公式
        )

        # 将列表居中显示
        bulleted_list.scale(1.5)  # 调整文本大小以便更好展示

        # 动画展示项目符号列表
        self.play(Write(bulleted_list))

        # 保持显示2秒
        self.wait(2)
```

3) 标题

`Title` 会将传入的字符串以标题的方式展示出来。

```
# manim -pql 01TitleDemo.py TitleDemo
from manim import *
```

```python
import manim

class TitleDemo(Scene):
    def construct(self):
        # 获取当前 Manim 版本号
        version_text = f"Manim version {manim.__version__}"

        # 创建标题对象
        title = Title(version_text)

        # 动画显示标题
        self.play(Write(title))

        # 保持显示2秒
        self.wait(2)
```

`Title` 类基于 LaTeX 渲染器，而在渲染中文时，LaTeX 的默认设置可能无法正确处理中文字符，因此会导致 LaTeX 编译错误。为了解决这个问题，我们可以使用 Title 类中的 Tex 模式来处理非中文字符，而对于中文文本则使用 `Text` 类。

```python
from manim import *

class FEMHistory(Scene):
    def construct(self):
        #1. 定义有限元方法的历史事件
        history = [
            "1943年: 结构工程师柯朗提出有限元方法的早期概念。",
            "1956年: Turner等人正式提出有限元方法，用于解决结构分析问题。",
            "1960年代: 有限元方法被广泛应用于结构力学、热传导和流体力学领域。",
            "1970年代: 有限元方法理论化，逐渐推广到非线性问题和复杂几何形状。",
            "1980年代: 高性能计算的发展推动了大型有限元模型的应用。",
            "1990年代: 有限元方法成为工程、航空航天、汽车等行业的标准工具。",
            "2000年后: 有限元方法与其他数值方法结合，如多尺度模拟和优化。"
        ]

        # 2.创建标题
        # 1) 使用Title显示英文标题
        #title = Title("FEM History").scale(1.2)  # 使用Title只能显 示英文
```

```
19          # 2) 使用Text显示中文标题
20          # (1)创建标题
21          title = Text("有限元方法的发展历史",
    font="SimSun").scale(1.2)  # SimSun 字体支持中文
22          # (2)获取标题的宽度并创建下划线
23          underline = Line(
24              start=[-title.width / 2, -0.1, 0],  #
    根据标题宽度设置下划线的起始点
25              end=[title.width / 2, -0.1, 0],     # 下划线的终止点
26              stroke_width=2,                     # 下划线的宽度
27              color=WHITE                         # 下划线颜色为白色
28          )
29          # (3)将标题与下划线组合成一个VGroup，便于统一操作
30          title_group = VGroup(title, underline).arrange(DOWN,
    buff=0.2)
31
32          # 3.播放动画
33          # self.play(FadeIn(title))      # 显示Title
34          self.play(FadeIn(title_group))  # 显示Text标题
35          self.wait(1)
36          #self.play(title.animate.to_edge(UP))
37          self.play(title_group.animate.to_edge(UP))
38
39          # 4.动画显示每个历史事件
40          for event in history:
41              event_text = Text(event, font="SimSun", font_size=24)
42              self.play(FadeIn(event_text))
43              self.wait(2)
44              self.play(FadeOut(event_text))
45
46          # 5.结束语
47          thank_you = Text("感谢观看！ ", font="SimSun").scale(1.2)
48          self.play(FadeIn(thank_you))
49          self.wait(2)
50
51  if __name__ == "__main__":
52      from manim import *
53      scene = FEMHistory()
54      scene.render()
```

三、 文本样式

Manim 支持自定义文本样式，允许用户修改字体、大小、颜色等属性。

属性	备注
font_size	字体大小，默认为 48
font	字体名称，例如"Arial" 或"SimSun"
color	文本颜色，默认为白色
gradient	渐变颜色，可以指定多个颜色
slant	倾斜角度，选项有 ITALIC（斜体）
weight	字重，选项有 BOLD（粗体）
t2c	给定文字的特定部分自定义颜色
line_spacing	文本行间距
alignment	文本对齐方式，选项有"center", "left" 等

表 35.1 manim 中的文字样式

1) 颜色相关

(1) color 单色

```python
# manim -pql 02TextAndMathDemo.py TextAndMathDemo
from manim import *

class TextAndMathDemo(Scene):
    def construct(self):
        # 创建各种文本和数学公式对象，并为它们指定不同的颜色
        text_english = Text("hello world", color=RED)
        text_chinese = Text("你好，世界", color=YELLOW)
        text_japanese = Text("こんにちは世界", color=GREEN)

        tex_pythagoras = Tex(r"$a^2 + b^2 = c^2$", color=RED)
        tex_einstein = Tex(r"$E = mc^2$", color=YELLOW)
        tex_euler = Tex(r"$e^{i\pi} + 1 = 0$", color=GREEN)

        # 设置每个文本和公式的位置
        text_english.to_edge(UP)
        text_chinese.next_to(text_english, DOWN)
        text_japanese.next_to(text_chinese, DOWN)

        tex_pythagoras.next_to(text_japanese, DOWN, buff=1)
        tex_einstein.next_to(tex_pythagoras, DOWN)
        tex_euler.next_to(tex_einstein, DOWN)

        # 依次显示文本和公式
        self.play(Write(text_english))
        self.play(Write(text_chinese))
        self.play(Write(text_japanese))

```

```
        self.play(Write(tex_pythagoras))
        self.play(Write(tex_einstein))
        self.play(Write(tex_euler))

        # 保持场景显示一段时间
        self.wait(2)

if __name__ == "__main__":
    from manim import config
    config.background_color = "#fafafa"  # 可选：设置背景颜色
    scene = TextAndMathDemo()
    scene.render()
```

(2) gradient 渐变色

`set_color_by_gradient` 方法可以为文本和公式设置渐变颜色。

```
from manim import *

class GradientTextAndMath(Scene):
    def construct(self):
        # 创建英文文本并设置渐变颜色
        text_english = Text("hello world").set_color_by_gradient(RED, GREEN)
        # 创建中文文本并设置渐变颜色
        text_chinese = Text("你好，世界").set_color_by_gradient(YELLOW, BLUE)
        # 创建日文文本并设置渐变颜色
        text_japanese = Text("こんにちは世界").set_color_by_gradient(BLUE, RED)

        # 创建数学公式并设置渐变颜色
        tex_pythagoras = Tex(r"$a^2 + b^2 = c^2$").set_color_by_gradient(RED, GREEN)
        tex_einstein = Tex(r"$E = mc^2$").set_color_by_gradient(YELLOW, BLUE)
        tex_euler = Tex(r"$e^{i\pi} + 1 = 0$").set_color_by_gradient(BLUE, RED)

        # 设置每个文本和公式的位置
        text_english.to_edge(UP)  # 将英文文本移至屏幕上方
        text_chinese.next_to(text_english, DOWN, buff=0.5)  # 中文文本在英文下方
        text_japanese.next_to(text_chinese, DOWN, buff=0.5)  # 日文 文本在中文下方
```

```
21
22          tex_pythagoras.next_to(text_japanese, DOWN, buff=1)  #
        数学 公式在文本下方
23          tex_einstein.next_to(tex_pythagoras, DOWN, buff=0.5)  #
        公式E=mc^2在其下方
24          tex_euler.next_to(tex_einstein, DOWN, buff=0.5)  #
        欧拉公式 在最下方
25
26          # 使用动画依次显示文本和公式
27          self.play(Write(text_english))
28          self.play(Write(text_chinese))
29          self.play(Write(text_japanese))
30
31          self.play(Write(tex_pythagoras))
32          self.play(Write(tex_einstein))
33          self.play(Write(tex_euler))
34
35          # 保持场景显示2秒
36          self.wait(2)
37
38  if __name__ == "__main__":
39      from manim import config
40      config.background_color = "#fafafa"  # 设置浅色背景
41      scene = GradientTextAndMath()
42      scene.render()
```

(3) 按字符设置颜色

1) 文本的颜色设置：`t2c`：给定文字的特定部分自定义颜色

```
1       # manim -pql 02TextWithColor.py
2       from manim import *
3
4       class TextAndMathWithColors(Scene):
5           def construct(self):
6               # 文本的颜色设置
7               text = Text("JLU.com", t2c={"JL": BLUE, "U":
        RED, "com": GREEN})
8               text.to_edge(UP)  # 将文本移到屏幕的上边缘
9
10              text2 = Text("我爱计算机", t2c={"我": BLUE, "爱":
        RED, "计算机": GREEN})
11
12
13              # 动画显示文本和公式
```

```
            self.play(Write(text))
            self.wait(1)
            self.play(ReplacementTransform(text, text2))

            # 保持场景显示2秒
            self.wait(2)

    if __name__ == "__main__":
        from manim import config
        config.background_color = "#ffffff"  # 设置白色背景
        scene = TextAndMathWithColors()
        scene.render()

```

2) LaTeX 数学公式的颜色设置

```
    # manim -pql 02TextWithColor.py
    from manim import *

    class TextAndMathWithColors(Scene):
        def construct(self):
            # LaTeX 数学公式的颜色设置
            tex = Tex(r"$a^2$", " + ", r"$b^2$", " = ",
    r"$c^2$")
            # 分别为公式中的不同部分设置颜色
            tex.set_color_by_tex("a^2", RED)
            tex.set_color_by_tex("b^2", YELLOW)
            tex.set_color_by_tex("c^2", GREEN)

            # 动画显示文本和公式
            self.play(Write(tex))

            # 保持场景显示2秒
            self.wait(2)

    if __name__ == "__main__":
        from manim import config
        config.background_color = "#ffffff"  # 设置白色背景
        scene = TextAndMathWithColors()
        scene.render()

```

2) 字体相关

(1) font 字体

manim 通过 `font` 属性设置字体。

```python
# manim -pql 01CustomFontText.py CustomFontText
from manim import *

class CustomFontText(Scene):
    def construct(self):
        # 英文文本，使用 Consolas 字体
        text_english = Text("I love", font="Consolas")
        text_english.to_edge(UP)  # 将文本移到屏幕的上方

        # 中文文本，使用华文行楷字体
        text_chinese_xingkai = Text("数学", font="STXingkai")  # 华 文行楷字体
        text_chinese_xingkai.next_to(text_english, DOWN, buff=0.5)  # 将中文放在英文下方

        # 中文文本，使用华文彩云字体
        text_chinese_caiyun = Text("计算机", font="STCaiyun")  # 华 文彩云字体
        text_chinese_caiyun.next_to(text_chinese_xingkai, DOWN, buff=0.5)  # 将“之旅”放在“数学”下方

        # 动画依次显示文本
        self.play(Write(text_english))
        self.play(Write(text_chinese_xingkai))
        self.play(Write(text_chinese_caiyun))

        # 保持场景显示2秒
        self.wait(2)

if __name__ == "__main__":
    from manim import config
    config.background_color = "#fafafa"  # 设置浅色背景
    scene = CustomFontText()
    scene.render()
```

引用本地字体文件

(2) font_size 字体大小

```python
# manim -pql  01TextWithDifferentSize.py TextWithDifferentSizes
from manim import *

class TextWithDifferentSizes(Scene):
    def construct(self):
        # 创建不同大小的文本
        text_1 = Text("I", font_size=20)         # 字体大小为20
        text_2 = Text("love", font_size=30)      # 字体大小为30
        text_3 = Text("数学", font_size=40)       #
    字体大小为40，中文字符
        text_4 = Text("&", font_size=50)         # 字体大小为50
        text_5 = Text("计算机", font_size=60)     #
    字体大小为60，中文字符

        # 调整文本位置，使它们按顺序显示在同一行上
        text_1.to_edge(LEFT)  # 将"I"移到屏幕左边缘
        text_2.next_to(text_1, RIGHT, buff=0.5)  # "love" 放在 "I"
     的右边，间隔0.5
        text_3.next_to(text_2, RIGHT, buff=0.5)  # "数学" 放在
    "love" 的右边，间隔0.5
        text_4.next_to(text_3, RIGHT, buff=0.5)  # "&" 放在 "数学"
     的右边，间隔0.5
        text_5.next_to(text_4, RIGHT, buff=0.5)  # "计算机" 放在
    "&" 的右边，间隔0.5

        # 依次显示每个文本
        self.play(Write(text_1))
        self.play(Write(text_2))
        self.play(Write(text_3))
        self.play(Write(text_4))
        self.play(Write(text_5))

        # 保持场景显示2秒
        self.wait(2)

if __name__ == "__main__":
    from manim import config
    config.background_color = "#fafafa"  # 设置浅色背景
    scene = TextWithDifferentSizes()
    scene.render()
```

(3) weight 字重

```python
# manim -pql 01TextWithDifferentWeights.py TextWithDifferentWeights
from manim import *

class TextWithDifferentWeights(Scene):
    def construct(self):
        # 正常字体权重的英文文本
        text_normal = Text("NORMAL font weight")  # 正常字体
        text_normal.to_edge(UP)  # 将文本移到屏幕的上方

        # 加粗字体权重的英文文本
        text_bold_english = Text("BOLD font weight", weight=BOLD)  # 加粗字体
        text_bold_english.next_to(text_normal, DOWN, buff=0.5)  # 在正常字体下方显示

        # 正常字体权重的中文文本，使用宋体
        text_normal_chinese = Text("正常字重", font="SimSun")  # 正 常宋体
        text_normal_chinese.next_to(text_bold_english, DOWN, buff=0.5)  # 在英文加粗字体下方显示

        # 加粗字体权重的中文文本
        text_bold_chinese = Text("加粗字体", font="SimSun", weight=BOLD)  # 加粗宋体
        text_bold_chinese.next_to(text_normal_chinese, DOWN, buff=0.5)  # 在正常中文字体下方显示

        # 依次显示每个文本
        self.play(Write(text_normal))
        self.play(Write(text_bold_english))
        self.play(Write(text_normal_chinese))
        self.play(Write(text_bold_chinese))

        # 保持场景显示2秒
        self.wait(2)

if __name__ == "__main__":
    from manim import config
    config.background_color = "#fafafa"  # 设置浅色背景
    scene = TextWithDifferentWeights()
    scene.render()
```

(4) slant 倾斜

```python
#  manim -pql 01TextWithItalicAndNormal.py TextWithItalicAndNormal
from manim import *

class TextWithItalicAndNormal(Scene):
    def construct(self):
        # 正常字体权重的英文文本
        text_normal = Text("NORMAL font weight")  # 正常字体
        text_normal.to_edge(UP)  # 将文本移到屏幕的上方

        # 斜体字体权重的英文文本
        text_italic_english = Text("ITALIC font", slant=ITALIC)
    #  斜体字体
        text_italic_english.next_to(text_normal, DOWN, buff=0.5)
    # 在正常字体下方显示

        text_bold_itatic = Text("BOLD ITALIC", slant=ITALIC,
    weight=BOLD)  # 粗斜体字体
        text_bold_itatic.next_to(text_italic_english, DOWN,
    buff=0.5)

        # 正常字体权重的中文文本，使用宋体
        text_normal_chinese = Text("正常字重", font="SimSun")  #
    正 常宋体
        text_normal_chinese.next_to(text_bold_itatic, DOWN,
    buff=0.5)

        # 斜体字体权重的中文文本
        text_italic_chinese = Text("斜体", font="SimSun",
    slant=ITALIC)  # 斜体宋体
        text_italic_chinese.next_to(text_normal_chinese, DOWN,
    buff=0.5)  # 在正常中文字体下方显示

        # 斜体加粗字体权重的中文文本
        text_bold_italic_chinese = Text("粗斜体", font="SimSun",
    slant=ITALIC, weight=BOLD)
        text_bold_italic_chinese.next_to(text_italic_chinese,
    DOWN, buff=0.5)

        # 依次显示每个文本
        self.play(Write(text_normal))
        self.play(Write(text_italic_english))
        self.play(Write(text_bold_itatic))
        self.play(Write(text_normal_chinese))
        self.play(Write(text_italic_chinese))
        self.play(Write(text_bold_italic_chinese))
```

```
36
37
38          # 保持场景显示2秒
39          self.wait(2)
40
41  if __name__ == "__main__":
42      from manim import config
43      config.background_color = "#fafafa"  # 设置浅色背景
44      scene = TextWithItalicAndNormal()
45      scene.render()
```

(5) 按字符设置

1) `t2f`：按字符设置字体

2) `t2s`：按字符设置斜体

3) `t2w`：按字符设置粗体

```
1   # manim -pql 01TextWithStyles.py TextWithStyles
2   from manim import *
3
4   class TextWithStyles(Scene):
5       def construct(self):
6           # 创建带有不同样式的文本
7           text = Text(
8               "I love 数学计算机",  # 目标文本
9               t2f={"数学": "STXingkai", "之旅": "SimSun"},  #
    为"数学"部分使用STXingkai字体，为"之旅"使用SimSun字体
10              t2s={"I": ITALIC},  # 将"ma"部分设置为斜体
11              t2w={"love": BOLD},   # 将"nim"部分设置为加粗
12          )
13
14          # 将文本放置在屏幕中央
15          text.move_to(ORIGIN)
16
17          # 动画显示文本
18          self.play(Write(text))
19
20          # 保持场景显示2秒
21          self.wait(2)
22
23  if __name__ == "__main__":
24      from manim import config
```

```
25      config.background_color = "#fafafa"  # 设置浅色背景
26      scene = TextWithStyles()
27      scene.render()
```

第三十六章 manim 动画效果

一、 创建

展示某个元素或者文字时，一下子就全显示出来会显得比较突兀，通过创建效果的动画，让各个元素的出现更加的自然。常用的创建效果动画主要有：

1) `Create()`：创建效果，元素从无到有。

2) `Write()`：书写效果，元素逐渐显示。

3) `FadeIn()`：淡入效果，元素从透明到不透明。

1) Create 创建图形

`Create()` 用于创建图形，绘制时图形逐步显示出来。例如，创建一个圆形：

```
# manim -pql 01CreateCircle.py
from manim import *

class CreateCircle(Scene):
    def construct(self):
        circle = Circle()
        self.play(Create(circle))
```

```
# manim -pql 01CreateSquare.py CreateSquare
from manim import *
class CreateSquare(Scene):
    def construct(self):
        s = Square(side_length=2, color=BLUE)
        self.play(Create(s))
```

2) Write 书写效果

```python
# manim -pql 02WriteText.py WriteText
from manim import *

class WriteText(Scene):
    def construct(self):
        text = Text(
            "Welcome to Manim",
            t2c={"Welcome": BLUE, "Manim": RED},
            t2f={"Manim": "STCaiyun"},
        )
        self.play(Write(text))
```

3) FadeIn 淡入效果

```python
# manim -pql 01FadeInExample.py FadeInExample
from manim import *

# 使用不同的类名以避免与 Manim 内置类的冲突
class FadeInExample(Scene):
    def construct(self):
        # 创建文本对象
        text = Text("Welcome to Manim")

        # 渐入显示文本
        self.play(FadeIn(text))
        self.wait(2)

        # 渐出文本
        self.play(FadeOut(text))

        # 创建一个蓝色的正方形
        square = Square(side_length=2, color=BLUE,
    fill_opacity=0.6)
```

```
19
20          # 渐入显示正方形
21          self.play(FadeIn(square))
22          self.wait(2)
```

4) add 添加元素

`add()` 方法用于添加元素，可以一次添加多个元素。没有动画效果，元素会立即显示出来。

例如，添加一个圆形和一个正方形：

```
1  # manim -pql 01AddExample.py AddExample
2  from manim import *
3
4  class AddExample(Scene):
5      def construct(self):
6          # 创建两个对象
7          circle = Circle(color=BLUE)
8          square = Square(side_length=2, color=RED)
9
10         # 添加对象
11         self.add(circle, square)
```

二、 销毁

销毁的效果一般用在移除图形和文字的场合。

1) Uncreate 擦除图形

```
1  # manim -pql 02UncreateSquare.py UncreateSquare
2  from manim import *
3
4  class UncreateSquare(Scene):
5      def construct(self):
6          square = Square(side_length=2, color=BLUE)
7          self.add(square)
8          self.wait(0.5)
```

```
9           self.play(Uncreate(square))
```

2) Unwrite 擦除文字

```
1  # manim -pql 02UnwriteText.py UnwriteText
2  from manim import *
3
4  class UnwriteText(Scene):
5      def construct(self):
6          text = Text(
7              "Welcome to Manim",
8              t2c={"Welcome": BLUE, "Manim": RED},
9              t2f={"Manim": "STCaiyun"},
10         )
11         self.add(text)
12         self.wait(0.5)
13         self.play(Unwrite(text))
```

3) FadeOut 淡出效果

```
1  # manim -pql 02FadeOutExample.py FadeOutExample
2  from manim import *
3
4  class FadeOutExample(Scene):
5      def construct(self):
6          # 创建两个对象
7          circle = Circle(color=BLUE)
8          square = Square(side_length=2, color=RED)
9
10         # 渐出对象
11         self.play(FadeOut(circle))
12         self.wait(0.5)
13         self.play(FadeOut(square))
```

三、 移动

1) shift 移动指定距离

```python
# manim -pql 03ShiftExample.py ShiftExample
from manim import *
class ShiftExample(Scene):
    def construct(self):
        # 创建两个对象
        circle = Circle(color=RED)
        square = Square(side_length=1, color=BLUE,
    fill_opacity=0.6)

        # 移动指定距离
        self.play(circle.animate.shift(UP))     # 上移1个单位
        self.play(circle.animate.shift(RIGHT))  # 右移1个单位
        self.play(circle.animate.shift(DOWN*3)) # 下移3个单位
        self.play(circle.animate.shift(LEFT*2)) # 左移2个单位
        self.wait(0.5)
```

2) move_to 移动到指定位置

```python
# manim -pql 03MoveToExample.py MoveToExample
from manim import *

class MoveToExample(Scene):
    def construct(self):
        # 创建两个对象
        circle = Circle(color=BLUE)
        square = Square(side_length=2, color=RED)

        # 移动到指定位置
        self.play(circle.animate.move_to(LEFT))
        self.wait(0.5)
        self.play(circle.animate.move_to(UP))
        self.wait(0.5)
        self.play(circle.animate.move_to(RIGHT))
        self.wait(0.5)
        self.play(circle.animate.move_to(DOWN))
        self.wait(0.5)
```

```
        self.play(circle.animate.move_to(ORIGIN))
        self.wait(0.5)
        self.play(square.animate.move_to(RIGHT))
        self.wait(0.5)
        self.play(square.animate.move_to(ORIGIN))
        self.wait(0.5)
        self.play(square.animate.move_to(ORIGIN+UP))
        self.wait(0.5)
```

3) MoveAlongPath 沿路径移动

常用的动画一般都是直线运动或者圆弧运动，而 `MoveAlongPath` 可以让元素沿着任意的函数轨迹来运动。

```
# manim -pql 01CosineGraph.py CosineGraph
from manim import *
import numpy as np

class CosineGraph(Scene):
    def construct(self):
        # 创建一个复杂的余弦函数图像
        cos_func = FunctionGraph(
            lambda t: np.cos(t) + 0.5 * np.cos(7 * t) + (1 / 7) * np.cos(14 * t),
            color=RED,  # 设定图像的颜色为红色
            x_range=[-PI, PI],  # 设定x轴的范围
        )

        # 将函数图像添加到场景中
        self.add(cos_func)

        # 创建一个点并放置在曲线的起点位置
        d1 = Dot(cos_func.get_start(), color=BLUE)

        # 将点添加到场景中
        self.add(d1)

        # 使点沿着余弦函数图像移动
        self.play(MoveAlongPath(d1, cos_func), rate_func=linear, run_time=3)

        # 暂停2秒以查看最终效果
        self.wait(2)

if __name__ == "__main__":
```

```
    scene = CosineGraph()
    scene.render()
```

4) TracedPath 保留运动路径

```
# manim -pql 01TracedDotScene.py TracedDotScene
from manim import *

class TracedDotScene(Scene):
    def construct(self):
        # 创建一个点，初始位置在右边 (2, 0, 0)
        d = Dot(RIGHT * 2, color=BLUE)

        # 创建一个路径跟踪器，用于记录点的运动轨迹
        # TracedPath 会跟踪点的中心点（get_center方法）
        b = TracedPath(d.get_center, stroke_color=YELLOW,
    stroke_width=4)

        # 将点和路径添加到场景中
        self.add(d, b)

        # 播放点的动画，点先沿着一个圆弧（向上的圆弧）移动并向左移动
        self.play(d.animate(path_arc=PI / 4).shift(LEFT * 2),
    run_time=2)

        #
    播放点的动画，点沿着另一个圆弧（向下的圆弧）移动并继续向左移动
        self.play(d.animate(path_arc=-PI / 4).shift(LEFT * 2),
    run_time=2)

        # 暂停2秒以便查看最终效果
        self.wait(2)

if __name__ == "__main__":
    scene = TracedDotScene()
    scene.render()
```

四、 Rotate 旋转

1) 自转

```
# manim -pql 04RotateExample.py RotateExample
from manim import *

class RotateExample(Scene):
    def construct(self):
        # 创建一个蓝色的正方形
        square = Square(side_length=2, color=BLUE,
fill_opacity=0.6)

        # 显示正方形
        self.play(FadeIn(square))

        # 逆时针旋转90度
        self.play(square.animate.rotate(PI / 2))
        self.wait(0.5)

        # 顺时针自转一周 (360度)
        self.play(Rotate(square, angle=-2 * PI),
run_time=2)  # 自旋转1周
        # self.play(square.animate.rotate(-2 * PI),
run_time=2) # 这条语句没有效果
        self.wait(0.5)

```

2) 围绕某个点旋转

```
# manim -pql 04RotateSquareWithRadius.py
RotateSquareWithRadius
from manim import *

class RotateSquareExample(Scene):
    def construct(self):
        # 创建一个蓝色的正方形，并将其上移
        s = Square(side_length=1, color=BLUE,
fill_opacity=0.6).shift(UP)

        # 将正方形添加到场景中
        self.add(s)

```

```
            # 1. 绕屏幕中心 (ORIGIN) 旋转一周
            self.play(Rotate(s, angle=2 * PI,
        about_point=ORIGIN), run_time=2)
            self.wait(0.5)

            # 2. 绕 [2, 3, 0] 这个点旋转 3/2
            self.play(Rotate(s, angle=3 * PI / 2,
        about_point=[2, 3, 0]), run_time=2)
            self.wait(0.5)

```

五、 变换

变换也是使用的比较多的一种动画效果。尤其是在数学视频中，经常遇到随着参数的变化，图形随之变形；或者反之。

1) 图形变换

(1) TransformFromCopy 保留原图形

```
# manim -pql 01TransformExample.py TransformExample
from manim import *

class TransformExample(Scene):
    def construct(self):
        # 创建一个蓝色方形，并向左移动2个单位
        s = Square(side_length=1, color=BLUE,
    fill_opacity=0.6).shift(LEFT * 2)

        # 创建一个红色圆形，并向右移动2个单位
        c = Circle(radius=1, color=RED,
    fill_opacity=0.6).shift(RIGHT * 2)

        # 将方形添加到场景中
        self.add(s)
        self.wait(0.5)  # 暂停0.5秒

        # 使用 TransformFromCopy 将方形复制并变换为圆形
        self.play(TransformFromCopy(s, c))

        # 暂停查看最终效果
        self.wait(2)
```

(2) ReplacementTransform 原图形直接变换成新的图形

```python
# manim -pql 01ShapeTransform.py ShapeTransform
from manim import *

class ShapeTransform(Scene):
    def construct(self):
        # 创建一个蓝色方形，并向左移动2个单位
        s = Square(side_length=1, color=BLUE,
    fill_opacity=0.6).shift(LEFT * 2)

        # 创建一个红色圆形，并向右移动2个单位
        c = Circle(radius=1, color=RED,
    fill_opacity=0.6).shift(RIGHT * 2)

        # 将方形添加到场景中
        self.add(s)
        self.wait(0.5)  # 暂停0.5秒

        # 使用 ReplacementTransform 从方形逐渐变换为圆形
        self.play(ReplacementTransform(s, c))

        # 暂停查看最终效果
        self.wait(2)

if __name__ == "__main__":
    from manim import *
    scene = ShapeTransform()
    scene.render()
```

2) 文字变换

(1) TransformFromCopy 保留原文字

```python
# manim -pql 02TransformTextExample.py TransformTextExample
from manim import *
```

```python

class TransformTextExample(Scene):
    def construct(self):
        # 创建一个文本对象，并设置字体大小和颜色
        text = Text("Hello, World!", font_size=48, color=BLUE)

        # 将文本添加到场景中
        self.add(text)
        self.wait(0.5)  # 暂停0.5秒

        # 使用 TransformFromCopy 将文本复制并变换为新文本
        test2 = Text("Goodbye, World!", font_size=48, color=RED)
        test2.next_to(text, DOWN)
        self.play(TransformFromCopy(text, test2))

        # 暂停查看最终效果
        self.wait(2)
```

```python
# manim -pql 02TransformTextExample.py TransformTextExample
from manim import *

class TransformTextExample(Scene):
    def construct(self):
        # 创建一个文本对象，并设置字体大小和颜色
        t1 = Tex(r"$(a+b)^2$").shift(UP)
        t2 = Tex(r"$a^2 + 2ab + b^2$")

        # 将文本添加到场景中
        self.add(t1)
        self.wait(0.5)  # 暂停0.5秒

        # 使用 TransformFromCopy 将文本复制并变换为新文本
        self.play(TransformFromCopy(t1, t2))

        # 暂停查看最终效果
        self.wait(2)
```

(2) ReplacementTransform 原文字直接变换成新的文字

```python
# manim -pql 02TextTransform.py TextTransform
from manim import *

class TextTransform(Scene):
    def construct(self):
        # 创建一个文本对象，并设置字体大小和颜色
        text = Text("Hello, World!", font_size=48, color=BLUE)

        # 将文本添加到场景中
        self.add(text)
        self.wait(0.5)  # 暂停0.5秒

        # 使用 ReplacementTransform 从文本逐渐变换为另一个文本
        test2 = Text("Goodbye, World!", font_size=48, color=RED)
        test2.next_to(text, DOWN)
        self.play(ReplacementTransform(text, test2))

        # 暂停查看最终效果
        self.wait(2)
```

```python
# manim -pql 02TextTransform.py TextTransform
from manim import *

class TextTransform(Scene):
    def construct(self):
        # 创建一个文本对象，并设置字体大小和颜色
        t1 = Tex(r"$(a+b)^2$").shift(UP)
        t2 = Tex(r"$a^2 + 2ab + b^2$")

        # 将文本添加到场景中
        self.add(t1)
        self.wait(0.5)  # 暂停0.5秒

        # 使用 ReplacementTransform 从文本逐渐变换为另一个文本
        self.play(ReplacementTransform(t1, t2))

        # 暂停查看最终效果
        self.wait(2)
```

六、 动画效果属性

在创建动画时，Manim 提供了多种动画效果和过渡样式。可以自定义动画的播放速度、曲线变化、方向等。

属性	备注
run_time	动画持续时间，默认 1 秒
rate_func	动画速率函数，用于控制动画的速度曲线
lag_ratio	延迟比例，控制多个动画对象的时间偏移
direction	方向，常用于旋转或移动动画
path_arc	移动路径的弧度，通常用于圆周移动
reverse	是否反向播放动画

表 36.1 manim 中的动画样式

七、 光照样式

Manim 允许用户添加光照效果，使用 OpenGL 渲染时尤其有用。光照可以创建更具深度感的三维效果。

属性	备注
ambient_light	环境光，控制整体场景的基础光照强度
directional_light	定向光，模拟来自特定方向的光源
point_light	点光源，模拟来自单点的光源
shadow	控制是否生成阴影
gloss	表面光泽度
reflection_opacity	反射的透明度

表 36.2 manim 中的光照样式

八、 三维对象样式

在三维空间中，Manim 提供了一些额外的样式，用于调整三维对象的外观。

属性	备注
shininess	物体表面光泽度
reflectiveness	物体的反射率
ambient_strength	环境光照强度
diffuse_strength	漫反射光强度
specular_strength	镜面反射光强度

表 36.3 manim 中的三维对象样式

九、 动画组合

在常用的动画效果中，一般都是逐个显示各个动画效果，manim 中提供了几个控制动画组合的方式。用的较多的两个分别是：

1) 多个动画组合时，设置每个动画的延迟启动时间

2) 多个动画组合时，前一个动画成功执行后再执行下一个动画

1) LaggedStart 延迟启动

```python
# manim -pql 01LaggedStart_demo.py LaggedStart_demo
from manim import *

# 创建一个场景
class DottedLineAnimation(Scene):
    def construct(self):
        # 创建两个点，d1为红色，d2为默认颜色
        d1 = Dot(LEFT * 2 + UP, color=RED)
        d2 = Dot(LEFT * 2 + DOWN)

        #
    创建一条虚线，颜色为绿色，虚线长度为0.1，虚线与空隙比为0.8，宽度为2
        dl = DashedLine(
            RIGHT * 2 + UP * 3,
            RIGHT * 2 + DOWN * 3,
            dash_length=0.1,
            dashed_ratio=0.8,
            stroke_width=2,
            color=GREEN,
        )

        # 将两个点和虚线添加到场景中
        self.add(d1, d2, dl)
        self.wait(0.5)

        # 创建两个动画:
        # d1以1秒时间向右移动4个单位
        # d2延迟0.5秒启动，但以0.5秒时间向 右移动4个单位
        ani1 = d1.animate(run_time=1).shift(RIGHT * 4)
        ani2 = d2.animate(run_time=0.5).shift(RIGHT * 4)

        # 使用LaggedStart函数实现两个动画之间的延迟启动，
        # lag_ratio为0.5表示动画之间有0.5秒的间隔
        self.play(LaggedStart(ani1, ani2, lag_ratio=0.5))

```

```
        # 场景结束时等待0.5秒
        self.wait(0.5)

# 使用Manim的命令行渲染代码
if __name__ == "__main__":
    scene = DottedLineAnimation()
    scene.render()
```

2) Succession 顺序启动

```
# manim -pql 01SuccessionStart_demo.py SuccessionStart_demo
from manim import *

# 创建一个场景
class DotMovement(Scene):
    def construct(self):
        # 创建四个点，并设置它们的颜色、位置和半径
        dot1 = Dot(point=LEFT * 2 + UP * 2, radius=0.16, color=BLUE)
        dot2 = Dot(point=LEFT * 2 + DOWN * 2, radius=0.16, color=MAROON)
        dot3 = Dot(point=RIGHT * 2 + DOWN * 2, radius=0.16, color=GREEN)
        dot4 = Dot(point=RIGHT * 2 + UP * 2, radius=0.16, color=YELLOW)

        # 将四个点添加到场景中
        self.add(dot1, dot2, dot3, dot4)

        # 创建动画：点1移动到点2，点2移动到点3，点3移动到点4，点4移动到点1
        self.play(
            Succession(
                dot1.animate.move_to(dot2),  # 点1移动到点2的位置
                dot2.animate.move_to(dot3),  # 点2移动到点3的位置
                dot3.animate.move_to(dot4),  # 点3移动到点4的位置
                dot4.animate.move_to(dot1),  # 点4移动到点1的位置
            )
        )

        # 等待一会儿以查看最终效果
        self.wait(1)

# 使用Manim的命令行渲染代码
```

```
if __name__ == "__main__":
    scene = DotMovement()
    scene.render()
```

十、 动画联动

动画联动是比动画组合更复杂的一种模式，动画组合时，每个动画其实是固定的，只是启动的时机不一样。而在动画联动中，一个动画如何运行是完全基于另一个动画的，而不是实现预设的。

```
# manim -pql 01AngleRotation.py AngleRotation
from manim import *

class AngleRotation(Scene):
    def construct(self):
        # 设置旋转中心点为左侧
        rotation_center = LEFT

        # 创建一个角度值追踪器，初始值为110度
        theta_tracker = ValueTracker(110)

        # 创建固定的水平线
        line1 = Line(LEFT, RIGHT)
        # 创建可旋转的线段
        line_moving = Line(LEFT, RIGHT)
        line_ref = line_moving.copy()  # 参考线段，用于保持线条的一致性

        # 根据初始角度值旋转线条
        line_moving.rotate(theta_tracker.get_value() * DEGREES, about_point=rotation_center)

        # 绘制角度并标记
        a = Angle(line1, line_moving, radius=0.5, other_angle=False)
        tex = MathTex(r"\theta").move_to(
            Angle(
                line1, line_moving, radius=0.5 + 3 * SMALL_BUFF, other_angle=False
            ).point_from_proportion(0.5)
        )

        # 创建显示角度的文本，红色字体
        theta_tex = Tex(
```

```
            r"$\theta = $",
            str(int(theta_tracker.get_value())),
            r"$^\circ$",
            font_size=50,
            color=RED,
        ).shift(UP * 2)

        # 将线条、角度和文字添加到场景中
        self.add(line1, line_moving, a, tex, theta_tex)
        self.wait()

        # 为线条设置更新器，以便实时旋转线条
        line_moving.add_updater(
            lambda x: x.become(line_ref.copy()).rotate(
                theta_tracker.get_value() * DEGREES,
    about_point=rotation_center
            )
        )

        # 为角度添加更新器，确保角度变化时自动更新
        a.add_updater(
            lambda x: x.become(Angle(line1, line_moving,
    radius=0.5, other_angle=False))
        )

        # 为角度符号添加更新器，确保符号随角度移动
        tex.add_updater(
            lambda x: x.move_to(
                Angle(
                    line1, line_moving, radius=0.5 + 3 *
    SMALL_BUFF, other_angle=False
                ).point_from_proportion(0.5)
            )
        )

        # 为角度显示的文本添加更新器，确保文本内容与角度值实时同步
        theta_tex.add_updater(
            lambda x: x.become(
                Tex(
                    r"$\theta = $",
                    str(int(theta_tracker.get_value())),
                    r"$^\circ$",
                    font_size=50,
                    color=RED,
                ).shift(UP * 2)
            )
        )

        # 动画部分：将角度从110度变到90度
        self.play(theta_tracker.animate.set_value(90))
```

```
78          # 动画部分：增加角度值到230度，动画时间为3秒
79          self.play(theta_tracker.animate.increment_value(140),
        run_time=3)
80
81          # 最后等待以展示结果
82          self.wait()
83
84  # 使用Manim的命令行渲染代码
85  if __name__ == "__main__":
86      scene = AngleRotation()
87      scene.render()
```

例子 36.1

导数推导的证明动画。

方法:

```
1       from manim import *
2
3       class DerivationDerivative(Scene):
4           def construct(self):
5               # 首先展示函数乘积导数公式
6               self.MyTopic()
7               self.wait(1)
8               # 然后绘制坐标系并展示曲线切线的变化
9               self.MyAxes()
10              self.wait(2)
11              # 最后展示公式的推导过程
12              self.MyFormula()
13              self.wait(5)
14
15          # 函数乘积导数公式的展示
16          def MyTopic(self):
17              # 创建标题文本
18              s = Text("证明uv函数乘积的导数公式:\t\t",
        font_size=30, color=BLUE)
19              s.to_corner(UP + LEFT)  # 将标题放置在左上角
20              self.play(Write(s))  # 播放文本写入动画
21
22              # 创建乘积导数公式的数学公式
23              m = MathTex("(uv)\' = ", "u\'v", " + ", "uv\'",
        font_size=50)
24              m.next_to(s, RIGHT)  # 将公式放置在标题旁边
25              self.play(Write(m))  # 播放公式写入动画
```

```
26
27              # 给公式中的各部分加上矩形框强调
28              framebox1 = SurroundingRectangle(m[1], buff=0.1)
        # 高亮u'v 部分
29              framebox2 = SurroundingRectangle(m[3], buff=0.1)
        # 高亮uv' 部分
30              framebox3 = SurroundingRectangle(m, buff=0.1)  #
       高亮整个公 式
31              self.play(Create(framebox1))
32              self.play(ReplacementTransform(framebox1,
       framebox2))
33              self.play(ReplacementTransform(framebox2,
       framebox3))
34
35          # 绘制坐标系及曲线，展示切线及导数的几何意义
36          def MyAxes(self):
37              # 创建坐标系，设定x轴、y轴的范围及刻度
38              ax = Axes(
39                  x_range=[-4, 4], y_range=[0, 5],
40                  x_axis_config={"numbers_to_include": [0, 1,
       2]},  # x轴 刻度
41                  y_axis_config={"numbers_to_include": [0, 1,
       4]},  # y轴 刻度
42                  tips=True  # 显示箭头
43              )
44
45              # 创建曲线y = x^2
46              curve = ax.plot(lambda x: x ** 2, x_range=[-2,
       2], color=RED)
47              self.play(Create(ax), Create(curve))
48              self.wait(2)
49
50              # 在曲线上标出一个点，设定为x = 1处
51              point = Dot(ax.i2gp(1, curve), color=YELLOW)
52              self.play(Create(point))
53              self.wait(2)
54
55              # 绘制x=1处的切线，展示导数的几何意义
56              tangent_line = ax.get_secant_slope_group(
57                  1, curve, dx_label="\\Delta x",
       dy_label="\\Delta y",
58                  dx_line_color=YELLOW, dy_line_color=YELLOW
59              )
60              self.play(Create(tangent_line), runtime=3)
61              self.wait(2)
62
63              #
       缩小整个图像并将其移到左上角，为后续公式推导留出空间
64              group = VGroup(ax, curve, tangent_line, point)
65              self.play(group.animate.scale(0.4))
```

```
66              self.play(group.animate.shift(np.array([4.5, 3,
    0])))
67              self.wait(2)
68
69          # 详细展示公式推导过程
70          def MyFormula(self):
71              # 定义公式推导的各个步骤
72              s1 = Tex(r"$\frac{dy}{dx}=\frac{\Delta y}{\Delta
    x} = \lim\limits_{\Delta x\to0} \frac{f(x+\Delta
    x)-f(x)}{\Delta x}$")
73              s2 = Tex(r"$=\lim\limits_{\Delta x\to0}
    \frac{u(x+ \Delta x)v(x+ \Delta x)-u(x)v(x)}{\Delta x}$")
74              s3 = Tex(r"$=\lim\limits_{\Delta x\to0}
    \frac{u(x+ \Delta x)v(x+ \Delta x)-u(x)v(x+\Delta x) +
    u(x)v(x+\Delta x)-u(x)v(x)}{\Delta x}$")
75              s4 = Tex(r"$=\lim\limits_{\Delta x\to0}
    \frac{u(x+ \Delta x)-u(x)}{\Delta x}v(x+\Delta
    x)+u(x)\lim\limits_{\Delta x\to0} \frac{v(x+ \Delta
    x)-v(x)}{\Delta x}$")
76              s5 = Tex(r"$=u'v+uv'$", font_size=100,
    color=YELLOW)
77
78              # 设置颜色并将公式1移到上方
79              s1.set_color([BLUE, YELLOW, RED])
80              s1.to_edge(np.array([-3, 3, 0]))
81              self.play(Write(s1))
82
83              # 依次展示后续的公式推导步骤
84              s2.next_to(s1, DOWN, aligned_edge=LEFT)
85              self.play(Write(s2))
86
87              s3.next_to(s2, DOWN, aligned_edge=LEFT)
88              self.play(Write(s3))
89
90              # 对第3步公式进行高亮处理
91              sr = SurroundingRectangle(s3, color=PINK,
    fill_opacity=0.2)
92              self.play(Create(sr))
93
94              s4.next_to(s3, DOWN, aligned_edge=LEFT)
95              self.play(Write(s4))
96
97              # 最后展示最终的结果公式
98              s5.next_to(s4, DOWN, aligned_edge=LEFT)
99              self.play(Write(s5))
100
101             # 对最终公式进行高亮处理
102             sr1 = SurroundingRectangle(s5)
103             self.play(Create(sr1))
104
```

十一、 相机

相机（Camera）在二维的场景下使用不多，一般在 3D 场景中提及的比较多。相机相当于我们看动画的视角，简单来理解的话，相当于我们的眼睛（实际情况会复杂一些，相机还有其他一些辅助功能）。

默认的相机焦点在屏幕的中心位置，相机默认是以俯视的视角查看所有的元素。

之前的介绍的常用动画效果和高级动画效果，都没有对相机进行调整过，所以，是通过移动和变换各个元素来实现动画效果。

☞ 相机则是另一种制作动画的方式，它不改变元素在屏幕或者说在坐标系中的位置，通过改变相机的位置和角度来实现动画效果。

1) 相机概念

举个现实世界中的简单例子，如果有个杯子，我们可以通过转动杯子来从各个角度观察杯子；而如果是一栋楼的话，我们无法移动它，只能围着楼走一圈来观察它，这个过程就相当于移动相机。

2) 相机移动

相机移动常用的两种方式：

1) 移动焦点: 改变相机的焦点，焦点在那个元素，那个元素就会在屏幕中心

2) 改变视野: 改变相机与元素的距离，离得越远，物体越小。

(1) move_to 移动焦点

manim 中移动焦点要继承 `MovingCameraScene` 类，
然后通过 `self.camera.frame.animate.move_to` 函数来移动焦点。

```python
# manim -pql 03CameraSample1.py CameraSample1

from manim import *

class CameraSample1(MovingCameraScene):
    def _move_focus(self):
        s = Square(color=RED, fill_opacity=0.5)
        t = Triangle(color=GREEN, fill_opacity=0.5)
```

```
        vg = VGroup(s, t)
        vg.arrange(RIGHT, buff=MED_LARGE_BUFF)
        self.add(vg)

        self.play(self.camera.frame.animate.move_to(s))
        self.play(self.camera.frame.animate.move_to(t))
        self.play(self.camera.frame.animate.move_to(vg))

    def construct(self):
        self._move_focus()

        self.wait()
```

(2) set 改变视野

改变视野通过`self.camera.frame.animate.set`方法，通过这个方法设置视野的宽度，可以形成缩放元素的效果。

```
# manim -pql 03CameraSample1.py CameraSample1
from manim import *

class CameraSample1(MovingCameraScene):
    def _scale(self):
        s = Square(color=RED, fill_opacity=0.5)
        self.add(s)

        self.camera.frame.save_state()
        self.play(self.camera.frame.animate.set(width=s.width *
    2))
        self.wait(0.3)
        self.play(self.camera.frame.animate.set(width=s.width *
    8))
        self.wait(0.3)
        self.play(Restore(self.camera.frame))

    def construct(self):
        self._scale()

        self.wait()
```

3) 鹰眼效果

鹰眼的效果是通过两个相机来实现的，两个相机的焦点一样，但是视野不一样。鹰眼效果一般用在提供全局视图的场合，特别是当元素特别多的时候。

```python
# manim -pql 03CameraSample1.py CameraSample1
from manim import *

class CameraSample2(ZoomedScene):
    def __init__(self, **kwargs):
        ZoomedScene.__init__(
            self,
            zoom_factor=3,
            zoomed_display_height=1,
            zoomed_display_width=2,
            image_frame_stroke_width=5,
            zoomed_camera_config={
                "default_frame_stroke_width": 3,
            },
            **kwargs
        )

    def construct(self):
        s = Square(color=RED, fill_opacity=0.5, side_length=1.5)
        t = Triangle(color=GREEN, fill_opacity=0.5).scale(0.5)
        vg = VGroup(s, t)
        vg.arrange(RIGHT, buff=SMALL_BUFF)
        self.add(vg)
        self.activate_zooming(animate=False)
        self.play(s.animate.shift(LEFT), t.animate.shift(RIGHT))
        self.play(s.animate.rotate(2 * PI / 3),
    t.animate.rotate(PI / 2))
        self.play(s.animate.shift(RIGHT), t.animate.shift(LEFT))
        self.wait()
```

4) 追踪物体

追踪物体就是将相机的焦点定位在移动的物体上，就像我们坐在火车上的感觉一样，那时，我们觉得火车没动，而是车外的风景不断向后移动。

```python
# manim -pql 03CameraSample1.py CameraSample1
```

```python
from manim import *

class CameraSample3(MovingCameraScene):
    def construct(self):
        # 保存当前摄像机状态
        self.camera.frame.save_state()

        # 创建一个正弦函数图像
        graph = FunctionGraph(
            lambda x: np.sin(x),  # 正弦函数
            x_range=[-3, 3],      # x范围
            color=RED,            # 颜色为红色
        )

        # 创建一个点，并将其放置在曲线的起始位置
        d = Dot(graph.get_start(), color=BLUE)
        self.add(graph, d)  # 将图形和点添加到场景中

        # 动画：摄像机缩放并移动到点的位置
        self.play(self.camera.frame.animate.scale(0.5).move_to(d))

        # 定义一个更新器，使摄像机帧与点同步移动
        def update_curve(mob):
            mob.move_to(d.get_center())  # 更新摄像机的位置到点的中心

        # 为摄像机添加更新器
        self.camera.frame.add_updater(update_curve)

        # 让点沿着曲线移动，并让摄像机跟随点
        self.play(MoveAlongPath(d, graph), rate_func=linear, run_time=2)

        # 移除摄像机更新器，停止跟随点的移动
        self.camera.frame.remove_updater(update_curve)

        # 恢复摄像机到初始状态
        self.play(Restore(self.camera.frame))
```

5) 3D 场景

相机其实主要就是应用在 3D 场景中的，所以 manim 的 3D 场景类 ThreeDScene 中提供了一个非常方便的移动相机的方法 `move_camera`。

```python

```

```python
# manim -pql 03CameraSample1.py CameraSample1
from manim import *

class CameraSample4(ThreeDScene):
    def construct(self):
        axes = ThreeDAxes()
        sphere = Surface(
            lambda u, v: np.array(
                [
                    1.5 * np.cos(u) * np.cos(v),
                    1.5 * np.cos(u) * np.sin(v),
                    1.5 * np.sin(u),
                ]
            ),
            v_range=[0, TAU],
            u_range=[-PI / 2, PI / 2],
            checkerboard_colors=[BLUE_D, BLUE_E],
            resolution=(15, 32),
        )
        self.add(axes, sphere)

        self.move_camera(phi=75 * DEGREES, theta=30 * DEGREES)
        self.move_camera(zoom=1.5)
        self.move_camera(zoom=0.5)
        self.wait()
```

第七部分

互联

第三十七章 urllib

一、 简介

`urllib`是 Python 的一个标准库，它提供了一系列用于操作 URL 的功能。它包含以下四个模块：

1) `urllib.request`：用于打开和读取 URL。

2) `urllib.error`：包含了`urllib.request`模块中产生的异常。

3) `urllib.parse`：用于解析 URL。

4) `urllib.robotparser`：用于解析 robots.txt 文件。

二、 urllib.request

`urllib.request`模块提供了打开和读取 URL 的方法

```
# 错误导入
import urllib
urllib.request

# 正确导入
from urllib import request

urllib.request.urlopen(url, data=None, [timeout, ]*, cafile=None,
    capath=None, cadefault=False, context=None)
'''
url: URL地址
data: POST请求的数据
timeout: 超时时间
cafile: CA证书
capath: CA证书路径
cadefault: 是否使用默认的CA证书
context: SSL上下文
'''
```

```
from urllib import request

url = 'http://www.baidu.com'
```

```
response = request.urlopen(url,timeout=1) #
    打开URL,timeout=1表示超时时间为1s
print(response.read().decode('utf-8'))    # 读取URL内容并解码

# 获取状态码
print(response.status)
```

1) Get

httpbin是一个 HTTP 测试网站，可以用来测试 HTTP 请求，如下是一个简单的例子：`Get`请求: 是从指定的资源请求数据。

```
from urllib import request

url = 'https://httpbin.org/get'
response = request.urlopen(url,timeout=1) #
    打开URL,timeout=1表示超时时间为1s
print(response.read().decode('utf-8'))    # 读取URL内容并解码
```

2) Post

`Post`请求: 向指定的资源提交数据进行处理请求（例如提交表单或上传文件）。数据被包含在请求体中。POST 请求可能会导致新的资源的建立和/或已有资源的修改。

```
from urllib import request, parse

url = 'https://httpbin.org/post'
data = bytes(parse.urlencode({'word': 'hello'}), encoding='utf8')
response = request.urlopen(url, data=data, timeout=1)
print(response.read().decode('utf-8'))
```

3) Http Headers

`Http Headers`是 HTTP 请求和响应的头部信息，可以通过`add_header`方法添加头部信息。

```
from urllib import request

url = 'https://httpbin.org/get'
headers = {
    'User-Agent': 'Mozilla/4.0 (compatible; MSIE 5.5; Windows NT)',
    'Host': 'httpbin.org',
    'Accept': '*/*',
    'Accept-Language': 'zh-CN,zh;q=0.9',
    'Accept-Encoding': 'gzip, deflate',
    'Connection': 'keep-alive',
```

```
    'Cookie': '_gauth=1; _ga=GA1.2.123456789.1599999999;
    _gid=GA1.2.123456789.1599999999'
}

# Create the Request object with the custom headers
req = request.Request(url, headers=headers)

# Send the request and get the response
response = request.urlopen(req)

# Read and print the response
print(response.read().decode('utf-8'))
```

4) Proxy

Proxy是一种网络服务，它充当客户端和其他服务器之间的中间服务器。客户端发送请求到代理服务器，代理服务器再发送请求到其他服务器，并将其他服务器的响应返回给客户端。

```
import requests

url = 'https://httpbin.org/get'
headers = {
    'User-Agent': 'Mozilla/4.0 (compatible; MSIE 5.5; Windows NT)',
    'Host': 'httpbin.org',
    'Accept': '*/*',
    'Accept-Language': 'zh-CN,zh;q=0.9',
    'Accept-Encoding': 'gzip, deflate',
    'Connection': 'keep-alive',
    'Cookie': '_gauth=1; _ga=GA1.2.123456789.1599999999;
    _gid=GA1.2.123456789.1599999999'
}

# Define your proxy
proxies = {
    'http': 'http://your_proxy_ip:your_proxy_port',
    'https': 'http://your_proxy_ip:your_proxy_port'
}

# Send the GET request with the headers and proxy
response = requests.get(url, headers=headers, proxies=proxies)

# Print the response
print(response.text)
```

三、 requests

requests是一个 Python 的第三方库，它提供了更多的功能和更简单的API，相比于urllib。

1) 安装

```
# 安装
conda install requests
pip install requests
```

2) Get

```
import requests

url = 'https://httpbin.org/get'
data = {
    'word': 'hello'
}
response = requests.get(url, params=data)
print(response.text)
```

3) Post

```
import requests

url = 'https://httpbin.org/post'
data = {
    'word': 'hello'
}
response = requests.post(url, data=data)
print(response.text)
```

第三十八章 django 概述

一、 概述

3.0 的官方文档

二、 MVC 模型

MVC 模式（Model－view－controller）是软件工程中的一种软件架构模式，把软件系统分为三个基本部分：模型（Model）、视图（View）和控制器（Controller）。

MVC 以一种插件式的、松耦合的方式连接在一起。

Django 本身基于 MVC 模型，即 Model（模型）+ View（视图）+ Controller（控制器）设计模式，MVC 模式使后续对程序的修改和扩展简化。

1) 模型（Model）：负责应用程序的数据和业务逻辑。通过将数据和逻辑从用户界面分离出来，使得模型可以独立于用户界面进行测试和修改。

2) 视图（View）：负责显示用户界面，但通常没有直接访问应用程序的数据。这使得可以更容易地更改应用程序的外观而不影响数据处理。

3) 控制器（Controller）：处理用户输入、更新模型和调整视图。通过将用户输入和应用程序逻辑分离，可以更容易地更改用户界面的交互方式而不影响数据和业务逻辑。

1) MTV 模型

Django 的 MTV 模式本质上和 MVC 是一样的，也是为了各组件间保持松耦合关系，只是定义上有些许不同，Django 的 MTV 分别是指：

1) M 表示模型（Model）：编写程序应有的功能，负责业务对象与数据库的映射 (ORM)。

2) T 表示模板 (Template)：负责如何把页面 (html) 展示给用户。

3) V 表示视图（View）：负责业务逻辑，并在适当时候调用 Model 和 Template。

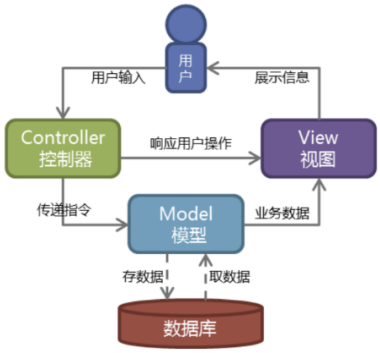

图 38.1 MVC 模型示意图

除了以上三层之外，还需要一个 URL 分发器，它的作用是将一个个 URL 的页面请求分发给不同的 View 处理，View 再调用相应的 Model 和 Template。

2) Django 的工作流程

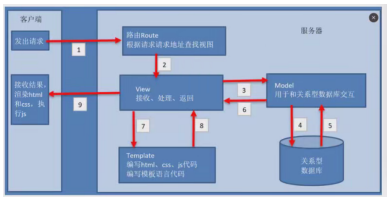

图 38.2 Django 流程

1) 浏览器通过域名发送一个 HTTP 请求到 Django 服务器。
2) 路由 (URLconf) 接收到请求，网站的功能模块被映射到一个 URL(路由)。
3) django 根据请求的 URL 调用相应的视图函数 (views)。
4) 视图函数处理请求，可能会与数据库交互，然后返回一个 HTTP 响应。
    1) 数据模型 (Model)：与数据库交互，处理数据。
    2) 当数据模型获取到数据后,视图函数将数据传递给模板 (template, 存储前端的 html 文件)。
    3) 模板将数据渲染到前端页面。
5) Django 将响应发送回浏览器，浏览器将其呈现给用户。

三、 Django 的安装

1) conda 安装

```
$> conda create --name mydjango python=3.10
$> conda activate mydjango
$> conda install django==3.0.3
$ python3 -m django --version
3.0.3
```

2) pip 安装

```
$> sudo pip3 install Django -i https://pypi.tuna.tsinghua.edu.cn/simple
$> sudo pip3 install Django==3.0.6 -i
    https://pypi.tuna.tsinghua.edu.cn/simple
```

四、 主项目

1) 创建项目

```
$> django-admin startproject recruitment_demo
$> tree
|-- manage.py # 命令行工具，用于管理项目
`-- recruitment_demo # 项目的容器
    |-- __init__.py # 一个空文件，告诉 Python 该目录是一个 Python 包
    |-- __pycache__ # 编译后的文件
       |-- __init__.cpython-310.pyc
       |-- settings.cpython-310.pyc
       |-- urls.cpython-310.pyc
       `-- wsgi.cpython-310.pyc
    |-- asgi.py # 一个 ASGI 兼容的 Web 服务器入口点
    |-- settings.py # 项目的配置文件
    |-- urls.py # 项目的 URL 声明，一份由 Django 驱动的网站"目录"
    `-- wsgi.py # 一个 WSGI 兼容的 Web 服务器的入口点
```

2) 查询帮助

```
$> python manage.py help
Type 'manage.py help <subcommand>' for help on a specific subcommand.

Available subcommands:

[auth]
    changepassword
```

```
    createsuperuser

[contenttypes]
    remove_stale_contenttypes

[django]
    check
    compilemessages
    createcachetable
    dbshell
    diffsettings
    dumpdata
    flush
    inspectdb
    loaddata
    makemessages
    makemigrations
    migrate
    sendtestemail
    shell
    showmigrations
    sqlflush
    sqlmigrate
    sqlsequencereset
    squashmigrations
    startapp
    startproject
    test
    testserver

[sessions]
    clearsessions

[staticfiles]
    collectstatic
    findstatic
    runserver
```

1) auth：用户认证系统, 与 Django 的认证系统（用户、权限、组等）相关
    1) `changepassword`：修改用户密码。
    2) `createsuperuser`：创建超级用户。
2) contenttypes：内容类型框架相关，内容类型框架允许你以统一的方式处理不同类型的模型。
    1) `remove_stale_contenttypes`：删除无用的内容类型。
3) django：Django 管理命令, 涉及数据库迁移、开发环境管理、数据导入导出等。
    1) `check`：检查 Django 项目中是否存在任何错误或潜在问题。适用于快速检测项目状态。

2) `compilemessages`：编译 Django 项目中的翻译文件（.po 文件转.mo 文件）。用于多语言支持。

3) `createcachetable`：创建数据库表以便使用缓存系统。用于缓存的数据库存储。

4) `dbshell`：打开一个数据库 shell（命令行界面），可以直接操作项目的数据库。

5) `diffsettings`：显示当前项目配置与默认配置的差异，帮助你查看哪些设置被修改过。

6) `dumpdata`：将数据库中的数据导出为 JSON 格式，通常用于备份或者数据迁移。

7) `flush`：重置数据库，将所有的数据清除并恢复到初始状态。会清空所有表，通常用于开发时清理数据。

8) `inspectdb`：根据现有的数据库表生成对应的 Django 模型代码（即反向生成模型）。

9) `loaddata`：加载之前导出的数据文件（通常是 JSON 格式）到数据库中。

10) `makemessages`：扫描代码，查找需要翻译的字符串，并生成相应的翻译文件（.po）。

11) `makemigrations`：生成数据库迁移文件，记录模型的更改。迁移是 Django 对数据库结构变化的管理。

12) `migrate`：应用迁移文件，更新数据库结构，使其与当前的模型状态一致。

13) `sendtestemail`：发送一个测试电子邮件，用于测试邮件设置是否正常。

14) `shell`：启动 Django 的交互式 Python shell，允许你直接在命令行中操作 Django 项目，例如查询数据库、操作模型等。

15) `showmigrations`：显示当前数据库迁移的状态，哪些迁移已应用，哪些没有应用。

16) `sqlflush`：生成并显示清空数据库表（flush）所需要的 SQL 语句。

17) `sqlmigrate`：查看某个迁移文件对应的 SQL 语句。

18) `sqlsequencereset`：查看重置数据库表的序列（主要是用于自增字段）所需要的 SQL 语句。

19) `squashmigrations`：将多个迁移合并为一个迁移文件，帮助减少迁移文件的数量。

20) `startapp`：创建一个新的 Django 应用，并生成默认的目录结构。

21) `startproject`：创建一个新的 Django 项目，初始化基本的项目结构。

22) `test`：运行 Django 测试框架中的测试，通常用于验证你的代码是否正常工作。

23) `testserver`：启动一个测试服务器，用于测试环境中的调试和运行。

3) 启动项目

```
$> python manage.py runserver ip地址:端口

# 本地访问
$> python manage.py runserver 0.0.0.0:8000
```

然后访问`http://localhost:8000/`，可以看到如图38.3所示的页面。

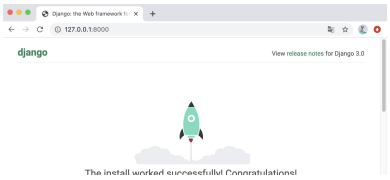

图 38.3 Django 启动成功

五、 创建子应用

(1) 子应用的作用

> **例子 38.1**
>
> Django 的主项目相当一家公司，公司下面有很多部门：开发部、运营部、财务部等等。
>
> 1) 公司-> 主项目
> 2) 部门-> 子应用
>
> 每个子应用之间相互独立，可以单独运行，也可以被其他项目引用。

1) 创建子应用

```
$> python manage.py startapp 子应用名称
```

```
$> python manage.py startapp jobs
$> tree
|-- manage.py
|-- recruitment_demo # 主项目的容器
   |-- __init__.py
```

```
...
`-- jobs  # 子应用的文件夹
    |-- __init__.py
    |-- admin.py  # 管理后台
    |-- apps.py   # 子应用的配置文件
    |-- migrations # 数据库迁移文件
       `-- __init__.py
    |-- models.py
    |-- tests.py   # 单元测试
    `-- views.py   # 视图函数
```

2) 注册子应用

在主项目的`settings.py`文件中注册子应用。

```
$> ls
jobs  recruitment_demo
$> cat recruitment_demo/settings.py # recruitment_demo是主项目的文件夹名称
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'jobs',  # 注册子应用，jobs是子应用的文件夹名称
    # 或者
    'job.apps.JobConfig',  # 注册子应用，job是子应用的文件夹名称
]
```

在注册子应用有两种写法

1) `'子模块名'`

这种写法会自动加载子应用的配置文件`apps.py`，并执行其中的代码。

2) `'子模块名.apps.子模块名Config'`

这个`子模块名Config`是`apps.py`中个一个类，一般约定的写法。

该方法显式指定应用的配置入口，Django 会加载该配置，而不是使用默认配置。可以在`子模块名Config`中定义一些特定的行为

```
class BaseInfoConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField' #
    指定模型的主键类型为大整型
    name = 'base_info' # 指定应用的名称，就是startapp时的应用名称
    verbose_name = '基础信息' # 指定应用的中文名称，在admin后台显示
```

第三十九章 Django 模型 (数据库)

一、 Django 模型简介

Django 对各种数据库提供了很好的支持，包括：PostgreSQL、MySQL、SQLite、Oracle。

Django 为这些数据库提供了统一的调用 API。我们可以根据自己业务需求选择不同的数据库。

二、 数据库的配置

1) sqlite3 数据库配置

Django 默认使用的是 sqlite3 数据库，不需要配置，也不需要安装额外的数据库驱动。

(1) settings.py 配置

```
$> cd 主项目目录
$> cat settings.py
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': os.path.join(BASE_DIR, 'db.sqlite3'),
    }
}
```

2) MySQL 数据库配置

(1) 安装 MySQL 驱动

```
$> sudo pip3 install pymysql
$> conda install pymysql
```

如果使用`pymysql`需要在 主项目的`__init__.py`文件中添加如下代码：

```
import pymysql
pymysql.install_as_MySQLdb()
```

(2) 提前配置数据库

ORM 无法操作到数据库级别，只能操作到数据表

1) 创建一个新数据库

```
$> mysql -u root -p
mysql> create database 数据库名称 default charset=utf8; #
    防止编码问题，指定为 utf8
```

2) 创建一个新用户, 并授权 (刷新权限)

```
mysql> create user '用户名'@'主机名' identified by '密码'; #
    主机名可以是 localhost
mysql> grant all privileges on 数据库名.* to '用户名'@'主机名'; #
    授权
mysql> flush privileges; # 刷新权限
```

例子 39.1

创建一个数据库`django_db`，用户名`django_user`，密码`your_password`，并授权`django_user`可以访问`django_db`数据库。

方法:

```
mysql> # 创建数据库
mysql> CREATE DATABASE django_db DEFAULT CHARACTER SET utf8;
Query OK, 1 row affected (0.03 sec)

mysql> # 创建用户
mysql> CREATE USER 'django_user'@'%' IDENTIFIED BY 'your_password';
Query OK, 1 row affected (0.03 sec)

mysql> # 授予 django_user, 使其可以访问 django_db 数据库:
mysql> GRANT ALL PRIVILEGES ON django_db.* TO 'django_user'@'%';
Query OK, 0 rows affected (0.01 sec)

mysql> # 刷新权限
mysql> flush privileges;
Query OK, 0 rows affected (0.01 sec)
```

(3) 配置 settings.py

```
$> cat settings.py
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': '数据库名称',
        'USER': '数据库用户名',
        'PASSWORD': '数据库密码',
        'HOST': '数据库地址',
        'PORT': '数据库端口',
    }
}
```

例子 39.2

配置 MySQL 数据库

方法:

```
$> cat settings.py
DATABASES = {
    'default': {
        # 使用 MySQL 引擎
        'ENGINE': 'django.db.backends.mysql',
        # 数据库名称，修改为你在 MySQL 中创建的数据库名
        'NAME': 'django_db',
        # 在 MySQL 中创建的用户名
        'USER': 'django_user',
        # 该用户的密码
        'PASSWORD': 'your_password',
        # 如果 MySQL 在本地机器上，使用 localhost，或者是容器 IP
        'HOST': 'localhost',
        # MySQL 端口
        'PORT': '3306',
    }
}

# 运行数据库迁移
$> python manage.py migrate
Operations to perform:
  Apply all migrations: TestModel, admin, auth, contenttypes, jobs,
    sessions
Running migrations:
  Applying TestModel.0001_initial... OK
  Applying TestModel.0002_auto_20241114_0739... OK
  Applying TestModel.0003_auto_20241114_0804... OK
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
```

```
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying jobs.0001_initial... OK
  Applying sessions.0001_initial... OK


# 查看数据库
$> docker start mysql57
$> docker exec -it mysql57 mysql -u root -p
Enter password:
mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| django_db          |
| mysql              |
| performance_schema |
| sys                |
| testdb             |
| testdb1            |
+--------------------+
7 rows in set (0.00 sec)

mysql> USE django_db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SHOW TABLES;
+----------------------------+
| Tables_in_django_db        |
+----------------------------+
| auth_group                 |
| auth_group_permissions     |
| auth_permission            |
| auth_user                  |
| auth_user_groups           |
| auth_user_user_permissions |
| django_admin_log           |
| django_content_type        |
| django_migrations          |
| django_session             |
| jobs                       |
| jobs_job                   |
| people                     |
```

```
+----------------------------+
13 rows in set (0.00 sec)

mysql>
```

3) PostgreSQL 数据库配置

(1) 安装 psycopg2

```
$> sudo sh -c 'echo "deb http://apt.postgresql.org/pub/repos/apt
    $(lsb_release -cs)-pgdg main" > /etc/apt/sources.list.d/pgdg.list'
$> wget --quiet -O - https://www.postgresql.org/media/keys/ACCC4CF8.asc |
    sudo apt-key add -
$> sudo apt update
$> sudo apt install postgresql-server-dev-9.6
$> pg_config --version
PostgreSQL 9.6.24
$> pip install psycopg2==2.8.6 # 因为是Django3.X, 所以安装2.8.6版本
```

或者安装预编译的二进制文件 psycopg2-binary

```
$> pip install psycopg2-binary
```

(2) [可选] 配置 PostgreSQL 的远程访问

默认情况下，PostgreSQL 仅允许本地访问。如果需要从其他容器或主机访问数据库，修改以下配置：

1) 拷贝文件进行修改

```
$> docker ps
CONTAINER ID   IMAGE          COMMAND         NAMES
fcca6d8e3b95   postgres:9.6   "docker-ent…"   postgres96

$> docker exec postgres96 cp /var/lib/postgresql/data/pg_hba.conf
    /var/lib/postgresql/data/pg_hba.conf.bak
$> docker cp postgres96:/var/lib/postgresql/data/pg_hba.conf
    ./pg_hba.conf
$> vim pg_hba.conf
# 添加下面内容
host    django_db    django_user    0.0.0.0/0    md5

$> # 其中 host表示允许通过 TCP/IP 协议进行远程访问，而不是本地 Unix
    套接字连接(local)。
$> # django_db 是数据库名称，django_user 是数据库用户名，
$> # 0.0.0.0/0 表示允许从任何 IP 地址访问。
$> # md5 表示使用 md5 加密方式验证密码。
```

```
$> docker cp ./pg_hba.conf
    postgres96:/var/lib/postgresql/data/pg_hba.conf
$> docker restart postgres96
```

2) 直接echo修改

```
$> docker exec postgres96 cp /var/lib/postgresql/data/pg_hba.conf
    /var/lib/postgresql/data/pg_hba.conf.bak
$> docker exec postgres96 bash -c "echo 'host    django_db
    django_user    0.0.0.0/0    md5' >>
    /var/lib/postgresql/data/pg_hba.conf"
$> docker restart postgres96
```

测试连接

```
$> docker exec -it postgres96 psql -h localhost -U django_user -d
    django_db
```

(3) 配置 settings.py

```
$> cat settings.py
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'your_db',          # 数据库名称 (默认为 postgres)
        'USER': 'your_user',        # 数据库用户 (默认为 postgres)
        'PASSWORD': 'your_password',# 数据库密码
        'HOST': 'localhost',        # 主机地址
        'PORT': '5432',             # 端口号
    }
}
```

例子 39.3

创建一个数据库django_db, 用户名django_user, 密码django_password, 并授权django_user可以访问django_db数据库。

方法:

```
$> $ docker exec -it postgres96 psql -U postgres
psql (9.6.24)
Type "help" for help.

postgres=# CREATE USER django_user WITH PASSWORD 'django_passwd';
CREATE ROLE

postgres=# CREATE DATABASE django_db OWNER django_user;
CREATE DATABASE
```

```
postgres=# GRANT ALL PRIVILEGES ON DATABASE django_db TO
    django_user;
GRANT
```

```
$> docker restart postgres96
```

```
$> cat settings.py
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'django_db',          # 数据库名称
        'USER': 'django_user',        # 数据库用户名
        'PASSWORD': 'django_passwd',  # 数据库密码
        'HOST': 'localhost',          # 数据库主机
        'PORT': '5432',               # 数据库端口
    }
}
```

运行数据库迁移

```
$> python manage.py makemigrations
$> python manage.py migrate
```

4) Redis 数据库配置

三、 定义模型

1) 前提准备————创建子应用

Django 规定，如果要使用模型，必须要创建一个子应用 app。

```
$> python manage.py startapp APP名
# 或者
$> django-admin startapp APP名
```

创建好了子应用后，会在项目目录下生成一个子应用目录，目录结构如下：

```
$> django-admin startapp TestModel
$> $ tree TestModel/
TestModel/
|-- admin.py # 管理后台,
|-- apps.py # 应用配置
|-- __init__.py # 一个空文件，告诉 Python 该目录是一个 Python 包
|-- migrations  # 数据库迁移文件
   `-- __init__.py
```

```
|-- models.py    # 模型文件
|-- tests.py     # 测试文件
`-- views.py     # 视图文件
```

2) 定义表结构

1) Django 的模型类必须继承自 `django.db.models.Model` 类。

   `models.Model` 是 Django 中定义模型类的基类，它包含了一些基本的属性和方法，用于定义数据库表的结构。

2) 要在 子应用 目录下的 `models.py` 文件中定义模型类。

```
$> cat TestModel/models.py
from django.db import models # 导入模型类

"""
1.类名 People 表示数据库表名
2.类属性 name 表示数据库表的字段名
  类属性 age 表示数据库表的字段名
3.类属性的值表示字段的类型
  CharField: 字符串类型 varchar
  IntegerField: 整数类型 int
  max_length: 字符串类型的长度
"""
class People(models.Model):
    # id 在 Django 中会自动生成，不需要定义
    # id = models.AutoField(primary_key=True)
    name = models.CharField(max_length=20) # 相当于 varchar(20)
    age = models.IntegerField()
    gender = models.BooleanField(default=True)
    # 外键关联，当关联的数据被删除时，本表中的数据也会被删除
    marjor = models.ForeignKey('Jobs', on_delete=models.CASCADE)

class Jobs(models.Model):
    job_name = models.CharField(max_length=20)
    job_salary = models.IntegerField()
    job_desc = models.TextField()
    job_address = models.CharField(max_length=50)
```

(1) ORM 的字段类型

在 ORM 的类属性中，与 SQL 的数据类型对应关系如下：

1) `models.CharField`：字符串类型 `varchar`

2) `models.IntegerField`：整数类型 `int`

3) `models.FloatField`：浮点类型 `float`

4) `models.DateTimeField`：日期时间类型 `datetime`

5) `models.BooleanField`：布尔类型 `bool`

6) `models.AutoField`：自增长整数类型 `int`（自动生成并递增）

7) `models.DecimalField`：高精度小数类型 `decimal`

8) `models.TextField`：长文本类型 `text`

9) `models.DateField`：日期类型 `date`

10) `models.TimeField`：时间类型 `time`

11) `models.EmailField`：电子邮件类型 `varchar`（特定长度）

12) `models.URLField`：URL 类型 `varchar`（特定长度）

13) `models.FileField`：文件类型 `blob` 或 `varchar`（用于文件路径）

14) `models.ImageField`：图像类型 `blob` 或 `varchar`（用于图像路径）

15) `models.JSONField`：JSON 类型 `json`（在一些数据库中）

16) `models.UUIDField`：UUID 类型 `uuid`

17) `models.ForeignKey`：外键关系 `int`（指向另一表的主键）

18) `models.ManyToManyField`：多对多关系（通过连接表实现）

19) `models.OneToOneField`：一对一关系 `int`（指向另一表的主键）

(2) 外键关联

```
models.ForeignKey('关联的表名', on_delete=models.CASCADE)
# on_delete=models.CASCADE 表示当关联的数据被删除时，本表中的数据也会被删除
```

3) 添加 INSTALLED_APPS

在主项目的`settings.py`文件中，设置`INSTALLED_APPS`，添加创建的 app。

```
$> tree
.
|-- HelloWorld
   |-- __init__.py
   |-- settings.py
....
|-- TestModel
   |-- models.py
....
```

```
$> cat HelloWorld/settings.py
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'TestModel', # 添加创建的 app
]
```

4) 生成迁移文件

迁移(Migration) 是 Django 中的一个重要概念，用于将模型的修改 (在文件`models.py`) 同步到数据库中。

1) `makemigrations`:根据主项目中的`settings.py`文件中的`INSTALLED_APPS`，生成迁移文件

2) `migrate`：执行迁移文件

```
$ python3 manage.py migrate   # 创建表结构
$ python3 manage.py makemigrations TestModel  # 让 Django
    知道我们在我们的模型有一些变更
$ python3 manage.py migrate TestModel   # 创建表结构
```

```
$> python3 manage.py migrate   # 创建表结构
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, jobs, sessions
Running migrations:
  No migrations to apply.
  Your models have changes that are not yet reflected in a migration, and
    so won't be applied.
  Run 'manage.py makemigrations' to make new migrations, and then re-run
    'manage.py migrate' to apply them.
$> python3 manage.py makemigrations TestModel
Migrations for 'TestModel':
  TestModel/migrations/0001_initial.py
    - Create model Test

$> python3 manage.py migrate   # 创建表结构
Operations to perform:
  Apply all migrations: TestModel, admin, auth, contenttypes, jobs,
    sessions
Running migrations:
  Applying TestModel.0001_initial... OK
```

如果迁移失败可以，删除迁移文件，重新迁移

```
$> find . -path "*/migrations/*.py" -not -name "__init__.py" -delete
$> find . -path "*/migrations/*.pyc"  -delete
```

(1) 查看数据库

```
$> python manage.py dbshell # 不同的数据库，命令不同
sqlite> .tables
auth_user                auth_user_user_permissions
auth_user_groups         django_admin_log
auth_group               django_content_type
auth_group_permissions   django_migrations
auth_permission          django_session
TestModel_People
TestModel_Jobs

sqlite> PRAGMA table_info(TestModel_People);
0|id|integer|1||1
1|name|varchar(20)|1||0
2|age|integer|1||0
3|gender|bool|1||0
4|marjor_id|integer|1||0

sqlite> PRAGMA table_info(TestModel_Jobs);
0|id|integer|1||1
1|job_name|varchar(20)|1||0
2|job_salary|integer|1||0
3|job_desc|text|1||0
4|job_address|varchar(50)|1||0
```

5) 常见报错

注意：如果没有在`INSTALLED_APPS`中添加创建的 app，会出现如下错误：

```
$ python3 manage.py migrate   # 创建表结构
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, jobs, sessions
Running migrations:
  No migrations to apply.
  Your models have changes that are not yet reflected in a migration, and
    so won't be applied.
  Run 'manage.py makemigrations' to make new migrations, and then re-run
    'manage.py migrate' to apply them.

$> python3 manage.py makemigrations TestModel
No installed app with label 'TestModel'.

$> python3 manage.py migrate TestModel
CommandError: No installed app with label 'TestModel'.
```

(1) mysqlclient 报错

```
$> python3 manage.py migrate
File
    "/usr/local/lib/python3.6/dist-packages/django/db/backends/mysql/base.py",
    line 36, in <module>
    raise ImproperlyConfigured('mysqlclient 1.3.13 or newer is required;
    you have %s.' % Database.__version__)
django.core.exceptions.ImproperlyConfigured: mysqlclient 1.3.13 or newer
    is required; you have 0.9.3.
```

MySQLclient 目前只支持到Python3.4,因此如果使用的更高版本的 python 需要修改如下：通过报错信息的文件路径找到

```
$> vim
    /usr/local/lib/python3.6/dist-packages/django/db/backends/mysql/base.py
# 把这两行代码注释掉
if version < (1, 3, 13):
    raise ImproperlyConfigured('mysqlclient 1.3.13 or newer is required;
    you have %s.' % Database.__version__)
```

四、 对象关系映射 (ORM)

Django 模型使用自带的 ORM。

对象关系映射（Object Relational Mapping，简称 ORM ）用于实现面向对象编程语言里不同类型系统的数据之间的转换。

1) ORM 与数据库的对应关系

1) Models 类：数据库表
2) Models 类的属性：数据库表的字段
3) Models 类的方法：数据库表的操作
4) Models 类的方法与数据库的操作：数据库表的增删改查
5) 对象实例：数据库表的记录

2) 数据库操作

(1) 增

增加数据一般使用`save()`方法。
在 app 的`views.py`文件中添加代码。

```
$> cat TestModel/views.py
from django.shortcuts import render
from django.http import HttpResponse
from .models import Test

def insert(request):
    test1 = Test(name='ZJW', age=18)
    test1.save()
    return HttpResponse("<p>数据添加成功！</p>")
```

(2) 查询

1. `all()`：获取全部数据
2. `filter()`：根据条件过滤数据
3. `get()`：根据条件获取单个对象
4. `order_by()`：根据字段排序

在 app 的`views.py`文件中添加代码。

```
$> cat TestModel/views.py
from django.shortcuts import render
from django.http import HttpResponse
from .models import Test

def select(request):
    # 初始化
    response = ""
    response1 = ""


    # 通过objects这个模型管理器的all()获得所有数据行，相当于SQL中的SELECT *
    FROM
    listTest = Test.objects.all()

    # filter相当于SQL中的WHERE，可设置条件过滤结果
    response2 = Test.objects.filter(id=1)

    # 获取单个对象
    response3 = Test.objects.get(id=1)

    # 限制返回的数据 相当于 SQL 中的 OFFSET 0 LIMIT 2;
    Test.objects.order_by('name')[0:2]

    #数据排序
    Test.objects.order_by("id")

    # 上面的方法可以连锁使用
    Test.objects.filter(name="runoob").order_by("id")
```

```
    # 输出所有数据
    for var in listTest:
        response1 += var.name + " "
    response = response1
    return HttpResponse("<p>" + response + "</p>")
```

(3) 改

修改数据可以使用`save()`或`update()`方法。
在 app 的`views.py`文件中添加代码。

```
$> cat TestModel/views.py
from django.shortcuts import render
from django.http import HttpResponse
from .models import Test

def update(request):
    # 修改其中一个id=1的name字段，再save
    test1 = Test.objects.get(id=1)
    test1.name = 'Google'
    test1.save()

    # 另外一种方式
    Test.objects.filter(id=1).update(name='Google')

    # 修改所有的列
    Test.objects.all().update(name='Google')

    return HttpResponse("<p>修改成功</p>")
```

(4) 删

删除数据可以使用`delete()`方法。
在 app 的`views.py`文件中添加代码。

```
$> cat TestModel/views.py
from django.shortcuts import render
from django.http import HttpResponse
from .models import Test

def delete(request):
    # 删除id=1的数据
    test1 = Test.objects.get(id=1)
    test1.delete()

    # 另外一种方式
    Test.objects.filter(id=1).delete()

    # 删除所有数据
    # Test.objects.all().delete()
```

```
    return HttpResponse("<p>删除成功</p>")
```

五、 后台管理

1) 创建超级用户

```
$> django-admin startproject mysite
$> cd mysite
$> python manage.py createsuperuser
Username (leave blank to use 'admin'): admin
Email address: admin@example.com
Password:
Password (again):
```

访问`http://127.0.0.1:8000/admin/`

2) 设置中文

在主项目的`settings.py`文件中设置中文。

```
$> cat mysite/settings.py
LANGUAGE_CODE = 'en-us'
# LANGUAGE_CODE = 'zh-hans'  # 设置中文
```

3) 注册模型

将子应用的模型在后台管理页面也可以看到,需要在子应用的`admin.py`文件中注册模型。

```
$> cat TestModel/models.py
...
class People(models.Model):
...
class Jobs(models.Model):
...


$> cat TestModel/admin.py
from django.contrib import admin # 导入admin模块
from TestModel.models import People, Jobs # 导入模型类


# Register your models here.
admin.site.register(People) # 注册模型类
admin.site.register(Jobs) # 注册模型类
```

如何对后台定义更多的信息，可以在模型类中设置`Meta`类。

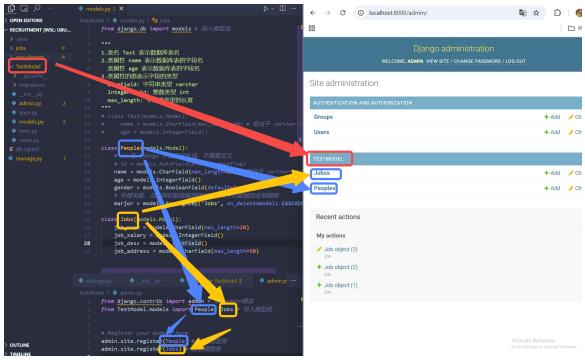

图 39.1 后台注册 models

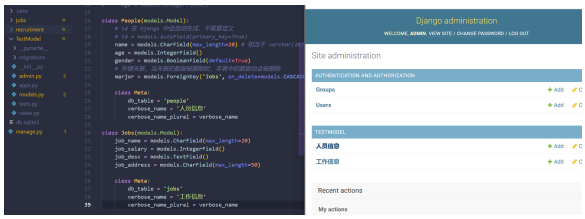

图 39.2 设置 Meta 信息

如果想要在后台管理页面中显示更多的字段,可以在模型类中设置`__str__()`方法。

如果想修改子应用在后台的名称,在子应用的`apps.py`文件中设置`verbose_name`,如图39.3所示。

```
$> cat TestModel/apps.py
from django.apps import AppConfig


class TestmodelConfig(AppConfig):
    name = 'TestModel'
    verbose_name = '测试模型'

$> cat mysite/settings.py
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    # 'TestModel', # 添加创建的 app
    'TestModel.apps.TestmodelConfig', # 添加创建的 app
]
```

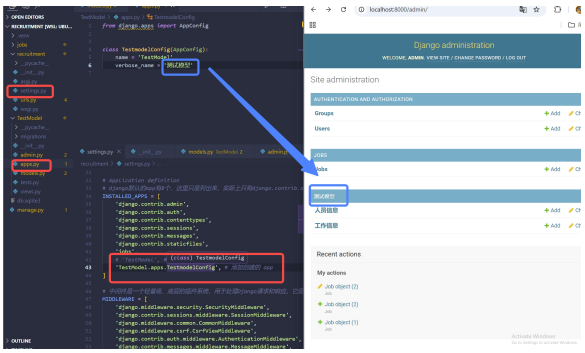

图 39.3 修改子应用在后台的名称

4) 问题

出现点击图39.2中的子项出现 404 问题，解决方法如下：

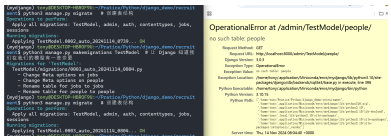

图 39.4 caption

```
$ python3 manage.py makemigrations TestModel  # 让 Django
    知道我们在我们的模型有一些变更
Migrations for 'TestModel':
  TestModel/migrations/0003_auto_20241114_0804.py
    - Change Meta options on jobs
    - Change Meta options on people
    - Rename table for jobs to jobs
    - Rename table for people to people

$ python3 manage.py migrate   # 创建表结构
Operations to perform:
  Apply all migrations: TestModel, admin, auth, contenttypes, jobs,
    sessions
Running migrations:
  Applying TestModel.0003_auto_20241114_0804... OK
```

第四十章 视图

一、 视图

Django 的视图是一个 Python 函数，它接受一个 Web 请求并返回一个 Web 响应。Django 的视图可以以多种方式编写，最简单的方式是作为一个函数。

二、 视图函数

1) 视图函数——HttpResponse

视图函数是 Django 中的一个重要概念，它是一个 Python 函数，接收一个 Web 请求 (`request`) 并返回一个 Web 响应 (`HttpResponse`)。视图函数的定义如下：

```
def my_view(request):
    # 业务逻辑
    return HttpResponse('Hello, World!')
```

2) 渲染模板——render

```
render(request, '模板名称', {'参数1': '值1', '参数2': '值2'})
```

1) 在项目的根目录下创建一个`templates`文件夹，用于存放模板文件。
2) 在`templates`文件夹下创建一个与子应用同名的文件夹`TestModel`，用于存放子应用`TestModel`的模板文件。
3) 在`TestModel`文件夹下创建一个`test.html`文件，用于存放模板文件。
4) 在主项目的`settings.py`中配置模板路径。

```
$> cat mysite/settings.py
...
BASE_DIR =
    os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
```

```
...
TEMPLATES = [
    {
        'BACKEND':
    'django.template.backends.django.DjangoTemplates',
        # 'DIRS': [],
        'DIRS': [os.path.join(BASE_DIR, 'templates')], #
    指定模板文件的目录
        'APP_DIRS': True,
        ...
    },
]
...
```

5) 在子应用目录下的urls.py中，修改test_view函数，使用render函数渲染模板。

```
from django.urls import path
from .views import test_view, test_index

urlpatterns = [
    path('test/', test_view),
    path('index/', test_index),
]
```

6) 在子应用目录下views.py中定义test_index函数，使用render函数渲染模板。

```
from django.shortcuts import render
from django.http import HttpResponse

# Create your views here.
def test_view(request):
    return HttpResponse('Hello, Django!') # 返回一个字符串

# 渲染模板 —— render

def test_index(request):
    return render(request, 'TestModel/index.html') #
    渲染index.html模板
```

7) 在浏览器中输入http://localhost:8000/index/,即可看到如图40.1所示页面。

三、 URL 配置 (URLconf)

URLconf 是 Django 中的 URL 配置，它是 URL 模式（URL 模式是 URL 和 Python 回调函数的映射）的集合。

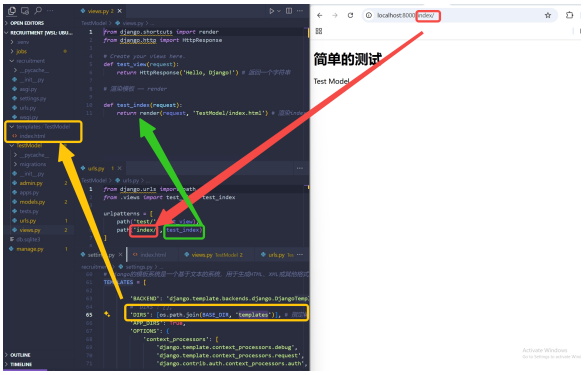

图 40.1 渲染 html 的过程

一般在 主项目目录下的`urls.py`文件中配置 URLconf，用于将 URL 映射到 视图函数 或 `include`子应用的 URLconf。

一般的格式为

1) 主项目的`urls.py`文件中，导入`include`函数和子应用的`urls.py`文件。

```
from django.urls import path, include

urlpatterns = [
    # 直接引入视图函数
    path('路由名/', views.视图函数名),
    path('路由名/', views.视图函数名, name='视图函数名'),
    url(r'^路由名/', include('子应用.urls')),
    re_path(r'^路由名/', include('子应用.urls')),

    # 引入子应用的 URLconf
    path('子应用名/', include('子应用.urls')),
    url(r'^路由名/', include('子应用.urls')),
    re_path(r'^路由名/', include('子应用.urls')),
]
```

2) 子应用的`urls.py`文件中，导入视图函数。

```
from django.urls import path, include
# 如果子应用有下面的一行，表示引入应用的 views.py 文件
from . import views

urlpatterns = [
    path('路由名/', views.视图函数名),
    path('路由名/', views.视图函数名, name='视图函数名'),
]
```

1) path() 将 URL 模式映射到视图

`path()`函数用于将 URL 模式映射到视图。

它采用四个参数，两个必选参数：route 和 view，两个可选参数：kwargs 和 name。

```
from django.urls import path

path(route, view, kwargs=None, name=None)
```

1) `route`: 字符串，定义 URL 的路径部分。

   可以包含变量，例如 `<int:my_variable>`，以从 URL 中捕获参数并将其传递给视图函数。

2) `view`: 视图函数，处理与给定路由匹配的请求。

   可以是一个函数或一个基于类的视图。

3) `kwargs`(可选): 一个字典，包含传递给视图函数的额外关键字参数。

4) `name`(可选): 为 URL 路由指定一个唯一的名称，以便在代码的其他地方引用它。这对于在模板中生成 URL 或在代码中进行重定向等操作非常有用。

2) [弃用] url()/re_path 将 URL 模式映射到视图

`url()`函数用于将 URL 模式映射到视图。

```
from django.urls import include, url

url(regex, view, kwargs=None, name=None, prefix=None)
```

1) `regex`: 字符串，包含正则表达式的 URL 模式。

2) `view`: 视图函数，处理与给定路由匹配的请求。

3) `kwargs`: 一个字典，包含传递给视图函数的额外关键字参数。

4) `name`: 为 URL 路由指定一个唯一的名称，以便在代码的其他地方引用它。

5) `prefix`: 字符串，用于给 URL 模式添加前缀。

Django2. 0 中可以使用`re_path()`函数替代`url()`函数。

```
from django.urls import include, re_path

re_path(regex, view, kwargs=None, name=None)
```

3) [不单独使用] include() 导入 URL 模式

`include()`函数用于包含其他 URL 模式。

```
from django.urls import include

include(urlconf_module, namespace=None, app_name=None)
```

1) `urlconf_module`: 字符串，包含 URL 模式的 Python 模块。
2) `namespace`: 字符串，用于给 URL 模式命名空间。
3) `app_name`: 字符串，用于给 URL 模式命名空间。

例子 40.1

有下面的结构

```
$ tree
.
|-- HelloWorld
|   |-- __init__.py
|   |-- __init__.pyc
|   |-- settings.py
|   |-- settings.pyc
|   |-- urls.py              # url 配置
|   |-- urls.pyc
|   |-- views.py              # 添加的视图文件
|   |-- views.pyc             # 编译后的视图文件
|   |-- wsgi.py
|   `-- wsgi.pyc
`-- manage.py
```

其中`views.py`文件内容如下：

```
$> cat HelloWorld/views.py 文件代码:
from django.http import HttpResponse

def hello(request):
    return HttpResponse("Hello world ! ")
```

方法:

URL 配置文件`urls.py`有两种写法：

1) 指向网站的根 URL

```
$> cat HelloWorld/urls.py
from django.urls import path
```

```
from . import views

urlpatterns = [
    path("", views.hello, name="hello"),
]
```

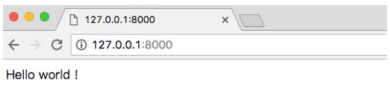

图 40.2 urlconf 指向网站的根

2) 适用于访问 `/hello/` 路径

```
$> cat HelloWorld/urls.py
from django.urls import path

from . import views

urlpatterns = [
    path('hello/', views.hello),
]
```

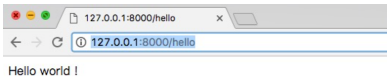

图 40.3 urlconf 指向 hello 目录

四、 路由视图过程

1) 在子应用`TestModel`中的`views.py`中定义一个视图函数`test_view`，接收一个 Web 请求，返回一个 Web 响应。

2) 在子应用`TestModel`中的新建一个`urls.py`文件，定义一个路由，将`test_view`与路由绑定。

```
$> cat TestModel/urls.py
from django.urls import path
from .views import test_view

urlpatterns = [
    path('test/', test_view),
]
```

3) 在主应用`mysite`中的`urls.py`文件中，将子应用`TestModel`的路由包含进来。

```
from django.contrib import admin
from django.urls import path

from django.conf.urls import include, url

urlpatterns = [
    url(r"^", include('jobs.urls')),
    path('admin/', admin.site.urls),
    path('', include('TestModel.urls')),
]
```

4) 在浏览器中输入`http://localhost:8000/test/`，即可看到如下页面：

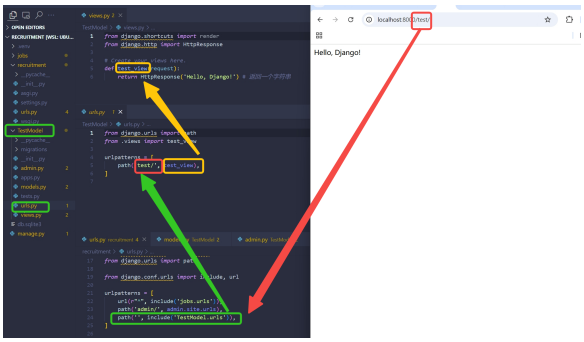

图 40.4 路由视图过程

其中路由的过程

1) 用户在浏览器中输入`http://localhost:8000/test/`。
2) Django 接收到请求，根据`mysite/urls.py`中的路由配置，
   这里面是依次遍历`jobs.urls`、`admin`和`TestModel.urls`。看这里面哪里个里面有`test/`这个路由，找到了`TestModel.urls`。
3) 根据`TestModel/urls.py`中的路由配置，找到了`test_view`。
4) 调用`test_view`，返回一个`HttpResponse`。
5) 将`HttpResponse`返回给浏览器。

第四十一章 模板 template

Django 模板是一个文本文件，它包含静态的 HTML，以及嵌入式的 Django 模板语言 (DTL) 代码。

一、 模板概述

在 Django 中，创建一个`base.html`文件，作为模板的基础文件，其他模板文件可以继承这个基础文件，从而减少重复代码。

1) 一般在主项目根目录创建一个`templates`文件夹，用于存放模板文件。

```
$> tree
.
|-- manage.py
|-- TestModel  # 子应用
|   |-- admin.py
|   |-- apps.py
|   |-- views.py
|   |-- urls.py
|   |-- models.py
|   |-- tests.py
|   `-- migrate
|-- mysite   # 主项目目录
   |-- __init__.py
   |-- settings.py
   |-- urls.py
   `-- wsgi.py
`-- templates # 模板文件夹
```

2) 在主项目的`settings.py`文件中配置模板文件路径。

```
$> cat mysite/settings.py
TEMPLATES = [
    {
        'BACKEND':
    'django.template.backends.django.DjangoTemplates',
        # 'DIRS': [], # 之前的配置
        # 修改为
        'DIRS': [os.path.join(BASE_DIR, 'templates')],
        'APP_DIRS': True,
```

```
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',

    'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

3) 在templates文件夹下创建一个base.html文件，作为模板的基础文件，其他模板文件可以继承这个基础文件，从而减少重复代码。

```
$> cat templates/base.html
<!DOCTYPE html>
<html lang="en">
<head>
    {% block delihead %}
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
    initial-scale=1.0">
    <title>测试文件标题</title>
    <link rel="icon" href="static/images/logo.png">
    <link rel="stylesheet" href="static/style.css" type="text/css">
    {% endblock %}
</head>

<body>
    <h2>美使馆</h2>
    {% block content %}
    {% endblock %}
</body>

<footer>
    <small>&copy 2020 美使馆<a
    href="http://www.baidu.com">JJW</a></small>
</footer>

</html>
```

4) 在urls.py文件中配置路径。

```
$> cat urls.py
from django.urls import path

from . import views

urlpatterns = [
    path('hello/', views.hello),
]
```

5) 在子应用的目录可以创建一个templates文件夹，用于存放子应用的模板文件。

```
$> tree TestModel
TestModel/
|-- __init__.py
|-- admin.py
|-- apps.py
|-- __pycache__
   |-- __init__.cpython-39.pyc
   |-- views.cpython-39.pyc
   `-- urls.cpython-39.pyc
|-- templates
   `-- hello.html
|--tests.py
|-- urls.py
`-- views.py
```

其中 hello.html 是子应用TestModel的模板文件。

```
$> cat templates/hello.html
{% extends 'base.html' %} <!-- 继承temples的base.html -->

{% block content %} <!-- 重写base.html的content -->
    <h3>Register</h3>
    <form action="/login/create_user" method="POST">
        {% csrf_token %}
        <input type="text" name="username" placeholder="Username"
    required>
        <input type="email" name="email" placeholder="Email"
    required>
        <input type="password" name="password"
    placeholder="Password" required>
        <input type="password" name="password2"
    placeholder="Confirm Password" required>
        <button type="submit">Register</button>
    </form>
{% endblock %}
```

6) 在子应用的views.py文件中增加一个新的对象，用于向模板提交数据。

```
$> cat TestModel/views.py
from django.shortcuts import render

"""
一个视图函数 hello
  接受一个 request 对象作为参数，并返回一个 HTTP 响应
"""
def hello(request):
    context = {}
    # context
    是一个字典，用来存储传递给模板的上下文数据。在这个字典中，键是模板变量的名称，值是要传递给模板的实际数据。
    context['hello'] = 'Hello World!'
```

```
    # 将 'Hello World!' 赋值给 context 字典中的 'hello'
    键。这样，hello
    将作为模板变量传递给模板，并且在模板中可以使用这个值。
    return render(request, 'hello.html', context)
```

7) 启动服务器，访问http://localhost:8000/hello/，可以看到页面显示Hello World!。

1) 模板的定义

1) 模板变量：{{ 变量名 }}

2) 从 URL 获取模板变量

```
{% url '路由名' 参数 %}
```

例子 41.1

在urls.py中定义一个

```
re_path('(?P<year>[0-9]{4}).html', views.myyear,
    name='urlyear'),

# 1.(?P<year>[0-9]{4}):
这是正则表达式中的一个命名捕获组，(?P<year>[0-9]{4})的含义是匹配4位数字（0-9之间）
# 2.views.myyear: 当URL匹配成功后，Django会调用 views.myyear
    这个视图函数，并将匹配到的 year 参数传递给它。
例如，如果URL是 2023.html，那么 views.myyear 会接收到
    year=2023。
# 3.name='urlyear':
这是给这个URL模式起的一个名字，方便在Django的其他地方通过名称来引用该URL。
通过这个名字，你可以使用 reverse() 函数或 {% url %}
    模板标签来生成对应的URL，例如 {% url 'urlyear' year=2023 %}
    会生成 2023.html。
```

而在模板文件夹的test.html中，

```
<a href="{% url 'urlyear' 2019 %}">2019年度</a>
```

方法:

当找到test.html模板文件时，会到urls.py中找到urlyear这个路由，并将2019作为参数传递给myyear函数。从而在浏览器中显示2019 年度的超链接。

```
<a href="/2019.html">2019年度</a>
```

3) 读取静态资源内容

```
{% static "css/style.css" %}
```

4) for 遍历：`{% for 变量名 in 列表名%}`

```
{% for item in list %}
    {{ item }}
{% endfor %}
```

5) if 判断：`{% if 条件%}`

```
{% if 条件 %}
    {{ 变量 }}
{% endif %}
```

二、 静态文件

1) 静态文件的作用

静态文件是指不需要经过服务器处理，直接可以被浏览器访问的文件，例如：图片、CSS、JavaScript 等。

1) 图片：`.jpg`、`.png`、`.gif`等。

2) CSS：`.css`文件，用于设置网页的样式。

3) JavaScript：`.js`文件，用于设置网页的交互效果。

2) 静态文件的配置

1) 在 主项目的`settings.py`文件中配置静态文件路径。

```
STATIC_URL = '/static/'
if not DEBUG:
    STATIC_ROOT = os.path.join(BASE_DIR, 'static')

# 重写 STATICFILES_DIRS
STATICFILES_DIRS = (os.path.join(BASE_DIR, 'static'),) #
    必须是一个元组(,)要有,
```

2) 想要引用 static 资源文件的子项目，需要在子项目的`urls.py`文件中的`urlpatterns`列表中添加一条配置，用于访问静态文件。

```
from django.urls import path
from . import views  # Import the views module
```

```
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path("", views.open_register, name="open register"),

] + static(settings.STATIC_URL,
    document_root=settings.STATICFILES_DIRS[0])
# 在 settings.py 中配置了 STATICFILES_DIRS, 所以这里可以直接使用
    settings.STATICFILES_DIRS[0] 来获取静态文件目录
```

在子项目的视图`views.py`中，可以通过`render`函数将静态文件传递给模板。

```
from django.shortcuts import render
from django.template import loader
from django.http import HttpResponse

# Create your views here.
def open_register(request):
    template = loader.get_template('login/register.html')
    return HttpResponse(template.render({}, request))
```

第八部分

数据处理与分析

第四十二章 [Python 库] Pandas

一、 简介

Pandas是一个开源的数据分析库，它为 Python 编程语言提供了高性能、易于使用的数据结构和数据分析工具。

它基于一种叫作 DataFrame 的数据结构，这种数据结构模仿了 R 语言中的 DataFrame。简单来说，一个 pandas DataFrame 是一张表格，类似于 Excel 表格。pandas 中包含大量用于修改表格和操作表格的方法，尤其是可以像 SQL 一样对表格进行查询和连接。NumPy 要求数组中的所有元素类型必须完全相同，而 pandas 不是这样，每一列数据的类型可以互不相同（比如整型、日期、浮点数和字符串）。

pandas 的另一个强大之处在于，它可以从许多文件格式和数据库中提取数据，如 SQL、Excel 文件和逗号分隔值（CSV）文件。

二、 安装

```
# 安装
conda install pandas
# 或
pip install pandas
```

第四十三章 Pandas 的数据结构

一、 Series

二、 DataFrame

三、 Index

第四十四章 Pandas 数据的引用和读取

一、 数据的引用

二、 文件的读取与写入

1) CSV
2) Excel
3) JSON
4) HDF5

第四十五章 Pandas 数据的聚合和排序

第九部分

人工智能

第四十六章 PyTorch

一、 概述

PyTorch 是一个 Python 的开源机器学习库，是 Python 优先的深度学习框架。PyTorch 是科学计算框架 Torch 在 Python 上的衍生，PyTorch 的产生受到了 Torch 和 Chainer 这两个框架的启发。Torch 使用 Lua 作为开发语言，PyTorch 则使用了 Python 作为开发语言。与 Chainer 类似，PyTorch 具有自动求导的动态图功能，也就是所谓的 define by run, 即当 Python 解释器运行到相应的行时才创建计算图。

PyTorch 就像带 GPU 的 NumPy, 采用了动态计算图（Dynamic Computational Graph）结构，是基于 tape 的 autograd 系统的深度神经网络。通过 PyTorch 的一种称为**反向模式自动微分**（Reverse-Mode Auto Differentiation）的技术，我们可以非常方便地构建网络。

`torch`是 PyTorch 的核心库，提供了很多用于创建和操作张量的函数。

PyTorch 由 4 个主要包组成，具体如下

1) `torch` 类似于 NumPy 的通用数组库,可将张量类型转换为`torch.cuda.TensorFloat`, 并在 GPU 上进行计算。
2) `torch.autograd` 用于构建计算图形并自动获取梯度的包。
3) `torch.nn` 神经网络构建的包，包括各种神经网络层，损失函数等。
4) `torch.optim` 优化器，包括各种优化算法，如 SGD、Adam 等。
5) `torch.utils` 数据处理工具，包括数据集加载、数据增强等。

1) 与 TensorFlow 的比较

1) 运行机制两个框架都是在张量上进行运算，并将任意一个模型看成是有向无环图 (Direct Acyclic Graph, DAG), 但 TensorFlow 遵循“数据即代码，代码即数据”的理念，当在 TensorFlow 中运行代码时，DAG 是以静态方式定义的，若需要实现动态 DAG, 则需要借助 TensorFlow Fold 库；

   而 PyTorch 属于更 Python 化的框架，动态 DAG 是内置的，可以随时定义、随时更改、随时执行节点，并且没有特殊的会话接口或占位符，相当灵活

2) 训练模式在分布式训练中，TensorFlow 和 PyTorch 的一个主要差异特点是数据并行化，使用 TensorFlow 时，使用者必须手动编写代码，并微调在特定设备上运行的每个操作，以实现分布式训练；

   而 PyTorch 则是利用异步执行的本地支持来实现的，其自身在分布式中的训练是比较欠缺的。

3) 可视化在可视化方面，TensorFlow 内置的 TensorBoard 非常强大，能够显示模型图，绘制标量变量，实现图像、嵌入可视化，甚至是播放音频等功能：

   PyTorch 的可视化情况，则显得有点差强人意，开发者可以使用 Visdom，但是 Visdom 提供的功能很简单且有限，可视化效果远远比不上 TensorBoard。

4) 生产部署对于生产部署而言，TensorFlow 具有绝对的优势，它可以直接使用 TensorFlow Serving 在 TensorFlow 中部署模型；

   而 PyTorch 没有提供任何用于在网络上直接部署模型的框架，需要使用 Flask 或者另一种替代方法来基于模型编写一个 RESTAPI。

二、 PyTorch 的历史

这里的 Py 我们不陌生，它就是 Python，那 Torch 是什么？从字面翻译过来是一个“火炬”。

什么是火炬呢？其实这跟 TensorFlow 中的 Tensor 是一个意思，我们可以把它看成是能在 GPU 中计算的矩阵。

那 PyTorch 框架具体是怎么用的呢？说白了就是一个计算的工具。借助它，我们就能用计算机完成复杂的计算流程。

但是我们都知道，机器跟人类的“语言”并不相通，想要让机器替我们完成对数据的复杂计算，就得先把数据翻译成机器能够理解的内容。无论是图像数据、文本数据还是数值数据，都要转换成矩阵才能进行后续的变化和运算。

搞定了读入数据这一步，我们就要靠 PyTorch 搞定后面各种复杂的计算功能。这些所有的计算功能，包括了从前向传播到反向传播，甚至还会涉及其它非常复杂的计算，而这些计算统统要交给 PyTorch 框架实现。

PyTorch 会把我们需要计算的矩阵传入到 GPU（或 CPU）当中，在 GPU（或 CPU）中实现各种我们所需的计算功能。因为 GPU 做矩阵运算比较快，所以在神经网络中的计算一般都首选使用 GPU，但对于学习来说，我们用 CPU 就可以了。

三、 PyTorch 的主要应用

PyTorch 主要应用于以下几个方面：

1) 计算机视觉: 使用 PyTorch, 程序员可以处理图像和视频，开发高度准确和精确的计算机视觉模型，如图像分类模型、对象检测模型和生成模型等。

2) 自然语言处理:PyTorch 可用于开发语言翻译、语言建模和聊天机器人等。利用 PyTorch, 程序员可使用长短期记忆网络、门控循环单元等处理自然语言、时间序列等方面的任务。

3) 强化学习:PyTorch 可用于开发强化学习模型，如深度 Q 网络、策略梯度

四、 Pytorch 环境搭建

1) 前提条件

Windows 平台还需要安装 Microsoft Visual C++ (VC_redist.x64.exe), 可从微软网站下载安装最新支持的 Visual C++。

2) CPU 版本

对于 CPU 版本直接通过 pip 或者 anaconda 命令安装即可，如：

```
pip install torch torchvision torchaudio
```

3) GPU 版本

目前 PyTorch 不仅支持 NVIDIA 的 GPU，还支持 AMD 的 ROCm 的 GPU。

对于 N 卡，什么型号是 pytorch 支持的呢？首先，需要计算能力（compute capability）≥3.0 的 GPU。可以到NVIDIA 官网查看支持的 GPU 型号。

本地下载

(1) conda 安装

```
conda create --name mypytorch python=3.10
# 安装 CUDA 12.1 版本
conda install pytorch torchvision torchaudio pytorch-cuda=12.1 -c pytorch
    -c nvidia
## pytorch 是 PyTorch 的核心库，提供了深度学习的基本功能
## torchvision 是基于 PIL 的 Python 图像库，主要用于图像处理和计算机视觉任务
## torchaudio 是 PyTorch 的音频库，提供了一些音频处理的工具
## pytorch-cuda=12.1 是指定安装的 CUDA 版本
## -c pytorch -c nvidia 是指定安装的渠道

# 验证pytorch是否安装成功
python -c "import torch; print(torch.__version__)"

$ python
```

```
Python 3.10.14 (main, May  6 2024, 19:42:50) [GCC 11.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import torch
nt(torch.__version__)
>>> print(torch.__version__)
2.4.0
>>> print(torch.cuda.is_available())
True
>>> print(torch.version.cuda)
12.1
>>>
```

(2) whl 安装

如果网速不好的话，Pytroch 离线下载 whl搜索对应的 whl 进行下载。

若是手动下载的 whl，需要注意 pytorch 与 torchvision 之间版本对应关系，这个可以到torchvision Github查看，这点非常重要，CV 中一些报错就是因为 torchvision 与 pytorch 版本不匹配导致的。

```
pip install *.whl
```

4) 问题

为什么我安装的明明是 GPU 版本，但是代码却返回了“False”，显示 GPU 不可用呢？

1) 检查计算机上是否有支持 CUDA 的 GPU 首先查看电脑的显卡型号以及是否有独立显卡，如果没有以“NVIDIAN”名称开头的独立显卡，则不能支持 CUDA，因此 GPU 不可用

2) 检查显卡驱动版本。在终端中输入“`nvidia-smi`”命令，会显示出显卡驱动的版本和 CUDA 的版本。

   如果显卡驱动的版本过低，与 CUDA 版本不匹配，那么 GPU 也不可用，需要根据显卡的型号更新显卡驱动。

3) 检查 PyTorch 版本和 CUDA 版本是否匹配

5) 后续辅助

完成 PyTorch 库的安装后，还需要安装辅助库，如绘图库 Matplotlib> 数据分析库 pandas 以及开发编辑工具 Jupyter Notebook 等，可在 Anaconda Prompt 命令行或者终端执行以下安装命令。

```
pip install pandas matplotlib notebook
```

五、 PyTorch 模块结构

1) Torch 相关

torch 是 PyTorch 框架的核心库，提供了深度学习的基本功能。

(1) PyTorch 安装目录

通过`conda info -envs`命令查看当前环境下的所有虚拟环境的位置。

```
$ conda info --envs
# conda environments:
#
base                     /home/tony/.application/Miniconda
mypytorch                /home/tony/.application/Miniconda/envs/mypytorch

# 因此PyTorch的安装位置在
/home/tony/.application/Miniconda/envs/mypytorch/lib/python3.10/site-packages/
```

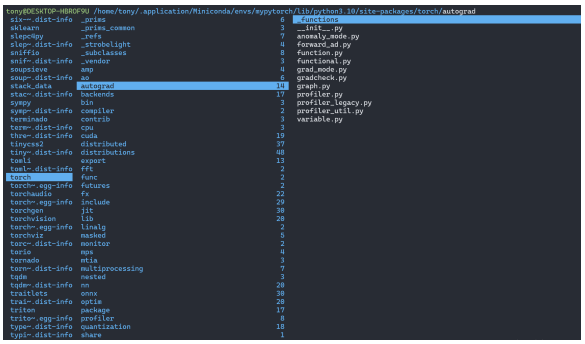

图 46.1 PyTorch 安装目录

(2) PyTorch 的主要模块

1) _C

从文件夹名称就知道它和 C 语言有关，其实它是辅助 C 语言代码调用的一个模块，该文件夹里存放了一系列 pyi 文件，pyi 文件是 python 用来校验数据类型的，如果调用数据类型不规范，会报错。

PyTorch 的底层计算代码采用的是 C++ 语言编写，并封装成库，供 pytorch 的 python 语言进行调用。这一点非常重要，后续我们会发现一些 pytorch 函数无法跳转到具体实现，这是因为具体的实现通过 C++ 语言，我们无法在 Pycharm 中跳转查看。

2) include

上面讲到 pytorch 许多底层运算用的是 C++ 代码，那么 C++ 代码在哪里呢？它们在这里, 在 torch/csrc 文件夹下可以看到各个`.h/.hpp`文件，而在 python 库中，只包含头文件，这些头文件就在 include 文件夹下。

但如果直接下载的是 PyTorch 的 whl 文件，是没有这个文件夹的，因为这个文件夹是源码编译的时候才会有的。

3) lib

torch 文件夹中最重要的一个模块，torch 文件夹占 3.2GB 的空间，98% 的内容都在 lib 中，占了 3.16GB 空间。pytorch 的内容几乎都在 lib 里面，让我们看看里面是什么。

lib 文件夹下包含大量的.lib .so 文件（分别是静态链接库和动态链接库），例如大名鼎鼎的`libcudnn_*.so`，`libc*_cuda.so`。这些底层库都会被各类顶层 python api 调用。

4) autograd：自动求导，自动计算张量的梯度，用于反向传播

该模块是 pytorch 的核心模块与概念，它实现了梯度的自动求导，极大地简化了深度学习研究者开发的工作量，开发人员只需编写前向传播代码，反向传播部分由`autograd`自动实现，再也不用手动去推导数学公式，然后编写代码了（很多朋友可能不知道，在早期的深度学习框架中是没有这个功能的，例如 caffe，它需要手动编写反向传播的公式代码）

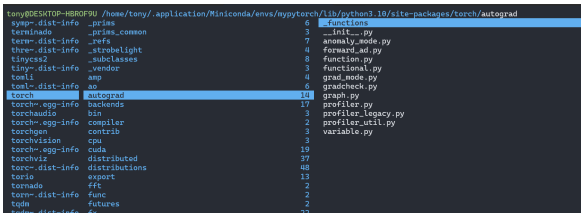

图 46.2 autograd 中的算法

5) nn：神经网络模块

相信这个模块是 99%pytorch 开发者使用频率最高的模块，搭建网络的网络层就在`nn.modules`里边。我们可以到`torch/nn/modules`里面看常用的网络层，包括常见的层（如全连接层、卷积层、循环层等）以及损失函数。

6) onnx

pytorch 模型转换到 onnx 模型表示的核心模块，进入文件夹可以看到大量的`opset**.py`，

7) optim

优化模块，深度学习的学习过程，就是不断的优化，而优化使用的方法函数，都暗藏在了 optim 文件夹中，进入该文件夹，可以看到熟悉的优

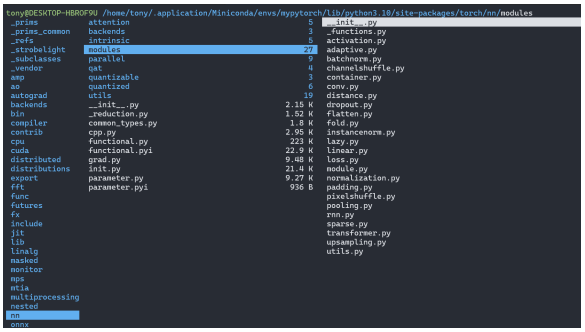

图 46.3 nn.modules 中的网络层

化方法：“Adam”、“SGD”、“ASGD”等。以及非常重要的学习率调整模块，`lr_scheduler.py`。

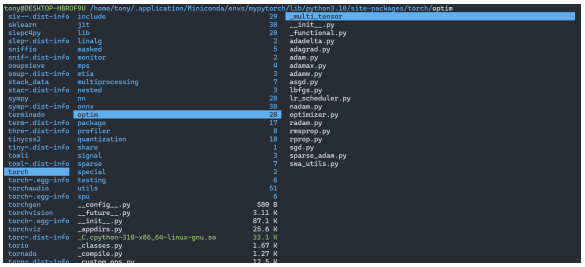

图 46.4 optim 中的算法

8) utils

utils 是各种软件工程中常见的文件夹，其中包含了各类常用工具，其中比较关键的是 data 文件夹，tensorboard 文件夹，这些工具都将在后续章节详细展开。

2) torchvision 相关

专门为计算机视觉任务设计的附加库，扩展了 PyTorch 的功能。

(1) torchvision 安装目录

(2) torchvision 的主要模块

1) datasets

提供了一系列常用的计算机视觉数据集的接口，如 CIFAR-10、ImageNet、MNIST 等。用户可以通过简单的 API 下载、加载和预处理这些数据集。

2) models: 预训练模型

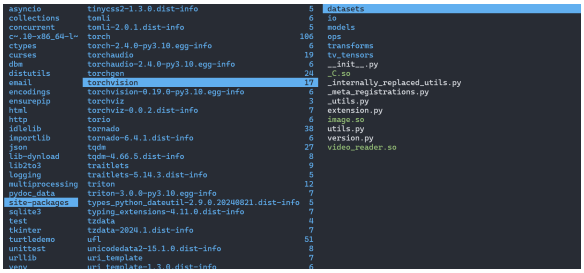

图 46.5 torchvision 的安装目录

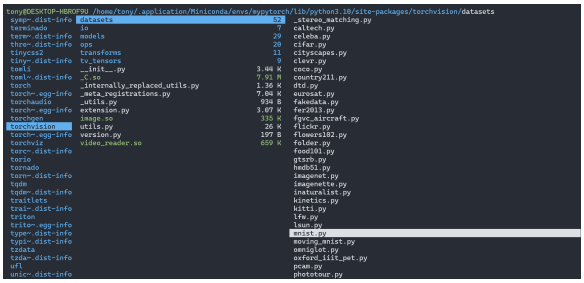

图 46.6 datasets 内容

这里是宝藏库，里边存放了经典的、可复现的、有训练权重参数可下载的视觉模型，例如分类的 alexnet、densenet、efficientnet、mobilenet-v1/2/3、resnet 等，分割模型、检测模型、视频任务模型、量化模型。

3) ops

视觉任务特殊的功能函数，例如检测中用到的 roi_align, roi_pool，boxes 的生成，以及 focal_loss 实现，都在这里边有实现。

4) transforms: 图像转换

提供了一系列常用的图像预处理方法。数据增强，数据预处理，数据增强的函数都在这里，例如缩放 (resize), 裁剪 (crop), 归一化 (normalize), 旋转 (rotate), 翻转 (flip) 等。

通过`torchvision/transforms/transforms.py`, 可以看到 torchvision 包含了这些功能

```
__all__ = ["Compose", "ToTensor", "PILToTensor",
"ConvertImageDtype", "ToPILImage", "Normalize", "Resize",
"Scale",
       "CenterCrop", "Pad", "Lambda", "RandomApply",
"RandomChoice", "RandomOrder", "RandomCrop",
       "RandomHorizontalFlip", "RandomVerticalFlip",
"RandomResizedCrop", "RandomSizedCrop", "FiveCrop", "TenCrop",
       "LinearTransformation", "ColorJitter", "RandomRotation",
"RandomAffine", "Grayscale", "RandomGrayscale",
       "RandomPerspective", "RandomErasing", "GaussianBlur",
"InterpolationMode", "RandomInvert", "RandomPosterize",
```

```
            "RandomSolarize", "RandomAdjustSharpness",
    "RandomAutocontrast", "RandomEqualize"]
```

第四十七章 PyTorch 基础

一、 Tensor 概述

1) Tensor 简介

pytorch 的核心数据结构——Tensor（张量）。

Tensor 中文被翻译为张量。张量在不同学科中有不同的意义，在深度学习中张量表示的是一个多维数组，它是标量、向量、矩阵的拓展。

张量的每一个方向被称为**模**（Mode）。

张量的阶数就是它的维数，一阶张量就是向量，二阶张量就是矩阵，三阶及其以上的张量统称为高阶张量。

Tensor 可以是零维（又称为标量或一个数）、一维 (向量)、二维 (矩阵) 及多维的数组。

一个 RGB 图像的数组就是一个三维张量，第一维是图像的高，第二维是图像的宽，第三维是图像的颜色通道。

2) Tensor 的作用

tensor 自称为神经网络界的 NumPy, 它与 NumPy 的 ndarray 相似，二者均可共享内存，且它们之间的转换非常方便和高效。不过它们也有不同之处，最大的区别就是 NumPy 会把`ndarray`放在 CPU 中加速运算，而`Tensor`会放在 GPU 中进行加速运算（假设当前环境有 GPU）。

tensor 用于表达各类数据，如输入数据、模型的参数、模型的特征图、模型的输出等。这里边有一个很重要的数据，就是模型的参数。对于模型的参数，我们需要更新它们，而更新操作需要记录梯度，梯度的记录功能正是被张量所实现的（求梯度是 autograd 实现的）。

3) torch.Tensor 和 torch.tensor

在 pytorch 中，有两个张量的相关概念极其容易混淆，分别是`torch.Tensor`和`torch.tensor`。

☞ 习惯上不区分 Tensor 和 tensor，但主要用小写的 tensor 表示张量这个数据类型。

注意：`torch.Tensor`和`torch.tensor`的区别

1) `torch.Tensor`是`torch.tensor`和`torch.empty`的一种混合。但是当传入数据时，`torch.Tensor`使用全局默认数据类型 (FloatTensor), `torch.tensor`从数据中推断数据类型。

2) `torch.tensor(1)` 是一个标量,而 `torch.Tensor(1)`(类似`torch.tensor([1])`)是一个一维张量。

3) `torch.tensor` 函数确实创建了一个 `torch.Tensor` 实例，但它不是直接调用 `torch.Tensor` 类构造函数。

   `torch.tensor` 是 PyTorch 提供的一个高层次的函数,主要用于将 Python 的数组或列表对象转换为 `torch.Tensor` 对象。

其实,通过命名规范,可知道 torch.Tensor 是 Python 的一个类, torch.tensor 是 Python 的一个函数。通常我们调用 torch.tensor 进行创建张量，而不直接调用 torch.Tensor 类进行创建。

```
1 import torch
2 tl=torch.Tensor(1)
3 t2=torch.tensor(1)
4 t3=torch.tensor([1])
5 print ("tl 的值{}, tl 的数据类型{} ".format (tl, tl.type()))
6 print ("t2 的值{}, t2 的数据类型{} ".format (t2, t2.type()))
7 print ("t3 的值{}, t3 的数据类型{} ".format (t3, t3.type()))
```

```
tl 的值tensor([-2.5500e-22]), tl 的数据类型torch.FloatTensor
t2 的值1, t2 的数据类型torch.LongTensor
t3 的值tensor([1.]), t3 的数据类型torch.FloatTensor
```

4) Tensor 的历史演变

5) Tensor 的结构属性

张量是 PyTorch 的基本数据结构，在使用时需要表示成`torch.Tensor`的形式，它有 8 个主要的属性前 4 个属性与数据相关，后 4 个属性与梯度求导相关

1) `data`: 被包装的张量。

2) `dtype`：张量的数据类型。

   如表47.1所示，PyTorch 中有 9 种不同的数据类型。

3) `shape`：张量的形状/维度。

4) `device`: 张量所在设备，加速计算的关键，GPU 或 CPU。

5) `grad`: data 的梯度。

6) `grad_fn`：创建张量的函数，这是自动求导的关键。

Data Type	dtype	CPU Tensor	GPU Tensor
32-bit floating point	torch.float32 or torch.float	torch.FloatTensor	torch.cuda.FloatTensor
64-bit floating point	torch.float64 or torch.double	torch.DoubleTensor	torch.cuda.DoubleTensor
16-bit floating point	torch.float16 or torch.half	torch.HalfTensor	torch.cuda.HalfTensor
8-bit integer (unsigned)	torch.uint8	torch.ByteTensor	torch.cuda.ByteTensor
8-bit integer (signed)	torch.int8	torch.CharTensor	torch.cuda.CharTensor
16-bit integer (signed)	torch.int16 or torch.short	torch.ShortTensor	torch.cuda.ShortTensor
32-bit integer (signed)	torch.int32 or torch.int	torch.IntTensor	torch.cuda.IntTensor
64-bit integer (signed)	torch.int64 or torch.long	torch.LongTensor	torch.cuda.LongTensor
Boolean	torch.bool	torch.BoolTensor	torch.cuda.BoolTensor

表 47.1 张量数据类型

7) `requires_grad`: 指示是否需要计算梯度。

8) `is_leaf`：指示是否是叶子节点。

二、 Tensor 操作

1) Tensor 操作分类

对 Tensor 的操作很多，从接口的角度来划分，可以分为两类：

1) `torch.function` 接口中的函数是 PyTorch 的全局函数，通常用于创建、操作、转换和计算张量。这些函数通常以张量作为输入并返回新的张量。

    1) `torch.tensor(data)`: 从`data`创建一个`Tensor`。
    2) `torch.zeros(size)`: 创建一个大小为`size`的全 0 张量。
    3) `torch.ones(size)`: 创建一个大小为`size`的全 1 张量。
    4) `torch.rand(size)`: 创建一个大小为`size`的随机张量。
    5) `torch.arange(start, end, step)`: 创建一个从`start`到`end`，步长为`step`的张量。
    6) `torch.add(tensor1, tensor2)`: 将两个张量相加。
    7) `torch.sub(tensor1, tensor2)`: 将两个张量相减。
    8) `torch.mul(tensor1, tensor2)`: 将两个张量相乘。
    9) `torch.div(tensor1, tensor2)`: 将两个张量相除。
    10) `torch.pow(tensor1, tensor2)`: 将`tensor1`的`tensor2`次方。
    11) `torch.sqrt(tensor)`: 计算`tensor`的
    12) `torch.exp(tensor)`: 计算`tensor`的指数。
    13) `torch.log(tensor)`: 计算`tensor`的对数。
    14) `torch.sin(tensor)`: 计算`tensor`的正弦。
    15) `torch.cos(tensor)`: 计算`tensor`的余弦。
    16) `torch.tan(tensor)`: 计算`tensor`的正切。
    17) `torch.sigmoid(tensor)`: 计算`tensor`的 Sigmoid 函数。

18) `torch.relu(tensor)`: 计算`tensor`的 ReLU 函数。
19) `torch.max(tensor)`: 计算`tensor`的最大值。
20) `torch.min(tensor)`: 计算`tensor`的最小值。
21) `torch.mean(tensor)`: 计算`tensor`的平均值。
22) `torch.sum(tensor)`: 计算`tensor`的和。
23) `torch.dot(tensor1, tensor2)`: 计算`tensor1`和`tensor2`的点积。
24) `torch.mm(tensor1, tensor2)`: 计算`tensor1`和`tensor2`的矩阵乘法。
25) `torch.cat(tensor1, tensor2)`: 沿着指定维度拼接张量。
26) `torch.stack(tensor1, tensor2)`: 沿着新维度拼接张量。
27) `torch.reshape(tensor, shape)`: 改变张量的形状。

2) `tensor.function` 接口是张量对象的方法，这些方法直接作用于张量本身。它们通常是 `torch.function` 接口的实例方法版本。

1) `tensor.item()`: 将张量转换为标量。
2) `tensor.numpy()`: 将张量转换为 NumPy 数组。
3) `tensor.to(device)`: 将张量移动到指定设备。
4) `tensor.cpu()`: 将张量移动到 CPU。
5) `tensor.cuda()`: 将张量移动到 GPU。
6) `tensor.add(tensor)`: 将两个张量相加。
7) `tensor.sub(tensor)`: 将两个张量相减。
8) `tensor.mul(tensor)`: 将两个张量相乘。
9) `tensor.div(tensor)`: 将两个张量相除。
10) `tensor.pow(tensor)`: 将`tensor`的`tensor`次方。
11) `tensor.sqrt()`: 计算`tensor`的平方根。
12) `tensor.exp()`: 计算`tensor`的指数。
13) `tensor.log()`: 计算`tensor`的对数。
14) `tensor.transpose(dim1, dim2)`: 交换张量的两个维度。
15) `tensor.t()`: 转置张量。
16) `tensor.transpose(dim1, dim2)`: 交换张量的两个维度。
17) `tensor.view(shape)`: 改变张量的形状。

2) 创建 Tensor

如表47.2所示，PyTorch 提供了很多创建张量的函数。
如果分类的话，创建张量的方式可以分为以下几种：

函数	功能
`Tensor(*size)`	直接从参数创建一个张量，支持 `list`、`numpy` 数组
`eye(row, column)`	创建指定行数、列数的二维单位张量
`linspace(start, end, steps)`	从 `start` 到 `end`，均匀切分成 `steps` 份
`logspace(start, end, steps)`	从 10^{start} 到 10^{end}，均匀切分成 `steps` 份
`rand/randn(*size)`	生成 [0,1] 均匀分布/标准正态分布数据
`ones(*size)`	返回指定形状的张量，元素初始为 1
`zeros(*size)`	返回指定形状的张量，元素初始为 0
`ones_like(t)`	返回与 `t` 形状相同的张量，且元素初始为 1
`zeros_like(t)`	返回与 `t` 形状相同的张量，且元素初始为 0
`arange(start, end, step)`	在区间 `[start, end)` 上以间隔 `step` 生成一个序列张量
`from_numpy(ndarray)`	把 `ndarray` 转换为 Tensor

表 47.2 常见的创建 Tensor 的函数

(1) 直接创建

通过`torch.tensor`创建张量

```
torch.tensor(data, dtype=None, device=None, requires_grad=False,
    pin_memory=False)
# data: 数据
#     PyTorch 支持通过list、 tuple、numpy array、scalar
    等多种类型进行数据传入，
#     并转换为 tensor。
# dtype: 张量数据类型
# device: 存放设备
# requires_grad: 是否需要梯度
#      只有当一个 Tensor 设置 requires_grad 为 True 的情况下，
#      才会对这个 Tensor 以及由这个 Tensor 计算出来的其他 Tensor 进行求导，
#      然后将导数值存在Tensor 的 grad 属性中，便于优化器来更新参数。
#
#      依赖于叶子节点的节点，requires_grad 默认为 True
#
#   如果是训练过程就要设置为 true，目的是方便求导、更新参数。
#   而到了验证或者测试过程，我们的目的是检查当前模型的泛化能力，
#   那就要把 requires_grad 设置成 Fasle，避免这个参数根据 loss 自动更新
# pin_memory: 是否存放在锁页内存，以加速数据传输。
#      在GPU上，如果数据存放在锁页内存中，那么GPU的显存占用会更少，
#      从而可以更快的进行运算。
```

```
1  import torch
2  a = torch.tensor([1,2,3], dtype=torch.float32)
3
4  b = torch.tensor([[1,2,3],[4,5,6]],
5                   dtype=torch.float32,
6                   device=torch.device('cuda:0'), # 存放在第一个GPU上
7                   requires_grad=True,
8                   pin_memory=True
9                   )
```

(2) 从 NumPy 数组创建

```
# 功能：从 numpy 数组创建张量
torch.from_numpy(ndarray)
```

```
import numpy as np
a = np.zeros(3)

b = torch.from_numpy(a)
```

注意:从`torch.from_numpy()`创建的张量和原始的`ndarray`共享内存，修改其中一个，另一个也会改变。

```
import numpy as np
import torch
arr = np.array([[1, 2, 3], [4, 5, 6]])
print("arr: ", arr)
t = torch.from_numpy(arr)
print("t: ", t)

print("\n修改arr")
arr[0, 0] = 0
print("arr: ", arr)
print("t: ", t)
```

```
arr:  [[1 2 3]
       [4 5 6]]
t:  tensor([[1, 2, 3],
       [4, 5, 6]])

修改arr
arr:  [[0 2 3]
       [4 5 6]]
t:  tensor([[0, 2, 3],
       [4, 5, 6]])
```

(3) 概率分布创建张量

1) 从正态分布 (高斯分布) 中抽取随机数创建张量

```
torch.normal(size, mean, std)   # 指定均值和标准差的正态分布
# size: 仅在mean和std均为标量时使用，表示创建张量的形状。
# mean: 正态分布的均值
# std: 正态分布的标准差
```

```
import torch

mean = torch.arange(1, 5, dtype=torch.float)
std = torch.arange(1, 5, dtype=torch.float)
# 将会为每一对 mean_i 和 std_i 生成一个服从正态分布的随机数 t_i。
# 每次运行的结果都不一样。
# t_0: 生成一个随机数 t_0，服从 N(1.0,1.0^2)。
# t_1: 生成一个随机数 t_1，服从 N(2.0,2.0^2)。
t = torch.normal(mean, std)
print("mean:{}\nstd:{}\n{}".format(mean, std, t))

print("------------------------------")
# mean是一个张量，std是一个标量
t = torch.normal(mean, 1.0)
print("mean:{}\nstd:{}\n{}".format(mean, 1.0, t))

print("------------------------------")
# mean是一个标量，std是一个张量
t = torch.normal(0.0, std)
print("mean:{}\nstd:{}\n{}".format(0.0, std, t))

print("------------------------------")
# mean和std都是标量，size指定张量的形状
t = torch.normal(mean=0.0, std=1.0, size=(4,))
print("mean:{}\nstd:{}\n{}".format(mean, std, t))
```

```
$ python3 test.py
mean:tensor([1., 2., 3., 4.])
std:tensor([1., 2., 3., 4.])
tensor([1.9618, 3.5882, 2.0549, 3.7797])
------------------------------
mean:tensor([1., 2., 3., 4.])
std:1.0
tensor([0.1122, 0.2529, 2.5417, 4.8538])
------------------------------
mean:0.0
```

```
std:tensor([1., 2., 3., 4.])
tensor([-0.2920, -0.8096, -4.0819, -2.9697])
------------------------------
mean:tensor([1., 2., 3., 4.])
std:tensor([1., 2., 3., 4.])
tensor([-0.8268, -0.1924, -0.1457, -1.2918])

$ python3 test.py
mean:tensor([1., 2., 3., 4.])
std:tensor([1., 2., 3., 4.])
tensor([1.9121, 0.8003, 5.2724, 3.5752])
------------------------------
mean:tensor([1., 2., 3., 4.])
std:1.0
tensor([-0.4556,  3.1790,  0.8509,  3.6606])
------------------------------
mean:0.0
std:tensor([1., 2., 3., 4.])
tensor([-0.0166,  1.5871,  3.0519,  1.8443])
------------------------------
mean:tensor([1., 2., 3., 4.])
std:tensor([1., 2., 3., 4.])
tensor([-0.8179, -0.5369,  0.3769,  1.6214])
```

2) 从标准正态分布中抽取随机数创建张量

```
torch.randn(size,         # 数据类型为浮点型且维度指定
                out=None,     # 输出张量
                dtype=None,   # 数据类型
                layout=torch.strided, # 默认为默认布局
                device=None, # 设备
                requires_grad=False)  # 是否需要梯度
# 数据类型为浮点型且维度指定,随机数满足均值为 0、方差为 1标准正态分布

torch.randn_like(input,  # 生成一个与 input 形状相同的张量
                        dtype=None,  # 数据类型
                        layout=None, # 默认为默认布局
                        device=None, # 默认为当前设备
                        requires_grad=False)  # 是否需要梯度
# 生成一个与 input 形状相同的张量, 随机数满足均值为 0、方差为
    1标准正态分布
```

```
# randn(*size)
import torch
randn_tensor = torch.randn(2, 2)
print("\nrandn(*size):")
print("Shape:", randn_tensor.shape)
print("Content:\n", randn_tensor)

```

```
# randn_like(input)
randn_like_tensor = torch.randn_like(randn_tensor)
print("\nrandn_like(input):")
print("Shape:", randn_like_tensor.shape)
print("Content:\n", randn_like_tensor)
```

```
$> python3 test.py
randn(*size):
Shape: torch.Size([2, 2])
Content:
tensor([[-0.5008,  1.1298],
        [ 0.7826,  0.1865]])

randn_like(input):
Shape: torch.Size([2, 2])
Content:
tensor([[ 0.0411, -0.9603],
        [-1.4047, -1.2398]])

$> python3 test.py
randn(*size):
Shape: torch.Size([2, 2])
Content:
tensor([[-0.9106,  0.6807],
        [-0.1289, -0.5300]])

randn_like(input):
Shape: torch.Size([2, 2])
Content:
tensor([[-0.2527, -0.8041],
        [ 0.9786,  2.0623]])
```

3) 从 0-1 均匀分布中抽取随机数创建张量

```
# 数据类型为浮点型且维度指定
torch.rand(size,          # 数据类型为浮点型且维度指定
                out=None,     # 输出张量
                dtype=None,   # 数据类型
                device=None, # 设备
                requires_grad=False)  # 是否需要梯度

# 生成一个与 input 形状相同的张量
torch.rand_like(input,  # 生成一个与 input 形状相同的张量
                        dtype=None,   # 数据类型
                        layout=None, # 默认为默认布局
                        device=None, # 默认为当前设备
                        requires_grad=False)  # 是否需要梯度
```

```python
# rand(*size)
import torch
rand_tensor = torch.rand(2, 2)
print("\nrand(*size):")
print("Shape:", rand_tensor.shape)
print("Content:\n", rand_tensor)

# rand_like(input)
rand_like_tensor = torch.rand_like(rand_tensor)
print("\nrand_like(input):")
print("Shape:", rand_like_tensor.shape)
print("Content:\n", rand_like_tensor)
```

```
$> python3 test.py
rand(*size):
Shape: torch.Size([2, 2])
Content:
tensor([[0.6926, 0.4433],
        [0.3605, 0.7165]])

rand_like(input):
Shape: torch.Size([2, 2])
Content:
tensor([[0.5039, 0.9240],
        [0.8755, 0.7009]])

$> python3 test.py
rand(*size):
Shape: torch.Size([2, 2])
Content:
tensor([[0.7404, 0.5186],
        [0.6042, 0.0406]])

rand_like(input):
Shape: torch.Size([2, 2])
Content:
tensor([[0.4726, 0.9517],
        [0.9636, 0.8840]])
```

```
# 同 torch.rand(size)  只不过得到的是整数随机数
torch.randint(low=0,    # 生成的随机数下界
              high,     # 生成的随机数上界
              size,     # 张量的形状
              out=None, # 输出张量
              dtype=None,   # 数据类型
              layout=None,  # 默认为默认布局
              device=None,  # 默认为当前设备
```

```
                requires_grad=False)  # 是否需要梯度

# 生成一个与 input 形状相同的张量
torch.randint_like(input,  # 输入张量
                        low,    # 生成的随机数下界
                        high,   # 生成的随机数上界
                        dtype=None,   # 数据类型
                        layout=None,  # 默认为默认布局
                        device=None,  # 默认为当前设备
                        requires_grad=False)  # 是否需要梯度
```

```python
import torch

# randint(low, high, *size)
rand_tensor = torch.randint(low=3, high=10, size=(2, 3))
print("\nrand(*size):")
print("Shape:", rand_tensor.shape)
print("Content:\n", rand_tensor)

print("------------------------------")
# randint_like(input)
rand_like_tensor = torch.randint_like(low=5, high=10,
    input=rand_tensor)
print("\nrand_like(input):")
print("Shape:", rand_like_tensor.shape)
print("Content:\n", rand_like_tensor)
```

```
rand(*size):
Shape: torch.Size([2, 3])
Content:
tensor([[7, 6, 9],
        [8, 7, 9]])
------------------------------

rand_like(input):
Shape: torch.Size([2, 3])
Content:
tensor([[5, 7, 7],
        [8, 7, 7]])
```

4) 生成从 0 到 n-1 的随机排列

```
torch.randperm(n,       # 生成的随机排列的长度
               out=None, # 输出张量
               dtype=None,   # 数据类型
               layout=torch.strided,  # 默认为默认布局
```

```
                device=None,  # 默认为当前设备
                requires_grad=False)  # 是否需要梯度
```

```
import torch

# randperm(n)
randperm_tensor = torch.randperm(10)
print("\nrandperm(n):")
print("Shape:", randperm_tensor.shape)
print("Content:\n", randperm_tensor)
```

```
randperm(n):
Shape: torch.Size([10])
Content:
        tensor([6, 2, 1, 0, 3, 4, 5, 7, 8, 9])
```

5) 生成以 input 为概率，生成伯努利分布 (0-1 分布，两点分布) 的随机数

```
torch.bernoulli(input,  # 输入张量
                out=None, # 输出张量
                generator=None)  # 生成器
```

```
import torch

# bernoulli(input)
bernoulli_tensor = torch.bernoulli(torch.tensor([[0.2,
    0.8],[0.5, 0.5]]))
print("\nbernoulli(input):")
print("Shape:", bernoulli_tensor.shape)
print("Content:\n", bernoulli_tensor)
```

```
bernoulli(input):
Shape: torch.Size([2, 2])
Content:
tensor([[0., 1.],
        [0., 0.]])
```

6) 生成一个空张量

```
torch.empty(size)    # 随机初始化
```

```
import torch

# empty(*size)
empty_tensor = torch.empty(2, 2)
print("\nempty(*size):")
print("Shape:", empty_tensor.shape)
print("Content:\n", empty_tensor)
```

(4) 根据数值创建 Tensor

1) 创建全 0 张量

```
torch.zeros(size, out, layout, device, requires_grad)   # 全0
# size 张量的形状
# out 输出张量
# layout 布局有strided、sparse_coo、mkldnn、sparse_csr、sparse_csc
#       strided: 一般的张量
#       sparse_coo: COO格式的稀疏张量
#       mkldnn: MKL-DNN格式的张量
#       sparse_csr: CSR格式的稀疏张量
#       sparse_csc: CSC格式的稀疏张量
# device 设备，如cpu、cuda、cuda:0、cpu:0
# requires_grad 是否需要梯度
```

```
import torch

zeros_tensor = torch.zeros(2, 2) # 创建一个2x2的全0张量
print("\nzeros(*size):")
print("Shape:", zeros_tensor.shape)
print("Content:\n", zeros_tensor)
```

```
$> python3 test.py
zeros(*size):
Shape: torch.Size([2, 2])
Content:
tensor([[0., 0.],
        [0., 0.]])
```

其中out类似 C++ 中的引用，可以直接修改原来的张量。

```python
import torch
out_t =torch.tensor([2])
print(out_t)
zeros_tensor = torch.zeros(3, 2, out=out_t)

print("zeros_tensor Shape:", zeros_tensor.shape)
print("zeros_tensor Content:\n", zeros_tensor)
print("zeros_tensor id", id(zeros_tensor))
print("-----------------------------")
print("out_t Shape:", out_t.shape)
print("out_t Content:\n", out_t)
print("out_t id", id(out_t))
print("out_t ?= zeros_tensor:", out_t is zeros_tensor)
```

```
$> python3 test.py
tensor([2])
zeros_tensor Shape: torch.Size([3, 2])
zeros_tensor Content:
tensor([[0, 0],
        [0, 0],
        [0, 0]])
zeros_tensor id 140167783140704
-----------------------------
out_t Shape: torch.Size([3, 2])
out_t Content:
tensor([[0, 0],
        [0, 0],
        [0, 0]])
out_t id 140167783140704

out_t ?= zeros_tensor: True
```

```
torch.zeros_like(input, dtype=None, layout=None, device=None,
    requires_grad=False)   # 全0
# input 输入张量
# dtype 数据类型
# layout 布局有strided、sparse_coo、mkldnn、sparse_csr、sparse_csc
#       strided: 一般的张量
#       sparse_coo: COO格式的稀疏张量
#       mkldnn: MKL-DNN格式的张量
#       sparse_csr: CSR格式的稀疏张量
#       sparse_csc: CSC格式的稀疏张量
```

```
# device 设备, 如cpu、cuda、cuda:0、cpu:0
# requires_grad 是否需要梯度
```

```
import torch

zeros_like_tensor = torch.zeros_like(torch.tensor([2]))
print("\nzeros_like(input):")
print("Shape:", zeros_like_tensor.shape)
print("Content:\n", zeros_like_tensor)
```

```
$> python3 test.py
zeros_like(input):
Shape: torch.Size([2])
Content:
tensor([0., 0.])
```

2) 创建全 1 张量

```
torch.ones(size, out, layout, device, requires_grad)   # 全1
# size 张量的形状
# out 输出张量
# layout 布局有strided、sparse_coo、mkldnn、sparse_csr、sparse_csc
#       strided: 一般的张量
#       sparse_coo: COO格式的稀疏张量
#       mkldnn: MKL-DNN格式的张量
#       sparse_csr: CSR格式的稀疏张量
#       sparse_csc: CSC格式的稀疏张量
# device 设备, 如cpu、cuda、cuda:0、cpu:0
# requires_grad 是否需要梯度
```

```
torch.ones_like(input, dtype=None, layout=None, device=None,
    requires_grad=False)   # 全1
# input 输入张量
# dtype 数据类型
# layout 布局有strided、sparse_coo、mkldnn、sparse_csr、sparse_csc
#       strided: 一般的张量
#       sparse_coo: COO格式的稀疏张量
#       mkldnn: MKL-DNN格式的张量
#       sparse_csr: CSR格式的稀疏张量
#       sparse_csc: CSC格式的稀疏张量
# device 设备, 如cpu、cuda、cuda:0、cpu:0
# requires_grad 是否需要梯度
```

```python
import torch
# ones(*size)
ones_tensor = torch.ones(2, 2)
print("\nones(*size):")
print("Shape:", ones_tensor.shape)
print("Content:\n", ones_tensor)

# ones_like(t)
ones_like_tensor = torch.ones_like(tensor)
print("\nones_like(t):")
print("Shape:", ones_like_tensor.shape)
print("Content:\n", ones_like_tensor)
```

```
$> python3 test.py
ones(*size):
Shape: torch.Size([2, 2])
Content:
tensor([[1., 1.],
        [1., 1.]])

ones_like(t):
Shape: torch.Size([2, 2])
Content:
tensor([[1., 1.],
        [1.,1.]])
```

3) 自定义全是某一数值

```
torch.full(size, fill_value, out, layout, device, requires_grad)
    # 自定义
# size 张量的形状
# fill_value 填充值
# out 输出张量
# layout 布局有strided、sparse_coo、mkldnn、sparse_csr、sparse_csc
#       strided: 一般的张量
#       sparse_coo: COO格式的稀疏张量
#       mkldnn: MKL-DNN格式的张量
#       sparse_csr: CSR格式的稀疏张量
#       sparse_csc: CSC格式的稀疏张量
# device 设备，如cpu、cuda、cuda:0、cpu:0
# requires_grad 是否需要梯度
```

```
torch.full_like(input, fill_value, dtype=None, layout=None,
    device=None, requires_grad=False)   # 自定义
# input 输入张量
# fill_value 填充值
```

```
# dtype 数据类型
# layout 布局有strided、sparse_coo、mkldnn、sparse_csr、sparse_csc
# device 设备，如cpu、cuda、cuda:0、cpu:0
# requires_grad 是否需要梯度
```

```
import torch

# full(*size, fill_value)
full_tensor = torch.full((2, 2), 3)
print("\nfull(*size, fill_value):")
print("Shape:", full_tensor.shape)
print("Content:\n", full_tensor)

tensor = torch.tensor([[1, 2, 6], [3, 4, 5]])
# full_like(input, fill_value)
full_like_tensor = torch.full_like(tensor, 5)
print("\nfull_like(input, fill_value):")
print("Shape:", full_like_tensor.shape)
print("Content:\n", full_like_tensor)
```

```
$> python3 test.py
full(*size, fill_value):
Shape: torch.Size([2, 2])
Content:
tensor([[3, 3],
        [3, 3]])

full_like(input, fill_value):
Shape: torch.Size([2, 3])
Content:
tensor([[5, 5, 5],
        [5, 5, 5]])
```

4) 创建等差数列

```
torch.arange(start, end, step, out, layout, device, requires_grad)
     # 等差数列
# start 起始值
# end 结束值
# step 步长
# out 输出张量
# layout 布局有strided、sparse_coo、mkldnn、sparse_csr、sparse_csc
# device 设备，如cpu、cuda、cuda:0、cpu:0
# requires_grad 是否需要梯度
```

```python
import torch

arange_tensor = torch.arange(2, 20, step=3)
print("\narange(start, end, step):")
print("Shape:", arange_tensor.shape)
print("Content:\n", arange_tensor)
```

```
$> python3 test.py
arange(start, end, step):
Shape: torch.Size([6])
Content:
tensor([ 2,  5,  8, 11, 14, 17])
```

5) 创建均分的1 维张量

```
torch.linspace(start, end, steps, out, layout, device,
    requires_grad)   # 均分数列
# start 起始值
# end 结束值
# steps 步数
# out 输出张量
# layout 布局有strided、sparse_coo、mkldnn、sparse_csr、sparse_csc
# device 设备，如cpu、cuda、cuda:0、cpu:0
# requires_grad 是否需要梯度
```

```python
import torch

linspace_tensor = torch.linspace(0, 1, steps=4)
print("\nlinspace(start, end, steps):")
print("Shape:", linspace_tensor.shape)
print("Content:\n", linspace_tensor)
```

```
$> python3 test.py
linspace(start, end, steps):
Shape: torch.Size([4])
Content:
tensor([0.0000, 0.3333, 0.6667, 1.0000])
```

6) 创建对数均分的1 维张量

```
torch.logspace(start, end, steps, base, out, dtype, layout, device,
    requires_grad)   # 对数均分数列
# start 起始值
# end 结束值
# steps 步数
# base 对数的底
# out 输出张量
# dtype 数据类型
# layout 布局有strided、sparse_coo、mkldnn、sparse_csr、sparse_csc
# device 设备，如cpu、cuda、cuda:0、cpu:0
# requires_grad 是否需要梯度
```

```
import torch

# 在 10^0 到 10^2 之间均匀取4个点
logspace_tensor = torch.logspace(0, 2, steps=6, base=10)
print("\nlogspace(start, end, steps):")
print("Shape:", logspace_tensor.shape)
print("Content:\n", logspace_tensor)

print("-----------------------------")
logspace_tensor = torch.logspace(0, 10, steps=6, base=2)
print("Shape:", logspace_tensor.shape)
print("Content:\n", logspace_tensor)
```

```
$> python3 test.py
logspace(start, end, steps):
Shape: torch.Size([6])
Content:
tensor([  1.0000,   2.5119,   6.3096,  15.8489,  39.8107, 100.0000])
-----------------------------
Shape: torch.Size([6])
Content:
tensor([1.0000e+00, 4.0000e+00, 1.6000e+01, 6.4000e+01, 2.5600e+02,
    1.0240e+03])
```

(5) 创建特殊张量

对角的 2 维张量

```
torch.eye(n, m=None, out, layout, device, requires_grad)   #
    对角线为1，其他为0
# n 行数
# m 列数
# out 输出张量
```

```
# layout 布局有strided、sparse_coo、mkldnn、sparse_csr、sparse_csc
# device 设备，如cpu、cuda、cuda:0、cpu:0
# requires_grad 是否需要梯度
```

```
import torch

eye_tensor = torch.eye(2, 3)
print("\neye(row, column):")
print("Shape:", eye_tensor.shape)
print("Content:\n", eye_tensor)
```

```
$> python3 test.py
eye(row, column):
Shape: torch.Size([2, 3])
Content:
tensor([[1., 0., 0.],
        [0., 1., 0.]])
```

3) Tensor 的转换

在实际项目中，我们接触到的数据类型有很多，比如 Int、list、NumPy 等。为了让数据在各个阶段畅通无阻，不同数据类型与 Tensor 之间的转换就非常重要了。

```
import torch
import numpy as np

# int 和 Tensor 之间的转换
a = torch.tensor(1)
print("Original Tensor a:", a)
b = a.item()  # Tensor 转 int
print("Converted to int b:", b)
print()

# list 和 Tensor 之间的转换
a = torch.tensor([1, 2, 3])
print("Original Tensor a:", a)
b = a.tolist()  # Tensor 转 list
print("Converted to list b:", b)
print()

```

```python
# NumPy 和 Tensor 之间的转换
a = torch.tensor([1, 2, 3])
print("Original Tensor a:", a)
b = a.numpy()  # Tensor 转 NumPy
print("Converted to NumPy array b:", b)
c = torch.from_numpy(b)  # NumPy 转 Tensor
print("Converted back to Tensor c:", c)
print()

# CPU 的 Tensor 和 GPU 的 Tensor 之间的转换
a = torch.tensor([1, 2, 3])
print("Original CPU Tensor a:", a)
if torch.cuda.is_available():
    b = a.cuda()  # CPU 转 GPU
    print("Converted to GPU Tensor b:", b)
    c = b.cpu()  # GPU 转 CPU
    print("Converted back to CPU Tensor c:", c)
else:
    print("CUDA is not available. Skipping GPU conversion.")
```

```
Original Tensor a: tensor(1)
Converted to int b: 1

Original Tensor a: tensor([1, 2, 3])
Converted to list b: [1, 2, 3]

Original Tensor a: tensor([1, 2, 3])
Converted to NumPy array b: [1 2 3]
Converted back to Tensor c: tensor([1, 2, 3])

Original CPU Tensor a: tensor([1, 2, 3])
Converted to GPU Tensor b: tensor([1, 2, 3], device='cuda:0')
Converted back to CPU Tensor c: tensor([1, 2, 3])
```

4) 数学运算函数

(1) torch.add() 张量逐元素相加

将两个张量相加。

```python
import torch
# 创建两个张量
tensor_a = torch.tensor([1, 2, 3])
tensor_b = torch.tensor([4, 5, 6])
# 逐元素加法
```

```
result = torch.add(tensor_a, tensor_b)
print(result)
```

```
$> python3 test.py
tensor([5, 7, 9])
```

(2) torch.addcdiv() 张量逐元素相加并除以第三个张量

```
torch.addcdiv(input, tensor1, tensor2, value=1, tensor3, out=None)   #
    张量逐元素相加并除以第三个张量
# input 输入张量
# tensor1 第一个张量
# tensor2 第二个张量
# value 默认为1, 可以设置除法因子
# tensor3 第三个张量
# out 输出张量
```

$$\text{output} = \text{tensor1} + \text{value} \times (\text{tensor2}/\text{tensor3})$$

其中的 tensor2/tensor3 是逐元素的除法。

```
import torch

# 创建三个张量
tensor_a = torch.tensor([1.0, 2.0, 3.0])
tensor_b = torch.tensor([4.0, 5.0, 6.0])
tensor_c = torch.tensor([2.0, 2.0, 2.0])

# 逐元素加法并除以第三个张量
result_addcdiv = torch.addcdiv(tensor_a, 2.0, tensor_b, tensor_c)
print("torch.addcdiv 的结果: ", result_addcdiv)
```

```
$> python3 test.py
torch.addcdiv 的结果:  tensor([5., 7., 9.])
```

(3) torch.addcmul() 张量逐元素相加并乘以第三个张量

```
torch.addcmul(input, tensor1, tensor2, value=1, tensor3, out=None)   #
    张量逐元素相加并乘以第三个张量
# input 输入张量
```

```
# tensor1 第一个张量
# tensor2 第二个张量
# value 默认为1, 可以设置乘法因子
# tensor3 第三个张量
# out 输出张量
```

$$\text{output} = \text{tensor1} + \text{value} \times (\text{tensor2} \times \text{tensor3})$$

其中的 tensor2 × tensor3 是逐元素的乘法。

```python
import torch

# 创建三个张量
tensor_a = torch.tensor([1.0, 2.0, 3.0])
tensor_b = torch.tensor([4.0, 5.0, 6.0])
tensor_c = torch.tensor([2.0, 2.0, 2.0])

# 逐元素加法并乘以第三个张量
result_addcmul = torch.addcmul(tensor_a, 2.0, tensor_b, tensor_c)
print("torch.addcmul 的结果: ", result_addcmul)
```

```
$> python3 test.py
torch.addcmul 的结果:  tensor([17., 22., 27.])
```

(4) torch.sub()

将两个张量相减。

```python
import torch
# 创建两个张量
tensor_a = torch.tensor([1, 2, 3])
tensor_b = torch.tensor([4, 5, 6])
# 逐元素减法
result = torch.sub(tensor_a, tensor_b)
print(result)
```

(5) torch.mul() 逐元素相乘

将两个张量 逐元素 相乘。

```python
import torch
# 创建两个张量
tensor_a = torch.tensor([1, 2, 3])
tensor_b = torch.tensor([4, 5, 6])
# 逐元素乘法
result = torch.mul(tensor_a, tensor_b)

# 简化写法
# result = tensor_a * tensor_b
print(result)
```

```
tensor([ 4, 10, 18])
```

(6) torch.div()

将两个张量 逐元素 相除。

```python
import torch
# 创建两个张量
tensor_a = torch.tensor([1, 2, 3])
tensor_b = torch.tensor([4, 5, 6])
# 逐元素除法
result = torch.div(tensor_a, tensor_b)
print(result)
```

(7) torch.matmul() 矩阵乘法

计算两个张量的矩阵乘法。

```python
import torch
# 创建两个2x3和3x2的矩阵
matrix_a = torch.tensor([[1, 2, 3], [4, 5, 6]])
matrix_b = torch.tensor([[7, 8], [9, 10], [11, 12]])
# 矩阵乘法
result = torch.matmul(matrix_a, matrix_b)

# 简化写法
```

```
# result = matrix_a @ matrix_b
# result = torch.mm(matrix_a, matrix_b)

print(result)
```

```
tensor([[ 58,  64],
        [139, 154]])
```

(8) torch.dot()

计算两个张量的点积。

```
import torch
# 创建两个1x3的向量
vector_a = torch.tensor([1, 2, 3])
vector_b = torch.tensor([4, 5, 6])
# 点积
result = torch.dot(vector_a, vector_b)
print(result)
```

(9) torch.pow()

计算张量的幂。

```
import torch
# 创建一个张量
tensor = torch.tensor([2, 3])
# 计算张量的幂
result = torch.pow(tensor, 2)
print(result)
```

(10) torch.exp()

计算张量的指数。

(11) torch.log()

计算张量的以 e 为底的自然对数。

```
import torch
import numpy as np

# 创建一个张量
tensor = torch.tensor([np.e**2 , np.e**3])
# 计算张量的以e为底的对数
result = torch.log(tensor)
print(result)
```

```
$> python3 test.py
tensor([2., 3.])
```

(12) torch.log2()

计算张量的以 2 为底的对数。

```
import torch
# 创建一个张量
tensor = torch.tensor([2, 3])
# 计算张量的以2为底的对数
result = torch.log2(tensor)
print(result)
```

(13) torch.log10()

计算张量的以 10 为底的对数。

```
import torch

# 创建一个张量
tensor = torch.tensor([pow(10, 10), 10**3])
# 计算张量的以10为底的对数
result = torch.log10(tensor)
```

```
print(result)
```

```
$> python3 test.py
tensor([10., 3.])
```

(14) torch.sqrt()

计算张量的平方根。

```
import torch
# 创建一个张量
tensor = torch.tensor([2, 3])
# 计算张量的平方根
result = torch.sqrt(tensor)
print(result)
```

(15) torch.sin()

计算张量的正弦值 $\sin(x)$。

```
import torch

# 创建一个张量
tensor = torch.tensor([0.0, torch.pi / 2, torch.pi])

# 计算张量的正弦值
result_sin = torch.sin(tensor)
print("torch.sin 结果: ", result_sin)
```

(16) torch.cos()

计算张量的余弦值 $\cos(x)$。

```
import torch
```

```
2
3  # 创建一个张量
4  tensor = torch.tensor([0.0, torch.pi / 2, torch.pi])
5
6  # 计算张量的余弦值
7  result_cos = torch.cos(tensor)
8  print("torch.cos 结果: ", result_cos)
```

(17) torch.tan()

计算张量的正切值 $\tan(x)$。

```
1  import torch
2
3  # 创建一个张量
4  tensor = torch.tensor([0.0, torch.pi / 4, torch.pi / 2])
5
6  # 计算张量的正切值
7  result_tan = torch.tan(tensor)
8  print("torch.tan 结果: ", result_tan)
```

(18) torch.asin()

计算张量的反正弦值，其结果在 $[-\frac{\pi}{2}, \frac{\pi}{2}]$ 之间。

```
1  import torch
2
3  # 创建一个张量
4  tensor = torch.tensor([0.0, 0.5, 1.0])
5
6  # 计算张量的反正弦值
7  result_asin = torch.asin(tensor)
8  print("torch.asin 结果: ", result_asin)
```

(19) torch.acos()

计算张量的反余弦值，其结果在 $[0, \pi]$ 之间。

```python
import torch

# 创建一个张量
tensor = torch.tensor([1.0, 0.5, 0.0])

# 计算张量的反余弦值
result_acos = torch.acos(tensor)
print("torch.acos 结果: ", result_acos)
```

(20) torch.atan()

计算张量的反正切值，其结果在 $[-\frac{\pi}{2}, \frac{\pi}{2}]$ 之间。

```python
import torch

# 创建一个张量
tensor = torch.tensor([0.0, 1.0, -1.0])

# 计算张量的反正切值
result_atan = torch.atan(tensor)
print("torch.atan 结果: ", result_atan)
```

(21) torch.atan2()

计算两个张量的反正切值，结果为从 x 轴正方向逆时针到点 (x, y) 的角度。

```python
import torch

# 创建两个张量
tensor_y = torch.tensor([1.0, -1.0, 0.0])
tensor_x = torch.tensor([1.0, 1.0, 1.0])

# 计算张量的 atan2 值
```

```
result_atan2 = torch.atan2(tensor_y, tensor_x)
print("torch.atan2 结果: ", result_atan2)
```

(22) torch.sinh()

计算张量的双曲正弦值 $\sinh(x) = \frac{e^x - e^{-x}}{2}$。

```
import torch

# 创建一个张量
tensor = torch.tensor([0.0, 1.0, -1.0])

# 计算张量的双曲正弦值
result_sinh = torch.sinh(tensor)
print("torch.sinh 结果: ", result_sinh)
```

(23) torch.cosh()

计算张量的双曲余弦值 $\cosh(x) = \frac{e^x + e^{-x}}{2}$。

```
import torch

# 创建一个张量
tensor = torch.tensor([0.0, 1.0, -1.0])

# 计算张量的双曲余弦值
result_cosh = torch.cosh(tensor)
print("torch.cosh 结果: ", result_cosh)
```

(24) torch.tanh()

计算张量的双曲正切值 $\tanh(x) = \frac{\sinh(x)}{\cosh(x)} = \frac{e^x - e^{-x}}{e^x + e^{-x}}$。

```python
import torch

# 创建一个张量
tensor = torch.tensor([0.0, 1.0, -1.0])

# 计算张量的双曲正切值
result_tanh = torch.tanh(tensor)
print("torch.tanh 结果: ", result_tanh)
```

(25) torch.mean()

计算张量的平均值。

```python
import torch
# 创建一个张量
tensor = torch.tensor([1.0, 2.0, 3.0, 4.0])
# 计算均值
mean_value = torch.mean(tensor)
print(mean_value)
```

(26) torch.sum()

计算张量的和。

```python
import torch
# 创建一个张量
tensor = torch.tensor([1, 2, 3, 4])
# 计算元素和
total_sum = torch.sum(tensor)
print(total_sum)
```

(27) torch 取整函数

1) `torch.floor()`: 向下取整

2) `torch.ceil()`: 向上取整

3) `torch.round()`: 四舍五入

```
import torch
a = torch.tensor(3.1415926)
print(torch.floor(a))  # tensor(3.)
print(torch.ceil(a))  # tensor(4.)
print(torch.round(a))  # tensor(3.)
```

5) 随机抽样函数

(1) torch.seed()

用于生成不确定的随机数，返回一个 64 位的数值。

```
import torch
# 设置随机种子
torch.seed()
```

(2) torch.manual_seed(seed)

设置随机种子，使得每次运行程序时，得到的随机数都一样。返回一个 torch.Generator 对象。

```
# 函数功能：确保生成的随机数都是固定的
torch.manual_seed(seed)
# seed: 随机种子，为int类型或long类型
```

```
import torch
# 设置随机种子
torch.manual_seed(0)
```

(3) torch.initial_seed()

```
# 函数功能：得到生成随机数的原始种子值。
torch.initial_seed()
```

```
import torch
# 得到生成随机数的原始种子值
torch.initial_seed()
```

```
5462750180411213428
```

(4) torch.get_rng_state()

```
# 函数功能：返回生成随机数的状态。
torch.get_rng_state()
```

```
import torch
# 获取生成随机数的状态
torch.get_rng_state()
```

(5) torch.set_rng_state(new_state)

```
# 函数功能：设置生成随机数的状态。
torch.set_rng_state(new_state)
# new_state: 生成随机数的状态，为torch.Generator对象
```

```
import torch
# 获取生成随机数的状态
torch.get_rng_state()
```

```
tensor([116, 134,  28,  ...,   0,   0,   0], dtype=torch.uint8)
```

6) 统计函数

(1) torch.prob() 所有元素的积

```
# 函数功能：返回输入张量所有元素的积
torch.prod(input, dtype=None)
# input: 输入张量
# dtype: 张量数据类型
```

```
import torch
# 创建一个二维张量
tensor = torch.tensor([[3, 2], [3, 4]])
# 计算所有元素的积
result = torch.prod(tensor)
print(result)
```

```
tensor(72)
```

(2) torch.sum() 所有元素的和

```
# 函数功能：数对输入的tensor数据的某一维度求和
torch.sum(input, dim, keepdim=False, dtype=None)
# input: 输入张量
# dim: 指定要计算的维度,可以是一个列表,
##
    当dim=0时，即第0个维度会缩减，也就是说将N行压缩成一行，故相当于对列进行求和；
## 当dim=1时，对行进行求和。
# keepdim: 是否保持张量的维度不变，默认为False
# dtype: 张量数据类型
```

```
import torch
# 创建一个二维张量
a = torch.rand(2, 2)
print(a)

```

```python
# 设置参数input和dim
a1 = torch.sum(a)
a2 = torch.sum(a, dim=(0, 1))
a3 = torch.sum(a, dim=0)
a4 = torch.sum(a, dim=1)

print(a1)
print(a2)
print(a3)
print(a4)
```

```
tensor([[0.2900, 0.8773],
        [0.7114, 0.4693]])
tensor(2.3479)
tensor(2.3479)
tensor([1.0014, 1.3466])
tensor([1.1673, 1.1807])
-------------------------
设置keepdim
tensor([[2.3479]])
tensor([[1.0014, 1.3466]])
tensor([[1.1673],
        [1.1807]])
```

(3) torch.mean() 所有元素的平均值

```
# 函数功能：数对输入的tensor数据的某一维度求平均值
torch.mean(input, dim=None, keepdim=False, dtype=None)
# input: 输入张量
# dim: 指定要计算的维度,可以是一个列表,
##
    当dim=0时，即第0个维度会缩减，也就是说将N行压缩成一行，故相当于对列进行求和;
## 当dim=1时，对行进行求和。
# keepdim: 是否保持张量的维度不变，默认为False
# dtype: 张量数据类型
```

```python
import torch
# 创建一个二维张量
a = torch.rand(2, 2)
print(a)

# 设置参数input和dim
a1 = torch.mean(a)
a2 = torch.mean(a, dim=(0, 1))
```

```
a3 = torch.mean(a, dim=0)
a4 = torch.mean(a, dim=1)

print(a1)
print(a2)
print(a3)
print(a4)
```

```
tensor([[0.7310, 0.0844],
        [0.4992, 0.6728]])
tensor(0.4969)
tensor(0.4969)
tensor([0.6151, 0.3786])
tensor([0.4077, 0.5860])
```

(4) torch.max() 所有元素的最大值

```
# 函数功能：数对输入的tensor数据的某一维度求最大值
torch.max(input, dim, keepdim=False, dtype=None)
# input: 输入张量
# dim: 指定要计算的维度,只能是一个值
##
    当dim=0时，即第0个维度会缩减，也就是说将N行压缩成一行，故相当于对列进行求和；
## 当dim=1时，对行进行求和。
# keepdim: 是否保持张量的维度不变，默认为False
# dtype: 张量数据类型
```

```
import torch
# 创建一个二维张量
a = torch.rand(2, 2)
print(a)

# 设置参数input和dim
a1 = torch.max(a)
# dim只能是一个值，不能是列表
# a2 = torch.max(a, dim=(0, 1))
a3 = torch.max(a, dim=0)
a4 = torch.max(a, dim=1)

print(a1)
print(a3)
print(a4)

```

```
print("-------------------------")
print("设置keepdim")
a5 = torch.max(a, dim=0, keepdim=True)
a6 = torch.max(a, dim=1, keepdim=True)
print(a5)
print(a6)
```

```
tensor([[0.4984, 0.7238],
        [0.9155, 0.6009]])
tensor(0.9155)
torch.return_types.max(
values=tensor([0.9155, 0.7238]),
indices=tensor([1, 0]))
torch.return_types.max(
values=tensor([0.7238, 0.9155]),
indices=tensor([1, 0]))
-------------------------
设置keepdim
torch.return_types.max(
values=tensor([[0.9155, 0.7238]]),
indices=tensor([[1, 0]]))
torch.return_types.max(
values=tensor([[0.7238],
        [0.9155]]),
indices=tensor([[1],
        [0]]))
```

(5) torch.min() 所有元素的最小值

```
# 函数功能: 数对输入的tensor数据的某一维度求最小值
torch.min(input, dim, keepdim=False, dtype=None)
# input: 输入张量
# dim: 指定要计算的维度,只能是一个值
##
    当dim=0时，即第0个维度会缩减，也就是说将N行压缩成一行，故相当于对列进行求和;
## 当dim=1时，对行进行求和。
# keepdim: 是否保持张量的维度不变，默认为False
# dtype: 张量数据类型
```

```
import torch
# 创建一个二维张量
a = torch.rand(2, 2)
print(a)

# 设置参数input和dim
```

```python
a1 = torch.min(a)
# dim只能是一个值，不能是列表
# a2 = torch.max(a, dim=(0, 1))
a3 = torch.min(a, dim=0)
a4 = torch.min(a, dim=1)

print(a1)
print(a3)
print(a4)

print("-------------------------")
print("设置keepdim")
a5 = torch.min(a, dim=0, keepdim=True)
a6 = torch.min(a, dim=1, keepdim=True)
print(a5)
print(a6)
```

```
tensor([[0.2834, 0.0715],
        [0.8795, 0.6110]])
tensor(0.0715)
torch.return_types.min(
values=tensor([0.2834, 0.0715]),
indices=tensor([0, 0]))
torch.return_types.min(
values=tensor([0.0715, 0.6110]),
indices=tensor([1, 1]))
-------------------------
设置keepdim
torch.return_types.min(
values=tensor([[0.2834, 0.0715]]),
indices=tensor([[0, 0]]))
torch.return_types.min(
values=tensor([[0.0715],
        [0.6110]]),
indices=tensor([[1],
        [1]]))
```

(6) torch.median() 所有元素的中位数

```python
# 函数功能：数对输入的tensor数据的某一维度求中位数
torch.median(input, dim, keepdim=False, dtype=None)
# input: 输入张量
# dim: 指定要计算的维度,只能是一个值
##
    当dim=0时，即第0个维度会缩减，也就是说将N行压缩成一行，故相当于对列进行求中位数;
## 当dim=1时，对行进行求中位数。
# keepdim: 是否保持张量的维度不变，默认为False
# dtype: 张量数据类型
```

```python
import torch
# 创建一个二维张量
a = torch.rand(2, 2)
print(a)

# 设置参数input和dim
a1 = torch.median(a)
# dim只能是一个值，不能是列表
# a2 = torch.median(a, dim=(0, 1))
a3 = torch.median(a,dim=0)
a4 = torch.median(a,dim=1)
print(a1)
print(a3)
print(a4)

print("-------------------------")
print("设置keepdim")
a5 = torch.median(a, dim=0, keepdim=True)
a6 = torch.median(a, dim=1, keepdim=True)
print(a5)
print(a6)
```

```
tensor([[0.8446, 0.7220],
        [0.2677, 0.3347]])
tensor(0.3347)
torch.return_types.median(
values=tensor([0.2677, 0.3347]),
indices=tensor([1, 1]))
torch.return_types.median(
values=tensor([0.7220, 0.2677]),
indices=tensor([1, 0]))
-------------------------
设置keepdim
torch.return_types.median(
values=tensor([[0.2677, 0.3347]]),
indices=tensor([[1, 1]]))
torch.return_types.median(
values=tensor([[0.7220],
        [0.2677]]),
indices=tensor([[1],
        [0]]))
```

(7) torch.mode() 所有元素的众数

```
# 函数功能：数对输入的tensor数据的某一维度求众数
torch.mode(input, dim, keepdim=False, dtype=None)
# input: 输入张量
# dim: 指定要计算的维度,只能是一个值
##
    当dim=0时，即第0个维度会缩减，也就是说将N行压缩成一行，故相当于对列进行求众数；
## 当dim=1时，对行进行求众数。
# keepdim: 是否保持张量的维度不变，默认为False
# dtype: 张量数据类型
```

```
import torch
# 创建一个二维张量
a = torch.rand(2, 2)
print(a)

# 设置参数input和dim
a1 = torch.mode(a)
# dim只能是一个值，不能是列表
# a2 = torch.mode(a, dim=(0, 1))
a3 = torch.mode(a, dim=0)
a4 = torch.mode(a, dim=1)
print(a1)
print(a3)
print(a4)

print("-------------------------")
print("设置keepdim")
a5 = torch.mode(a, dim=0, keepdim=True)
a6 = torch.mode(a, dim=1, keepdim=True)
print(a5)
print(a6)
```

```
tensor([[0.5236, 0.7934],
        [0.1625, 0.6901]])
torch.return_types.mode(
values=tensor([0.5236, 0.1625]),
indices=tensor([0, 0]))
torch.return_types.mode(
values=tensor([0.1625, 0.6901]),
indices=tensor([1, 1]))
torch.return_types.mode(
values=tensor([0.5236, 0.1625]),
indices=tensor([0, 0]))
```

```
-------------------------
设置keepdim
torch.return_types.mode(
values=tensor([[0.1625, 0.6901]]),
indices=tensor([[1, 1]]))
torch.return_types.mode(
values=tensor([[0.5236],
        [0.1625]]),
indices=tensor([[0],
        [0]]))
```

(8) torch.var() 所有元素的方差

$$\begin{cases} \mathrm{Var}(X) = \dfrac{1}{N-1}\sum_{i=1}^{N}(X_i - \bar{X})^2 & \text{(unbiased=True)} \\ \mathrm{Var}(X) = \dfrac{1}{N}\sum_{i=1}^{N}(X_i - \bar{X})^2 & \text{(unbiased=False)} \end{cases}$$

1) N 是样本数。

2) X_i 是样本中的每个元素。

3) $\bar{X}$ 是样本的平均值。

```
# 函数功能：数对输入的tensor数据的某一维度求方差
torch.var(input, dim, keepdim=False, unbiased=True, *, out=None,
    dtype=None)
# input: 输入张量
# dim: 指定要计算的维度,只能是一个值
#
    当dim=0时，即第0个维度会缩减，也就是说将N行压缩成一行，故相当于对列进行求方差；
#       当dim=1时，对行进行求方差。
# keepdim: 是否保持张量的维度不变，默认为False
# unbiased: 是否使用无偏估计，
#       当unbiased=True时，将使用“贝塞尔校正”更正。
#       当unbiased=False时，将通过有偏估计量计算方差
# out: 输出张量
# dtype: 张量数据类型
```

```
1 import torch
2 # 创建一个二维张量
3 a = torch.rand(2, 2)
4 print(a)
```

```
5
6  # 设置参数input和dim
7  a1 = torch.var(a)
8  # dim只能是一个值，不能是列表
9  # a2 = torch.var(a, dim=(0, 1))
10 a3 = torch.var(a, dim=0)
11 a4 = torch.var(a, dim=1)
12 print(a1)
13 print(a3)
14 print(a4)
15
16 print("-------------------------")
17 print("设置unbiased")
18 a5 = torch.var(a, dim=0, unbiased=False)
19 a6 = torch.var(a, dim=1, unbiased=False)
20 a7 = torch.var(a, dim=0, unbiased=True)
21 a8 = torch.var(a, dim=1, unbiased=True)
22 print(a5)
23 print(a6)
24 print(a7)
25 print(a8)
26
27 print("-------------------------")
28 print("设置keepdim")
29 a9 = torch.var(a, dim=0, keepdim=True)
30 a10 = torch.var(a, dim=1, keepdim=True)
31 print(a9)
32 print(a10)
```

```
tensor([[0.9427, 0.5881],
        [0.5058, 0.5132]])
tensor(0.0428)
tensor([0.0955, 0.0028])
tensor([6.2864e-02, 2.7513e-05])
-------------------------
设置unbiased
tensor([0.0477, 0.0014])
tensor([3.1432e-02, 1.3756e-05])
tensor([0.0955, 0.0028])
tensor([6.2864e-02, 2.7513e-05])
-------------------------
设置keepdim
tensor([[0.0955, 0.0028]])
tensor([[6.2864e-02],
        [2.7513e-05]])
```

例子 47.1

有一个二维张量

$$a = \begin{bmatrix} 0.9427 & 0.5881 \\ 0.5058 & 0.5132 \end{bmatrix}$$

，求其方差。

方法:

样本均值为：

$$\bar{a} = \frac{0.9427 + 0.5881 + 0.5058 + 0.5132}{4} = \frac{2.5498}{4} = 0.63745$$

计算每个元素与均值的差的平方：

$$(0.9427 - 0.63745)^2 = (0.30525)^2 = 0.09318,$$
$$(0.5881 - 0.63745)^2 = (-0.04935)^2 = 0.00244,$$
$$(0.5058 - 0.63745)^2 = (-0.13165)^2 = 0.01733,$$
$$(0.5132 - 0.63745)^2 = (-0.12425)^2 = 0.01543.$$

将以上结果相加：

$$\sum = 0.09318 + 0.00244 + 0.01733 + 0.01543 = 0.12838$$

1) a_1 未指定 `unbiased`，默认 `unbiased=True`，所以使用无偏估计：

$$\mathrm{Var}(a_1) = \frac{\sum}{4-1} = \frac{0.12838}{3} = 0.04279 \approx 0.0428.$$

2) a_3 计算沿第 0 维（列）方向的方差，表示对每一列进行计算。

1) 对第 1 列：[0.9427, 0.5058]

均值为：

$$\bar{a}_{\mathrm{col1}} = \frac{0.9427 + 0.5058}{2} = 0.72425.$$

方差计算：

$$(0.9427 - 0.72425)^2 = (0.21845)^2 = 0.0477,$$
$$(0.5058 - 0.72425)^2 = (-0.21845)^2 = 0.0477.$$

无偏估计方差：

$$\mathrm{Var} = \frac{0.0477 + 0.0477}{2-1} = \frac{0.0954}{1} = 0.0954 \approx 0.0955.$$

2) 对第 2 列：[0.5881, 0.5132]

均值为：

$$\bar{a}_{\mathrm{col2}} = \frac{0.5881 + 0.5132}{2} = 0.55065.$$

方差计算：

$$(0.5881 - 0.55065)^2 = (0.03745)^2 = 0.0014,$$
$$(0.5132 - 0.55065)^2 = (-0.03745)^2 = 0.0014.$$

无偏估计方差：

$$\mathrm{Var} = \frac{0.0014 + 0.0014}{2-1} = \frac{0.0028}{1} = 0.0028$$

因此 $a_3 = [0.0955, 0.0028]$。

3) a_4 计算沿第 1 维（行）方向的方差，表示对每一行进行计算。

1) 对第 1 行：[0.9427, 0.5881]

均值为：

$$\bar{a}_{\text{row1}} = \frac{0.9427 + 0.5881}{2} = 0.7654$$

方差计算：

$$(0.9427 - 0.7654)^2 = (0.1773)^2 = 0.03145,$$

$$(0.5881 - 0.7654)^2 = (-0.1763)^2 = 0.03145.$$

无偏估计方差：

$$\text{Var} = \frac{0.03145 + 0.03145}{2 - 1} = \frac{0.06864}{1} = 0.06864 \approx 0.0686.$$

2) 对第 2 行：[0.5058, 0.5132]

均值为：

$$\bar{a}_{\text{row2}} = \frac{0.5058 + 0.5132}{2} = 0.5095$$

方差计算：

$$(0.5058 - 0.5095)^2 = (0.00307)^2 = 0.00001369,$$

$$(0.5132 - 0.5095)^2 = (0.00435)^2 = 0.00001369.$$

无偏估计方差：

$$\text{Var} = \frac{0.00001369 + 0.00001369}{2 - 1} = \frac{0.000027513}{1} = 0.000027513 \approx 0.00002751.$$

因此 $a_4 = [6.2864e-02, 2.7513e-05]$。

4) a_5 `unbiased=False`，使用有偏估计，公式为：

$$\text{Var}(X) = \frac{1}{N}\sum_{i=1}^{N}(X_i - \bar{X})^2$$

1) 对第 1 列：[0.9427, 0.5058]

$$\text{Var} = \frac{0.0477 + 0.0477}{2} = \frac{0.0954}{2} = 0.0477.$$

2) 对第 2 列：[0.5881, 0.5132]

$$Var = \frac{0.0014 + 0.0014}{2} = \frac{0.0028}{2} = 0.0014.$$

因此 $a_5 = [0.0477, 0.0014]$。

5) a_6 `unbiased=False`，使用有偏估计

1) 对第 1 列：[0.9427, 0.5881]

$$\text{Var} = \frac{0.03145 + 0.03145}{2} = \frac{0.06864}{2} = 0.031432.$$

2) 对第 2 列：[0.5058, 0.5132]

$$\text{Var} = \frac{0.00001369 + 0.00001369}{2} = \frac{0.000027513}{2} = 0.0000137565.$$

因此 $a_6 = [3.1432e-02, 1.3756e-05]$。

(9) torch.std() 所有元素的标准差

```
torch.std(input, dim=None, *, unbiased=True, keepdim=False)
```

```
import torch
# 创建一个二维张量
a = torch.rand(2, 2)
print(a)
print(torch.std(a))
print(torch.std(a, dim=0))
print(torch.std(a, dim=1))
```

```
tensor([[0.4939, 0.3728],
        [0.5331, 0.4765]])
tensor(0.0684)
tensor([0.0277, 0.0733])
tensor([0.0857, 0.0400])
```

7) Tensor 的形状

在处理数据、构建网络层等过程中，我们经常需要了解并修改 Tensor 的形状。

如表47.3所示，PyTorch 提供了很多修改 Tensor 形状的函数。

函数	说明
`size()`	返回张量的 `shape` 属性值，与函数 `shape(0.4 版新增)` 等价
`numel(input)`	计算 Tensor 的元素个数
`view(*shape)`	修改 Tensor 的形状，与 `reshape(0.4 版新增)` 类似， 但 `view` 返回的对象与源 Tensor 共享内存， 修改一个 Tensor 会同时修改另一个 Tensor， `reshape` 将生成全新的 Tensor，而且不要求源 Tensor 是连续的。
`resize`	类似于 `view`，但在 `size` 超出阈值时会重新分配内存空间
`item`	若 Tensor 为单元素，则返回 Python 的标量
`unsqueeze`	在指定维度增加一个“1”
`squeeze`	在指定维度压缩一个“1”

表 47.3 常用修改 Tensor 形状的函数

(1) 获取 Tensor 的形状

为了得到 Tensor 的形状，我们可以使用 `shape` 或 `size` 来获取。两者的不同之处在于，`shape` 是 `torch.tensor` 的一个属性，而 `size()` 则是一个 `torch.tensor` 拥有的方法。

```python
import torch
# 生成一个形状为2X3的矩阵
a = torch.randn (2, 3)
print("a :", a)
# 查看a的形状
print("a.shape :", a.shape)  # torch.Size([3])
print("a.size() :", a.size())  # torch.Size([3])
# 查看a的维度
print("a.dim() :", a.dim())  # 2
# 查看a的元素个数
print("a.numel() :", a.numel())  # 6
```

```
a : tensor([[ 1.2820, -0.0560,  1.4295],
        [ 0.1141,  0.2860,  0.6796]])
a.shape : torch.Size([2, 3])
a.size() : torch.Size([2, 3])
a.dim() : 2
a.numel() : 6
```

(2) Tensor 的形状变换

在 PyTorch 中有两种常用的改变形状的函数，分别是 `view` 和 `reshape`。

1) `view` `view()`。只可以由 `torch.Tensor.view()` 调用

```
# view() 用于改变形状
torch.view(input, shape)
# input: 输入张量
# shape: 新的形状
```

```python
import torch

# 生成一个形状为 2x3x5 的张量
a = torch.rand(2, 3, 5)
print("Original a:", a)
```

```
    print("Original a.shape:", a.shape)  # torch.Size([2, 3,
    5])
    print("Original a.dim():", a.dim())  # 3
    print("Original a.numel():", a.numel())  # 30

    # 将张量 a 的形状变为 (2, 15)
    # 调用 a.view(2, 15) 和 a.view(-1) 并没有改变 a
    张量的形状,
    # 这是因为 view()
    方法不会在原地 (in-place) 修改张量的形状,
    # 而是返回一个具有新形状的张量。
    a = a.view(2, 15)
    print("After view(2, 15), a:", a)
    print("a.shape:", a.shape)  # torch.Size([2, 15])
    print("a.dim():", a.dim())  # 2
    print("a.numel():", a.numel())  # 30

    # 将张量 a 变平为一维张量
    a = a.view(-1)
    print("After view(-1), a:", a)
    print("a.shape:", a.shape)  # torch.Size([30])
    print("a.dim():", a.dim())  # 1
    print("a.numel():", a.numel())  # 30

```

```
Original a: tensor([[[0.7896, 0.0760, 0.3403, 0.2851, 0.5863],
        [0.7447, 0.4550, 0.4071, 0.2395, 0.2282],
        [0.8083, 0.2299, 0.2049, 0.7512, 0.1109]],

        [[0.8561, 0.9217, 0.1821, 0.8229, 0.8569],
        [0.3844, 0.5223, 0.9659, 0.3908, 0.9212],
        [0.2532, 0.2833, 0.9911, 0.9758, 0.8310]]])
Original a.shape: torch.Size([2, 3, 5])
Original a.dim(): 3
Original a.numel(): 30

After view(2, 15), a: tensor([[0.7896, 0.0760, 0.3403, 0.2851,
0.5863, 0.7447, 0.4550, 0.4071, 0.2395,
        0.2282, 0.8083, 0.2299, 0.2049, 0.7512, 0.1109],
        [0.8561, 0.9217, 0.1821, 0.8229, 0.8569, 0.3844,
0.5223, 0.9659, 0.3908,
        0.9212, 0.2532, 0.2833, 0.9911, 0.9758, 0.8310]])
a.shape: torch.Size([2, 15])
a.dim(): 2
a.numel(): 30

After view(-1), a: tensor([0.7896, 0.0760, 0.3403, 0.2851,
0.5863, 0.7447, 0.4550, 0.4071, 0.2395,
        0.2282, 0.8083, 0.2299, 0.2049, 0.7512, 0.1109, 0.8561,
0.9217, 0.1821,
```

```
        0.8229, 0.8569, 0.3844, 0.5223, 0.9659, 0.3908, 0.9212,
0.2532, 0.2833,
        0.9911, 0.9758, 0.8310])
a.shape: torch.Size([30])
a.dim(): 1
a.numel(): 30
```

2) `reshape` `reshape()`。可以由 `torch.reshape()` 与 `torch.Tensor.reshape()` 调用

```
# reshape() 用于改变形状
torch.reshape(input, shape)
# input: 输入张量
# shape: 新的形状
```

注意：`reshape` 方法是一个不改变原张量的操作，它返回的是一个新的张量，而不是在原地对 a 进行修改。

```
import torch

a = torch.rand(2,3,5)  # (2,3,4)
print("a shape:", a.shape)
b = a.reshape(2,15)  # 将张量a的形状变为(2,15)
print("a shape:", a.shape)
print("b shape:", b.shape)

```

```
a shape: torch.Size([2, 3, 5])
a shape: torch.Size([2, 3, 5])
b shape: torch.Size([2, 15])
```

注意：当张量在内存中是连续时，使用`reshape`之后新张量和原张量共享内存，即修改一个张量会影响另一个张量。

```
import torch

a = torch.rand(2,1,3)
print("a shape:", a.shape)
print("a:", a)
print("----------------------")
```

```
    b = a.reshape(3,2)  # 将张量a的形状变为(2,15)
    print("a shape:", a.shape)
    print("b shape:", b.shape)
    a[0,0,0]= 100
    print("a:", a)
    print("b:", b)

```

```
a shape: torch.Size([2, 1, 3])
a: tensor([[[0.2559, 0.8705, 0.6568]],

        [[0.9167, 0.8845, 0.8788]]])
----------------------
a shape: torch.Size([2, 1, 3])
b shape: torch.Size([3, 2])
# 发现下面的a和b的第一个元素都变成了100
a: tensor([[[100.0000,   0.8705,   0.6568]],

        [[  0.9167,   0.8845,   0.8788]]])
b: tensor([[100.0000,   0.8705],
        [  0.6568,   0.9167],
        [  0.8845,   0.8788]])
```

上面看起来操作是一样的，但是 `view` 和 `reshape` 的区别在于，`view` 函数只能用于 contiguous Tensor，也就是说必须是内存中连续存储的，而 `reshape` 函数没有这个限制。

```
x = torch.randn(4, 4)
x = x.permute(1, 0)
x = x.view(16)  # 报错
x = x.reshape(16)  # 不报错
```

`permute(1, 0)` 会交换张量的第 0 和第 1 个维度，使得 x 的形状变为 `[4, 4]`，但元素顺序会改变，内存布局可能不再是连续的。

```
x:
tensor([[-1.3285,  0.1734,  1.3999,  1.2230],
        [-0.2893,  1.1102,  1.2439,  0.2771],
        [ 1.0310,  0.7060,  1.1739,  0.2482],
        [ 0.9299,  0.9445,  2.2493, -0.1438]])

x.permute(1, 0):
tensor([[-1.3285, -0.2893,  1.0310,  0.9299],
```

```
        [ 0.1734,  1.1102,  0.7060,  0.9445],
        [ 1.3999,  1.2439,  1.1739,  2.2493],
        [ 1.2230,  0.2771,  0.2482, -0.1438]])
```

8) Tensor 的索引

但有的时候，我们只需要其中的一部分而不是全部，这时候就需要用到索引操作。

函数	说明
`index_select(input, dim, index)`	在指定维度上选择一些行或列
`nonzero(input)`	获取非 0 元素的下标
`masked_select(input, mask)`	使用二元值进行选择
`gather(input, dim, index)`	在指定维度上选择数据，输出的形状与 `index`（`index` 的类型必须是 `LongTensor` 类型）一致
`scatter_(input, dim, index, src)`	`gather` 的反操作，根据指定索引补充数据

表 47.4 常用的选择函数

PyTorch 中的 Tensor 支持索引操作，可以通过索引获取 Tensor 中的元素。就是`index_select`和`masked_select`函数。

(1) index_select() 按照索引获取张量中的元素

```
# index_select() 按照索引获取张量中的元素
torch.index_select(tensor, dim, index, out=None)
# tensor: 输入张量
# dim: 指定维度
# index: 索引，可以是一个整数，也可以是一个一维张量
```

```
1  import torch
2
3  # 示例1: 一维张量，按索引选择元素
4  tensor_1d = torch.tensor([10, 20, 30, 40, 50])
5
6  # 使用index_select在dim=0上根据索引[0, 2, 4]选择元素
7  selected_1d = torch.index_select(tensor_1d, 0, torch.tensor([0,
       2, 4]))
8  print("示例1: 一维张量，按索引选择元素")
9  print("原始张量:", tensor_1d)
10 print("选择结果:", selected_1d)
11
12 # 示例2: 二维张量，按行选择
```

```
13 tensor_2d = torch.tensor([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
14
15 # 使用index_select在dim=0上根据索引[0, 2]选择行
16 selected_rows = torch.index_select(tensor_2d, 0, torch.tensor([0,
       2]))
17 print("\n示例2: 二维张量, 按行选择")
18 print("原始张量:\n", tensor_2d)
19 print("选择结果 (行) :\n", selected_rows)
20
21 # 示例3: 二维张量, 按列选择
22 # 使用index_select在dim=1上根据索引[1, 2]选择列
23 selected_cols = torch.index_select(tensor_2d, 1, torch.tensor([1,
       2]))
24 print("\n示例3: 二维张量, 按列选择")
25 print("选择结果 (列) :\n", selected_cols)
26
27 # 示例4: 三维张量, 按指定维度选择元素
28 tensor_3d = torch.tensor([
29     [[1, 2], [3, 4]],
30     [[5, 6], [7, 8]],
31     [[9, 10], [11, 12]]
32 ])
33
34 # 使用index_select在dim=0上根据索引[0, 2]选择
35 selected_3d_dim0 = torch.index_select(tensor_3d, 0,
       torch.tensor([0, 2]))
36 print("\n示例4: 三维张量, 按dim=0选择")
37 print("原始张量:\n", tensor_3d)
38 print("选择结果 (dim=0) :\n", selected_3d_dim0)
39
40 # 使用index_select在dim=1上根据索引[1]选择
41 selected_3d_dim1 = torch.index_select(tensor_3d, 1,
       torch.tensor([1]))
42 print("\n按dim=1选择:\n", selected_3d_dim1)
43
44 # 使用index_select在dim=2上根据索引[0]选择
45 selected_3d_dim2 = torch.index_select(tensor_3d, 2,
       torch.tensor([0]))
46 print("\n按dim=2选择:\n", selected_3d_dim2)
47
48 # 示例5: 四维张量, 按任意维度选择
49 tensor_4d = torch.randn(2, 3, 4, 5)  # 生成一个随机的四维张量
50
51 # 在dim=1上根据索引[0, 2]选择
52 selected_4d = torch.index_select(tensor_4d, 1, torch.tensor([0,
       2]))
53 print("\n示例5: 四维张量, 按dim=1选择")
54 print("选择结果 (dim=1) : \n", selected_4d)
```

```
示例1: 一维张量, 按索引选择元素
原始张量: tensor([10, 20, 30, 40, 50])
选择结果: tensor([10, 30, 50])

示例2: 二维张量, 按行选择
原始张量:
    tensor([[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]])
选择结果 (行) :
    tensor([[1, 2, 3],
        [7, 8, 9]])

示例3: 二维张量, 按列选择
选择结果 (列) :
    tensor([[2, 3],
        [5, 6],
        [8, 9]])

示例4: 三维张量, 按dim=0选择
原始张量:
    tensor([[[ 1,  2],
             [ 3,  4]],

            [[ 5,  6],
             [ 7,  8]],

            [[ 9, 10],
             [11, 12]]])
选择结果 (dim=0) :
    tensor([[[ 1,  2],
             [ 3,  4]],

            [[ 9, 10],
             [11, 12]]])

按dim=1选择:
    tensor([[[ 3,  4]],

            [[ 7,  8]],

            [[11, 12]]])

按dim=2选择:
    tensor([[[ 1],
             [ 3]],

            [[ 5],
             [ 7]],

            [[ 9],
             [11]]])

示例5: 四维张量, 按dim=1选择
```

```
选择结果 (dim=1) :
    tensor([[[[-0.1890, -0.7456, -1.1377, -1.4363,  1.7878],
              [-0.5825,  2.6052,  0.6801,  0.6008,  1.4784],
              [-0.4649, -1.1073, -1.7590,  0.1953, -0.7053],
              [ 0.1035, -1.5305,  0.5679, -0.7610,  1.1759]],

             [[ 0.4405,  0.8257, -1.6868,  0.5713, -0.8757],
              [ 0.8236,  0.8513, -1.2844, -1.3713, -0.5326],
              [ 0.9026, -0.0555, -0.2652, -0.3538, -1.1889],
              [-0.4444, -1.6979,  0.8575, -0.2297,  0.3860]]],


            [[[ 0.2512, -0.0789,  0.5589,  0.5068,  1.7785],
              [-0.8088, -0.3661, -2.0435, -1.0728, -0.5933],
              [-0.4169, -0.4006,  0.3270,  0.9387, -0.1128],
              [-1.1071, -2.2229, -0.1993, -1.2302,  0.2496]],

             [[-1.2770, -2.5906, -0.3730, -1.1978,  1.0446],
              [-0.8497, -1.8485,  0.0233, -0.5376,  0.2462],
              [ 0.0170, -0.8262,  0.3325, -0.5890,  0.2073],
              [ 1.0971,  2.3874,  0.9752, -0.1202, -0.0352]]]])
```

使用索引列表的时候，索引列表的类型**必须**是 `torch.long` 类型。

```python
import torch

t = torch.randint(0, 10, (2, 3))
# 索引张量
idx = torch.tensor([0, 1], dtype=torch.long) # 这里的dtype必须是torch.long
print("t: ", t)
print("t[idx]: ", t[idx])
t_select = torch.index_select(t, dim=0, index=idx)
print("torch.index_select(t, dim=0, index=idx): ", t_select)
```

```
t:  tensor([[7, 0, 9],
        [6, 5, 2]])
t[idx]:  tensor([[7, 0, 9],
        [6, 5, 2]])
torch.index_select(t, dim=0, index=idx):  tensor([[7, 0, 9],
        [6, 5, 2]])
```

(2) masked_select() 按照掩码获取张量中的元素

`indexed_select`是基于给定的索引来进行数据提取的。但有的时候，我们还想通过一些判断条件来进行选择，比如提取深度学习网络中某一层中数

值大于 0 的参数。

```
# masked_select() 按照掩码获取张量中的元素
torch.masked_select(tensor, mask, out=None)
# tensor: 输入张量
# mask: 掩码，与输入张量的形状相同，元素为0或1
```

```python
import torch

# 示例1：一维张量，使用布尔掩码选择元素
tensor_1d = torch.tensor([10, 20, 30, 40, 50])

# 使用布尔掩码选择大于30的元素
mask_1d = tensor_1d > 30
selected_1d = torch.masked_select(tensor_1d, mask_1d)
print("示例1：一维张量，使用布尔掩码选择元素")
print("原始张量:", tensor_1d)
print("掩码:", mask_1d)
print("选择结果:", selected_1d)

# 示例2：二维张量，选择符合条件的元素
tensor_2d = torch.tensor([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

# 使用掩码选择大于等于5的元素
mask_2d = tensor_2d >= 5
selected_2d = torch.masked_select(tensor_2d, mask_2d)
print("\n示例2：二维张量，选择符合条件的元素")
print("原始张量:\n", tensor_2d)
print("掩码:\n", mask_2d)
print("选择结果:", selected_2d)

# 示例3：二维张量，自定义布尔掩码
# 使用自定义布尔掩码选择指定位置的元素
custom_mask_2d = torch.tensor([[True, False, True], [False, True,
    False], [True, False, True]])
selected_custom_2d = torch.masked_select(tensor_2d,
    custom_mask_2d)
print("\n示例3：二维张量，自定义布尔掩码")
print("自定义掩码:\n", custom_mask_2d)
print("选择结果:", selected_custom_2d)

# 示例4：三维张量，按掩码选择
tensor_3d = torch.tensor([
    [[1, 2], [3, 4]],
    [[5, 6], [7, 8]],
    [[9, 10], [11, 12]]
])
```

```python

# 使用掩码选择大于5的元素
mask_3d = tensor_3d > 5
selected_3d = torch.masked_select(tensor_3d, mask_3d)
print("\n示例4: 三维张量, 按掩码选择")
print("原始张量:\n", tensor_3d)
print("掩码:\n", mask_3d)
print("选择结果:", selected_3d)

# 示例5: 四维张量, 随机生成掩码选择
tensor_4d = torch.randn(2, 3, 4)  # 生成一个随机四维张量

# 生成随机布尔掩码, 选择正数的元素
mask_4d = tensor_4d > 0
selected_4d = torch.masked_select(tensor_4d, mask_4d)
print("\n示例5: 四维张量, 随机生成掩码选择")
print("原始张量:\n", tensor_4d)
print("掩码:\n", mask_4d)
print("选择结果:", selected_4d)
```

```
示例1: 一维张量, 使用布尔掩码选择元素
原始张量: tensor([10, 20, 30, 40, 50])
掩码: tensor([False, False, False,  True,  True])
选择结果: tensor([40, 50])

示例2: 二维张量, 选择符合条件的元素
原始张量:
    tensor([[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]])
掩码:
    tensor([[False, False, False],
        [False,  True,  True],
        [ True,  True,  True]])
选择结果: tensor([5, 6, 7, 8, 9])

示例3: 二维张量, 自定义布尔掩码
自定义掩码:
    tensor([[ True, False,  True],
        [False,  True, False],
        [ True, False,  True]])
选择结果: tensor([1, 3, 5, 7, 9])

示例4: 三维张量, 按掩码选择
原始张量:
    tensor([[[ 1,  2],
             [ 3,  4]],

            [[ 5,  6],
             [ 7,  8]],
```

```
        [[ 9, 10],
            [11, 12]]])
掩码:
    tensor([[[False, False],
             [False, False]],

            [[False,  True],
             [ True,  True]],

            [[ True,  True],
             [ True,  True]]])
选择结果: tensor([ 6,  7,  8,  9, 10, 11, 12])

示例5: 四维张量, 随机生成掩码选择
原始张量:
    tensor([[[ 1.6190, -1.5189,  0.2127, -0.1679],
             [ 1.5070,  0.9289,  0.8709, -1.2303],
             [ 0.7638, -0.3387,  0.4020, -0.7013]],

            [[ 1.4183,  0.8994, -1.4394,  1.0644],
             [-0.0089,  0.0351, -0.5800, -0.8311],
             [ 1.0684, -0.7077,  1.0300, -1.2010]]])
掩码:
    tensor([[[ True, False,  True, False],
             [ True,  True,  True, False],
             [ True, False,  True, False]],

            [[ True,  True, False,  True],
             [False,  True, False, False],
             [ True, False,  True, False]]])
选择结果: tensor([1.6190, 0.2127, 1.5070, 0.9289, 0.8709, 0.7638, 0.4020,
    1.4183, 0.8994,
        1.0644, 0.0351, 1.0684, 1.0300])
```

9) Tensor 的广播机制

在 PyTorch 中，如果两个张量的形状不同，PyTorch 会尝试使用广播机制来尝试将两个张量的形状进行适配。

广播机制的规则如下：

1. 如果张量的维度不同，将维度较小的张量进行扩展，直到两个张量的维度都一样。
2. 如果两个张量在某个维度上的长度是相同的，或者其中一个张量在该维度上的长度为 1，那么我们就说这两个张量在该维度上是相容的。
3. 如果两个张量在所有维度上都是相容的，它们就能使用广播。
4. 如果两个张量在某个维度上的长度不同且不为 1，那么这两个张量是不兼容的。如果两个张量是不兼容的，那么 PyTorch 将会抛出一个错误。

```python
import torch
import numpy as np

A = np.arange(0, 40,10).reshape(4, 1)
B = np.arange(0, 3)
# 把 ndarray 转换为 Tensor ,
A1=torch.from_numpy (A) # 形状为 4 x 1
B1=torch.from_numpy (B) # 形状为 3
print("A1:", A1)
print("A1.shape:", A1.shape)
print("B1:", B1)
print("B1.shape:", B1.shape)

# Tensor自动实现广播
C=A1+B1
print("C:", C)
print("C.shape:", C.shape)

##################################
# 我们可以根据广播机制手工进行配置
# 根据规则1，需要向A1看齐，把B1变为(1, 3 )
B2=B1.unsqueeze (0) #B2 的形状为 1X3
print("B2:", B2)
print("B2.shape:", B2.shape)
# 使用expand函数重复数组，分别转变为4X3的矩阵
A2=A1.expand(4, 3)
print("A2:", A2)
print("A2.shape:", A2.shape)
B3=B2.expand(4, 3)
print("B3:", B3)
print("B3.shape:", B3.shape)
# 然后进行相加，C1与C结果一致
C1=A2+B3
print("C1:", C1)
print("C1.shape:", C1.shape)
```

```
A1: tensor([[ 0],
        [10],
        [20],
        [30]])
A1.shape: torch.Size([4, 1])
B1: tensor([0, 1, 2])
B1.shape: torch.Size([3])
C: tensor([[ 0,  1,  2],
        [10, 11, 12],
        [20, 21, 22],
```

```
        [30, 31, 32]])
C.shape: torch.Size([4, 3])
B2: tensor([[0, 1, 2]])
B2.shape: torch.Size([1, 3])
A2: tensor([[ 0,  0,  0],
        [10, 10, 10],
        [20, 20, 20],
        [30, 30, 30]])
A2.shape: torch.Size([4, 3])
B3: tensor([[0, 1, 2],
        [0, 1, 2],
        [0, 1, 2],
        [0, 1, 2]])
B3.shape: torch.Size([4, 3])
C1: tensor([[ 0,  1,  2],
        [10, 11, 12],
        [20, 21, 22],
        [30, 31, 32]])
C1.shape: torch.Size([4, 3])
```

10) 归并操作

对输入进行归并或合计等操作，这类操作的输入、输出的形状一般不相同，而且往往是输入大于输出。归并操作可以对整个张量进行归并，也可以沿着某个维度进行归并。

常见的归并操作可参考表 47.5。

函数	说明
`cumprod(t, axis)`	在指定维度对 `t` 进行累积积
`cumsum(t)`	在指定维度对 `t` 进行累加
`dist(a, b, p=2)`	返回 `a, b` 之间的 `p` 阶范数
`mean/median`	均值 / 中位数
`std/var`	标准差 / 方差
`norm(t, p=2)`	返回 `t` 的 `p` 阶范数
`prod(t)/sum(t)`	返回 `t` 所有元素的积 / 和

表 47.5 常见的归并操作

归并操作一般涉及 dim 参数，用于指定沿哪个维度进行归并。

另一个参数是`keepdim`, 用于说明输出结果中是否保留含 1 的维度，默认情况是`False`, 即不保留

```python
import torch

# Initial tensor for demonstration
t = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
```

```
a = torch.tensor([1.0, 2.0])
b = torch.tensor([4.0, 6.0])

print("Initial Tensor t:\n", t)
print("Tensor a:\n", a)
print("Tensor b:\n", b)

print("################################################")
# 1. cumprod(t, axis)
cumprod_result = torch.cumprod(t, dim=0)
print("\ncumprod(t, axis=0):\n", cumprod_result)

# 2. cumsum(t)
cumsum_result = torch.cumsum(t, dim=1)
print("\ncumsum(t, axis=1):\n", cumsum_result)

# 3. dist(a, b, p=2)
dist_result = torch.dist(a, b, p=2)
print("\ndist(a, b, p=2):\n", dist_result)

# 4. mean/median
mean_result = torch.mean(t)
median_result = torch.median(t)
print("\nmean(t):\n", mean_result)
print("median(t):\n", median_result)

# 5. std/var
std_result = torch.std(t)
var_result = torch.var(t)
print("\nstd(t):\n", std_result)
print("var(t):\n", var_result)

# 6. norm(t, p=2)
norm_result = torch.norm(t, p=2)
print("\nnorm(t, p=2):\n", norm_result)

# 7. prod(t)/sum(t)
prod_result = torch.prod(t)
sum_result = torch.sum(t)
print("\nprod(t):\n", prod_result)
print("sum(t):\n", sum_result)
```

```
Initial Tensor t:
 tensor([[1., 2.],
        [3., 4.]])
Tensor a:
 tensor([1., 2.])
Tensor b:
```

```
 tensor([4., 6.])
##############################################

cumprod(t, axis=0):
 tensor([[1., 2.],
        [3., 8.]])

cumsum(t, axis=1):
 tensor([[1., 3.],
        [3., 7.]])

dist(a, b, p=2):
 tensor(5.)

mean(t):
 tensor(2.5000)
median(t):
 tensor(2.)

std(t):
 tensor(1.2910)
var(t):
 tensor(1.6667)

norm(t, p=2):
 tensor(5.4772)

prod(t):
 tensor(24.)
sum(t):
 tensor(10.)
```

11) 比较操作

比较操作一般进行逐元素比较操作，有些是按指定方向比较。常用的比较函数可参考表 47.6。

函数	说明
`eq`	比较张量是否相等，支持广播机制
`equal`	比较张量是否有相同的形状与值
`ge/le/gt/lt`	大于/小于比较，大于或等于/小于或等于比较
`max/min(t, axis)`	返回最值，若指定 `axis`，则额外返回下标
`topk(t, k, axis)`	在指定的 `axis` 维上取最高的 `k` 个值

表 47.6 常用的比较函数

```
1 import torch
```

```
2
3  # Initial tensors for demonstration
4  t1 = torch.tensor([[1, 2], [3, 4]])
5  t2 = torch.tensor([[1, 0], [3, 5]])
6  t3 = torch.tensor([3, 4])
7
8  print("Tensor t1:\n", t1)
9  print("Tensor t2:\n", t2)
10 print("##############################################")
11
12 # 1. eq
13 eq_result = torch.eq(t1, t2)
14 print("\neq(t1, t2):\n", eq_result)
15
16 # 2. equal
17 equal_result = torch.equal(t1, t2)
18 print("\nequal(t1, t2):\n", equal_result)
19
20 # 3. ge/le/gt/lt
21 ge_result = torch.ge(t1, t2)
22 le_result = torch.le(t1, t2)
23 gt_result = torch.gt(t1, t2)
24 lt_result = torch.lt(t1, t2)
25 print("\nge(t1, t2):\n", ge_result)
26 print("le(t1, t2):\n", le_result)
27 print("gt(t1, t2):\n", gt_result)
28 print("lt(t1, t2):\n", lt_result)
29
30 # 4. max/min(t, axis)
31 max_value, max_indices = torch.max(t1, dim=1)
32 min_value, min_indices = torch.min(t1, dim=1)
33 print("\nmax(t1, axis=1):\nValues:\n", max_value, "\nIndices:\n",
       max_indices)
34 print("min(t1, axis=1):\nValues:\n", min_value, "\nIndices:\n",
       min_indices)
35
36 # 5. topk(t, k, axis)
37 topk_values, topk_indices = torch.topk(t1, k=2, dim=1)
38 print("\ntopk(t1, k=2, axis=1):\nValues:\n", topk_values,
       "\nIndices:\n", topk_indices)
```

```
Tensor t1:
 tensor([[1, 2],
        [3, 4]])
Tensor t2:
 tensor([[1, 0],
        [3, 5]])
##############################################
```

```
eq(t1, t2):
 tensor([[ True, False],
        [ True, False]])

equal(t1, t2):
 False

ge(t1, t2):
 tensor([[ True,  True],
        [ True, False]])
le(t1, t2):
 tensor([[ True, False],
        [ True,  True]])
gt(t1, t2):
 tensor([[False,  True],
        [False, False]])
lt(t1, t2):
 tensor([[False, False],
        [False,  True]])

max(t1, axis=1):
Values:
 tensor([2, 4])
Indices:
 tensor([1, 1])
min(t1, axis=1):
Values:
 tensor([1, 3])
Indices:
 tensor([0, 0])

topk(t1, k=2, axis=1):
Values:
 tensor([[2, 1],
        [4, 3]])
Indices:
 tensor([[1, 0],
        [1, 0]])
```

12) 矩阵操作

机器学习和深度学习中存在大量的矩阵运算，常用的有两种，一种是逐元素乘法，另外一种是点积乘法。

PyTorch 提供了大量的矩阵操作函数，如表 47.7 所示。

1) Torch 的`dot`函数与 NumPy 的`dot`函数有点不同, Torch 中的 dot 函数是对两个 1 维张量进行点积运算，NumPy 中的`dot`函数则无此限制。

2) mm 是对 2 维矩阵进行点积运算，`bmm`是对含批量的 3 维矩阵进行点积运算。

函数	说明
`dot(t1, t2)`	计算张量（1 维）的内积（或点积）
`mm(mat1, mat2)` / `bmm(batch1, batch2)`	计算矩阵乘法 / 含批量的 3 维矩阵乘法
`mv(t1, v1)`	计算矩阵与向量乘法
`t`	转置
`transpose(t, dim1, dim2)`	转置指定维度
`svd(t)`	计算 `t` 的 SVD 分解

表 47.7 常用的矩阵函数

3) 转置运算会导致存储空间不连续，需要调用`contiguous`方法转为连续

```python
import torch

# Initial tensors for demonstration
t1 = torch.tensor([1.0, 2.0, 3.0])
t2 = torch.tensor([4.0, 5.0, 6.0])

mat1 = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
mat2 = torch.tensor([[5.0, 6.0], [7.0, 8.0]])

batch1 = torch.rand(2, 3, 4)
batch2 = torch.rand(2, 4, 5)

v1 = torch.tensor([1.0, 2.0])

print("Tensor t1:\n", t1)
print("Tensor t2:\n", t2)
print("Matrix mat1:\n", mat1)
print("Matrix mat2:\n", mat2)
print("Batch tensor batch1:\n", batch1)
print("Batch tensor batch2:\n", batch2)

print("################################################")
# 1. dot(t1, t2)
dot_result = torch.dot(t1, t2)
print("\ndot(t1, t2):\n", dot_result)

# 2. mm(mat1, mat2)
mm_result = torch.mm(mat1, mat2)
print("\nmm(mat1, mat2):\n", mm_result)

# 3. bmm(batch1, batch2)
bmm_result = torch.bmm(batch1, batch2)
print("\nbmm(batch1, batch2):\n", bmm_result)

```

```python
# 4. mv(mat1, v1)
mv_result = torch.mv(mat1, v1)
print("\nmv(mat1, v1):\n", mv_result)

# 5. t
t_result = mat1.t()
print("\nt(mat1):\n", t_result)

# transpose(t, dim1, dim2)
transpose_result = mat1.transpose(0, 1)
print("\ntranspose(mat1, 0, 1):\n", transpose_result)

# 6. svd(t)
svd_u, svd_s, svd_v = torch.svd(mat1)
print("\nsvd(mat1):")
print("U matrix:\n", svd_u)
print("S vector:\n", svd_s)
print("V matrix:\n", svd_v)
```

```
Tensor t1:
 tensor([1., 2., 3.])
Tensor t2:
 tensor([4., 5., 6.])
Matrix mat1:
 tensor([[1., 2.],
        [3., 4.]])
Matrix mat2:
 tensor([[5., 6.],
        [7., 8.]])
Batch tensor batch1:
 tensor([[[0.3983, 0.4416, 0.1070, 0.5512],
         [0.7837, 0.0907, 0.4018, 0.9702],
         [0.6895, 0.6196, 0.0774, 0.6120]],

        [[0.3102, 0.1542, 0.8083, 0.5189],
         [0.2689, 0.4970, 0.0489, 0.4435],
         [0.4309, 0.8844, 0.2440, 0.8552]]])
Batch tensor batch2:
 tensor([[[0.2242, 0.5657, 0.0059, 0.5609, 0.6435],
         [0.7252, 0.1577, 0.6345, 0.1926, 0.8432],
         [0.5642, 0.7624, 0.9616, 0.4454, 0.3097],
         [0.6465, 0.9147, 0.0551, 0.9040, 0.1125]],

        [[0.5233, 0.9517, 0.8298, 0.8518, 0.2571],
         [0.5407, 0.6427, 0.6459, 0.2627, 0.3871],
         [0.4580, 0.5309, 0.2760, 0.6062, 0.5461],
         [0.5776, 0.2420, 0.7489, 0.5053, 0.7210]]])
##############################################

dot(t1, t2):
```

```
 tensor(32.)

mm(mat1, mat2):
 tensor([[19., 22.],
        [43., 50.]])

bmm(batch1, batch2):
 tensor([[[0.8262, 0.8807, 0.4158, 0.8544, 0.7238],
          [1.0954, 1.6514, 0.5020, 1.5131, 0.8144],
          [1.0432, 1.1065, 0.5054, 1.0937, 1.0589]],

         [[0.9156, 0.9490, 0.9687, 1.0569, 0.9549],
          [0.6880, 0.7086, 0.8898, 0.6133, 0.6080],
          [1.3094, 1.3150, 1.6367, 1.1794, 1.2030]]])

mv(mat1, v1):
 tensor([ 5., 11.])

t(mat1):
 tensor([[1., 3.],
        [2., 4.]])

transpose(mat1, 0, 1):
 tensor([[1., 3.],
        [2., 4.]])

svd(mat1):
U matrix:
 tensor([[-0.4046, -0.9145],
        [-0.9145,  0.4046]])
S vector:
 tensor([5.4650, 0.3660])
V matrix:
 tensor([[-0.5760,  0.8174],
        [-0.8174, -0.5760]])
```

13) 矩阵转秩 (维度转换)

矩阵转秩就是将多维矩阵的维度进行转换，总数不变但是维度发生了变化。

在 PyTorch 中，分别是 `permute()` 和 `transpose()` 可以用来实现矩阵的转秩 (交换不同维度的数据)。

在在调整卷积层的尺寸、修改 channel 的顺序、变换全连接层的大小的时候，我们就要用到它们。

(1) permute()

```
# permute() 用于交换维度
torch.permute(*dims) # *dims: 交换维度的顺序
```

```
a = torch.rand(2,3,5)  # (2,3,4)
a.shape # torch.Size([2, 3, 5])
a.permute(2, 1, 0)  # 将第0维(2)和第2维(5)交换
a.shape # torch.Size([5, 3, 2])
```

(2) transpose()

`transpose`，不同于 `permute`，它每次只能转换两个维度，或者说交换两个维度的数据。

```
# transpose() 用于交换维度
torch.transpose(dim0, dim1)
```

```
a = torch.rand(2,3,5)  # (2,3,4)
a.shape # torch.Size([2, 3, 5])
a.transpose(0, 2)  # 将第0维(2)和第2维(5)交换
a.shape # torch.Size([5, 3, 2])
```

注意：经过了 `transpose` 或者 `permute` 处理之后的数据，变得不再连续了。

14) 增减维度

有时候我们需要对 Tensor 增加或者删除某些维度，比如删除或者增加图片的几个通道。

PyTorch 提供了 `squeeze()` 和 `unsqueeze()` 函数解决这个问题。

(1) squeeze() 删除维度为 1 的维度

```
# squeeze() 用于删除维度为1的维度
torch.squeeze(input, dim=None, out=None)
# input: 输入张量
# dim: 删除的维度
#       如果指定维度，那么只有该维度大小为1才能被删除
# dim=None: 默认删除所有维度为1的维度
# out: 输出张量
```

`squeeze` 函数不在原地操作，需要重新赋值给原来的变量。

```python
import torch

a = torch.rand(2,1,5)  # (2,3,4)
print("a shape:", a.shape)
b =  a.squeeze()  # 删除第1维(3)
print("a shape:", a.shape)
print("b shape:", b.shape)
```

```
a shape: torch.Size([2, 1, 5])
a shape: torch.Size([2, 1, 5]) # 不是原地操作
b shape: torch.Size([2, 5])
```

例子 47.2

如何理解维度，为什么`squeeze`函数可以删除维度为 1 的维度？

方法: 这个维度直观上就是`[]`的层数

```python
import torch
# 创建一个张量，具有多个维度为1的维度
print("torch.rand(1)", torch.rand(1))
print("torch.rand(3)", torch.rand(3))
print("torch.rand(3, 1)", torch.rand(3, 1))
print("torch.rand(1, 3)", torch.rand(1, 3))
print("torch.rand(1, 3, 1)", torch.rand(1, 3, 1))
print("torch.rand(3, 1, 1)", torch.rand(3, 1, 1))

```

```
torch.rand(1) tensor([0.5110])      # 随机数
torch.rand(3) tensor([0.6516, 0.6921, 0.0996])
torch.rand(3, 1) tensor([[0.5129],  # 2维,多了1层[],不过是在内部
                         [0.9296],
                         [0.8092]])
torch.rand(1, 3) tensor([[0.0461, 0.5447, 0.9819]]) #
2维,多了1层[],不过是在外部
torch.rand(1, 3, 1) tensor([[[0.9945], # 3维,多了1层[]
        [0.7740],
        [0.4791]]])
torch.rand(3, 1, 1) tensor([[[0.1434]], # 3维,多了2层[]
```

```
           [[0.3084]],

           [[0.7733]]])
```

根据上面的例子发现如果维度为 1 的话，仅仅是多了一层`[]`，所以`squeeze`函数就是删除这一层`[]`。

```python
import torch

# 创建一个张量，具有多个维度为1的维度
x = torch.rand(1, 3, 1, 5, 1)
print("原始张量 x 的形状: ", x.shape)

# 情况 1: 不指定 dim 参数，删除所有大小为1的维度
x_squeezed_all = torch.squeeze(x)
print("不指定 dim 的 squeeze 后张量形状: ", x_squeezed_all.shape)

# 情况 2: 指定 dim 参数为 0（删除第 0 维度，如果其大小为1）
x_squeezed_dim0 = torch.squeeze(x, dim=0)
print("指定 dim=0 的 squeeze 后张量形状: ", x_squeezed_dim0.shape)

# 情况 3: 指定 dim 参数为 2（删除第 2 维度，如果其大小为1）
x_squeezed_dim2 = torch.squeeze(x, dim=2)
print("指定 dim=2 的 squeeze 后张量形状: ", x_squeezed_dim2.shape)

# 情况 4: 指定 dim 参数为 4（删除第 4 维度，如果其大小为1）
x_squeezed_dim4 = torch.squeeze(x, dim=4)
print("指定 dim=4 的 squeeze 后张量形状: ", x_squeezed_dim4.shape)

# 情况 5: 指定 dim 参数为 1（不删除第 1 维度，因为其大小不为1）
x_squeezed_dim1 = torch.squeeze(x, dim=1)
print("指定 dim=1 的 squeeze 后张量形状: ", x_squeezed_dim1.shape)
```

```
原始张量 x 的形状:  torch.Size([1, 3, 1, 5, 1])
不指定 dim 的 squeeze 后张量形状:  torch.Size([3, 5])
指定 dim=0 的 squeeze 后张量形状:  torch.Size([3, 1, 5, 1])
指定 dim=2 的 squeeze 后张量形状:  torch.Size([1, 3, 5, 1])
指定 dim=4 的 squeeze 后张量形状:  torch.Size([1, 3, 1, 5])
指定 dim=1 的 squeeze 后张量形状:  torch.Size([1, 3, 1, 5, 1]) # 没有删除
```

当然，有时为了找 Bug，可能会不注意到某个维度的大小为 1，这时候就需要手动指定要删除的维度。

```python
import torch

# 创建一个张量，具有多个维度为1的维度
x = torch.rand(1, 3, 1, 5, 1)
print("原始张量 x 的形状: ", x.shape)

# 情况 1: 不指定 dim 参数，删除所有大小为1的维度
x_squeezed_all = torch.squeeze(x)
print("不指定 dim 的 squeeze 后张量形状: ", x_squeezed_all.shape)

# 情况 2: 指定 dim 参数为 0（删除第 0 维度，如果其大小为1）
if x.size(0) == 1:
    x_squeezed_dim0 = torch.squeeze(x, dim=0)
    print("指定 dim=0 的 squeeze 后张量形状: ",
    x_squeezed_dim0.shape)
else:
    print("指定 dim=0 的 squeeze 失败，因为维度大小不为1。")

# 情况 3: 指定 dim 参数为 2（删除第 2 维度，如果其大小为1）
if x.size(2) == 1:
    x_squeezed_dim2 = torch.squeeze(x, dim=2)
    print("指定 dim=2 的 squeeze 后张量形状: ",
    x_squeezed_dim2.shape)
else:
    print("指定 dim=2 的 squeeze 失败，因为维度大小不为1。")

# 情况 4: 指定 dim 参数为 4（删除第 4 维度，如果其大小为1）
if x.size(4) == 1:
    x_squeezed_dim4 = torch.squeeze(x, dim=4)
    print("指定 dim=4 的 squeeze 后张量形状: ",
    x_squeezed_dim4.shape)
else:
    print("指定 dim=4 的 squeeze 失败，因为维度大小不为1。")

# 情况 5: 指定 dim 参数为 1（尝试删除第 1 维度，但大小不为1）
if x.size(1) == 1:
    x_squeezed_dim1 = torch.squeeze(x, dim=1)
    print("指定 dim=1 的 squeeze 后张量形状: ",
    x_squeezed_dim1.shape)
else:
    print("指定 dim=1 的 squeeze 失败，因为维度大小不为1。")
```

```
原始张量 x 的形状:  torch.Size([1, 3, 1, 5, 1])
不指定 dim 的 squeeze 后张量形状:  torch.Size([3, 5])
指定 dim=0 的 squeeze 后张量形状:  torch.Size([3, 1, 5, 1])
```

```
指定 dim=2 的 squeeze 后张量形状： torch.Size([1, 3, 5, 1])
指定 dim=4 的 squeeze 后张量形状： torch.Size([1, 3, 1, 5])
指定 dim=1 的 squeeze 失败，因为维度大小不为1。
```

(2) 手动删除维度

如果某个维度的大小不为1,而仍想删除该维度,可以使用 `torch.index_select`、`torch.cat` 方法来手动删除该维度。

这实际上意味着你需要重新组织张量的内容，将不需要的维度“去掉”。

```python
import torch

# 创建一个张量，具有多个维度
x = torch.rand(1, 3, 1, 5, 1)
print("原始张量 x 的形状：", x.shape)

# 要删除的维度索引
dim_to_remove = 1

# 手动删除指定维度
# 方法：通过选择所有其他维度的数据，重新创建一个新的张量
x_manual_remove_dim = torch.cat([x.select(dim_to_remove, i) for i
    in range(x.size(dim_to_remove))], dim=dim_to_remove)

# 打印手动删除指定维度后的张量形状
print(f"手动删除 dim={dim_to_remove} 后的张量形状：",
    x_manual_remove_dim.shape)
```

```
原始张量 x 的形状： torch.Size([1, 3, 1, 5, 1])
手动删除 dim=1 后的张量形状： torch.Size([1, 3, 5, 1])
```

(3) unsqueeze() 增加维度

```python
# unsqueeze() 用于增加维度
torch.unsqueeze(input, dim, out=None)
# input: 输入张量
# dim: 增加的维度
# out: 输出张量
```

注意：`unsqueeze` 函数不在原地操作，需要重新赋值给原来的变量。

```
a = torch.rand(2,3,5)  # (2,3,4)
a.shape # torch.Size([2, 3, 5])
a.unsqueeze(1)  # 增加第1维(3)
a.shape # torch.Size([2, 1, 3, 5])
```

```
import torch

# 创建一个初始张量
x = torch.tensor([1.0, 2.0, 3.0])
print("原始张量 x: ", x)
print("原始张量 x 的形状: ", x.shape)

# 情况 1: 在第0个维度增加一个维度
x_unsqueeze_0 = torch.unsqueeze(x, dim=0)
print("\n在第0个维度增加一个维度后的张量: ", x_unsqueeze_0)
print("在第0个维度增加一个维度后的张量形状: ", x_unsqueeze_0.shape)

# 情况 2: 在第1个维度增加一个维度
x_unsqueeze_1 = torch.unsqueeze(x, dim=1)
print("\n在第1个维度增加一个维度后的张量: ", x_unsqueeze_1)
print("在第1个维度增加一个维度后的张量形状: ", x_unsqueeze_1.shape)

# 创建一个二维张量
y = torch.tensor([[1, 2, 3], [4, 5, 6]])
print("\n原始二维张量 y: ", y)
print("原始二维张量 y 的形状: ", y.shape)

# 情况 3: 在第0个维度增加一个维度
y_unsqueeze_0 = torch.unsqueeze(y, dim=0)
print("\n在第0个维度增加一个维度后的张量: ", y_unsqueeze_0)
print("在第0个维度增加一个维度后的张量形状: ", y_unsqueeze_0.shape)

# 情况 4: 在第1个维度增加一个维度
y_unsqueeze_1 = torch.unsqueeze(y, dim=1)
print("\n在第1个维度增加一个维度后的张量: ", y_unsqueeze_1)
print("在第1个维度增加一个维度后的张量形状: ", y_unsqueeze_1.shape)

# 情况 5: 在第2个维度增加一个维度
y_unsqueeze_2 = torch.unsqueeze(y, dim=2)
print("\n在第2个维度增加一个维度后的张量: ", y_unsqueeze_2)
print("在第2个维度增加一个维度后的张量形状: ", y_unsqueeze_2.shape)
```

```
原始张量 x:  tensor([1., 2., 3.])
原始张量 x 的形状:  torch.Size([3])

在第0个维度增加一个维度后的张量:  tensor([[1., 2., 3.]]) # 多了1层 []
在第0个维度增加一个维度后的张量形状:  torch.Size([1, 3])

在第1个维度增加一个维度后的张量:  tensor([[1.],
        [2.],
        [3.]])
在第1个维度增加一个维度后的张量形状:  torch.Size([3, 1])

原始二维张量 y:  tensor([[1, 2, 3],
        [4, 5, 6]])
原始二维张量 y 的形状:  torch.Size([2, 3])

在第0个维度增加一个维度后的张量:  tensor([[[1, 2, 3],
         [4, 5, 6]]])
在第0个维度增加一个维度后的张量形状:  torch.Size([1, 2, 3])

在第1个维度增加一个维度后的张量:  tensor([[[1, 2, 3]],

        [[4, 5, 6]]])
在第1个维度增加一个维度后的张量形状:  torch.Size([2, 1, 3])

在第2个维度增加一个维度后的张量:  tensor([[[1],
         [2],
         [3]],

        [[4],
         [5],
         [6]]])
在第2个维度增加一个维度后的张量形状:  torch.Size([2, 3, 1])
```

15) Tensor 的连接操作

在项目开发中，深度学习某一层神经元的数据可能有多个不同的来源，那么就需要将数据进行组合，这个组合的操作，我们称之为连接。

PyTorch 提供了 `cat()` 和 `stack()` 函数来实现连接操作。

(1) cat() 不变维度连接

cat 是 concatnate 的意思，也就是拼接、联系的意思。

```
# cat() 用于连接张量
torch.cat(tensors, dim=0, out=None)
# tensors: 张量序列
# dim: 沿着哪个维度拼接
```

```
# out: 输出张量
```

```
import torch

# 两个二维张量
A=torch.ones(3,3,3)
B=2*torch.ones(3,3,3)

print("A.size():",A.size())
print("B.size():",B.size())

print("torch.cat((A,B),0).size():",torch.cat((A,B),0).size())
print("torch.cat((A,B),1).size():",torch.cat((A,B),1).size())
print("torch.cat((A,B),2).size():",torch.cat((A,B),2).size())
```

```
A.size(): torch.Size([3, 3, 3])
B.size(): torch.Size([3, 3, 3])
torch.cat((A,B),0).size(): torch.Size([6, 3, 3])
torch.cat((A,B),1).size(): torch.Size([3, 6, 3])
torch.cat((A,B),2).size(): torch.Size([3, 3, 6])
```

推导 cat 的拼接方式

1) 维度 0
对于 A 的维度 0 就是第 0 个括号开始的

```
[[[1., 1., 1.],
  [1., 1., 1.],
  [1., 1., 1.]],

 [[1., 1., 1.],
  [1., 1., 1.],
  [1., 1., 1.]],

 [[1., 1., 1.],
  [1., 1., 1.],
  [1., 1., 1.]]]
```

同理 B 的维度 0 就是第 0 个括号开始的

```
[[[2., 2., 2.],
  [2., 2., 2.],
  [2., 2., 2.]],
```

```
[[2., 2., 2.],
 [2., 2., 2.],
 [2., 2., 2.]],

[[2., 2., 2.],
 [2., 2., 2.],
 [2., 2., 2.]]]
```

在维度 0 上拼接就是将 A 和 B 的第 0 个括号拼接在一起

```
[[[1., 1., 1.],
  [1., 1., 1.],
  [1., 1., 1.]],

 [[1., 1., 1.],
  [1., 1., 1.],
  [1., 1., 1.]],

 [[1., 1., 1.],
  [1., 1., 1.],
  [1., 1., 1.]],

 [[2., 2., 2.],
  [2., 2., 2.],
  [2., 2., 2.]],

 [[2., 2., 2.],
  [2., 2., 2.],
  [2., 2., 2.]],

 [[2., 2., 2.],
  [2., 2., 2.],
  [2., 2., 2.]]]
```

2) 维度 1
对于 A 的维度 0 就是第 1 个括号开始的

```
[   # 这层括号不看了
    # 第1部分
    [[1., 1., 1.],
     [1., 1., 1.],
     [1., 1., 1.]]

    # 第2部分
    [[1., 1., 1.],
     [1., 1., 1.],
     [1., 1., 1.]],

    # 第3部分
    [[1., 1., 1.],
     [1., 1., 1.],
```

```
            [1., 1., 1.]]
        ]    # 这层括号不看了
```

同理 B 的维度 0 就是第 1 个括号开始的

```
        [   # 这层括号不看了
            # 第1部分
            [[2., 2., 2.],
             [2., 2., 2.],
             [2., 2., 2.]]

            # 第2部分
            [[2., 2., 2.],
             [2.,2., 2.],
             [2., 2., 2.]],

            # 第3部分
            [[2., 2., 2.],
             [2., 2., 2.],
             [2., 2., 2.]]
        ]   # 这层括号不看了
```

在维度 1 上拼接就是将 A 和 B 的第 1 个括号拼接在一起

```
        [   # 这层括号不看了
            [[1., 1., 1.],
             [1., 1., 1.],
             [1., 1., 1.],
             [2., 2., 2.],
             [2., 2., 2.],
             [2., 2., 2.]],

            [[1., 1., 1.],
             [1., 1., 1.],
             [1., 1., 1.],
             [2., 2., 2.],
             [2., 2., 2.],
             [2., 2., 2.]],

            [[1., 1., 1.],
             [1., 1., 1.],
             [1., 1., 1.],
             [2., 2., 2.],
             [2., 2., 2.],
             [2., 2., 2.]]
        ]    # 这层括号不看了
```

3) 维度 2

对于 A 的维度 0 就是第 2 个括号开始的

```
        [[  # 这层括号不看了
```

```
        # 第1部分
        [1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.]

        # 第2部分
        [1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.],

        # 第3部分
        [1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.]
    ]]  # 这层括号不看了
```

同理 B 的维度 0 就是第 2 个括号开始的

```
    [[  # 这层括号不看了
        # 第1部分
        [2., 2., 2.],
        [2., 2., 2.],
        [2., 2., 2.]

        # 第2部分
        [2.,2., 2.],
        [2., 2., 2.],
        [2., 2., 2.],

        # 第3部分
        [2., 2., 2.],
        [2., 2., 2.],
        [2., 2., 2.],
    ]]
```

在维度 2 上拼接就是将 A 和 B 的第 2 个括号拼接在一起

```
[[  # 这层括号不看了
    # 第1部分
    [1., 1., 1., 2., 2., 2.],
    [1., 1., 1., 2., 2., 2.],
    [1., 1., 1., 2., 2., 2.],

    # 第2部分
    [1., 1., 1., 2., 2., 2.],
    [1., 1., 1., 2., 2., 2.],
    [1., 1., 1., 2., 2., 2.],

    # 第3部分
    [1., 1., 1., 2., 2., 2.],
    [1., 1., 1., 2., 2., 2.],
    [1., 1., 1., 2., 2., 2.]
```

```
    ]]
```

```
>>> A=torch.ones(3,3,3)
>>> B=2*torch.ones(3,3,3)
>>> A
tensor([[[1., 1., 1.],
         [1., 1., 1.],
         [1., 1., 1.]],

        [[1., 1., 1.],
         [1., 1., 1.],
         [1., 1., 1.]],

        [[1., 1., 1.],
         [1., 1., 1.],
         [1., 1., 1.]]])
>>> B
tensor([[[2., 2., 2.],
         [2., 2., 2.],
         [2., 2., 2.]],

        [[2., 2., 2.],
         [2., 2., 2.],
         [2., 2., 2.]],

        [[2., 2., 2.],
         [2., 2., 2.],
         [2., 2., 2.]]])

# 沿着第0维度拼接
>>> torch.cat((A,B),0)
tensor([[[1., 1., 1.],
         [1., 1., 1.],
         [1., 1., 1.]],

        [[1., 1., 1.],
         [1., 1., 1.],
         [1., 1., 1.]],

        [[1., 1., 1.],
         [1., 1., 1.],
         [1., 1., 1.]],

        [[2., 2., 2.],
         [2., 2., 2.],
         [2., 2., 2.]],

        [[2., 2., 2.],
         [2., 2., 2.],
         [2., 2., 2.]],
```

```
    [[2., 2., 2.],
        [2., 2., 2.],
        [2., 2., 2.]]])

# 沿着第1维度拼接
>>> torch.cat((A,B),1)
tensor([[[1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.],
        [2., 2., 2.],
        [2., 2., 2.],
        [2., 2., 2.]],

    [[1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.],
        [2., 2., 2.],
        [2., 2., 2.],
        [2., 2., 2.]],

    [[1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.],
        [2., 2., 2.],
        [2., 2., 2.],
        [2., 2., 2.]]])

# 沿着第2维度拼接
>>> torch.cat((A,B),2)
tensor([[[1., 1., 1., 2., 2., 2.],
        [1., 1., 1., 2., 2., 2.],
        [1., 1., 1., 2., 2., 2.]],

    [[1., 1., 1., 2., 2., 2.],
        [1., 1., 1., 2., 2., 2.],
        [1., 1., 1., 2., 2., 2.]],

    [[1., 1., 1., 2., 2., 2.],
        [1., 1., 1., 2., 2., 2.],
        [1., 1., 1., 2., 2., 2.]]])
```

`cat` 实际上是将多个 Tensor 在已有的维度上进行连接，

(2) stack() 增加维度连接

那如果想增加新的维度进行连接，这时候就需要 `stack` 函数登场了。

```
# stack() 用于连接张量
torch.stack(tensors, dim=0, out=None)
# tensors: 张量序列
# dim: 沿着哪个维度拼接
# out: 输出张量
```

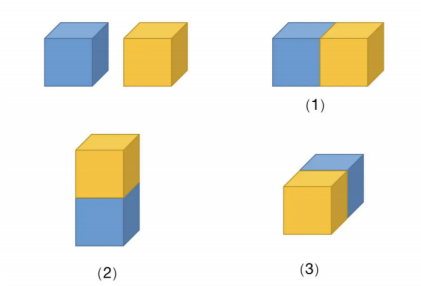

图 47.1 cat 的拼接方式

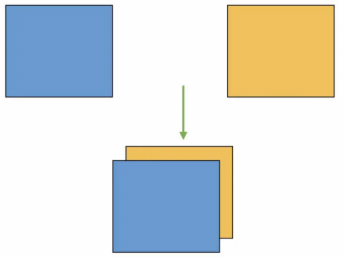

图 47.2 stack 的拼接方式

图 47.3 cat 和 stack 的拼接方式

```python
import torch

# 两个二维张量
A=torch.ones(3,3)
B=2*torch.ones(3,3)

print("A.size():",A.size())
print("B.size():",B.size())

print("torch.stack((A,B),0).size():",torch.stack((A,B),0).size())
print("torch.stack((A,B),1).size():",torch.stack((A,B),1).size())
print("torch.stack((A,B),2).size():",torch.stack((A,B),2).size())
```

```
A.size(): torch.Size([3, 3])
B.size(): torch.Size([3, 3])
torch.stack((A,B),0).size(): torch.Size([2, 3, 3])
torch.stack((A,B),1).size(): torch.Size([3, 2, 3])
torch.stack((A,B),2).size(): torch.Size([3, 3, 2])
```

推导 stack 的拼接方式

1) 维度 0

对于 A 的维度 0 就是第 0 个括号开始的

```
[[1., 1., 1.],
 [1., 1., 1.],
 [1., 1., 1.]]
```

同理 B 的维度 0 就是第 0 个括号开始的

```
          [[2., 2., 2.],
           [2., 2., 2.],
           [2., 2., 2.]]
```

在维度 0 上拼接就是将 A 和 B 在维度 0 上合并 (新增第 0 个 [] 括号)

```
      [ # 新增的维度0
          [[1., 1., 1.],
           [1., 1., 1.],
           [1., 1., 1.]],

          [[2., 2., 2.],
           [2., 2., 2.],
           [2., 2., 2.]]
      ]
```

2) 维度 1
对于 A 的维度 0 就是第 1 个括号开始的

```
      [   # 这层括号不看了
          [1., 1., 1.],
          [1., 1., 1.],
          [1., 1., 1.]
      ]   # 这层括号不看了
```

同理 B 的维度 0 就是第 1 个括号开始的

```
      [   # 这层括号不看了
          [2., 2., 2.],
          [2., 2., 2.],
          [2., 2., 2.]
      ]   # 这层括号不看了
```

在维度 1 上拼接就是将 A 和 B 在维度 0 上合并 (新增第 1 个 [] 括号)

```
      [
          [ # 新增第1个[]括号
           [1., 1., 1.],
           [2., 2., 2.]
          ],

          [
           [1., 1., 1.],
           [2., 2., 2.]
          ],

         [
           [1., 1., 1.],
```

```
            [2., 2., 2.]
        ]
    ]
```

3) 维度 2
对于 A 的维度 0 就是第 2 个括号开始的

```
    [   # 这层括号不看了
        [# 这层括号不看了
            1.,
            1.,
            1.
        ],

        [# 这层括号不看了
            1.,
            1.,
            1.
        ],

        [# 这层括号不看了
            1.,
            1.,
            1.
        ]
    ]   # 这层括号不看了
```

同理 B 的维度 0 就是第 2 个括号开始的

```
[   # 这层括号不看了
    [# 这层括号不看了
        2.,
        2.,
        2.
    ],

    [# 这层括号不看了
        2.,
        2.,
        2.
    ],

    [# 这层括号不看了
        2.,
        2.,
        2.
    ]
]   # 这层括号不看了
```

在维度 2 上拼接就是将 A 和 B 在维度 0 上合并 (新增第 2 个 [] 括号)

```
    [
        [
            [1., 2.], # 新增第2个[]括号
            [1., 2.],
            [1., 2.]
        ],

        [
            [1., 2.],
            [1., 2.],
            [1., 2.]
        ],

        [
            [1., 2.],
            [1., 2.],
            [1., 2.]
        ]
    ]
```

详细展开发现

```
>>> A=torch.ones(3,3)
>>> B=2*torch.ones(3,3)
>>> A
tensor([[1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.]])
>>> B
tensor([[2., 2., 2.],
        [2., 2., 2.],
        [2., 2., 2.]])
>>> torch.stack((A,B),0)
tensor([[[1., 1., 1.],
         [1., 1., 1.],
         [1., 1., 1.]],

        [[2., 2., 2.],
         [2., 2., 2.],
         [2., 2., 2.]]])
>>> torch.stack((A,B),1)
tensor([[[1., 1., 1.],
         [2., 2., 2.]],

        [[1., 1., 1.],
         [2., 2., 2.]],

        [[1., 1., 1.],
         [2., 2., 2.]]])
>>> torch.stack((A,B),2)
tensor([[[1., 2.],
```

```
         [1., 2.],
         [1., 2.]],

        [[1., 2.],
         [1., 2.],
         [1., 2.]],

        [[1., 2.],
         [1., 2.],
         [1., 2.]]])
```

```
>>> A=torch.arange(0,4)
tensor([0, 1, 2, 3])
>>> B=torch.arange(5,9)
>>> B
tensor([5, 6, 7, 8])

>>> C=torch.stack((A,B),0)
>>> C
tensor([[0, 1, 2, 3],
        [5, 6, 7, 8]])
>>> D=torch.stack((A,B),1)
>>> D
tensor([[0, 5],
        [1, 6],
        [2, 7],
        [3, 8]])
```

16) Tensor 的切片

切分就是连接的逆过程，有了刚才的经验

PyTorch 中的 Tensor 支持切片操作，但是切片操作会返回一个新的 Tensor，所以切片操作不会改变原来的 Tensor。

PyTorch 提供了`chunk()`、`split()`和`unbind()`函数来实现切片操作。

(1) chunk() 均分

```
# chunk() 按照声明的 dim, 进行尽可能平均的划分。
# 如果不能整除，则最后一个张量的维度会小于其他张量

torch.chunk(tensor, chunks, dim=0)
# tensor: 输入张量
# chunks: 切分后的块数
# dim: 切分维度
```

```
1 import torch
```

```python

# 示例1: 一维张量的均分
A = torch.tensor([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

# 将A沿着dim=0方向均分为2个块
B = torch.chunk(A, 2, 0)
print("示例1: 一维张量的均分")
for i, chunk in enumerate(B):
    print(f"块{i+1}:", chunk)

# 示例2: 一维张量不能均分的情况
# 将A沿着dim=0方向分成3个块, 不能整除的情况下最后一个块包含剩余元素
C = torch.chunk(A, 3, 0)
print("\n示例2: 一维张量不能均分的情况")
for i, chunk in enumerate(C):
    print(f"块{i+1}:", chunk)

# 示例3: 二维张量的分块
D = torch.tensor([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]])

# 将D沿着dim=0 (行方向) 分成2个块
E = torch.chunk(D, 2, 0)
print("\n示例3: 二维张量沿行方向的均分")
for i, chunk in enumerate(E):
    print(f"块{i+1}:\n", chunk)

# 将D沿着dim=1 (列方向) 分成2个块
# chunk 函数是先做除法, 然后再向上取整得到每组的数量。
F = torch.chunk(D, 2, 1)
print("\n示例4: 二维张量沿列方向的均分")
for i, chunk in enumerate(F):
    print(f"块{i+1}:\n", chunk)

# 示例5: 如果 chunk 参数大于 Tensor 可以切分的长度
# 将D沿着dim=1 (列方向) 分成4个块, 不均分时, 最后一个块包含剩余的元素
G = torch.chunk(D, 4, 1)
print("\n示例5: 二维张量沿列方向不能均分的情况")
for i, chunk in enumerate(G):
    print(f"块{i+1}:\n", chunk)

# 示例6: 三维张量的分块
H = torch.tensor([[[1, 2], [3, 4]], [[5, 6], [7, 8]], [[9, 10],
    [11, 12]]])

# 将H沿着dim=0 (最外层维度) 分成3个块
I = torch.chunk(H, 3, 0)
print("\n示例6: 三维张量沿最外层维度的均分")
for i, chunk in enumerate(I):
    print(f"块{i+1}:\n", chunk)
```

```
示例1: 一维张量的均分
块1: tensor([1, 2, 3, 4, 5])
块2: tensor([ 6,  7,  8,  9, 10])

示例2: 一维张量不能均分的情况
块1: tensor([1, 2, 3, 4])
块2: tensor([5, 6, 7, 8])
块3: tensor([ 9, 10])

示例3: 二维张量沿行方向的均分
块1:
tensor([[1, 2, 3],
        [4, 5, 6]])
块2:
tensor([[ 7,  8,  9],
        [10, 11, 12]])

示例4: 二维张量沿列方向的均分
块1:
tensor([[ 1,  2],
        [ 4,  5],
        [ 7,  8],
        [10, 11]])
块2:
tensor([[ 3],
        [ 6],
        [ 9],
        [12]])

示例5: 如果 chunk 参数大于 Tensor 可以切分的长度
块1:
tensor([[ 1],
        [ 4],
        [ 7],
        [10]])
块2:
tensor([[ 2],
        [ 5],
        [ 8],
        [11]])
块3:
tensor([[ 3],
        [ 6],
        [ 9],
        [12]])

示例6: 三维张量沿最外层维度的均分
块1:
tensor([[[1, 2],
        [3, 4]]])
```

```
块2:
tensor([[[5, 6],
         [7, 8]]])
块3:
tensor([[[ 9, 10],
         [11, 12]]])
```

(2) split() 按照指定大小切分

```
# split() 按照声明的 dim, 进行划分。
torch.split(tensor, split_size_or_sections, dim=0)
# split_size_or_sections 这个参数。
#   当它为整数时，表示将 tensor 按照每块大小为这个整数的数值来切割；
#   当这个参数为列表时，则表示将此 tensor 切成和列表中元素一样大小的块。
#
#   如果使用list,要保证 list的和等于 改维度的大小
```

```python
import torch

# 示例1: 一维张量，按照固定大小分割l
A = torch.tensor([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

# 将A按照每块大小为3进行分割，dim=0 (默认)
B = torch.split(A, 3, 0)
print("示例1: 一维张量，按照固定大小分割")
for i, chunk in enumerate(B):
    print(f"块{i+1}:", chunk)

# 示例2: 一维张量，不能均分的情况
# 将A按照每块大小为4进行分割，最后一个块包含剩余元素
C = torch.split(A, 4, 0)
print("\n示例2: 一维张量，不能均分的情况")
for i, chunk in enumerate(C):
    print(f"块{i+1}:", chunk)

# 示例3: 一维张量，按照列表分割
# 将A按照列表[2, 3, 5]指定的块大小进行分割
#
# 这里第0维度一共10个元素，因此 list 中的元素之和应该等于 10=2+3+5
D = torch.split(A, [2, 3, 5], 0)
print("\n示例3: 一维张量，按照列表分割")
for i, chunk in enumerate(D):
    print(f"块{i+1}:", chunk)

# 示例4: 二维张量，按行分割
E = torch.tensor([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]])
```

```python

# 将E沿着dim=0（行方向）按照每块大小为2进行分割
F = torch.split(E, 2, 0)
print("\n示例4：二维张量，按行分割")
for i, chunk in enumerate(F):
    print(f"块{i+1}:\n", chunk)

# 示例5：二维张量，按列分割
# 将E沿着dim=1（列方向）按照每块大小为2进行分割
G = torch.split(E, 2, 1)
print("\n示例5：二维张量，按列分割")
for i, chunk in enumerate(G):
    print(f"块{i+1}:\n", chunk)

# 示例6：二维张量，不能均分的情况
# 将E沿着dim=1（列方向）分成[1, 2]两块
H = torch.split(E, [1, 2], 1)
print("\n示例6：二维张量，不能均分的情况")
for i, chunk in enumerate(H):
    print(f"块{i+1}:\n", chunk)

# 示例7：三维张量，按最外层维度分割
I = torch.tensor([[[1, 2], [3, 4]], [[5, 6], [7, 8]], [[9, 10],
    [11, 12]]])

# 将I沿着dim=0分割成每块大小为1的块
J = torch.split(I, 1, 0)
print("\n示例7：三维张量，按最外层维度分割")
for i, chunk in enumerate(J):
    print(f"块{i+1}:\n", chunk)

# 示例8：三维张量，按第二维度分割
# 将I沿着dim=1（第二维度）分割成每块大小为1的块
K = torch.split(I, 1, 1)
print("\n示例8：三维张量，按第二维度分割")
for i, chunk in enumerate(K):
    print(f"块{i+1}:\n", chunk)
```

```
示例1：一维张量，按照固定大小分割
块1: tensor([1, 2, 3])
块2: tensor([4, 5, 6])
块3: tensor([7, 8, 9])
块4: tensor([10])

示例2：一维张量，不能均分的情况
块1: tensor([1, 2, 3, 4])
块2: tensor([5, 6, 7, 8])
块3: tensor([ 9, 10])
```

```
示例3: 一维张量, 按照列表分割
块1: tensor([1, 2])
块2: tensor([3, 4, 5])
块3: tensor([ 6,  7,  8,  9, 10])

示例4: 二维张量, 按行分割
块1:
tensor([[1, 2, 3],
        [4, 5, 6]])
块2:
tensor([[ 7,  8,  9],
        [10, 11, 12]])

示例5: 二维张量, 按列分割
块1:
tensor([[ 1,  2],
        [ 4,  5],
        [ 7,  8],
        [10, 11]])
块2:
tensor([[ 3],
        [ 6],
        [ 9],
        [12]])

示例6: 二维张量, 不能均分的情况
块1:
tensor([[ 1],
        [ 4],
        [ 7],
        [10]])
块2:
tensor([[ 2,  3],
        [ 5,  6],
        [ 8,  9],
        [11, 12]])

示例7: 三维张量, 按最外层维度分割
块1:
tensor([[[1, 2],
        [3, 4]]])
块2:
tensor([[[5, 6],
        [7, 8]]])
块3:
tensor([[[ 9, 10],
        [11, 12]]])

示例8: 三维张量, 按第二维度分割
块1:
tensor([[[ 1,  2]],

        [[ 5,  6]],
```

```
        [[ 9, 10]]])
块2:
tensor([[[ 3,  4]],

        [[ 7,  8]],

        [[11, 12]]])
```

(3) unbind() 按照指定维度切分

如果我们现在有一个 3 channel 图像的 Tensor，想要逐个获取每个 channel 的数据, 假如用 `chunk` 的话，我们需要将 `chunks` 设为 3；如果用 `split` 的话，需要将 `split_size_or_sections` 设为 1.

虽然它们都可以实现相同的目的，但是如果 channel 数量很大，逐个去取也比较折腾。这时候，就需要用到另一个函数：`unbind`，它的函数定义如下

```
# unbind() 按照声明的 dim, 进行切分。
torch.unbind(tensor, dim=0)
```

```
import torch

# 示例1: 三通道图像的三维张量, 按第一个维度分割
# 假设我们有一个三通道的图像张量, 大小为 (3, 4, 4)
image_tensor = torch.tensor([
    [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [13, 14, 15, 16]],    # 第一个通道
    [[17, 18, 19, 20], [21, 22, 23, 24], [25, 26, 27, 28], [29, 30, 31, 32]], # 第二个通道
    [[33, 34, 35, 36], [37, 38, 39, 40], [41, 42, 43, 44], [45, 46, 47, 48]]  # 第三个通道
])

# 使用unbind方法按dim=0分割 (将每个通道分开)
channels = torch.unbind(image_tensor, dim=0)
print("示例1: 三通道图像张量按dim=0分割")
for i, channel in enumerate(channels):
    print(f"通道{i+1}:\n", channel)

# 示例2: 二维张量, 按行分割
tensor_2d = torch.tensor([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

# 使用unbind方法按dim=0分割 (按行分割)
rows = torch.unbind(tensor_2d, dim=0)
print("\n示例2: 二维张量按行分割")
for i, row in enumerate(rows):
```

```python
    print(f"行{i+1}:", row)

# 使用unbind方法按dim=1分割（按列分割）
cols = torch.unbind(tensor_2d, dim=1)
print("\n示例3：二维张量按列分割")
for i, col in enumerate(cols):
    print(f"列{i+1}:", col)

# 示例4：三维张量，按第二维度分割
tensor_3d = torch.tensor([
    [[1, 2], [3, 4], [5, 6]],
    [[7, 8], [9, 10], [11, 12]]
])

# 使用unbind方法按dim=1分割（按第二维度分割，即分割每个二维张量）
split_second_dim = torch.unbind(tensor_3d, dim=1)
print("\n示例4：三维张量按第二维度分割")
for i, sub_tensor in enumerate(split_second_dim):
    print(f"分块{i+1}:\n", sub_tensor)

# 示例5：四维张量，按第三维度分割
tensor_4d = torch.tensor([
    [[[1], [2]], [[3], [4]]],
    [[[5], [6]], [[7], [8]]]
])

# 使用unbind方法按dim=2分割（按第三维度分割）
split_third_dim = torch.unbind(tensor_4d, dim=2)
print("\n示例5：四维张量按第三维度分割")
for i, sub_tensor in enumerate(split_third_dim):
    print(f"分块{i+1}:\n", sub_tensor)
```

```
示例1：三通道图像张量按dim=0分割
通道1:
    tensor([[ 1,  2,  3,  4],
        [ 5,  6,  7,  8],
        [ 9, 10, 11, 12],
        [13, 14, 15, 16]])
通道2:
    tensor([[17, 18, 19, 20],
        [21, 22, 23, 24],
        [25, 26, 27, 28],
        [29, 30, 31, 32]])
通道3:
    tensor([[33, 34, 35, 36],
        [37, 38, 39, 40],
        [41, 42, 43, 44],
        [45, 46, 47, 48]])
```

```
示例2: 二维张量按行分割
行1: tensor([1, 2, 3])
行2: tensor([4, 5, 6])
行3: tensor([7, 8, 9])

示例3: 二维张量按列分割
列1: tensor([1, 4, 7])
列2: tensor([2, 5, 8])
列3: tensor([3, 6, 9])

示例4: 三维张量按第二维度分割
分块1:
    tensor([[1, 2],
        [7, 8]])
分块2:
    tensor([[ 3,  4],
        [ 9, 10]])
分块3:
    tensor([[ 5,  6],
        [11, 12]])

示例5: 四维张量按第三维度分割
分块1:
    tensor([[[1],
             [3]],

            [[5],
             [7]]])
分块2:
    tensor([[[2],
             [4]],

            [[6],
             [8]]])
```

17) 根据操作张量中的元素分类

(1) 逐元素操作

如表 47.8 所示，PyTorch 提供了很多逐元素操作的函数，可以对张量进行逐元素操作。

```python
import torch

# Initial tensor for demonstration
t = torch.tensor([[1.0, -2.0], [3.0, -4.0]])
t1 = torch.tensor([[2.0, 2.0], [2.0, 2.0]])
t2 = torch.tensor([[1.0, -1.0], [-1.0, 1.0]])

```

函数	说明
`abs/add`	绝对值 / 加法
`addcdiv(t, t1, t2, value=1)`	`t1` 与 `t2` 按元素除后，乘以 `value` 加 `t`
`addcmul(t, t1, t2, value=1)`	`t1` 与 `t2` 按元素乘后，乘以 `value` 加 `t`
`ceil/floor`	向上取整 / 向下取整
`clamp(t, min, max)`	将张量元素限制在指定区间
`exp/log/pow`	指数 / 对数 / 幂
`mul(*tensor) / neg`	逐元素乘法 / 取反
`sigmoid/tanh/softmax`	激活函数
`sign/sqrt`	取符号 / 开根号

表 47.8 常见的逐元素操作

```
print("Initial Tensor t:\n", t)
print("Tensor t1:\n", t1)
print("Tensor t2:\n", t2)

# 1. abs/add
abs_result = torch.abs(t)
add_result = torch.add(t, t1)
print("\nabs(t):\n", abs_result)
print("add(t, t1):\n", add_result)

# 2. addcdiv(t, t1, t2, value=1)
addcdiv_result = torch.addcdiv(t, t1, t2, value=1)
print("\naddcdiv(t, t1, t2, value=1):\n", addcdiv_result)

# 3. addcmul(t, t1, t2, value=1)
addcmul_result = torch.addcmul(t, t1, t2, value=1)
print("\naddcmul(t, t1, t2, value=1):\n", addcmul_result)

# 4. ceil/floor
ceil_result = torch.ceil(t)
floor_result = torch.floor(t)
print("\nceil(t):\n", ceil_result)
print("floor(t):\n", floor_result)

# 5. clamp(t, min, max)
clamp_result = torch.clamp(t, min=-1, max=1)
print("\nclamp(t, min=-1, max=1):\n", clamp_result)

# 6. exp/log/pow
exp_result = torch.exp(t)
log_result = torch.log(torch.abs(t) + 1)  # Add 1 to avoid log(0)
pow_result = torch.pow(t, 2)
print("\nexp(t):\n", exp_result)
print("log(abs(t) + 1):\n", log_result)
print("pow(t, 2):\n", pow_result)
```

```
43
44 # 7. mul(*tensor)/neg
45 mul_result = torch.mul(t, t1)
46 neg_result = torch.neg(t)
47 print("\nmul(t, t1):\n", mul_result)
48 print("neg(t):\n", neg_result)
49
50 # 8. sigmoid/tanh/softmax
51 sigmoid_result = torch.sigmoid(t)
52 tanh_result = torch.tanh(t)
53 softmax_result = torch.softmax(t, dim=0)
54 print("\nsigmoid(t):\n", sigmoid_result)
55 print("tanh(t):\n", tanh_result)
56 print("softmax(t, dim=0):\n", softmax_result)
57
58 # 9. sign/sqrt
59 sign_result = torch.sign(t)
60 sqrt_result = torch.sqrt(torch.abs(t))  # sqrt on abs to avoid
       sqrt of negative
61 print("\nsign(t):\n", sign_result)
62 print("sqrt(abs(t)):\n", sqrt_result)
```

```
Initial Tensor t:
 tensor([[ 1., -2.],
        [ 3., -4.]])
Tensor t1:
 tensor([[2., 2.],
        [2., 2.]])
Tensor t2:
 tensor([[ 1., -1.],
        [-1.,  1.]])

abs(t):
 tensor([[1., 2.],
        [3., 4.]])
add(t, t1):
 tensor([[ 3.,  0.],
        [ 5., -2.]])

addcdiv(t, t1, t2, value=1):
 tensor([[ 3., -4.],
        [ 1., -2.]])

addcmul(t, t1, t2, value=1):
 tensor([[ 3., -4.],
        [ 1., -2.]])

ceil(t):
 tensor([[ 1., -2.],
        [ 3., -4.]])
```

```
floor(t):
 tensor([[ 1., -2.],
        [ 3., -4.]])

clamp(t, min=-1, max=1):
 tensor([[ 1., -1.],
        [ 1., -1.]])

exp(t):
 tensor([[2.7183e+00, 1.3534e-01],
        [2.0086e+01, 1.8316e-02]])
log(abs(t) + 1):
 tensor([[0.6931, 1.0986],
        [1.3863, 1.6094]])
pow(t, 2):
 tensor([[ 1.,  4.],
        [ 9., 16.]])

mul(t, t1):
 tensor([[ 2., -4.],
        [ 6., -8.]])
neg(t):
 tensor([[-1.,  2.],
        [-3.,  4.]])

sigmoid(t):
 tensor([[0.7311, 0.1192],
        [0.9526, 0.0180]])
tanh(t):
 tensor([[ 0.7616, -0.9640],
        [ 0.9951, -0.9993]])
softmax(t, dim=0):
 tensor([[0.1192, 0.8808],
        [0.8808, 0.1192]])

sign(t):
 tensor([[ 1., -1.],
        [ 1., -1.]])
sqrt(abs(t)):
 tensor([[1.0000, 1.4142],
        [1.7321, 2.0000]])
```

(2) in-place 操作

在 PyTorch 中，in-place 操作是指那些直接在原地修改张量内容的操作，而不是创建新的张量。in-place 操作通常以 `_` 结尾。

1) 数学运算

    1) `add_()`: 加法操作
    2) `sub_()`: 减法操作
    3) `mul_()`: 乘法操作

4) `div_()`: 除法操作
5) `pow_()`: 幂操作

2) 张量变形和索引操作

1) `view_()`: 变形操作
2) `reshape_()`: 变形操作
3) `resize_()`: 调整张量大小
4) `zero_()`: 将张量的所有元素置为 0
5) `fill_()`: 用指定的值填充张量

3) 其他操作

1) `copy_()`: 将一个张量的内容复制到另一个张量
2) `index_add_()`: 按指定的索引对张量进行加法操作
3) `index_copy_()`: 按指定的索引对张量进行复制操作
4) `index_fill_()`: 按指定的索引对张量进行填充操作

4) 归一化操作 `normalize_()`：对张量进行归一化操作

```python
import torch

# 创建一个张量
x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)

# In-place 操作示例
x.add_(1.0)       # x = [2.0, 3.0, 4.0]
x.mul_(2.0)       # x = [4.0, 6.0, 8.0]
x.zero_()        # x = [0.0, 0.0, 0.0]

# 需要注意，进行 in-place 操作时，可能会导致计算图被破坏
y = x.sum()
y.backward()     # 如果x是叶子节点，可能会报错
```

(3) 内存布局连续

1) 连续的 Tensor（Contiguous Tensor）：如果张量的元素在内存中是按行优先（row-major order）连续存储的，则称该张量是连续的。换句话说，张量的内存布局按照其自身的存储顺序（从前到后）逐行排列，不存在任何跳跃或间隔。

2) 不连续的 Tensor（Non-contiguous Tensor）：不连续的张量是在内存中不按行优先存储的。这通常发生在张量经过某些操作（如转置、切片等）后，其数据不再按顺序排列，而是产生了间隔或跳跃。在这种情况下，张量的数据虽然依然可以访问，但可能需要额外的内存和计算来访问这些非连续存储的数据。

PyTorch 提供了 `is_contiguous()` 方法来检查张量是否是连续的，如果张量是连续的，则返回 `True`，否则返回 `False`。

```python
import torch

# 创建一个张量
x = torch.tensor([[1.0, 2.0, 3.0], [4, 5, 6]], requires_grad=True)

# 检查张量是否是连续的
is_contiguous = x.is_contiguous()
print("x 是否是连续的: ", x.is_contiguous())

# 创建一个不连续的张量
y = x.t()
print("y 是否是连续的: ", y.is_contiguous())
```

```
x 是否是连续的:  True
y 是否是连续的:  False
```

导致张量不连续的原因一下的操作：

1) 转置操作（Transpose）：

转置操作 `x.t()` 或 `x.transpose(dim0, dim1)` 会导致张量的存储顺序改变。因为转置操作会交换维度，使得在内存中的数据顺序不再是行优先的。

2) 切片（Slicing）：

对张量的部分切片操作，例如 `x[:, ::2]`，也可能导致不连续。因为切片操作可能会跳过一部分数据，导致数据在内存中不再按顺序排列。

3) 视图操作（View）：

某些视图操作，如 `x.view(-1, 2)`，可能需要重新排列内存中的数据。如果操作需要重新安排数据，结果张量可能是非连续的。

4) 拼接（Concatenation）：

例如 `torch.cat([x1, x2], dim=0)` 如果在非连续维度上拼接，也可能产生不连续的结果。

5) 高维交换（Permute）：

`x.permute(dims)` 操作会重新排列张量的维度，使其不再连续。

```python
import torch

# 创建一个张量
x = torch.tensor([[1.0, 2.0, 3.0], [4, 5, 6]], requires_grad=True)

# 检查张量是否是连续的
print("x 是否是连续的: ", x.is_contiguous())

# 转置操作 (Transpose)
y = x.t()
print("转置后的 y 是否是连续的: ", y.is_contiguous())  # 转置操作导致不连续

# 切片操作 (Slicing)
z = x[:, ::2]  # 每隔一个元素切片
print("切片后的 z 是否是连续的: ", z.is_contiguous())  # 切片操作导致不连续

# 视图操作 (View)
v = x.view(-1)  # 将张量展平为一维
print("视图后的 v 是否是连续的: ", v.is_contiguous())  # 视图操作不改变内存顺序，仍然是连续的

# 拼接操作 (Concatenation)
a = torch.tensor([[7.0, 8.0, 9.0], [10, 11, 12]], requires_grad=True)
w = torch.cat([x, a], dim=0)  # 沿第0维进行拼接
print("拼接后的 w 是否是连续的: ", w.is_contiguous())  # 沿着同一个维度拼接，结果是连续的

# 高维交换 (Permute)
p = x.permute(1, 0)  # 交换维度
print("高维交换后的 p 是否是连续的: ", p.is_contiguous())  # 高维交换操作导致不连续

# 结果分析
# 1. 转置操作 (Transpose) 会导致张量不连续，因为转置改变了元素在内存中的排列顺序。
# 2. 切片操作 (Slicing) 可能导致不连续，特别是切片间隔 (如 [::2]) 操作会跳过一些数据，破坏了连续性。
# 3. 视图操作 (View) 在张量连续的前提下，不改变内存顺序，因此结果仍然是连续的。
# 4. 沿着同一个维度进行拼接 (Concatenation) 一般不会改变张量的连续性，拼接后的张量是连续的。
```

```
# 5. 高维交换（Permute）操作改变了维度顺序，从而导致不连续。

# 对比总结
# - 转置（Transpose）、切片（Slicing，尤其是步长不为1的切片）和高维交换（Permute）操
# - 视图操作（View）和沿着同一维度的拼接（Concatenation）不会影响连续性，只要它们不
```

```
x 是否是连续的： True
转置后的 y 是否是连续的： False
切片后的 z 是否是连续的： False
视图后的 v 是否是连续的： True
拼接后的 w 是否是连续的： True
高维交换后的 p 是否是连续的： False
```

三、 autograd 自动求导微分

1) Tensor 与 Variable

☞ 在 PyTorch 0.4.0 版本之后，Variable已经被弃用，Tensor已经整合了 Variable 的功能。

虽然 Variable 仍可用，但 Variable 返回值已经是一个 Tensor（原来返回值是 Variable），所以今后无需再用 Variable 包装 Tensor。但是了解 Variable 的概念有助于理解 PyTorch 的自动求导机制。

Variable 不仅能对 Tensor 包装，而且能记录生成 Tensor 的运算（这是自动求导的关键）。

```
torch.autograd.Variable{
    data,           # 就是 Tensor 张量数据
    # 下面的是Tensor梯度运算的记录
    requires_grad,  # 是否需要梯度，
                    # 并不是所有的张量都需要求导，为了节省内存
                    # 只有需要求导的张量才会计算梯度
    grad,           # 对应于data的梯度，形状与data一致
    grad_fn,        # 记录创建该Tensor时用到的Function，
                    # 该Function在反向传播计算中使用
                    # 是自动求导的关键
    is_leaf         # 是否是叶子节点
                    # 为叶子结点时，反向传播结束，其梯度仍会保存，
                    # 非叶子结点的梯度被释放，以节省内存。
}
```

0.4.0 版本以后，Variable 并入 Tensor，可见第??页的 Tensor 属性。

```
torch.Tensor{
    # 与数据相关
    data,           # 张量数据，多维数组，最核心的属性
    dtype,          # 张量类型
    shape,          # 张量形状
```

```
    device,          # 张量在CPU或GPU上
    # 与梯度相关
    requires_grad,   # 是否需要梯度,
                     # 并不是所有的张量都需要求导，为了节省内存
                     # 只有需要求导的张量才会计算梯度
    grad,            # 梯度
    grad_fn,         # 创建Tensor的Function,是自动求导的关键
    is_leaf          # 是否是叶子节点
}
```

神经网络中的一个重要内容就是参数学习，而参数学习离不开求导。

几乎所有机器学习算法在训练或预测时都归结为求解最优化问题，如果目标函数可导，那么问题变为训练函数的驻点（即一阶导数等于零的点）。自动微分也称自动求导，算法能够计算可导函数在某点处的导数值，是反向传播算法的一般化。

`torch.autograd`包就是用来自动求导的。autograd 包为张量上所有的操作提供了自动求导功能，`torch.Tensor`和`torch.Function`为 autograd 包的两个核心类，它们相互连接并生成一个有向非循环图。

2) 自动求导要点

`autograd`包为对 Tensor 进行自动求导，自动求导时需考虑如下事项：

1) 创建叶子节点（leaf node）：通过设置`requires_grad=True`，可以记录对张量的操作，以便后续利用`backward()`进行梯度计算。

   默认情况下，`requires_grad`为 False。

2) 修改`requires_grad`：可以通过`.detach()`或使用`torch.no_grad()`方法来临时禁止梯度计算，不记录计算历史。这种方式通常在评估或测试模型时使用。

   历史记录包含了如何创建张量，以及对张量的操作，这些信息被保存在`grad_fn`属性中。

3) 非叶子节点的梯度计算:通过非叶子节点计算的张量会自动赋予`grad_fn`属性，这表明该张量是通过某个操作计算得到的。

4) 调用`backward()`：当对最终得到的张量调用`backward()`方法时，会计算其梯度，并且会将梯度累加到叶子节点的 grad 属性中。

5) 指定 backward() 的梯度参数：可以指定`backward()`中的 gradient 参数，其形状应该与调用`backward()`的张量维度匹配或是可广播的。

6) 反向传播的计算图:默认情况下,计算图在反向传播后会被释放,如果需要多次进行反向传播,则需要在调用`backward()`时将`retain_graph=True`。

7) 非叶子节点的梯度：非叶子节点的梯度在调用`backward()`时会被清除。

8) 禁止自动求导：通过`torch.no_grad()`可以暂时禁止自动求导，例如在测试或评估模型时。

在整个过程中，PyTorch 采用计算图的形式进行组织，该计算图为动态图，它的计算图在每次前向传播时都会重新构建，这样可以避免计算图过大导致的内存占用问题。

3) 计算图

计算图是一种有向无环图，用来表示算子与变量之间关系的有向无环图。

(1) 计算图的推导

推导计算图过程

用计算图表示表达式 $z = w * x + b$

如图47.4所示，表达式 $z = wx + b$ 可写成两个表示式：如果 $y = wx, z = y + b$。

其中 x, w, b 为变量，是用户创建的变量，不依赖于其他变量，故又称为**叶子节点**。

为计算各叶子节点的梯度，需要把对应的张量参数`requires_grad`属性设置为`True`，这样就可自动跟踪其历史记录。y, z 是计算得到的变量，非叶子节点，z 为根节点。`mul`和`add`是算子（或操作或函数）。这些变量及算子就构成一个完整的计算过程（或正向传播过程）。

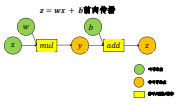

图 47.4 前向传播计算图

图 47.5 反向传播计算图

图 47.6 前向传播和反向传播计算图

图47.5所示，我们的目标是更新各叶子节点的梯度，根据复合函数导数的链式法则，不难算出各叶子节点的梯度。

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y}\frac{\partial y}{\partial x} = w$$
$$\frac{\partial z}{\partial w} = \frac{\partial z}{\partial y}\frac{\partial y}{\partial w} = x$$
$$\frac{\partial z}{\partial b} = 1$$

PyTorch 调用`backward`函数自动计算各节点的梯度，这是一个反向传播过程。在反向传播过程中，`autograd`沿着图47.5，从当前根节点 z 反向溯源，利用导数链式法则，计算所有叶子节点的梯度，并将梯度值累加到`grad`属性中。

对非叶子节点的计算操作 (或 function) 记录在`grad_fn`属性中，叶子节点的`grad_fn`值为`None`。

PyTorch 使用自动微分的计算图的生命周期如图47.7所示，PyTorch 中的计算图是动态计算图，动态计算图有两个特点：

1) 正向传播是立即执行的;

2) 反向传播后计算图立即销毁。

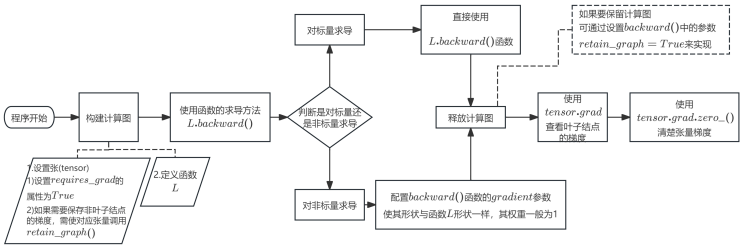

图 47.7 PyTorch 计算图的生命周期

例子 47.3

用计算图表示 $y=(x+w)\times(w+1)$，在 $x=2,w=1$ 时的计算图。

方法:

令 $\begin{cases} a=x+w \\ b=w+1 \\ y=a\times b \end{cases}$

表示成有向无环图如图47.8所示。

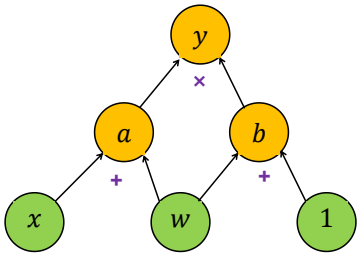

图 47.8 计算图例题演示

带入 $x=2,w=1$，计算 y 的值，就是前向传播的过程。

y 对 w 计算偏导

$$\begin{aligned}\frac{\partial y}{\partial w} &= \frac{\partial y}{\partial a}\frac{\partial a}{\partial w}+\frac{\partial y}{\partial b}\frac{\partial b}{\partial w} \\ &= b\times 1+a\times 1 \\ &= b+a \\ &= (w+1)+(x+w) \\ &= 2*w+x+1 \\ &= 2\star 1+2+1=5\end{aligned}$$

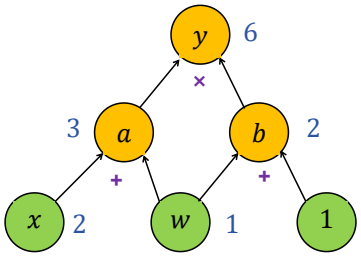

图 47.9 计算图例题演示-前向传播

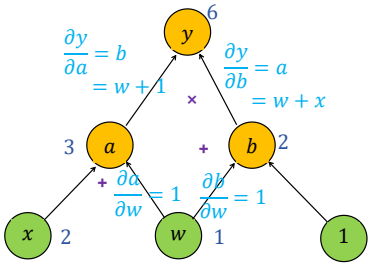

图 47.10 计算图例题演示-y 对 w 求梯度

用代码实现如下：

```
import torch

w = torch.tensor([1.0], requires_grad=True)  # 初始化权重 w
x = torch.tensor([2.0], requires_grad=True)  # 初始化输入 x

a = torch.add(w, x)  # retain_grad() 保留梯度
b = torch.add(w, 1)
y = torch.mul(a, b)  # 计算 y

y.backward()  # 反向传播
print("y 对 w 的梯度: ", w.grad)  # 输出 y 对 w 的梯度

```

```
y 对 w 的梯度:  tensor([5.])
```

同理，y 对 x 的梯度为

$$\begin{aligned}\frac{\partial y}{\partial x} &= \frac{\partial y}{\partial a}\frac{\partial a}{\partial x} \\ &= b \times 1 \\ &= b \\ &= w + 1 \\ &= 1 + 1 = 2\end{aligned}$$

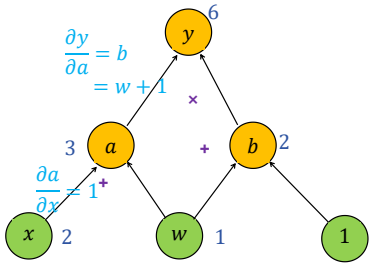

图 47.11 计算图例题演示-y 对 x 求梯度

用代码实现如下：

```
import torch

w = torch.tensor([1.0], requires_grad=True)  # 初始化权重 w
x = torch.tensor([2.0], requires_grad=True)  # 初始化输入 x

a = torch.add(w, x)  # retain_grad() 保留梯度
b = torch.add(w, 1)
y = torch.mul(a, b)  # 计算 y

y.backward()  # 反向传播
print("y 对 w 的梯度: ", w.grad)  # 输出 y 对 w 的梯度
print("y 对 x 的梯度: ", x.grad)  # 输出 y 对 x 的梯度

```

```
y 对 w 的梯度:  tensor([5.])
y 对 x 的梯度:  tensor([2.])
```

(2) 计算图的元素

1) 结点：

结点表示数据：向量、矩阵、张量等，

结点有两种类型：

1) 叶子节点: 用户创建的变量，不依赖于其他变量。
2) 非叶子节点：计算得到的变量，依赖于其他变量。

只有叶子结点才有**梯度**，梯度值保存在`grad`属性中。`requires_grad`属性为`True`。

```
import torch

w = torch.tensor([1.0], requires_grad=True)  # 初始化权重 w
x = torch.tensor([2.0], requires_grad=True)  # 初始化输入 x

a = torch.add(w, x)  # retain_grad() 保留梯度
b = torch.add(w, 1)
y = torch.mul(a, b)  # 计算 y

# 查看是否是叶子节点
print("w 是否是叶子节点: ", w.is_leaf, "梯度为: ", w.grad)
print("x 是否是叶子节点: ", x.is_leaf, "梯度为: ", x.grad)
print("a 是否是叶子节点: ", a.is_leaf, "梯度为: ", a.grad)
print("b 是否是叶子节点: ", b.is_leaf, "梯度为: ", b.grad)
print("y 是否是叶子节点: ", y.is_leaf, "梯度为: ", y.grad)

```

```
w 是否是叶子节点:  True 梯度为:  tensor([5.])
x 是否是叶子节点:  True 梯度为:  tensor([2.])
# 非叶子结点在反向传播结束后梯度会被自动清空
a 是否是叶子节点:  False 梯度为:  None
b 是否是叶子节点:  False 梯度为:  None
y 是否是叶子节点:  False 梯度为:  None
```

非叶子结点在反向传播结束后梯度会被自动清空。如果希望保留非叶子节点的梯度，可以使用`retain_grad()`方法。

```

```

```python
import torch

w = torch.tensor([1.0], requires_grad=True)  # 初始化权重 w
x = torch.tensor([2.0], requires_grad=True)  # 初始化输入 x

a = torch.add(w, x)  # retain_grad() 保留梯度
a.retain_grad()
b = torch.add(w, 1)
y = torch.mul(a, b)  # 计算 y

y.backward()  # 反向传播


# 查看是否是叶子节点
print("w 是否是叶子节点: ", w.is_leaf, "梯度为: ", w.grad)
print("x 是否是叶子节点: ", x.is_leaf, "梯度为: ", x.grad)
print("a 是否是叶子节点: ", a.is_leaf, "梯度为: ", a.grad)
print("b 是否是叶子节点: ", b.is_leaf, "梯度为: ", b.grad)
print("y 是否是叶子节点: ", y.is_leaf, "梯度为: ", y.grad)

```

```
w 是否是叶子节点:  True 梯度为:  tensor([5.])
x 是否是叶子节点:  True 梯度为:  tensor([2.])
a 是否是叶子节点:  False 梯度为:  tensor([2.])
b 是否是叶子节点:  False 梯度为:  None
y 是否是叶子节点:  False 梯度为:  None
```

2) 边：

边表示操作：加法、乘法、激活函数等。

(3) 计算图的属性 grad_fn

`grad_fn`属性表示该张量是通过什么操作得到的，是一个`Function`对象。`grad_fn`的值：

1) **None**：当一个张量是手动创建的

例如使用 `torch.tensor()` 或 `torch.rand()`且没有参与任何需要自动求导的操作时，其 `grad_fn` 为 `None`，这种张量被称为叶子节点（leaf tensor）。

2) **AddBackward**：表示加法操作的反向传播函数。

例如，`z = x + y`，`z.grad_fn` 为 `AddBackward`。

3) **SubBackward**：表示减法操作的反向传播函数。
   例如，`z = x - y`，`z.grad_fn` 为 `SubBackward`。

4) **MulBackward**：表示乘法操作的反向传播函数。
   例如，`z = x * y`，`z.grad_fn` 为 `MulBackward`。

5) **DivBackward**：表示除法操作的反向传播函数。
   例如，`z = x / y`，`z.grad_fn` 为 `DivBackward`。

6) **PowBackward**：表示幂运算的反向传播函数。
   例如，`z = x.pow(2)`，`z.grad_fn` 为 `PowBackward`。

7) **MatMulBackward**：表示矩阵乘法操作的反向传播函数。
   例如，`z = torch.matmul(x, y)` 或 `z = x @ y`，`z.grad_fn` 为 `MatMulBackward`。

8) **MeanBackward**：表示求均值操作的反向传播函数。
   例如，`z = x.mean()`，`z.grad_fn` 为 `MeanBackward`。

9) **SumBackward**：表示求和操作的反向传播函数。
   例如，`z = x.sum()`，`z.grad_fn` 为 `SumBackward`。

10) **ExpBackward**：表示指数函数的反向传播函数。
    例如，`z = x.exp()`，`z.grad_fn` 为 `ExpBackward`。

11) **SqrtBackward**：表示平方根函数的反向传播函数。
    例如，`z = x.sqrt()`，`z.grad_fn` 为 `SqrtBackward`。

12) **LogBackward**：表示对数函数的反向传播函数。
    例如，`z = x.log()`，`z.grad_fn` 为 `LogBackward`。

13) **TBackward**：表示转置操作的反向传播函数。
    例如，`z = x.t()`（二维矩阵转置），`z.grad_fn` 为 `TBackward`。

14) **ReluBackward**：表示 ReLU 激活函数的反向传播函数。
    例如，`z = torch.relu(x)`，`z.grad_fn` 为 `ReluBackward`。

15) **SigmoidBackward**：表示 Sigmoid 激活函数的反向传播函数。
    例如，`z = torch.sigmoid(x)`，`z.grad_fn` 为 `SigmoidBackward`。

16) **ViewBackward**：表示视图操作（即张量视图转换）的反向传播函数。
    例如，`z = x.view(-1)`，`z.grad_fn` 为 `ViewBackward`。

17) **CatBackward**：表示张量拼接操作的反向传播函数。
    例如，`z = torch.cat((x, y), dim=0)`，`z.grad_fn` 为 `CatBackward`。

18) **CloneBackward**：表示克隆操作的反向传播函数。
例如，`z = x.clone()`，`z.grad_fn` 为 `CloneBackward`。

19) **IndexBackward**：表示索引操作的反向传播函数。
例如，`z = x[0]`，`z.grad_fn` 为 `IndexBackward`。

```python
import torch

w = torch.tensor([1.0], requires_grad=True)  # 初始化权重 w
x = torch.tensor([2.0], requires_grad=True)  # 初始化输入 x

a = torch.add(w, x)  # retain_grad() 保留梯度
a.retain_grad()
b = torch.add(w, 1)
y = torch.mul(a, b)  # 计算 y

y.backward()  # 反向传播


# 查看grad_fn
print("w.grad_fn: ", w.grad_fn)
print("x.grad_fn: ", x.grad_fn)
print("a.grad_fn: ", a.grad_fn)
print("b.grad_fn: ", b.grad_fn)
print("y.grad_fn: ", y.grad_fn)
```

```
w.grad_fn:  None # 因为是叶子结点，所以没有grad_fn
x.grad_fn:  None
a.grad_fn:  <AddBackward0 object at 0x7fae3975ae60>
b.grad_fn:  <AddBackward0 object at 0x7fae43a57fa0>
y.grad_fn:  <MulBackward0 object at 0x7fae3975ae60>
```

4) autograd 包

(1) torch.autograd 简介

1) 自动求导机制通过有向无环图（directed acyclic graph ， DAG）实现

2) 在 DAG 中，记录数据（对应 tensor.data）以及操作（对应 tensor.grad_fn）

3) 操作在 pytorch 中统称为 Function，如加法、减法、乘法、ReLU、conv、Pooling 等，统统是 Function

autograd 的使用有很多方法，这里重点常用的

1) `torch.autograd.backward()`：自动求取梯度

2) `torch.autograd.grad()`：求取梯度

3) `torch.autograd.Function`：自定义 Function

(2) torch.autograd.backward()

PyTorch 提供了`torch.autograd.backward()`函数用于实现自动求取梯度。

```
# 功能：实现自动求取梯度
torch.autograd.backward(
    tensors,                # 用于计算梯度的张量
    grad_tensors = None,    #
    计算非标量的梯度。其形状一般与前面的张量保持一致。
    retain_graph=None,      # 由于PyTorch采用动态图机制，
                            #
    在调用一次backward函数后PyTorch会自动销毁计算图
                            #
    如果想对某个变量重复调用backward函数，则需要将该参数设置为True。
    create_graph=False,     # 创建导数计算图，用于高阶求导
                            # 当设置为True的时候可以用来计算更高阶的梯度。
    grad_variables=None     #
    这个参数后面版本中应该会丢弃，直接使用grad_tensors就好了。
)
```

注意：我们使用时候都是在张量上直接调用`.backward()`函数，但这里却是`torch.autograd.backward`，为什么不一样呢？其实`Tensor.backward()`接口内部调用了`autograd.backward`。

1) `retain_grad`参数使用：

```
#####  retain_graph=True
import torch
w = torch.tensor([1.], requires_grad=True)
x = torch.tensor([2.], requires_grad=True)

a = torch.add(w, x)
b = torch.add(w, 1)
y = torch.mul(a, b)

y.backward(retain_graph=True)
print(w.grad)
y.backward()
print(w.grad)

```

```
tensor([5.])
tensor([10.])
```

```
#####  retain_graph=False
import torch
w = torch.tensor([1.], requires_grad=True)
x = torch.tensor([2.], requires_grad=True)

a = torch.add(w, x)
b = torch.add(w, 1)
y = torch.mul(a, b)

y.backward()
print(w.grad)
y.backward()
print(w.grad)

```

```
tensor([5.])

---------------------------------------------------------------------------
RuntimeError                              Traceback (most
recent call last)
<ipython-input-9-64bccc64184d> in <module>
    10 y.backward()
    11 print(w.grad)
---> 12 y.backward()
    13 print(w.grad)
RuntimeError: Trying to backward through the graph a second
time (or directly access saved tensors after they have already
been freed). Saved intermediate values of the graph are freed
when you call .backward() or autograd.grad(). Specify
retain_graph=True if you need to backward through the graph a
second time or if you need to access saved tensors after
calling backward.
```

2) `grad_tensors`参数使用：

```
w = torch.tensor([1.], requires_grad=True)
x = torch.tensor([2.], requires_grad=True)

a = torch.add(w, x)
b = torch.add(w, 1)

y0 = torch.mul(a, b)    # y0 = (x+w) * (w+1)    dy0/dw = 2w + x + 1
y1 = torch.add(a, b)    # y1 = (x+w) + (w+1)    dy1/dw = 2

loss = torch.cat([y0, y1], dim=0)       # [y0, y1]

grad_tensors = torch.tensor([1., 2.])

loss.backward(gradient=grad_tensors)    # Tensor.backward中的 gradient 传入 torch.autograd.backward()中的grad_tensors

# w =  1* (dy0/dw)  +   2*(dy1/dw)
# w =  1* (2w + x + 1)  +   2*(w)
# w =  1* (5)  +   2*(2)
# w =  9

print(w.grad)

```

```
tensor([9.])
```

例子 47.4

令 $\begin{cases} a = w + x \\ b = w + 1 \end{cases}$，$\begin{cases} y_0 = a \times b = w^2 + w + wx + x \\ y_1 = a + b = 2w + x + 1 \end{cases}$

假定计算在 $w = 1, x = 2$ 时的梯度

当 $gradf = \nabla f_{loss} = (\frac{\partial f}{\partial y_0}, \frac{\partial f}{\partial y_1}) = (1,2)$, 计算 w 的梯度 $w.grad$

当 $\nabla f_{loss} = (3,4)$, 计算 w 的梯度 $w.grad$

方法:

1) 当 $\nabla f_{loss} = (1,2)$

(1) 计算在 $w=1, x=2$ 时的 y_0, y_1 的值

$$\begin{cases} y_0 = a \times b = (x+w) \times (w+1) = (2+1) \times (1+1) = 6 \\ y_1 = a + b = (x+w) + (w+1) = (2+1) + (1+1) = 5 \end{cases}$$

(2) 构造损失函数因为在进行反向传播时，通常会对损失函数（loss）进行求导，以获得各个参数的梯度, 因此定义函数 f_{loss}

$$f_{loss} = f(y_0, y_1)$$

上面的 f_{loss} 使用 python 实现如下

```
                        loss =
torch.cat([y_0,y_1],dim=0)=[6,5]

```

$$\begin{aligned} \frac{\partial y_0}{\partial w} &= \frac{\partial}{\partial w}[(x+w) \times (w+1)] \\ &= \frac{\partial}{\partial w}[(2+w) \times (1+w)] \\ &= (2+w) + (1+w) = 3 + 2w \end{aligned}$$

当 $w=1, \frac{\partial y_0}{\partial w} = 3 + 2 \times 1 = 5$

同理 $\frac{\partial y_1}{\partial w} = \frac{\partial}{\partial w}[(x+w) + (w+1)] = 1 + 1 = 2$

(3) 调用`backward()`函数计算梯度当 loss 的梯度为 $\nabla f_{loss} = (\frac{\partial \text{loss}}{\partial y_0} = 1, \frac{\partial \text{loss}}{\partial y_1} = 2) = (1, 2)$

上面赋值 ∇f_{loss} 的代码如下：

```
                        grad_tensors =
torch.tensor([1., 2.])

loss.backward(gradient=grad_tensors)

```

因此

$$\begin{aligned} w.\text{grad} &= \frac{\partial \text{loss}}{\partial y_0} \times \frac{\partial y_0}{\partial w} + \frac{\partial \text{loss}}{\partial y_1} \times \frac{\partial y_1}{\partial w} \\ &= 1 \times 5 + 2 \times 2 \\ &= 9 \end{aligned}$$

2) 当 $\nabla f_{loss} = (3,4)$

$$
\begin{aligned}
w.\text{grad} &= 3 \times \frac{\partial y_0}{\partial w} + 4 \times \frac{\partial y_1}{\partial w} \\
&= 3 \times 5 + 4 \times 2 \\
&= 23
\end{aligned}
$$

```
        import torch

        # 依赖于叶子节点的节点，requires_grad 默认为 True
        w = torch.tensor([1.], requires_grad=True)
        x = torch.tensor([2.] ,requires_grad=True)
        a = torch.add(w, x)
        b = torch.add(w, 1)

        y_0 = torch.mul(a, b)
        y_1 = torch.add(a, b)
        loss = torch.cat([y_0, y_1], dim=0)
        grad_tensors = torch.tensor([1., 2.])
        # loss.backward(gradient=grad_tensors) # 如果就计算1次
        # 当∇f_loss = (1,2)时
        loss.backward(gradient=grad_tensors,
    retain_graph=True)
        print("w.grad:", w.grad)

        # 在设置 retain_graph=True 的情况下，可以多次调用
    backward
        # 当∇f_loss = (3,4)时
        # 梯度不能自动清零，在每次反向传播中会叠加
        w.grad.zero_()
        grad_tensors = torch.tensor([3., 4.])
        loss.backward(gradient=grad_tensors)
        print("w.grad:", w.grad)

```

```
    w.grad: tensor([9.])
    w.grad: tensor([23.])
```

[I] 标量反向传播

```python
import torch
# 定义输入张量x
x = torch.Tensor([3])

# 初始化权重参数w和偏置b，并设置 requires_grad=True 表示需要计算梯度
w = torch.randn(1, requires_grad=True) # 随机初始化一个权重参数
b = torch.randn(1, requires_grad=True) # 随机初始化一个偏置

print("初始化的权重:", w, "\n初始化的偏置:", b, "\n输入张量x:", x)
# 实现正向传播
y = torch.mul(w, x) # 等价于 y = w * x
z = torch.add(y, b) # 等价于 z = y + b

print("张量y:", y, "\n张量z:", z)
# 查看 x, w, b 的 requires_grad 属性
print("x, w, b 的 requires_grad 属性分别为:", x.requires_grad,
    w.requires_grad, b.requires_grad)

# 查看非叶子结点的 requires_grad 属性
## 因与w、b有依赖关系,故y、z的requires_grad属性也是: True、 True
print("y, z 的 requires_grad 属性分别为:", y.requires_grad,
    z.requires_grad)

# 查看个节点是否为叶子节点
print("x, w, b, y, z 是否为叶子结点:", x.is_leaf, w.is_leaf,
    b.is_leaf, y.is_leaf, z.is_leaf)

#查看叶子节点的grad_fn属性
print("x, w, b 的 grad_fn 属性分别为:", x.grad_fn, w.grad_fn,
    b.grad_fn)

# 查看非叶子结点的 grad_fn 属性
print("y, z 的 grad_fn 属性分别为:", y.grad_fn, z.grad_fn)

# 基于 z 张量进行梯度反向传播，执行backward()操作
z.backward()
#
    如果需要多次使用backward函数，需要修改参数retain_graph为True,此时梯度是累加的
# z.backward(retain_graph=True)
#查看叶子节点的梯度,
# x是叶子节点但因为不需要更新，因此它无须求导,，故其梯度为None
print("各个叶子结点的梯度x, w, b 分别为:", x.grad, w.grad, b.grad)
# 非叶子节点的梯度，执行backward函数之后，会自动清空
print("非叶子结点的梯度:", y.grad, z.grad)
```

```
初始化的权重: tensor([0.4014], requires_grad=True)
初始化的偏置: tensor([-0.4248], requires_grad=True)
输入张量x: tensor([3.])
张量y: tensor([1.2041], grad_fn=<MulBackward0>)
张量z: tensor([0.7792], grad_fn=<AddBackward0>)
x, w, b 的 requires_grad 属性分别为: False True True
y, z 的 requires_grad 属性分别为: True True
x, w, b, y, z 是否为叶子结点: True True True False False
x, w, b 的 grad_fn 属性分别为: None None None
y, z 的 grad_fn 属性分别为: <MulBackward0 object at 0x7fc7a549f8e0>
    <AddBackward0 object at 0x7fc7a549f8b0>
各个叶子结点的梯度x, w, b 分别为: None tensor([3.]) tensor([1.])
非叶子结点的梯度: None None
```

如果需要多次使用`backward`函数，需要修改参数`retain_graph`为`True`，此时梯度是累加的

```python
import torch

# 定义输入张量x
x = torch.Tensor([2])

# 初始化权重参数w和偏置b, 并设置 requires_grad=True 表示需要计算梯度
w = torch.randn(1, requires_grad=True)  # 随机初始化一个权重参数
b = torch.randn(1, requires_grad=True)  # 随机初始化一个偏置

print("初始化的权重:", w.item(), "\n初始化的偏置:", b.item(),
    "\n输入张量x:", x.item())

# 实现正向传播
y = torch.mul(w, x)  # 等价于 y = w * x
z = torch.add(y, b)  # 等价于 z = y + b

print("张量y:", y.item(), "\n张量z:", z.item())

# 查看 x, w, b 的 requires_grad 属性
print("x, w, b 的 requires_grad 属性分别为:", x.requires_grad,
    w.requires_grad, b.requires_grad)

# 查看非叶子结点的 requires_grad 属性
print("y, z 的 requires_grad 属性分别为:", y.requires_grad,
    z.requires_grad)

# 查看个节点是否为叶子节点
print("x, w, b, y, z 是否为叶子结点:", x.is_leaf, w.is_leaf,
    b.is_leaf, y.is_leaf, z.is_leaf)

# 查看叶子节点的grad_fn属性
print("x, w, b 的 grad_fn 属性分别为:", x.grad_fn, w.grad_fn,
```

```
    b.grad_fn)

# 查看非叶子结点的 grad_fn 属性
print("y, z 的 grad_fn 属性分别为:", y.grad_fn, z.grad_fn)

# 第一次反向传播
print("\n==========================================================")
z.backward(retain_graph=True)
#
    如果需要多次使用backward函数, 需要修改参数retain_graph为True,此时梯度是累加的
print("第一次反向传播后各个结点的值x, w, b, y, z分别为:", x.item(),
    w.item(), b.item(), y.item(), z.item())
print("第一次反向传播后各个叶子结点的梯度 x, w, b 分别为:", x.grad,
    w.grad.item(), b.grad.item())
print("第一次反向传播后非叶子结点的梯度 y, z:", y.grad, z.grad)

# 第二次反向传播
print("\n==========================================================")
z.backward(retain_graph=True)
print("第二次反向传播后各个结点的值x, w, b, y, z分别为:", x.item(),
    w.item(), b.item(), y.item(), z.item())
print("第二次反向传播后各个叶子结点的梯度 x, w, b 分别为:", x.grad,
    w.grad.item(), b.grad.item())
print("第二次反向传播后非叶子结点的梯度 y, z:", y.grad, z.grad)

# 第三次反向传播
print("\n==========================================================")
z.backward()
print("第三次反向传播后各个结点的值x, w, b, y, z分别为:", x.item(),
    w.item(), b.item(), y.item(), z.item())
print("第三次反向传播后各个叶子结点的梯度 x, w, b 分别为:", x.grad,
    w.grad.item(), b.grad.item())
print("第三次反向传播后非叶子结点的梯度 y, z:", y.grad, z.grad)
```

```
初始化的权重: -0.74216628074646
初始化的偏置: -0.9749470949172974
输入张量x: 2.0
张量y: -1.48433256149292
张量z: -2.4592795372009277
x, w, b 的 requires_grad 属性分别为: False True True
y, z 的 requires_grad 属性分别为: True True
x, w, b, y, z 是否为叶子结点: True True True False False
x, w, b 的 grad_fn 属性分别为: None None None
y, z 的 grad_fn 属性分别为: <MulBackward0 object at 0x7fc7a5337eb0>
    <AddBackward0 object at 0x7fc7a5337ee0>

==========================================================
第一次反向传播后各个结点的值x, w, b, y, z分别为: 2.0 -0.74216628074646
    -0.9749470949172974 -1.48433256149292 -2.4592795372009277
```

```
第一次反向传播后各个叶子结点的梯度 x, w, b 分别为: None 2.0 1.0
第一次反向传播后非叶子结点的梯度 y, z: None None

============================================================
第二次反向传播后各个结点的值x, w, b, y, z分别为: 2.0 -0.74216628074646
    -0.9749470949172974 -1.48433256149292 -2.4592795372009277
第二次反向传播后各个叶子结点的梯度 x, w, b 分别为: None 4.0 2.0
# 这里的梯度是在上一次的基础上累加的, 因此就是2倍
第二次反向传播后非叶子结点的梯度 y, z: None None

============================================================
第三次反向传播后各个结点的值x, w, b, y, z分别为: 2.0 -0.74216628074646
    -0.9749470949172974 -1.48433256149292 -2.4592795372009277
第三次反向传播后各个叶子结点的梯度 x, w, b 分别为: None 6.0 3.0
第三次反向传播后非叶子结点的梯度 y, z: None None
```

[II] 非标量反向传播

1) 一阶偏导，梯度

对于非标量的反向传播，不能直接使用`backward()`函数，Y 为一个向量，我们想办法把这个向量转变成一个标量不就好了？比如我们可以对 Y 求和然后用求和得到的标量对 X 求导，这样不会对结果有影响，例如：

$$Y_{\text{sum}} = \sum y_i = x_1^2 + x_2^2 + 6$$

$$\frac{\partial Y_{sum}}{\partial x_1} = 2x_1, \frac{\partial Y_{sum}}{\partial x_2} = 2x_2$$

实际上，对 Y 求和等价于 Y 点积一个全为 1 的向量或矩阵。这个向量矩阵 V 也就是我们需要传入的`grad_tensors`参数。(点积只是相对于一维向量而言的，对于矩阵或更高维的张量，可以看作对每一个维度做点积运算。)

```python
import torch
X = torch.ones(2, requires_grad=True)
print("X:", X)
Y = X**2 + 3
print("Y:", Y)
# Y.backward() # 因为Y非标量, 报错
Y.sum().backward()
print("X.grad:", X.grad)

```

```
X: tensor([1., 1.], requires_grad=True)
Y: tensor([4., 4.], grad_fn=<AddBackward0>)
```

```
        X.grad: tensor([2., 2.])
```

2) Jacobian 矩阵

例子 47.5

我们先手工计算一下 y 对 x 的梯度，以验证 PyTorch 的 backward 函数的结果是否正确。y 对 x 的梯度是一个雅可比矩阵，各项的值可通过以下方法进行计算。

方法: 假设 $x=(x_1=2,x_2=3), y=\left(y_1=x_1^2+3x_2, y_2=x_2^2+2x_1\right)$, 不难得到

$$J=\begin{bmatrix}\frac{\partial y_1}{\partial x_1} & \frac{\partial y_1}{\partial x_2}\\ \frac{\partial y_2}{\partial x_1} & \frac{\partial y_2}{\partial x_2}\end{bmatrix}=\begin{bmatrix}2x_1 & 3\\ 2 & 2x_2\end{bmatrix}$$

$$J^{\mathrm{T}}\times[1,0]^{\mathrm{T}}=\begin{bmatrix}\frac{\partial y_1}{\partial x_1}\\ \frac{\partial y_1}{\partial x_2}\end{bmatrix}$$

$$J^{\mathrm{T}}\times[0,1]^{\mathrm{T}}=\begin{bmatrix}\frac{\partial y_2}{\partial x_1}\\ \frac{\partial y_2}{\partial x_2}\end{bmatrix}$$

当 $x_1=2, x_2=3$ 时

$$J=\begin{bmatrix}4 & 3\\ 2 & 6\end{bmatrix}$$

所以

$$J^{T}=\begin{bmatrix}4 & 2\\ 3 & 6\end{bmatrix}$$

由此可得

$$J^{\mathrm{T}}\times[1,0]^{\mathrm{T}}=\begin{bmatrix}\frac{\partial y_1}{\partial x_1}\end{bmatrix}=\begin{bmatrix}4\\ 3\end{bmatrix}$$

$$J^{T}\times[0,1]^{T}=\begin{bmatrix}\frac{\partial y_2}{\partial x_1}\\ \frac{\partial y_2}{\partial x_2}\end{bmatrix}=\begin{bmatrix}2\\ 6\end{bmatrix}$$

```
        import torch

        # 定义叶子结点张量 x 形状为 1x2
        x = torch.tensor([[2.0, 3.0]],dtype=torch.float32,
    requires_grad=True)
        print("x:", x)

        # 初始化雅可比矩阵
        J = torch.zeros(2, 2)

```

```
        # 初始化目标张量, 形状为 1x2
        y = torch.tensor([[1.0, 2.0]],dtype=torch.float32)
        print("初始化y:", y)

        # 定义 y 与 x 之间的映射关系
        y[0, 0] = x[0, 0] ** 2 + 3 * x[0, 1]
        y[0, 1] = x[0, 1] ** 2 + 2 * x[0, 0]
        print("定义之后的y:", y)

        ################################################
        # 1.先让 v = (1, 0) 得到 y_1 对 x 的梯度
        # 需要重复使用 backward() 函数, 因此需要设置
    retain_graph=True
        y.backward(torch.tensor([[1.0, 0.0]]),
    retain_graph=True)
        J[0] = x.grad
        ## 因为 x.grad 是累加的, 所以需要清零
        x.grad = torch.zeros_like(x.grad)
        print("y_1对x的梯度:", J[0])

        # 2.再让 v = (0, 1) 得到 y_2 对 x 的梯度
        y.backward(torch.tensor([[0.0, 1.0]]))
        J[1] = x.grad
        print("y_2对x的梯度:", J[1])

        # 输出雅可比矩阵
        print("雅可比矩阵J:", J)
```

```
x: tensor([[2., 3.]], requires_grad=True)
初始化y: tensor([[1., 2.]])
定义之后的y: tensor([[13., 13.]], grad_fn=<CopySlices>)
y_1对x的梯度: tensor([4., 3.])
y_2对x的梯度: tensor([2., 6.])
雅可比矩阵J: tensor([[4., 3.],
                     [2., 6.]])
```

注意：这里的`backward`函数的参数`grad_tensors`是一个向量，而不是一个标量。而且选取不对的`grad_tensors`会导致计算出的梯度不对。

例子 47.6

如果 $v=(1,1)$ 计算 y 对 x 的梯度，结果是多少？

方法:

$$J^{\mathrm{T}} \times v^{\mathrm{T}} = \begin{bmatrix} 4 & 2 \\ 3 & 6 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 6 \\ 9 \end{bmatrix}$$

```
        import torch

        # 定义叶子结点张量 x 形状为 1x2
        x = torch.tensor([[2.0, 3.0]],dtype=torch.float32,
    requires_grad=True)
        print("x:", x)

        # 初始化目标张量, 形状为 1x2
        y = torch.tensor([[1.0, 2.0]],dtype=torch.float32)
        print("初始化y:", y)

        # 定义 y 与 x 之间的映射关系
        y[0, 0] = x[0, 0] ** 2 + 3 * x[0, 1]
        y[0, 1] = x[0, 1] ** 2 + 2 * x[0, 0]
        print("定义之后的y:", y)

        y.backward(torch.tensor([[1.0, 1.0]]),
    retain_graph=True)
        print("y_1对x的梯度:", x.grad)

```

```
x: tensor([[2., 3.]], requires_grad=True)
初始化y: tensor([[1., 2.]])
定义之后的y: tensor([[13., 13.]], grad_fn=<CopySlices>)
y_1对x的梯度: tensor([[6., 9.]])
```

(3) torch.autograd.grad()

PyTorch 提供了`torch.autograd.grad()`函数用于实现自动求取梯度。

```
# 功能: 实现自动求取梯度
torch.autograd.grad(
    outputs,                # 用于求导的张量
    inputs,                 # 需要梯度的张量
    grad_outputs=None,      #
    计算非标量的梯度。其形状一般与前面的张量保持一致。
    retain_graph=None,      # 由于PyTorch采用动态图机制,
                            #
    在调用一次backward函数后PyTorch会自动销毁计算图
                            #
    如果想对某个变量重复调用backward函数, 则需要将该参数设置为True。
    create_graph=False      # 创建导数计算图, 用于高阶求导
                            # 当设置为True的时候可以用来计算更高阶的梯度。
    allow_unused=False      #
    如果为True, 即使某个输入张量的梯度未被使用, 也不会报错
```

```
)
```

其实`torch.autograd.grad()`函数与`torch.autograd.backward()`函数的功能类似，

1) `torch.autograd.backward()`函数是一个较为通用的函数，适用于计算损失函数相对于所有输入的梯度。

2) `torch.autograd.grad()`函数是一个更灵活的函数，允许用户指定计算哪些张量的梯度，并可以返回这些梯度，而不直接修改原始张量的`.grad`属性。

例子 47.7

$z = x \times y$, 在 $x = 2, y = 3$ 时，计算 $\frac{\partial z}{\partial x}$ 和 $\frac{\partial z}{\partial y}$

方法:

1) 使用`torch.autograd.grad()`函数计算

```
import torch

# 创建张量并设置 requires_grad=True
x = torch.tensor([2.0], requires_grad=True)
y = torch.tensor([3.0], requires_grad=True)

# 定义一个简单的计算
z = x * y  # z = 2.0 * 3.0 = 6.0

# 调用 grad() 计算梯度
grad_x, grad_y = torch.autograd.grad(outputs=z, inputs=[x, y])

# 打印计算得到的梯度
print("grad_x:", grad_x)  # 输出: tensor([3.])
print("grad_y:", grad_y)  # 输出: tensor([2.])

```

```
grad_x: tensor([3.])
grad_y: tensor([2.])
```

2) 使用`torch.autograd.backward()`函数计算

```
import torch

# 创建张量并设置 requires_grad=True
x = torch.tensor([2.0], requires_grad=True)
y = torch.tensor([3.0], requires_grad=True)

# 定义一个简单的计算
z = x * y  # z = 2.0 * 3.0 = 6.0

# 调用 backward() 计算梯度
z.backward()

# 打印 x 和 y 的梯度
print("x.grad:", x.grad)  # 输出: tensor([3.])
print("y.grad:", y.grad)  # 输出: tensor([2.])

```

```
grad_x: tensor([3.])
grad_y: tensor([2.])
```

(4) torch.autograd.Function

有的时候，想要实现自己的一些操作（op），如特殊的数学函数、pytorch 的 module 中没有的网络层，那就需要自己写一个 Function，在 Function 中定义好`forward`的计算公式、`backward`的计算公式，然后将这些 op 组合到模型中，模型就可以用`autograd`完成梯度求取。

例子 47.8

假设需要一个计算指数的功能，并且能组合到模型中，实现 autograd

1) 继承 Function
2) 实现 forward 和 backward 函数

方法:

1) forward 和 backward 函数第一个参数为`ctx`, 它的作用类似于类函数的`self`一样。

   `ctx`是一个上下文对象，具体为 context，它在自定义的 autograd 函数中用于在前向传播和反向传播之间共享信息。

   1) 在 forward 中
      ctx 用来存储中间计算结果，供反向传播时使用。通过调用`ctx.save_for_backward`可以将中间变量存储到 ctx 对象中（如代码中的 result）。
   2) 在 backward 中
      可以通过`ctx.saved_tensors`取出这些保存的变量，计算梯度。这是 PyTorch 提供的机制，确保在反向传播时能够获取到前向传播中的关键变量。

2) backward 函数返回的参数个数与 forward 的输入参数个数相同, 即，传入该 op 的参数，都需要给它们计算对应的梯度。

```python
import torch
from torch.autograd.function import Function

class Exp(Function):
    @staticmethod # 静态方法
    # 函数功能：计算输入 i 的指数值 i.exp()
    # ctx: 一个上下文对象，用于在 forward 和 backward 之间传递信息
    def forward(ctx, i):
        # ============== step1: 函数功能实现 ==============
        result = i.exp()
        # ============== step1: 函数功能实现 ==============

        # ============== step2: 结果保存，用于反向传播 ==============
        ctx.save_for_backward(result)
        # ============== step2: 结果保存，用于反向传播 ==============

        return result

    @staticmethod
    # 函数功能：计算梯度
    def backward(ctx, grad_output):
        # ============== step1: 取出结果，用于反向传播 ==============
        result, = ctx.saved_tensors
        # ============== step1: 取出结果，用于反向传播 ==============


```

```
27              # ============== step2: 反向传播公式实现
    ==============
28              grad_results = grad_output * result
29              # ============== step2: 反向传播公式实现
    ==============
30
31
32              return grad_results
33
34      x = torch.tensor([1.], requires_grad=True)
35      # 前向传播
36      y = Exp.apply(x)    # 需要使用apply方法调用自定义autograd
    function
37                          # 继承 torch.autograd.Function
    的类有一个静态方法 apply,
38                          # 用于执行这个自定义的函数
39                          # 调用 Exp.apply(x)
    时，实际上是调用了 Exp 类的 forward
    方法，这个方法实现了前向传播逻辑（即 i.exp()）。
40                          # apply 的作用就是把输入张量传给
    forward，同时构建计算图，并将其存储在 ctx 上，以便后续在
    backward 中使用。
41                          # 然后在反向传播阶段（调用
    .backward() 时），backward
    会根据前向传播时保存的信息来计算梯度。
42      print(y)                                    #  y = e^x =
    e^1 = 2.7183
43      # 反向传播
44      y.backward()
45      print(x.grad)                              # 反传梯度,
    x.grad = dy/dx = e^x = e^1  = 2.7183
46
```

```
tensor([2.7183], grad_fn=<ExpBackward>)
tensor([2.7183])
```

例子 47.9

反向传梯度时乘以一个自定义系数

方法:

```
        import torch
        from torch.autograd.function import Function

        class GradCoeff(Function):

                @staticmethod
                # 函数功能：实现前向传播
                def forward(ctx, x, coeff):

                # ============== step1: 函数功能实现
    ==============
                ctx.coeff = coeff   # 将coeff存为ctx的成员变量
                # ============== step1: 函数功能实现
    ==============
                return x

                @staticmethod
                # 函数功能：为梯度乘以一定系数 ctx.coeff
                def backward(ctx, grad_output):
                return ctx.coeff * grad_output, None    #
    backward的输出个数，应与forward的输入个数相同，此处coeff不需要梯度，因此返


        x = torch.tensor([2.], requires_grad=True)
        # 前向传播
        ret = GradCoeff.apply(x, -0.1)                  #
    前向需要同时提供x及coeff，设置coeff为-0.1
        ret = ret ** 3
        print(ret)                                      #
    注意看： ret.grad_fn
        # 反向传播
        ret.backward()
        print(x.grad)
```

```
tensor([8.], grad_fn=<PowBackward0>)
tensor([-1.2000])
# d(x^3)/dx = 3x^2
# x = 2 ⇒ d(2^3)/dx = 3 × 2^2 = 12
# grad = −0.1 × 12 = −1.2
```

例子 47.10

手动实现 2D 卷积

方法:

```
        import torch
        from torch.autograd.function import
    once_differentiable
        import torch.nn.functional as F

        def convolution_backward(grad_out, X, weight):
        """

    计算卷积操作的反向传播，返回两个梯度：输入张量X的梯度和卷积核weight的梯度
        参数：
        - grad_out：前向传播的输出梯度
        - X：输入张量
        - weight：卷积核（权重）

        返回：
        - grad_X：输入张量X的梯度
        - grad_weight：卷积核weight的梯度
        """
        # 计算grad_input，即输入张量的梯度
        grad_input = F.conv2d(X.transpose(0, 1),
    grad_out.transpose(0, 1)).transpose(0, 1)
        # 计算grad_X，即权重的梯度
        grad_X = F.conv_transpose2d(grad_out, weight)
        return grad_X, grad_input

        class MyConv2D(torch.autograd.Function):
        @staticmethod
        def forward(ctx, X, weight):
                """
                前向传播：实现卷积操作
                参数：
                - ctx：上下文对象，保存中间结果以供backward使用
                - X：输入张量
                - weight：卷积核（权重）

                返回：
                - ret：卷积结果
                """
                # 保存输入张量X和卷积核weight，供反向传播使用
                ctx.save_for_backward(X, weight)

                # 使用PyTorch内置的conv2d函数进行卷积计算
                ret = F.conv2d(X, weight)
                return ret

        @staticmethod
```

```
44          def backward(ctx, grad_out):
45                  """
46                  反向传播：计算卷积操作的梯度
47                  参数：
48                  - ctx：上下文对象，获取前向传播保存的变量
49                  - grad_out：前向传播输出的梯度
50
51                  返回：
52                  - 两个梯度：输入张量X的梯度和卷积核weight的梯度
53                  """
54                  # 从上下文中获取保存的X和weight
55                  X, weight = ctx.saved_tensors
56                  # 调用convolution_backward函数，计算并返回梯度
57                  return convolution_backward(grad_out, X,
    weight)
58
59          # 初始化权重和输入张量
60          weight = torch.rand(5, 3, 3, 3, requires_grad=True,
    dtype=torch.double)
61          X = torch.rand(10, 3, 7, 7, requires_grad=True,
    dtype=torch.double)
62
63          # 使用gradcheck来检查自定义操作的梯度计算是否正确
64          #
    gradcheck会通过数值方法检查自定义操作是否实现了正确的梯度
65          print("梯度检查：",
    torch.autograd.gradcheck(MyConv2D.apply, (X, weight)))
66
67          # 调用自定义的卷积操作
68          y = MyConv2D.apply(X, weight)
69
70          # 生成目标标签，计算损失
71          label = torch.randn_like(y)
72          loss = F.mse_loss(y, label)
73
74          # 在反向传播前，打印weight的梯度，初始值为None
75          print("反向传播前，weight.grad：", weight.grad)
76
77          # 反向传播，计算梯度
78          loss.backward()
79
80          # 打印反向传播后的weight梯度
81          print("反向传播后，weight.grad：", weight.grad)
82
```

```
梯度检查： True
反向传播前，weight.grad： None
反向传播后，weight.grad： tensor([[[[1.3423, 1.3445, 1.3271],
```

```
               [1.3008, 1.3262, 1.2493],
               [1.2969, 1.3269, 1.2369]],
      ......
               [[1.2302, 1.2134, 1.2619],
               [1.2397, 1.2048, 1.2447],
               [1.2419, 1.2548, 1.2647]]], dtype=torch.float64)
```

5) autograd 的注意事项

(1) 梯度不会自动清零

在 PyTorch 中，梯度不会自动清零，每次计算完梯度后，需要手动清零，否则会累加。

```
import torch
w = torch.tensor([1.], requires_grad=True)
x = torch.tensor([2.], requires_grad=True)

for i in range(4):
    a = torch.add(w, x)
    b = torch.add(w, 1)
    y = torch.mul(a, b)

    y.backward()
    print(w.grad)  # 梯度不会自动清零，数据会累加， 通常需要采用 optimizer.zero_grad() 完成对参数的梯度清零

#     w.grad.zero_()
```

```
tensor([5.])
tensor([10.])
tensor([15.])
tensor([20.])
```

使用 `w.grad.zero_()` 可以手动将梯度清零，如下所示：

```
import torch
w = torch.tensor([1.], requires_grad=True)
x = torch.tensor([2.], requires_grad=True)

```

```
for i in range(4):
    a = torch.add(w, x)
    b = torch.add(w, 1)
    y = torch.mul(a, b)

    y.backward()
    print(w.grad)  # 输出累加的梯度

    # 手动将梯度清零
    w.grad.zero_()
```

```
tensor([5.])
tensor([5.])
tensor([5.])
tensor([5.])
```

(2) 叶子结点不可执行 in-place

叶子节点不可以执行 in-place 操作 (`torch.add_`、`torch.mul_`等)，否则会报错，如下所示：

```
import torch

# 依赖于叶子节点的节点，requires_grad 默认为 True
w = torch.tensor([1.], requires_grad=True)
x = torch.tensor([2.] ,requires_grad=True)
a = torch.add(w, x)
b = torch.add(w, 1)

y = torch.mul(a, b)
w.add_(1.0)
# RuntimeError: a leaf Variable that requires grad is being used in an
    in-place operation.
```

(3) detach 切断一些分支的反向传播

训练网络时，有时候我们希望保持一部分的网络参数不变，只对其中一部分的参数进行调整，只训练部分分支网络，不让其梯度对主网络的梯度造成影响，这时候可以使用`detach`。函数来切断一些分支的反向传播。

`detach()`将张量从创建它的计算图 (Graph) 中分离，把它作为叶子节点，其中参数`grad_fn = None`且`requires_grad = False`。

```
import torch
```

```
2
3  # x = torch.ones(2, requires_grad=True)
4  x = torch.tensor([2., 2.], requires_grad=True)
5  # 对 x 每个元素*2
6  print("x:", x)
7  y = 2*x**3 + 4
8  print("y:", y)
9
10 ## 对分离变量 y，生成一个新的变量 c
11 c = y.detach()
12 print("c:", c)
13 z = c*x # 将此刻的 c 作为常数，不再参与梯度计算
14 print("z:", z)
15 print("z.sum():", z.sum())
16 z.sum().backward()
17
18 print("-------------------------")
19 # 对 c 进行反向传播
20 print("x.grad:", x.grad)
21 print("c.grad_fn:", c.grad_fn)
22 print("c.requires_grad:", c.requires_grad)
23
24 print("-------------------------")
25 x.grad.zero_()
26 y.sum().backward()
27 print("x.grad:", x.grad)
28 print("y.grad_fn:", y.grad_fn)
29 print("y.requires_grad:", y.requires_grad)
```

```
x: tensor([2., 2.], requires_grad=True)
y: tensor([20., 20.], grad_fn=<AddBackward0>)
c: tensor([20., 20.])
z: tensor([40., 40.], grad_fn=<MulBackward0>)
z.sum(): tensor(80., grad_fn=<SumBackward0>)
-------------------------
x.grad: tensor([20., 20.])
c.grad_fn: None
c.requires_grad: False
-------------------------
x.grad: tensor([24., 24.])
y.grad_fn: <AddBackward0 object at 0x7f598a399870>
y.requires_grad: True
```

四、 Tensor 与 Numpy 比较

PyTorch 与 NumPy 有很多类似的地方，并且有很多相同的操作函数名称，所以有时很容易混淆，下面我们对一些主要的区别进行汇总，具体可参考表47.9。

操作类别	NumPy	PyTorch
数据类型	np.ndarray	torch.Tensor
	np.float32	torch.float32; torch.float
	np.float64	torch.float64; torch.double
	np.int64	torch.int64; torch.long
从已有数据构建	np.array([3.2, 4.3], dtype=np.float16)	torch.tensor([3.2, 4.3], dtype=torch.float16)
	x.copy()	x.clone()
	np.concatenate	torch.cat
线性代数	np.dot	torch.mm
属性	x.ndim	x.dim()
	x.size	x.nelement()
形状操作	x.reshape	x.reshape; x.view
	x.flatten	x.view(-1)
类型转换	np.floor(x)	torch.floor(x); x.floor()
比较	np.less	x.lt
	np.less_equal/np.greater	x.le/x.gt
	np.greater_equal/np.equal/np.not_equal	x.ge/x.eq/x.ne
随机种子	np.random.seed	torch.manual_seed

表 47.9 PyTorch 与 NumPy 函数对照表

例子 47.11

实现一个有关回归的机器学习任务

方法:

1) 首先，给出一个数组 x, 然后基于表达式 $y = 3x^2 + 2$, 加上一些噪声数据到达另一组数据 y。
2) 然后，构建一个机器学习模型，学习表达式 $y = wx^2 + b$ 的两个参数 w、b。利用数组 x、y 的数据训练模型。
3) 最后，采用梯度下降法，通过多次迭代学习到 w、b 的值。

1) 使用 NumPy 实现

```
# 导入所需的库
import numpy as np
from matplotlib import pyplot as plt

# 生成输入数据 x 及目标数据 y
# 设置随机种子，生成同一份数据，以便结果复现
np.random.seed(985)
x = np.linspace(-1, 1, 100).reshape(100, 1)
```

```
    y = 3 * np.power(x, 2) + 2 + 0.2 *
    np.random.rand(x.size).reshape(100, 1)
    plt.title("Generated data")
    plt.scatter(x, y)
    plt.show()

    ################################
    # 回归模型
    # 1.初始化参数
    w = np.random.rand(1, 1)
    b = np.random.rand(1, 1)

    # 2.训练模型
    lr = 0.001  # 学习率
    epochs = 1000  # 迭代次数
    for i in range(epochs):
        # 1)前向传播
        y_pred = np.power(x, 2).dot(w) + b
        # 2)计算损失
        loss = 0.5 * (y_pred - y) ** 2
        loss = loss.sum()
        # 3)计算梯度
        grad_w = np.sum((y_pred - y) * np.power(x, 2))
        grad_b = (y_pred - y).sum()
        # 4)更新参数,使用梯度下降法使得损失函数最小
        w -= lr * grad_w
        b -= lr * grad_b
        if i % 100 == 0:
            print(f"Epoch {i}, Loss {loss}")

    ################################
    # 可视化结果
    plt.title("Regression result")
    plt.plot(x, y_pred, 'r-', label='predict')
    plt.scatter(x, y, color='blue', marker='o', label='true')
    plt.show()

```

```
Epoch 0, Loss 346.44044950441815
Epoch 100, Loss 4.032199214623452
Epoch 200, Loss 0.9025039659613794
Epoch 300, Loss 0.30119252296143295
Epoch 400, Loss 0.18566196029014184
Epoch 500, Loss 0.1634649589416306
Epoch 600, Loss 0.1592002271038078
Epoch 700, Loss 0.15838083995493227
Epoch 800, Loss 0.15822341029105794
Epoch 900, Loss 0.1581931631739274
```

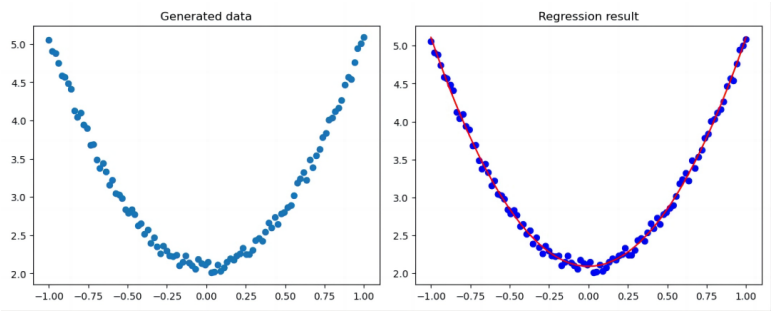

图 47.12 Numpy 版实现回归问题

2) 使用 Tensor 和 autograd 实现

```
# 导入所需的库
import numpy as np
from matplotlib import pyplot as plt

# 生成输入数据 x 及目标数据 y
# 设置随机种子，生成同一份数据，以便结果复现
torch.manual_seed(985) # np.random.seed(985)
dtype = torch.float
# x = np.linspace(-1, 1, 100).reshape(100, 1)
x = torch.unsqueeze(torch.linspace(-1, 1, 100), dim=1)
# 在第1维增加一个维度
# 为了后面的矩阵相乘y_pred = x.pow(2).mm(w)，需要将 x
转换为矩阵形式
#y = 3 * np.power(x, 2) + 2 + 0.2 *
np.random.rand(x.size).reshape(100, 1)
y = 3 * x.pow(2) + 2 + 0.2 * torch.rand(x.size())
plt.title("Generated data")
plt.scatter(x.numpy(), y.numpy()) #
画图时需要将张量转换为 numpy 数组
plt.show()

##################################
# 回归模型
# 1.初始化参数
# w = np.random.rand(1, 1)
w = torch.rand(1, 1, dtype=dtype, requires_grad=True)
# b = np.random.rand(1, 1)
b = torch.rand(1, 1, dtype=dtype, requires_grad=True)

# 2.训练模型
```

```
    lr = 0.001  # 学习率
    epochs = 1000  # 迭代次数
    for i in range(epochs):
        # 1)前向传播
        # y_pred = np.power(x, 2).dot(w) + b
        y_pred = x.pow(2).mm(w) + b # .mm()表示矩阵相乘
        # 2)计算损失
        loss = 0.5 * (y_pred - y) ** 2
        loss = loss.sum()
        # 3)自动计算梯度
        # grad_w = np.sum((y_pred - y) * np.power(x, 2))
        # grad_b = (y_pred - y).sum()
        loss.backward()
        # 4)更新参数,使用梯度下降法使得损失函数最小
        # w -= lr * grad_w
        # b -= lr * grad_b
        with torch.no_grad(): # 不记录梯度
            w -= lr * w.grad
            b -= lr * b.grad
            # 梯度清零
            w.grad.zero_()
            b.grad.zero_()

        # if i % 100 == 0:
        #     print(f"Epoch {i}, Loss {loss}")
        if i % 100 == 0:
            print(f"Epoch {i}, Loss {loss.item()}")

    ################################
    # 可视化结果
    plt.title("Regression result")
    plt.plot(x.numpy(), y_pred.detach().numpy(), 'r-',
    label='predict', linewidth=4) # detach() 从计算图中分离
    plt.scatter(x.numpy(), y.numpy(), color='blue',
    marker='o', label='true')
    plt.legend()
    plt.show()

```

```
Epoch 0, Loss 431.41571044921875
Epoch 100, Loss 2.9688427448272705
Epoch 200, Loss 0.6887450218200684
Epoch 300, Loss 0.2506664991378784
Epoch 400, Loss 0.16649889945983887
Epoch 500, Loss 0.1503276824951172
Epoch 600, Loss 0.1472204625606537
Epoch 700, Loss 0.14662344753742218
Epoch 800, Loss 0.1465088278055191
```

```
Epoch 900, Loss 0.14648674428462982
```

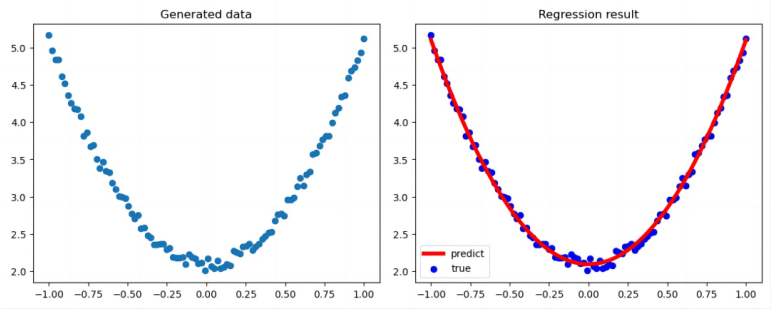

图 47.13 使用 Tensor 和 autograd 实现

如果 1 元 2 次函数使用两个参数 w_1、w_2 和 b，则代码如下：

```
# 导入所需的库
import torch
from matplotlib import pyplot as plt

# 生成输入数据 x 及目标数据 y
# 设置随机种子，生成同一份数据，以便结果复现
torch.manual_seed(985)  # 设置随机种子

# 使用 PyTorch 生成训练数据
x = torch.linspace(-1, 1, 100).reshape(100, 1)  # 生成 100 个数据点
# y = 3 * x.pow(2) + 2 + 0.2 * torch.rand(x.size())  # 生成目标数据 y
y = 3 * x**2 + 2 + 0.2 * torch.randn(100, 1)  # 生成目标数据 y

plt.title("Generated data")
plt.scatter(x.numpy(), y.numpy())  # 画图时需要将张量转换为 numpy 数组
plt.show()

#################################
# 回归模型
# 1.初始化参数
# w1 x^2 + w2 x + b
w1 = torch.rand((1, 1), dtype=torch.float, requires_grad=True)  # 初始化权重，启用梯度
w2 = torch.rand((1, 1), dtype=torch.float, requires_grad=True)  # 初始化权重，启用梯度
b = torch.rand((1, 1), dtype=torch.float,
```

```
    requires_grad=True)  # 初始化偏置，启用梯度
25
26  # 2.训练模型
27  lr = 0.01  # 学习率
28  epochs = 1000  # 迭代次数
29  for i in range(epochs):
30      # 1)前向传播
31      y_pred = x.pow(2).mm(w1) + x.mm(w2) + b  # 使用
    PyTorch 矩阵乘法
32
33      # 2)计算损失
34      loss = 0.5 * (y_pred - y).pow(2).sum()  # 使用
    PyTorch 的张量操作计算损失
35      # loss = (0.5 * (y_pred - y) ** 2).mean()
36
37      # 3)反向传播
38      loss.backward()  # 自动计算梯度
39
40      # 4)更新参数,使用梯度下降法使得损失函数最小
41      with torch.no_grad():
42          w1 -= lr * w1.grad  # 更新权重
43          w2 -= lr * w2.grad
44          b -= lr * b.grad  # 更新偏置
45          # 清零梯度
46          w1.grad.zero_()
47          w2.grad.zero_()
48          b.grad.zero_()
49
50      # 每100次迭代输出一次损失值
51      if i % 100 == 0:
52          print("epoch: {}, loss: {}".format(i,
    loss.item()))
53          plt.scatter(x.numpy(), y.numpy())
54          plt.plot(x.numpy(), y_pred.detach().numpy(),
    'r-', label='predict')
55          plt.title("Regression result")
56          plt.legend()
57          plt.show()
58
59  print("w1: ", w1, "w2: ", w2, "b: ", b)
60
```

```
epoch: 0, loss: 386.6900939941406
epoch: 100, loss: 1.4194867610931396
epoch: 200, loss: 1.4194862842559814
epoch: 300, loss: 1.4194862842559814
epoch: 400, loss: 1.4194862842559814
epoch: 500, loss: 1.4194862842559814
```

```
epoch: 600, loss: 1.4194862842559814
epoch: 700, loss: 1.4194862842559814
epoch: 800, loss: 1.4194862842559814
epoch: 900, loss: 1.4194862842559814
w1:  tensor([[3.0083]], requires_grad=True)
w2:  tensor([[-0.0228]], requires_grad=True)
b:  tensor([[1.9803]], requires_grad=True)
```

3) 使用优化器及自动微分实现

```python
# 导入所需的库
import numpy as np
from matplotlib import pyplot as plt

# 生成输入数据 x 及目标数据 y
# 设置随机种子，生成同一份数据，以便结果复现
torch.manual_seed(985) # np.random.seed(985)
dtype = torch.float
# x = np.linspace(-1, 1, 100).reshape(100, 1)
x = torch.unsqueeze(torch.linspace(-1, 1, 100), dim=1) # 在第1维增加一个维度
#y = 3 * np.power(x, 2) + 2 + 0.2 * np.random.rand(x.size).reshape(100, 1)
y = 3 * x.pow(2) + 2 + 0.2 * torch.rand(x.size())
plt.title("Generated data")
plt.scatter(x.numpy(), y.numpy()) # 画图时需要将张量转换为 numpy 数组
plt.show()

##################################
# 回归模型
# 1.初始化参数
# w = np.random.rand(1, 1)
w = torch.rand(1, 1, dtype=dtype, requires_grad=True)
# b = np.random.rand(1, 1)
b = torch.rand(1, 1, dtype=dtype, requires_grad=True)

# 2.定义损失函数和优化器
loss_func = torch.nn.MSELoss()
optimizer = torch.optim.SGD([w, b], lr=0.001)

# 3.训练模型
# lr = 0.001  # 学习率
epochs = 1000  # 迭代次数
for i in range(epochs):
# 1)前向传播
# y_pred = np.power(x, 2).dot(w) + b
```

```
    y_pred = x.pow(2).mm(w) + b # .mm()表示矩阵相乘
    loss = loss_func(y_pred, y)
    # 计算损失
    # loss = 0.5 * (y_pred - y) ** 2
    # loss = loss.sum()
    # 2)自动计算梯度
    # grad_w = np.sum((y_pred - y) * np.power(x, 2))
    # grad_b = (y_pred - y).sum()
    loss.backward()
    # 3)更新参数,使用梯度下降法使得损失函数最小
    # w -= lr * grad_w
    # b -= lr * grad_b
    # with torch.no_grad(): # 不记录梯度
    #     w -= lr * w.grad
    #     b -= lr * b.grad
    #     # 梯度清零
    #     w.grad.zero_()
    #     b.grad.zero_()
    optimizer.step()
    #
    因通过autograd计算的梯度会累加到grad中，故每次循环需把梯度清零
    optimizer.zero_grad()


    # if i % 100 == 0:
    #     print(f"Epoch {i}, Loss {loss}")
    if i % 100 == 0:
            print(f"Epoch {i}, Loss {loss.item()}")

    ################################
    # 可视化结果
    plt.title("Regression result")
    plt.plot(x.numpy(), y_pred.detach().numpy(), 'r-',
    label='predict', linewidth=4) # detach() 从计算图中分离
    plt.scatter(x.numpy(), y.numpy(), color='blue',
    marker='o', label='true')
    plt.legend()
    plt.show()

```

```
Epoch 0, Loss 8.628314018249512
Epoch 100, Loss 5.594874858856201
Epoch 200, Loss 3.659278631210327
Epoch 300, Loss 2.423168182373047
Epoch 400, Loss 1.632761836051941
Epoch 500, Loss 1.1263880729675293
Epoch 600, Loss 0.8010491132736206
Epoch 700, Loss 0.5911281108856201
```

```
Epoch 800, Loss 0.4548189640045166
Epoch 900, Loss 0.36548832058906555
```

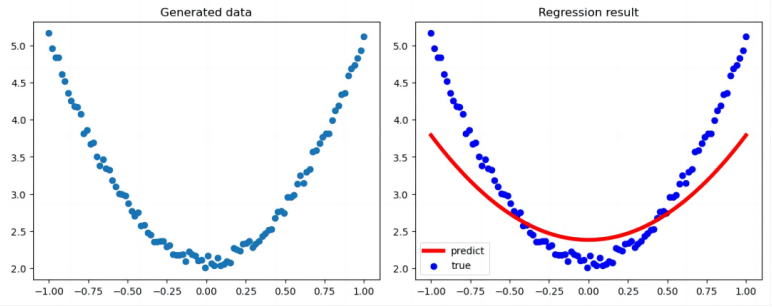

图 47.14 使用优化器及自动微分实现

4) 把数据集转换为带批量处理功能的迭代器把数据集转换为带批量处理功能的迭代器，这样训练时就可进行批量处理。如果数据量比较大，采用批量处理可提升训练模型的效率及性能。

```
# 导入所需的库
import numpy as np
from matplotlib import pyplot as plt
import torch

# 生成输入数据 x 及目标数据 y
# 设置随机种子，生成同一份数据，以便结果复现
# np.random.seed(985)
# x = np.linspace(-1, 1, 100).reshape(100, 1)
# y = 3 * np.power(x, 2) + 2 + 0.2 *
np.random.rand(x.size).reshape(100, 1)
# plt.title("Generated data")
# plt.scatter(x, y)
# plt.show()


##################################
# 回归模型
# 1.初始化参数
# w = np.random.rand(1, 1)
# b = np.random.rand(1, 1)
def deta_iter(features, labels, batch_size=8):
num_examples = len(features)
indices = list(range(num_examples))
# 随机读取样本，打乱顺序
np.random.shuffle(indices)
```

```
    for i in range(0, num_examples, batch_size):
            indexs = torch.LongTensor(indices[i: min(i +
batch_size, num_examples)]) # 最后一次可能不足一个batch
            yield features.index_select(0, indexs),
labels.index_select(0, indexs)

# 2.定义损失函数和优化器
loss_func = torch.nn.MSELoss()
optimizer = torch.optim.SGD([w, b], lr=0.001)

# 3.训练模型
lr = 0.001  # 学习率
epochs = 1000  # 迭代次数
for i in range(epochs):
for features, labels in deta_iter(x, y, 10):
        # 1)前向传播
        y_pred = features.pow(2).mm(w) + b
        loss = loss_func(y_pred, labels)

        # 2)自动计算梯度
        loss.backward()

        # 3)更新参数,使用梯度下降法使得损失函数最小
        optimizer.step()
        optimizer.zero_grad()

if i % 100 == 0:
        print(f"Epoch {i}, Loss {loss.item()}")
print("w:", w, "b:", b)

#################################
# 可视化结果
y_pred = x.pow(2).mm(w).detach().numpy() +
b.detach().numpy()
plt.title("Regression result")
plt.plot(x.numpy(), y_pred, 'r-', label='predict',
linewidth=4)
plt.scatter(x.numpy(), y.numpy(), color='blue',
marker='o', label='true')
plt.legend()
plt.show()

```

```
poch 0, Loss 0.004008688032627106
Epoch 100, Loss 0.002615853678435087
Epoch 200, Loss 0.002760492265224457
Epoch 300, Loss 0.001538299024105072
Epoch 400, Loss 0.004123619291931391
```

```
Epoch 500, Loss 0.003147989511489868
Epoch 600, Loss 0.0033883980941027403
Epoch 700, Loss 0.0021358653903007507
Epoch 800, Loss 0.004264053888618946
Epoch 900, Loss 0.002425102051347494

w: tensor([[3.0139]], requires_grad=True) b: tensor([[2.0920]],
requires_grad=True)
```

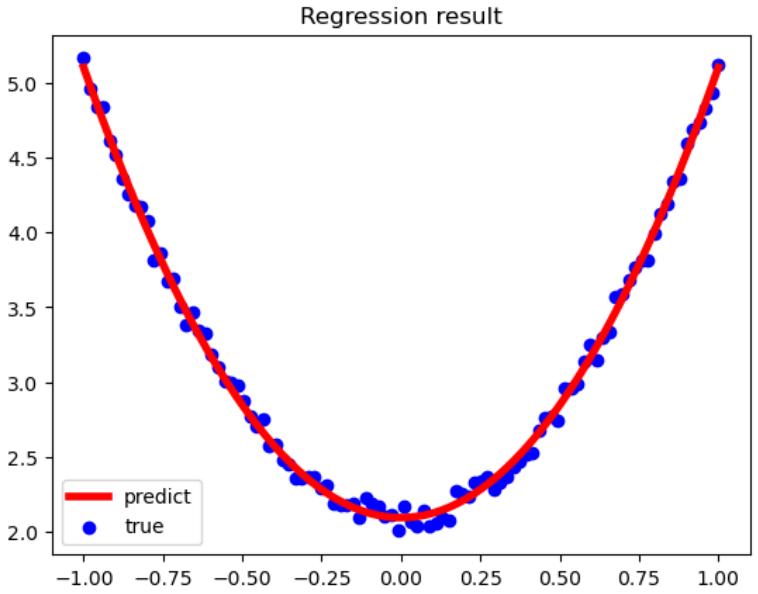

图 47.15 把数据集转换为带批量处理功能的迭代器

第四十八章 PyTorch 数据处理

一、 数据处理工具箱概述

PyTorch 提供了一系列数据处理工具箱，主要包括以下几个部分。

1) torch.utils.data：数据加载工具箱，提供了 Dataset、DataLoader 等工具。

2) torchvision：图形处理工具箱，提供了常用的数据集加载、数据预处理、图像处理等工具。

3) torchaudio：音频处理工具箱，提供了音频数据加载、音频数据处理等工具。

4) torchtext：文本处理工具箱，提供了文本数据加载、文本数据处理等工具。

二、 utils.data

是 torch.utils.data 工具包，它包括以下 4 个类。

1) Dataset：一个抽象类，其他数据集需要继承这个类，并且覆写其中的两个方法 (`__getitem__`、`__len__`)。

2) (**数据模块核心**)DataLoader：定义一个新的迭代器，实现批量 (batch) 读取，打乱数据 (shuffle) 并提供并行加速等功能。因为`__getitem__`**一次只能**获取一个样本，所以通过`torch.utils.data.DataLoader`来实现批量读取。

3) `random_split`：把数据集随机拆分为给定长度的非重叠新数据集。

4) `*Sampler`：多种采样函数。

dataset 负责与磁盘打交道，将磁盘上的数据读取并预处理好，提供给 DataLoader，而 DataLoader 只需要关心如何组装成批数据，以及如何采样。采样的体现是出现在传入 getitem 函数的索引，这里采样的规则可以通过 sampler 由用户自定义，可以方便地实现均衡采样、随机采样、有偏采样、渐进式采样等。

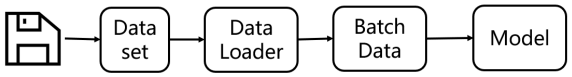

图 48.1 Dataset 与 DataLoader 的关系

1) Dataset 数据交互模块

pytorch 提供的 torch.utils.data.Dataset 类是一个抽象基类 An abstract class representing a Dataset，供用户继承，编写自己的 dataset，实现对数据的读取。

在 Dataset 类的编写中必须要实现的两个函数是

1) `__getitem__`：通过给定索引 (index) 获取数据、标签或一个样本。然后实现从磁盘中读取数据，并进行预处理（包括 online 的数据增强），然后返回一个样本的数据。

   数据可以是包括模型需要的输入、标签，也可以是其他元信息，例如图片的路径。`__getitem__`返回的数据会在 dataloader 中组装成一个 batch。即，通常情况下是在 dataloader 中调用 Dataset 的`__getitem__`函数获取一个样本。

2) `__len__`：返回数据集的长度。

```python
# 导入需要的库
import torch
from torch.utils import data
import numpy as np

# 定义获取数据集的类
class TestDataset(data.Dataset): # 继承 Dataset, 自定义数据集类
    def __init__(self):
        self.Data = np.asarray([[0, 1], [2, 3], [2, 1], [3, 4], [4, 5]])
        self.Label = np.asarray([0,1,3,5,2])

    # __getitem__方法用来获取数据集中第index个样本
    def __getitem__(self, index):
        # 把numpy转换为Tensor
        txt = torch.from_numpy(self.Data[index])
        label = torch.tensor(self.Label[index])
        return txt, label

    def __len__(self):
        return len(self.Data)

# 获取数据集中的数据
Test = TestDataset()
```

```
print(Test[2]) # 相当于调用 __getitem__(2) 方法
print(Test.__getitem__(2))
print(Test.__len__())
print(Test[0]) # 相当于调用 __getitem__(0) 方法
```

```
(tensor([2, 1]), tensor(3))
(tensor([2, 1]), tensor(3))  # 两者等价
5
(tensor([0, 1]), tensor(0))
```

以上数据以元组格式返回，每次只返回一个样本。

2) DataLoader 数据加载模块

实际上，Dataset 只负责数据的抽取，一次调用`__getitem__`只返回一个样本。如果希望批量处理，同时进行数据打乱和并行加速等操作，可选择`DataLoader`。DataLoader 的格式为：

```
data.DataLoader(
    dataset,        # 数据集
    batch_size=1,   # 批量大小
    shuffle=False,  # 是否打乱数据
    sampler=None,   # 样本抽样
    batch_sampler=None,  # 批量抽样
    num_workers=0,  # 并行线程数,0代表不使用多进程
    collate_fn=None,    #
    : 如何将多个样本数据拼接成一个批量，一般使用默认的拼接方式即可
    pin_memory=False,   # 是否将数据保存在锁页内存(pin_memory)区，
                        # pin_memory中的数据转到GPU会快一些。
    drop_last=False,    # 是否丢弃最后一个不完整的 batch
                        # dataset中的数据个数可能不是batch_size的整数倍，
                        #
    将drop_last设置为True时会将多出来且不足一个批量的数据丢弃。
    timeout=0,          # 加载数据超时时间
    worker_init_fn=None,    # 每个 worker 初始化函数
    multiprocessing_context=None,  # 多进程上下文
    generator=None, *, # 生成器
    prefetch_factor=2,  # 预取因子
    persistent_workers=False # 是否使用持久化工作进程
)
```

```
# 导入需要的库
import torch
from torch.utils import data
import numpy as np

```

```python
# 定义获取数据集的类
class TestDataset(data.Dataset): # 继承 Dataset, 自定义数据集类
    def __init__(self):
        self.Data = np.asarray([[0, 1], [2, 3], [2, 1], [3, 4], [4, 5]])
        self.Label = np.asarray([0, 1, 0, 1, 2])

    def __getitem__(self, index):
        # 把numpy转换为Tensor
        txt = torch.from_numpy(self.Data[index])
        label = torch.tensor(self.Label[index])
        return txt, label

    def __len__(self):
        return len(self.Data)

# 获取数据集中的数据
Test = TestDataset()
######################
# 1)单个获取
# print(Test[2]) # 相当于调用 __getitem__(2) 方法
# print(Test.__len__()) # 相当于调用 __len__() 方法
# print(Test[0]) # 相当于调用 __getitem__(0) 方法

######################
# 2)使用DataLoader加载数据
test_loader = data.DataLoader(Test, batch_size=3, shuffle=False, num_workers=2)
for i, data in enumerate(test_loader):
    inputs, labels = data
    print(f"Batch {i}:")
    print(f"数据: {inputs}")
    print(f"标签: {labels}\n")

print("-------------------------")
#
    批量读取。我们可以像使用迭代器一样使用它，如对它进行循环操作。不过它不是迭代器，
dataiter = iter(test_loader)
inputs, labels = next(dataiter)
print(f"数据: {inputs}")
print(f"标签: {labels}")
```

批量读取我们可以像使用迭代器一样使用它，如对它进行循环操作。不过它不是迭代器，可以通过iter命令转换为迭代器。

```
Batch 0:
数据: tensor([[0, 1],
        [2, 3],
```

```
        [2, 1]])
标签: tensor([0, 1, 0])

Batch 1:
数据: tensor([[3, 4],
        [4, 5]])
标签: tensor([1, 2])

-------------------------
数据: tensor([[0, 1],
        [2, 3],
        [2, 1]])
标签: tensor([0, 1, 0])
```

三、 torchvision 图形处理工具箱

torchvision 包括 4 个类，各类的主要功能如下。

1) datasets:提供常用的数据集加载,设计上都是继承`torch.utils.data.Dataset`, 主要包括 MNIST、CIFAR10/100. ImageNet. COCO 等。

2) models: 提供深度学习中各种经典的网络结构以及训练好的模型（如果要选择训练好的模型，则设置`pretrained = True`)，包括 AlexNet、VGG 系列、ResNet 系列 Inception 系列等。

3) transforms: 常用的数据预处理操作，主要包括对`Tensoi`及`PIL Image`对象的操作。

4) utils: 含两个函数，一个是`make_grid`, 它能将多张图像拼接在一个网格中； 另一个是`savejmg`, 它能将`Tensor`保存为图像。

1) torchvision 安装

torchvision 是 PyTorch 的一个视觉处理工具包， 独立于 PyTorch, 需要另外安装。

```
pip install torchvision
# 或者
conda install torchvision
```

2) torchvision.datasets

3) torchvision.models

4) torchvision.transforms

transforms 提供了对 PIL Image 对象和 Tensor 对象的常用操作。

对 PIL Image 的操作包括：

1) `Scale/Resize`：调整图片尺寸，长宽比保持不变。

2) `CenterCrop`：从中心开始裁剪图片。

3) `RandomCrop`：随机裁剪。

4) `RandomSizeCrop`：随机尺寸裁剪。

5) `Pad`: 填充。

6) `ToTensor`：将 PIL Image 对象转为 Tensor，会自动将 [0,255] 归一化至 [0,1]。

7) `RandomHorizontalFlip`：随机水平翻转, 概率为 0.5。

8) `RandomVerticalFlip`：随机垂直翻转, 概率为 0.5。

9) `ColorJitter`：调整亮度、对比度、饱和度和色相。

对 Tensor 的操作包括：

1) `Normalize`：标准化。

2) `ToPILImage`：将 Tensor 转为 PIL Image 对象。

如果要对数据集进行多个操作，可通过 Compose 将这些操作像管道一样拼接起来，类似于`nn.Sequential`

```
# 例如
transforms.Compose([
    # 从图片中间裁剪出224*224的图片
    transforms.RandomResizedCrop(224),
    # 随机水平翻转，概率为0.5
    transforms.RandomHorizontalFlip(),
    # 将图片像素值归一化
    transforms.ToTensor(),
    # 标准化
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])
```

5) torchvision.utils

四、 可视化工具

1) TensorBoard

TensorBoard 是 Google TensorFlow 的可视化工具，可以记录训练数据、评估数据、网络结构、图像等，并且可以在 Web 上展示，对于观察神经网络训

练的过程非常有帮助。PyTorch 支持`tensorboard_logger`、`visdom`等可视化工具。

TensorBoard 功能很强大，支持 scalar、image、figureA histogram、audio、text、graph、onnx_graph、embedding、pr_curve、videosummaries 等可视化方式。

(1) 安装

```
pip install tensorboard
```

(2) 使用

执行下面命令

```
tensorboard --logdir=<event文件目录>
```

复制网址http://localhost:6006/ 到浏览器中进行打开

1) 如果 conda 环境下没有安装 tensorboard，可以通过以下命令启动 tensorboard。

```
tensorboard --logdir=/home/tony/Pratice/Python/Pytorch/NFEM
--port 6006 --load_fast=false
```

2) 在 tensorboard 启动的过程中，可能发生的问题：

6006 端口被占用：

```
port 6006 was already in use
E0117 15:58:38.631224 MainThread program.py:260] TensorBoard
attempted to bind to port 6006, but it was already in use
TensorBoard attempted to bind to port 6006, but it was already
in use
```

解决方法：修改端口为 6007

```
tensorboard --logdir=your_dir --port=6007
```

```
1 # 可视化神经网络
2 # 导入需要的模块。
```

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
from torch.utils.tensorboard import SummaryWriter

import os
log_dir = os.getcwd()  # 或者指定一个相对于当前工作目录的子目录，如 'logs'

# 1.构建神经网络。
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(1, 10, kernel_size=5)
        self.conv2 = nn.Conv2d(10, 20, kernel_size=5)
        self.conv2_drop = nn.Dropout2d()
        self.fc1 = nn.Linear(320, 50)
        self.fc2 = nn.Linear(50, 10)
        self.bn = nn.BatchNorm2d(20)

    def forward(self, x):
        x = F.max_pool2d(self.conv1(x), 2)
        x = F.relu(x) + F.relu(-x)  # 这里是双边ReLU，通常不常见，可能是个实验性设计
        x = F.relu(F.max_pool2d(self.conv2_drop(self.conv2(x)), 2))
        x = self.bn(x)
        x = x.view(-1, 320)
        x = F.relu(self.fc1(x))
        x = F.dropout(x, training=self.training)
        x = self.fc2(x)
        x = F.softmax(x, dim=1)
        return x

# 2.把模型保存为图像
# 1) 定义输入。
input_tensor = torch.rand(1, 1, 28, 28)
# 2) 实例化模型。
model = Net()
# 3) 把模型保存为图像。
with SummaryWriter(log_dir=log_dir, comment='Net') as w:
    w.add_graph(model, (input_tensor, ))
```

```
# 可视化损失函数
```

```
2  #
       可视化损失值时需要用到add_scalar函数，这里利用一层全连接神经网络，训练一元二次函数的参数。
3  import torch
4  import torch.nn as nn
5  from torch.utils.tensorboard import SummaryWriter
6  import numpy as np
7
8  input_size = 1
9  output_size = 1
10 num_epoches = 60
11 learning_rate = 0.01
12
13 dtype = torch.FloatTensor
14 writer = SummaryWriter(log_dir='logs',comment='Linear')
15 np.random.seed(100)
16 x_train = np.linspace(-1, 1, 100).reshape(100,1)
17 y_train = 3*np.power(x_train, 2) +2+
       0.2*np.random.rand(x_train.size).reshape(100,1)
18
19 model = nn.Linear(input_size, output_size)
20
21 criterion = nn.MSELoss()
22 optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
23
24 for epoch in range(num_epoches):
25     inputs = torch.from_numpy(x_train).type(dtype)
26     targets = torch.from_numpy(y_train).type(dtype)
27
28     output = model(inputs)
29     loss = criterion(output, targets)
30
31     optimizer.zero_grad()
32     loss.backward()
33     optimizer.step()
34     # 保存loss的数据与epoch数值
35     writer.add_scalar('训练损失值', loss, epoch)
```

第四十九章 PyTorch 神经网络工具

利用这个工具箱设计一个神经网络就像搭积木一样，可以极大简化构建模型的任务。

一、 神经网络核心组件

神经网络核心部分或组件不多，我们确定这些组件后，这个就基本确定了神经网络。这些核心组件分析如下

1) 层：神经网络的基本结构，将输入张量转换为输出张量。
2) 模型：由层构成的网络。
3) 损失函数：参数学习的目标函数，通过最小化损失函数来学习各种参数。
4) 优化器：如在使损失值最小时，就会涉及优化器。

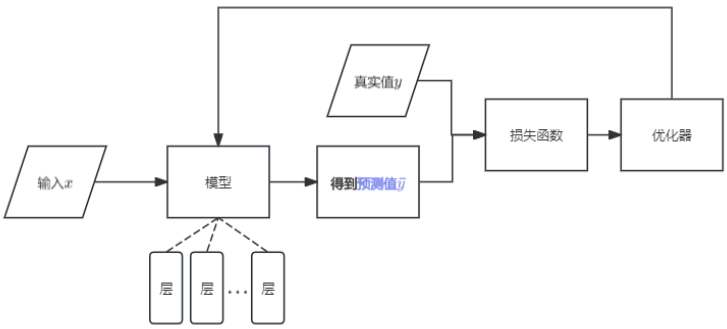

图 49.1 神经网络关键组件及相互关系示意图

如图49.1所示，多个层链接在一起构成一个模型或网络，输入数据通过这个模型转换为预测值。预测值与真实值共同构成损失函数的输入，损失函数输出损失值（损失值可以是距离、概率值等），该损失值用于衡量预测值与

目标结果的匹配或相似程度。优化器利用损失值更新权重参数，目标是使损失值越来越小。这是一个循环过程，当损失值达到一个阈值或循环次数到达指定次数时，循环结束。

二、 构建神经网络的主要工具

1) nn.Module

使用 autograd 及 Tensor 实现机器学习实例时，需要做不少设置，如将叶子节点的参数`requires_grad`设置为`True`，然后调用`backward`函数从 grad 属性中提取梯度。对于大规模的网络，autograd 太过于底层和烦琐。为了简单、有效地解决这个问题，`nn`是一个有效工具。它是专门为深度学习设计的一个模块，而`nn.Module`是`nn`的一个核心数据结构。`nn.Module`可以是神经网络的某个层，也可以是包含多层的神经网络。在实际使用中，最常见的做法是继承`nn.Module`生成自己的网络层，

`nn.Module`是 PyTorch 中所有神经网络模块的基类，所有神经网络模块都应该继承自`nn.Module`。

2) nn.functional

`nn.functional`是一个函数模块，包含了神经网络中的一些常用函数，如激活函数、损失函数、池化函数等。

(1) nn.Module 和 nn.functional 的区别

`nn`中的层，

1) 继承了`nn.Module`，名称一般为`nn.Xxx`(第一个是大写)

   如`nn.Linear`、`nn.Conv2d`、`nn.CrossEntropyLoss`等。

2) `nn.functional`中的函数，名称一般为`nn.functional.xxx`(第一个是小写)，

   如`nn.functional.linear`、`nn.functional.conv2d`、`nn.functional.cross_entropy`等。

从功能方面来说两者相当，基于`nn.Module`实现的层，也可以基于`nn.functional`实现，反之亦然。从性能方面来说两者也没有太大差异。不过在具体使用时，两者还是有区别的

1) `nn.Xxx`继承于`nn.Module`，需要先实例化并传入参数，然后以函数调用的方式调用实例化的对象并传入输入数据。它能够很好地与`nn.Sequential`结合使用

   而`nn.functional.xxx`是一个函数。它无法保存参数，因此无法使用`nn.Sequential`。

2) nn.Xxx中的层可以自动定义、保存参数，如nn.Linear中的权重和偏置，
而nn.functional.xxx中的函数不具备保存参数的功能，不仅需要自己定义参数 (权重 (weights) 和偏置 (bias) 等)，还需要手动传入参数。

3) dropout操作在训练和测试阶段是不同的，使用nn.Xxx方式定义的dropout层，在调用model.eval()后，自动实现状态的转换，
而使用nn.functional.xxx方式定义的dropout函数，需要手动设置状态。

总的来说，两种函数的功能都是相同的，但 PyTorch 官方推荐：

1) 具有学习参数的（例如，conv2d、linear、batchjnorm、dropout 等）情况采用nn.Xxx方式，

2) 没有学习参数的（例如，maxpool, loss func, activation fimc 等）情况采用 nn.functional.xxx 或者 nn.Xxx 方式。

```python
# 使用 nn.Module 定义一个简单的神经网络
import torch
import torch.nn as nn
import torch.optim as optim

# 定义一个简单的神经网络，继承自 nn.Module
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        # 使用 nn.Xxx 自动定义和保存参数
        self.fc1 = nn.Linear(10, 20)
        self.fc2 = nn.Linear(20, 10)
        self.dropout = nn.Dropout(p=0.5)  # 定义 Dropout 层

    def forward(self, x):
        x = self.fc1(x)
        x = self.dropout(x)
        x = self.fc2(x)
        return x

# 使用 nn.Sequential 结合多层
net = nn.Sequential(
    nn.Linear(10, 20),
    nn.ReLU(),
    nn.Dropout(p=0.5),
    nn.Linear(20, 10)
)

# 打印 Dropout 在训练模式下的状态
```

```
print("Training mode:")
print(net)

# 创建一个随机输入张量
input_data = torch.randn(5, 10)

# 模型实例化并调用
model = Net()
output = model(input_data)

# 切换到 eval 模式，Dropout 状态将自动切换
print("Dropout before eval:", model.dropout.training)
model.eval()
print("Eval mode:")
print(model)

# 在训练和测试模式下输出模型的 Dropout 状况
model.eval()  # 切换到评估模式
print("Dropout after eval:", model.dropout.training)

# 训练优化器设置
optimizer = optim.SGD(model.parameters(), lr=0.01)
```

```
Training mode:
Sequential(
  (0): Linear(in_features=10, out_features=20, bias=True)
  (1): ReLU()
  (2): Dropout(p=0.5, inplace=False)
  (3): Linear(in_features=20, out_features=10, bias=True)
)
Dropout before eval: True
Eval mode:
Net(
  (fc1): Linear(in_features=10, out_features=20, bias=True)
  (fc2): Linear(in_features=20, out_features=10, bias=True)
  (dropout): Dropout(p=0.5, inplace=False)
)
Dropout after eval: False
```

```
# 使用 nn.functional 定义一个简单的神经网络
import torch
import torch.nn.functional as F

# 自定义一个简单的神经网络，不继承 nn.Module
class FunctionalNet:
```

```
    def __init__(self):
        # 手动定义权重和偏置
        self.weight1 = torch.randn(20, 10, requires_grad=True)
        self.bias1 = torch.randn(20, requires_grad=True)
        self.weight2 = torch.randn(10, 20, requires_grad=True)
        self.bias2 = torch.randn(10, requires_grad=True)
        self.dropout_p = 0.5
        self.training = True

    def forward(self, x):
        x = F.linear(x, self.weight1, self.bias1)
        if self.training:  # 手动设置 dropout 状态
            x = F.dropout(x, p=self.dropout_p)
        x = F.linear(x, self.weight2, self.bias2)
        return x

    def eval(self):
        self.training = False  # 手动切换为评估模式

# 实例化模型并传入输入数据
model = FunctionalNet()
input_data = torch.randn(5, 10)

# 模型训练模式下
print("Training mode")
output = model.forward(input_data)

# 手动切换到评估模式，关闭 dropout
print("Dropout before eval:", model.training)
model.eval()
print("Eval mode")
print("Dropout after eval:", model.training)
output = model.forward(input_data)

# 手动更新权重（这里只是为了演示，通常在训练步骤中会通过优化器更新）
optimizer = optim.SGD([model.weight1, model.bias1, model.weight2,
    model.bias2], lr=0.01)
```

```
Training mode
Dropout before eval: True
Eval mode
Dropout after eval: False
```

三、 构建模型的方法

使用 PyTorch 构建模型的方法大致有以下 3 种。

1) 继承nn.Module基类构建模型。

2) 使用nn.Sequential按层顺序构建模型。

3) 继承nn.Module基类构建模型,再使用相关模型容器(如nn.Sequential、nn.ModuleList、nn.ModuleDict等）进行封装。

例子 49.1

如图49.2所示，使用 PyTorch 构建一个简单的神经网络模型，使用 3 种方法构建模型。

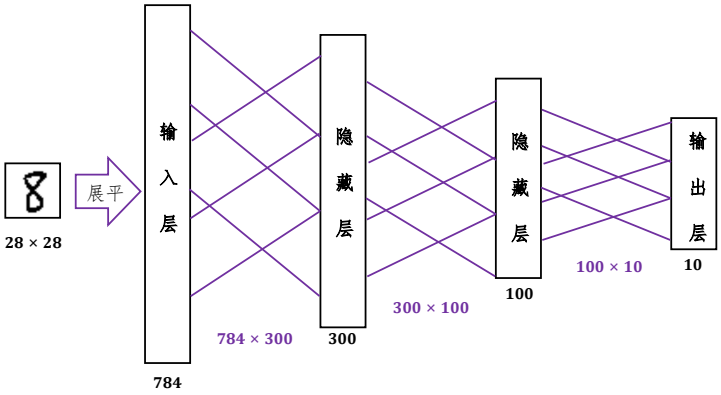

图 49.2 使用 PyTorch 构建模型的演示

1) 继承 nn.Module 基类构建模型

利用这种方法构建模型，先定义一个类，使之继承nn.Module基类。把模型中需要用到的层放在构造函数__init__()中，在forward方法中实现模型的正向传播。

```
# 构建神经网络
# 方法1.继承 nn.Module 类
import torch
from torch import nn # 导入nn 模块
import torch.nn.functional as F # 导入函数模块

############################
class Model(nn.Module):
    def __init__(self, in_dim, n_hidden_1, n_hidden_2, out_dim):
        super(Model, self).__init__()
```

```
        self.layer1 = nn.Linear(in_dim, n_hidden_1)
        self.layer2 = nn.Linear(n_hidden_1, n_hidden_2)
        self.layer3 = nn.Linear(n_hidden_2, out_dim)

    def forward(self, x):
        x = F.relu(self.layer1(x))
        x = F.relu(self.layer2(x))
        x = self.layer3(x)
        return x

## 实例化模型
model = Model(10, 20, 10, 6) # 输入是10, 输出是6
print(model)

x = torch.randn(10) # 这里是10
print("x:", x)
print(model.forward(x))
```

```
Model(
  (layer1): Linear(in_features=10, out_features=20, bias=True)
  (layer2): Linear(in_features=20, out_features=10, bias=True)
  (layer3): Linear(in_features=10, out_features=6, bias=True)
)
x: tensor([-1.0035,  0.4780,  0.8982,  0.0782, -1.2985, -0.5568,  0.2565,
    1.2554,
        0.1004,  0.4208])
tensor([-0.0519,  0.0176,  0.0493, -0.2996, -0.1996,  0.1357], #
    输出层是6个神经元
      grad_fn=<ViewBackward0>)
```

```
# 导入模块
import torch
from torch import nn            # 导入 nn 模块
import torch.nn.functional as F # 导入函数模块

################################
# 构建模型
class Model_inh_mod(nn.Module): # 通过继承 nn.Module 来构建模型
    def __init__(self, in_dim, n_hidden_1, n_hidden_2, out_dim):
        super(Model_use_Seq, self).__init__()
        self.flatten = nn.Flatten() # 展平操作
        self.linear1 = nn.Linear(in_dim, n_hidden_1)
        self.bn1 = nn.BatchNorm1d(n_hidden_1) # 批量归一化
        self.linear2 = nn.Linear(n_hidden_1, n_hidden_2)
        self.bn2 = nn.BatchNorm1d(n_hidden_2)
```

```
        self.out = nn.Linear(n_hidden_2, out_dim)

    def forward(self, x):
        x = self.flatten(x)
        x = self.linear1(x)
        x = self.bn1(x)
        x = F.relu(x)
        x = self.linear2(x)
        x = self.bn2(x)
        x = F.relu(x)
        x = self.out(x)
        x = F.softmax(x, dim=1)
        return x

# 查看模型
# 定义一个28*28的输入，
# 第一个隐藏层300个神经元，第二个隐藏层100个神经元，
# 输出层10个神经元
in_dim, n_hidden_1, n_hidden_2, out_dim = 28*28, 300, 100, 10
model = Model_inh_mod(in_dim, n_hidden_1, n_hidden_2, out_dim)
print(model)
```

```
Model_inh_mod(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (linear1): Linear(in_features=784, out_features=300, bias=True)
  (bn1): BatchNorm1d(300, eps=1e-05, momentum=0.1, affine=True,
    track_running_stats=True)
  (linear2): Linear(in_features=300, out_features=100, bias=True)
  (bn2): BatchNorm1d(100, eps=1e-05, momentum=0.1, affine=True,
    track_running_stats=True)
  (out): Linear(in_features=100, out_features=10, bias=True)
)
```

2) 使用 nn.Sequential 按层顺序构建模型

1) 利用可变参数 Python 中的函数的参数个数是可变 (或称为不定长参数) 的，PyTorch 中的有些函数也与此类似，`nn.Sequential(*args)`。

```
# 方法2 使用nn.Sequential
import torch
from torch import nn

# 定义模型
in_dim, n_hidden_1, n_hidden_2, out_dim = 10, 20, 10, 6
```

```
7
8       model = nn.Sequential(
9           nn.Linear(in_dim, n_hidden_1),
10          nn.ReLU(),
11          nn.Linear(n_hidden_1, n_hidden_2),
12          nn.ReLU(),
13          nn.Linear(n_hidden_2, out_dim)
14      )
15
16      # 打印模型
17      print(model)
18
19      # 测试模型
20      x = torch.tensor([1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
        dtype=torch.float32)
21      print("x:", x)
22      print(model(x))
23
```

```
1  # 导入模块
2  import torch
3  from torch import nn            # 导入 nn 模块
4  # 初始化参数
5  in_dim, n_hidden_1, n_hidden_2, out_dim = 28*28, 300, 100, 10
6
7  ###################################
8  # 构建模型
9  Model_use_Seq = nn.Sequential(
10     nn.Flatten(), # 展平操作
11     nn.Linear(in_dim, n_hidden_1),
12     nn.BatchNorm1d(n_hidden_1), # 批量归一化
13     nn.ReLU(),
14     nn.Linear(n_hidden_1, n_hidden_2),
15     nn.BatchNorm1d(n_hidden_2),
16     nn.ReLU(),
17     nn.Linear(n_hidden_2, out_dim),
18     nn.Softmax(dim=1)
19     )
20
21 # 查看模型
22 model = Model_use_Seq
23 print(model)
```

```
Sequential(
  (0): Flatten(start_dim=1, end_dim=-1)
  (1): Linear(in_features=784, out_features=300, bias=True)
  (2): BatchNorm1d(300, eps=1e-05, momentum=0.1, affine=True,
    track_running_stats=True)
  (3): ReLU()
  (4): Linear(in_features=300, out_features=100, bias=True)
  (5): BatchNorm1d(100, eps=1e-05, momentum=0.1, affine=True,
    track_running_stats=True)
  (6): ReLU()
  (7): Linear(in_features=100, out_features=10, bias=True)
  (8): Softmax(dim=1)
)
```

2) 使用`add_module`方法

```python
# 导入模块
import torch
from torch import nn            # 导入 nn 模块
# 初始化参数
in_dim, n_hidden_1, n_hidden_2, out_dim = 28*28, 300, 100, 10

###################################
# 构建模型
Seq_model = nn.Sequential()
Seq_model.add_module('flatten', nn.Flatten())
Seq_model.add_module('linear1', nn.Linear(in_dim,
    n_hidden_1))
Seq_model.add_module('bn1', nn.BatchNorm1d(n_hidden_1))
Seq_model.add_module('relu1', nn.ReLU())
Seq_model.add_module('linear2', nn.Linear(n_hidden_1,
    n_hidden_2))
Seq_model.add_module('bn2', nn.BatchNorm1d(n_hidden_2))
Seq_model.add_module('relu2', nn.ReLU())
Seq_model.add_module('out', nn.Linear(n_hidden_2, out_dim))
Seq_model.add_module('softmax', nn.Softmax(dim=1))

# 查看模型
model = Seq_model
print(model)
```

```
Sequential(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (linear1): Linear(in_features=784, out_features=300, bias=True)
  (bn1): BatchNorm1d(300, eps=1e-05, momentum=0.1, affine=True,
```

```
    track_running_stats=True)
  (relu1): ReLU()
  (linear2): Linear(in_features=300, out_features=100, bias=True)
  (bn2): BatchNorm1d(100, eps=1e-05, momentum=0.1, affine=True,
    track_running_stats=True)
  (relu2): ReLU()
  (out): Linear(in_features=100, out_features=10, bias=True)
  (softmax): Softmax(dim=1)
)
```

3) 使用`OrderedDict`方法

```python
# 导入模块
import torch
from torch import nn            # 导入 nn 模块
from collections import OrderedDict

# 初始化参数
in_dim, n_hidden_1, n_hidden_2, out_dim = 28*28, 300, 100, 10

###################################
# 构建模型
Seq_model = nn.Sequential(OrderedDict([
    ('flatten', nn.Flatten()),
    ('linear1', nn.Linear(in_dim, n_hidden_1)),
    ('bn1', nn.BatchNorm1d(n_hidden_1)),
    ('relu1', nn.ReLU()),
    ('linear2', nn.Linear(n_hidden_1, n_hidden_2)),
    ('bn2', nn.BatchNorm1d(n_hidden_2)),
    ('relu2', nn.ReLU()),
    ('out', nn.Linear(n_hidden_2, out_dim)),
    ('softmax', nn.Softmax(dim=1))
    ]))

# 查看模型
model = Seq_model
print(model)
```

```
Sequential(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (linear1): Linear(in_features=784, out_features=300, bias=True)
  (bn1): BatchNorm1d(300, eps=1e-05, momentum=0.1, affine=True,
    track_running_stats=True)
  (relu1): ReLU()
  (linear2): Linear(in_features=300, out_features=100, bias=True)
  (bn2): BatchNorm1d(100, eps=1e-05, momentum=0.1, affine=True,
```

```
    track_running_stats=True)
  (relu2): ReLU()
  (out): Linear(in_features=100, out_features=10, bias=True)
  (softmax): Softmax(dim=1)
)
```

3) 继承 nn.Module 基类构建模型，再使用相关模型容器进行封装

1) 使用`nn.Sequential`封装

```
# 导入模块
import torch
from torch import nn            # 导入 nn 模块
import torch.nn.functional as F # 导入函数模块

#################################
# 构建模型
class Model_use_Seq(nn.Module): # 通过继承 nn.Module 来构建模型
    def __init__(self, in_dim, n_hidden_1, n_hidden_2, out_dim):
        super(Model_use_Seq, self).__init__()
        self.flatten = nn.Flatten() # 展平操作
        self.layer1 = nn.Sequential(nn.Linear(in_dim, n_hidden_1), nn.BatchNorm1d(n_hidden_1)) # 线性层和批量归一化层
        self.layer2 = nn.Sequential(nn.Linear(n_hidden_1, n_hidden_2), nn.BatchNorm1d(n_hidden_2))
        self.out = nn.Linear(n_hidden_2, out_dim)

    def forward(self, x):
        x = self.flatten(x)
        x = F.relu(self.layer1(x))
        x = F.relu(self.layer2(x))
        x = F.softmax(self.out(x), dim=1)
        return x

# 查看模型
# 定义一个28*28的输入，
# 第一个隐藏层300个神经元，第二个隐藏层100个神经元，
# 输出层10个神经元
in_dim, n_hidden_1, n_hidden_2, out_dim = 28*28, 300, 100, 10
model = Model_use_Seq(in_dim, n_hidden_1, n_hidden_2, out_dim)
print(model)
```

```
Model_use_Seq(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (layer1): Sequential(
    (0): Linear(in_features=784, out_features=300, bias=True)
    (1): BatchNorm1d(300, eps=1e-05, momentum=0.1, affine=True,
    track_running_stats=True)
  )
  (layer2): Sequential(
    (0): Linear(in_features=300, out_features=100, bias=True)
    (1): BatchNorm1d(100, eps=1e-05, momentum=0.1, affine=True,
    track_running_stats=True)
  )
  (out): Linear(in_features=100, out_features=10, bias=True)
)
```

2) 使用`nn.ModuleList`封装

```
# 导入模块
import torch
from torch import nn            # 导入 nn 模块
import torch.nn.functional as F # 导入函数模块

#################################
# 构建模型
class Model_use_modlist(nn.Module): # 通过继承 nn.Module
    来构建模型
    def __init__(self, in_dim, n_hidden_1, n_hidden_2,
    out_dim):
        super(Model_use_modlist, self).__init__()
        self.layers = nn.ModuleList([nn.Flatten(),
                                     nn.Linear(in_dim,
    n_hidden_1),

    nn.BatchNorm1d(n_hidden_1),
                                     nn.ReLU(),
                                     nn.Linear(n_hidden_1,
    n_hidden_2),

    nn.BatchNorm1d(n_hidden_2),
                                     nn.ReLU(),
                                     nn.Linear(n_hidden_2,
    out_dim),
                                     nn.Softmax(dim=1)])

```

```
    def forward(self, x):
        for layer in self.layers:
            x = layer(x)
        return x

# 查看模型
# 定义一个28*28的输入，
# 第一个隐藏层300个神经元，第二个隐藏层100个神经元，
# 输出层10个神经元
in_dim, n_hidden_1, n_hidden_2, out_dim = 28*28, 300, 100, 10
model = Model_use_modlist(in_dim, n_hidden_1, n_hidden_2,
    out_dim)
print(model)
```

```
Model_use_modlist(
  (layers): ModuleList(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=784, out_features=300, bias=True)
    (2): BatchNorm1d(300, eps=1e-05, momentum=0.1, affine=True,
    track_running_stats=True)
    (3): ReLU()
    (4): Linear(in_features=300, out_features=100, bias=True)
    (5): BatchNorm1d(100, eps=1e-05, momentum=0.1, affine=True,
    track_running_stats=True)
    (6): ReLU()
    (7): Linear(in_features=100, out_features=10, bias=True)
    (8): Softmax(dim=1)
  )
)
```

3) 使用`nn.ModuleDict`封装

```
# 导入模块
import torch
from torch import nn            # 导入 nn 模块

################################
# 构建模型
class Model_use_dict(nn.Module): # 通过继承 nn.Module
    来构建模型
    def __init__(self, in_dim, n_hidden_1, n_hidden_2,
    out_dim):
        super(Model_use_dict, self).__init__()
        self.layers = nn.ModuleDict({
            'flatten': nn.Flatten(),
            'linear1': nn.Linear(in_dim, n_hidden_1),
```

```
            'bn1': nn.BatchNorm1d(n_hidden_1),
            'relu1': nn.ReLU(),
            'linear2': nn.Linear(n_hidden_1, n_hidden_2),
            'bn2': nn.BatchNorm1d(n_hidden_2),
            'relu2': nn.ReLU(),
            'out': nn.Linear(n_hidden_2, out_dim),
            'softmax': nn.Softmax(dim=1)
        })

    def forward(self, x):
        for layer in self.layers.values():
            x = layer(x)
        return x

# 查看模型
in_dim, n_hidden_1, n_hidden_2, out_dim = 28*28, 300, 100, 10
model = Model_use_dict(in_dim, n_hidden_1, n_hidden_2,
    out_dim)
print(model)
```

其中激活函数 ReLU 在模型中应该出现 2 次，但在定义字典时，只需定义一次，在定义 forward 函数的列表中则需要出现 2 次。

```
Model_use_dict(
  (layers): ModuleDict(
    (flatten): Flatten(start_dim=1, end_dim=-1)
    (linear1): Linear(in_features=784, out_features=300, bias=True)
    (bn1): BatchNorm1d(300, eps=1e-05, momentum=0.1, affine=True,
    track_running_stats=True)
    (relu1): ReLU()
    (linear2): Linear(in_features=300, out_features=100, bias=True)
    (bn2): BatchNorm1d(100, eps=1e-05, momentum=0.1, affine=True,
    track_running_stats=True)
    (relu2): ReLU()
    (out): Linear(in_features=100, out_features=10, bias=True)
    (softmax): Softmax(dim=1)
  )
)
```

四、 损失优化

1) Pytorch 激活函数

第**??**页激活函数一节中介绍了常见的激活函数，PyTorch 中也提供了这些激活函数，如表49.1所示。

激活函数	表达式	调用方法
nn.Sigmoid	$\text{Sigmoid}(x) = \frac{1}{1+\exp(-x)}$	torch.nn.Sigmoid()
nn.Tanh	$\text{Tanh}(x) = \frac{\exp(x)-\exp(-x)}{\exp(x)+\exp(-x)}$	torch.nn.Tanh()
nn.ReLU	$\text{ReLU}(x) = \max(0, x)$	torch.nn.ReLU(inplace=False)
nn.LeakyReLU	$\text{LeakyReLU}(x) = \begin{cases} x, & \text{if } x \geq 0 \\ \text{negative_slope} \cdot x, & \text{if } x < 0 \end{cases}$	torch.nn.LeakyReLU(negative_slope=0.01, inplace=False)
nn.ELU	$\text{ELU}(x) = \begin{cases} x, & \text{if } x > 0 \\ \alpha(\exp(x)-1), & \text{if } x \leq 0 \end{cases}$	torch.nn.ELU(alpha=1.0, inplace=False)
nn.Hardshrink	$\text{Hardshrink}(x) = \begin{cases} x, & \text{if } \lvert x \rvert > \lambda \\ 0, & \text{if } \lvert x \rvert \leq \lambda \end{cases}$	torch.nn.Hardshrink(lambd=0.5)
nn.Hardsigmoid	$\text{Hardsigmoid}(x) = \max(0, \min(1, \frac{x+1}{2}))$	torch.nn.Hardsigmoid(inplace=False)
nn.Hardtanh	$\text{Hardtanh}(x) = \begin{cases} 1, & \text{if } x > 1 \\ -1, & \text{if } x < -1 \\ x, & \text{otherwise} \end{cases}$	torch.nn.Hardtanh(min_val=-1.0, max_val=1.0)
nn.Hardswish	$\text{Hardswish}(x) = x \cdot \frac{\min(\max(x+3), 6)}{6}$	torch.nn.Hardswish(inplace=False)
nn.LogSigmoid	$\text{LogSigmoid}(x) = \log(\frac{1}{1+\exp(-x)})$	torch.nn.LogSigmoid()
nn.MultiheadAttention	-	torch.nn.MultiheadAttention(embed_dim, num_heads, ...)
nn.PReLU	$\text{PReLU}(x) = \begin{cases} x, & \text{if } x \geq 0 \\ a \cdot x, & \text{if } x < 0 \end{cases}$	torch.nn.PReLU(num_parameters=1, init=0.25)
nn.ReLU6	$\text{ReLU6}(x) = \min(\max(0, x), 6)$	torch.nn.ReLU6(inplace=False)
nn.RReLU	$\text{RReLU}(x) = \begin{cases} x, & \text{if } x \geq 0 \\ a \cdot x, & \text{if } x < 0 \end{cases}$	torch.nn.RReLU(lower=0.125, upper=0.33, inplace=False)
nn.SELU	$\text{SELU}(x) = \lambda \cdot \begin{cases} x, & \text{if } x > 0 \\ \alpha \cdot (\exp(x)-1), & \text{if } x \leq 0 \end{cases}$	torch.nn.SELU(inplace=False)
nn.CELU	$\text{CELU}(x) = \begin{cases} x, & \text{if } x > 0 \\ \alpha(\exp(\frac{x}{\alpha})-1), & \text{if } x \leq 0 \end{cases}$	torch.nn.CELU(alpha=1.0, inplace=False)
nn.GELU	$\text{GELU}(x) = 0.5 \cdot x \cdot \left[1 + \tanh(\sqrt{\frac{2}{\pi}}(x + 0.044715 \cdot x^3))\right]$	torch.nn.GELU()
nn.SiLU	$\text{SiLU}(x) = \frac{x}{1+\exp(-x)}$	torch.nn.SiLU(inplace=False)
nn.Softplus	$\text{Softplus}(x) = \frac{1}{\beta}\log(1+\exp(\beta x))$	torch.nn.Softplus(beta=1, threshold=20)
nn.Softshrink	$\text{Softshrink}(x) = \begin{cases} x-\lambda, & \text{if } x > \lambda \\ x+\lambda, & \text{if } x < -\lambda \\ 0, & \text{otherwise} \end{cases}$	torch.nn.Softshrink(lambd=0.5)
nn.Softsign	$\text{Softsign}(x) = \frac{x}{1+\lvert x \rvert}$	torch.nn.Softsign()
nn.Tanhshrink	$\text{Tanhshrink}(x) = x - \tanh(x)$	torch.nn.Tanhshrink()
nn.Threshold	$\text{Threshold}(x) = \begin{cases} x, & \text{if } x > \text{threshold} \\ \text{value}, & \text{if } x \leq \text{threshold} \end{cases}$	torch.nn.Threshold(threshold, value, inplace=False)
nn.Softmin	$\text{Softmin}(x_i) = \frac{\exp(-x_i)}{\sum_j \exp(-x_j)}$	torch.nn.Softmin(dim=None)
nn.Softmax	$\text{Softmax}(x_i) = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$	torch.nn.Softmax(dim=None)
nn.Softmax2d	-	torch.nn.Softmax2d()
nn.LogSoftmax	$\text{LogSoftmax}(x_i) = \log\left(\frac{\exp(x_i)}{\sum_j \exp(x_j)}\right)$	torch.nn.LogSoftmax(dim=None)
nn.AdaptiveLogSoftmaxWithLoss	-	torch.nn.AdaptiveLogSoftmaxWithLoss(...)

表 49.1 PyTorch 激活函数及其表达式和调用方法

2) Pytorch 损失函数

第**??**页误差函数一节中介绍了常见的损失函数，主要出现在反向传播中，如表49.2所示，就是 PyTorch 中的损失函数。

损失函数	函数说明	表达式	调用方法
nn.L1Loss	L1 范数损失函数	$\text{L1Loss}(x) = \sum_i \lvert x_i - y_i \rvert$	torch.nn.L1Loss(size_average=None, reduce=None, reduction='mean')
nn.MSELoss	均方误差损失函数	$\text{MSELoss}(x) = \frac{1}{n}\sum_i (x_i - y_i)^2$	torch.nn.MSELoss(reduce=True, size_average=True, reduction='mean')
nn.CrossEntropyLoss	交叉熵损失函数	$\text{CrossEntropyLoss}(x) = -\sum_i y_i \log(p(x_i))$	torch.nn.CrossEntropyLoss(weight=None, size_average=None, ignore_index=-100, reduce=None, reduction='mean')
nn.CosineEmbeddingLoss	余弦相似度损失函数	$\text{CosineEmbeddingLoss}(x, y) = \begin{cases} 1-\cos(x_1, x_2), & y=1 \\ \max(0, \cos(x_1, x_2) - margin), & y=-1 \end{cases}$	torch.nn.CosineEmbeddingLoss(margin=0.0, reduction='mean')
nn.CTCLoss	连接时序分类损失	-	torch.nn.CTCLoss()
nn.NLLLoss	负对数似然损失	$\text{NLLLoss}(x) = -\sum_i y_i \log(p(x_i))$	torch.nn.NLLLoss()
nn.PoissonNLLLoss	泊松负对数似然损失	$\text{PoissonNLLLoss}(x) = \log(\frac{x^y}{y!})$	torch.nn.PoissonNLLLoss()
nn.GaussianNLLLoss	高斯负对数似然损失	$\text{GaussianNLLLoss}(x) = \frac{1}{2}\log(2\pi\sigma^2) + \frac{(x-\mu)^2}{2\sigma^2}$	torch.nn.GaussianNLLLoss()
nn.KLDivLoss	KL 散度损失	$D_{KL}(P \lvert\rvert Q) = \sum_i P(x) \log(\frac{P(x)}{Q(x)})$	torch.nn.KLDivLoss()
nn.BCELoss	二进制交叉熵损失	$\text{BCELoss}(x) = -\sum_i y \log(p(x)) + (1-y)\log(1-p(x))$	torch.nn.BCELoss()
nn.BCEWithLogitsLoss	逻辑二进制交叉熵损失	-	torch.nn.BCEWithLogitsLoss()
nn.MarginRankingLoss	间隔排序损失	$\text{MarginRankingLoss}(x) = \max(0, -y(x_1 - x_2) + \text{margin})$	torch.nn.MarginRankingLoss()
nn.HingeEmbeddingLoss	铰链嵌入损失	$\text{HingeEmbeddingLoss}(x) = \max(0, 1 - yx)$	torch.nn.HingeEmbeddingLoss()
nn.MultiLabelMarginLoss	多标签分类损失	-	torch.nn.MultiLabelMarginLoss()
nn.SoftMarginLoss	两分类逻辑损失	-	torch.nn.SoftMarginLoss()
nn.MultiLabelSoftMarginLoss	多标签逻辑损失	-	torch.nn.MultiLabelSoftMarginLoss()
nn.SmoothL1Loss	平滑 L1 损失	$\text{SmoothL1Loss}(x) = \begin{cases} 0.5(x-y)^2, & \text{if } \lvert x-y \rvert < 1 \\ \lvert x-y \rvert - 0.5, & \text{otherwise} \end{cases}$	torch.nn.SmoothL1Loss()
nn.MultiMarginLoss	多类别分类损失	-	torch.nn.MultiMarginLoss()
nn.TripletMarginLoss	三元组损失	-	torch.nn.TripletMarginLoss()
nn.TripletMarginWithDistanceLoss	距离三元组损失	-	torch.nn.TripletMarginWithDistanceLoss()

表 49.2 损失函数及其函数说明、表达式和调用方法

其中常用的

1) L1 范数损失函数：`nn.L1Loss`

```
loss = nn.L1Loss(size_average=False, reduce=False,
reduction='mean')
# size_average: 当reduce=True时有效。
#    为True时，返回的loss为平均值；
#    为False时，  返回各样本的loss数值之和。
```

```
    # reduce:返回值是否为标量，默认为True。
```

```
import torch

# L1Loss 使用 reduction='mean'，即对所有样本的损失取均值
loss_mean = torch.nn.L1Loss(reduction='mean')
input = torch.tensor([1.0, 2.0, 6.0, 5.0])
target = torch.tensor([4.0, 5.0, 6.0, 7.0])
output_mean = loss_mean(input, target)
print(f"使用 reduction='mean' 的输出：{output_mean.item()}")
    # 输出：2.0

# L1Loss 使用 reduction='sum'，即对所有样本的损失取和
loss_sum = torch.nn.L1Loss(reduction='sum')
output_sum = loss_sum(input, target)
print(f"使用 reduction='sum' 的输出：{output_sum.item()}")  #
    输出：8.0

# L1Loss 使用 reduction='none'，即不对损失进行任何聚合
loss_none = torch.nn.L1Loss(reduction='none')
output_none = loss_none(input, target)
print(f"使用 reduction='none' 的输出：{output_none}")  #
    输出：tensor([3., 3., 0., 2.])

##################################################
# 下面的方法在 PyTorch 1.0.0 版本后已经被废弃
# 使用 size_average 和 reduce 参数来模拟上述情况
# size_average=True, reduce=True 等效于 reduction='mean'
loss_size_average = torch.nn.L1Loss(size_average=True,
    reduce=True)
output_size_average = loss_size_average(input, target)
print(f"使用 size_average=True, reduce=True 的输出：
    {output_size_average.item()}")  # 输出：2.0

# size_average=False, reduce=True 等效于 reduction='sum'
loss_reduce_sum = torch.nn.L1Loss(size_average=False,
    reduce=True)
output_reduce_sum = loss_reduce_sum(input, target)
print(f"使用 size_average=False, reduce=True 的输出：
    {output_reduce_sum.item()}")  # 输出：8.0

# size_average=False, reduce=False 等效于 reduction='none'
loss_reduce_none = torch.nn.L1Loss(size_average=False,
    reduce=False)
output_reduce_none = loss_reduce_none(input, target)
print(f"使用 size_average=False, reduce=False 的输出：
```

```
        {output_reduce_none}")  # 输出: tensor([3., 3., 0., 2.])
```

```
使用 reduction='mean' 的输出: 2.0
使用 reduction='sum' 的输出: 8.0
使用 reduction='none' 的输出: tensor([3., 3., 0., 2.])

##################################################
使用 size_average=True, reduce=True 的输出: 2.0
使用 size_average=False, reduce=True 的输出: 8.0
使用 size_average=False, reduce=False 的输出: tensor([3., 3., 0.,
    2.])
```

例子 49.2

给定输入 $input = [1.0, 2.0, 6.0, 5.0]$, 目标 $target = [4.0, 5.0, 6.0, 7.0]$，计算 L1 范数损失函数。

方法: L1 损失函数 ('torch.nn.L1Loss') 的定义是目标值和预测值之间的绝对差值。数学公式如下：

$$\text{L1Loss} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

其中：y_i 是目标值。$\hat{y}_i$ 是预测值。N 是样本的数量。

$|1.0-4.0| = 3.0, |2.0-5.0| = 3.0, |6.0-6.0| = 0.0, |5.0-7.0| = 2.0$，所以损失值为 $[3.0, 3.0, 0.0, 2.0]$。

1) 当 `reduction='mean'` 时：对损失进行平均，输出值为 $output = \frac{3.0+3.0+0.0+2.0}{4} = 2.0$。
2) 当 `reduction='sum'` 时：对损失进行求和，输出值为 $output = 3.0 + 3.0 + 0.0 + 2.0 = 8.0$。
3) 当 `reduction='none'` 时：不进行任何聚合操作，输出的张量为每个样本的绝对误差：`tensor([3., 3., 0., 2.])`。

2) 均方误差损失函数：`nn.MSELoss`

```
    loss = nn.MSELoss(reduce=False, size_average=False,
    reduction='mean')
    # size_average: 当reduce=True时有效。
    #   为True时，返回的loss为平均值;
    #   为False时，返回各样本的loss数值之和。
    # reduce:返回值是否为标量，默认为True。
```

```
```

```python
import torch

# 使用 MSELoss（均方误差损失函数）
# reduction='mean': 对所有样本的平方误差取均值
loss_mse = torch.nn.MSELoss(reduction='mean')
input = torch.tensor([1.5, 3.0, 5.0, 4.0])
target = torch.tensor([4.0, 2.0, 6.0, 7.0])
output_mse = loss_mse(input, target)
print(f"使用 reduction='mean' 的 MSE 损失输出: {output_mse.item()}")  # 输出: 4.25

# 使用 MSELoss, reduction='sum': 对所有样本的平方误差取和
loss_mse_sum = torch.nn.MSELoss(reduction='sum')
output_mse_sum = loss_mse_sum(input, target)
print(f"使用 reduction='sum' 的 MSE 损失输出: {output_mse_sum.item()}")  # 输出: 17.0

# 使用 MSELoss, reduction='none': 不对损失进行任何聚合
loss_mse_none = torch.nn.MSELoss(reduction='none')
output_mse_none = loss_mse_none(input, target)
print(f"使用 reduction='none' 的 MSE 损失输出: {output_mse_none}")  # 输出: tensor([6.25, 1.00, 1.00, 8.75])
```

```
使用 reduction='mean' 的 MSE 损失输出: 4.3125
使用 reduction='sum' 的 MSE 损失输出: 17.25
使用 reduction='none' 的 MSE 损失输出: tensor([6.2500, 1.0000, 1.0000, 9.0000])
```

例子 49.3

给定输入 $input = [1.5, 3.0, 5.0, 4.0]$，目标 $target = [4.0, 2.0, 6.0, 7.0]$，计算均方误差损失函数。

方法: MSE 损失函数 (均方误差) 的定义为：

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

其中：y_i 是目标值。$\hat{y}_i$ 是预测值。N 是样本的数量。

$(1.5-4.0)^2 = 6.25, (3.0-2.0)^2 = 1.0, (5.0-6.0)^2 = 1.0, (4.0-7.0)^2 = 9.0$，所以损失值为 $[6.25, 1.0, 1.0, 9.0]$。

1) 当 `reduction='mean'` 时：对损失进行平均，输出值为 $output = \frac{6.25+1.0+1.0+9.0}{4} = 4.3125$。
2) 当 `reduction='sum'` 时：对损失进行求和，输出值为 $output = 6.25 + 1.0 + 1.0 + 9.0 = 17.25$。

3) 当`reduction='none'`时：不进行任何聚合操作，输出值为 $output = [6.25, 1.0, 1.0, 9.0]$。不进行任何聚合操作，输出的张量为每个样本的平方误差：`tensor([6.25, 1.0, 1.0, 9.0])`。

3) 交叉熵损失函数：`nn.CrossEntropyLoss`

```
import torch

# 使用 CrossEntropyLoss (交叉熵损失函数)
# reduction='mean': 对所有样本的交叉熵取均值
loss_ce = torch.nn.CrossEntropyLoss(weight=None, size_average=None,
    ignore_index=-100, reduction='mean')
input = torch.Tensor([[-0.1181, -0.3682, -0.2209]])
target = torch.tensor([0])
output_ce = loss_ce(input, target)
print(f"使用 reduction='mean' 的 CrossEntropy 损失输出:
    {output_ce.item()}")  # 输出: 1.0213

# 使用 CrossEntropyLoss, reduction='sum': 对所有样本的交叉熵取和
loss_ce_sum = torch.nn.CrossEntropyLoss(weight=None,
    size_average=None, ignore_index=-100, reduction='sum')
output_ce_sum = loss_ce_sum(input, target)
print(f"使用 reduction='sum' 的 CrossEntropy 损失输出:
    {output_ce_sum.item()}")  # 输出: 1.0213

# 使用 CrossEntropyLoss, reduction='none': 不对损失进行任何聚合
loss_ce_none = torch.nn.CrossEntropyLoss(weight=None,
    size_average=None, ignore_index=-100, reduction='none')
output_ce_none = loss_ce_none(input, target)
print(f"使用 reduction='none' 的 CrossEntropy 损失输出:
    {output_ce_none}")  # 输出: tensor([1.0213])
```

```
使用 reduction='mean' 的 CrossEntropy 损失输出: 0.9862011671066284
使用 reduction='sum' 的 CrossEntropy 损失输出: 0.9862011671066284
使用 reduction='none' 的 CrossEntropy 损失输出: tensor([0.9862])
```

例子 49.4

给定输入 $input = [-0.1181, -0.3682, -0.2209]$，目标 $target = 0$，计算交叉熵损失函数。

方法: 交叉熵损失函数的定义为：

$$CrossEntropyLoss = -\sum_{i=1}^{N} y_i \cdot \log(\hat{y}_i)$$

其中：y_i 是目标类标签 (在此例中为 0，即目标类为第一个类别)。· $\hat{y}_i$ 是预测的概率。

计算 Softmax 函数的输出值：$\hat{y}_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$。

$$\hat{y}_0 = \frac{e^{-0.1181}}{e^{-0.1181} + e^{-0.3682} + e^{-0.2209}} = \frac{0.8884}{0.8884 + 0.6923 + 0.8015} = 0.37299093229950897$$

计算交叉熵损失函数:$CrossEntropyLoss = -1 \cdot \log_2(0.37299093229950897) = 0.9862011671066284$。

4) 余弦相似度损失函数：`nn.CosineEmbeddingLoss`

```
loss = nn.CosineEmbeddingLoss(margin=0.0, size_average=None,
reduce=None, reduction='mean')
```

```python
import torch

# 使用 CosineEmbeddingLoss (余弦相似度损失函数)
# reduction='mean': 对所有样本的余弦相似度损失取均值
loss_cosine = torch.nn.CosineEmbeddingLoss(margin=0.0,
    reduction='mean')

# 将1D张量转换为2D张量
a = torch.tensor([1.0, 2.0, 3.0, 4.0]).unsqueeze(0)  #
    变成形状为 [1, 4]
b = torch.tensor([4.1, 6.1, 7.1, 8.1]).unsqueeze(0)  #
    变成形状为 [1, 4]
target = torch.tensor([1])  # 相似性目标为正 (1 表示相似)

output_cosine = loss_cosine(a, b, target)
print(f"使用 reduction='mean' 的 CosineEmbedding 损失输出:
    {output_cosine.item()}")  # 输出: 0.0082

# 使用 CosineEmbeddingLoss, reduction='sum':
    对所有样本的余弦相似度损失取和
loss_cosine_sum = torch.nn.CosineEmbeddingLoss(margin=0.0,
    reduction='sum')
output_cosine_sum = loss_cosine_sum(a, b, target)
print(f"使用 reduction='sum' 的 CosineEmbedding 损失输出:
    {output_cosine_sum.item()}")  # 输出: 0.0082

# 使用 CosineEmbeddingLoss, reduction='none':
    不对损失进行任何聚合
loss_cosine_none = torch.nn.CosineEmbeddingLoss(margin=0.0,
    reduction='none')
output_cosine_none = loss_cosine_none(a, b, target)
print(f"使用 reduction='none' 的 CosineEmbedding 损失输出:
    {output_cosine_none}")  # 输出: tensor([0.0082])
```

```
使用 reduction='mean' 的 CosineEmbedding 损失输出:
    0.019919216632843018
使用 reduction='sum' 的 CosineEmbedding 损失输出:
    0.019919216632843018
使用 reduction='none' 的 CosineEmbedding 损失输出: tensor([0.0199])
```

例子 49.5

给定输入 $a=[1.0,2.0,3.0,4.0]$，$b=[4.1,6.1,7.1,8.1]$，目标 $target=1$，计算余弦相似度损失函数。

方法: 余弦相似度损失函数的定义为：

$$CosineEmbeddingLoss=\begin{cases} margin-cos(\hat{y},y), & \text{if } target=1 \\ margin+cos(\hat{y},y), & \text{if } target=-1 \end{cases}$$

在这里写成

$$\text{CosineEmbeddingLoss}=1-\frac{a\cdot b}{\|a\|\|b\|}$$

$$\begin{cases} a\cdot b=(1.0\times 4.1)+(2.0\times 6.1)+(3.0\times 7.1)+(4.0\times 8.1)=4.1+12.2+21.3+32.4 \\ \|a\|=\sqrt{1^2+2^2+3^2+4^2}=\sqrt{1+4+9+16}=\sqrt{30} \\ \|b\|=\sqrt{4.1^2+6.1^2+7.1^2+8.1^2}\approx\sqrt{16.81+37.21+50.41+65.61}=\sqrt{170.04} \end{cases}$$

余弦相似度计算为：

$$\cos(\theta)=\frac{70.0}{\sqrt{30}\times\sqrt{170.04}}\approx 0.9800807619823181$$

余弦相似度损失为：

$$CosineEmbeddingLoss=1-\cos(\theta)=1-0.9800807619823181=0.019919216632843018$$

3) Pytorch 优化器

优化器就是在深度学习反向传播过程中，指引损失函数的各个参数往正确的方向更新合适的大小，使得更新后的各个参数让损失函数（目标函数）值不断逼近全局最小。

第**??**页介绍了梯度下降法，常见的优化器有 SGD、Adam、RMSprop 等，

```python
# 导入相关第三方库
import torch
import torch.nn as nn
import torch.utils.data as Data
import matplotlib.pyplot as plt
import matplotlib
from tqdm import tqdm  # 导入tqdm库

matplotlib.rcParams['font.sans-serif'] = ['SimHei']

# 准备数据
x = torch.unsqueeze(torch.linspace(-1, 1, 500), dim=1)
y = x.pow(3)

# 设置超参数
# 超参数: 学习率、迭代次数、隐藏层神经元个数
LR = 0.01
EPOCHs = 1000
BATCH_SIZE = 15
torch.manual_seed(985)

# 设置数据加载器
dataset = Data.TensorDataset(x, y)
data_loader = Data.DataLoader(
    dataset=dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=2,
)

# 设置神经网络
class Net(nn.Module):
    def __init__(self, n_input, n_hidden, n_output):  # 修正构造函数名
        super(Net, self).__init__()
        self.hidden = nn.Linear(n_input, n_hidden)
        self.output = nn.Linear(n_hidden, n_output)

    def forward(self, input):
        x = torch.relu(self.hidden(input))
        output = self.output(x)
        return output

# 训练模型
def train():
    net_SDG = Net(1, 10, 1)
    net_Momentum = Net(1, 10, 1)
    net_RMSprop = Net(1, 10, 1)
```

```
48      net_Adam = Net(1, 10, 1)
49      net_AdaGrad = Net(1, 10, 1)
50      nets = [net_SDG, net_Momentum, net_RMSprop, net_Adam,
        net_AdaGrad]
51
52      # 设置优化器
53      optimizer_SDG = torch.optim.SGD(net_SDG.parameters(), lr=LR)
54      optimizer_Momentum =
        torch.optim.SGD(net_Momentum.parameters(), lr=LR, momentum=0.8)
55      optimizer_RMSprop =
        torch.optim.RMSprop(net_RMSprop.parameters(), lr=LR, alpha=0.9)
56      optimizer_Adam = torch.optim.Adam(net_Adam.parameters(),
        lr=LR, betas=(0.9, 0.99))
57      optimizer_AdaGrad =
        torch.optim.Adagrad(net_AdaGrad.parameters(), lr=LR)
58
59      # 设置损失函数
60      loss_func = nn.MSELoss()
61      losses = [[], [], [], [], []] # 用于存储损失值
62
63      # 训练模型
64      for epoch in tqdm(range(EPOCHs), desc='训练进度'):  #
        添加tqdm进度条
65          for step, (batch_x, batch_y) in enumerate(data_loader):
66              for net, optimizer, loss_list in zip(nets,
        [optimizer_SDG, optimizer_Momentum, optimizer_RMSprop,
        optimizer_Adam, optimizer_AdaGrad], losses):
67                  pred_y = net(batch_x)
68                  loss_value = loss_func(pred_y, batch_y)
69                  optimizer.zero_grad()
70                  loss_value.backward()
71                  optimizer.step()
72                  loss_list.append(loss_value.item())
73
74      # 可视化结果
75      labels = ['SGD', 'Momentum', 'RMSprop', 'Adam', 'AdaGrad']
76      plt.figure(figsize=(12, 7), dpi=160)
77      for i, loss in enumerate(losses):
78          plt.plot(loss, label=labels[i])
79
80      plt.tick_params(labelsize=13)
81      plt.xlabel("训练步骤", size=15)
82      plt.ylabel("模型损失", size=15)
83      plt.ylim((0, 0.3))
84      plt.legend(loc="upper right", fontsize=15)
85      plt.show()
86
87  train()
88  # if __name__ == '__main__':
89  #     train()
```

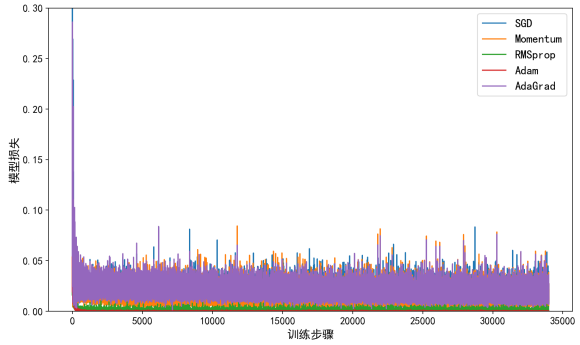

图 49.3 多种优化器比较

五、 训练模型

PyTorch 训练模型主要包括加载数据集、损失计算、定义优化算法、反向传播、参数更新等主要步骤。

1) 加载和预处理数据集

可以使用 PyTorch 的数据处理工具，如`torch.utils`和`torchvision`等

2) 定义损失函数

可以通过自定义方法或使用 PyTorch 内置的损失函数定义损失函数，如回归使用的`loss_fun=nn.MSELoss()`, 分类使用的`nn.BCELoss`等损失函数

3) 定义优化方法

PyTorch 常用的优化方法都封装在`torch.optim`中，其设计很灵活，可以扩展为自定义的优化方法。

所有的优化方法都是继承了基类`otim.Optimizer`, 并实现了自己的优化步骤。

如使用 SGD 优化器时，可设置`optimizer = torch.optim.SGD(params,lr = 0.001)`。

4) 循环训练模型

1) 设置为训练模式：调用`model.train()`会把所有的 module 设置为训练模式。

2) 梯度清零在默认情况下梯度是累加的，需要手工把梯度初始化或清零，调用`optimizer.zero_grad()`即可。

3) 求损失值

```
y_pred = model(x)
loss = loss_fun(y_pred, y)
```

4) 反向传播 (自动求导) 调用`loss.backward()`即可。

5) 更新参数调用`optimizer.step()`即可。

5) 循环测试或验证模型

1) 设置为测试或验证模式：调用`model.eval()`会把所有的`training`属性设置为`False`。

2) 在不跟踪梯度的模式下计算损失值、预测值等

```
with torch.no_grad():
    y_pred = model(x)
    loss = loss_fun(y_pred, y)
```

(1) 比较 model.train() 和 model.eval() 的区别

如果模型中有批量归一化 (Batch Normalization, BH) 层和 dropout 层，需要在训练时添加`model.train()`, 在测试时添加`model.eval()`

其中`model.train()`是保证 BN 层用每一批数据的均值和方差,而`model.eval()`是保证 BN 层用全部训练数据的均值和方差;

而对于 dropout 层,`model.train()`是随机取一部分网络连接来训练更新参数，而`model.eval()`是利用所有网络连接。

6) 可视化结果

六、 正则化

正则化是一种用于防止过拟合的技术，主要有 L1 正则化、L2 正则化、Dropout 等方法。

1) L1 正则化

2) L2 正则化

3) Dropout

4) Batch Normalization

七、 训练技巧

八、 保存和加载模型

PyTorch 中保存和加载模型的方法有多种，如保存和加载整个模型、保存和加载模型参数等。

九、 实例

1) 模拟余弦函数曲线

```
# 导入相关第三方库
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from torch.utils.data import TensorDataset
import numpy as np
import matplotlib
import matplotlib.pyplot as pit
matplotlib.rcParams['font.sans-serif'] = ['SimHei']
matplotlib.rcParams['axes.unicode_minus'] = False

# 准备拟合数据
x=np.linspace(-2*np.pi, 2*np.pi, 400)
y=np.cos(x)
X=np. expand_dims (x, axis=1)
Y=y.reshape(400,-1)
dataset=TensorDataset(torch.tensor(X,dtype=torch.float) ,
    torch.tensor(Y,dtype=torch.float))
dataloader=DataLoader(dataset,batch_size=10,shuffle=True)

# 设置神经网络，这里就用一个简单的线性结构
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.net=nn.Sequential(
            nn.Linear(1,100),
            nn.ReLU(),
            nn.Linear(100,1)
        )
    def forward(self, input:torch.FloatTensor):
```

```
        return self.net(input)

net=Net()

# 设置优化器和损失函数
optim=torch.optim.Adam(Net.parameters(net),lr=0.001)
Loss=nn.MSELoss()

# 开始训练
epochs=1000
for epoch in range(epochs):
    loss=None
    for batch_x, batch_y in dataloader:
        y_predict=net(batch_x)
        loss=Loss(y_predict, batch_y)
        optim.zero_grad()
        loss.backward()
        optim.step()
    if epoch%100==0:
        print(f'epoch:{epoch},loss:{loss.item()}')

predict=net(torch.tensor(X,dtype=torch.float))
# 可视化结果
pit.figure(figsize=(12, 7), dpi=160)
pit.plot(x, y, label="实际值", marker = "X")
pit.plot(x, predict.detach().numpy(), label="预测值", marker= 'o')
pit.xlabel("x", size=15)
pit.ylabel ("cos (x)", size=15)
pit.xticks(size=15)
pit.yticks(size=15)
pit.legend(fontsize=15)
pit.show()
```

```
epoch:0,loss:0.413939893245697
epoch:100,loss:0.005030750297009945
epoch:200,loss:0.0018504426116123796
epoch:300,loss:0.0018474120879545808
epoch:400,loss:0.0006889037904329598
epoch:500,loss:0.0005864549311809242
epoch:600,loss:0.0002862799447029829
epoch:700,loss:0.001682802801951766
epoch:800,loss:0.0007093378226272762
epoch:900,loss:3.737903170986101e-05
```

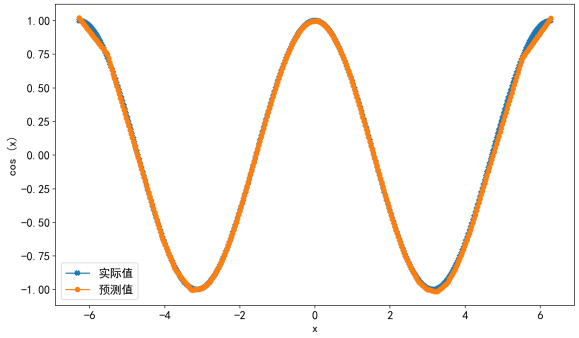

图 49.4 模拟余弦函数曲线

第十部分

办公自动化

第五十章 excel 自动化概述

Python 操作 excel 的库有xlrd、xlwt、openpyxl、xlutils。

一、 Excel 介绍

Excel 工作簿有两种格式:`xls`和`xlsx`。

☞ 如果存储的数据大于工作簿的极限时，就可以考虑使用数据库来存储数据。

1) `xls`格式的文件是二进制文件，它是 Excel 2003 及以前版本的文件格式。
   `xls`格式的文件最多可写入 65 535 行、256 列的数据量
2) `xlsx`格式的文件是 XML 文件，它是 Excel 2007 及以后版本的文件格式。
   `xlsx`格式的文件最多可写入 1048576 行、16384 列的数据量

二、 处理 Excel 文件的 Python 模块

可以处理 Excel 文件的 Python 模块有很多，如 XlsxWriter、xlrd、xlwt、xlutils、openpyxl 和 xlwings 等。

功能 \ 模块	XlsxWriter	xlrd	xlwt	xlutils	openpyxl	xlwings
读	x	✓	x	✓	✓	✓
写	✓	x	✓	✓	✓	✓
修改	x	x	x	✓	✓	✓
支持 xls 格式	x	✓	✓	✓	x	✓
支持 xlsx 格式	✓	✓	✓	x	✓	✓
支持批量操作	x	x	x	x	x	✓

表 50.1 python Excel 不同模块支持功能情况

通过上表的对比可以发现，xlwings 模块的功能是最齐全的。它不仅能读、写和修改两种格式的 Excel 文件（xls 和 xlsx），而且能批量处理多个 Excel 文件。此外，xlwings 模块还能与 Excel VBA 结合使用，实现更加强大的数据输入和分析功能。

第五十一章 Pandas 处理 Excel

与 NumPy 模块相比，pandas 模块更擅长处理二维数据，其主要有`Series`和`DataFrame`两种数据结构。

一、 Excel 文件操作

1) read_excel 文件读取

通过 pandas 模块可以从多种格式的数据文件中读取数据，也可以将处理后的数据写入这些文件中。

```
import pandas as pd
df = pd.read_excel('xlsx文件')

# 或者
df = pd.read_excel('xlsx文件', sheetname='工作表名', endcoding='utf-8')
df = pd.read_excel('xlsx文件', sheetname=0, endcoding='utf-8',index_col=0)
##
    sheetname用于指定工作表，可以是工作表名称，也可以是数字（默认为0，即第1个工作表）
## encoding用于指定编码格式，默认为utf-8
## index_col用于指定索引列，默认为0，即第一列作为索引
```

除了读取 Excel 工作簿，pandas 模块还可以读取 CSV 文件。CSV 文件本质上是一个文本文件，它仅存储数据，不能像 Excel 工作簿那样存储格式、公式、宏等信息，所以所占存储空间通常较小。CSV 文件一般用逗号分隔一系列值，既可以用 Excel 打开，也可以用文本编辑器（如“记事本”）打开。

```
# 读取csv文件
df = pd.read_csv('csv文件')

# 或者
data = pd.read_csv('csv文件.csv', delimiter=',', encoding='utf-8')
## delimiter用于指定分隔符，默认为逗号
## encoding用于指定编码格式，默认为utf-8
```

2) DataFrame.to_excel 文件写入

```
import pandas as pd

DataFrame.to_excel('xlsx文件')

# 或者
DataFrame.to_excel('xlsx文件', sheet_name='工作表名', encoding='utf-8',
    index=False)
## sheet_name用于指定工作表名称，默认为'Sheet1'
## encoding用于指定编码格式，默认为utf-8
## index=False用于不写入索引列

DataFrame.to_excel('xlsx文件',index=['A','B','C'])
```

图 51.1 index=True

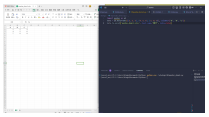

图 51.2 sheetname=’ 数字’,index=False

图 51.3 pandas 写入 Excel

二、 Sheet 工作表 (sheet)

1) 读取工作表 (sheet)

```
# 读取工作表(sheet)
df = pd.read_excel('xlsx文件', sheet_name='工作表名')
```

2) 写入工作表 (sheet)

```
# 写入工作表(sheet)
df.to_excel('xlsx文件', sheet_name='工作表名')

# 写入多个 Sheet: pd.ExcelWriter
with pd.ExcelWriter('xlsx文件') as writer:
    df1.to_excel(writer, sheet_name='Sheet1')
    df2.to_excel(writer, sheet_name='Sheet2')
```

三、 Cell 单元格

Pandas 不直接提供单元格级别操作，但可以通过 DataFrame 来间接实现：

1) 获取单元格

(1) 按行/列索引获取单元格

1) `df['列名']`: 获取整列数据
2) `df[['列名1', '列名2']]`: 获取多列数据
3) `df.iloc[行索引]`: 获取整行数据
4) `df.iloc[[行索引1, 行索引2]]`: 获取多行数据
5) `df.head(n)`: 获取前 n 行数据

(2) 按区块获取单元格

1) `df['列名'][行索引]`: 先根据列名获取列数据，再根据行索引获取单元格数据
2) `df.iloc[行索引][列索引]`: 先根据行索引获取行数据，再根据列索引获取单元格数据

2) 数据筛选

```
import pandas as pd
df = pd.read_excel('xlsx文件')
df[df['列名'] > 0] # 筛选列名大于0的行

# 多条件筛选
df[(df['列名1'] > 0) & (df['列名2'] > 0)] # 筛选列名1和列名2都大于0的行
```

3) sort_values 排序

```
import pandas as pd
df = pd.read_excel('xlsx文件')
df.sort_values('列名', ascending=True) # 升序排序
df.sort_values('列名', ascending=False) # 降序排序
```

4) 数据运算

```
import pandas as pd
df = pd.read_excel('xlsx文件')
df['新的列名'] = df['列名1'] + df['列名2'] # 列名1和列名2相加
df['新的列名'] = df['列名1'] - df['列名2'] # 列名1和列名2相减
df['新的列名'] = df['列名1'] * df['列名2'] # 列名1和列名2相乘
df['新的列名'] = df['列名1'] / df['列名2'] # 列名1和列名2相除
```

5) drop 删除行/列

```
import pandas as pd
df = pd.read_excel('xlsx文件')
df.drop(0) # 删除第一行
df.drop([0, 1]) # 删除第一行和第二行
df.drop('列名') # 删除列名对应的列
```

6) 数据表的拼接

(1) merge 拼接

`merge()`函数可以根据一个或多个同名的列将不同数据表中的行连接起来。

```
pd.merge(left, right, how='inner', on='key_column')
# left和right是要连接的两个数据表
#
    how用于指定连接方式，inner（内连接）、outer（外连接）、left（左连接）、right（右连接）
# on用于指定连接的列名
```

```
1  import pandas as pd
2
3  # 两个数据表
4  df1 = pd.DataFrame({'ID': [1, 2, 3], 'Name': ['Alice', 'Bob',
       'Charlie']})
5  df2 = pd.DataFrame({'ID': [2, 3, 4], 'Score': [85, 90, 95]})
6  print("df1:{}\n".format(df1))
7  print("df2:{}\n".format(df2))
8
9
10 # 按 ID 列进行内连接
11 result = pd.merge(df1, df2, how='inner', on='ID')
12 print(result)
```

```
df1:   ID     Name
0   1    Alice
1   2      Bob
2   3  Charlie

df2:   ID  Score
0   2     85
1   3     90
2   4     95
```

```
   ID     Name  Score
0   2      Bob     85
1   3  Charlie     90
```

(2) concat 全连接

`concat()`函数使用全连接（UNION ALL）方式完成拼接，它不需要对齐，而是直接进行合并，即不需要两个表有相同的列或索引，只是把数据整合到一起。

因此，该函数没有参数`how`和`on`，而是用参数`axis`指定连接的轴向。该参数默认值为 0，指按行方向连接（纵向拼接）。

```
pd.concat([df1, df2], axis=0)
# axis=0表示按行方向连接（纵向拼接）
# axis=1表示按列方向连接（横向拼接）
```

```
import pandas as pd
# 两个数据表
df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
df2 = pd.DataFrame({'A': [5, 6], 'B': [7, 8]})
print("df1:{}\n".format(df1))
print("df2:{}\n".format(df2))

# 按行拼接
result_row = pd.concat([df1, df2], axis=0, ignore_index=True)

# 按列拼接
result_col = pd.concat([df1, df2], axis=1)
print("按行拼接:")
print(result_row)
print("\n按列拼接:")
print(result_col)
```

```
df1:   A  B
0  1  3
1  2  4

df2:   A  B
0  5  7
1  6  8

按行拼接:
   A  B
```

```
0  1  3
1  2  4
2  5  7
3  6  8

按列拼接:
   A  B  A  B
0  1  3  5  7
1  2  4  6  8
```

(3) append 纵向拼接

`append()` 是 `concat()` 的简化版本，通常用于按行追加数据 (`axis=0`)。

```
df1.append(df2, ignore_index=True)
# ignore_index=True表示忽略原索引，生成新的索引
```

```
1  import pandas as pd
2  # 两个数据表
3  df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
4  df2 = pd.DataFrame({'A': [5, 6], 'B': [7, 8]})
5  print("df1:{}\n".format(df1))
6  print("df2:{}\n".format(df2))
7  result = df1._append(df2, ignore_index=True)
8  print(result)
9  result = df1._append(df2, ignore_index=False)
10 print(result)
```

```
df1:   A  B
0  1  3
1  2  4

df2:   A  B
0  5  7
1  6  8

   A  B
0  1  3
1  2  4
2  5  7
3  6  8

   A  B
0  1  3
1  2  4
```

```
0  5  7
1  6  8
```

第五十二章 [麻烦] xlrd、xlwt 与 xlutils 库

1) `xlrd`:
支持 Excel 读取的扩展库
支持 `xls`和`xlsx`格式的文件

2) `xlwt`:
支持 Excel 写入的扩展库
仅支持 `xls`格式的文件

3) `xlutils`
用于 Excel 修改的扩展库
仅支持 `xls`格式的文件
而且需要安装 `xlrd` 和 `xlwt` 才能使用
`xlutils` 是一个用于读取、写入和复制 Excel 文件的库，它是 `xlrd` 和 `xlwt` 的结合体。

xlutils 依赖于 xlrd 与 xlwt, 它最常用的功能就是将 xlrd 的 Book 对象复制成 xlwt 的 Workbook 对象，从而实现 xlrd 与 xlwt 之间的数据流通，它起到了“桥梁”的作用。

☞ `xlrd` 和 `xlwt` 分别是读取和写入，但是如果需要修改工作簿中的数据，就需要使用 `xlutils` 库。
如果只靠 `xlrd` 和 `xlwt` 来修改工作簿中的数据，其过程会很复杂，需要通过 xlrd 获取工作簿中所有的数据，然后通过 xlwt 建立新的工作簿，再将 xlrd 读取的数据写入，在写入的过程中修改数据

一、 安装

```
# pip 安装
pip install xlrd
pip install xlwt

# conda 安装
conda install xlrd
conda install xlwt
# 在安装xlutils之前，需要先安装xlrd与xlwt
# pip 安装
pip install xlutils

# conda 安装
```

```
conda install xlutils
```

二、 读取 Excel 内容

```
# xlrd 的函数
xlrd.open_workbook(file)              # 打开文件
xlrd.open_workbook().sheets()         # 获取工作簿中所有的sheet
xlrd.open_workbook().sheet_names()    # 获取sheet的名字
xlrd.open_workbook().sheets_count()   # 获取sheet的个数
xlrd.open_workbook().sheet_by_index() # 根据索引获取sheet
xlrd.open_workbook().sheet_by_name('Sheet name')  # 根据名字获取sheet
xlrd.open_workbook().sheets().nrows   # 获取sheet的行数
xlrd.open_workbook().sheets().ncols   # 获取sheet的列数
xlrd.open_workbook().sheets().cell_value(rowx, colx) # 获取单元格的值
```

```
import xlrd

file = r"C:\Users\tony\Documents\Pythonauto\excel\example.xls"
## r :告诉Python不对反斜杠进行转义(windows独有)
## 这里要使用xls格式的文件，如果是xlsx格式的文件，需要使用openpyxl库

wb = xlrd.open_workbook(file)
# 获取全部的sheet
sheets = xlrd.open_workbook(file).sheets()
print(sheets)

# 使用wb.sheet_by_index() 来获取sheet
sheet = wb.sheet_by_index(0)
# 打印sheet的名字
print("sheet 的名字是", sheet.name)
# 打印sheet的行数
print("sheet 的行数是", sheet.nrows)
# 打印sheet的列数
print("sheet 的列数是", sheet.ncols)
# 打印第一行的值
print("第一行的值是", sheet.row_values(0))
# 打印第一列的值
print("第一列的值是", sheet.col_values(0))
# 打印第一行第一列的值
print("第一行第一列的值是", sheet.cell_value(0, 0))

# 使用wb.sheets() 来获取sheet
table = wb.sheets()[1]
print("sheet 的名字是", table.name)
print("sheet 的行数是", table.nrows)
```

```
print("sheet 的列数是", table.ncols)
print("第一行的值是", table.row_values(0))
print("第一列的值是", table.col_values(0))
# 打印第二行第三列的值
print("第二行第三列的值是", table.cell_value(1, 2))

# 使用wb.sheet_by_name() 来获取sheet
sheets_teacher = wb.sheet_by_name("教师")
sheets_tea_nrows = sheets_teacher.nrows
sheets_tea_ncols = sheets_teacher.ncols
for i in range(sheets_tea_nrows):
    for j in range(sheets_tea_ncols):
        print(sheets_teacher.cell_value(i, j), end="\t")
    print()
```

三、 写入 Excel 内容

```
# xlwt 的函数
## Workbook() 创建一个工作簿
## add_sheet() 创建一个工作表
## write() 写入数据
## save() 保存数据
```

```
import xlwt

file = r"C:\Users\tony\Documents\Pythonauto\excel\example1.xls"

# 创建一个workbook对象，相当于创建一个Excel文件
wb = xlwt.Workbook()
# 创建一个sheet对象，相当于创建一个sheet
sheet = wb.add_sheet("第一个sheet")

# 写入数据
# 写入第一行
sheet.write(0, 0, "姓名")
sheet.write(0, 1, "年龄")
sheet.write(0, 2, "性别")
# 写入第二行
sheet.write(1, 0, "张三")
sheet.write(1, 1, 18)
sheet.write(1, 2, "男")

# 创建第二个sheet
```

```
21 sheet1 = wb.add_sheet("第二个sheet")
22 # 写入数据
23 # 写入第一行
24 sheet1.write(0, 0, "姓名")
25 sheet1.write(0, 1, "年龄")
26 sheet1.write(0, 2, "性别")
27 # 写入第二行
28 sheet1.write(1, 0, "李四")
29 sheet1.write(1, 1, 20)
30 sheet1.write(1, 2, "女")
31
32 # 保存文件
33 wb.save(file
```

四、 类型转换

当读取 Excel 文件时，xlrd库会将所有的数据都当做字符串来处理，如果需要将数据转换为其他类型，可以使用int()、float()等函数进行转换。

```
#Python
    中没有与工作簿中日期格式相对应的数据类型，所以日期格式的数据会被读取为浮点数
# 如果直接使用 cell_value 会将日期转换为浮点数，需要使用 xldate_as_tuple()
    函数将日期转换为元组或者使用 xldate_as_datetime() 函数将日期转换为
    datetime 对象

# xldate_as_tuple() 函数将日期转换为元组
time_tuple = xlrd.xldate_as_tuple(cell.value, workbook.datemode) #
    将日期转换为元组
# 将日期元组转换为字符串
time_str = datetime(*time_tuple).strftime('%Y-%m-%d')

# xldate_as_datetime() 函数将日期转换为 datetime 对象
time_value = xlrd.xldate_as_datetime(cell.value, workbook.datemode) #
    将日期转换为datetime对象
# 将日期对象转换为字符串
time_str = time_value.strftime('%Y-%m-%d')
## datemode 参数指示日期模式，它有两个可能的值:
## 0: 1900年作为基准，Excel for Windows的默认模式,Excel日期值减去1
## 1: 1904年作为基准，Excel for Mac的默认模式,Excel日期值减去1

# 如果单元格的数据类型是布尔类型，可以使用 cell.ctype ==
    xlrd.XL_CELL_BOOLEAN 来判断
if cell.ctype == xlrd.XL_CELL_BOOLEAN:
    print(cell.value)
    sheet.write(row, col, xlwt.Formula('TRUE')) # 写入布尔值

# easyxf() 函数可以设置单元格的样式
xlwt.easyxf(num_format_str='YYYY-MM-DD') # 设置日期格式以 "年-月-日"
    的形式显示
```

```
xlwt.easyxf(num_format_str='BOOLEAN') # 设置布尔格式，显示为 TRUE 或 FALSE
```

```
import xlwt, xlrd
from datetime import datetime

# 创建一个新的Excel工作簿
workbook = xlwt.Workbook()

# 创建第一个工作表，包含字符串和数字
sheet1 = workbook.add_sheet('包含字符串和数字')
sheet1.write(0, 0, 'Name')
sheet1.write(0, 1, 'Age')
sheet1.write(1, 0, 'Alice')
sheet1.write(1, 1, 30)
sheet1.write(2, 0, 'Bob')
sheet1.write(2, 1, 25)

# 创建第二个工作表，包含日期类型
sheet2 = workbook.add_sheet('包含日期')
sheet2.write(0, 0, 'Event')
sheet2.write(0, 1, 'Date')
date_style = xlwt.easyxf(num_format_str='YYYY-MM-DD')
sheet2.write(1, 0, 'Conference')
sheet2.write(1, 1, datetime(2023, 6, 15), date_style)
sheet2.write(2, 0, 'Meeting')
sheet2.write(2, 1, datetime(2023, 7, 20), date_style)

# 创建第三个工作表，包含布尔类型
sheet3 = workbook.add_sheet('包含布尔值')
sheet3.write(0, 0, 'Task')
sheet3.write(0, 1, 'Completed')
sheet3.write(1, 0, 'Task1')
sheet3.write(1, 1, xlwt.Formula('TRUE'))
sheet3.write(2, 0, 'Task2')
sheet3.write(2, 1, xlwt.Formula('FALSE'))

# 保存工作簿
workbook.save('不同类型的excel.xls')

# 打开之前创建的Excel文件
workbook = xlrd.open_workbook('不同类型的excel.xls')

# 读取第一个工作表
sheet1 = workbook.sheet_by_name('包含字符串和数字')
print("Sheet1 ", sheet1.name, "Data:")
for row in range(sheet1.nrows):
    for col in range(sheet1.ncols):
```

```
46          print(sheet1.cell_value(row, col), end=' ')
47      print()
48
49  # 读取第二个工作表
50  sheet2 = workbook.sheet_by_name('包含日期')
51  print("\nSheet2 ", sheet2.name, "Data:")
52  for row in range(sheet2.nrows):
53      for col in range(sheet2.ncols):
54          cell = sheet2.cell(row, col)
55          if cell.ctype == xlrd.XL_CELL_DATE:
56              #date_value = xlrd.xldate_as_tuple(cell.value,
    workbook.datemode) # 将日期转换为元组
57              # print(datetime(*date_value).strftime('%Y-%m-%d'),
    end=' ') # 将日期元组转换为字符串
58              # 或者使用
59              data_value = xlrd.xldate_as_datetime(cell.value,
    workbook.datemode) # 将日期转换为datetime对象
60              print(data_value.strftime('%Y-%m-%d'), end=' ')
61          else:
62              print(cell.value, end=' ')
63      print()
64
65  # 读取第三个工作表
66  sheet3 = workbook.sheet_by_name('包含布尔值')
67  print("\nSheet3 ", sheet3.name, "Data:")
68  for row in range(sheet3.nrows):
69      for col in range(sheet3.ncols):
70          print(sheet3.cell_value(row, col), end=' ')
71      print()
```

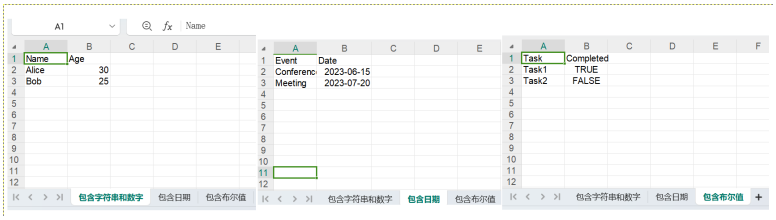

图 52.1 数据类型转换结果演示

```
Sheet1  包含字符串和数字 Data:
Name Age
Alice 30.0
Bob 25.0

Sheet2  包含日期 Data:
Event Date
Conference 2023-06-15
```

```
Meeting 2023-07-20

Sheet3  包含布尔值 Data:
Task Completed
Task1
Task2
```

五、 合并单元格

```
# 使用write_merge() 函数可以合并单元格
## write_merge(row1, row2, col1, col2, value, style)
## row1: 起始行
## row2: 结束行
## col1: 起始列
## col2: 结束列
## value: 写入的值
## style: 单元格样式
```

```python
import xlrd
import xlwt
from xlutils.copy import copy

# 打开现有的 Excel 文件
input_file = 'example.xls'
workbook = xlrd.open_workbook(input_file, formatting_info=True)
sheet = workbook.sheet_by_index(0)

# 创建一个新的工作簿，并将现有工作簿的内容复制到新的工作簿中
new_workbook = copy(workbook)
new_sheet = new_workbook.get_sheet(0)

# 合并单元格并设置内容
# 合并第2行第2列到第2行第4列的单元格（索引从0开始）
new_sheet.write_merge(1, 1, 1, 3, 'Merged Cell Content')

# 保存修改后的工作簿
output_file = 'merged_example.xls'
new_workbook.save(output_file)

print(f"合并单元格后的 Excel 文件已保存为 {output_file}")
```

六、 使用 Excel 函数

```
# 使用 xlwt.Formula() 函数可以在单元格中使用 Excel 函数
## 以下是一些常用的 Excel 函数
## SUM(): 求和
## AVERAGE(): 求平均值
## MAX(): 求最大值
## MIN(): 求最小值
## COUNT(): 计数
## IF(): 条件判断
## VLOOKUP(): 垂直查找
## HLOOKUP(): 水平查找
## CONCATENATE(): 连接字符串
## LEFT(): 提取字符串左边的字符
## RIGHT(): 提取字符串右边的字符
## MID(): 提取字符串中间的字符
## UPPER(): 将字符串转换为大写
## LOWER(): 将字符串转换为小写
## TRIM(): 删除字符串两端的空格
## LEN(): 计算字符串的长度
## DATE(): 返回日期
## TIME(): 返回时间
## NOW(): 返回当前日期和时间
## RAND(): 返回随机数
## PI(): 返回圆周率
## SQRT(): 求平方根
## POWER(): 求幂
## LOG(): 求对数
```

```python
import xlwt
from datetime import datetime

# 创建一个新的Excel工作簿
workbook = xlwt.Workbook()

# 数据样本
sample_data = [1, 2, 3, 4, 5]
lookup_data = [("A", 100), ("B", 200)]

# 演示SUM函数
sheet = workbook.add_sheet('演示SUM函数')
for row, value in enumerate(sample_data, start=1):
    sheet.write(row, 0, value)
sheet.write(0, 0, 'SUM')
sheet.write(0, 1, 'SUM(A2:A6)')
sheet.write(1, 2, xlwt.Formula('SUM(A2:A6)'))

# 演示AVERAGE函数
```

```
sheet = workbook.add_sheet('演示AVERAGE函数')
for row, value in enumerate(sample_data, start=1):
    sheet.write(row, 0, value)
sheet.write(0, 0, 'AVERAGE')
sheet.write(0, 1, 'AVERAGE(A2:A6)')
sheet.write(1, 2, xlwt.Formula('AVERAGE(A2:A6)'))

# 演示MAX函数
sheet = workbook.add_sheet('演示MAX函数')
for row, value in enumerate(sample_data, start=1):
    sheet.write(row, 0, value)
sheet.write(0, 0, 'MAX')
sheet.write(0, 1, 'MAX(A2:A6)')
sheet.write(1, 2, xlwt.Formula('MAX(A2:A6)'))

# 演示MIN函数
sheet = workbook.add_sheet('演示MIN函数')
for row, value in enumerate(sample_data, start=1):
    sheet.write(row, 0, value)
sheet.write(0, 0, 'MIN')
sheet.write(0, 1, 'MIN(A2:A6)')
sheet.write(1, 2, xlwt.Formula('MIN(A2:A6)'))

# 演示COUNT函数
sheet = workbook.add_sheet('演示COUNT函数')
for row, value in enumerate(sample_data, start=1):
    sheet.write(row, 0, value)
sheet.write(0, 0, 'COUNT')
sheet.write(0, 1, 'COUNT(A2:A6)')
sheet.write(1, 2, xlwt.Formula('COUNT(A2:A6)'))

# 演示IF函数
sheet = workbook.add_sheet('演示IF函数')
sheet.write(1, 0, 5)
sheet.write(0, 0, 'IF')
sheet.write(0, 1, 'IF(A2>10, "Greater than 10", "10 or less")')
sheet.write(1, 2, xlwt.Formula('IF(A2>10, "Greater than 10", "10
    or less")'))

# 演示VLOOKUP函数
sheet = workbook.add_sheet('演示VLOOKUP函数')
for row, (key, value) in enumerate(lookup_data, start=6):
    sheet.write(row, 0, key)
    sheet.write(row, 1, value)
sheet.write(0, 0, 'VLOOKUP')
sheet.write(0, 1, 'VLOOKUP("B", $A$6:$B$7, 2, FALSE)')
sheet.write(1, 2, xlwt.Formula('VLOOKUP("B", $A$6:$B$7, 2,
    FALSE)'))

# 演示HLOOKUP函数
```

```
68  sheet = workbook.add_sheet('演示HLOOKUP函数')
69  for col, (key, value) in enumerate(lookup_data, start=6):
70      sheet.write(0, col, key)
71      sheet.write(1, col, value)
72  sheet.write(0, 0, 'HLOOKUP')
73  sheet.write(0, 1, 'HLOOKUP("B", $A$6:$B$7, 2, FALSE)')
74  sheet.write(2, 2, xlwt.Formula('HLOOKUP("B", $A$6:$B$7, 2,
        FALSE)'))
75
76  # 演示CONCATENATE函数
77  sheet = workbook.add_sheet('演示CONCATENATE函数')
78  sheet.write(1, 0, 'Hello')
79  sheet.write(1, 1, 'World')
80  sheet.write(0, 0, 'CONCATENATE')
81  sheet.write(0, 1, 'CONCATENATE(A2, " ", B2)')
82  sheet.write(1, 2, xlwt.Formula('CONCATENATE(A2, " ", B2)'))
83
84  # 演示LEFT函数
85  sheet = workbook.add_sheet('演示LEFT函数')
86  sheet.write(1, 0, 'Hello')
87  sheet.write(0, 0, 'LEFT')
88  sheet.write(0, 1, 'LEFT(A2, 3)')
89  sheet.write(1, 2, xlwt.Formula('LEFT(A2, 3)'))
90
91  # 演示RIGHT函数
92  sheet = workbook.add_sheet('演示RIGHT函数')
93  sheet.write(1, 0, 'Hello')
94  sheet.write(0, 0, 'RIGHT')
95  sheet.write(0, 1, 'RIGHT(A2, 3)')
96  sheet.write(1, 2, xlwt.Formula('RIGHT(A2, 3)'))
97
98  # 演示MID函数
99  sheet = workbook.add_sheet('演示MID函数')
100 sheet.write(1, 0, 'HelloWorld')
101 sheet.write(0, 0, 'MID')
102 sheet.write(0, 1, 'MID(A2, 2, 3)')
103 sheet.write(1, 2, xlwt.Formula('MID(A2, 2, 3)'))
104
105 # 演示UPPER函数
106 sheet = workbook.add_sheet('演示UPPER函数')
107 sheet.write(1, 0, 'hello')
108 sheet.write(0, 0, 'UPPER')
109 sheet.write(0, 1, 'UPPER(A2)')
110 sheet.write(1, 2, xlwt.Formula('UPPER(A2)'))
111
112 # 演示LOWER函数
113 sheet = workbook.add_sheet('演示LOWER函数')
114 sheet.write(1, 0, 'HELLO')
115 sheet.write(0, 0, 'LOWER')
116 sheet.write(0, 1, 'LOWER(A2)')
```

```
117 sheet.write(1, 2, xlwt.Formula('LOWER(A2)'))
118
119 # 演示TRIM函数
120 sheet = workbook.add_sheet('演示TRIM函数')
121 sheet.write(1, 0, '   Hello   ')
122 sheet.write(0, 0, 'TRIM')
123 sheet.write(0, 1, 'TRIM(A2)')
124 sheet.write(1, 2, xlwt.Formula('TRIM(A2)'))
125
126 # 演示LEN函数
127 sheet = workbook.add_sheet('演示LEN函数')
128 sheet.write(1, 0, 'Hello')
129 sheet.write(0, 0, 'LEN')
130 sheet.write(0, 1, 'LEN(A2)')
131 sheet.write(1, 2, xlwt.Formula('LEN(A2)'))
132
133 # 演示DATE函数
134 sheet = workbook.add_sheet('演示DATE函数')
135 sheet.write(0, 0, 'DATE')
136 sheet.write(0, 1, 'DATE(2023, 6, 15)')
137 sheet.write(1, 2, xlwt.Formula('DATE(2023, 6, 15)'))
138
139 # 演示TIME函数
140 sheet = workbook.add_sheet('演示TIME函数')
141 sheet.write(0, 0, 'TIME')
142 sheet.write(0, 1, 'TIME(12, 0, 0)')
143 sheet.write(1, 2, xlwt.Formula('TIME(12, 0, 0)'))
144
145 # 演示NOW函数
146 sheet = workbook.add_sheet('演示NOW函数')
147 sheet.write(0, 0, 'NOW')
148 sheet.write(0, 1, 'NOW()')
149 sheet.write(1, 2, xlwt.Formula('NOW()'))
150
151 # 演示RAND函数
152 sheet = workbook.add_sheet('演示RAND函数')
153 sheet.write(0, 0, 'RAND')
154 sheet.write(0, 1, 'RAND()')
155 sheet.write(1, 2, xlwt.Formula('RAND()'))
156
157 # # 演示RANDBETWEEN函数
158 # sheet = workbook.add_sheet('演示RANDBETWEEN函数')
159 # sheet.write(0, 0, 'RANDBETWEEN')
160 # sheet.write(0, 1, 'RANDBETWEEN(1, 100)')
161 # sheet.write(1, 2, xlwt.Formula('RANDBETWEEN(1, 100)'))
162
163 # 演示PI函数
164 sheet = workbook.add_sheet('演示PI函数')
165 sheet.write(0, 0, 'PI')
166 sheet.write(0, 1, 'PI()')
```

```
sheet.write(1, 2, xlwt.Formula('PI()'))

# 演示SQRT函数
sheet = workbook.add_sheet('演示SQRT函数')
sheet.write(1, 0, 16)
sheet.write(0, 0, 'SQRT')
sheet.write(0, 1, 'SQRT(A2)')
sheet.write(1, 2, xlwt.Formula('SQRT(A2)'))

# 演示POWER函数
sheet = workbook.add_sheet('演示POWER函数')
sheet.write(1, 0, 2)
sheet.write(0, 0, 'POWER')
sheet.write(0, 1, 'POWER(A2, 2)')
sheet.write(1, 2, xlwt.Formula('POWER(A2, 2)'))

# 演示LOG函数
sheet = workbook.add_sheet('演示LOG函数')
sheet.write(1, 0, 100)
sheet.write(0, 0, 'LOG')
sheet.write(0, 1, 'LOG(A2)')
sheet.write(1, 2, xlwt.Formula('LOG(A2)'))

# 保存工作簿
workbook.save('excel函数演示.xls')

import xlrd
# 打开已创建的Excel文件
workbook = xlrd.open_workbook('excel函数演示.xls')

# 读取每个工作表的数据
sheet_names = [
    '演示SUM函数', '演示AVERAGE函数', '演示MAX函数', '演示MIN函数',
    '演示COUNT函数',
    '演示IF函数', '演示VLOOKUP函数', '演示HLOOKUP函数',
    '演示CONCATENATE函数', '演示LEFT函数',
    '演示RIGHT函数', '演示MID函数', '演示UPPER函数',
    '演示LOWER函数', '演示TRIM函数',
    '演示LEN函数', '演示DATE函数', '演示TIME函数', '演示NOW函数',
    '演示RAND函数',
    '演示PI函数', '演示SQRT函数', '演示POWER函数', '演示LOG函数'
]

for sheet_name in sheet_names:
    sheet = workbook.sheet_by_name(sheet_name)
    print(f"\n{sheet_name} Data:")
    for row in range(sheet.nrows):
        for col in range(sheet.ncols):
            cell = sheet.cell(row, col)
            if cell.ctype == 2:  # xlrd.XL_CELL_NUMBER
```

```
213                 print(f"Number: {cell.value}", end=' ')
214             elif cell.ctype == 3:  # xlrd.XL_CELL_DATE
215                 date_value = xlrd.xldate_as_tuple(cell.value,
    workbook.datemode)
216                 print(f"Date: {datetime(*date_value)}", end=' ')
217             elif cell.ctype == 4:  # xlrd.XL_CELL_BOOLEAN
218                 print(f"Boolean: {cell.value}", end=' ')
219             elif cell.ctype == 5:  # xlrd.XL_CELL_ERROR
220                 print(f"Error: {cell.value}", end=' ')
221             elif cell.ctype == 6:  # xlrd.XL_CELL_FORMULA
222                 print(f"Formula: {cell.value}", end=' ')
223             else:  # xlrd.XL_CELL_TEXT (1) or xlrd.XL_CELL_EMPTY
    (0)
224                 print(cell.value, end=' ')
225         print()
```

七、 复制现有的 Excel 文件并添加新的数据

```
# xlutils 的函数
## copy() 复制工作簿
## add_sheet() 添加工作表
## save() 保存工作簿
```

```
1  import xlrd
2  import xlwt
3  from xlutils.copy import copy
4
5  # 打开现有的 Excel 文件
6  input_file = 'excel/example.xls'
7  workbook = xlrd.open_workbook(input_file, formatting_info=True)
8  sheet = workbook.sheet_by_index(0)
9
10 # 创建一个新的工作簿，并将现有工作簿的内容复制到新的工作簿中
11 new_workbook = copy(workbook)
12 new_sheet = new_workbook.get_sheet(0)
13
14 # 在新的工作簿中添加新的数据
15 new_data = [
16     ['Name', 'Age', 'City'],
17     ['Alice', 30, 'New York'],
18     ['Bob', 25, 'Los Angeles'],
19     ['Charlie', 35, 'Chicago']
20 ]
```

```python

# 添加新的数据到现有工作表中
start_row = sheet.nrows  # 从现有数据的下一行开始添加新数据
for row_idx, row_data in enumerate(new_data, start=start_row):
    for col_idx, cell_value in enumerate(row_data):
        new_sheet.write(row_idx, col_idx, cell_value)

# 添加一个新的工作表
additional_sheet = new_workbook.add_sheet('NewDataSheet')

# 在新的工作表中添加数据
additional_data = [
    ['Product', 'Price', 'Quantity'],
    ['Apple', 1.2, 50],
    ['Banana', 0.5, 100],
    ['Cherry', 2.5, 200]
]

# 添加新的数据到新的工作表中
for row_idx, row_data in enumerate(additional_data):
    for col_idx, cell_value in enumerate(row_data):
        additional_sheet.write(row_idx, col_idx, cell_value)

# 保存新的工作簿
output_file = 'new_example_with_additional_sheet.xls'
new_workbook.save(output_file)

print(f"新的 Excel 文件已保存为 {output_file}")
```

八、 合并多个 Excel 文件

```python
import xlrd
import xlwt
from xlutils.copy import copy

def read_excel_data(file_name):
    workbook = xlrd.open_workbook(file_name)
    sheet = workbook.sheet_by_index(0)
    data = []
    for row_idx in range(sheet.nrows):
        row_data = sheet.row_values(row_idx)
        data.append(row_data)
    return data
```

```
13
14 def write_data_to_sheet(sheet, data, start_row=0):
15     for row_idx, row_data in enumerate(data, start=start_row):
16         for col_idx, cell_value in enumerate(row_data):
17             sheet.write(row_idx, col_idx, cell_value)
18
19 # 读取多个 Excel 文件
20 file_names = ['file1.xls', 'file2.xls', 'file3.xls']
21 all_data = []
22 for file_name in file_names:
23     data = read_excel_data(file_name)
24     all_data.append(data)
25
26 # 创建一个新的工作簿
27 new_workbook = xlwt.Workbook()
28 new_sheet = new_workbook.add_sheet('MergedData')
29
30 # 写入数据到新的工作表中
31 start_row = 0
32 for data in all_data:
33     write_data_to_sheet(new_sheet, data, start_row)
34     start_row += len(data)
35
36 # 保存新的工作簿
37 output_file = 'merged_example.xls'
38 new_workbook.save(output_file)
39
40 print(f"新的合并后的 Excel 文件已保存为 {output_file}")
```

九、 修改现有的 Excel 文件

```
1 import xlrd
2 import xlwt
3 from xlutils.copy import copy
4
5 # 打开现有的 Excel 文件
6 input_file = 'excel/example.xls'
7 workbook = xlrd.open_workbook(input_file, formatting_info=True)
8 sheet = workbook.sheet_by_index(0)
9
10 # 创建一个新的工作簿，并将现有工作簿的内容复制到新的工作簿中
11 new_workbook = copy(workbook)
12 new_sheet = new_workbook.get_sheet(0)
```

```
13
14 # 修改现有的数据
15 # 例如，修改第1行第1列（索引从0开始）的数据为 'Modified'
16 new_sheet.write(0, 0, 'Modified')
17
18 # 添加新数据到现有的工作表中
19 new_data = [
20     ['New Name', 'New Age', 'New City'],
21     ['David', 40, 'San Francisco'],
22     ['Eva', 28, 'Seattle']
23 ]
24
25 # 确定从哪一行开始添加新数据
26 start_row = sheet.nrows  # 从现有数据的下一行开始添加新数据
27 for row_idx, row_data in enumerate(new_data, start=start_row):
28     for col_idx, cell_value in enumerate(row_data):
29         new_sheet.write(row_idx, col_idx, cell_value)
30
31 # 保存修改后的工作簿
32 output_file = 'modified_example.xls'
33 new_workbook.save(output_file)
34
35 print(f"修改后的 Excel 文件已保存为 {output_file}")
```

第五十三章 [简洁] openpyxl 库

`openpyxl` 是一个用于读取、写入和修改 Excel 文件的库，它支持 `xlsx` 格式的文件, 不用再担心格式问题。

相较于 `xlrd` 和 `xlwt`，`openpyxl` 更加强大，支持更多的功能，例如合并单元格、设置单元格样式、添加图表等。

`openpyxl`唯一的缺点就是对 Excel 中 VBA(Visual Basic Applications) 支持的并不友好。

一、 安装

```
# pip 安装
pip install openpyxl

# conda 安装
conda install openpyxl
```

二、 创建 Excel 文件

```
from openpyxl import Workbook

# 创建一个新的工作簿
wb = Workbook()

# 保存工作簿到文件
wb.save("new_excel_file.xlsx")
```

三、 读取 Excel 内容

需要注意，openpyxl 与 xlrd、xlwt 不同，cell 方法中的 row、column 参数从 1 开始，即工作表中的行号、列号与 Excel 工作表一样，都以 1 作为起始坐标，而 xlrd、xlwt 却以 0 作为起始坐标。

1) load_workbook() 打开文件

```
from openpyxl import load_workbook
# 打开一个Excel文件
wb = load_workbook('excel/example.xlsx')
```

2) sheet 表单

针对 Excel 文件中的 sheet 表单，openpyxl 提供了以下几个方法：

1) `get_sheet_names()`：获取工作簿中所有的 sheet 名称
2) `get_sheet_by_name()`：根据名字获取 sheet
3) `get_sheet_by_index()`：根据索引获取 sheet

```
## max_row 获取最大行数
## max_column 获取最大列数
## cell() 获取单元格
## cell().value 获取单元格的值
```

```
1  from openpyxl import load_workbook
2
3  # 1. 打开一个Excel文件
4  workbook = load_workbook('excel/example.xlsx')
5
6  # 2. 获取工作簿中所有的sheet名称
7  sheet_names = workbook.sheetnames
8  print(f"Sheet names: {sheet_names}")
9
10 # 3. 根据名字获取sheet
11 sheet_by_name = workbook[sheet_names[0]]
12 print(f"Sheet by name: {sheet_by_name.title}")
13
14 # 4. 根据索引获取sheet
       (openpyxl没有直接根据索引获取sheet的方法，我们通过sheetnames间接获取)
15 sheet_by_index = workbook[sheet_names[1]]
16 print(f"Sheet by index: {sheet_by_index.title}")
17
18 # 5. 获取最大行数和最大列数
19 max_row = sheet_by_name.max_row
20 max_column = sheet_by_name.max_column
21 print(f"Max row: {max_row}, Max column: {max_column}")
22
23 # 6. 获取特定单元格
```

```python
cell = sheet_by_name.cell(row=1, column=1)
print(f"Cell at (1, 1): {cell}")

# 7. 获取单元格的值
cell_value = cell.value
print(f"Value of cell at (1, 1): {cell_value}")

# 打印所有单元格值的示例
for row in sheet_by_name.iter_rows(min_row=1, max_row=max_row,
    min_col=1, max_col=max_column):
    for cell in row:
        print(f"Row {cell.row}, Column {cell.column}:
    {cell.value}")
```

```
Sheet names: ['教师', '学生', 'Sheet3']
Sheet by name: 教师
Sheet by index: 学生
Max row: 2, Max column: 3
Cell at (1, 1): <Cell '教师'.A1>
Value of cell at (1, 1): 姓名
Row 1, Column 1: 姓名
Row 1, Column 2: 性别
Row 1, Column 3: 学号
Row 2, Column 1: 贾继伟
Row 2, Column 2: 男
Row 2, Column 3: 20233120098
```

四、 写入 Excel 内容

openpyxl除可以创建新工作簿来写数据外，还可以修改已存在的工作簿，该过程不再需要使用xlutils库。

```
# openpyxl 的函数
## Workbook() 创建一个新的工作簿
## create_sheet() 创建一个新的工作表
## save() 保存工作簿
## append() 添加数据
## merge_cells() 合并单元格
## unmerge_cells() 取消合并单元格
```

```python
from openpyxl import Workbook

# 1. 创建一个新的工作簿
workbook = Workbook()
```

```python

# 2. 创建一个新的工作表
sheet = workbook.active
sheet.title = 'NewSheet1'  # 修改工作表的标题
print(f"Created new sheet with title: {sheet.title}")

# 3. 添加数据到工作表
data = [
    ["Name", "Age", "City"],
    ["Alice", 30, "New York"],
    ["Bob", 25, "Los Angeles"],
    ["Charlie", 35, "Chicago"]
]

for row in data:
    sheet.append(row)

# 4. 合并和取消合并单元格
# 合并单元格
sheet.merge_cells('A1:C1')
print("Merged cells A1 to C1")
sheet.merge_cells('A2:C2')
print("Merged cells A2 to C2")

# 保存之前取消合并单元格
sheet.unmerge_cells('A1:C1')
print("Unmerged cells A1 to C1")

# 5.在添加一个工作表
new_sheet = workbook.create_sheet(title='AnotherSheet')
print(f"Created another sheet with title: {new_sheet.title}")

new_sheet['A1'] = 'This is another sheet'
new_sheet.cell(row=2, column=1).value='贾继位'

# 5.保存工作簿
workbook.save('new_workbook.xlsx')
print("Workbook saved as new_workbook.xlsx")
```

五、 修改 Excel 文件单元格内容

如果想修改已存在的工作簿,只需将创建工作簿的`openpyxl.Workbook()`方法替换为`openpyxl.load_workbook()`方法即可。

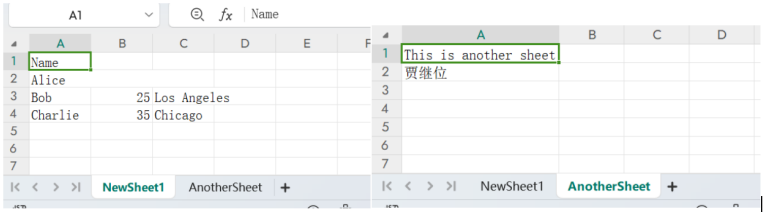

图 53.1 openpyxl 写入、合并演示

六、 修改 Excel 文件样式

`openpyxl`支持设置单元格的样式，例如字体、颜色、对齐方式等。
主要分为以下几个部分:

1) Font: 字体样式
2) PatternFill: 填充样式
3) Border: 边框样式
4) Alignment: 对齐样式

1) 基本样式

```
# openpyxl 的函数
## Font() 创建字体样式
## PatternFill() 创建填充样式
## Border() 创建边框样式
## Side() 创建边框样式
## Alignment() 创建对齐样式
## NamedStyle() 创建命名样式
## Style() 创建样式
```

```
from openpyxl import Workbook
from openpyxl.styles import Font, PatternFill, Border, Side,
    Alignment

# 1. 创建一个新的工作簿和工作表
workbook = Workbook()
sheet = workbook.active
sheet.title = "StyledSheet"

# 2. 定义字体样式
font = Font(name='Arial', size=14, bold=True, italic=False,
    color='FF0000')
# 其中颜色也可以这样写 color=colors.RED
```

```
12
13  # 3. 定义不同的PatternFill效果
14  fill_types = [
15      'solid', 'lightVertical', 'lightGray', 'gray0625',
        'darkGray', 'gray125',
16      'lightDown', 'lightHorizontal', 'darkDown', 'lightUp',
        'lightTrellis', 'darkGrid',
17      'darkTrellis', 'lightGrid', 'darkVertical', 'darkHorizontal',
        'mediumGray', 'darkUp'
18  ]
19
20  # 4. 定义边框样式和不同颜色
21  border_colors = ['FF0000', '00FF00', '0000FF', 'FFFF00',
        'FF00FF', '00FFFF']
22  border_styles = [Border(
23      left=Side(style='thin', color=color),
24      right=Side(style='thin', color=color),
25      top=Side(style='thin', color=color),
26      bottom=Side(style='thin', color=color)
27  ) for color in border_colors]
28
29  # 5. 定义对齐样式
30  alignment = Alignment(horizontal='center', vertical='center')
31
32  # 6. 将不同的PatternFill效果应用到不同单元格
33  for i, fill_type in enumerate(fill_types):
34      print(f"Applying {fill_type} fill")
35      fill = PatternFill(fill_type=fill_type, start_color='FFFF00',
        end_color='FFFF00')
36      cell = sheet.cell(row=i + 1, column=1, value=f"{fill_type}
        fill")
37      cell.font = font
38      cell.fill = fill
39      cell.border = border_styles[i % len(border_styles)]
40      cell.alignment = alignment
41
42  # 7.设置行高和列宽
43  sheet.row_dimensions[1].height = 30 # 设置第一行的行高为30
44  sheet.column_dimensions['A'].width = 20 # 设置第一列的列宽为20
45
46  # 8. 保存工作簿
47  workbook.save('styled_workbook_complete.xlsx')
48  print("Workbook saved as styled_workbook_complete.xlsx")
```

2) 自定义样式

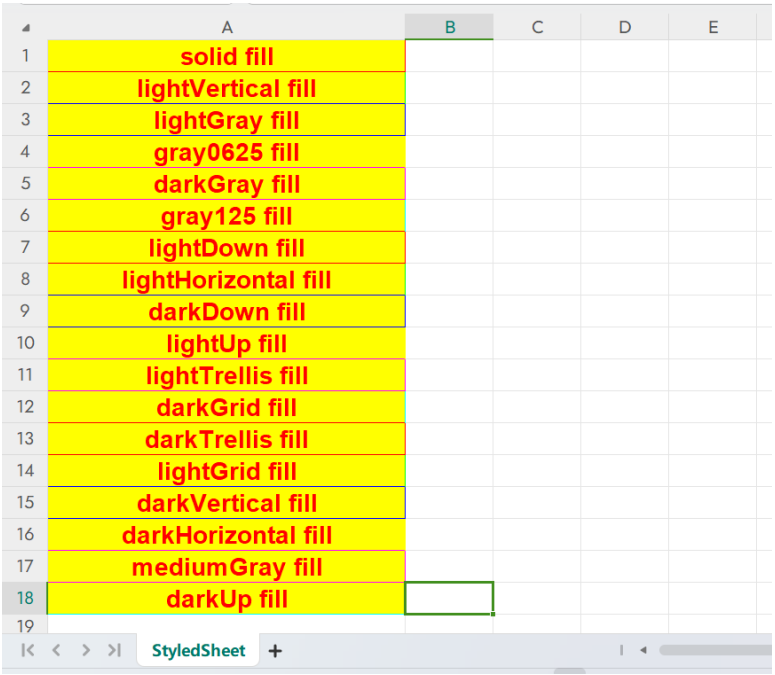

图 53.2 openpyxl 修改样式效果

```python
from openpyxl import Workbook
from openpyxl.styles import Font, PatternFill, Border, Side,
    Alignment, NamedStyle

# 1. 创建一个新的工作簿和工作表
workbook = Workbook()
sheet = workbook.active
sheet.title = "StyledSheet"

# 2. 创建字体样式
font = Font(name='Arial', size=14, bold=True, italic=False,
    color='FF0000')

# 3. 创建填充样式
fill = PatternFill(fill_type='solid', start_color='FFFF00',
    end_color='FFFF00')

# 4. 创建边框样式
side = Side(style='thin', color='000000')
border = Border(left=side, right=side, top=side, bottom=side)

# 5. 创建对齐样式
alignment = Alignment(horizontal='center', vertical='center')
```

```
# 6. 将这些样式应用到特定单元格
cell = sheet.cell(row=1, column=1, value="Styled Text")
cell.font = font
cell.fill = fill
cell.border = border
cell.alignment = alignment

# 7. 创建命名样式
named_style = NamedStyle(name="highlight")
named_style.font = Font(name='Arial', size=12, bold=True,
    color='FFFFFF')
named_style.fill = PatternFill(fill_type='solid',
    start_color='FF5733')
named_style.border = Border(left=Side(style='thin',
    color='FF5733'),
                            right=Side(style='thin',
    color='FF5733'),
                            top=Side(style='thin',
    color='FF5733'),
                            bottom=Side(style='thin',
    color='FF5733'))
named_style.alignment = Alignment(horizontal='center',
    vertical='center')

# 注册命名样式
workbook.add_named_style(named_style)

# 应用命名样式到特定单元格
cell2 = sheet.cell(row=2, column=1, value="Named Style")
cell2.style = named_style

# 8. 保存工作簿
workbook.save('styled_workbook_complete.xlsx')
print("Workbook saved as styled_workbook_complete.xlsx")
```

七、 操作大型 Excel 文件

当要处理大型工作簿时，直接将整个工作簿载人程序的操作是不合理的，数据量过大会占用过多内存，从而影响程序本身的运行速度，导致程序出现“卡顿”、闪退等情况。

`openpyxl`提供了一种更加高效的方式来处理大型工作簿,即使用`read_only`模式和`write_only`模式。

1) 在`read_only`模式：

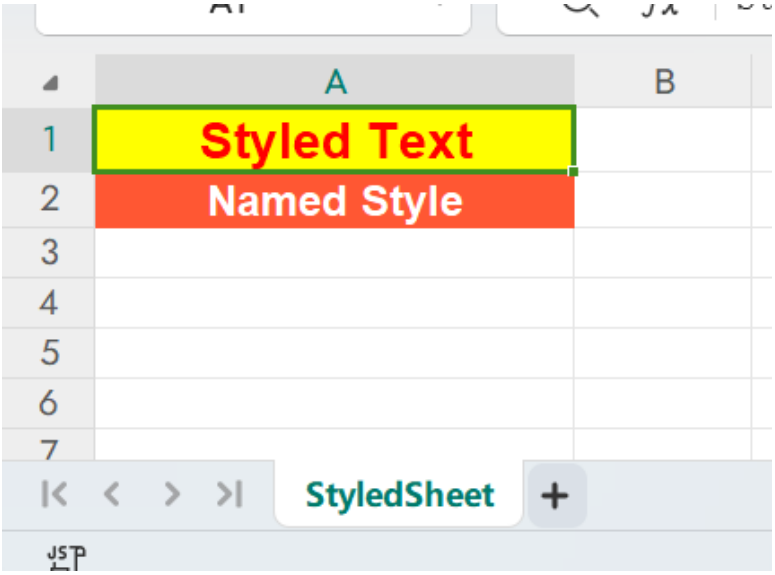

图 53.3 openpyxl 自定义样式效果

openpyxl 可以使用恒定的内存来处理无限的数据，其原理就是先处理一部分数据，处理完后，释放占用的内存，然后继续读入部分数据进行处理。这种方式可以快速读取大型工作簿中的数据。但需要注意，在该模式下，不允许对工作表进行写操作。

2) 在`write_only`模式：

openpyxl 也是通过类似的原理将大量数据分批写入工作簿中的 0 不同之处在于，该模式下不允许对工作表进行读操作。

1) read_only 模式

使用 openpyxl 的 read only 模式非常简单，只需在使用 load_workbook 方法时将 read only 参数设置为 True 即可

```
from openpyxl import load_workbook

# 在read_only模式下载入大型工作簿
wb = load_workbook('excel/large_workbook.xlsx', read_only=True)
# 选择 big_sheet1 工作表
sheet = workbook['big_sheet1']

for row in wb.iter_rows:  # 逐行读取数据
    for cell in row:      # 逐列读取数据
        print(cell.value)
```

2) write_only 模式

使用 openpyxl 的 write only 模式也非常简单，只需在使用 load_workbook 方法时将 write only 参数设置为 True 即可

```
from openpyxl import Workbook
from openpyxl.cell import WriteOnlyCell
from openpyxl.comments import Comment
from openpyxl.styles import Font

# write_only 设置为 True
wb = Workbook(write_only=True)

#     write_only模式下不会包含任何工作表，需要使用create_sheet方法自行仓U建
ws = wb.create_sheet()

#     write_on1y模式下，单元格想要具有样式，就只能使用WriteOnlyCell创建单元格
cell = WriteOnlyCell (ws, 状态写入的内容')

# 为单元格设置字体样式
cell. font = Font (name='微软雅黑', size=36)

# 插入Excel批注
cell. comment = Comment (text='这是一个批注', author='贾继伟')

# write_on1y模式下只能使用append方法添力口数据
## 在write_only模式下，不可在任意位置使用cell 或 iter_rows方法，
## 在添加数据时只能使用append方法
ws.append([cell, 2.333, None])

# 保存
wb save('write_only.xlsx')
```

第五十四章 [必须安装 Excel] xlwings 库

一、 简介

使用 xlwings必须安装 Excel 软件，xlwings 是一个用于在 Excel 中调用 Python 的库，

1) xlwings 的数据结构

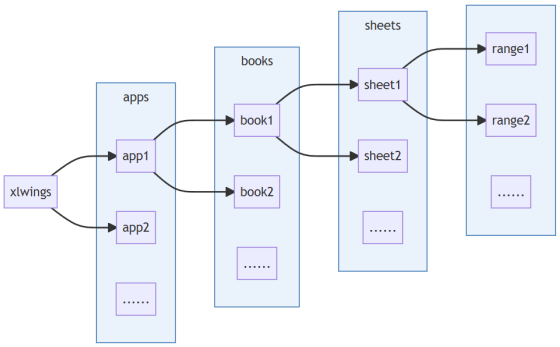

图 54.1 xlwings 的数据结构

如图54.1所示，xlwings 的数据结构主要有以下几个部分：

1) App：单个 Excel 程序 APP 可以是一个 Excel 的实例，在此实例下创建工作薄，要创建工作簿，就必须先创建 App 实例。

   APP 也可以是 Excel 程序集合, 使用 xlwings 可以创建一个或者多个 App, 多个 App 之间是相互独立的, 每个 App 对应一个 PID 值，PID 值是数字编号，可以用来识别不同的 App

2) Book：单个工作簿

3) Sheet：工作表

4) Range: 单元格范围

二、 安装

```
# pip 安装
pip install xlwings

pip install xlwings -i https://pypi.tuna.tsinghua.edu.cn/simple
# conda 安装
conda install xlwings
```

三、 App 应用程序

也正是因为 App 的存在，所以`xlwings`必须安装 Excel 软件，否则无法使用 xlwings 库。

```
import xlwings as xw
## App打开Excel
## visible = True 显示Excel界面
## add_book = False 不添加新的工作簿
app = xw.App(visible = True, add_book = False)
# app = xw.App(visible = False, add_book = False)

app.quit() # 关闭Excel
```

虽然新版的 xlwings, 可以直接对 Book 进行操作，但是功能很有限，所以还是建议使用 App 来操作 Excel。

四、 Book 工作簿 (Excel 文件)

工作簿的含义是 Excel 文件，可以通过 xlwings 创建一个新的工作簿

1) 新建/保存关闭工作簿

```
import xlwings as xw

# 1.创建一个新的工作簿
app = xw.App(visible = True, add_book = False)
## 1.2 创建一个新的工作簿
wb = app.books.add()

# 2.保存工作簿
wb.save('test.xlsx')
wb.close() # 关闭工作簿
app.quit() # 关闭Excel
```

2) 获取当前活动工作簿

```
import xlwings as xw

# 获取当前活动的工作簿
active_wb = xw.books.active
```

```
import xlwings as xw

app = xw.App(visible=True, add_book=False)
for i in range(1, 21):
    # 新建工作簿
    workbook = app.books.add()

    # 保存为特定文件名
    file_path =
    f'c:\\Users\\bingx\\Documents\\Python\\分公司{i}.xlsx'
    workbook.save(file_path)


    # 输出保存的文件路径（或名称）
    # print(f"已保存：{file_path}")

    workbook = app.books.open(file_path)  # 重新打开
    print(f"当前工作簿：{workbook.name}")

    # 关闭工作簿
    workbook.close()

# 关闭 Excel 应用
app.quit()
```

```
前工作簿：分公司1.xlsx
当前工作簿：分公司2.xlsx
当前工作簿：分公司3.xlsx
当前工作簿：分公司4.xlsx
当前工作簿：分公司5.xlsx
当前工作簿：分公司6.xlsx
当前工作簿：分公司7.xlsx
当前工作簿：分公司8.xlsx
当前工作簿：分公司9.xlsx
当前工作簿：分公司10.xlsx
当前工作簿：分公司11.xlsx
当前工作簿：分公司12.xlsx
当前工作簿：分公司13.xlsx
```

```
当前工作簿: 分公司14.xlsx
当前工作簿: 分公司15.xlsx
当前工作簿: 分公司16.xlsx
当前工作簿: 分公司17.xlsx
当前工作簿: 分公司18.xlsx
当前工作簿: 分公司19.xlsx
当前工作簿: 分公司20.xlsx
```

3) 删除工作簿

xlwings 本身不能直接删除文件，但你可以结合 Python 的 `os` 模块实现：

```
import os

# 删除文件
file_path = 'test.xlsx'
os.remove(file_path)
```

五、 Sheet 工作表 (sheet)

1) 新建工作表 (sheet)

```
import xlwings as xw

# 1.创建一个新的工作簿
app = xw.App(visible = True, add_book = False)
wb = app.books.add()

# 2.新建一个工作表
## 2.1 根据索引新建工作表
worksheet = wb.sheets[0] # 获取第一个工作表
worksheet.name = '员工信息' # 设置工作表的名称
## 2.2 在开头插入新的工作表
worksheet = wb.sheets.add('产品统计表') # 添加一个新的工作表
# 2.3在指定位置插入新的工作表
worksheet = wb.sheets.add('产品销售表', after='员工信息') #
    添加一个新的工作表

# 3.保存工作簿
wb.save('test.xlsx')
wb.close() # 关闭工作簿
app.quit() # 关闭Excel
```

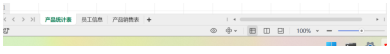

图 54.2 新建工作表演示

(1) 获取工作表 (sheet)

1) 按索引：`wb.sheets[索引值]`, 索引值从 0 开始

2) 按名称：`wb.sheets['工作表名']`

3) 当前活动工作表：`wb.sheets.active`

4) 获取所有工作表：`wb.sheets`（返回所有工作表的集合）

(2) 按索引/名称获取工作表

```
import xlwings as xw

# 打开工作簿
wb = xw.Book('example.xlsx')

# 1.获取第一个工作表（索引从 0 开始）
sheet = wb.sheets[0]
print(sheet.name)  # 打印工作表名称

# 2.获取名为 'Sheet1' 的工作表
sheet = wb.sheets['Sheet1']
print(sheet.name)

# 3.获取当前活动的工作表
active_sheet = wb.sheets.active
print(active_sheet.name)
```

2) 获取工作簿中所有工作表

```
import xlwings as xw
wb = xw.Book("xlsx路径")
sheets = wb.sheets
```

```
1  import xlwings as xw
2  #
3  wb =
       xw.Book("openpyxl/吉林2024-2021招生录取2024.11.14日姚老师校对基础表.xlsx")
4
5  # 获取工作簿中的所有工作表
6  sheets = wb.sheets
7
8  # 打印所有工作表的名称
9  print("工作簿中的所有工作表:")
10 for sheet in sheets:
```

```
    print(sheet.name)

# 关闭工作簿
wb.close()
```

```
工作簿中的所有工作表:
录取表
普通专科物理
普通专科历史
普通本科物理
普通本科历史
提前A物理
提前A历史
查找表
错误名称表
```

图 54.3 获取工作簿中所有工作表

3) 删除工作表 (sheet)

1) 先获取工作表对象

2) 然后调用 `.delete()` 方法

```
# 删除指定工作表
sheet_to_delete = wb.sheets['Sheet1']
sheet_to_delete.delete()
```

4) 重命名工作表

1) 先获取工作表对象

2) 然后调用 `.name` 属性

```
# 重命名工作表
sheet_to_rename = wb.sheets['Sheet1']
sheet_to_rename.name = '新名词'
```

六、 Range 区域/单元格

具体对某个 Sheet 中行、列、单元格的操作，都是通过 `sheet` 对象来实现的。

1) 获取单元格

使用`sheet.range`方法获取单元格，`range`方法的参数可以是单元格的坐标，也可以是单元格的范围。

```
# 获取单个单元格
cell = sheet.range('A1')

# 获取多个单元格
cells = sheet.range('A1:B3')
```

1) 如果是单个单元格，返回的是 `Range` 对象
2) 如果多个单元格是一行/列，返回的是该行/列的 `Range` 对象
3) 如果多个单元格是一个矩形区域，返回的是该矩形区域的 `Range` 对象，如图54.4所示

如果要获取单元格的值，可以使用 `range.value` 属性：

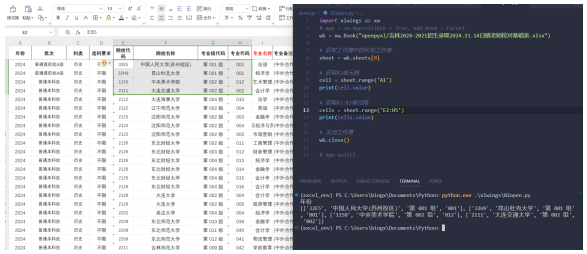

图 54.4 range 获取某一区间

```
# 读取整行
row_values = sheet.range('1:1').value

# 读取整列
col_values = sheet.range('A:A').value
```

2) 清除单元格

清除单元格的方法是`range.clear_contents()`,仅清除单元格内容（文字、数字、公式等）。不改变工作表的结构（例如行列位置保持不变）。

```python
import xlwings as xw

app = xw.App(visible=False, add_book=False)
wb = xw.Book("openpyxl/吉林2024-2021招生录取2024.11.14日姚老师校对基础表.xlsx")
sheet = wb.sheets[0]

# 清空A1:C3范围内的单元格
sheet.range('A1:C3').clear_contents()

wb.save("openpyxl/clear_contents.xlsx")
wb.close()
app.quit()
```

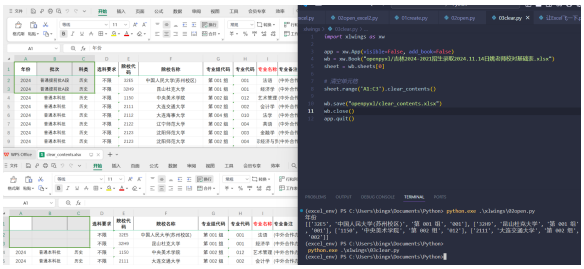

图 54.5 清空单元格演示

3) 删除单元格

删除单元格的方法是`range.delete(shift=None)`,删除单元格后，影响到整个工作表的结构。可选择向上或向左移动其余单元格。

1) (**不推荐**)`shift=None`: 默认值，删除单元格后，不会移动其余单元格，但不一定有时候会移动其余单元格，不推荐使用

2) `shift='up'`: 删除单元格后，向上移动其余单元格

3) `shift='left'`: 删除单元格后，向左移动其余单元格

```python
# 删除一行
sheet.range('1:1').delete()

# 删除一列
sheet.range('A:A').delete()
```

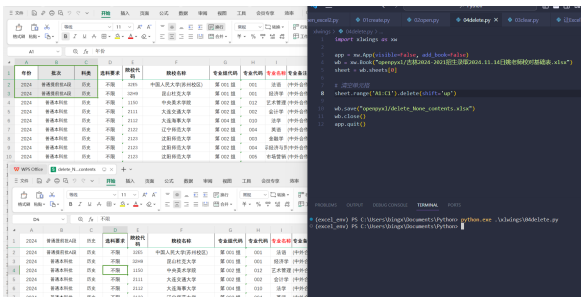

图 54.6 shift='up' 效果

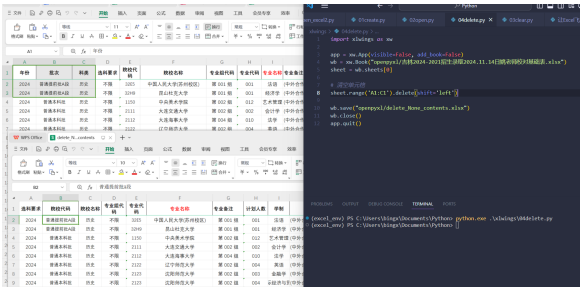

图 54.7 shift='left' 效果

4) 插入单元格

插入单元格的方法是`range.insert(shift=None)`,插入单元格后，影响到整个工作表的结构。可选择向下或向右移动其余单元格。

1) (不推荐)`shift=None`: 默认值，插入单元格后，不会移动其余单元格，但不一定有时候会移动其余单元格，不推荐使用

2) `shift='down'`: 插入单元格后，向下移动其余单元格

3) `shift='right'`: 插入单元格后，向右移动其余单元格

```
# 插入一行
sheet.range('1:1').insert()

# 插入一列
sheet.range('A:A').insert()
```

七、 xlwings 与 panads 的结合

xlwings 模块可以与 pandas 模块进行交互。

例如，用 pandas 模块创建数据表格，再用 xlwings 模块将表格写入工作簿，演示代码如下：

```
import xlwings as xw
import pandas as pd
app = xw.App(visible=False)

# 创建一个新的工作簿
workbook = app.books.add()
worksheet = workbook.sheets.add('新工作表')

# 将DataFrame写入Excel
df = pd.DataFrame([[1, 2], [3, 4]], columns=['a', 'b'])
worksheet.range('B3').value = df # 在A1单元格写入DataFrame

# 保存工作簿
workbook.save(r'table.xlsx')
workbook.close()
app.quit()
```

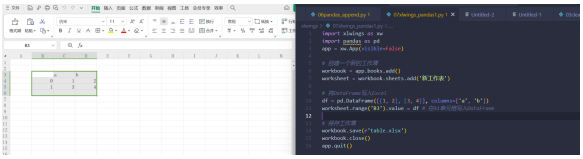

图 54.8 xlwings 与 pandas 结合 1

第五十五章 word 自动化

一、 docx 库

python-docx 是 Python 专门用来编辑 Word 文档的库，使用它的好处就是不必自己研究 docx 文件类型的底层实现细节，你可以像操作.txt 文本一样直接打开、修改和保存关闭文件。可以说，python-docx 扩展库降低了用户使用 Python 的复杂度。

1) 安装

```
# pip 安装
pip install python-docx

# conda 安装
conda install python-docx
```

2) 创建 Word 文档

```
import docx

doc = docx.Document() # Document对象表示整个文档
doc.add_heading('文章标题', 0) # 添加标题,0 表示标题的级别
doc.add_paragraph('Hello world!') # 添加段落
doc.add_heading('子标题', 1) # 添加标题,1 表示标题的级别
doc.add_paragraph('这是子标题段落') # 添加段落
doc.save('./word/docx创建文档.docx') # 保存文档
```

第五十六章 web 自动化

一、 selenium 介绍

Selenium是一个开源的测试工具，用于测试 Web 应用程序。Selenium 是一个用于测试 Web 应用程序的开源工具。它提供了一种简单的 API，可以通过编程方式控制浏览器的行为，从而实现自动化测试。

1) 组成部分

1) Selenium WebDriver：这是 Selenium 的核心组件，用于直接与浏览器进行交互。WebDriver 提供了丰富的 API，允许开发者通过代码控制浏览器的行为，如打开网页、点击按钮、填写表单等。

2) Selenium IDE：这是一个浏览器插件，主要用于录制和回放用户的操作。Selenium IDE 适合初学者快速创建简单的测试脚本，但它不支持复杂的逻辑和条件判断。

3) Selenium Grid：这是一个用于并行执行测试的工具。通过 Selenium Grid，你可以在多个浏览器和操作系统上同时运行测试，从而加快测试速度并提高测试覆盖率。

2) Selenium WebDriver

Selenium WebDriver 是 Selenium 的核心组件，它提供了与浏览器交互的 API，允许开发者通过编程语言控制浏览器并执行各种操作。

在 Selenium 4 中，引入了 `Service` 对象来设置驱动程序路径，这种方式更加灵活，且符合 Selenium 4 的设计规范。

二、 selenium 安装

1) pip 安装 selenium

```
$> pip install selenium
$> pip show selenium
Name: selenium
Version: 4.38.0
Summary: Official Python bindings for Selenium WebDriver
```

浏览器	驱动程序	下载地址
Chrome	ChromeDriver	点击下载
Edge	EdgeDriver	点击下载
Firefox	GeckoDriver	点击下载
Safari	SafariDriver	点击下载

```
Home-page: https://www.selenium.dev
Author:
Author-email:
License: Apache-2.0
Location: d:\application\miniconda\envs\pyauto\lib\site-packages
Requires: certifi, trio, trio-websocket, typing_extensions,
    urllib3, websocket-client
Required-by:
```

2) 下载 selenium 驱动程序

在 selenium4.6 以前的版本中，selenium 需要下载对应的浏览器的驱动程序，才能进行自动化测试。

三、 启动浏览器

1) selenium4.6 以前的版本

```
from selenium import webdriver

# 使用 Chrome 浏览器
driver = webdriver.Chrome(executable_path='/path/to/chromedriver')

# 或者使用 Firefox 浏览器
# driver = webdriver.Firefox(executable_path='/path/to/geckodriver')

# 或者使用 Edge 浏览器
# driver = webdriver.Edge(executable_path='/path/to/msedgedriver')
```

2) selenium4.6 及以后版本

在 Selenium 4 中，不再直接在 webdriver.Chrome 中设置驱动程序路径，而是通过引入 Service 对象来设置。这样可以避免弃用警告，并确保驱动程序的正确加载。例如：

```
from selenium import webdriver
driver = webdriver.Chrome() # 如果使用其他浏览器，如
    Firefox，需要相应修改

# 访问网址
driver.get(f"https://www.baidu.com/")

# 保持浏览器不退出
```

```
s = input("输入回车退出")
```

图 56.1 Selenium4.0 以后自动下载相应的驱动

当国内的网络环境，自动检测下载驱动需要不一样的网络环境，所以建议手动下载驱动，然后指定驱动路径。

```
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
# 在 Selenium 4 中，不再直接在 webdriver.Chrome
    中设置驱动程序路径，而是通过引入 Service 对象来设置。
# 这样可以避免弃用警告，并确保驱动程序的正确加载。

service =
    Service("D:/Practice/python/selenium_chrome/chromedriver.exe")
    # ChromeDriver路径
browser = webdriver.Chrome(service=service)      # 创建浏览器对象

# 访问网址
browser.get(f"https://www.baidu.com/")

# 定位元素

# 保持浏览器不退出
s = input("输入回车退出")
```

图 56.2 Selenium4 指定驱动位置

1) Selenium 操作流程

```
from selenium import webdriver
from selenium.webdriver.chrome.service import Service as ChromeService

# 1.设置正确的驱动路径
```

```
service =
    ChromeService(executable_path="D:/Practice/python/selenium_chrome/chromedriver.exe")

# 2.[可选] 设置开机启动参数
options = webdriver.ChromeOptions() # 创建ChromeOptions对象
options.add_argument("--start-maximized") # 窗口最大化

# 3.创建浏览器对象
browser = webdriver.Chrome(service=service, options=options)

# 4.访问网址
browser.get(f"https://www.baidu.com/")

# 5.定位元素

# 6.操作元素

# 7.[可选] 保持浏览器不退出

# 7.[可选] 操作完成后关闭浏览器
browser.quit()
```

第五十七章 OCR 识别

一、 在线识别

二、 离线识别

1) pytesseract

参考文献

[1] Iammyself001. Arch 安装 anaconda. Website. https://www.bilibili.com/video/BV1Fo4y1L7Jo/.